

Сетевое программирование на Python

Ключевые возможности SSL в Python 3

Цели протокола TLS и методы их достижения на Python

Архитектура серверов и создание серверных приложений

Работа с электронной почтой в приложениях

Применение Flask и Django для создания сетевых веб-приложений



Джон Галбрейт

Network Programming in Python : The Basic

*A Detailed Guide to Python 3 Network Programming
and Management*

John Galbraith



www.bpbonline.com

Джон Галбрейт

**Сетевое
программирование
на Python**

Санкт-Петербург
«БХВ-Петербург»

2024

УДК 004.438 Python
ББК 32.973.26-018.1
Г15

Галбрейт Дж.

Г15 Сетевое программирование на Python: Пер. с англ. — СПб.:
БХВ-Петербург, 2024. — 448 с.: ил. — (Профессиональное программирование)
ISBN 978-5-9775-1899-4

Книга посвящена разработке серверных приложений и клиент-серверных архитектур на Python. Рассказано о поддержке SSL в Python 3, представлены примеры работы с протоколами TCP, UDP, HTTP, SMTP, IMAP, FTP, RPC, взаимодействия с сервисами DNS. Освещена работа с электронной почтой в приложениях. Описаны цели протокола TLS и методы их достижения на Python. Подробно описаны возможности модуля asyncio, входящего в состав Python 3.4, даны рекомендации по разработке сетевых приложений с использованием веб-фреймворков Flask и Django.

Для программистов

УДК 004.438 Python
ББК 32.973.26-018.1

Группа подготовки издания:

Руководитель проекта	<i>Павел Шалин</i>
Зав. редакцией	<i>Людмила Гауль</i>
Перевод с английского	<i>Марины Попович</i>
Редактор	<i>Анна Кузьмина</i>
Компьютерная верстка	<i>Натальи Смирновой</i>
Оформление обложки	<i>Зои Канторович</i>

Copyright 2022 BPB Publications, India. All rights reserved.
First published in the English language under the title **Network Programming in Python : The Basic: Detailed Guide to Python 3 Network Programming and Management** by BPB Publications India.
Russian translation rights arranged with BPB Publications, India.

© 2022 BPB Publications, Индия. Все права защищены.
Впервые опубликовано на английском языке под названием **Network Programming in Python : The Basic: Detailed Guide to Python 3 Network Programming and Management** издательством BPB Publications India.
Права на перевод на русский язык предоставлены издательством BPB Publications, Индия.

"БХВ-Петербург", 191036, Санкт-Петербург, Гончарная ул., 20.

ISBN 978-93-5551-257-4 (англ.)
ISBN 978-5-9775-1899-4 (рус.)

© BPB Publications, India, 2022
© Перевод на русский язык, оформление.
ООО «БХВ-Петербург», ООО «БХВ», 2023

Содержание

https://t.me/IT_Portal

Об авторе.....	14
О рецензенте	15
Благодарности.....	16
Предисловие	17
Пакет кода	19
ГЛАВА 1. Введение в сетевое взаимодействие между клиентом и сервером	21
Содержание главы	21
Цель	22
Основы: стеки и библиотеки	22
Уровни приложения	25
Что такое протокол.....	26
Сетевое взаимодействие	28
Слой за слоем.....	30
Кодирование и декодирование.....	32
Протокол IP	33
IP-адреса	34
Маршрутизация	35
Фрагментация пакетов	37
Подробно об IP	38
Резюме	38
ГЛАВА 2. Протокол UDP	41
Содержание главы	42
Цель	42
Множество сервисов в одной системе	42
Сокет — точка соединения.....	44
Клиенты, принимающие любые пакеты.....	48
Отсрочка, блокировка и время ожидания	50
UDP-сокеты.....	54
Идентификаторы запросов	56

От привязки до интерфейсов	57
Фрагментация UDP	59
Параметры сокетов	61
Широковещание	62
Сценарии применения UDP	64
Резюме	64
 ГЛАВА 3. Протокол TCP	67
Содержание главы	67
Цель	68
Как работает TCP	68
Когда использовать TCP	69
Сокеты TCP	70
TCP-клиент и TCP-сервер	71
Одно взаимодействие — один сокет	75
Адрес	77
От привязки до интерфейсов	78
Взаимоблокировка	79
Полуоткрытые соединения, закрытые соединения	84
Потоки TCP для передачи файлов	86
Резюме	86
 ГЛАВА 4. DNS и имена сокетов	89
Содержание главы	89
Цель	89
Имена сокетов и хостов	90
Пять координат сокетов	91
IPv6	92
Современное разрешение адресов	93
Привязка сервера к порту с помощью <i>getaddrinfo()</i>	94
Метод <i>getaddrinfo()</i> для привязки к сервису	95
Получение канонического имени хоста с помощью <i>getaddrinfo()</i>	96
Другие флаги <i>getaddrinfo()</i>	98
Примитивные процедуры службы имен	99
Метод <i>getsockaddr()</i>	100
Протокол DNS	101
Почему не стоит использовать DNS напрямую	104
Python для DNS-запросов	105
Разрешение почтовых доменов	106
Резюме	109
 ГЛАВА 5. Данные и ошибки в Интернете	111
Содержание главы	111
Цель	111
Строки и байты	111

Строки символов	113
Сетевой порядок байтов и двоичные числа	116
Кадрирование	119
Pickle и форматы с разделителями	125
JSON и XML	126
Сжатие	127
Исключения в сети	128
Специфические исключения	130
Исключения в сети: обнаружение и сообщение об ошибках	131
Резюме	133

ГЛАВА 6. Протокол SSL/TLS..... 135

Содержание главы	135
Цель	136
От чего не защищает TLS	136
Что худшее может случиться?	137
Создание сертификатов	139
TLS Offloading	141
Контексты по умолчанию в Python 3.4.....	142
Подходы к обертке сокетов	146
Выбор шифров вручную и Perfect Forward Security	147
Поддержка протокола TLS	149
Дальнейшее изучение	151
Резюме	157

ГЛАВА 7. Архитектура сервера..... 159

Содержание главы	159
Цель	160
Несколько слов о развертывании	160
Базовый протокол	162
Однопоточный сервер	166
Многопроцессорный и многопоточный серверы	169
Фреймворк SocketServer из прошлого	171
Асинхронные серверы	172
Фреймворк asyncio с обратными вызовами	177
Фреймворк asyncio с сопрограммами	179
Устаревший модуль asyncore	181
Комбинированный подход	182
Под влиянием inetd	183
Резюме	185

ГЛАВА 8. Очереди сообщений и кеши..... 187

Содержание главы	187
Цель	188
Использование Memcached (кеширование в памяти)	188

Хеширование и сегментирование	191
Очереди сообщений	194
Очереди сообщений в Python	196
Резюме	201
ГЛАВА 9. HTTP-клиенты	203
Содержание главы	203
Цель	204
Библиотеки клиентов Python	204
Кадрирование, шифрование и порты	206
Методы	208
Хосты и пути	209
Коды состояний	210
Валидация и кеширование	213
Кодирование содержимого	216
Согласование содержимого	216
Тип содержимого	218
Аутентификация по HTTP	219
Файлы cookie	221
Поддержание соединения и http lib	222
Резюме	223
ГЛАВА 10. Серверы для работы с HTTP	225
Содержание главы	225
Цель	226
Стандарт WSGI	226
Серверные фреймворки для асинхронной обработки	228
Прямые и обратные прокси	229
Четыре архитектурных стиля	230
Python на Apache	232
HTTP-серверы на Python	232
Преимущество обратных прокси	233
Платформа как услуга	234
REST и паттерны GET и POST	236
WSGI без фреймворка	238
Резюме	242
ГЛАВА 11. Всемирная паутина	245
Содержание главы	245
Цель	246
URL и гипермедиа	246
Создание и парсинг URL	247
Относительные URL	250
Язык гипертекстовой разметки HTML	251
Чтение и запись с использованием базы данных	253

Ужасное веб-приложение на Flask.....	255
Методы и формы HTTP	261
Когда формы используют неподходящие методы	263
Опасные и безопасные cookie	264
Непостоянный межсайтовый скриптинг	266
Постоянный межсайтовый скриптинг.....	268
Подделка межсайтовых запросов	269
Улучшенная программа	270
Приложение для оплаты на Django.....	273
Выбор фреймворка для веб-сайта	277
Веб-сокеты	279
Веб-скрейпинг	279
Получение страниц.....	281
Страницы для веб-скрейпинга	285
Рекурсивный веб-скрейпинг.....	287
Резюме	291

ГЛАВА 12. Составление и парсинг сообщений электронной почты..... 293

Содержание главы	293
Цель	294
Форматирование электронного письма.....	294
Составление электронного письма	296
HTML и мультимедиа	298
Создание контента.....	303
Парсинг электронного письма	305
Использование MIME	307
Кодирование заголовков.....	309
Парсинг дат	311
Резюме	312

ГЛАВА 13. Протокол SMTP 313

Содержание главы	313
Цель	314
Веб-сервисы электронной почты и почтовые клиенты	314
Все началось с командной строки	314
Развитие клиентов	315
Переход на веб-сервисы электронной почты	317
Функции SMTP	318
Передача электронной почты	319
Получатель на конверте и заголовки.....	320
Несколько прыжков	320
Библиотека для работы с протоколом SMTP.....	322
Обработка ошибок и отладка	323
EHLO для сбора информации	326
SSL и TLS.....	329

Аутентификация SMTP.....	332
Советы по SMTP.....	333
Резюме	334
ГЛАВА 14. Протокол POP	335
Содержание главы	335
Цель	336
Серверы POP и стандарты	336
Аутентификация и подключение	336
Получение информации о почтовом ящике.....	339
Загрузка и удаление писем	340
Резюме	343
ГЛАВА 15. Протокол IMAP.....	345
Содержание главы	346
Цель	347
Реализация IMAP в Python.....	347
Клиент IMAP.....	349
Просмотр папок	351
UID и номера писем	351
Интервалы между письмами	352
Общая информация	352
Получение всего почтового ящика	354
Загрузка отдельных писем.....	356
Добавление и удаление флагов	363
Удаление писем	364
Поиск	364
Работа с папками	366
Асинхронность	367
Резюме	367
ГЛАВА 16. Протоколы SSH и Telnet.....	369
Содержание главы	369
Цель	370
Автоматизация с помощью командной строки	370
Раскрытие выражения и экранирование в командной строке.....	371
Аргументы в командах UNIX.....	372
Экранирование символов	374
Ужасная командная строка Windows	376
Терминал	377
Терминалы и буферизация	380
Telnet.....	381
SSH: безопасная оболочка	386
SSH: краткий обзор	386
Ключи хоста для SSH.....	387

Аутентификация в SSH.....	390
Отдельные команды и сеансы	391
Протокол SFTP	396
Дополнительные возможности	399
Резюме	399
ГЛАВА 17. Протокол FTP.....	401
Содержание главы	402
Цель	402
Что делать, если невозможно использовать FTP.....	402
Каналы коммуникации.....	403
FTP в Python	404
Двоичные файлы и файлы ASCII.....	405
Расширенная загрузка двоичных файлов с сервера	407
Отправка данных на удаленный компьютер.....	409
Расширенная отправка двоичных данных	410
Обработка ошибок.....	411
Поиск по каталогам	412
Обнаружение каталогов и загрузка в рекурсивном режиме.....	414
Создание и удаление каталогов	416
Безопасное использование FTP.....	416
Резюме	416
ГЛАВА 18. RPC — удаленный вызов процедур.....	419
Содержание главы	420
Цель	421
Характеристики RPC.....	421
XML-RPC	422
JSON-RPC.....	429
Самодокументируемые данные	432
Объекты: Руго и RPyC.....	434
Пример RPyC	435
Очереди сообщений, RPC и веб-фреймворки.....	438
Восстановление после ошибок в сети	438
Резюме	439
Предметный указатель.....	441

Посвящается

моей семье

Об авторе

Джон Гэлбрейт (John Galbraith) — архитектор, дизайнер, инженер, художник, сценарист и комедийный автор. Последние 14 лет работал с клиентами, вендорами, сервисными интеграторами. Учился в MIT.

О рецензенте

Джомни Карл (Jomny Carle) — архитектор корпоративных решений, музыкальный критик и поэт. За 22 года работы он помог множеству клиентов в разработке и реализации корпоративных стратегий в соответствии с требованиями безопасности и бизнес-целями.

Благодарности

Я хочу поблагодарить несколько человек за поддержку при написании этой книги. Во-первых, я хочу сказать спасибо моим родителям за то, что подбадривали и вдохновляли меня. Я бы никогда не дописал книгу без их поддержки.

Выражаю благодарность команде BPB Publications за то, что дали возможность написать книгу, не торопили меня и предоставляли всю необходимую поддержку.

И спасибо нашим близким за то, что решали все бытовые вопросы, пока мы занимались книгой.

Предисловие

Из *главы 1* вы узнаете, что современное сетевое оборудование может передавать небольшие сообщения, называемые пакетами, размер которых обычно не превышает несколько тысяч байтов. Вы увидите, как объединять отдельные сообщения в связанное общение между браузером и сервером или почтовым клиентом и почтовым сервером провайдера.

В *главе 2* мы рассмотрим протокол UDP. Он решает две из обозначенных в первой главе задач. Он назначает номера портов пакетам, предназначенным для разных сервисов в одной системе. Если пакеты теряются, дублируются или приходят не по порядку, это не его забота.

Глава 3 посвящена протоколу TCP. Он использует те же правила, что и UDP, чтобы учитывать номера портов, и обеспечивает упорядоченность и надежность потоков данных, скрывая тот факт, что непрерывный поток на самом деле разделен на отдельные пакеты, которые приходится собирать в месте назначения.

В *главе 4* мы обсудим два важных вопроса, которые нужно решить независимо от используемого протокола данных: TCP или UDP. В этой главе мы рассмотрим сетевые адреса и распределенный сервис, который преобразует имена в IP-адреса.

Как вы узнаете из *главы 5*, если приложение должно искать имя хоста DNS, где-то за кулисами обязательно будет работать UDP. Хотя TCP на практике используется по умолчанию, когда двум программам в Интернете нужно обмениваться данными, мы рассмотрим несколько ситуаций, когда лучше выбрать другой протокол.

В начале *главы 6* мы определим цели TLS и методы их достижения. Затем мы посмотрим, как активировать и настроить TLS на TCP-сокете с помощью простых и сложных примеров кода Python. Наконец, мы увидим, как TLS работает в протоколах, которые мы будем рассматривать далее.

В *главе 7* мы подробно рассмотрим архитектуру серверов и создание программного обеспечения сервера.

Глава 8 — короткая, но, пожалуй, самая важная глава в этой книге. Мы рассмотрим две технологии — кеши и очереди сообщений, на которых строятся масштабные системы.

Глава 9 — первая из трех глав об HTTP. В этой главе вы узнаете, как использовать HTTP в клиентском приложении, которое хочет извлечь и кешировать документы, а

также отправить запросы или данные на сервер. Вы также узнаете, по каким правилам работает протокол.

В *главе 10* мы рассмотрим проектирование и развертывание HTTP-серверов. В обеих главах мы будем изучать протокол на базовом уровне, т. е. как механизм извлечения и загрузки документов.

В *главе 11* мы рассмотрим шаблоны и формы, а также веб-фреймворки, которые позволяют объединить все эти паттерны и упростить их создание.

В *главе 12* мы поговорим о том, как создаются электронные сообщения, и обратим особое внимание на включение в них мультимедиа и использование символов из других языков.

Глава 13 посвящена протоколу SMTP. Он передает электронные письма с компьютера пользователя на сервер, который хранит сообщения и передает их получателю.

В *главе 14* вы узнаете, как работает устаревший протокол POP для передачи электронных писем.

Глава 15 посвящена современному протоколу для передачи электронной почты — IMAP. При использовании IMAP можно не только извлекать и просматривать сообщения, но также отмечать их прочитанными и хранить в разных папках на сервере.

В *главе 16* рассматривается, как подключаться к командной строке по сети и устранять возможные проблемы при ее использовании.

Глава 17 описывает, что FTP использовался для четырех основных задач. Во-первых, его главным предназначением была загрузка файлов. Во-вторых, FTP часто применялся для анонимной загрузки файлов на сервер. В-третьих, протокол позволял синхронизировать целое дерево файлов между компьютерами. В-четвертых, FTP использовался в своем изначальном предназначении — полноценное интерактивное управление файлами.

Глава 18 посвящена RPC и особенностям работы с протоколами удаленного вызова процедур.

Пакет кода

Код для этой книги можно загрузить по ссылке:

<https://rebrand.ly/twfyjg1>

или на GitHub: **<https://github.com/bpbpublications/Network-Programming-in-Python>**.

Код в репозитории на GitHub будет обновляться в случае необходимости.

Введение в сетевое взаимодействие между клиентом и сервером

В этой книге мы рассматриваем сетевое программирование на Python, в том числе основные принципы, модули и сторонние библиотеки для взаимодействия с удаленными компьютерами через Интернет с помощью распространенных протоколов связи.

Эта книга предназначена для тех, кто уже знаком с Python, здесь не рассматриваются основные принципы этого языка. Мы приводим наглядные примеры кода и надеемся, что они будут вам полезны. Мы не станем подробно рассматривать расширенные возможности самого Python, но в некоторых местах будем пояснять особенно интересные или удачные подходы и конструкции.

С другой стороны, эта книга подходит для тех, кто ничего не знает о сетевом программировании. Если вы хоть раз использовали браузер или отправляли электронную почту, этих знаний вам будет достаточно. Мы будем рассматривать сеть с точки зрения разработчика приложений, который создает сервис, подключенный к сети, например сайт, почтовый сервер или онлайн-игру, или проектирует клиентское программное обеспечение, которое использует такой сервис.

Однако в этой книге мы не будем рассматривать настройку и конфигурирование сетей. Сетевая архитектура, серверное администрирование и автоматизированное конфигурирование сети — это отдельные дисциплины, которые не пересекаются с сетевым программированием, рассматриваемым в этой книге. Сегодня Python все шире используется для конфигурирования сети благодаря таким проектам, как OpenStack, SaltStack и Ansible. Если вы хотите узнать больше о многочисленных технологиях конфигурирования сети, изучите книги и документацию по этой теме.

Содержание главы

- ◆ Основы: стеки и библиотеки.
- ◆ Уровни приложения.
- ◆ Что такое протокол.
- ◆ Сетевое взаимодействие.

- ◆ Слой за слоем.
- ◆ Кодирование и декодирование.
- ◆ Протокол IP.
- ◆ IP-адреса.
- ◆ Маршрутизация.
- ◆ Фрагментация пакетов.
- ◆ Подробно об IP.
- ◆ Резюме.

Цель

В этой главе мы рассмотрим такие темы, как уровни приложения, протокол IP, кодирование и декодирование на Python, библиотеки Python, маршрутизация и т. д.

Основы: стеки и библиотеки

Мы начнем изучение сетевого программирования на Python с двух главных концепций.

- ◆ Первая из них — сетевой стек с базовыми сетевыми сервисами, которые служат основой для более сложных сервисов.
- ◆ Вторая — библиотеки Python. Взаимодействие с сетевым протоколом часто прописано в готовом коде Python, который мы будем использовать в виде модулей из стандартных встроенных библиотек или пакетов из сторонних решений.

Сетевое программирование часто сводится к выбору и реализации библиотеки, которая уже содержит нужные функции. Основная цель этой книги — познакомить вас с различными сетевыми библиотеками для Python, а также с низкоуровневыми сетевыми сервисами, на которых эти библиотеки основаны. Зная базовые концепции, вы будете лучше понимать, как работают библиотеки и что происходит, когда на нижнем уровне что-то идет не так.

Начнем с базового примера. Возьмем почтовый адрес:

Тадж-Махал

г. Агра, штат Уттар-Прадеш, Индия

Мы хотим узнать широту и долготу этой точки. Для этого Google предлагает удобный API геокодирования. Что нужно сделать, чтобы воспользоваться сетевым сервисом Python?

Если речь о новом сетевом сервисе, для начала желательно узнать, вдруг кто-то уже разработал протокол, который понадобится вашей программе. Например, протокол

геокодирования Google. Давайте попробуем найти что-нибудь про геокодирование (geocoding) в документации к стандартной библиотеке Python.

<https://docs.python.org/3/library/>

К сожалению, там про это ничего нет. В стандартной библиотеке есть не все функции, но стоит регулярно проверять список тем, чтобы продолжать изучение сервисов Python.

Также советую читать блог Python Module of the Week, в котором Даг Хеллманн (Doug Hellmann) рассказывает о возможностях стандартной библиотеки Python.

Раз стандартная библиотека не предлагает пакет для этого сценария, можно поискать универсальное решение в репозитории PyPI (Python Package Index), где собраны пакеты от других программистов и организаций со всего мира. Кроме того, можно поискать библиотеку Python на сайте вендора, сервис которого вы собираетесь использовать. Также можно ввести в поисковой системе слово "Python" и название веб-сервиса, который вы собираетесь использовать. Возможно, получится найти подходящий пакет.

В этом примере я использовал Python Package Index:

<https://pypi.org/>

Я ввел "geocoding" (геокодирование) и нашел пакет pygeocoder, который предоставляет удобный интерфейс для функций геокодирования Google (хотя, судя по описанию, пакет создан не самим Google).

<https://pypi.org/project/pygeocoder/>

Кстати, нередко можно найти пакет Python, который вроде бы содержит нужные возможности, а мы хотим испытать его в своей системе. Давайте рассмотрим лучшую технологию Python для быстрых экспериментов с новыми библиотеками — virtualenv.

Раньше установка пакета Python была очень сложным и необратимым процессом, который требовал прав администратора на компьютере, в итоге установка Python постоянно менялась. Спустя несколько месяцев в системе скапливалось несколько десятков пакетов Python, причем новые пакеты могли уже не устанавливаться из-за проблем с совместимостью с устаревшими пакетами из проекта, который давно закончился.

Думающие разработчики Python больше не попадают в такие ситуации. Многие из нас устанавливают только virtualenv в качестве пакета Python для всей системы. После этого можно создавать небольшие изолированные виртуальные среды Python, чтобы в свое удовольствие устанавливать и удалять пакеты и экспериментировать с новыми решениями, не захламляя всю систему. Закончив с экспериментами, можно просто удалить каталог виртуального окружения, и все будет как прежде.

Сейчас нам надо будет создать виртуальное окружение, чтобы протестировать пакет pygeocoder. Если у вас еще не установлен virtualenv, загрузите и установите его:

<https://pypi.org/project/virtualenv/>

Следуйте инструкциям по созданию нового окружения. (В Windows каталог Python с виртуальным окружением будет называться Scripts, а не bin.)

```
$ virtualenv -p python3 geo_env
```

```
$ cd geo_env
```

```
$ ls
```

```
bin/ include/ lib/
```

```
$ . bin/activate
```

```
$ python -c 'import pygeocoder'
```

```
Traceback (most recent call last):
```

```
File "<string>", line 1, in
```

```
ImportError: No module named 'pygeocoder'
```

Как видите, пока у нас нет пакета pygeocoder. Его можно установить командой `pip` в виртуальном окружении, в которое мы вошли командой `activate`.

```
$ pip install pygeocoder
```

Загрузим и распакуем pygeocoder:

```
Downloading pygeocoder-1.2.1.1.tar.gz
```

```
Running setup.py egg_info for package pygeocoder
```

```
Downloading/unpacking requests>=1.0 (from pygeocoder)
```

```
Downloading requests-2.0.1.tar.gz (412kB): 412kB downloaded
```

```
Running setup.py egg_info for package requests
```

```
Installing collected packages: pygeocoder, requests
```

```
Running setup.py install for pygeocoder
```

```
Running setup.py install for requests
```

```
Successfully installed pygeocoder requests
```

```
2
```

The pygeocoder package will now be available in the virtualenv's python binary.

```
$ python -c 'import pygeocoder'
```

Итак, мы установили пакет pygeocoder и теперь можем запустить программу `search1.py`, как в листинге 1.1.

Листинг 1.1. Получение широты и долготы

```
#!/usr/bin/env python3
```

```
# Network Programming in Python: The Basics
```

```
from pygeocoder import Geocoder
```

```
if __name__ == '__main__':
```

```
    address = 'taj mahal'
```

```
    print(Geocoder.geocode(address)[0].coordinates)
```

В командной строке увидим следующие результаты:

```
$ python3 search1.py (27.1751° N, 78.0421° E)
```

Как видите, мы легко получили ответ на наш вопрос о долготе и широте определенного адреса. Информация получена напрямую с сайта Google. Мы взяли первый попавшийся пакет и угадали.

Удивлены, что книга по сетевому программированию начинается с установки стороннего пакета, с которым потенциально интересная задача превратилась в несколько строчек скрипта на Python? Привыкайте! Вы увидите, что именно так решается 90% задач: мы находим решение другого разработчика, а затем аккуратно и эффективно встраиваем его в свою программу.

Мы пока не закончили рассматривать этот пример. Мы увидели, как можно относительно просто обратиться к сложному сетевому сервису, но что лежит за привлекательным интерфейсом `rugeocoder`? Как именно мы используем сервис? Представьте, что этот сложный сервис — всего лишь верхний уровень сетевого стека.

Уровни приложения

Для решения задачи мы взяли стороннюю библиотеку Python из репозитория PyPI. Она отлично подошла для работы с API геокодирования от Google. Но что если этой библиотеки не было бы? Что если нам пришлось бы писать собственный клиент для Google Maps API?

Для того чтобы ответить на этот вопрос, давайте посмотрим на `search2.py` в листинге 1.2. Здесь используется не сторонняя библиотека, которая умеет работать с геокодированием, а популярная библиотека `requests`, на основе которой разработан `rugeocodin`. Как видите, она уже установлена в виртуальном окружении.

Листинг 1.2. Использование API геокодирования от Google для получения документа JSON

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics

import requests

def geocode(address):
    base = 'https://nominatim.openstreetmap.org/search'
    parameters = {'q': address, 'format': 'json'}
    user_agent = ' Client-Server Networking: An Overview search2.py'
    headers = {'User-Agent': user_agent}
    response = requests.get(base, params=parameters, headers=headers)
    reply = response.json()
    print(reply[0]['lat'], reply[0]['lon'])
```



```
if __name__ == '__main__':
    geocode('taj mahal')
```

Результат мы получим почти такой же, как в первом случае.

\$ python3 search2.py

```
{'lat': 27.1751° N, 'lng': - 78.0421° E }
```

Результаты немного различаются (например, библиотека requests представила данные в виде словаря Python), но в целом мы получили те же значения.

Первое, что бросается в глаза в этом коде, — отсутствие общей семантики `pygeocoder`. Если не вглядываться, можно не заметить, что у нас просят почтовый адрес. В отличие от кода `search1.py`, который запрашивал преобразование адреса в значения широты и долготы, во втором коде мы создаем базовый URL и серию параметров запроса, цель которых неочевидна, если не изучить документацию Google. Кстати, с документацией по API можно ознакомиться здесь:

<https://developers.google.com/maps/documentation/geocoding/>

Если внимательно посмотреть на словарь параметров запроса в файле `search2.py`, можно заметить, что параметр адреса содержит конкретный почтовый адрес. Второй аргумент сообщает Google, что вы не используете датчик геолокации мобильного устройства, чтобы получить данные в этом запросе.

Мы вручную вызываем ответ при получении документа в результате поиска этого URL. Для того чтобы преобразовать его в JSON, мы используем метод `json()` и изучим многоуровневую структуру данных, чтобы найти нужный элемент с широтой и долготой.

Скрипт `search2.py` дает тот же результат, что и `search1.py`, но вместо того, чтобы напрямую использовать адреса и значения широты и долготы, он создает URL, получает ответ и составляет из него JSON. Если мы спустимся на уровень вниз по сетевому стеку, то увидим, что здесь код занимается только спецификой обработки запроса, а не самой сутью запроса.

Что такое протокол

Во втором скрипте создается URL и извлекается расположенный по нему документ. Это довольно простое действие, как вы уже знаете по работе с браузером. По сути, URL содержит описание того, где найти и как получить определенный документ в Интернете. URL начинается с имени протокола, за которым следует имя компьютера, где хранится документ, и, наконец, путь к конкретному документу на этом компьютере. URL содержит инструкции, по которым низкоуровневый протокол находит документ, так что приложение `search2.py` может разрешить URL и получить страницу.

Знаменитый протокол HTTP (Hypertext Transfer Protocol — протокол передачи гипертекста), который применяется почти во всех современных веб-соединениях, —

это низкоуровневый протокол, используемый URL. Мы рассмотрим его подробнее в главах 9–11. HTTP предлагает метод, с помощью которого библиотека requests извлекает результаты из Google.

А что будет, если убрать библиотеку и попробовать получить результат через HTTP? Давайте посмотрим на search3.py в листинге 1.3.

Листинг 1.3. Использование Google Maps напрямую через HTTP

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics

import http.client
import json
from urllib.parse import quote_plus base = '/search'

def geocode(address):
    path = '{}?q={}&format=json'.format(base, quote_plus(address))
    user_agent = b' Client-Server Networking: An Overview.py'
    headers = {'User-Agent': user_agent}
    connection = http.client.HTTPSConnection('nominatim.openstreetmap. org')
    connection.request('GET', path, None, headers)
    rawreply = connection.getresponse().read()
    reply = json.loads(rawreply.decode('utf-8'))
    print(reply[0]['lat'], reply[0]['lon'])

if __name__ == '__main__':
    geocode('taj mahal')
```

В этом коде мы работаем напрямую с HTTP — мы просим его подключиться к определенному компьютеру, выполнить запрос GET по URL, который мы создали вручную, а затем получаем ответ из HTTP-соединения. Параметры запроса необязательно предоставлять в виде отдельных ключей и значений. Можно сделать проще и вставить их напрямую в путь в виде словаря, записав их в формате "имя = значение" через &, а в конце поставив вопросительный знак (?).

Результат этой программы очень похож на результаты предыдущих:

```
$ python3 search2.py
{'lat': 27.1751° N, 'lng': - 78.0421° E }
```

HTTP — это лишь один из множества протоколов, для которых в стандартной библиотеке Python есть встроенная реализация. Мы не обязаны учитывать всю специфику работы HTTP. В коде search3.py мы просто велит отправить запрос, а затем изучаем ответ. Мы опустились на еще один уровень стека протоколов, поэтому имеем дело с более низкоуровневыми деталями, чем в search2.py, но стандартная библиотека все же позволяет эффективно обрабатывать сетевые данные.

Сетевое взаимодействие

Протокол HTTP не передает данные между двумя устройствами сам по себе — он работает на еще более базовой абстракции. По сути, он использует возможности современных операционных систем передавать трафик между двумя программами в виде открытого текста по IP-сети с помощью протокола TCP. Иными словами, протокол HTTP передает точный текст сообщений, которыми обмениваются два хоста с помощью TCP.

Под HTTP находится самый нижний уровень сетевого стека, но мы можем взаимодействовать с ним с помощью Python. Изучите код `search4.py` в листинге 1.4. В нем мы отправляем в Google Maps точно такой же запрос, что и в предыдущих трех программах, но здесь мы передаем необработанное текстовое сообщение через Интернет и также получаем текст в качестве ответа.

Листинг 1.4. Взаимодействие с Google Maps через сокет

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics

import socket
from urllib.parse import quote_plus

request_text = """\
GET /maps/api/geocode/json?address={}&sensor=false HTTP/1.1\r\n
Host: maps.google.com:80\r\n
User-Agent: search4.py (Network Programming in Python: The Basics)\ r\n
Connection: close\r\n
\r\n
"""

def geocode(address):
    sock = socket.socket() sock.connect(('maps.google.com', 80))
    request = request_text.format(quote_plus(address))
    sock.sendall(request.encode('ascii'))
    raw_reply = b''
    while True:
        more = sock.recv(4096)
        if not more:
            break
        raw_reply += more
    print(raw_reply.decode('utf-8'))

if __name__ == '__main__':
    geocode('taj mahal')
```

Между `search3.py` и `search4.py` есть принципиальное различие. В предыдущих примерах для взаимодействия со сложным сетевым протоколом мы использовали библиотеку Python, написанную на Python. Здесь мы применили метод `socket()` самой операционной системы, чтобы обеспечить взаимодействие по IP-сети. Если бы мы писали эту сетевую функцию на языке C, мы использовали бы те же методы, которые применяют разработчики низкоуровневых систем.

Мы еще будем возвращаться к сокетам в следующих главах. Пока мы видим, что сетевое взаимодействие на низшем уровне (`search4.py`) выглядит как отправка и получение байтовых строк. Запрос представляет собой байтовую строку, ответ — это еще одна огромная байтовая строка, которую мы просто выводим на экран, чтобы увидеть, как она выглядит. (В разд. "Кодирование и декодирование" далее в этой главе мы узнаем, зачем нужно декодировать строку, прежде чем выводить на экран.) HTTP-запрос, содержимое которого мы видим в функции `sendall()`, состоит из названия операции (GET), расположения нужного документа и версии HTTP.

`GET/maps/api/geocode/json?address=taj+mahal&sensor=false HTTP/1.1`

Далее следует серия заголовков с именем, столбцом и значением, а затем символы возврата каретки и новой строки, которые закрывают запрос.

В листинге 1.5 представлен ответ, который выводится как выходные данные скрипта `search4.py`. Вместо того чтобы писать много кода для обработки текста, в этом примере мы выводим на экран весь ответ. Полезнее просто посмотреть, как выглядит HTTP-ответ, чем расшифровывать код для его анализа.

Листинг 1.5. Результат выполнения `search4.py`

`HTTP/1.1 200 OK`

`Server: nginx`

`Date: Tue, 25 Jan 2022 22:50:14 GMT`

`Content-Type: application/json; charset=UTF-8`

`Transfer-Encoding: chunked`

`Connection: close`

`Access-Control-Allow-Origin: *`

`Access-Control-Allow-Methods: OPTIONS,GET`

`37c`

```
[{"place_id":188987579,"licence":"Data © OpenStreet- Map contributors, ODbL 1.0.
https://osm.org/copyright","osm_type":"way","osm_id":375257537,"boundingbox-
":["27.1745358","27.1754823","78.0415593","78.0426212"],"lat":"2
7.1750123","lon":"78.04209683661315","display_name":"Taj Mahal, Taj Mahal Internal
Path, Taj Ganj, Agra, Uttar Pradesh, 282001,
India","class":"tourism","type":"attraction","importance":1.0489
056883572618,"icon":"https://nominatim.openstreetmap.org/ui/mapi-
cons//poi_point_of_interest.p.20.png"},"{"place_id":191576149,"li- cence":"Data ©
OpenStreetMap contributors, ODbL 1.0. https://osm.
org/copyright","osm_type":"way","osm_id":382063175,"boundingbox-
":["27.1674585","27.1682576","78.0506999","78.0507466"],"lat":"27.1682576","lon":"78.0
```

```
507466","display_name":"gali no 1, Taj Ganj, Agra, Uttar Pradesh, 282001, India","class":"highway","type":"residential","importance":0.5}]
```

0

По своей структуре HTTP-ответ очень похож на HTTP-запрос. Сначала идет строка состояния, затем серия заголовков. Далее следуют пустая строка и содержимое ответа: структура данных JavaScript в простом формате JSON, которая отвечает на наш запрос, описывая географическое положение, предоставленное API геокодирования Google.

Все эти строки состояния и заголовки в предыдущих примерах обрабатывались библиотекой Python `httplib`. Теперь мы видим, как выглядит взаимодействие, если убраться более высокие уровни стека.

Слой за слоем

Надеюсь, вам понравилось первое знакомство с сетевым программированием на Python. Давайте теперь подробнее рассмотрим несколько аспектов, опираясь на эти примеры.

Во-первых, мы увидели стек протоколов, начиная с высшего, семантически сложного взаимодействия ("Я хочу узнать географическое расположение для этого почтового адреса"), вплоть до простейшего обмена текстовыми строками между двумя компьютерами с помощью сетевого оборудования.

Стек протоколов в наших примерах состоит из четырех протоколов.

- ◆ Также у нас есть API геокодирования от Google, который объясняет, как выражать географические запросы в виде URL, которые возвращают данные JSON с координатами.
- ◆ URL — это уникальные идентификаторы для документов, которые можно извлекать по HTTP.
- ◆ HTTP использует сырые сокеты TCP/IP для поддержки операций с документами, например GET.
- ◆ Сокеты TCP/IP могут получать и отправлять только байтовые строки.

Каждый уровень стека использует инструменты низших уровней и предоставляет инструменты высшим.

Во-вторых, в этих примерах мы увидели, что с помощью Python можно работать с каждым уровнем сетевого стека. Нам пришлось применить стороннюю библиотеку только для работы с протоколом вендора, чтобы форматировать запросы в понятном для Google виде. Во втором листинге мы использовали библиотеку `requests` не потому, что в стандартной библиотеке нет модуля `urllib.request`, а потому что у этого модуля очень громоздкий API. Стандартная библиотека Python предлагает хорошую поддержку для всех предыдущих уровней протокола. У Python есть функ-

ции и классы, с помощью которых мы можем извлечь документ по определенному URL или отправить и получить текст через сырой сокет.

В-третьих, когда мы работали с низкоуровневыми протоколами, качество программ заметно упало. Например, в `search2.py` и `search3.py` нам пришлось жестко кодировать структуру и имена хостов. Такой негибкий код будет сложнее обслуживать в дальнейшем. В `search4.py` код получился еще хуже — он включает написанный вручную, непараметризованный HTTP-запрос, структура которого непонятна Python. В нем не хватает логики для обработки и оценки HTTP-ответа, а также понимания потенциальных сетевых ошибок.

Поэтому помните: правильно реализовать сетевые протоколы очень сложно и по возможности следует всегда использовать стандартную библиотеку или сторонние библиотеки. Обычно возникает искушение максимально упростить код, особенно при написании сетевого клиента. Например, игнорировать возможные ошибки, готовиться только к наиболее вероятным ответам, избегать корректного экранирования параметров, потому что кажется, что строки запросов должны включать только буквы, и в целом писать очень хрупкий код, мало что знающий о сервисе, с которым взаимодействует. Используйте библиотеки, разработчики которых уже придумали, как обходить острые углы и решать возможные проблемы.

В-четвертых, высокоуровневые сетевые протоколы, вроде API геокодирования Google для разрешения почтовых адресов, скрывают под собой уровни сетевого стека. Если использовать библиотеку `pygeocoder`, можно и не узнать, что для обработки запросов используются URL и HTTP.

Обработка проблем на нижних уровнях зависит от используемой библиотеки Python. Например, если Google недоступен из вашего расположения, а код пытается узнать координаты адреса, будет вызвано исключение на нижнем уровне? Или это будет считаться ошибкой на уровне геокодирования? При чтении книги обращайтесь внимание на обработку проблем с сетью, особенно применительно к низкоуровневым протоколам.

Наконец, мы дошли до темы, которой будет посвящена оставшаяся часть первой главы: интерфейс `socket()`, который мы использовали в `search4.py`, находится не на самом нижнем уровне при отправке запроса в Google. Под абстракцией сокетов есть еще протоколы, недоступные для Python, которые обрабатываются операционной системой.

- ◆ Протокол TCP (Transmission Control Protocol — протокол управления передачей) обеспечивает двусторонний поток байтов, передавая (в том числе повторно), получая и переупорядочивая небольшие фрагменты трафика, называемые пакетами.
- ◆ IP (Internet Protocol — интернет-протокол) — это протокол, с помощью которого пакеты передаются между компьютерами.
- ◆ В самом низу находится канальный уровень, состоящий из сетевого оборудования вроде Ethernet-портов и плат беспроводной связи, которые обеспечивают физическое взаимодействие между компьютерами, подключенными напрямую.

Оставшаяся часть этой главы, а также следующие две главы посвящены протоколам на низших уровнях. В этой главе мы начнем рассматривать уровень IP, а в следующих обсудим, как UDP и TCP поддерживают два основных типа взаимодействия между приложениями на двух хостах, подключенных к Интернету.

Однако сначала давайте узнаем, что такое байты и символы.

Кодирование и декодирование

Python 3 проводит различие между строками символов и низкоуровневыми последовательностями байтов. *Байты* — это реальные бинарные числа, которыми обмениваются компьютеры по сети. Они состоят из восьми цифр от 00000000 до 11111111, или от 0 до 255 в десятичной системе. В строках символов Python можно встретить символы Unicode, например латинскую букву "а" в нижнем регистре или правую фигурную скобку. У каждого символа Unicode есть числовой идентификатор, который называется *кодовой точкой*, но нам необязательно их знать, потому что Python 3 корректно обрабатывает символы сам, и мы будем иметь дело с последовательностями байтов, только если сами попросим Python преобразовать символы в байты или обратно.

У этих операций есть названия.

Декодирование. Когда в приложение поступают байты и мы хотим знать, что они значат, мы используем декодирование. Представьте, что программа — это шпион, которому поручили расшифровать последовательность байтов, передаваемых по каналу коммуникации.

Кодирование. Это процесс преобразование строк символов, понятных внешнему миру, в байты с помощью одной из нескольких кодировок, используемых компьютерами для передачи или хранения символов в единственной твердой валюте — байтах. Представьте, что шпион преобразует свои сообщения в цифры перед отправкой.

В Python 3 две эти операции реализованы как функция `decode()` для байтовых строк после чтения и метод `encode()` для символьных строк, когда их нужно будет записать обратно. В листинге 1.6 мы видим эти методы в действии.

Листинг 1.6. Кодирование символов для исходящей передачи и декодирование входящих байтов

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics

if __name__ == '__main__':
    # Преобразование входящих байтов в символы Unicode.
    input_bytes = b'\xff\xfe4\x001\x003\x00 \x00i\x00s\x00 \x00i\x00n\x00.\x00'
    input_characters = input_bytes.decode('utf-16')
    print(repr(input_characters))
```

```
# Преобразование символов обратно в байты перед отправкой.
output_characters = 'We copy you down, Eagle.\n'
output_bytes = output_characters.encode('utf-8')
with open('eagle.txt', 'wb') as f:
    f.write(output_bytes)
```

В примерах в этой книге мы будем стараться различать байты и символы. Вы могли заметить, что байтовые строки начинаются с буквы `b` и выглядят как `b'Hello'`, а у символьных строк нет никаких букв в начале — просто `'world'`. Для того чтобы избежать путаницы между байтовыми и символьными строками, Python 3 поддерживает *только символьные строки* для большинства строковых функций.

Протокол IP

Взаимодействие по сети и между сетями используется для обмена ресурсами. Сети физически соединяют множество компьютеров и позволяют им общаться, а сами сети соединяются друг с другом, образуя более масштабные системы, такие как Интернет.

Операционная система отвечает за распределение ресурсов дисков, оперативной памяти и центрального процессора между программами на компьютере, а также за распределение сетевых ресурсов, чтобы приложения общались друг с другом и не мешали взаимодействию других объектов в сети.

Физическое сетевое оборудование, например Ethernet-плата, беспроводные передатчики и USB-порты, с помощью которых взаимодействуют компьютеры, поддерживает совместное использование общих физических ресурсов. К одному хабу можно подключить десяток Ethernet-плат; 30 плат беспроводной связи могут делить один радиоканал; а DSL-модем использует частотное мультиплексирование, чтобы его цифровые сигналы не создавали помех для аналоговых сигналов, которые передаются по проводу во время телефонного разговора.

Пакет — это основная единица передачи данных между сетевыми устройствами. Он представляет собой байтовую строку длиной от нескольких байтов до нескольких тысяч. Существуют и специализированные сети, особенно в сфере телекоммуникаций, в которых можно отдельно управлять каждым байтом, но обычно мы все же передаем данные пакетами.

На физическом уровне пакет состоит всего из двух частей: байтовой строки с данными и адреса места назначения. *Адрес* — это обычно уникальный идентификатор одной из сетевых плат, входящих в тот же сегмент Ethernet или беспроводной сети, что и компьютер-отправитель. Задача сетевой платы — отправить и получить пакет таким образом, чтобы абстрагировать от операционной системы компьютера детали работы сети со всеми ее кабелями, напряжением и сигналами.

Какое место во всем этом занимает протокол IP?

Протокол IP отвечает за назначение стандартной системы адресов всем компьютерам, подключенным к Интернету по всему миру, чтобы им можно было передавать пакеты. Например, у браузера должна быть возможность подключаться к хосту из любого места, при этом ему необязательно знать, по какому лабиринту сетевых устройств проходит каждый пакет. Программы Python редко работают на таком низком уровне, что видят протокол IP в действии, но все же полезно понимать, как он работает.

IP-адреса

В исходной версии протокола IP каждому компьютеру, который подключается к глобальной сети, выдается адрес длиной четыре байта. Обычно IP-адрес выглядит как четыре целых числа (каждое число — это байт) от 0 до 255, разделенных точкой:

130.207.244.244

Люди, как правило, пользуются именами хостов, а не IP-адресами, потому что последовательность чисел гораздо сложнее запомнить. Пользователь может просто ввести **google.com** и не думать, что на самом деле его компьютер отправляет пакеты на адрес 74.125.67.103.

В листинге 1.7 приводится базовая программа Python, `getname.py`, которая запрашивает у операционной системы (Linux, Mac OS X, Windows и т. д.) разрешить имя хоста **www.python.org**. Система DNS (Domain Name System — система доменных имен), которая отвечает за поиск имен хостов, устроена довольно сложно, и мы подробно рассмотрим ее в *главе 4*.

Листинг 1.7. Преобразование имени хоста в IP-адрес

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics

import socket

if __name__ == '__main__':
    hostname = 'www.python.org'
    addr = socket.gethostbyname(hostname)
    print('The IP address of {} is {}'.format(hostname, addr))
```

Сейчас достаточно запомнить два момента:

- ◆ во-первых, какими бы сложными ни казались интернет-приложения, мы всегда используем IP-адреса, чтобы направлять пакеты в нужное место;
- ◆ во-вторых, операционная система обычно обрабатывает все детали разрешения имени хоста в IP-адрес.

Ни вы, ни ваш код на Python не видите, как это происходит.

Правда, сейчас адреса устроены чуть сложнее, чем раньше.

В мире заканчиваются 4-байтные IP-адреса, поэтому сейчас внедряется расширенная схема IPv6 с 16-байтными адресами, которых человечеству хватит надолго. Они выглядят не так, как привычные 4-байтные IP-адреса:

```
fe80::fcfd:4aff:fecf:ea4e
```

Нам необязательно знать разницу между IPv4 и IPv6, если наш код принимает IP-адреса или имена хостов от пользователя и передает их на обработку в соответствующую библиотеку. Операционная система, в которой работает код Python, сама распознает версию IP и обрабатывает адреса соответственно.

Традиционные IP-адреса читаются слева направо. Первый байт или два указывают на организацию, а следующий — на подсеть, в которой расположен целевой компьютер. В последнем байте указывается конкретный компьютер или сервис. Несколько диапазонов IP-адресов зарезервированы для особого применения.

- ◆ **127.*.*.** IP-адреса, которые начинаются со 127, зарезервированы для ресурсов на том же компьютере, где работает приложение. Когда браузер, FTP-клиент или программа Python обращаются к адресу в этом диапазоне, они общаются с сервисом или программой на том же компьютере. IP-адрес 127.0.0.1 обычно назначается самому компьютеру, на котором выполняется это программное обеспечение, и часто имеет имя хоста localhost.
- ◆ **10.*.*.**, **172.16–31.*.*** и **192.168.*.***. Эти IP-адреса зарезервированы для частных подсетей. Администраторы договорились не назначать эти IP-адреса сторонним серверам или сервисам. Эти адреса не обозначают определенный хост, а используются во внутренних сетях организации и не доступны широкой публике.

Некоторые из этих частных адресов могут быть назначены сетевому оборудованию у вас дома. Например, беспроводные маршрутизаторы или DSL-модемы часто назначают адреса из этих частных диапазонов домашним компьютерам и ноутбукам, скрывая весь ваш интернет-трафик за одним реальным IP-адресом, выданным вашим интернет-провайдером.

Маршрутизация

Когда приложение запрашивает отправку данных на конкретный IP-адрес, операционная система определяет, как передать данные через одну из физических сетей, к которым подключен компьютер. *Маршрутизация* — это процесс принятия решения о том, куда следует отправить каждый IP-пакет в зависимости от указанного IP-адреса назначения.

Почти весь или даже весь код Python, который вы напишете в своей жизни, будет выполняться на хостах на границе сети и общаться с остальным миром через один сетевой интерфейс. В таких случаях решения о маршрутизации принимаются довольно легко.

- ◆ Если IP-адрес начинается с 127.*.*, операционная система понимает, что этот пакет предназначен для другого приложения на той же машине. Он будет сразу передан другой программе путем внутреннего копирования данных операционной системой, а не через сетевое устройство.
- ◆ Если IP-адрес находится в той же подсети, что и компьютер-отправитель, целевой хост можно найти в локальном Ethernet-сегменте, беспроводном канале или другой локальной сети, а затем передать пакет на компьютер, подключенный к этой сети.
- ◆ В противном случае пакет пересылается в шлюз, который соединяет локальную подсеть с Интернетом. Шлюз сам решает, куда отправить пакет.

На границе сети принимать решения очень просто: пакет можно либо оставить в локальной сети, либо передать через Интернет. Специализированные сетевые устройства, на которых работает Интернет, принимают гораздо более сложные решения. Приходится составлять подробные таблицы маршрутизации, которые нужно постоянно обновлять на коммутаторах, соединяющих целые континенты. Пакет, предназначенный Google, идет в одном направлении, пакет на IP-адрес Amazon — в другом, а пакет для вашего компьютера — в третьем. Однако с нашей стороны маршрутизация выглядит гораздо проще.

Как компьютер определяет, каким образом поступить с пакетом для указанного IP-адреса — переслать по локальной подсети или отправить в шлюз для пересылки в Интернет? Выше мы обозначали IP-адреса в виде префиксов, за которыми следовали звездочки, чтобы показать концепцию подсети, в которой у всех хостов одинаковый префикс IP-адреса. Разумеется, в реальном мире сетевой стек операционной системы не вставляет адреса со звездочками в таблицу маршрутизации. Подсети определяются путем объединения IP-адреса с маской, которая указывает, сколько значимых битов адреса должны совпадать, чтобы хост считался принадлежащим этой подсети. Эти обозначения легко читать, если помнить, что каждый байт в IP-адресе состоит из 8 бит бинарных данных. Они выглядят следующим образом:

- ◆ **127.0.0.0/8.** Этот шаблон относится к уже знакомому нам диапазону IP-адресов. Он зарезервирован для локального хоста (localhost). Здесь указано, что первые 8 бит (1 байт) должны совпадать с числом 127, а следующие 24 бита (3 байта) могут иметь любое значение.
- ◆ **192.168.0.0/16.** Здесь должны точно совпадать первые 16 бит, т. е. под этот шаблон подходят все IP-адреса в диапазоне 192.168. Последние 16 бит 32-битного адреса могут принимать любое значение.
- ◆ **192.168.5.0/24.** Это адрес для одной подсети. Это, пожалуй, самая распространенная маска подсети в Интернете. Для того чтобы IP-адрес входил в этот диапазон, первые 3 байта должны соответствовать шаблону. Отличаться может только последний байт (последние 8 бит). Итого получается 256 уникальных адресов. 0 обычно обозначает имя подсети, а 255 используется для передачи широковещательных пакетов всем хостам в подсети (подробнее об этом — в сле-

дующей главе). Таким образом, у нас остаются 254 адреса, которые можно назначать компьютерам. Хотя шлюзу, который соединяет подсеть с Интернетом, обычно назначается адрес 1, некоторые организации выбирают другой номер.

Как и в случае с разрешением имен хостов в IP-адреса, код Python почти всегда оставляет решения о маршрутизации пакетов за операционной системой.

Фрагментация пакетов

Наконец, в связи с IP-протоколом нам осталось рассмотреть только концепцию фрагментации пакетов. Этот процесс происходит незаметно для программы, но иногда вызывает проблемы, так что о нем следует хотя бы упомянуть.

Протокол IP поддерживает очень большие пакеты, до 64 Кбайт, и иногда их приходится разбивать на части — фрагментировать, потому что сетевое оборудование часто не может обрабатывать большие пакеты. Сети Ethernet, например, пропускают пакеты не больше 1500 байт. Существует флаг, запрещающий фрагментацию (DF, don't fragment), который указывает, что именно должно произойти, если пакет слишком большой для одной из физических сетей между исходным и целевым компьютерами:

- ◆ если флаг DF не установлен, фрагментация разрешена, и когда пакет попадает в сеть, для которой он слишком велик, шлюз разделяет его на несколько пакетов и назначает их сборку на другом конце;
- ◆ если флаг DF задан, фрагментация запрещена, так что если пакет не помещается, он будет отброшен, а отправитель получит сообщение об ошибке в виде пакета ICMP (Internet Control Message Protocol — протокол управления интернет-сообщениями), попытается разделить пакет на части поменьше и отправит их повторно.

Обычно флаг DF задается операционной системой, и программа Python им не управляет. Операционная система придерживается следующей логики: если используется протокол UDP (ему посвящена *глава 2*), который передает данные в виде датаграмм, операционная система не задает флаг DF, чтобы каждая датаграмма прибыла в место назначения в любом количестве фрагментов, но если используется протокол TCP (*см. главу 3*) с длинными потоками данных из сотен и тысяч пакетов, операционная система задает флаг DF, чтобы можно было выбрать подходящий размер пакета для передачи данных без фрагментации, которая снижает надежность данных.

Максимальная единица передачи (maximum transmission unit, MTU) — это самый большой пакет, который может поместиться в подсети, и раньше обработка MTU создавала много проблем для пользователей Интернета. В 1990-е годы интернет-провайдеры (как правило, телефонные компании, предоставляющие DSL-соединения) начали использовать PPPoE — протокол, который инкапсулирует IP-пакеты в блоки по 1492 байта, а не 1500 байт, как для Ethernet. Поскольку они использовали пакеты на 1500 байт по умолчанию и отключили все пакеты ICMP в качестве

ошибочной меры безопасности, многие веб-сайты оказались к этому не готовы. Серверы не получали через ICMP уведомлений о том, что большие пакеты на 1500 байт с запретом на фрагментацию доходили до DSL-линий пользователей, но не могли войти в них.

В результате маленькие файлы и страницы работали без проблем, как и интерактивные протоколы, такие как Telnet и SSH, потому что отправляли маленькие пакеты (меньше 1492 байт). Если пользователь пытался загрузить большой файл или команда Telnet или SSH создавала много экранов с выходными данными сразу, соединение зависало и переставало работать.

Сегодня мы с таким не сталкиваемся, но эта история показывает, как низкоуровневая функция протокола IP может вызвать проблемы у пользователя и почему важно знать характеристики IP при создании сетевых программ и устранении неполадок в них.

Подробно об IP

В следующих главах мы будем рассматривать уровни выше IP и узнаем, как приложения Python могут использовать различные сервисы, созданные поверх протокола IP, для различных взаимодействий по сети. Если вы хотите узнать больше о принципах работы IP, читайте официальные ресурсы — запросы комментариев (requests for comment, RFC), выпущенные Инженерным советом Интернета (Internet Engineering Task Force, IETF). Эти документы содержат подробную информацию обо всех тонкостях протокола IP. На изучение уйдет несколько часов.

RFC часто ссылаются на другие RFC с более подробным описанием протокола или схемы адресации.

Если вы хотите узнать все об IP и протоколах, которые работают поверх него, читайте книгу Кевина Р. Фолла и У. Ричарда Стивенса "Протоколы TCP/IP: практическое руководство. Том 1"¹. В этой книге подробно описываются процессы, происходящие в протоколах. Вы можете найти и другие хорошие книги о сетях в целом и конфигурировании сетей в частности, если планируете заниматься созданием IP-сетей и маршрутизацией на работе или дома, для подключения своих компьютеров к Интернету.

Резюме

Не считая наиболее фундаментальных сетевых функций, все сетевые сервисы строятся на других, более базовых уровнях.

¹ Kevin R. Fall, W. Richard Stevens. TCP/IP Illustrated, Volume 1: The Protocols. — 2nd Edition. — Addison-Wesley Professional, 2011. — ISBN: 9780132808200.

В первых разделах этой главы мы рассматривали сетевой стек. Протокол TCP/ IP (который мы обсудим в *главе 3*) позволяет передавать потоки байтов между клиентом и сервером. Протокол HTTP (*см. главу 9*) дает возможность клиенту использовать соединение для запроса конкретного документа, и сервер может отправить этот документ в ответ. Когда документ, возвращенный сервером, должен предоставить клиенту структурированные данные, World Wide Web (*см. главу 11*) кодирует инструкции для извлечения документов по определенному адресу в стандартном формате JSON. Поверх этого стека Google предоставляет сервис геокодирования, чтобы можно было создать URL, по которому будет отвечать Google, присылая документ JSON с географическим расположением.

Символы нужно преобразовывать в байты, чтобы передавать по сети или помещать в хранилище, содержащее данные в виде байтов, например на диск. Существуют специальные системы для кодирования символов в байты. Чаще всего в Интернете используется простая, пусть и ограниченная, кодировка ASCII и расширенная система Unicode, в частности ее стандарт UTF-8. Мы можем использовать функцию `decode()` в Python, чтобы преобразовывать символьные строки в байтовые, и метод `encode()`, чтобы преобразовывать байтовые строки обратно в символы. Python 3 избегает прямого преобразования байтов в строки, потому что неизвестно, какая кодировка вам нужна, поэтому код Python 3 часто включает больше вызовов `decode()` и `encode()`, чем код Python 2.

Для того чтобы IP-сеть отправляла пакеты от имени приложения, сетевые администраторы, производители сетевого оборудования и разработчики операционных систем должны договориться о назначении IP-адресов отдельным компьютерам, создании таблиц маршрутизации на уровне компьютера и роутера и настройке системы DNS (*см. главу 4*) для привязки IP-адресов к именам, которые отображаются для пользователей. Разработчики Python должны понимать, что каждый IP-пакет следует в сети по своему пути к месту назначения, и пакет можно фрагментировать, если он слишком большой, чтобы пройти по одному из сегментов сети.

В большинстве приложений есть два способа использовать IP: обрабатывать каждый пакет как отдельное сообщение или запросить поток данных, который будет автоматически делиться на пакеты. Эти протоколы называются UDP и TCP. Мы рассмотрим их *главах 2 и 3* этой книги.

Протокол UDP

В предыдущей главе мы узнали, что современное сетевое оборудование передает небольшие сообщения, называемые пакетами, которые обычно содержат не больше пары тысяч байтов. Как объединять отдельные сообщения в связанное общение между браузером и сервером или почтовым клиентом и почтовым сервером провайдера?

Протокол IP отвечает только за доставку пакета на нужный компьютер. Для взаимодействия приложений обычно требуются еще две возможности, которые обеспечивает протокол, работающий поверх IP.

- ◆ Многочисленные пакеты, передаваемые между двумя хостами, нужно пометить, чтобы отличать веб-пакеты от почтовых, и оба этих вида от других типов сетевых взаимодействий. Передача по одному каналу нескольких потоков данных называется мультиплексированием.
- ◆ Если в потоке пакетов при передаче возникли повреждения, их нужно исправить. Пропавшие пакеты нужно отправлять повторно, пока они не дойдут до получателя. Если для пакетов важен порядок, его нужно соблюдать. Наконец, дубликаты следует уничтожить, чтобы избежать дублирования данных в потоке. При соблюдении этих условий передачу данных можно будет назвать надежной.

Двум протоколам, расположенным поверх IP, посвящены отдельные главы этой книги.

В данной главе мы поговорим о протоколе UDP (User Datagram Protocol — протокол пользовательских датаграмм). Он решает две из обозначенных нами задач. Он назначает номера портов пакетам, предназначенным для разных сервисов в одной системе, как описано далее. Если пакеты теряются, дублируются или приходят не по порядку, это не его забота.

Этими проблемами занимается протокол TCP (Transmission Control Protocol). Он использует те же правила, что и UDP, чтобы учитывать номера портов, и обеспечивает упорядоченность и надежность потоков данных, скрывая тот факт, что непрерывный поток на самом деле разделен на пакеты, которые приходится собирать в месте назначения. Об этом протоколе мы поговорим в *главе 3*.

Стоит отметить, что некоторые специализированные приложения, например для обмена мультимедийными ресурсами среди хостов по локальной сети, не используют ни один из этих протоколов, а создают новый протокол на основе IP, который сосуществует с парой TCP и UDP и по-новому обрабатывает передачу данных по

IP-сети. В этой книге мы не будем говорить о создании протоколов на Python. Создание необработанных пакетов ICMP и получение ответа ICMP в конце *главы 1* — это единственная попытка создания пакетов напрямую поверх IP в этой книге.

На самом деле, вы вряд ли будете использовать UDP в своих приложениях. Если вам кажется, что UDP хорошо подойдет для вашего приложения, рассмотрите лучшие очереди сообщений (*см. главу 8*). Тем не менее вы должны знать, как с помощью UDP происходит мультиплексирование необработанных пакетов, чтобы лучше понять протокол TCP, который мы рассмотрим в *главе 3*.

Содержание главы

- ◆ Множество сервисов в одной системе.
- ◆ Сокет — точка соединения.
- ◆ Клиенты, принимающие любые пакеты.
- ◆ Отсрочка, блокировка и время ожидания.
- ◆ UDP-сокеты.
- ◆ Идентификаторы запросов.
- ◆ От привязки до интерфейсов.
- ◆ Фрагментация UDP.
- ◆ Параметры сокетов.
- ◆ Широковещание.
- ◆ Сценарии применения UDP.
- ◆ Резюме.

Цель

В этой главе мы подробно рассмотрим протокол UDP.

Множество сервисов в одной системе

В сфере компьютерных сетей и электромагнитных сигналов мы сталкиваемся со сложностями при попытке различить многочисленные сигналы, использующие один канал. Мультиплексирование позволяет разным потокам путешествовать одним транспортом. Радиосигналы можно различать по частотам. Создатели UDP решили помечать каждый UDP-пакет парой 16-битных номеров портов без знака в диапазоне от 0 до 65 536, чтобы разные потоки не смешивались. Исходный (source) порт обозначает отправителя, а целевой (destination) — получателя сообщения, который находится по указанному IP-адресу.

На уровне IP-сети мы видим только пакеты, которые передаются на определенный хост.

Source (IP-адрес) | Destination (IP-адрес)

Однако на одном компьютере может быть несколько программ, которые отправляют и получают пакеты. У них, кроме адреса, есть еще и номер порта.

Source (IP-адрес:номер порта) | Destination (IP-адрес:номер порта)

Эти четыре значения будут всегда одинаковыми у пакетов из конкретного взаимодействия. У обратных пакетов просто поменяются местами координаты отправителя и получателя.

Допустим, мы создаем DNS-сервер (см. главу 4) с IP-адресом 192.168.1.9 на одной из рабочих станций. Сервер запросит у операционной системы разрешение получать пакеты, поступающие на порт UDP, по стандартному номеру порта DNS — 53, чтобы другие компьютеры видели этот сервис. Если этот порт не занят другим процессом, DNS-сервер получит его.

Допустим, у нас есть клиентский компьютер с IP-адресом 192.168.1.30, который хочет отправить запрос на сервер. Он создаст запрос в памяти и попросит операционную систему передать этот блок данных в виде UDP-пакета. Поскольку мы должны определить клиента при возврате пакета, а клиент не запросил номер порта, операционная система назначит ему случайный порт, например 44137.

В результате пакет отправится к порту 53 со следующими адресами:

Source (192.168.1.30:44137) | Destination (192.168.1.9:53)

Когда DNS-сервер создаст ответ, он попросит операционную систему передать UDP-пакет с этими двумя адресами в обратном порядке, и ответ вернется напрямую отправителю.

Source (192.168.1.9:53) | Destination (192.168.1.30:44137)

UDP работает довольно просто — ему достаточно знать IP-адрес и порт получателя, чтобы отправить пакет.

Откуда клиентская программа узнает, к какому порту подключиться? Существует три подхода.

- ◆ IANA (Internet Assigned Numbers Authority — администрация адресного пространства Интернета) занимается стандартизацией назначения имен и номеров. Поэтому в предыдущем случае мы искали DNS по порту UDP 53.
- ◆ Автоматическая конфигурация. Когда компьютер подключается к сети через какой-то протокол, например DHCP, он узнает IP-адреса ключевых сервисов, вроде DNS. Затем программа может обращаться к этим сервисам, сопоставляя IP-адреса с известными номерами портов.
- ◆ Конфигурация вручную. В остальных случаях администратору или пользователю необходимо вручную присвоить сервису IP-адрес или имя хоста. Например, мы можем вручную ввести имя веб-сервера в браузер.

Выбирая номер порта, например 53 для DNS, IANA делит их на две категории. Для портов UDP и TCP есть три диапазона номеров портов.

- ◆ Самые важные и популярные сервисы получают известные номера от 0 до 1023. Во многих UNIX-подобных операционных системах эти порты нельзя назначить обычным пользовательским программам. Благодаря этому запрету пользователи не могут выдавать свои приложения за критически важные системные сервисы.
- ◆ Зарегистрированные порты от 1024 до 49151 операционная система разрешает использовать кому угодно. Например, пользовательское приложение может ожидать передачи данных по порту 5432 и выполнять функции базы данных PostgreSQL. IANA рекомендует регистрировать их для конкретных сервисов и использовать только для них.
- ◆ Все остальные номера портов (49152–65535) доступны для свободного использования. Операционная система выбирает произвольные номера портов из этого пула, когда клиенту не важно, какой порт будет назначен исходящему соединению.

Когда мы пишем приложения, которые принимают номера портов от пользователя, например из командной строки или файла конфигурации, лучше дополнять числовые значения известных портов именами, понятными человеку. Это стандартные имена, которые можно получить с помощью метода `getservbyname()` стандартного модуля Python `socket`. Например, с помощью этого метода мы можем узнать порт Domain Name Service.

```
>>> import socket
>>> socket.getservbyname('domain')
53
```

Более сложная функция `getaddrinfo()`, которая также предоставляется в модуле `socket`, тоже может декодировать имена портов, как мы увидим в *главе 4*.

На устройствах Linux и Mac OS X база данных имен и номеров портов известных сервисов обычно хранится в файле `/etc/services`, который вы можете изучить при желании. Например, на первых нескольких страницах файла приводятся старые протоколы с зарезервированными номерами, хотя они уже много лет не получают никаких пакетов. IANA также предоставляет актуальную и более подробную версию: www.iana.org/assignments/port-numbers.

Сокет — точка соединения

В Python используется интересная альтернатива API сетевого программирования. На базовом уровне стандартная библиотека Python предоставляет только интерфейс на основе объекта для всех низкоуровневых вызовов операционной системы, с помощью которых обычно осуществляются сетевые операции в операционных системах, соответствующих стандарту POSIX. Эти вызовы даже названы по процессам, которые инкапсулируют. Не зря Python стал глотком свежего воздуха для всех, кто работал с низкоуровневыми языками в начале 1990-х годов, ведь он предоставлял

стандартные системные вызовы, которые уже были всем знакомы. Наконец-то у нас появился высокоуровневый язык, с помощью которого можно было напрямую работать с низкоуровневыми вызовами операционной системы и не использовать громоздкий и ограниченный, но внешне более красивый API для того или иного языка. Было гораздо проще запомнить один набор вызовов, который работал как с C, так и с Python.

В системах Windows и POSIX (например, Linux и Mac OS X) базовые системные вызовы для сетевых операций работали с конечной точкой, называемой *сокетом*. Операционная система идентифицирует сокеты по целым числам, а Python предлагает более удобный объект `socket.socket`. По сути, он сохраняет целое число (его можно посмотреть, вызвав метод `fileno()`) и использует его автоматически, когда мы вызываем один из методов для запроса системного вызова к сокету.

Обратите внимание, что целое число `fileno()`, которое идентифицирует сокет в системах POSIX, также является дескриптором файла, выбранным из пула целых чисел, представляющих открытые файлы. Иногда в окружениях POSIX код получает это целое число и с его помощью выполняет с дескриптором файла операции, которые обычно применяются к файлам, а не сетевым объектам, например `os.read()` и `os.write()`. Однако в этой книге мы будем выполнять с сокетом только стандартные операции, потому что наши примеры кода должны работать и в Windows.

Как выглядят используемые сокеты? В листинге 2.1 приводится пример простого UDP-сервера и UDP-клиента. Мы видим, что он использует стандартную библиотеку Python только для одного вызова, к `socket.socket()`, а остальные вызовы направлены к методам возвращенного объекта сокета.

Листинг 2.1. В интерфейсе обратной связи у нас UDP-сервер и UDP-клиент

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# UDP-клиент и сервер на localhost

import argparse, socket
from datetime import datetime

MAX_BYTES = 65535

def server(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('127.0.0.1', port))
    print('Listening at {}'.format(sock.getsockname()))
    while True:
        data, address = sock.recvfrom(MAX_BYTES)
```

```

    text = data.decode('ascii')
    print('The client at {} says {!r}'.format(address, text))
    text = 'Your data was {} bytes long'.format(len(data))
    data = text.encode('ascii')
    sock.sendto(data, address)

def client(port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    text = 'The time is {}'.format(datetime.now())
    data = text.encode('ascii')
    sock.sendto(data, ('127.0.0.1', port))
    print('The OS assigned me the address {}'.format(sock.getsockname()))
    data, address = sock.recvfrom(MAX_BYTES) # Осторожно! См. главу 2
    text = data.decode('ascii')
    print('The server {} replied {!r}'.format(address, text))

if __name__ == '__main__':
    choices = {'client': client, 'server': server}
    parser = argparse.ArgumentParser(description='Send and receive UDP locally')
    parser.add_argument('role', choices=choices, help='which role to play')
    parser.add_argument('-p', metavar='PORT', type=int, default=1060,
                        help='UDP port (default 1060)')
    args = parser.parse_args()
    function = choices[args.role]
    function(args.p)

```

Клиент и сервер используют только IP-адрес в localhost, который должен быть доступен локально, поэтому мы сможем выполнить скрипт на компьютере даже без подключения к сети. Сначала запускаем сервер:

```
$ python udp_local.py server
```

```
Listening at ('127.0.0.1', 1060)
```

Сервер ждет входящее сообщение после вывода этой строки.

Как видно из исходного кода, сервер отработал в три этапа.

Сначала он вызвал метод `socket()`, чтобы создать простой сокет. Пока у него нет ни IP, ни порта.

Если попытаться с ним связаться, мы получим ошибку, потому что пока он ни к чему не подключен. Мы видим, что у сокета есть определенный тип: он относится к семейству адресов `AF_INET`. Тип датаграммы — `SOCK_DGRAM`, т. е. он будет работать в IP-сети с помощью UDP. В следующем разделе вы поймете, почему я провожу параллель между датаграммами и пакетами для расчета максимальной единицы передачи (maximum transmission unit, MTU).

Затем инструкция `bind()` запрашивает сетевой адрес UDP, который представляет собой базовый кортеж Python, состоящий из строкового значения IP-адреса (имя хоста тоже подойдет) и целочисленного значения номера порта UDP. Если другая программа уже использует этот порт UDP и серверный скрипт не может получить к нему доступ, этот шаг завершится ошибкой. Если мы запустим еще одну копию сервера, то получим ошибку с сообщением о том, что этот адрес уже используется:

```
$ python udp_local.py server
```

```
Traceback (most recent call last):
```

```
...
```

```
OSError: [Errno 98] Address already in use
```

Компьютер уже использует UDP-порт 1060, поэтому есть вероятность, что при первом запуске сервера возникнет эта ошибка. Когда я выбирал номер порта для первого примера, я сомневался. Разумеется, это число должно было быть больше 1023, потому что иначе скрипт нужно было запускать от имени системного администратора. Я ручаюсь за качество своих скриптов, но не хочу, чтобы их запускали от имени администратора. Я мог бы позволить операционной системе самой выбрать номер порта (я поступил так с клиентом, как вы вскоре сами увидите), вывести этот номер, а затем передать его клиенту как один из аргументов командной строки. Однако тогда я не смог бы показать вам синтаксис запроса определенного номера порта. Наконец, я подумывал выбрать порт из диапазона больших чисел, но этот порт в тот момент мог быть занят другим приложением на вашем компьютере, например браузером или SSH-клиентом.

В итоге я решил, что у меня есть только один вариант — использовать порт в зарезервированном, но неизвестном диапазоне выше 1023. Я посмотрел на список и подумал, что, скорее всего, у вас на компьютере не установлен SAP BusinessObjects Polestar. Если установлен, используйте параметр `-p`, чтобы выбрать для сервера другой номер порта.

Программа Python всегда может использовать метод сокета `getsockname()`, чтобы получить кортеж, содержащий текущий IP-адрес и порт этого сокета.

Итак, мы привязали сокет, и теперь сервер готов получать запросы. Он входит в цикл и непрерывно вызывает `recvfrom()`, сообщая функции, что готов принимать сообщения длиной до 65 535 байт — максимальная длина датаграммы UDP, чтобы мы всегда видели полное содержимое каждой датаграммы. Метод `recvfrom()` ждет сообщения от клиента неопределенное время.

При получении датаграммы `recvfrom()` возвращает адрес клиента, отправившего ее, а также ее содержимое в виде байтов. Мы выводим сообщение на консоль, а затем отправляем клиенту ответную датаграмму, потому что Python умеет преобразовывать байты в строки.

Итак, давайте запустим наш клиент и посмотрим, что будет. Листинг 2.1 включает код клиента.

В этой книге я часто привожу код сервера и клиента в одном листинге, используя параметры командной строки. Надеюсь, вы не запутаетесь. Лично мне так удобнее,

потому что так логика клиента и сервера находится на одной странице, а мне легче понять, какой фрагмент серверного кода относится к тому или иному фрагменту клиентского кода.

Откройте еще одно командное окно на компьютере при работающем сервере и попробуйте дважды запустить клиента:

```
$ python udp_local.py client
```

```
The OS assigned me the address ('0.0.0.0', 46056)
```

```
The server ('127.0.0.1', 1060) replied 'Your data was 46 bytes long'
```

```
$ python udp_local.py client
```

```
The OS assigned me the address ('0.0.0.0', 39288)
```

```
The server ('127.0.0.1', 1060) replied 'Your data was 46 bytes long'
```

Каждое соединение, обслуживаемое сервером, должно отражаться в командном окне сервера.

```
The client at ('127.0.0.1', 46056) says 'The time is 2014-06-05 10:34:53.448338'
```

```
The client at ('127.0.0.1', 39288) says 'The time is 2014-06-05 10:34:54.065836'
```

Клиентский код чуть проще серверного и состоит всего из трех строк, но в нем мы видим сразу две новые концепции. Метод `sendto()` принимает от клиента сообщение и адрес назначения. Этого однократного вызова достаточно, чтобы отправить датаграмму на сервер. Но если взаимодействие должно быть двусторонним, нам потребуется IP-адрес и номер порта клиента. Как видно в выходных данных `getsockname()`, операционная система назначает их автоматически. Все номера портов клиентов находятся в диапазоне, который IANA выделила для временных портов, как мы и ожидали. По крайней мере это то, что я вижу на своем компьютере с Linux. Если у вас другая операционная система, результаты могут быть иными.

Закончив использовать сервер, нажимаем комбинацию клавиш `<Ctrl>+<C>` в терминале, чтобы завершить его работу.

Клиенты, принимающие любые пакеты

Клиентский код в листинге 2.1 на самом деле связан с рисками. Обратите внимание, что метод `recvfrom()` предоставляет адрес входящей датаграммы, но функция не проверяет исходный адрес датаграммы с целью убедиться, что ответ действительно поступает от сервера.

Это можно проверить, если задержать ответ от сервера и посмотреть, может ли другой источник предоставить ответ, который будет принят нашим наивным клиентом. В менее производительных операционных системах, например Windows, нужно будет добавить значительную задержку. Для того чтобы имитировать медленный сервер, вставьте функцию `sleep()` между получением запроса и отправкой ответа на сервере. Если сервер привязан к сокету, в Mac OS X и Linux можно приостановить его, нажав комбинацию клавиш `<Ctrl>+<Z>`, чтобы симитировать сер-

вер с большой задержкой. Запускаем новый сервер и приостанавливаем его нажатием комбинации клавиш <Ctrl>+<Z>.

```
$ python udp_local.py server
```

```
Listening at ('127.0.0.1', 1060)
```

```
^Z
```

```
[1] + 9370 suspended python udp_local.py server
```

```
$
```

Если запустить клиента теперь, он отправит датаграмму и зависнет в ожидании ответа.

```
$ python udp_local.py client
```

```
The OS assigned me the address ('0.0.0.0', 39692)
```

Допустим, злоумышленник хочет подделать ответ от сервера, передав датаграмму до того, как сервер успеет ответить. Клиент сообщил операционной системе, что готов получить любую датаграмму, и никак не проверяет поступающие данные, поэтому он поверит, что поддельный ответ исходит от сервера. Давайте попробуем отправить такой пакет из командной строки Python.

```
$ python3
```

```
Python 3.4.0 (default, jan 25 2021, 13:05:18)
```

```
[GCC 4.8.2] on linux
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> import socket
```

```
>>> sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
>>> sock.sendto('FAKE'.encode('ascii'), ('127.0.0.1', 39692))
```

```
4
```

Клиент без раздумий примет поддельный ответ за настоящий.

```
The server ('127.0.0.1', 37821) replied 'FAKE'
```

Теперь давайте введем fg, чтобы снова запустить сервер. Он увидит пакет клиента в очереди и попытается ответить на запрос через клиентский сокет, который уже закрыт. Для того чтобы завершить процесс, нажмите комбинацию клавиш <Ctrl>+<C>.

Следует отметить, что отправить клиенту UDP-пакет может кто угодно. Здесь речь не идет об атаке посредника (man-in-the-middle), который получает контроль над сетью и может подделывать пакеты с фальшивых адресов, так что защититься от него можно только шифрованием (см. главу 6). Просто любой отправитель, который соответствует требованиям, может отправить пакет, и он будет принят.

В неразборчивом (promiscuous) режиме клиент, ждущий пакеты, принимает все, что ему поступает, не глядя на источник. Иногда мы создаем таких клиентов намеренно, например, когда отслеживаем поведение сети и хотим собрать все пакеты, поступающие на определенный интерфейс. В нашем сценарии неразборчивый режим не подходит.

Мы должны использовать продуманное шифрование, чтобы код был уверен, что общается с нужным сервером. А пока можно выполнить две небольшие проверки. Во-первых, мы должны создать или использовать протоколы, которые предоставляют уникальное обозначение или указывают в запросе идентификатор, который затем должен повториться в ответе. Если ответ содержит нужный идентификатор, значит, он поступает от источника, который по крайней мере видел наш запрос. Однако диапазон идентификаторов должен быть достаточно большим, чтобы нас не завалило тысячами и миллионами пакетов со всеми возможными идентификаторами. Во-вторых, мы должны использовать `connect()`, чтобы запретить другим адресам отправлять нам пакеты на основе сравнения адреса ответного пакета с адресом, по которому вы его предоставили (если помните, кортежи в Python можно легко сравнивать с помощью `==`). Подробнее об этом *см. в разд. "UDP-сокеты" и "Идентификаторы запросов" далее в этой главе.*

Отсрочка, блокировка и время ожидания

В листинге 2.1 пакеты никак не могли потеряться, потому что клиент и сервер работали на одном компьютере и общались через интерфейс обратной связи (а не через физическую сетевую плату, у которой мог случиться сбой). Что нужно добавить в код в том случае, если риск потери пакетов существует?

Давайте посмотрим на пример в листинге 2.2. Сервер отвечает не на все клиентские запросы, а только на случайно выбранные, чтобы мы увидели, как реализовать надежность в клиентском коде, и при этом не ждали часами, когда потеря пакета произойдет сама собой.

Листинг 2.2. Сервер и клиент UDP на разных компьютерах

```
#!/usr/bin/env python3

# Network Programming in Python: The Basics
# Клиент и сервер UDP общаются по сети

import argparse, random, socket, sys

MAX_BYTES = 65535

def server(interface, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((interface, port))
    print('Listening at', sock.getsockname())
    while True:
        data, address = sock.recvfrom(MAX_BYTES)
```



```

    if random.random() < 0.5:
        print('Pretending to drop packet from {}'.format(address))
        continue
    text = data.decode('ascii')
    print('The client at {} says {!r}'.format(address, text))
    message = 'Your data was {} bytes long'.format(len(data))
    sock.sendto(message.encode('ascii'), address)

def client(hostname, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    hostname = sys.argv[2]
    sock.connect((hostname, port))
    print('Client socket name is {}'.format(sock.getsockname()))

    delay = 0.1 # секунд
    text = 'This is another message'
    data = text.encode('ascii')
    while True:
        sock.send(data)
        print('Waiting up to {} seconds for a reply'.format(delay))
        sock.settimeout(delay)
        try:
            data = sock.recv(MAX_BYTES)
        except socket.timeout as exc:
            delay *= 2 # ждать дольше следующего запроса
            if delay > 2.0:
                raise RuntimeError('I think the server is down')
        else:
            break # мы закончили, можно разорвать обратную связь

    print('The server says {!r}'.format(data.decode('ascii')))

if __name__ == '__main__':
    choices = {'client': client, 'server': server}
    parser = argparse.ArgumentParser(description='Send and receive UDP, '
                                                'pretending packets are often dropped')
    parser.add_argument('role', choices=choices, help='which role to take')
    parser.add_argument('host', help='interface the server listens at;'
                        'host the client sends to')
    parser.add_argument('-p', metavar='PORT', type=int, default=1060,
                        help='UDP port (default 1060)')
    args = parser.parse_args()
    function = choices[args.role]
    function(args.host, args.p)

```

В предыдущем примере сервер сообщал операционной системе, что принимает пакеты только с частного IP-адреса 127.0.0.1 от других процессов на том же компьютере. На этот раз мы можем убрать это ограничение, определив IP-адрес сервера как пустую строку. Система интерпретирует это как "любой локальный интерфейс". На моем компьютере с Linux это приводит к тому, что мы запрашиваем у операционной системы IP-адрес 0.0.0.0.

```
$ python udp_remote.py server ""
```

```
Listening at ('0.0.0.0', 1060)
```

При получении запроса сервер использует функцию `random()`, чтобы решить, стоит ли отвечать на запрос. Так нам не придется целый день ждать, пока пакет настоящему потеряется. Какое бы решение он ни принял, сервер выдает сообщение, чтобы держать вас в курсе. Как создать "настоящий" UDP-клиент, который будет обрабатывать возможную потерю пакетов?

Из-за нестабильности UDP клиенту придется выполнять запрос циклом. Он должен быть готов ждать ответа либо бесконечно, либо в течение заданного времени, после которого нужно будет отправить запрос повторно. Принять решение сложно, потому что большинство клиентов не различают эти три типа событий:

- ◆ ответ идет долго, но скоро придет;
- ◆ ответ никогда не придет, потому что потерялся;
- ◆ сервер не работает и не отвечает на запросы.

Если через какое-то время ответ так и не поступил, UDP-клиент должен отправить повторный запрос. Конечно, так мы можем зря потратить время сервера, ведь, возможно, ответ уже в пути, и придется обрабатывать тот же запрос снова, но все же в какой-то момент клиент должен решить, стоит ли отправить запрос повторно или ждать бесконечно.

Для того чтобы не позволять операционной системе замораживать сокет на неопределенное время при выполнении операции `recv()`, этот клиент сначала вызывает метод `settimeout()`. Система понимает, что клиент не хочет ждать дольше, чем указано, и в какой-то момент вызов нужно прервать. Если заданное число секунд прошло, вызывается исключение окончания времени ожидания.

Вызывающий объект считается заблокированным, пока ждет ответа от сети. Эта блокировка связана с вызовом `recv()`, который заставляет клиента ждать новые данные. Подробнее о блокирующих и неблокирующих сетевых запросах мы поговорим в *главе 7*, которая посвящена серверной архитектуре.

Например, у этого клиента установлена задержка в 1/10 секунды. В моей домашней сети, где время ответа составляет всего несколько десятков миллисекунд, это редко приводит к тому, что клиент отправляет повторный запрос, просто потому что ответ задерживается.

Очень важно правильно написать логику поведения на случай, когда время ожидания истечет. Нельзя просто отправлять повторные запросы через указанный период. Перегрузка канала является самой частой причиной потери пакетов. Это знают

все, кто хоть раз пытался отправить обычные данные через DSL-модем, пока загружается картинка или видео. В таких случаях отправлять еще больше пакетов взамен потерянных не лучшее решение.

Поэтому клиенты используют стратегию экспоненциальной задержки, в соответствии с которой повторные попытки предпринимаются все реже. Это позволяет продолжить работу в случае потери нескольких запросов или ответов и при этом постепенно восстановить работоспособность перегруженной сети, поскольку все активные клиенты начинают отправлять меньше пакетов. Существуют более сложные алгоритмы экспоненциальной задержки, например версия для Ethernet, которая включает случайный фактор, чтобы две конкурирующие сетевые платы не активировали задержку по одному расписанию, но в целом достаточно будет удваивать время отсрочки каждый раз, когда ответ не приходит.

Если запросы отправляются на сервер, до которого 200 мс, этот наивный метод успеет отправить не меньше двух копий каждого запроса, потому что он не знает, что путь до сервера занимает дольше 0,1 с. Если вы создаете долгосрочный UDP-клиент, пусть он запоминает длительность предыдущих нескольких запросов, чтобы соответствующим образом рассчитывать задержку до первой повторной попытки.

Укажите имя хоста другого компьютера, на котором выполняется серверный скрипт, для клиента в листинге 2.2, как указано ранее. Если клиенту повезет, он получит ответ сразу.

```
$ python udp_remote.py client Guinness
```

```
Client socket name is ('127.0.0.1', 45420)
```

```
Waiting up to 0.1 seconds for a reply
```

```
The server says 'Your data was 23 bytes long'
```

Однако часто запросы будут оставаться без ответа, и придется отправлять их снова. Мы можем наблюдать экспоненциальную задержку в реальном времени, потому что выходные данные будут выводиться все реже и реже.

```
$ python udp_remote.py client guinness
```

```
Client socket name is ('127.0.0.1', 58414)
```

```
Waiting up to 0.1 seconds for a reply
```

```
Waiting up to 0.2 seconds for a reply
```

```
Waiting up to 0.4 seconds for a reply
```

```
Waiting up to 0.8 seconds for a reply
```

```
The server says 'Your data was 23 bytes long'
```

Можно проверить, доходит ли запрос до адресата или у вас в сети на самом деле теряются пакеты, в терминале, где выполняется сервер. Я вижу, что при проведении предыдущего теста все пакеты дошли до сервера.

```
Pretending to drop packet from ('192.168.5.10', 53322)
```

```
Pretending to drop packet from ('192.168.5.10', 53322)
```

```
Pretending to drop packet from ('192.168.5.10', 53322)
```

```
Pretending to drop packet from ('192.168.5.10', 53322)
```

```
The client at ('192.168.5.10', 53322) says, 'This is another message'
```

А если сервер полностью недоступен? К сожалению, UDP не видит разницы между недоступным сервером и перегруженной сетью. Все, что остается клиенту, — в какой-то момент прекратить попытки. Мы перезапускаем клиента после завершения серверного процесса.

```
$ python udp_remote.py client Guinness
```

```
Client socket name is ('127.0.0.1', 58414)
```

```
Waiting up to 0.1 seconds for a reply
```

```
Waiting up to 0.2 seconds for a reply
```

```
Waiting up to 0.4 seconds for a reply
```

```
Waiting up to 0.8 seconds for a reply
```

```
Waiting up to 1.6 seconds for a reply
```

```
Traceback (most recent call last):
```

```
...
```

```
socket.timeout: timed out
```

```
The preceding exception resulted in the following exception:
```

```
Traceback (most recent call last):
```

```
...
```

```
RuntimeError: I think the server is down
```

Прекращать повторные попытки следует только в том случае, если программа пытается выполнить небольшую задачу, которая требует выдать выходные данные или вернуть результат пользователю. Попытки могут быть и бесконечными, если это программа-демон, которая выполняется весь день, например погодный виджет в углу экрана, который показывает температуру и прогноз с удаленного UDP-сервиса. Ноутбук может быть долго отключен от сети, и код часами или даже днями будет ждать ответа от сервера погоды.

Если мы пишем демон, который не прекращает повторные попытки, не стоит настраивать строгую экспоненциальную задержку, или очень скоро между повторными попытками будут проходить часы, а мы рискуем пропустить короткий период, когда пользователь с ноутбуком зайдет в кафе и наконец подключится к сети. Лучше указать максимальное значение, скажем, пять минут, до которого должна дойти экспоненциальная задержка, чтобы запрос точно прошел, если пользователь оказался в сети хотя бы на пять минут.

Нам не придется высчитывать время и гадать, когда подключение к сети восстановится, если операционная система позволяет процессам получать оповещения о таких событиях, как подключение к сети. К сожалению, в этой книге мы не будем рассматривать такие процедуры. Давайте вернемся к UDP.

UDP-сокеты

В листинге 2.2 осталась еще одна концепция, которую мы не рассмотрели. Мы уже говорили о привязках. Как о явных привязках с помощью вызова `bind()`, который сервер отправляет для получения нужного адреса, так и о неявных привязках, кото-

рые возникают, когда клиент впервые пытается использовать сокет, а операционная система назначает ему случайный временный номер порта.

Удаленный UDP-клиент в листинге 2.2 использует еще незнакомый нам метод: `connect()`. Как видите, он прекрасно справляется. Вместо того чтобы использовать `sendto()` с явным кортежем адресов при каждой отправке запроса на сервер, можно применять вызов `connect()`, который заранее сообщает операционной системе удаленные адреса, по которым мы хотим отправить пакеты, чтобы нам оставалось только предоставить данные в вызове `send()` и не приходилось указывать адрес сервера снова.

Метод `connect()` делает еще кое-что, чего мы не видим в листинге 2.2, — он решает проблему клиента, который принимает любые пакеты! Если мы повторим наш тест, то увидим, что клиент не принимает посторонние пакеты. Дело в том, что с помощью `connect()` мы настроили предпочитаемый адрес назначения для UDP-сокета: после выполнения `connect()` любые входящие пакеты, обратный адрес которых не совпадает с адресом, к которому вы подключились, будут отбрасываться операционной системой.

Есть два способа создать UDP-клиента, который обращает внимание на обратный адрес возвращаемых пакетов.

- ◆ Можно использовать `sendto()`, чтобы направить исходящий пакет в указанное место назначения, а затем применить `recvfrom()`, чтобы получать ответы и сравнивать каждый обратный адрес со списком серверов, от которых клиент ждет ответа.
- ◆ Или можно напрямую использовать `connect()` сразу после создания сокета, а затем указывать `send()` и `recv()` для взаимодействия. Операционная система будет отфильтровывать нежелательные пакеты. Если снова подключиться к тому же сокету, второй адрес назначения не добавится, так что этот способ подходит лишь для взаимодействия с одним сервером за раз, потому что исходный адрес полностью удаляется, чтобы следующий трафик с этого адреса не проходил к приложению.

Можно использовать метод `getpeername()` UDP-сокета, чтобы запомнить адрес, к которому он был подключен после использования `connect()`. Не вызывайте этот метод для сокета, который еще не подключен, — вы получите ошибку `socket.error`, а не `0.0.0.0` или подобный ответ с подстановочными символами. Осталось прояснить еще пару моментов, касающихся функции `connect()`.

Во-первых, при подключении к UDP-сокету по сети не передаются никакие данные, а сервер не начинает ждать пакеты. Просто в памяти операционной системы сохраняется адрес, который потом будет использоваться в вызовах `send()` и `recv()`.

Во-вторых, помните, что подключение к серверу или даже отфильтровывание нежелательных пакетов с помощью обратного адреса — это небезопасный метод! Злоумышленник в сети сможет создавать пакеты с обратным адресом сервиса, которые пройдут через этот фильтр.

Отправка пакетов с обратным адресом другого компьютера называется *спуфингом*, и разработчики протоколов должны учитывать этот риск при создании протоколов, защищенных от вмешательства извне.

Подробнее об этом мы поговорим в *главе 6*.

Идентификаторы запросов

В листингах 2.1 и 2.2 мы видим открытые текстовые сообщения в кодировке ASCII. Если мы захотим создать свою схему UDP-запросов и ответов, возможно, следует назначать каждому запросу порядковый номер и следить за тем, чтобы у принимаемого ответа был тот же номер. Достаточно просто скопировать номер из запроса в ответ на сервере. Это дает нам минимум два важных преимущества.

Так можно будет различать дублирующие ответы на запросы, которые клиент отправил несколько раз с экспоненциальной задержкой.

Допустим, мы отправляем запрос *A*. Ответ не приходит, и мы отправляем запрос *A* снова. Наконец, мы получаем ответ *A*. Мы думаем, что первый запрос, наверное, потерялся, и продолжаем работу.

А если до сервера дошли оба запроса, просто ответы идут с задержкой?

Что если мы получили один из двух ответов, но ждали другой? Если теперь мы отправим к серверу запрос *B* и будем ждать ответ, но получим ответ на запрос *A*, мы можем принять его за ответ на запрос *B*. В итоге возникнет серьезная путаница — мы будем считать ответ на предыдущий запрос ответом на текущий.

Идентификаторы запросов защищают нас от таких ситуаций. Если у всех копий запросов *A* будет идентификатор 42496, а у всех копий запроса *B* — идентификатор 16916, и мы ждем ответ на запрос *B*, мы будем просто удалять все ответы с другими идентификаторами, пока не получим ответ 16916. Такой подход защищает нас от повторяющихся ответов, которые могут поступать не только из-за повторных запросов, но и в тех редких случаях, когда из-за избыточности сетевой фабрики где-то между сервером и клиентом создаются две копии пакета.

Еще один вариант применения идентификаторов запросов — защита от спуфинга, хотя бы в ситуациях, когда злоумышленник не может читать пакеты. Если может, это вас не спасет: он будет видеть IP, номер пакета и идентификатор запроса в каждом передаваемом пакете, и сможет подделать ответ на любой запрос (и надеяться, что он придет раньше, чем ответ от законного сервера). Если злоумышленник не может читать трафик и отправляет UDP-пакеты на сервер вслепую, длинный идентификатор запроса повысит вероятность того, что клиент не будет принимать поддельные запросы.

Как видите, я использую идентификаторы запросов не по порядку, чтобы злоумышленник не мог просто угадать их. Если назначать идентификаторы по порядку, риск будет выше. Создавать большие целые числа можно с помощью модуля `random`. Если идентификатор будет случайным числом от 0 до N , вероятность того,

что злоумышленник его угадает, равна максимум $1/N$, и даже ниже, если придется перебрать все номера портов в системе.

Конечно, все это не самые надежные меры безопасности. Они защищают только от простейших попыток спуфинга со стороны злоумышленников, которые не видят наш сетевой трафик. В *главе 6* мы поговорим о том, как защититься даже от злоумышленников, которые могут отслеживать наш трафик и вставлять свои сообщения.

От привязки до интерфейсов

Пока мы видели две альтернативы IP-адреса, который можно использовать в методе `bind()` сервера. Если мы укажем `127.0.0.1`, значит, мы принимаем пакеты только от программ на том же компьютере. Если это будет пустая строка, это будет означать прием пакетов через любой сетевой интерфейс сервера.

Здесь есть варианты. Например, можно указать IP-адрес одного из внешних IP-интерфейсов, например платы Ethernet или беспроводной связи, и сервер будет ожидать пакеты только для этого IP-адреса. Вы могли заметить, что в листинге 2.2 можно указать строку сервера для метода `bind()`, чтобы провести тесты.

Что будет, если просто выполнить привязку к внешнему интерфейсу? Используйте для сервера внешний IP-адрес, который сообщит вам операционная система:

```
$ python udp_remote.py server 192.168.5.130
```

```
Listening at ('192.168.5.130', 1060)
```

Мы должны легко подключиться к этому IP-адресу с другого компьютера.

```
$ python udp_remote.py client Guinness
```

```
Client socket name is ('192.168.5.10', 35084)
```

```
Waiting up to 0.1 seconds for a reply
```

```
The server says 'Your data was 23 bytes'
```

Если мы попробуем подключиться к сервису через интерфейс обратной связи, когда скрипт клиента выполняется на том же компьютере, пакеты не достигнут места назначения.

```
$ python udp_remote.py client 127.0.0.1
```

```
Client socket name is ('127.0.0.1', 60251)
```

```
Waiting up to 0.1 seconds for a reply
```

```
Traceback (most recent call last):
```

```
...
```

```
socket.error: [Errno 111] Connection refused
```

Вообще-то результат получился лучше, чем если бы пакеты просто не были доставлены. Во всяком случае, в моей операционной системе. Поскольку операционная система может определить, что порт открыт, даже без отправки пакета, подключение к этому порту сразу запрещается. Однако имейте в виду, что получать от UDP такие удобные сообщения об отклонении соединения (`connection rejected`) мы можем только в интерфейсе обратной связи, но не в реальной сети. Там пакет просто

передается без указания, будет ли он получен целевым портом. Перезапустим клиента на том же компьютере, но на этот раз используем внешний IP-адрес:

```
$ python udp_remote.py client 192.168.5.130
```

```
Client socket name is ('192.168.5.130', 34919)
```

```
Waiting up to 0.1 seconds for a reply
```

```
The server says 'Your data was 23 bytes'
```

Видите, что происходит? Локальные программы могут отправлять запросы на любые IP-адреса компьютера, даже если они собираются использовать их только для общения с другим сервисом на том же компьютере.

В результате привязка к IP-интерфейсу ограничивает адреса, по которым внешние хосты могут взаимодействовать с вами. Однако она не мешает общению с другими клиентами на том же компьютере, если они знают, к какому IP-адресу следует подключиться.

Что будет, если запустить два сервера одновременно? Давайте остановим все выполняющиеся скрипты и запустим два сервера на одном компьютере. Интерфейс обратной связи подключится к одному из них.

```
$ python udp_remote.py server 127.0.0.1
```

```
Listening at ('127.0.0.1', 1060)
```

Мы не можем запустить другой сервер по этому адресу, ведь он уже занят. Операционная система не сможет различить, какой из двух процессов должен получить пакет, поступающий по этому адресу.

```
$ python udp_remote.py server 127.0.0.1
```

```
Traceback (most recent call last):
```

```
...
```

```
OSError: [Errno 98] Address already in use
```

Что еще более странно, мы не сможем запустить сервер по IP-адресу с подстановочными символами.

```
$ python udp_remote.py server
```

```
Traceback (most recent call last):
```

```
...
```

```
OSError: [Errno 98] Address already in use
```

Проблема в том, что этот адрес включает адрес 127.0.0.1, который уже занят первым сервером. А если мы запустим второй сервер не по всем IP, а только по внешнему IP-интерфейсу, который первый сервер не прослушивает? Давайте попробуем.

```
$ python udp_remote.py server 192.168.5.130
```

```
Listening at ('192.168.5.130', 1060)
```

Все получилось! Теперь два сервера с одним номером порта UDP работают на одном компьютере, только один обслуживает внутренние запросы, а другой направлен вовне и ждет пакеты от сети, к которой подключена плата беспроводной связи на моем компьютере. Можно запустить еще несколько серверов, по одному для каждого удаленного интерфейса, если у вас их несколько.

Затем попробуем отправить пакеты с помощью UDP-клиента. Каждый запрос попадет только на один сервер, и это всегда будет сервер с IP-адресом, на который мы направили UDP-запрос.

Получается, что сетевой стек IP не считает UDP-порт отдельным объектом, который либо свободен, либо занят. Он рассматривает его как имя сокета, которое всегда должно идти в паре с IP-адресом (даже с подстановочными символами). У каждого запущенного сервера должен быть свой уникальный идентификатор сокета.

Еще одно важное замечание. Учитывая, что привязка сервера к интерфейсу 127.0.0.1 защищает нас от потенциально опасных пакетов во внешней сети, можно подумать, что привязка к одному внешнему интерфейсу защитит нас от вредоносных пакетов в других внешних сетях. На крупных серверах с несколькими сетевыми платами, например, возникает искушение выполнить привязку к частной подсети, куда входят остальные серверы, и надеяться, что это предотвратит прием поддельных пакетов по общедоступному IP-адресу.

К сожалению, не все так просто. Входящие пакеты, направленные к одному интерфейсу, могут быть приняты или отклонены другим интерфейсом в зависимости от операционной системы и ее настройки. Вполне вероятно, что если через общедоступное интернет-соединение будут поступать пакеты, якобы от других ваших серверов, ваша система их примет. Для того чтобы узнать, как это работает у вас, изучите документацию к вашей операционной системе или обратитесь к системному администратору. Если ваша операционная система не предоставляет меры безопасности, можно настроить на компьютере межсетевой экран.

Фрагментация UDP

До сих пор в этой главе мы предполагали, что UDP позволяет отправлять необработанные датаграммы, упакованные как IP-пакеты, где в качестве дополнительной информации указывается только порт отправителя и получателя. В предыдущем листинге мы видели, что UDP-пакет может быть до 64 Кбайт, но вы уже знаете, что плата Ethernet или беспроводной сети может обрабатывать пакеты не больше 1500 байт.

UDP отправляет маленькие датаграммы в виде отдельных IP-пакетов, а большие датаграммы делит на несколько пакетов, чтобы можно было передать их по сети (как мы обсуждали в *главе 1*).

В результате у больших пакетов выше риск потеряться, потому что, если хотя бы один фрагмент не достигнет адресата, собрать пакет уже не получится.

Процесс фрагментации огромных UDP-пакетов должен происходить незаметно для приложения, не считая возросшего риска сбоя. На что еще нужно обратить внимание?

- ◆ Если вы волнуетесь об эффективности, можно использовать только маленькие пакеты, чтобы повторная передача требовалась реже, а удаленный IP-стек быстрее собирал пакеты и доставлял ожидающему их приложению.

- ◆ Если межсетевой экран некорректно отклоняет ICMP-пакет и хост не может автоматически определить MTU (максимальную единицу передачи) до удаленного хоста (как это случилось в конце 1990-х годов), большие UDP-пакеты будут исчезать незаметно. MTU обозначает размер пакета, принимаемый всеми сетевыми устройствами между двумя хостами.

Если протокол полностью контролирует разделение данных по датаграммам и вам нужна возможность корректировать размер в зависимости от фактического значения MTU между двумя хостами, некоторые операционные системы (например, Linux) позволяют отключать фрагментацию и получать исключение в случае передачи слишком большого UDP-пакета. Тогда достаточно будет создать датаграмму меньше этого лимита.

Давайте посмотрим на листинг 2.3, где передается большая датаграмма.

Листинг 2.3. Отправка большого UDP-пакета

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Отправляем большую UDP-датаграмму, чтобы узнать MTU в сети.

import IN, argparse, socket

if not hasattr(IN, 'IP_MTU'):
    raise RuntimeError('cannot perform MTU discovery on this combination'
                       ' of operating system and Python distribution')

def send_big_datagram(host, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.IPPROTO_IP, IN.IP_MTU_DISCOVER, IN.IP_PMTUDISC_DO)
    sock.connect((host, port))

    try:
        sock.send(b'#' * 65000)
    except socket.error:
        print('Alas, the datagram did not make it')
        max_mtu = sock.getsockopt(socket.IPPROTO_IP, IN.IP_MTU)
        print('Actual MTU: {}'.format(max_mtu))

    else:
        print('The big datagram was sent!')

if name == 'main':
    parser = argparse.ArgumentParser(description='Send UDP packet to get MTU')
```

```

parser.add_argument('host', help='the host to which to target the packet')
parser.add_argument('-p', metavar='PORT', type=int, default=1060,
                    help='UDP port (default 1060)')
args = parser.parse_args()
send_big_datagram(args.host, args.p)

```

Когда я запускаю это приложение на сервере в моей домашней сети, я вижу, что моя беспроводная сеть поддерживает физические пакеты не более 1500 байт, как в стандартных сетях Ethernet.

\$ python big_sender.py guinness

Alas, the datagram did not make it

Actual MTU: 1500

Странно то, что даже интерфейс обратной связи на моем компьютере, который теоретически может потреблять пакеты величиной с ОЗУ, тоже накладывает ограничения на размер пакетов.

\$ python big_sender.py 127.0.0.1

Alas, the datagram did not make it

Actual MTU: 65535

Не во всех операционных системах можно посмотреть значение MTU. См. документацию.

Параметры сокетов

Интерфейс сокетов POSIX включает различные параметры, влияющие на поведение сетевых сокетов. Параметр IP MTU DISCOVER в листинге 2.3 — один из примеров. Доступные параметры можно посмотреть в документации к вашей операционной системе. В Python для работы с ними используются функции `getsockopt()` и `setsockopt()`. Для того чтобы узнать больше об этих двух системных вызовах, в Linux см. документацию по `socket(7)` и `udp(7)`, а также страницы <https://manuals.sourceforge.net/manuals> и <https://manuals.sourceforge.net/tcp>.

При настройке параметров сначала указывается группа, а затем имя параметра. Имя групп см. в документации к вашей операционной системе. Вызов `setattr()` в Python принимает на один аргумент больше, чем метод `getattr()`.

```

value = s.getsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, value)

```

У операционных систем есть уникальные параметры и реализации параметров. Вот несколько самых популярных параметров.

◆ **SO_BROADCAST** позволяет отправлять и получать широковещательные UDP-пакеты (подробнее — в следующем разделе).

- ♦ **SO_DONTROUTE** отправляет только пакеты, адресованные хостам в подсетях, к которым этот компьютер подключен напрямую. Если этот параметр задан, то, например, мой ноутбук отправляет пакеты в сети 127.0.0.0/8 и 192.168.5.0/24 и никуда больше, потому что маршрутизация через шлюз запрещена.
- ♦ **SO_TYPE**. При передаче методу `getsockopt()` этот параметр определяет, принадлежит ли сокет типу `SOCK_DGRAM`, который можно использовать для UDP, или `SOCK_STREAM`, который поддерживает семантику TCP (см. главу 3).

В следующей главе мы рассмотрим еще несколько параметров, которые относятся только к сокетам TCP.

Широковещание

Поддержка *широковещания* — одно из важных преимуществ UDP. Можно отправить датаграмму не на один хост, а по всей подсети, к которой подключен компьютер, и физическая сетевая плата предоставит ее всем подключенным хостам без многочисленных пересылок.

Следует отметить, что широковещание считается устаревшим, потому что его вытесняет более современная технология — *мультивещание*, при котором современные операционные системы эффективнее используют интеллектуальные возможности многих сетей и сетевых устройств. Кроме того, мультивещание работает с хостами за пределами локальной подсети. Однако если вам нужен просто способ поддерживать актуальность каких-то данных в локальной сети, например для игровых клиентов или автоматизированных табло, и у каждого клиента время от времени могут теряться пакеты, можно эффективно использовать широковещание UDP.

Сервер, который может получать широковещательные пакеты, и клиент, который способен их передавать, показаны в листинге 2.4. Здесь вы заметите только одно важное отличие от тактики в предыдущих листингах. Для того чтобы включить широковещание, мы вызываем метод `setsockopt()` объекта сокета, прежде чем использовать его. Помимо этого, сокет используется сервером и клиентом обычным образом.

Листинг 2.4. Широковещание UDP

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# UDP-клиент и сервер для широковещательных сообщений в LAN

import argparse, socket

BUFSIZE = 65535

def server(interface, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```

sock.bind((interface, port))
print('Listening for datagrams at {}'.format(sock.getsockname()))
while True:
    data, address = sock.recvfrom(BUFSIZE)
    text = data.decode('ascii')
    print('The client at {} says: {!r}'.format(address, text))

def client(network, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    text = 'Broadcast datagram!'
    sock.sendto(text.encode('ascii'), (network, port))

if __name__ == '__main__':
    choices = {'client': client, 'server': server}
    parser = argparse.ArgumentParser(description='Send, receive UDP broadcast')
    parser.add_argument('role', choices=choices, help='which role to take')
    parser.add_argument('host', help='interface the server listens at;'
                        ' network the client sends to')
    parser.add_argument('-p', metavar='port', type=int, default=1060,
                        help='UDP port (default 1060)')
    args = parser.parse_args()
    function = choices[args.role]
    function(args.host, args.p)

```

Первое, что мы заметим при использовании этого сервера и клиента, — они работают как обычный сервер и клиент, пока мы просто отправляем с клиента пакеты на определенный IP-адрес сервера.

Если настроить сокет UDP на широковещание, это никак не повлияет на возможность отправлять и получать явно адресованные пакеты.

Все самое интересное начинается, когда мы смотрим на параметры локальной сети и используем широковещательный IP-адрес в качестве места назначения. Для начала выполним инструкции, чтобы запустить в сети один или два сервера.

\$ python udp_broadcast.py server ""

Listening for broadcasts at ('0.0.0.0', 1060)

Затем отправляем с клиента сообщения каждому серверу. Мы увидим, что сообщение будет доставлено только одному серверу.

\$ python udp_broadcast.py client 192.168.5.10

Если указать широковещательный адрес локальной сети, мы увидим, что пакеты поступают на все широковещательные серверы одновременно! Обычные серверы

его не увидят. Для того чтобы удостовериться в этом, достаточно запустить несколько клонов стандартных серверов во время широковещания. Если выполнить `ifconfig`, можно узнать широковещательный адрес своей локальной сети:

```
$ python udp_broadcast.py client 192.168.5.255
```

Разумеется, оба сервера подтверждают, что получили сообщение. Если в вашей операционной системе сложно узнать широковещательный адрес и вы не возражаете против того, чтобы запустить широковещание со всех сетевых портов хоста, в Python можно указать специальное имя хоста `<broadcast>` при настройке UDP-соединения. При отправке имени клиенту не забудьте кавычки, потому что символы `&` и `>` уникальны для оболочки POSIX.

```
$ python udp_broadcast.py client "<broadcast>"
```

К сожалению, нет универсального способа узнать широковещательный адрес подсети на любой платформе. Вам придется самостоятельно изучить документацию к вашей операционной системе.

Сценарии применения UDP

У вас наверняка сложилось впечатление, что UDP подходит для передачи коротких сообщений. На самом деле этот протокол эффективен, только если сервер отправляет по одному сообщению за раз, а затем ждет ответ. Если приложение отправляет сразу много сообщений, лучше использовать "умную" очередь сообщений. Она будет устанавливать таймер на короткое время, за которое накопится несколько небольших сообщений, чтобы затем отправить их вместе, и скорее всего по протоколу TCP, который успешно фрагментирует полезную нагрузку.

Используйте UDP, если:

- ◆ с его помощью вы реализуете существующий протокол;
- ◆ вы создаете поток данных в реальном времени с избыточностью, которая позволяет терять пакеты, и вы не хотите, чтобы данные за эту секунду ждали поступления старых данных (как это происходит при использовании TCP);
- ◆ вашему приложению требуется мультивещание в локальной сети.

В последующих главах мы рассмотрим и другие варианты реализовать коммуникации в приложении. Есть старая шутка о том, что если вы наладили UDP в своем приложении, скорее всего, вы перепутали его с TCP.

Резюме

Протокол UDP позволяет пользовательским программам отправлять отдельные пакеты по IP-сети. Клиентское приложение передает пакет на сервер, который в ответном UDP-сообщении всегда указывает обратный адрес.

Сетевой стек POSIX получает доступ к UDP через так называемый сокет — конечную точку, которая может передавать и получать датаграммы с помощью IP-адреса и номера порта UDP (имени или адреса сокета). Встроенный модуль `socket` в Python позволяет выполнять базовые операции в сети.

Прежде чем сервер сможет получать входящие пакеты, его нужно привязать к адресу и порту с помощью метода `bind()`. UDP-клиент может просто начать отправку сообщений, и операционная система автоматически назначит ему номер порта.

UDP считается ненадежным протоколом, потому что не обрабатывает потерю пакетов из-за сбоя сетевого устройства или перегрузки сегмента сети. Клиентам придется отправлять запросы повторно, пока ответ не придет. Для того чтобы не создавать слишком много запросов, клиенты должны использовать метод экспоненциальной задержки, а также увеличивать время ожидания, если круговой путь до сервера занимает дольше обычного времени.

Идентификаторы запросов помогают избежать дублирования ответов, которое происходит, когда ответ, который считался потерянным, все же поступает, и клиент ошибочно принимает его за ответ на более поздний запрос. Случайно выбранные идентификаторы запросов также служат защитой от простейшего спуфинга.

При использовании сокетов важно отличать процесс привязки, при котором мы резервируем определенный UDP-порт, от процесса подключения, который разрешает получение запросов только от определенного сервера.

UDP-сокеты предоставляют такую удобную возможность, как широковещание, благодаря которому мы можем передавать пакеты всем хостам подсети, не рассылая их по отдельности. Это одна из немногих причин, по которым можно выбрать UDP для современного приложения, если оно относится к играм по локальной сети или другим вариантам кооперативных вычислений.

Протокол TCP

https://t.me/IT_Portal

TCP (Transmission Control Protocol — протокол управления передачей) делает большую часть работы по передаче данных в Интернете. Технически этот протокол должен называться TCP/IP, но в данной книге мы будем обозначать его просто TCP. Впервые он появился в 1974 г. Протокол использует технологию передачи IP-пакетов, чтобы программы могли взаимодействовать друг с другом с помощью непрерывных потоков данных. TCP гарантирует целостность потока данных — без потерь, дубликатов или нарушения порядка, если только соединение не прерывается и не зависит из-за сбоя сети.

TCP почти всегда используется протоколами, которые передают документы и файлы, включая веб-страницы в браузере и электронную почту. TCP также лежит в основе таких протоколов, как SSH для подключения к терминалу, и многих популярных протоколов чат-ботов.

Раньше предпринимались попытки ускорить сеть с помощью приложений на основе UDP (*см. главу 2*) с точно заданным размером и временем каждой отдельной даграммы. Современные реализации TCP работают сложнее и имеют множество преимуществ, созданных за 30 с лишним лет разработок и исследований. Вряд ли кто-то, кроме экспертов по протоколам, сможет повысить производительность текущего стека TCP. Даже приложения, которым особенно важна скорость, например очереди сообщений (*см. главу 8*), сегодня используют TCP.

Содержание главы

- ◆ Как работает TCP.
- ◆ Когда использовать TCP.
- ◆ Сокеты TCP.
- ◆ TCP-клиент и TCP-сервер.
- ◆ Одно взаимодействие — один сокет.
- ◆ Адрес.
- ◆ От привязки до интерфейсов.
- ◆ Взаимоблокировка.

- ◆ Полуоткрытые соединения, закрытые соединения.
- ◆ Потоки TCP для передачи файлов.
- ◆ Резюме.

Цель

В этой главе мы узнаем, как передавать и получать потоки данных между двумя сокетами по сети с помощью протокола TCP.

Как работает TCP

Как вы уже успели узнать, с сетями не так-то просто сладить. Время от времени они теряют пакеты, а иногда создают лишние копии данных или доставляют ответы не по порядку. Если мы используем только датаграммы, как с UDP, в коде приложения должно быть прописано, как проверять поступление каждой датаграммы и как восстанавливать потерявшиеся пакеты. TCP скрывает пакеты, так что приложение может просто отправлять адресату поток данных, а все потерявшиеся данные будут передаваться, пока не достигнут места назначения.

RFC 793 от 1981 г. — это первая спецификация TCP/IP, но с тех пор были представлены и другие RFC с дополнениями и улучшениями.

Принципы работы TCP таковы.

- ◆ TCP последовательно считает не пакеты (1, 2, 3, ...), а переданные байты. Затем адресат собирает их в правильном порядке, а если заметит, что чего-то не хватает, попросит передать эти пакеты снова. Например, за пакетом на 1024 байта с номером 7200 следует пакет с номером 8224. Это значит, что сетевому стеку не придется запоминать, как поток данных разделен на пакеты. Если будет запрошена повторная передача, поток можно разделить на новые пакеты каким-то другим методом (который, например, позволит вместить в пакет больше данных, если теперь нужно передать больше байтов), и получатель все равно сможет собрать пакет.
- ◆ В хороших реализациях TCP первое число в последовательности выбирается случайным образом, чтобы злоумышленники не могли его угадать.
- ◆ TCP не ждет ответа, а отправляет пакеты большими группами. Размер окна TCP — это объем данных, который отправитель посылает в сеть в определенный момент.
- ◆ Реализация TCP со стороны получателя управляет своим размером окна и может замедлять или приостанавливать соединение. Этот процесс называется управлением потоком.
- ◆ Наконец, если TCP обнаруживает потерянные пакеты, он предполагает, что сеть перегружена, и ограничивает объем передаваемых данных в секунду. В беспро-

водных сетях или других средах передачи, где пакеты могут теряться просто из-за шума, это может стать настоящей проблемой. Могут пострадать соединения, которые работали нормально до перезагрузки роутера. Например, конечные точки не могут взаимодействовать больше 20 секунд, а после восстановления сети снова связываются друг с другом. Они определяют, что сеть серьезно перегружена, и примут меры. После восстановления связи они поначалу не будут отправлять друг другу данные.

Конечно, у TCP есть много нюансов и тонкостей, но я надеюсь, что это описание помогло вам получить представление о том, как работает этот протокол. Помните, что приложение будет видеть поток данных, но на самом деле это пакеты с порядковыми номерами, эффективно скрытые сетевым стеком операционной системы.

Когда использовать TCP

Если ваши сетевые приложения похожи на мои, TCP будет использоваться для большинства сетевых соединений. Можно за всю карьеру не создать в коде ни одного UDP-пакета намеренно. (Однако, как вы увидите в *главе 5*, если приложение должно искать имя хоста DNS, где-то за кулисами обязательно будет работать UDP.)

Хотя TCP на практике используется по умолчанию, когда двум программам в Интернете нужно обменяться данными, мы рассмотрим несколько ситуаций, когда лучше выбрать другой протокол.

TCP — довольно громоздкий протокол, если клиенты отправляют серверу одиночные маленькие запросы, а затем прерывают соединение. Для установки TCP-соединения между двумя хостами используется последовательный обмен рукопожатиями SYN, SYN-ACK и ACK.

- ◆ SYN: "Я хочу поговорить. Вот порядковый номер пакета, с которого я начну".
- ◆ SYN-ACK: "Хорошо. Вот порядковый номер, который я буду использовать в моем направлении".
- ◆ ACK: "Договорились".

Когда соединение завершено, требуется обменяться еще тремя или четырьмя пакетами, чтобы закрыть его: FIN, FIN-ACK и ACK, или более длинный вариант, где требуются отдельные пакеты FIN и ACK в каждом направлении. Итого для доставки одного запроса требуется минимум шесть пакетов! В таких ситуациях целесообразнее будет использовать UDP.

Вопрос в том, захочет ли клиент открыть TCP-соединение, а затем с его помощью сделать несколько запросов к тому же серверу в следующие несколько минут или часов? После установки соединения и всех рукопожатий каждый запрос и ответ будут умещаться в одном пакете, используя все возможности TCP, включая повторную передачу, экспоненциальную задержку и управление потоком.

Если долгосрочные отношения между клиентом и сервером не нужны, лучше использовать UDP, особенно если у вас так много клиентов, что стандартная реализация TCP исчерпала бы всю память, если бы пришлось обслуживать отдельные потоки данных для каждого активного клиента.

Вторая ситуация, когда TCP неэффективен: если приложение может не просто передать данные при утере пакета, а сделать что-то более умное. Возьмем, к примеру, аудиочат. Если потерялся пакет, а вместе с ним и данные за секунду, не стоит снова и снова стараться прислать эту секунду. Вместо этого клиент может заполнить паузу, чем получится, на основе имеющихся пакетов. Умный аудиопrotocol начинает и заканчивает каждый пакет битом сильно сжатого аудио из предыдущего и следующего моментов времени как раз для таких ситуаций. А затем можно просто продолжить, как ни в чем не бывало. С TCP это невозможно, потому что он будет упорно передавать потерянные данные, даже если они давно никому не нужны. Для стриминга мультимедиа чаще всего используются датаграммы UDP.

Сокеты TCP

TCP различает программы с одним IP-адресом по номерам портов, как и UDP. Тут действует тот же принцип известных и временных номеров портов (см. *разд. "Множество сервисов в одной системе"* о номерах портов в *главе 2*).

Для протокола UDP требуется всего один сокет: сервер может установить UDP-порт, а затем получать датаграммы от сотен разных клиентов. Конечно, можно подключить сокет UDP к определенному узлу методом `connect()`, чтобы затем сокет использовал методы `send()` и `recv()` для отправки и получения пакетов этого узла, но это необязательно. Метод `connect()` действует так, как если бы программа сама выбрала отправку только на один адрес с помощью методов `sendto()`, а затем игнорировала бы ответы со всех остальных адресов.

Если протокол хранит состояние, как TCP, вызов `connect()` становится начальной точкой для последующего сетевого взаимодействия. Это точка, в которой сетевой стек операционной системы инициирует рукопожатия, как описано выше. Если все прошло успешно, оба конца потока TCP доступны для использования.

Получается, что в отличие от сокета UDP, TCP-вызов `connect()` может завершиться неудачей. Удаленный хост может не ответить или отказать в подключении. Могут возникнуть и более редкие проблемы, например немедленное получение пакета RST (reset — сброс). Поскольку для потоковой передачи требуется установить постоянное соединение между двумя хостами, второй хост должен ожидать передачи и быть готовым принять ваше соединение.

Со стороны сервера, который не выполняет вызов `connect()`, а получает пакет SYN от клиента, входящее соединение порождает еще более значимое событие для приложения Python: создание нового сокета. Это связано с тем, что стандартный ин-

терфейс POSIX TCP использует два типа сокетов: пассивные слушающие сокет и активные подключенные.

- ◆ Пассивный сокет отслеживает имя сокета (адрес и номер порта) сервера, по которому могут приниматься соединения. Пассивный сокет не может получать и отправлять данные. Он не участвует в сетевых взаимодействиях. Зато с его помощью сервер оповещает операционную систему о том, что он готов принимать входящие соединения по определенному номеру порта TCP.
- ◆ Удаленный партнер по соединению с определенным IP-адресом и номером порта подключается к активному сокету. Его можно использовать для общения только с одним партнером. Через него можно передавать данные, не беспокоясь о том, как они будут разделены на пакеты. Поток похож на файл до такой степени, что в системах UNIX подключенный сокет TCP можно передать приложению, которое собирается читать из файла, и приложение даже не заметит, что данные передаются по сети.

У пассивного сокета есть адрес интерфейса и номер порта, которые обозначают его уникальность, и никто не сможет использовать этот адрес и порт, при этом у нас может быть много активных сокетов с одним локальным именем сокета. На веб-сервере с тысячей HTTP-соединений будет тысяча активных сокетов с одним общедоступным IP-адресом и TCP-портом 80. Активный сокет отличается по четырем координатам:

(удаленный IP, удаленный порт, локальный IP, локальный порт)

Операционная система использует эти четыре значения, чтобы идентифицировать каждое активное TCP-соединение, и проверяет все входящие TCP-пакеты, чтобы убедиться, что координаты отправителя и получателя соответствуют активным соединениям.

TCP-клиент и TCP-сервер

Давайте посмотрим на пример в листинге 3.1. Здесь объединены две отдельные программы, но у них есть общий код, и таким образом мы видим код клиента и сервера вместе.

Листинг 3.1. Пример TCP-сервера и TCP-клиента

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Простой TCP-клиент и сервер, которые отправляют и получают 16 байт

import argparse, socket

def recvall(sock, length):
    data = b''
```

```
while len(data) < length:
    more = sock.recv(length - len(data))
    if not more:
        raise EOFError('was expecting %d bytes but only received'
                        ' %d bytes before the socket closed'
                        % (length, len(data)))
    data += more
return data

def server(interface, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind((interface, port)) sock.listen(1)
    print('Listening at', sock.getsockname())
    while True:
        sc, sockname = sock.accept()
        print('We have accepted a connection from', sockname)
        print(' Socket name:', sc.getsockname())
        print(' Socket peer:', sc.getpeername())
        message = recvall(sc, 16)
        print(' Incoming sixteen-octet message:', repr(message))
        sc.sendall(b'Farewell, client')
        sc.close()
        print(' Reply sent, socket closed')

def client(host, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((host, port))
    print('Client has been assigned socket name', sock.getsockname())
    sock.sendall(b'Hi there, server')
    reply = recvall(sock, 16)
    print('The server said', repr(reply))
    sock.close()

if __name__ == '__main__':
    choices = {'client': client, 'server': server}
    parser = argparse.ArgumentParser(description='Send and receive over TCP')
    parser.add_argument('role', choices=choices, help='which role to play')
    parser.add_argument('host', help='interface the server listens at;'
                        ' host the client sends to')
    parser.add_argument('-p', metavar='PORT', type=int, default=1060,
                        help='TCP port (default 1060)') args = parser.parse_args()
    function = choices[args.role]
    function(args.host, args.p)
```

В *главе 2* мы подробно рассмотрели метод `bind()`, потому что адрес, который мы предоставляем в качестве его параметра, указывает, могут ли удаленные хосты подключиться к нашему серверу и может ли сервер подключиться к ним или он защищен от внешних соединений и доступен только для программ на том же компьютере. В *главе 2* мы начали с безопасной программы, доступной только для интерфейса обратной связи, а затем перешли к более рискованным вариантам — приложению, которое принимает соединения с других хостов в сети.

В листинге приводятся оба варианта. Мы можем привязать `127.0.0.1` или один из внешних IP-адресов компьютера с помощью аргумента хоста из командной строки. Или мы можем отправить пустую строку, чтобы указать, что готовы принимать соединения с любого IP-адреса нашего компьютера. Подробнее об этом читайте в *главе 2*. Эти правила одинаково применяются к соединениям и сокетам TCP и UDP.

Номера портов для TCP и UDP тоже выбираются одинаково.

В чем разница между предыдущими примерами с UDP и этим клиентом и сервером, которые используют TCP?

Клиент очень похож. Он открывает сокет, вызывает `connect()`, указывая адрес нужного сервера, а затем отправляет и получает данные. Однако есть несколько отличий.

Во-первых, вызов TCP `connect()` ведет себя не так безобидно, как в UDP, где он просто устанавливает удаленный адрес по умолчанию, который будет использоваться для последующих операций `send()` и `recv()`. `connect()` — это действие в сети в реальном времени, которое запускает тройное рукопожатие между клиентом и сервером, чтобы они подготовились к общению друг с другом. Это значит, что вызов `connect()` может завершиться сбоем, например, если запустить клиента, пока сервер не запущен.

```
$ python tcp_deadlock.py client localhost
```

```
Sending 16 bytes of data, in chunks of 16
bytes Traceback (most recent call last):
```

```
...
```

```
ConnectionRefusedError: [Errno 111] Connection refused
```

Во-вторых, этот TCP-клиент гораздо проще UDP-клиента в одном отношении: ему нет дела до потерянных пакетов. TCP предоставляет гарантии, поэтому можно использовать метод отправки `send()` и не проверять, получил ли адресат данные, и так же можно использовать метод `recv()` для получения и не думать о повторной отправке запроса. Клиент может быть уверен, что сетевой стек повторно перешлет все нужные данные.

В-третьих, в одном направлении программа выглядит сложнее, чем при использовании UDP. Это странно, ведь несмотря на все гарантии, поток TCP кажется проще, чем датаграммы UDP. TCP считает все входящие и исходящие данные одним потоком без начала и конца, так что он делит их на пакеты, как хочет. В результате методы `send()` и `recv()` ведут себя иначе. При использовании UDP они просто отправляют и получают датаграмму соответственно, и каждая датаграмма существует отдельно от остальных данных и при этом не делится на части. Нельзя отправить или

получить UDP-датаграмму наполовину. Приложение воспринимает их только целиком.

TCP же может разделять потоки данных на пакеты разных размеров, а затем собирать их при получении. Конечно, это вряд ли случится с маленьким сообщением на 16 байт из листинга 3.1, но мы должны подготовить код на случай, если это произойдет. Как потоки TCP влияют на функции `send()` и `recv()`?

Когда мы вызываем TCP `transmit()`, в сетевом стеке операционной системы происходит один из трех сценариев.

- ◆ Сетевой стек локальной системы может просто принять данные, потому что сетевая плата свободна для передачи или в системе достаточно места, чтобы скопировать данные во временный исходящий буфер. Поскольку отправляется целая строка, `send()` сразу возвращает результат, и возвращаемым значением будет длина строки данных.
- ◆ Еще один сценарий: сетевая плата может быть занята, буфер исходящих данных может быть полон, а система не сможет или не захочет выделить больше места. В этом случае вызов `send()` просто зависнет, и программа будет ждать, пока данные не будут приняты.
- ◆ Есть вероятность, что исходящий буфер будет полон, но не до конца, и тогда часть данных будет принята сразу. Остальным данным все равно придется ждать. В этом случае вызов `send()` завершается сразу и возвращает начальные байты, которые удалось принять, а остальные данные оставляет необработанными.

Из-за этой последней возможности нельзя просто вызвать `transmit()` для сокета потоковой передачи, не проверив возвращаемое значение. Нужно включить вызов `send()` в цикл, который в случае частичной передачи продолжит попытки отправить оставшиеся данные, пока не удастся передать всю байтовую строку. В коде это может выглядеть так:

```
bytes_sent = 0
while bytes_sent < len(message):
    message_remaining = message[bytes_sent:]
    bytes_sent += s.send(message_remaining)
```

К счастью, Python не заставляет вас писать этот код каждый раз, когда нужно передать блок данных. Реализация сокета в стандартной библиотеке включает удобную функцию `sendall()`, которая используется в листинге 3.1. Поскольку функция `sendall()` написана на C, она не только работает быстрее, но и снимает глобальную блокировку интерпретатора во время цикла, позволяя остальным потокам Python спокойно выполняться, пока все данные не будут переданы.

К сожалению, в стандартной библиотеке нет аналога для `recv()`, хотя он тоже связан с рисками неполной передачи. `Recv()` реализуется операционной системой на основе логики, которая очень похожа на логику передачи.

- ◆ Если данных нет, `recv()` блокирует программу до поступления данных.

- ◆ Если во входящем буфере накопилось достаточно данных, мы получим столько байтов, сколько мы указали для `recv()`.
- ◆ Если буфер содержит какие-то данные, но их меньше, чем настроено для `recv()`, мы получим то, что уже есть, даже если это противоречит нашим указаниям.

Вот почему метод `recv()` нужно использовать в цикле. Операционная система не понимает, что этот простой клиент и сервер отправляют сообщения фиксированной длины в 16 байт. Она не может предсказать, когда данных будет достаточно для того, чтобы программа считала их полным сообщением, поэтому она передает вам столько информации, сколько у нее есть.

Почему в стандартной библиотеке Python есть метод `sendall()`, но нет ничего подобного для `recv()`? Скорее всего, потому, что сообщения фиксированной длины — редкость в наши дни. При разделении входящего потока на части большинство протоколов подчиняются гораздо более сложным правилам, чем определенная длина сообщения. Цикл, который выполняет `recv()` в большинстве реальных приложений, выглядит гораздо сложнее, чем код в листинге 3.1, т. к. компьютеру обычно приходится читать или обрабатывать часть сообщения прежде, чем он поймет, сколько данных еще поступит.

HTTP-ответ, например, содержит заголовки, пустую строку и количество байтов в заголовке `Content-Length` (длина содержимого). Мы не знаем, сколько раз нужно вызывать `recv()`, прежде чем мы получим хотя бы заголовки и определим длину содержимого. Этот код нужно прописать в приложении, а не взять из стандартной библиотеки.

Одно взаимодействие — один сокет

Если присмотреться к коду сервера в листинге 3.1, можно заметить, что он отличается от предыдущих примеров, и это отличие связано с определением потокового сокета TCP. Потоковые сокеты делятся на две категории: слушающие (с их помощью серверы предоставляют порт для входящих соединений) и подключенные (конкретное взаимодействие сервера с клиентом).

Мы видим, как реализуется это отличие, в коде сервера из листинга 3.1. Слушающий сокет предоставляет новый подключенный сокет как результат, возвращаемый методом `accept()`. Давайте подробно рассмотрим, как действия сокета выполняются в коде.

Для того чтобы получить определенный порт, сервер сначала вызывает `bind()`. Пока ничто не указывает на то, как будет вести себя программа — как клиент или как сервер, т. е. будет ли она активно устанавливать соединения или пассивно ждать их. Этот код просто резервирует для программы определенный порт на одном или на всех интерфейсах. Клиенты могут использовать этот метод, если хотят общаться с сервером с определенного порта в системе, а не получать временный номер порта.

Сервер объявляет, что хочет использовать сокет для прослушивания вызова этого метода, и на этом этапе принимается решение. Выполнение этого кода для TCP-

сокета меняет его назначение. При вызове `listen()` он необратимо меняется, и его больше нельзя использовать для отправки или получения данных. Объект сокета больше не будет привязан к определенному клиенту. Теперь метод сокета `accept()` (который используется исключительно для того, чтобы включать слушающие TCP-сокеты) можно использовать лишь для получения входящих соединений, и каждый такой вызов ждет, пока новый клиент подключится, прежде чем вернуть новый сокет, который будет использоваться для нового диалога.

Как мы видим в коде, `getsockname()` работает и для слушающих, и для подключенных сокетов. Он показывает, какой локальной TCP-порт используется сокетом. Мы можем в любой момент выполнить метод `getpeername()`, чтобы узнать адрес клиента, к которому привязан подключенный сокет. Или мы можем сохранить имя сокета, полученное как второе возвращаемое значение из метода `accept()`. Как вы увидите, оба значения равны одному и тому же адресу.

```
$ python tcp_sixteen.py server ""
```

```
Listening at ('0.0.0.0', 1060)
```

```
Waiting to accept a new connection
```

```
We have accepted a connection from ('127.0.0.1', 57971)
```

```
Socket name: ('127.0.0.1', 1060)
```

```
Socket peer: ('127.0.0.1', 57971)
```

```
Incoming sixteen-octet message: b'Hi there, server'
```

```
Reply sent, socket closed
```

```
Waiting to accept a new connection
```

Следующие выходные данные получены, когда клиент устанавливает только одно соединение с сервером:

```
$ python3 tcp_sixteen.py client 127.0.0.1
```

```
Client has been assigned socket name ('127.0.0.1', 57971)
```

```
The server said b'Farewell, client'
```

В остальном серверном коде мы видим, что когда `accept()` вернул подключенный сокет, он ведет себя в точности как клиентский сокет, без дальнейших перекосов в их взаимодействии. `Recv()` возвращает данные по мере поступления, а `sendall()` — это идеальный способ отправить большой блок данных и убедиться, что все они будут отправлены.

Когда мы вызвали `listen()` на сокете сервера, ему был передан целочисленный аргумент. Это значение указывает, сколько ожидающих соединений можно разрешить, прежде чем операционная система начнет отклонять новые соединения и откладывать дальнейшие рукопожатия. В этом примере я указал очень маленькое значение (1), чтобы мог подключаться только один клиент. Мы подробнее поговорим об этом параметре в разделе проектирования сетевого сервера в *главе 7*.

Когда клиент и сервер обменяются всеми нужными данными, они закрывают свой конец соединения методом `close()`, велят операционной системе отправить оставшиеся данные в их исходящий буфер, а затем закрывают TCP-сеанс с помощью пакета FIN.

Адрес

В листинге 3.1 остался последний момент, который нам нужно рассмотреть. Почему сервер должен убедиться, что задан параметр сокета `SO_REUSEADDR`, прежде чем попытаться выполнить привязку к порту?

Если закомментировать эту строку и попытаться запустить сервер, мы увидим, что происходит без этого параметра. На первый взгляд может показаться, что все в порядке. Мы ничего не заметим, если будем только запускать и останавливать сервер (здесь я запускаю и останавливаю сервер простой командой — нажатием комбинации клавиш `<Ctrl>+<C>` — в строке терминала):

```
$ python tcp_sixteen.py server ""
Listening at ('127.0.0.1', 1060)
Waiting to accept a new connection
^C
```

```
Traceback (most recent call last):
```

```
...
```

```
KeyboardInterrupt
```

```
$ python tcp_sixteen.py server ""
Listening at ('127.0.0.1', 1060)
Waiting to accept a new connection
```

Однако если мы запустим сервер, подключим к нему клиента, а потом остановим и перезапустим сервер, мы увидим совсем другую ситуацию. При перезапуске сервера отобразится следующая ошибка:

```
$ python tcp_sixteen.py server
Traceback (most recent call last):
```

```
...
```

```
OSError: [Errno 98] Address already in use
```

Почему так происходит? Почему `bind()`, который может повторяться бесконечно, не выполняется просто потому, что был подключен клиент? Если сервер продолжит работу без параметра `SO_REUSEADDR`, адрес будет доступен только через несколько минут после последнего соединения с клиентом.

Это связано с крайней осторожностью сетевого стека операционной системы. Сокет сервера, который только ожидает передачи данных, можно отключить и забыть. Несмотря на то что клиент и сервер закрыли соединения и обменялись пакетами `FIN`, подключенный `TCP`-сокеты, который сейчас общается с клиентом, не может быстро исчезнуть. Почему? Потому что сетевой стек никак не может узнать, получен ли последний пакет, который закрывает сокеты. Если он потерялся, получатель не будет понимать, где он задержался, и снова отправит пакет `FIN` в надежде получить ответ.

У надежного протокола, вроде `TCP`, должен наступать момент, когда пора прекратить взаимодействие. В противном случае системы будут бесконечно обмениваться сообщениями о том, что взаимодействие между ними закончено, пока мы наконец

не выключим компьютеры. Даже последний пакет может потеряться, и его придется передавать несколько раз, пока он не достигнет получателя. Что же делать?

Когда действующее TCP-соединение закрывается с точки зрения приложения, сетевой стек операционной системы сохраняет запись о нем в состоянии ожидания до четырех минут. В RFC эти состояния называются TIME-WAIT и CLOSE-WAIT. Финальные пакеты FIN могут получить корректные ответы, а закрытый сокет будет по-прежнему пребывать в одной из этих двух фаз. Если бы реализация TCP просто забыла о соединении, она бы не смогла ответить на FIN соответствующим ACK.

Итак, если сервер пытается получить порт, на котором за последние несколько минут было установлено соединение, оказывается, что этот порт занят. Если мы попытаемся выполнить метод `bind()` для этого адреса, мы получим ошибку.

С помощью параметра сокета `SO_REUSEADDR` мы говорим приложению, что можно занять порт, на котором пока еще закрываются бывшие соединения. Когда я пишу код сервера, я всегда указываю `SO_REUSEADDR`, и проблем никогда не возникает.

От привязки до интерфейсов

IP-адрес, который мы объединяем с номером порта в операции `bind()`, указывает операционной системе, от каких сетевых интерфейсов мы хотим получать соединения, как мы уже видели в *главе 2* о протоколе UDP. Вызовы в листинге 3.1 использовали локальный IP-адрес 127.0.0.1, который изолирует код от подключений с других компьютеров.

Для того чтобы это проверить, можно выполнить листинг 3.1 в режиме сервера и подключиться к клиенту с другого компьютера, как показано ранее.

\$ python tcp_sixteen.py client 192.168.5.130

Traceback (most recent call last):

...

ConnectionRefusedError: [Errno 111] Connection refused

Мы видим, что сервер даже не отвечает, запущен ли он. Даже если входящее соединение к его порту отклоняется, операционная система не оповещает его об этом. (Если у системы есть межсетевой экран, клиент может просто зависнуть при попытке подключения, без сообщения о том, что соединение отклонено.)

Если мы запустим сервер с пустым именем хоста (т. е. Python `bind()` будет считать, что мы готовы принимать соединения от любого активного сетевого интерфейса компьютера), сервер будет принимать соединения. Затем клиент может успешно подключиться к другому хосту (пустая строка обозначается двумя двойными кавычками в конце командной строки).

\$ python tcp_sixteen.py server ""

Listening at ('0.0.0.0', 1060)

Waiting to accept a new connection

```
We have accepted a connection from ('127.0.0.1', 60359)
Socket name: ('127.0.0.1', 1060)
Socket peer: ('127.0.0.1', 60359)
Incoming sixteen-octet message: b'Hi there, server'
Reply sent, socket closed
Waiting to accept a new connection
```

Я уже говорил, что моя операционная система использует конкретный IP-адрес 0.0.0.0, чтобы обозначить готовность принимать соединения по любому интерфейсу, но ваша операционная система может использовать другие обозначения. Python скрывает от нас эти тонкости, позволяя просто указать пустую строку.

Взаимоблокировка

В информатике этот термин описывает ситуацию, в которой две программы с ограниченными ресурсами вынуждены бесконечно ждать друг друга из-за ошибок планирования. При использовании TCP взаимоблокировка возникает довольно часто.

Как мы уже знаем, стандартные стеки TCP хранят в буферах входящие пакеты, пока приложение не будет готово их прочитать, а также исходящие данные, пока сетевое оборудование не сможет передать исходящий пакет. Буферы обычно небольшие, и система не хочет, чтобы программы заполняли всю оперативную память неотправленными сетевыми данными. В конце концов, зачем тратить ресурсы на производство данных, если получатель все равно не готов их обработать?

Это ограничение не проблема, если мы используем клиент и сервер, как в листинге 3.1, где обе стороны читают все сообщение партнера, прежде чем развернуться и отправить данные в противоположном направлении. Если создать клиента и сервер, которые оставляют в буфере слишком много данных, без плана быстро их считать, мы очень скоро столкнемся со сложностями.

Давайте посмотрим на пример в листинге 3.2, где клиент и сервер пытаются по-умному решить задачу, не думая о последствиях. Сервер должен преобразовать текст, переведя все буквы в верхний регистр. Клиентские запросы могут быть очень большими, и если сервер попытается дочитать весь поток информации до конца, прежде чем приступить к его обработке, у него закончится память, так что сервер считывает и обрабатывает данные небольшими фрагментами по 1024 байта.

Листинг 3.2. TCP-сервер и TCP-клиент с риском взаимоблокировки

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# TCP-клиент и сервер оставляют слишком много данных в состоянии ожидания

import argparse, socket, sys
```

```
def server(host, port, bytecount):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind((host, port))
    sock.listen(1)
    print('Listening at', sock.getsockname())
    while True:
        sc, sockname = sock.accept()
        print('Processing up to 1024 bytes at a time from', sockname)
        n = 0
        while True:
            data = sc.recv(1024)
            if not data:
                break
            output = data.decode('ascii').upper().encode('ascii')
            sc.sendall(output)      # отправляет его обратно в верхнем регистре
            n += len(data)
            print('\r %d bytes processed so far' % (n,), end=' ')
            sys.stdout.flush()
        print()
        sc.close()
        print(' Socket closed')

def client(host, port, bytecount):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    bytecount = (bytecount + 15) // 16 * 16    # округление до числа, кратного 16
    message = b'capitalize this!'             # 16-байтовое сообщение повторяется снова и снова

    print('Sending', bytecount, 'bytes of data, in chunks of 16 bytes')
    sock.connect((host, port))

    sent = 0
    while sent < bytecount:
        sock.sendall(message) sent += len(message)
        print('\r %d bytes sent' % (sent,), end=' ')
        sys.stdout.flush()

    print()
    sock.shutdown(socket.SHUT_WR)

    print('Receiving all the data the server sends back')
```

```

received = 0
while True:
    data = sock.recv(42)
    if not received:
        print(' The first data received says', repr(data))
    if not data:
        break
    received += len(data)
    print('\r %d bytes received' % (received,), end=' ')

print()
sock.close()

if __name__ == '__main__':
    choices = {'client': client, 'server': server}
    parser = argparse.ArgumentParser(description='Get deadlocked over TCP')
    parser.add_argument('role', choices=choices, help='which role to play')
    parser.add_argument('host', help='interface the server listens at;'
                        ' host the client sends to')
    parser.add_argument('bytecount', type=int, nargs='?', default=16,
                        help='number of bytes for client to send (default 16)')
    parser.add_argument('-p', metavar='PORT', type=int, default=1060,
                        help='TCP port (default 1060)')
    args = parser.parse_args()
    function = choices[args.role]
    function(args.host, args.p, args.bytecount)

```

Код просто применяет метод `upper()` к строке ASCII, и эту задачу можно легко разбить на любые части. Это процедура, которую можно выполнять отдельно на каждом компьютере, и каждый блок данных не зависит от предыдущих или следующих. Все было бы сложнее, если бы мы применяли другой метод, например `title()`, который преобразует первую букву каждого слова в заглавную. Граница блоков может проходить посередине слова. Для потока данных, который разделен на блоки по 16 байт, ошибка может выглядеть следующим образом:

```

>>> message = 'the tragedy of macbeth'
>>> blocks = message[:16], message[16:]
>>> ''.join( b.upper() for b in blocks ) # все работает
'THE TRAGEDY OF MACBETH'
>>> ''.join( b.title() for b in blocks ) # что-то пошло не так
'The Tragedy Of Macbeth'

```

Мы не смогли бы обработать текст в кодировке UTF-8, разделенный на блоки фиксированной длины, потому что один символ из нескольких байтов мог бы разделиться на разные блоки. В таких случаях серверу пришлось бы проявлять больше осторожности и сохранять состояние между блоками.

И все-таки обрабатывать данные по частям — это эффективный подход, даже если блоки по 1024 байта, как в нашем примере, слишком малы для современных серверов и сетей. Когда данные разделены и часто отправляются клиентам, серверу не приходится хранить большие объемы данных в памяти. При таком подходе сервер может обслуживать сотни клиентов за раз, передавая потоки размером в несколько гигабайт без лишней нагрузки на память и другие аппаратные ресурсы.

Клиент и сервер в листинге 3.2 неплохо обрабатывают и небольшие потоки данных. Если запустить сервер, а затем выполнить клиент, с помощью параметра командной строки передав небольшой объем данных, скажем, 32 байта, весь текст придет в верхнем регистре. Для простоты значение будет округлено до числа, кратного 16 байтам.

```
$ python tcp_deadlock.py client 127.0.0.1 32
```

```
Sending 32 bytes of data, in chunks of 16 bytes
```

```
32 bytes sent
```

```
Receiving all the data the server sends back
```

```
Первыми полученными данными будет b'CAPITALIZE THIS!CAPITALIZE THIS!'.
```

```
32 bytes received
```

Сервер подтвердит, что действительно обработал 32 байта от последнего клиента. Кстати, для простоты сервер должен работать в той же системе, что и клиент, и в этом скрипте используется IP-адрес localhost.

```
Processing up to 1024 bytes at a time from ('127.0.0.1', 60461)
```

```
32 bytes processed so far
```

```
Socket closed
```

Этот код прекрасно работает на очень маленьком объеме данных, но он справится и с объемами побольше. Запустите клиента с сотнями тысяч байтов и проверьте, все ли работает.

Кстати, первый обмен данными демонстрирует поведение `recv()`, о котором я уже говорил. `Recv(1024)` доставлял только 16 байт, если больше данных пока не поступило, даже если сервер запросил 1024 байта.

Однако здесь может возникнуть опасная ситуация. Если значение будет достаточно большим, произойдет настоящая катастрофа. Попробуйте отправить с клиента большой поток данных, скажем, гигабит.

```
$ python tcp_deadlock.py client 127.0.0.1 1073741824
```

Клиент и сервер будут часто обновлять терминал, сообщая об объеме данных, которые они отправили и получили. Это число будет все увеличиваться, пока внезапно обе стороны не зависнут. Если приглядеться, можно заметить, что первым останавливается сервер, а за ним клиент. На моем ноутбуке с Ubuntu они успевают об-

работать разный объем данных, но в тестовом прогоне, который я только что завершил, скрипт Python обработал много данных. Сервер остановится, выдав следующее сообщение:

```
$ python tcp_deadlock.py server ""
```

```
Listening at ('0.0.0.0', 1060)
```

```
Processing up to 1024 bytes at a time from ('127.0.0.1', 60482)
```

```
4452624 bytes processed so far
```

Клиент примерно на 350 000 байт опережает сервер:

```
$ python tcp_deadlock.py client "" 16000000
```

```
Sending 16000000 bytes of data, in chunks of 16 bytes
```

```
8020912 bytes sent
```

Почему клиент и сервер зависли?

Исходящий буфер сервера и входящий буфер клиента переполнились, и TCP использовал протокол корректировки размера окна, чтобы соединение не отправляло больше данные, которые все равно потом придется удалить и отправить заново.

Почему же обмен данными остановился совсем? Давайте подумаем, что происходит, когда каждый блок данных проходит по сети. Клиент отправляет его с помощью метода `sendall()`. Сервер принимает его с помощью `recv()`, обрабатывает и отправляет снова, в верхнем регистре, также через вызов метода `sendall()`. Что происходит потом? Ничего. Клиент не вызывает `recv()`, пока не передал все данные, в итоге данных накапливается все больше и больше, пока они не перестают помещаться в буферы операционной системы.

При последней передаче операционная система буферизировала 4 Мбайт данных во входящую очередь клиента, и сетевой стек решил, что он переполнен. В этот момент вызов сервера `sendall()` перестает выполняться, и операционная система приостанавливает работу сервера до тех пор, пока не освободится место для отправки следующих данных. Раз сервер больше не обрабатывает данные и не отправляет запросы `recv()`, клиент может начать резервное копирование данных. Клиент создал много данных, прежде чем остановиться, и, судя по всему, операционная система установила лимит в 3,5 Мбайт на объем данных, который она готова поставить в очередь в этом направлении.

У вас в системе может быть другой лимит. Я здесь привожу значения для моей операционной системы на тот момент. Они никак не связаны с принципами работы TCP.

В этом примере мы видим, что метод `recv(1024)` действительно возвращает менее 1024 байт, если к передаче готов меньший объем данных. Буферы могут временно хранить данные, чтобы не пришлось отбрасывать и повторно отправлять пакеты, которые поступили, когда у получателя не действовал вызов `recv()`. Однако буферы не бесконечны. Если процесс TCP пытается записать данные, но никто не может их получить и обработать, в конце концов он не сможет больше их записывать, пока какие-то данные не будут считаны и буфер не освободится.

Кроме того, из этого примера мы видим, в чем проблема протоколов, которые не требуют, чтобы клиент ждал ответа или подтверждения от сервера после каждого запроса. Если протокол не заставляет сервер читать запрос только после того, как клиент закончил его отправлять, а затем послать полный ответ в противоположном направлении, возникает ситуация, при которой клиент и сервер могут зависнуть, и нам останется только вручную завершить обе программы и переписать код, чтобы исправить проблему.

Но как же клиентам и серверам обрабатывать большие объемы данных и не попасть в эту ловушку? Есть два варианта. Во-первых, они могут использовать параметры сокета, чтобы отключить блокировку и разрешить функциям `send()` и `recv()` прерываться, как только они не смогут передавать данные. В *главе 7* мы подробно обсудим этот вариант, когда будем рассматривать различные подходы к созданию программ сетевых серверов.

Во-вторых, программы могут обрабатывать данные из нескольких источников одновременно, используя отдельные потоки или процессы (один может заниматься отправкой данных в сокет, например, а другой — чтением данных) или применяя вызовы операционной системы, например `select()` или `poll()`, чтобы отслеживать оба сокета и работать с тем, который готов к работе. В *главе 7* мы подробно рассмотрим и эти детали.

Кстати, при использовании UDP мы никогда не попали бы в такую ситуацию, потому что UDP не поддерживает управление потоком. Если поступает больше данных, чем можно обработать, UDP просто отбрасывает их, и приложению приходится догадываться, что произошло.

Полуоткрытые соединения, закрытые соединения

Нам нужно рассмотреть еще два аргумента из предыдущего примера.

В листинге 3.2 мы видим, как объект сокета в Python отвечает на условие "конец файла". Когда сокет закрывается, он просто выдает пустую строку, точно так же, как объект файла Python выдает пустую строку, когда в нем не осталось данных для чтения.

В листинге 3.1 мы об этом не думали, потому что задали строгие ограничения (сообщения по 16 байт), и нам не нужно было закрывать соединение, чтобы показать, что коммуникация закончена. Клиент и сервер могли передать сообщение и оставить сокет открытым, а закрыть его позже, и не волноваться, что какой-то другой ресурс ждет закрытия сокета.

В листинге 3.2 клиент отправляет (а сервер обрабатывает и возвращает) произвольный объем данных, зависящий только от числа, которое пользователь указывает командной строке. В результате мы дважды видим в коде один и тот же паттерн: цикл `while`, который продолжается, пока не увидит пустую строку, возвращенную

методом `recv()`. Пока мы не перейдем к *главе 7*, где мы рассмотрим неблокирующие сокеты, мы будем получать исключение от `recv()`, просто потому что на определенный момент данные недоступны. В этом сценарии используются другие методы, чтобы определить, был ли сокет закрыт.

Вы увидите, что после отправки данных клиент вызывает `shutdown()` для сокета. Это позволяет решить серьезную проблему. Как клиенту избежать необходимости выполнять метод `close()` для сокета и лишая себя возможности отправлять несколько вызовов `recv()`, которые нужны, чтобы получить от сервера ответ, если сервер будет читать данные, пока не увидит конец файла? Решение — закрыть соединение наполовину, т. е. перманентно отключить соединение в одном направлении, не удаляя сокет.

Для двустороннего сокета можно использовать вызов `shutdown()`, чтобы остановить соединение в одном направлении, как показано в листинге 3.2. Доступны три значения аргумента.

- ◆ **SHUT_WR.** Это самое распространенное значение, потому что программы обычно знают, когда закончатся выходные данные с их стороны, но не со стороны собеседника. Это значение указывает, что вызывающий объект больше не будет отправлять данные на этот сокет, и для получателя это означает конец файла.
- ◆ **SHUT_RD.** Эта команда используется для отключения входящего потока сокета, что приводит к ошибке конца файла, если собеседник пытается отправить через этот сокет еще данные.
- ◆ **SHUT_RDWR.** Это значение блокирует коммуникацию на сокете в обоих направлениях. Поначалу может показаться, что от этого значения нет никакой пользы, ведь можно просто закрыть сокет с помощью `close()` и добиться того же эффекта. Однако между этими двумя способами есть довольно ощутимая разница. Если операционная система разрешает нескольким программам использовать один сокет, метод `close()` закроет связь одного процесса с сокетом, но он останется открытым, пока его использует другой процесс. Метод `shutdown()` отключает сокет для всех использующих его процессов.

Мы не можем создать однонаправленный сокет с помощью обычного метода `socket()`, поэтому если нам нужно отправлять данные через сокет только в одном направлении, мы сначала создадим его, а затем выполним `shutdown()` для направления, которое нам не нужно. Это значит, что буферы операционной системы не будут лишней раз наполняться, если собеседник отправляет данные в неверном направлении.

Когда мы используем метод `shutdown()` с сокетами, которые должны быть однонаправленными, собеседник получит сообщение об ошибке, если случайно пытается передать данные в неправильном направлении.

В противном случае ненужные данные будут игнорироваться или заполнять буфер, что приведет к взаимоблокировке, ведь данные не будут считываться.

Потоки TCP для передачи файлов

TCP поддерживает потоки данных, в чем-то похожие на традиционные файлы, которые тоже могут читать и записывать последовательные данные. Python прекрасно разделяет эти два понятия: файлы могут читать и писать, а сокеты только отправляют и получают данные. Нет такого объекта, который допускал бы обе пары операций. Это гораздо более разумный подход, чем в POSIX, где можно на C вызвать методы `read()` и `write()` для сокетов, как если бы они были дескрипторами файлов.

Однако в некоторых ситуациях мы все же хотим работать с сокетом, как со стандартным объектом файла в Python, например, если мы хотим отправить его в код, который может напрямую считывать и записывать данные в файл (допустим, `pickle`, `json` или `zlib`). Python предоставляет для сокета метод `makefile()`, который возвращает объект файла Python, а за кулисами выполняются вызовы `recv()` и `send()`.

```
>>> import socket
>>> sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
>>> hasattr(sock, 'read')
False
>>> f = sock.makefile()
>>> hasattr(f, 'read')
True
```

Сокеты в UNIX-подобных системах, таких как Ubuntu и Mac OS X, как и стандартные файлы Python, предлагают метод `fileno()`, с помощью которого можно узнать номер дескриптора файла для вызовов на нижних уровнях. Это пригодится нам в *главе 7*, когда мы будем рассматривать методы `select()` и `poll()`.

Резюме

Сокет потоковой передачи, использующий протокол TCP, делает все возможное, чтобы обеспечить передачу и прием потоков данных по сети между двумя сокетами, включая повторную передачу потерянных пакетов, восстановление порядка пакетов и разделение больших потоков данных на пакеты, оптимизированные для передачи по сети.

TCP, как и UDP, различает конечные точки в одной системе по номерам портов. Программа, которая хочет принимать входящие TCP-соединения, должна выполнить метод `bind()` для привязки к порту, метод `listen()` для прослушивания сокета, а затем метод `accept()` в цикле, чтобы получать входящие соединения, через которые она сможет взаимодействовать с отдельными клиентами. Для того чтобы подключиться к существующему порту сервера, программе достаточно создать сокет и выполнить метод `connect()` с указанием адреса.

На серверах должен быть задан параметр `SO_REUSEADDR` на сокетах, для которых выполняется метод `bind()`, иначе операционная система не будет разрешать

привязку, потому что старые соединения долго закрываются после последнего взаимодействия.

Методы `send()` и `recv()` используются для отправки и получения данных соответственно. Некоторые протоколы, работающие поверх TCP, помечают данные таким образом, что клиенты и серверы автоматически понимают, когда взаимодействие заканчивается. Другие протоколы воспринимают TCP-сокет как поток данных, отправляющий и получающий данные до конца файла. С помощью метода сокета `shutdown()` можно сообщить о конце файла в одном направлении (по умолчанию все сокеты двунаправленные), а другое направление оставить открытым.

Когда две стороны обмениваются данными, сокет может наполняться данными, которые не читаются, что приведет к взаимоблокировке. В итоге одно направление больше не сможет отправлять данные методом `send()` и зависнет, ожидая, пока место освободится.

Если мы хотим передать сокет в процедуру Python, которая читает и записывает данные в файл, метод `makefile()` возвращает объект Python, выполняющий методы `recv()` и `send()`, когда вызывающий объект хочет считать или записать данные.

DNS и имена сокетов

В предыдущих главах мы в общих чертах рассмотрели два главных транспортных протокола — UDP и TCP, которые используются в IP-сетях. В этой главе мы обсудим два важных вопроса, которые нужно решить независимо от используемого протокола данных.

Содержание главы

- ◆ Имена сокетов и хостов.
- ◆ Пять координат сокетов.
- ◆ IPv6.
- ◆ Современное разрешение адресов.
- ◆ Привязка сервера к порту с помощью `getaddrinfo()`.
- ◆ Метод `getaddrinfo()` для привязки к сервису.
- ◆ Получение канонического имени хоста с помощью `getaddrinfo()`.
- ◆ Другие флаги `getaddrinfo()`.
- ◆ Примитивные процедуры службы имен.
- ◆ Метод `getsockaddr()`.
- ◆ Протокол DNS.
- ◆ Почему не стоит использовать DNS напрямую.
- ◆ Python для DNS-запросов.
- ◆ Разрешение почтовых доменов.
- ◆ Резюме.

Цель

В этой главе мы рассмотрим сетевые адреса и распределенный сервис, который преобразует имена в IP-адреса.

Имена сокетов и хостов

Мы редко напрямую указываем IP-адреса в браузерах или почтовых клиентах. Вместо них мы используем доменные имена. Некоторые доменные имена, например `python.org`, обозначают целые организации. Иногда можно сократить имя хоста, если он находится локально, но мы всегда можем использовать полное доменное имя, которое включает все элементы, в том числе домен верхнего уровня.

Домен верхнего уровня — это `.com`, `.net`, `.org`, `.gov`, `.mil` или стандартный двухбуквенный код страны, например `.ru`. Сейчас появляются и другие имена доменов, вроде `.beer`, так что полные доменные имена сложнее отличать от частичных на вид, если только вы не запомнили весь список доменов верхнего уровня.

У каждого домена верхнего уровня есть свои серверы, которыми управляет организация, отвечающая за назначение доменов в рамках домена верхнего уровня. При регистрации домена на этих серверах создается запись об этом.

Когда клиент в любой точке мира пытается разрешить имя в вашем домене, серверы верхнего уровня направляют его на серверы вашего домена, а ваша компания возвращает адреса для ваших имен хостов. Служба доменных имен (Domain Name Service, DNS) — это сеть серверов, расположенных по всему миру, которая отвечает на запросы имен, используя систему имен верхнего уровня и ссылок.

Как мы уже знаем, имя сокета не может состоять из примитивного значения Python, например целого числа или строки. Протоколы TCP и UDP используют целочисленные номера портов, чтобы можно было дать один IP-адрес компьютера нескольким приложениям, поэтому имя сокета состоит из адреса и номера порта:

(115.114.148.6)

Мы уже говорили об именах сокетов в предыдущих главах, а сейчас давайте рассмотрим их подробнее.

Имена сокетов очень важны. Ниже приводятся все методы сокетов, которым нужно передать имя сокета в качестве аргумента.

- ◆ `mysocket.accept()`. Каждый раз, когда мы вызываем эту функцию для слушающего потокового TCP-сокета, у которого есть входящие соединения для приложения. Функция возвращает кортеж, второй элемент которого — удаленный адрес (а первый — новый сокет, подключенный к этому удаленному адресу).
- ◆ `mysocket.bind(адрес)`. Эта функция привязывает сокет к указанному локальному адресу. Исходящие пакеты получают адрес, с которого они будут поступать, а входящие соединения с других компьютеров получают имя, к которому можно подключиться.
- ◆ `mysocket.connect(адрес)`. Эта функция указывает сокету, что данные, проходящие через него, будут переданы на указанный удаленный адрес. Функция просто устанавливает адрес по умолчанию для UDP-сокетов, если вызывающий объект использует `send()` вместо `sendto()` или `recv()` вместо `recvfrom()`, но не передает по сети данные сразу. На TCP-сокетах при этом приходится обмениваться тройным

рукопожатием, чтобы договориться о новом потоке с другим компьютером, и если договориться не получается, в Python вызывается исключение.

- ◆ `mysocket.getpeername` возвращает удаленный адрес сокета.
- ◆ `getsockname()` возвращает адрес собственной локальной конечной точки сокета.
- ◆ `mysocket.recvfrom(...)` создает кортеж, который содержит строку возвращенных данных и адрес, с которого они были получены, для UDP-сокетов.
- ◆ `mysocket.sendto(данные, адрес)` отправляет пакет данных на удаленный адрес с неподключенного UDP-порта.

Это функции, которым важно знать адрес сокета. В целом любой из этих методов может получать или возвращать любые адреса, указанные ниже, так что они будут работать с IPv4, IPv6 или менее популярными семействами адресов, которые мы не будем рассматривать в этой книге.

Пять координат сокетов

В главах 2 и 3 мы рассматривали имена хостов и IP-адреса, используемые сокетами, но это только две последние координаты из пяти важных решений, которые нужно принять при проектировании и развертывании каждого объекта сокета, как в следующем коде:

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(('localhost', 1060))
```

Как видите, здесь мы указываем четыре значения: два для конфигурации сокета и два для адреса вызова `bind()`. Метод `socket()` принимает третий, необязательный, аргумент, который может стать пятой координатой. Мы рассмотрим каждую координату, начиная с трех возможных параметров сокета.

Самое важное решение — семейство адресов, которое указывает, с сетью какого типа мы будем взаимодействовать.

Мне кажется, что будет полезнее приводить примеры для IP-сети, потому что они пригодятся большинству программистов на Python и будут работать в Linux, Mac OS X и даже Windows, так что в этой книге всегда используется семейство `AF_INET`. Однако если вы импортируете модуль `socket`, выполните инструкцию `print dir(socket)` и поищите строку, которая начинается с `AF_` (address family — семейство адресов), вы увидите и другие варианты, например `AppleTalk` и `Bluetooth`. Семейство адресов `AF_UNIX` обычно используется на платформах POSIX. Эти соединения похожи на интернет-сокеты, но применяются между приложениями на одном компьютере и подключаются по именам файлов, а не именам хостов и номерам портов.

После семейства адресов указывается тип сокета. Он определяет тип стратегии коммуникации, которую вы хотите использовать в выбранной сети. Можно поду-

мать, что у каждого семейства адресов должны быть свои типы сокетов. В конце концов, какое еще семейство, кроме AF_INET, поддерживает UDP и TCP?

К счастью, все несколько проще. Протоколы UDP и TCP входят только в семейство AF_INET, но разработчики интерфейсов сокетов договорились называть все сокететы на основе пакетов SOCK_DGRAM, а на основе потоков — SOCK_STREAM. Поскольку многие семейства адресов поддерживают один из этих подходов или оба, этих двух обозначений достаточно, чтобы охватить широкий ряд протоколов в разных семействах.

Указав семейство адресов и тип сокета, мы обычно сужаем выбор до одного варианта, поэтому поле протокола в методе socket() используется редко. Обычно мы ничего не указываем или указываем 0, чтобы значение было выбрано автоматически. Если мы выбираем потоковый режим по IP, автоматически будет выбран TCP. Если мы выбираем датаграммы, это будет UDP. Поэтому в методе socket() в этой книге не указывается третий аргумент — на практике он никогда не нужен. Примеры протоколов для семейства AF_INET можно найти в модуле socket среди имен, начинающихся с IPPROTO. Под названиями IPPROTO_TCP и IPPROTO_UDP вы увидите два протокола, которые мы рассматриваем в этой книге.

Наконец, четвертым и пятым значениями указываются IP-адрес и номер порта, о которых мы говорили в предыдущих двух главах.

Мы уже знаем, что имя сокета для этих протоколов состоит из двух компонентов: имени хоста и порта. У семейств адресов AppleTalk, ATM и Bluetooth есть свои структуры данных. Это необязательно будет кортеж из строки и целого числа. По сути, набор из пяти координат содержит три фиксированных значения, необходимые для создания сокета, и дополнительные координаты, которые требуются для конкретного семейства адресов.

IPv6

Все-таки помимо AF_INET мы должны упомянуть о еще одном семействе адресов — AF_INET6, включающем адреса IPv6, которых миру хватит надолго.

Когда сеть ARPANET начала развиваться, стало очевидно, что нам не хватит 32-битных адресов, хотя во времена, когда память компьютеров измерялась в килобайтах, такой размер казался вполне обоснованным. 4 млрд адресов не хватит всем жителям планеты, особенно когда у многих по несколько устройств.

Пока стандарт IPv6 не стал повсеместным, но программистам на Python все же следует уже сейчас разрабатывать программы, совместимые с IPv6.

В Python можно проверить наличие булева атрибута ipv6 в модуле socket, чтобы узнать, поддерживает ли платформа IPv6.

```
>>> import socket
>>> socket.has_ipv6
True
```


Это не означает, что интерфейс IPv6 работает и может отправлять пакеты. Просто мы видим, что операционная система поддерживает IPv6.

Поначалу может показаться, что придется внести в код Python слишком много изменений, чтобы он смог поддерживать IPv6.

- ◆ Если вы работаете в сети IPv6, сокеты должны использовать семейство AF_INET6.
- ◆ Имена сокетов больше не ограничены двумя компонентами — адресом и номером порта. Можно использовать дополнительные координаты с информацией о потоке и области действия.
- ◆ Иногда IPv6-адреса заменяют привычные IPv4, вроде 18.9.22.69, которые вы уже видели в файлах конфигурации или параметрах командной строки. Возможно, у вас пока нет для них подходящих регулярных выражений.

IPv6 дает не только гораздо больше адресов, но и другие преимущества, например более полную поддержку средств безопасности на канальном уровне.

Конечно, если вы привыкли писать старомодный неловкий код, который сканирует или собирает IP-адреса и имена хостов с помощью специального регулярного выражения, может показаться, что придется внести слишком много изменений. Другими словами, если вы когда-нибудь сами прописывали обработку адресов, скорее всего, вы ожидаете, что переход на IPv6 потребует еще более сложного кода. Не беспокойтесь. Я вообще рекомендую всеми силами избегать интерпретации адресов. Ниже я покажу, как это сделать.

Современное разрешение адресов

Getaddrinfo — один из самых эффективных инструментов для работы с сокетами в Python, потому что он упрощает код и помогает перейти с IPv4 на IPv6.

Функция `getaddrinfo()`, как и большинство других операций, требующих адресов, находится в модуле `socket`.

Скорее всего, это единственная функция, которая понадобится вам, чтобы преобразовывать имена хостов и номера портов от пользователей в адреса, которые можно будет использовать в методах сокета. Не считая каких-то специализированных применений.

Здесь используется простой подход. Нам не придется решать вопрос с адресами по частям, как при использовании более старых функций модуля `socket`. Мы можем получить всю необходимую информацию за один запрос. В ответ мы получаем все нужные координаты, чтобы построить и подключить сокет к указанному расположению.

Ниже приводится пример использования этой функции. Модуль `pprint` никак не связан с сетями, он просто улучшает внешний вид кортежей при выводе:

```
>>> from pprint import pprint
>>> infolist = socket.getaddrinfo('google.com', 'www')
```

```
>>> pprint(infolist)
[(2, 1, 6, '', ('142.250.67.174', 80))]
>>> info = infolist[0]
>>> info[0:3]
(2, 1, 6)
>>> s = socket.socket(*info[0:3])
>>> info[4]
('142.250.67.174', 80)
>>> s.connect(info[4])
```

Мы получили всю нужную информацию, чтобы создать сокет и использовать его для установки соединения: семейство, тип, протокол, каноническое имя и адрес. Какие параметры принимает `getaddrinfo()`? Возвращенный список из двух элементов показывает, что есть два способа подключиться к сервису HTTP на хосте **gatech.edu**: путем создания сокета `SOCK_STREAM` (сокет типа 1), который использует протокол `IPPROTO_TCP` (протокол номер 6), или с помощью сокета `SOCK_DGRAM` (сокет типа 2) с протоколом `IPPROTO_UDP`. Обычно позже в скрипте мы явно указываем, какой сокет нам нужен при вызове `getaddrinfo()`.

В листингах в *главах 2 и 3* нужно было явно указывать `AF_INET` и другие значения, чтобы обозначить низкоуровневый протокол, а при использовании `getaddrinfo()` в коде Python мы не обязаны использовать эти обозначения из модуля `socket`.

Достаточно будет указать в `getaddrinfo()`, какой именно адрес нам нужен. Мы используем первые три компонента из возвращаемого значения `getaddrinfo()` как параметры для функции `socket()` `Object()` { [код] } и пятый элемент в качестве адреса для любых функций, которым требуется адрес, как мы уже видели в первом разделе этой главы.

В приведенном выше фрагменте кода `getaddrinfo()` принимает не только имя хоста, но и имя порта вместо целого числа, поэтому более старым скриптам Python не придется делать дополнительные вызовы, если пользователь указывает `www` или `smtp` вместо 80 или 25.

Прежде чем мы узнаем, какие параметры есть у `getaddrinfo()`, давайте посмотрим, как он используется для выполнения трех распространенных задач в сети. Они идут в том порядке, в котором обычно выполняются: привязка, подключение и распознавание удаленного хоста, который отправил нам данные.

Привязка сервера к порту с помощью *getaddrinfo()*

Если мы хотим передать адрес функции `bind()`, например, когда создаем сокет сервера или хотим, чтобы клиент подключился куда-то еще, но с предсказуемого адреса, мы используем `getaddrinfo()`, указав `None` в качестве имени хоста, но предоставив номер порта и тип сокета.

Обратите внимание, что в вызове `getaddrinfo()` ноль обозначает подстановочный символ в полях, которые должны включать цифры:

```
>>> from socket import getaddrinfo
>>> getaddrinfo(None, 'smtp', 0, socket.SOCK_STREAM, 0, socket.AI_PASSIVE)
[(2, 1, 6, '', ('0.0.0.0', 25)), (10, 1, 6, '', (':::', 25, 0, 0))]
>>> getaddrinfo(None, 53, 0, socket.SOCK_DGRAM, 0, socket.AI_PASSIVE)
[(2, 2, 6, '', ('0.0.0.0', 53)), (10, 2, 17, '', (':::', 53, 0, 0))]
```

Я задал два вопроса. В одном я указал текстовый идентификатор порта, а в другом — числовое значение номера порта. В первом запросе я хотел узнать, к какому IP-адресу нужно привязать сокет методом `bind()`, если я хочу обслуживать SMTP-трафик по протоколу TCP. Во втором запросе речь шла об использовании UDP для обслуживания трафика через порт 53 (DNS). В ответ я получил адреса с подстановочными символами, с помощью которых можно будет выполнить привязку к каждому интерфейсу IPv4 и IPv6 на локальном компьютере с указанием подходящего семейства сокетов, типа сокетов и параметров протокола для каждого случая.

Можно опустить параметр `AI_PASSIVE` и просто указать имя хоста, если мы хотим выполнить привязку методом `bind()` к конкретному IP-адресу, который настроен как локальный адрес для нашего компьютера. Вот два примера привязки к `localhost`:

```
>>> getaddrinfo('127.0.0.1', 'smtp', 0, socket.SOCK_STREAM, 0)
[(2, 1, 6, '', ('127.0.0.1', 25))]
>>> getaddrinfo('localhost', 'smtp', 0, socket.SOCK_STREAM, 0)
[(10, 1, 6, '', (':::1', 25, 0, 0)), (2, 1, 6, '', ('127.0.0.1', 25))]
```

Как видите, если указать для `localhost` IPv4-адрес, мы сможем устанавливать соединения только по IPv4. А если предоставить символьное имя `localhost`, компьютер будет доступен по IPv4 и IPv6 (во всяком случае, так это работает на моем ноутбуке Linux с корректно заданным файлом `/etc/hosts`).

Что делать, если мы объявляем, что хотим предоставить простой сервис, а `getaddrinfo()` возвращает несколько адресов? Ведь мы не можем создать один сокет и методом `bind()` привязать его к нескольким адресам. В *главе 7* мы подробно рассмотрим подходы, с помощью которых можно создать серверный код и запустить несколько привязанных сокетов сервера одновременно.

Метод `getaddrinfo()` для привязки к сервису

Мы используем метод `getaddrinfo()`, чтобы узнать о подключении к другим сервисам, если только мы не выполняем привязку к локальному адресу, чтобы предоставить сервис самостоятельно. При поиске сервисов можно указать пустую строку, чтобы связаться с локальным хостом через интерфейс обратной связи, или строку с IPv4-адресом, IPv6-адресом или именем хоста в качестве места назначения.

Можно вызвать `getaddrinfo()` с флагом `AI_ADDRCONFIG`, если нужно использовать функции `connect()` или `sendto()` для сервиса. Этот флаг отфильтрует адреса,

недоступные для нашего компьютера. Например, диапазон IP-адресов в организации может включать адреса IPv4 и IPv6. Если хост поддерживает лишь IPv4, мы хотим получить в результатах только адреса из этого семейства. Если указать флаг `AI_V4MAPPED`, будут возвращены IPv4-адреса, преобразованные в формат IPv6. Их можно использовать на локальном компьютере, у которого есть только интерфейс IPv6, если сервисы, к которым мы подключаемся, поддерживают лишь IPv4.

Сначала мы используем `getaddrinfo()`:

```
>>> getaddrinfo('google.com', 'www', 0, socket.SOCK_STREAM, 0,
... socket.AI_ADDRCONFIG | socket.AI_V4MAPPED)
[(2, 1, 6, '', ('142.250.67.174', 80)),
 (2, 1, 6, '', ('142.250.67.174', 80))]
```

Мы получили именно то, о чем просили: список всех способов подключиться к **ftp.kernel.org** через TCP-соединение к FTP-порту. Для распределения нагрузки этот сервис находится на множестве разных адресов в Интернете, поэтому мы получили несколько IP-адресов. Когда возвращается несколько адресов, мы обычно используем первый и пробуем следующие, только если подключение не удалось. Операторы удаленного сервиса предоставляют эти адреса в желаемом порядке, который мы должны соблюдать.

Как подключить ноутбук к интерфейсу HTTP IANA, который отвечает за назначение номеров портов?

```
>>> getaddrinfo('google.com', 'www', 0, socket.SOCK_STREAM, 0,
... socket.AI_ADDRCONFIG | socket.AI_V4MAPPED)
[(2, 1, 6, '', ('142.250.67.174', 80))]
```

Сайт IANA — это отличный пример возможности применения флага `AI_ADDRCONFIG`, потому что, как и любое другое порядочное агентство по стандартизации Интернета, IANA уже поддерживает IPv6. Мой ноутбук может взаимодействовать только с IPv4-адресами по беспроводной сети, к которой он сейчас подключен, поэтому предыдущий вызов вернул только IPv4-адреса. Если удалить указанные в шестом параметре флаги, мы увидим IPv6-адрес, который все равно не сможем использовать.

```
>>> getaddrinfo('google.com', 'www', 0, socket.SOCK_STREAM, 0)
[(2, 1, 6, '', ('142.250.67.174', 80)),
 (10, 1, 6, '', ('2001:4860:4860::8844', 80, 0, 0))]
```

Это нормально, если мы не планируем использовать адреса сами, а просто поставляем информацию о каталогах другим хостам или приложениям.

Получение канонического имени хоста с помощью `getaddrinfo()`

Наконец, иногда мы хотим узнать имя хоста, которое формально связано с IP-адресом на том конце серверного сокета, если мы создаем экземпляр нового соеди-

нения или только что приняли входящее соединение на одном из наших серверных сокетов.

Это желание вполне разумно, но сопряжено с серьезным риском: владелец IP-адреса мог настроить DNS-сервер таким образом, чтобы он возвращал любое имя в качестве канонического, когда наш компьютер запрашивает его. Он может притвориться, что он **google.com**, **python.org** и т. д. Когда мы спрашиваем, какие имена хоста принадлежит одному из IP-адресов собеседника, он может вернуть совершенно любую строку символов.

Прежде чем доверять результатам поиска канонического имени (также называется обратным DNS-поиском, потому что сопоставляет IP-адрес с именем хоста, а не наоборот), мы должны проверить возвращенное имя и убедиться, что оно действительно разрешается в исходный IP-адрес. Если нет, нас специально вводят в заблуждение или это был добросовестный ответ от домена, где некорректно настроено сочетание имен и IP-адресов.

Поиск канонических имен дорого обходится системе. Требуется лишний круговой путь к глобальной службе DNS, так что имена часто не заносятся в журнал. Сервисы, которые приостанавливаются на обратный поиск имени для каждого подключающегося IP-адреса, работают очень медленно, поэтому многие системные администраторы в стремлении сократить время отклика записывают просто IP-адреса. Если один из них вызывает проблемы, его всегда можно будет найти в файле журнала вручную.

Однако если нам все же требуется каноническое имя хоста и мы хотим его найти, можно выполнить `getaddrinfo()` с флагом `AI_CANONNAME`, и четвертый элемент возвращенного кортежа (который был пустой строкой в предыдущих примерах) и будет каноническим именем:

```
>>> getaddrinfo('google.com', 'www', 0, socket.SOCK_STREAM, 0,
... socket.AI_ADDRCONFIG | socket.AI_V4MAPPED | socket.AI_CANONNAME)
[(2, 1, 6, 'google.com', ('142.250.67.174', 80))]
```

`Getaddrinfo()` также может принять имя сокета, уже подключенного к удаленному сервису, и вернуть каноническое имя.

```
>>> mysock = server_sock.accept()
>>> addr, port = mysock.getpeername()
>>> getaddrinfo(addr, port, mysock.family, mysock.type, mysock.proto,
... socket.AI_CANONNAME)
[(2, 1, 6, 'rr.pmtpa.wikimedia.org', ('103.102.166.226', 80))]
```

Напоминаю, что это сработает, только если владелец IP-адреса назначил ему имя. Поскольку у многих IP-адресов в Интернете нет подходящего обратного имени, у нас отсутствует возможность определить, кто связался с нами, если только мы не используем шифрование, чтобы проверить собеседника.

Другие флаги `getaddrinfo()`

В предыдущих примерах мы уже рассмотрели три флага `getaddrinfo()`. Доступные флаги зависят от операционной системы (список и конфигурации см. в документации). Однако есть несколько флагов, доступных на любой платформе.

- ◆ **AI_ALL.** Флаг `AI_V4MAPPED` защищает нас от ситуаций, когда наш хост использует только IPv6-адреса, а хост, к которому мы хотим подключиться, предоставляет только IPv4-адреса. `AI_V4MAPPED` преобразует IPv4-адреса в их IPv6-эквиваленты. Если IPv6-адреса доступны, отображаться будут только они, а IPv4-адреса не будут входить в возвращаемые значения. Флаг `AI_ALL` решает эту проблему: если мы хотим видеть все адреса с хоста, подключенного через IPv6, даже если нам доступны IPv6-адреса, можно использовать флаг `AI_ALL` вместе с `AI_V4MAPPED`, и мы получим список со всеми известными адресами.
- ◆ **AI_NUMERICHOST.** Этот флаг отключает попытки интерпретировать имя хоста (первый параметр `getaddrinfo()`) в виде текста, например `cern.ch`. Вместо этого строка имени хоста читается как литерал IPv4 или IPv6, например `74.207.234.78` или `fe80::fcfd:4aff:fecf:ea4e`. Так гораздо быстрее, потому что пользователь или файл конфигурации, которые предоставляют адрес, не могут заставить программу связаться со службой DNS для поиска имени, и это позволяет помешать потенциальному злоумышленнику заставить систему отправить запрос на сервер имен, контролируемый кем-то другим.
- ◆ **AI_NUMERICSERV.** Отключает возможность указать символьное имя порта (например, `www`) и разрешает только числовые номера портов. Поскольку база данных с номерами портов обычно существует локально на устройствах с поддержкой IP, и не нужно совершать запрос на удаленный сервер, нам не приходится защищать программу от долгих поисков DNS. В системах POSIX разрешение символьного имени порта обычно сводится к быстрому поиску в файле `/etc/services` (но можно для верности проверить параметр в файле `/etc/nsswitch.conf`). Если включить этот флаг, можно будет делать удобные проверки правильности, когда мы знаем, что строка порта всегда должна быть целым числом.

Наконец, нам не нужно волноваться о флагах, связанных с IDN, которые предоставляются в некоторых операционных системах и велят методу `getaddrinfo()` решать новые сложные доменные имена с символами Unicode. Python определяет, требуется ли для строки специальная кодировка, и применяет подходящие параметры для ее преобразования:

```
>>> getaddrinfo(' उदाहरण.परीक्षा ', 'www', 0, socket.SOCK_STREAM, 0,
... socket.AI_ADDRCONFIG | socket.AI_V4MAPPED)
[(2, 1, 6, '', ('199.7.85.13', 80))]
```

Если вы хотите понять, как это работает, начните с RFC 3492 и обратите внимание на то, что у Python есть кодек `idna` для преобразования в международные доменные имена и обратно.

```
>>> ' उदाहरण.परीक्षा '.encode('idna')
```

```
B'xn--11b5bs3a9aj6g '
```

Если ввести доменное имя на хинди, как в предыдущем примере, в службу доменных имен будет передана эта строка ASCII. Python скрывает от нас эти тонкости.

Примитивные процедуры службы имен

Прежде чем метод `getaddrinfo()` стал популярен, программисты, работающие на уровне сокетов, использовали несколько более простых методов, предоставляемых операционной системой. Большинство из них подходит только для IPv4, поэтому сегодня их лучше избегать.

Документацию по ним см. на странице модуля `socket` стандартной библиотеки. Давайте рассмотрим несколько простых примеров для каждого вызова. Имя хоста текущего компьютера возвращается с помощью двух вызовов.

```
>>> socket.gethostname()
```

```
'bpbonline'
```

```
>>> socket.getfqdn()
```

```
'bpbonline'
```

Еще два вызова позволяют преобразовать имена хостов IPv4 в IP-адреса.

```
>>> socket.gethostbyname('bpbonline.com')
```

```
'23.227.38.65'
```

```
>>> socket.gethostbyaddr('23.227.38.65')
```

```
('myshopify.com', [], ['23.227.38.65'])
```

Наконец, понадобятся три метода, чтобы найти номера протоколов и портов по символьным именам, которые понимает операционная система.

```
>>> socket.getprotobyname('UDP')
```

```
17
```

```
>>> socket.getservbyname('www')
```

```
80
```

```
>>> socket.getservbyport(80)
```

```
'www'
```

Мы можем поместить полное имя хоста системы, в которой выполняется программа Python, в вызов `gethostbyname()`, чтобы получить основной IP-адрес этого компьютера.

Метод *getsockaddr()*

Давайте рассмотрим небольшой пример применения метода `getaddrinfo()` на практике (листинг 4.1).

Листинг 4.1. Создаем и подключаем сокет с помощью `getaddrinfo()`

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Находим сервис WWW произвольного хоста с помощью getaddrinfo().

import argparse, socket, sys

def connect_to(hostname_or_ip):
    try:
        infolist = socket.getaddrinfo(
            hostname_or_ip, 'www', 0, socket.SOCK_STREAM, 0,
            socket.AI_ADDRCONFIG | socket.AI_V4MAPPED | socket.AI_CANONNAME,
        )
    except socket.gaierror as e:
        print('Name service failure:', e.args[1])
        sys.exit(1)

    info = infolist[0] # согласно стандартным рекомендациям берем первый вариант
    socket_args = info[0:3]
    address = info[4]
    s = socket.socket(*socket_args)
    try:
        s.connect(address)
    except socket.error as e:
        print('Network failure:', e.args[1])
    else:
        print('Success: host', info[3], 'is listening on port 80')

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Try connecting to port 80')
    parser.add_argument('hostname', help='hostname that you want to contact')
    connect_to(parser.parse_args().hostname)
```

Этот скрипт пытается быстро подключиться к порту 80 через потоковый сокет, чтобы узнать, работает ли веб-сервер, который мы указываем в командной строке.

Пример того, как использовать этот скрипт:

```
$ python www_ping.py bpbonline
```

```
Success: host bpbonline is listening on port 80
```

```
$ python www_ping.py smtp.google.com
```

```
Network failure: Connection timed out
```

```
$ python www_ping.py no-such-host.com
```

```
Name service failure: Name or service not known
```

Три замечание об этом скрипте.

- ◆ Он очень общий, без указания IP в качестве протокола или TCP в качестве транспорта. Если пользователь вводит имя хоста и система понимает, что это хост, к которому она подключена через AppleTalk, метод `getaddrinfo()` может вернуть семейство сокетов, тип и протокол AppleTalk, и именно такой сокет мы создадим и подключим.
- ◆ Сбой `getaddrinfo()` приведет к определенной проблеме службы имен, которая в Python называется `gaierror`, а не к стандартной ошибке сокета, как в конце предыдущего скрипта.
- ◆ Мы не отправили список из трех элементов в функцию сокета `socket() Object() { [код] }`. Вместо этого звездочка перед списком аргументов указывает, что три компонента списка аргументов сокета поставляются в `Object() { [код] }` как три независимых параметра. Это отличается от того, что мы делаем с фактическим возвращенным адресом, передаваемым как единое значение всем процедурам сокета, которым он нужен.

Протокол DNS

DNS — это система, в которой миллионы интернет-хостов помогают сопоставлять имена хостов с IP-адресами. Благодаря DNS мы можем ввести в адресной строке браузера **python.org**, чтобы не запоминать IPv4-адрес 82.94.164.162 или IPv6-адрес 2001:888:2000:d::a2.

ПРОТОКОЛ DNS

Цель: разрешает имена хостов, возвращая IP-адреса.

Стандарт: RFC 1034 и RFC 1035 (с 1987 г.).

Базовые протоколы: UDP/IP и TCP/IP.

Номер порта: 53.

Библиотеки: сторонние, включая `dnspython3`.

Сообщения, которые компьютеры отправляют для этого разрешения, проходят через серию серверов в иерархии. Если локальный компьютер и сервер имен не могут

разрешить имя хоста, потому что он нелокальный или вызывался очень давно, так что в кеше сервера имен его нет, следующий шаг — запросить один из серверов имен верхнего уровня, чтобы определить, какие компьютеры отвечают за этот домен. Затем можно запросить имя домена с помощью возвращенных IP-адресов DNS-сервера.

Давайте посмотрим, как обычно выполняется эта операция.

Для примера возьмем доменное имя **www.python.org**. Если браузеру нужен его адрес, он вызывает метод `getaddrinfo()`, который просит операционную систему разрешить имя. Система знает, есть у нее свой сервер имен или она использует службу имен, предоставляемую сетью. Когда компьютер подключается к сети, будь то локальная сеть в офисе или учебном заведении, беспроводная сеть или домашнее проводное соединение, информация о сервере имен обычно автоматически настраивается через DHCP. В других обстоятельствах системный администратор, настраивающий ваш компьютер, мог бы вручную настроить IP-адреса DNS-сервера. В любом случае DNS-серверы должны предоставляться просто по IP-адресам, иначе мы не сможем отправлять DNS-запросы.

Если нам не нравится поведение или производительность службы DNS нашего интернет-провайдера, мы можем настроить сторонний DNS-сервер, например сервер Google по адресу 8.8.8.8 и 8.8.4.4. В редких случаях вместо локальной DNS используется другая система, например служба WINS от Windows, однако для разрешения имен требуется DNS-сервер.

Некоторые имена хостов известны компьютеру и без службы DNS. Когда мы делаем запрос, например `getaddrinfo`, в первую очередь операционная система запрашивает имя хоста у DNS. На самом деле DNS-запросы отнимают много времени, поэтому должны выполняться только как крайняя мера. В зависимости от элемента `hosts` в файле `/etc/nsswitch.conf` (POSIX) или параметров в Панели управления (Windows), операционная система может обращаться к нескольким ресурсам, прежде чем вызывать службу DNS. Например, на моем ноутбуке с Ubuntu каждый запрос имени хоста начинается с проверки файла `/etc/hosts`. По возможности используется специальный протокол Multicast DNS. Если решить вопрос таким путем не получается, система обращается к полноценной службе DNS.

Представим ситуацию, когда имя **www.python.org** не определяется локально на компьютере и не находится в локальном кеше. Компьютер обращается к локальному DNS-серверу и отправляет один пакет с запросом UDP DNS. DNS-сервер отвечает на вопрос. В этой главе под "нашим" DNS-сервером мы будем иметь в виду конкретный DNS-сервер, который ищет для нас имена хостов. Разумеется, сам сервер принадлежит кому-то другому, скажем, компании, интернет-провайдеру или Google, а не нам.

Итак, сначала наш DNS-сервер проверяет кеш ранее запрошенных доменных имен, чтобы узнать, запрашивал ли адрес **www.python.org** какой-то другой компьютер, обслуживаемый этим же сервером, за последние несколько минут или часов. Он вернет действующую запись, если найдет ее (владелец доменного имени сам ука-

зывает срок действия, потому что некоторые организации предпочитают быстро менять IP-адреса при необходимости, а другие не против, если их старые IP-адреса много часов и дней хранятся в DNS-кешах по всему миру). Но если сейчас утро и вы первый в офисе, кто сегодня запрашивает **www.python.org**, DNS-сервер будет искать имя хоста с нуля.

Наш DNS-сервер запросит **www.python.org** у верхушки в иерархии DNS-серверов в мире, которая знает все домены верхнего уровня (.com, .org, .net и т. д.), а также группы серверов, ответственные за них. IP-адреса серверов верхнего уровня обычно интегрированы в программное обеспечение сервера имен, чтобы можно было определить сервер доменных имен до подключения к службе доменных имен. DNS-сервер узнает (если эта информация не сохранилась после недавних запросов), какие серверы хранят полный индекс домена .org, за первый круговой путь UDP.

Второй DNS-запрос будет направлен на серверы .org с целью узнать информацию о владельце домена **python.org**. Выполните `whois` в командной строке в системе POSIX, чтобы получить информацию об этом домене от серверов верхнего уровня, или используйте одну из многочисленных веб-страниц `whois`, если эта команда не установлена локально.

```
$ whois python.org
```

```
Domain Name:PYTHON.ORG
Created On:27-Mar-1995 05:00:00 UTC
Last Updated On:07-Sep-2006 20:50:54 UTC
Expiration Date:28-Mar-2016 05:00:00 UTC
...
Registrant Name:Python Software Foundation
...
Name Server:NS2.XS4ALL.NL
Name Server:NS.XS4ALL.NL
```

Итак, мы получили ответ. Любой DNS-запрос имени хоста в **python.org** будет отправляться на один из двух DNS-серверов из этого ответа, в какой бы точке земного шара вы ни находились. Конечно, когда DNS-сервер отправляет запрос на сервер доменных имен верхнего уровня, он получает не только два имени. Он также получает IP-адреса, чтобы можно было подключиться сразу, обходясь без дальнейших ресурсоемких DNS-запросов.

Наш DNS-сервер может напрямую подключиться к NS2.XS4ALL.NL или NS.XS4ALL.NL, чтобы спросить о домене **python.org**, и прервать взаимодействие с DNS-серверами корневого и верхнего уровней. Если первый сервер будет недоступен, мы вызовем второй. Это повышает наши шансы получить ответ, но увеличивает время загрузки страницы в браузере.

В зависимости от того, как настроены серверы имен **python.org**, DNS-серверу может потребоваться всего один дополнительный запрос, чтобы вернуть ответ, или еще несколько запросов, если речь о большой организации с разными отделами и

подотделами, каждый из которых использует свой DNS-сервер для пересылки запросов. В этой ситуации один из двух найденных серверов может сразу ответить на запрос о **www.python.org**, и наш DNS-сервер сможет отправить UDP-пакет браузеру, указав, какому IP-адресу соответствует это имя хоста.

Для этой процедуры требуются четыре круговых пути по сети: наш компьютер выполнил запрос и получил ответ от нашего DNS-сервера, и чтобы ответить, наш DNS-сервер тоже выполнил рекурсивный запрос на другие серверы, для чего потребовались еще три круговых пути. Неудивительно, что браузер медленно открывает сайт, когда мы впервые вводим адрес.

Почему не стоит использовать DNS напрямую

Надеюсь, из описания типичного DNS-запроса вы поняли, что операционная система проделывает большую работу, когда нам нужно найти имя хоста. Если у вас нет особых причин взаимодействовать с DNS, я рекомендую всегда использовать `getaddrinfo()` или аналогичный подход для разрешения имен хостов. Вот какие преимущества мы получаем, когда операционная система ищет имена за нас.

- ◆ DNS не всегда является единственным источником информации об именах в системе. Иногда имена хоста, которые работают где-то в системе (в браузере, путях общих папок и т. д.), внезапно перестают работать, когда используют наше приложение, потому что мы не обращаемся к таким механизмам, как WINS или `/etc/hosts`, используемым операционной системой.
- ◆ Кеш локального компьютера с ранее запрашиваемыми доменными именами, скорее всего, содержит хост, IP-адрес которого мы запрашиваем. Если мы попробуем ответить на этот вопрос через DNS, мы проделаем двойную работу.
- ◆ Благодаря параметрам, установленным системным администратором, или механизму настройки сети, вроде DHCP, система, в которой выполняется скрипт Python, уже знает о локальных серверах доменных имен. Для того чтобы использовать DNS в программе Python, нужно узнать, как запросить эту информацию у операционной системы, но мы не будем обсуждать здесь данную тему, потому что это зависит от операционной системы.
- ◆ Без локального DNS-сервера мы не сможем использовать и его кеш, а значит, наше приложение и другие приложения в той же сети не смогут делать повторные запросы часто используемых имен хоста.
- ◆ Время от времени инфраструктура DNS в мире обновляется, а вместе с ней обновляются системные библиотеки и демоны. Если ваше приложение вызывает DNS, вы должны следить за этими изменениями и отражать в коде новые IP-адреса серверов доменов верхнего уровня, новые стандарты и корректировки протокола DNS.

Стандартная библиотека Python не содержит функции для работы с DNS. Выберите стороннюю библиотеку.

Python для DNS-запросов

Делать DNS-запросы с помощью Python вполне разумно. Например, в случаях если почтовый сервер или клиент, который пытается отправить почту получателям напрямую, а не через локальный почтовый ретранслятор, хочет найти записи MX, связанные с доменом, чтобы определить нужный почтовый сервер для адресов **@example.com**.

Давайте рассмотрим одну из сторонних библиотек DNS для Python, `dnspython3`, которую можно установить с помощью обычного инструмента пакетов Python. Пожалуй, это лучшая библиотека для подобных операций в Python 3.

\$ `pip install dnspython3`

Библиотека предоставляет собственные методы, чтобы мы могли определить, какие серверы доменных имен использует Windows или операционная система POSIX, а затем попросить у этих серверов выполнить нужные запросы. Поэтому в примерах кода в этой главе не требуется хост, для которого администратор или сервис конфигурации сети настроили серверы имен.

Базовый и полный поиск приводятся в листинге 4.2.

Листинг 4.2. Простой DNS-запрос с рекурсией

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Базовый DNS-запрос.

import argparse, dns.resolver

def lookup(name):
    for qtype in 'A', 'AAAA', 'CNAME', 'MX', 'NS':
        answer = dns.resolver.query(name, qtype, raise_on_no_answer=False)
        if answer.rrset is not None:
            print(answer.rrset)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Resolve a name using DNS')
    parser.add_argument('name', help='name that you want to look up in DNS')
    lookup(parser.parse_args().name)
```

Как видите, можно отправлять только по одному типу DNS-запросов за раз, поэтому данный скрипт выполняется циклом, запрашивая разные типы записей, которые относятся к одному имени хоста, указанному как аргумент командной строки. Давайте выполним запрос для **python.org** и посмотрим, что получится.

\$ `python dns_basic.py python.org`

```
python.org. 42945 IN A 140.211.10.69
```

```
python.org. 86140 IN MX 50 mail.python.org.  
python.org. 86146 IN NS ns4.p11.dynect.net.  
python.org. 86146 IN NS ns3.p11.dynect.net.  
python.org. 86146 IN NS ns1.p11.dynect.net.  
python.org. 86146 IN NS ns2.p11.dynect.net.
```

Ответы возвращаются как последовательность объектов. В каждой строке выводятся следующие ключи:

- ◆ имя, которое мы искали;
- ◆ период в секундах, в течение которого имя хранится в кеше;
- ◆ класс IN, который указывает, что мы получаем ответы с интернет-адресами;
- ◆ тип записи: A — IPv4-адреса, AAAA — IPv6-адреса, NS — сервер имен, MX — ответ, который указывает почтовый сервер для домена;
- ◆ наконец, в разделе данных содержится информация, которая нужна нам для подключения или обращения к сервису.

Мы узнаем о домене **python.org** три факта. Во-первых, A указывает, что мы хотим подключиться к реальной системе **python.org**, т. е. инициировать HTTP-соединение, начать SSH-сеанс или сделать что-то подобное, потому что пользователь указал **python.org** в качестве компьютера для подключения, и мы должны отправлять пакеты на IP-адрес 140.211.10.69. Во-вторых, NS указывает, что если мы хотим запросить имена хостов под **python.org**, нужно попросить серверы имен с ns1.p11.dynect.net по ns4.p11.dynect.net разрешить эти имена (в идеале мы должны обращаться к ним в указанном порядке, а не по порядковым номерам). В-третьих, нужно найти имя хоста **mail.python.org**, если мы хотим отправлять электронную почту пользователям в почтовом домене **@python.org**.

DNS-запрос может вернуть запись типа CNAME. Это будет означать, что мы ищем имя хоста, которое является псевдонимом для другого имени хоста, которое нужно будет искать отдельно. Этот тип записи больше не популярен, потому что для него требуется совершить два круговых пути, хотя иногда он все же встречается.

Разрешение почтовых доменов

В большинстве проектов Python, где требуется разрешение почтового домена, можно использовать DNS напрямую.

Требования к разрешению описаны в RFC 5321, где говорится, что если существуют записи MX, мы должны попытаться связаться с этими серверами SMTP, и если ни один из них не принимает сообщение, нужно вернуть пользователю ошибку или поместить сообщение в очередь для повторных попыток. Если приоритеты сообщений не равны, нужно повторять попытки в порядке приоритета. Если записей MX нет, а у домена есть запись A или AAAA, можно попробовать установить

SMTP-соединение с этим адресом. Если таких записей нет, но запрашивается CNAME, для указанного доменного имени следует искать записи MX или A в соответствии с теми же требованиями.

В листинге 4.3 показано, как использовать эти методы на практике. Там приводится серия DNS-запросов и показаны их ответы. Вы можете использовать почтовый менеджер Python, чтобы распределять электронную почту по удаленным хостам, изменив функцию таким образом, чтобы она возвращала адреса, а не просто выводила их.

Листинг 4.3. Разрешение имен почтовых доменов

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Поиск почтового домена – части адреса после @

import argparse, dns.resolver

def resolve_hostname(hostname, indent=''):
    "Print an A or AAAA record for `hostname`; follow CNAMEs if necessary."
    indent = indent + ' '
    answer = dns.resolver.query(hostname, 'A')
    if answer.rrset is not None:
        for record in answer:
            print(indent, hostname, 'has A address', record.address)
        return
    answer = dns.resolver.query(hostname, 'AAAA')
    if answer.rrset is not None:
        for record in answer:
            print(indent, hostname, 'has AAAA address', record.address)
        return
    answer = dns.resolver.query(hostname, 'CNAME')
    if answer.rrset is not None:
        record = answer[0]
        cname = record.address
        print(indent, hostname, 'is a CNAME alias for', cname) #?
        resolve_hostname(cname, indent)
        return
    print(indent, 'ERROR: no A, AAAA, or CNAME records for', hostname)

def resolve_email_domain(domain):
    "For an email address `name@domain` find its mail server IP addresses."
```

```

try:
    answer = dns.resolver.query(domain, 'MX', raise_on_no_answer=False)
except dns.resolver.NXDOMAIN:
    print('Error: No such domain', domain)
    return
if answer.rrset is not None:
    records = sorted(answer, key=lambda record: record.preference)
    for record in records:
        name = record.exchange.to_text(omit_final_dot=True)
        print('Priority', record.preference)
        resolve_hostname(name)
else:
    print('This domain has no explicit MX records')
    print('Attempting to resolve it as an A, AAAA, or CNAME')
    resolve_hostname(domain)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Find mailserver IP address')
    parser.add_argument('domain', help='domain that you want to send mail to')
    resolve_email_domain(parser.parse_args().domain)

```

Конечно, реализация функции `resolve_hostname()` здесь неоднозначная, потому что приходится динамически выбирать между записями A и AAAA в зависимости от того, к какой сети подключен текущий хост: IPv4 или IPv6. На самом деле вместо того, чтобы пытаться разрешить адрес самим, можно передать методу `getsockaddr()` имя хоста почтового сервера, но в листинге 4.3 мы хотим посмотреть, как работает DNS. Сделайте еще несколько запросов, чтобы увидеть, как разрешаются имена.

Конечно, настоящий почтовый сервер не стал бы публиковать адреса серверов, а просто попытался бы доставить им почту и прекратил операцию. Если бы он продолжил проходить по списку серверов после успешной отправки, он создал бы несколько копий одного письма, по одному для каждого сервера. Тем не менее данный пример кода дает нам представление об этой процедуре. Как видите, сейчас у **python.org** есть только один IP-адрес почтового сервера.

```
$ python dns_mx.py python.org
```

```
This domain has 1 MX records
```

```
Priority 50
```

```
mail.python.org has A address 82.94.164.166
```

Конечно, снаружи мы не видим, принадлежит IP одному компьютеру или группе хостов. Некоторые компании могут выделять несколько точек получения для вхо-

дящих писем. У IANA шесть почтовых серверов (или, во всяком случае, шесть IP-адресов, к которым можно подключиться, а серверов может быть сколько угодно).

```
$ python dns_mx.py iana.org
```

```
This domain has 6 MX records
```

```
Priority 10
```

```
    pechora7.icann.org has A address 192.0.46.73
```

```
Priority 10
```

```
    pechora5.icann.org has A address 192.0.46.71
```

```
Priority 10
```

```
    pechora8.icann.org has A address 192.0.46.74
```

```
Priority 10
```

```
    pechora1.icann.org has A address 192.0.33.71
```

```
Priority 10
```

```
    pechora4.icann.org has A address 192.0.33.74
```

```
Priority 10
```

```
    pechora3.icann.org has A address 192.0.33.73
```

Выполняйте этот скрипт на разных доменах, чтобы посмотреть, как большие и маленькие компании обрабатывают пересылку входящей почты по IP-адресам.

Резюме

Имена хостов часто необходимо преобразовывать в адреса сокетов, чтобы программы Python могли подключаться к ним.

Функция `getsockaddr()` в модуле `socket` подходит для большинства операций по поиску имени хоста, потому что она использует возможности вашей операционной системы и не только умеет искать доменные имена с помощью всех доступных ей механизмов, но и понимает, какой тип адреса (IPv4 или IPv6) поддерживается локальным IP-стеком.

IPv4-адреса по-прежнему наиболее популярны, хотя IPv6 используются все чаще и чаще. Программа Python может считать адреса непрозрачными строками и не заниматься их парсингом или интерпретацией, а просто поручать функции `getsockaddr()` поиск имен всех портов и хостов.

DNS — это глобально распределенная база данных, которая направляет запросы о доменных именах на серверы владельца домена. Это основная технология разрешения имен. Обычно код Python не работает с ней напрямую, но мы можем посмотреть, как сообщение электронной почты пересылается по почтовому домену, указанному в адресе после символа `@`.

Теперь мы знаем, как именовать хосты, к которым мы подключаем сокет. В главе 5 мы рассмотрим различные варианты шифрования и разграничения передаваемых данных.

Данные и ошибки в Интернете

В первых четырех главах этой книги мы увидели, как вызывать hosts в IP-сети и как устанавливать и прерывать TCP- и UDP-соединения между hosts. В этой главе мы поговорим о том, как подготовить данные к передаче.

Содержание главы

- ◆ Строки и байты.
- ◆ Строки символов.
- ◆ Сетевой порядок байтов и двоичные числа.
- ◆ Кадрование.
- ◆ Pickle и форматы с разделителями.
- ◆ JSON и XML.
- ◆ Сжатие.
- ◆ Исключения в сети.
- ◆ Специфические исключения.
- ◆ Исключения в сети: обнаружение и сообщение об ошибках.
- ◆ Резюме.

Цель

Какой формат и кодировку мы должны использовать? К каким ошибкам должны быть готовы приложения Python? Эти вопросы относятся к передаче как потоков, так и датаграмм, и в этой главе мы дадим на них ответы.

Строки и байты

Микросхемы памяти и сетевые платы используют байт в качестве стандартной единицы измерения. Мы передаем и храним данные этими маленькими фрагмента-

ми по 8 бит. Однако между микросхемами памяти и сетевыми платами есть важное различие. Python может полностью скрывать все решения о том, как представлять числа, строки, списки и словари в памяти. Мы не видим, что эти структуры хранятся в байтах, если только не используем специализированные инструменты отладки.

Интерфейс сокета работает с байтами, так что и приложение, и сам программист видят их. При работе с сетями мы должны думать, как данные будут представлены в сети. В иных случаях такой высокоуровневый язык, как Python, старается избежать подобных деталей.

Мельчайшая единица информации — *бит*, который может равняться нулю или единице. В электронике это можно представить как провод, который либо замкнут в цепь, либо заземлен. *Байт* состоит из 8 битов.

Биты должны находиться в логическом порядке. При записи двоичного числа, например 01100001, мы пишем цифры по порядку, как и в десятичной системе. Самым важным битом считается первый, так же как в десятичном числе 234 мы считаем цифру 2 самой важной, а цифру 4 — наоборот, потому что сотни больше, чем десятки и единицы.

Один байт представляет собой число от 00000000 до 11111111. В десятичной системе счисления это 0 и 255.

Мы можем считать числа от 0 до 255 в обратную сторону и интерпретировать их как отрицательные. Числа от 10000000 до 11111111, которые должны обозначать числа от 128 до 255, иногда воспринимаются как ряд от -128 до -1, и первые цифры показывают, является число положительным или отрицательным (так называемая *арифметика с дополнением до двух*). Мы можем использовать более сложные правила для интерпретации байта, например таблицу для назначения байту определенного символа или значения. Мы также можем объединять байты, чтобы получать большие числа.

Поскольку раньше байт мог иметь разную длину в разных системах, в сети 8-битный байт иногда называется *октетом*.

В Python байты обычно представлены одним из двух способов: как целое число со значением от 0 до 255 или как однобайтовая строка, единственным значением которой является байт. В коде Python мы можем выразить в байтах число из любой системы — двоичной, восьмеричной, десятичной и шестнадцатеричной.

```
>>> 0b1100010
```

```
98
```

```
>>> 0b1100010 == 0o142 == 98 == 0x62
```

```
True
```

Мы можем преобразовать список таких чисел в байтовую строку, используя тип `bytes()` в последовательности, или наоборот восстановить список путем итерации по байтовой строке.

```
>>> b = bytes([0, 1, 98, 99, 100])
```

```
>>> len(b)
```

```
5
```

```
>>> type(b)
<class 'bytes'>
>>> list(b)
[0, 1, 98, 99, 100]
```

Метод `repr()` объекта байтовой строки использует символы ASCII для обозначения элементов массива, значения которых соответствуют кодам печатаемых символов, а явный шестнадцатеричный формат `xNN` применяется только для байтов, которые не соответствуют печатаемым символам ASCII. Это может сбивать с толку.

```
>>> b
b'\x00\x01bcd'
```

Не стоит думать, что байтовые строки ограничиваются кодировкой ASCII и могут представлять только последовательности 8-битных байтов.

Строки символов

Нам нужна кодировка, которая назначает каждый символ допустимому байтовому значению, чтобы можно было отправлять строки символов через сокет. Обычно используется кодировка ASCII (American Standard Code for Information Interchange — американский стандартный код для обмена информацией). Она определяет коды символов от 0 до 127, которые помещаются в 7 бит. В результате самый значимый бит в ASCII всегда равен 0. Поскольку коды от 0 до 31 используются для управления, а не для передачи реальных символов (букв, цифр и знаков препинания), они не приводятся в таблицах. Остальные символы ASCII можно разделить на группы по 32. В первой группе находятся знаки препинания и цифры, во второй — буквы в верхнем регистре, а в третьей — буквы в нижнем регистре:

```
>>> for i in range(32, 128, 32):
...     print(' '.join(chr(j) for j in range(i, i+32)))
...
! " # $ % & ' ( ) * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
@ A B C D E F G H I J K L M N O P Q R S T U V W X Y Z [ \ ] ^ _
` a b c d e f g h i j k l m n o p q r s t u v w x y z { | } ~
```

Кстати, символ в левом верхнем углу — это пробел, с кодом 32. Невидимый символ в правом нижнем углу — последний контрольный символ: Delete на позиции 127. У этой кодировки, разработанной в 1960-х годах, есть две интересных особенности. Во-первых, цифры идут в таком порядке, что мы можем узнать числовое значение каждой цифры, вычтя из ее кода код нуля. Во-вторых, мы можем переключаться между буквами в верхнем и нижнем регистрах, вычитая и прибавляя 32 бита, даже в строке символов.

Однако Python 3 работает со строками не только в кодировке ASCII, которая включает всего 128 кодов. У нас есть более современный стандарт — Unicode, с тысячами и даже миллионами символов. Python считает строки последовательностью символов Unicode, а фактическое представление строк Python в ОЗУ, как и остальные

структуры данных Python, скрыто от нас. При работе с данными в файле или по сети нам нужно думать о том, как они будут выглядеть. Существует два процесса для преобразования символов:

- ◆ при *кодировании* символов мы преобразуем строку символов Unicode в байты, которые можно передавать за пределами программы Python;
- ◆ когда мы преобразуем байтовую строку в символы, это называется *декодированием*.

Представьте, что вне программы Python информация хранится в виде секретного кода (в байтах), который нужно расшифровать, прежде чем понять и выполнить. На выходе из программы Python данные нужно закодировать, а в обратном направлении — декодировать.

Кодировки бывают разные, но их можно разделить на две большие группы.

Кодировки могут быть *однобайтовыми*, т. е. вмещать только 256 уникальных символов, но при этом каждый символ укладывается в 1 байт. Эти кодировки удобно использовать в сетевом программировании. Мы знаем, что при считывании n байтов из сокета мы получим n символов, а когда поток байтов разделяется на фрагменты, каждый байт будет отдельным символом, который можно обработать, не зная, какой байт следует за ним. Мы также можем найти n -й символ в n -м байте.

Многобайтовые кодировки устроены сложнее и не имеют таких преимуществ. Некоторые кодировки, например UTF-32, используют фиксированное число байтов для каждого символа. Это неэффективно, если данные содержат в основном символы ASCII, зато у всех символов одинаковая длина. Другие кодировки, например UTF-8, передают символы в разном количестве байтов, так что нужно проявлять осторожность, ведь если поток данных передается по частям, мы никак не можем узнать, разделен символ или нет. А еще мы можем найти символ n , только если будем читать с самого начала.

В документации по модулю `codecs` в стандартной библиотеке есть список всех кодировок, которые поддерживает Python.

Большинство однобайтовых кодировок Python основаны на ASCII и используют остальные 128 символов для региональных букв и символов:

```
>>> b'\x67\x68\x69\xe7\xe8\xe9'.decode('latin1')
'ghiçée'
>>> b'\x67\x68\x69\xe7\xe8\xe9'.decode('latin2')
'ghiç
é'
>>> b'\x67\x68\x69\xe7\xe8\xe9'.decode('greek')
'ghiḡqi'
>>> b'\x67\x68\x69\xe7\xe8\xe9'.decode('hebrew')
'ghiḡqi'
```

То же можно сказать и о многочисленных кодовых страницах Windows, которые упоминаются в стандартной библиотеке. Кроме того, есть однобайтовые кодиров-

ки, которые никак не связаны с ASCII и основаны на устаревших стандартах мейн-фрейма IBM.

```
>>> b'\x67\x68\x69\xe7\xe8\xe9'.decode('EBCDIC-CP-BE')
'ÃÇÑXYZ'
```

Старая схема UTF-16 (которая недолго была популярна, когда Unicode был гораздо меньше и помещался в 16 бит), современная схема UTF-32 и популярная UTF-8 переменной ширины, которая похожа на ASCII, пока мы не начинаем включать символы с кодом больше 127, — все это популярные многобайтовые кодировки, которые будут часто вам попадаться. Вот как выглядит строка Unicode в этих трех кодировках:

```
>>> len('Namárië!')
8
>>> 'Namárië!'.encode('UTF-16')
b'\xff\xfeN\x00a\x00m\x00\xe1\x00r\x00i\x00\xeb\x00!\x00'
>>> len(_)
18
>>> 'Namárië!'.encode('UTF-32')
b'\xff\xfe\x00\x00N\x00\x00\x00a\x00\x00\x00m\x00\x00\x00\xe1\x00\x00\x00r\x00\x00\x00i\x00\x00\x00\xeb\x00\x00\x00!\x00\x00\x00'
>>> len(_)
36
>>> 'Namárië!'.encode('UTF-8')
b'Nam\xc3\xa1ri\xc3\xab!'
>>> len(_)
10
```

Здесь можно увидеть символы ASCII — N, a, m, r и i — среди байтовых значений, представляющих символы, которых нет в ASCII.

У многобайтовых кодировок есть дополнительный символ, поэтому в UTF-16 всего получается $(8 \times 2) + 2$ байта, а в UTF-32 — $(8 \times 4) + 4$ байта. Маркер последовательности байтов, или метка порядка байтов (byte order marker, BOM) — это специальный символ, который позволяет при считывании автоматически определять, какой байт идет первым — самый важный или наименее важный. О порядке байтов мы подробно поговорим в следующем разделе.

При работе с закодированным текстом мы часто встречаем две ошибки: попытку декодировать закодированную байтовую строку по правилам, которые не соответствуют этой кодировке, и попытку закодировать символы, которых нет в этой кодировке.

```
>>> b'\x80'.decode('ascii')
```

```
Traceback (most recent call last):
```

```
...
```

```
UnicodeDecodeError: 'ascii' codec can't decode byte 0x80 in position 0: ordinal not in range(128)
```

```
>>> 'ghihqi'.encode('latin-1')
```

```
Traceback (most recent call last):
```

```
...
```

```
UnicodeEncodeError: 'latin-1' codec can't encode characters in position 3-5: ordinal not in range(256)
```

Для того чтобы решать такие проблемы, мы обычно проверяем, какую кодировку мы используем, или пытаемся понять, почему наши данные не соответствуют ожидаемой кодировке. Если это не помогает решить проблему и в коде остаются несоответствия между указанными кодировками и фактическими строками данных, можно изучить документацию по стандартной библиотеке и найти другие способы, чтобы избегать исключений.

```
>>> b'ab\x80def'.decode('ascii', 'replace')
```

```
'ab[?]def'
```

```
>>> b'ab\x80def'.decode('ascii', 'ignore')
```

```
'abdef'
```

```
>>> 'ghihqi'.encode('latin-1', 'replace')
```

```
b'ghi??'
```

```
>>> 'ghihqi'.encode('latin-1', 'ignore')
```

```
b'ghi'
```

Эти примеры объясняются в документации по модулю `codecs` в стандартной библиотеке. Больше примеров можно найти в блоге "Python Module of the Week" Дага Хелмана (Doug Hellman).

Как вы уже знаете, рискованно декодировать частично полученное сообщение с помощью кодировки, в которой символ обозначается несколькими байтами, потому что один символ может случайно разделиться на два пакета. Некоторые подходы к этой проблеме приводятся в *разд. "Кадрирование" далее в этой главе*.

Сетевой порядок байтов и двоичные числа

Если мы собираемся отправлять по сети только текст, нам нужно понимать только кодировку и кадрирование (подробнее об этом *см. в следующем разделе*).

Однако иногда нам недостаточно простого текста, или мы пишем код, который будет взаимодействовать с сервисом, использующим двоичные данные. В этом случае мы должны понимать сетевой порядок байтов.

Для того чтобы понять, причем тут порядок байтов, давайте отправим по сети целое число, скажем, 4253.

Многие протоколы передадут целое число просто в виде строки '4253' из четырех символов. В распространенных текстовых кодировках нам понадобится минимум 4 байта для этого числа. Поскольку числа хранятся не в десятичном формате, нам

требуется несколько раз делить их и проверять остаток, чтобы программа поняла, что на самом деле это число равно $4000 + 200 + 50 + 3$. Когда мы получаем строку '4253', потребуется выполнить несколько операций сложения и умножения, чтобы превратить текст в число.

Несмотря на то что в текстовом виде числа занимают больше места, обычно именно в таком виде они и пересылаются в Интернете. Когда мы, например, запрашиваем веб-страницу, HTTP-протокол сообщает значение Content-Length результата как строку десятичных цифр — '4253'. Несмотря на все сложности, веб-сервер и клиент уверенно преобразуют десятичные числа. В последние десятилетия развития сетей существует тенденция к замене двоичных форматов на очевидные и понятные человеку, даже если для их обработки потребуется больше вычислительных ресурсов.

Конечно, современные процессы лучше справляются с умножением и делением, чем во времена, когда двоичные числа использовались гораздо чаще, причем не только потому, что сами процессоры стали быстрее, но и потому, что их разработчики теперь гораздо умнее реализуют арифметику с целыми числами, так что эти операции сегодня требуют меньше циклов, чем, скажем, в начале 1980-х годов.

В любом случае компьютер представляет значение этой переменной в Python не как строку '4253'.

Он сохраняет его как двоичное целое число в виде битов нескольких последовательных байтов. С помощью встроенной функции `hex()` Python мы можем посмотреть, как хранится целое число.

```
>>> hex(4253)
'0x109d'
```

Поскольку каждое шестнадцатеричное число занимает 4 бита, в полный байт умещаются два таких числа. Число хранится как самый важный байт `0x10` и наименее важный байт `0x9d` рядом друг с другом в памяти, а не как четыре десятичных числа (4, 2, 5 и 3), из которых 4 — самая важная цифра (поскольку если ее изменить, число изменится на тысячу), а 3 — наименее важная.

В какой последовательности должны стоять эти 2 байта? Это зависит от архитектуры процессора того или иного производителя. Они сходятся в том, что байты в памяти должны храниться по порядку, и что строка Content-Length: 4253 должна храниться в этом порядке, начиная с 4 и заканчивая 3, но порядок хранения двоичных чисел различается.

Некоторые компьютеры (например, старые процессоры SPARC) хранят байты в обратном порядке, от старшего к младшему, в том виде, в каком мы привыкли видеть числа. Другие компьютеры (например, популярная архитектура x86) первым размещают наименее значимый байт ("первый" здесь означает байт с наименьшим адресом в памяти).

Дэнни Коэн (Danny Cohen) в своей заметке IEN-137 "О священных войнах и призыв к миру" ("On Holy Wars and a Plea for Peace") придумал термины big-endian (обратный порядок байтов) и little-endian (прямой порядок байтов) и описал проблему как

пародию на спор о том, с какого конца лучше разбивать яйца, из книги Джонатана Свифта (www.ietf.org/rfc/ien/ien137.txt).

В Python различия между двумя подходами очевидны. Достаточно использовать модуль `struct`, в котором есть несколько процедур для преобразования данных между двоичными формами. Число 4253 сначала представлено в прямом порядке, а потом в обратном:

```
>>> import struct
>>> struct.pack('<i', 4253)
b'\x9d\x10\x00\x00'
>>> struct.pack('>i', 4253)
b'\x00\x00\x10\x9d'
```

4253 — небольшое число, так что я использовал форматирование `struct`, где 4 байта представляют целое число и первые 2 байта равны нулю. Для того чтобы было проще запомнить, представьте, что `>` и `<` — это стрелки, которые указывают на наименее значимый конец строки байтов. Полный список форматов данных, поддерживаемых модулем `struct`, см. в документации по стандартной библиотеке. В модуле есть метод `unpack()` для преобразования двоичных данных в числа Python.

```
>>> struct.unpack('>i', b'\x00\x00\x10\x9d')
(4253,)
```

Если обратный порядок кажется вам более логичным, вы будете рады узнать, что он "победил" и стал стандартом для сетевых взаимодействий. В результате в модуле `struct` появился новый символ `!` для методов `pack()` и `unpack()`, который сообщает другим программистам (и нам самим, когда мы будем читать код позже), что мы упаковываем данные для пересылки по сети.

Вот несколько рекомендаций по подготовке двоичных данных для передачи через сетевой сокет.

- ◆ Используйте модуль `struct`, чтобы создать двоичные данные для передачи по сети и распаковать их по прибытии.
- ◆ Если вы контролируете формат данных, используйте префикс `!`, чтобы выбрать сетевой порядок байтов. Если протокол создан кем-то другим и указан прямой порядок байтов, используйте `"`.

Всегда проверяйте структуру, чтобы посмотреть, как она упорядочивает ваши данные по сравнению со стандартом протокола. Используйте символы `"x"` для заполнения строки пустыми байтами.

Для преобразования целых чисел в байтовые строки в нужном порядке старый код Python использовал бы много функций со странными названиями из модуля `socket`, например `ntohl()` или `htons()`. Они соответствуют функциям библиотеки POSIX для работы с сетями, которые тоже включают вызовы `socket()` и `bind()`. Вместо этих неудобных функций советую использовать модуль `struct`. Он более гибкий, функциональный и понятный.

Кадрирование

Если мы отправляем данные в датаграммах UDP, они передаются отдельными фрагментами, которые мы можем идентифицировать. Если в сети что-то пойдет не так, мы сможем переупорядочить и передать эти фрагменты вручную, как описано в *главе 2*.

Но если мы выбрали более популярный протокол, TCP, возникает проблема кадрирования: как разграничить сообщение таким образом, чтобы получатель знал, когда заканчивается одно сообщение и начинается другое? Когда мы передаем данные через `sendall()`, они делятся на пакеты, и прежде чем приложение прочитает сообщение, ему придется сделать несколько вызовов `recv()`, или не придется, если все пакеты поступят до того, как операционная система снова запланирует этот процесс.

Когда получатель может прерывать выполняющуюся функцию `recv()`, поскольку сообщение пришло полностью и его уже можно обрабатывать целиком?

Существуют разные методы. Например, есть способ для очень простых сетевых протоколов, которые только доставляют данные и не ждут ответа, так что получателю не нужно в какой-то момент прерывать получение и переходить к ответу. Отправитель будет выполнять цикл, пока не передаст функции `sendall()` все исходящие данные, а затем завершит сокет методом `shut()`. Получателю просто нужно будет выполнить метод `recv()` несколько раз, пока вызов не вернет пустую строку, указывающую, что отправитель закрыл сокет. Этот метод использован в листинге 5.1.

Листинг 5.1. Отправка всех данных и завершение соединения

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Клиент, отправляющий данные, закрывает сокет, не дожидаясь ответа.

import socket
from argparse import ArgumentParser

def server(address):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(address)
    sock.listen(1)
    print('Run this script in another window with "-c" to connect')
    print('Listening at', sock.getsockname())
    sc, sockname = sock.accept()
    print('Accepted connection from', sockname)
    sc.shutdown(socket.SHUT_WR)
    message = b''
```

```

while True:
    more = sc.recv(8192) # произвольное значение 8k
    if not more:         # сокет закрывается, когда recv() возвращает ''
        print('Received zero bytes - end of file')
        break
    print('Received {} bytes'.format(len(more)))
    message += more
print('Message:\n')
print(message.decode('ascii'))
sc.close()
sock.close()

def client(address):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect(address)
    sock.shutdown(socket.SHUT_RD)
    sock.sendall(b'Beautiful is better than ugly.\n')
    sock.sendall(b'Explicit is better than implicit.\n')
    sock.sendall(b'Simple is better than complex.\n')
    sock.close()

if __name__ == '__main__':
    parser = ArgumentParser(description='Transmit & receive a data stream')
    parser.add_argument('hostname', nargs='?', default='127.0.0.1',
                        help='IP address or hostname (default: %(default)s)')
    parser.add_argument('-c', action='store_true', help='run as the client')
    parser.add_argument('-p', type=int, metavar='port', default=1060,
                        help='TCP port number (default: %(default)s)')
    args = parser.parse_args()
    function = client if args.c else server
    function((args.hostname, args.p))

```

Если мы запустим скрипт от имени сервера, а затем запустим клиент из другой командной строки, мы заметим, что все данные клиента доходят до сервера без изменений, и единственное кадрирование — событие конца файла, которое клиент иницирует при закрытии сокета.

\$ python streamer.py

Run this script in another window with "-c" to connect

Listening at ('127.0.0.1', 1060)

Accepted connection from ('127.0.0.1', 49057)

Received 96 bytes

Received zero bytes - end of file

Message:

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Клиент и сервер закрывают соединение в направлении, которое не собираются использовать, и это очень удобно, потому что этот сокет не должен больше получать данные. Теперь мы не сможем случайно использовать его в другом направлении, что могло бы привести к взаимоблокировке, как мы видели в листинге 3.2 из *главы 3*. Хотя вызвать метод `shutdown()` для сокета должен только клиент или только сервер, можно на всякий случай для симметрии закрыть его в оба направления.

Второй способ — вариация первого. Поначалу сокет остается открытым в оба направления. Сначала данные передаются в одном направлении (как в листинге 5.1), а затем это направление отключается. Сокет закрывается после того, как данные будут переданы в противоположном направлении. В листинге 3.2 из *главы 3* мы видели, как важно завершить передачу данных в одном направлении, прежде чем направить поток в другую сторону, иначе можно случайно заблокировать клиента и сервер.

Третий способ мы видели в листинге 3.1: использовать сообщения фиксированной длины. Можно отправить байтовую строку методом `sendall()`, а затем написать цикл `recv()`, чтобы получить сообщение целиком.

```
def recvall(sock, length):
    data = ''
    while len(data) < length:
        more = sock.recv(length - len(data))
        if not more:
            raise EOFError('socket closed {} bytes into a {}-byte'
                           ' message'.format(len(data), length))
        data += more
    return data
```

В наше время данные такие динамичные, что сообщения фиксированной длины — большая редкость. Однако в некоторых случаях мы можем передавать таким образом двоичные данные. Например, если какой-то формат всегда выдает блоки данных одинаковой длины.

Четвертый способ — использовать специальные символы для разделения сообщений. Получатель будет продолжать цикл `recv()`, как в примере выше, пока не получит разделитель, указывающий на конец сообщения. Если байты или символы сообщения будут находиться в определенном диапазоне, логично будет завершать сообщения символом за пределами этого диапазона. Если, например, мы отправляем строки ASCII, можно использовать символ `null` или символ, которого нет в ASCII, допустим `xff`.

Если сообщение может содержать любые данные, выбрать разделитель будет проблематично, ведь этот символ может появляться в данных. Конечно, можно использовать кавычки, например, как мы используем символ одинарной кавычки посреди строки Python, окруженной одинарными кавычками.

```
'All\'s well that ends well.'
```

Я рекомендую использовать этот разделитель, только если в сообщении встречается ограниченный набор символов, потому что расставить кавычки правильно бывает очень сложно, особенно с произвольными данными. Например, мы должны убедиться, что не путаем разделитель в кавычках с настоящими кавычками, которые закрывают сообщение. Вторая проблема — потом нужно будет снова пройти по сообщению и удалить кавычки, которые окружали литеральный разделитель. Кроме того, при таком подходе мы не сможем узнать длину сообщения, пока не декодируем его.

Пятый способ — добавлять к каждому сообщению префикс с указанием его длины. Блоки двоичных данных можно передавать без изменений (без парсинга, кавычек или интерполяции), так что это популярный выбор для высокопроизводительных протоколов. Длину необходимо указать с помощью одного из подходов, описанных ранее. Например, длина часто выражается как двоичное целое число фиксированной ширины или десятичная строка переменной длины, за которой следует текстовый разделитель. Прочитав и декодировав длину, получатель может циклически выполнять `recv()`, пока не получит сообщение целиком. Цикл может выглядеть как пример в листинге 3.1, только вместо числа 16 можно использовать переменную длину.

А что если нам нравится простота и эффективность пятого способа, но мы не знаем длину сообщений заранее? Например, отправитель считывает данные из другого источника и не может предугадать их длину?

Стоит ли отказываться от такого элегантного подхода и использовать разделители?

Шестой способ подходит для ситуаций, когда длина сообщений неизвестна. Вместо отправки одного блока данных мы отправляем несколько, и у каждого в начале указывается длина. Когда отправителю поступают новые данные, можно узнать длину каждого блока и добавить его в исходящий поток. Когда придет время завершить передачу, отправитель сможет послать согласованный сигнал, например поле длины со значением ноль, чтобы сообщить получателю, что последовательность блоков закончена.

В листинге 5.2 мы видим простую иллюстрацию этой концепции. Как и в предыдущем примере, в этом листинге данные отправляются только в одном направлении — от клиента к серверу, но структура данных гораздо интереснее.

Перед каждым сообщением идет структура длиной 4 байта. Размер кадра не должен превышать 4 Гбайт, потому что `I` означает 32-битное число без знака. В этом простом коде мы отправляем на сервер три блока, а затем сообщение с нулевой длиной (поле длины с нулями и без содержимого), чтобы сервер знал, что серия блоков завершена.

Листинг 5.2. Кадрирование данных с помощью указания длины перед каждым блоком

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Отправка данных потоком с разделением на блоки с указанием длины в префиксе.

import socket, struct
from argparse import ArgumentParser

header_struct = struct.Struct('!I')    # сообщения длиной до 2**32 - 1

def recvall(sock, length):
    blocks = []
    while length:
        block = sock.recv(length)
        if not block:
            raise EOFError('socket closed with %d bytes left'
                            ' in this block'.format(length))
        length -= len(block)
        blocks.append(block)
    return b''.join(blocks)

def get_block(sock):
    data = recvall(sock, header_struct.size)
    (block_length,) = header_struct.unpack(data)
    return recvall(sock, block_length)

def put_block(sock, message):
    block_length = len(message)
    sock.send(header_struct.pack(block_length))
    sock.send(message)

def server(address):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(address)
    sock.listen(1)
    print('Run this script in another window with "-c" to connect')
    print('Listening at', sock.getsockname())
    sc, sockname = sock.accept()
    print('Accepted connection from', sockname)
```

```

sc.shutdown(socket.SHUT_WR)
while True:
    block = get_block(sc)
    if not block:
        break
    print('Block says:', repr(block))
sc.close()
sock.close()

```

```

def client(address):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect(address)
    sock.shutdown(socket.SHUT_RD)
    put_block(sock, b'Beautiful is better than ugly.')
    put_block(sock, b'Explicit is better than implicit.')
    put_block(sock, b'Simple is better than complex.')
    put_block(sock, b'')
    sock.close()

if __name__ == '__main__':
    parser = ArgumentParser(description='Transmit & receive blocks over TCP')
    parser.add_argument('hostname', nargs='?', default='127.0.0.1',
                        help='IP address or hostname (default: %(default)s)')
    parser.add_argument('-c', action='store_true', help='run as the client')
    parser.add_argument('-p', type=int, metavar='port', default=1060,
                        help='TCP port number (default: %(default)s)')
    args = parser.parse_args()
    function = client if args.c else server
    function((args.hostname, args.p))

```

Обратите внимание, какую осторожность нужно проявлять! Поле длиной 4 байта так мало, что можно подумать, будто функция `recv()` не вернет его сразу, но все же код будет работать, только если `recv()` будет выполняться в цикле, а скрипт будет запрашивать данные, пока не поступят все 4 байта. Это нужно понимать.

Итак, у нас есть целых шесть способов разделить бесконечный поток данных на удобные фрагменты. Современные протоколы часто совмещают эти способы, и вы тоже можете их комбинировать.

Протокол HTTP, например, о котором мы поговорим чуть позже, — это хороший пример смешения нескольких подходов к кадрированию. Он использует пустую строку `'rnrn'` в качестве разделителя, чтобы указать окончание заголовков. Конец строки можно считать специальным символом, потому что заголовки — это текст.

Поскольку полезная нагрузка может включать только двоичные данные, например изображение или сжатый файл, заголовки включают параметр `Content-Length` в байтах, чтобы определить, сколько еще данных нужно считать из сокета после чтения заголовков.

Получается, что HTTP комбинирует четвертый и пятый способы. Кроме того, он может использовать и шестой метод: если сервер не способен оценить длительность ответа, HTTP может использовать механизм кодирования фрагментированной передачи (`chunked transfer encoding`), доставляющий последовательность блоков, у каждого из которых есть префикс с длиной ответа. Поле нулевой длины, как в листинге 5.2, указывает на конец передачи.

Pickle и форматы с разделителями

Стоит отметить, что у некоторых типов данных уже имеются встроенные разделители. Нет необходимости кадрировать данные, которые уже используют свой механизм разделения.

Например, в стандартной библиотеке Python есть модуль `pickle` для сериализации. Используя странное сочетание текстовых команд и данных, `pickle` сохраняет содержимое структуры данных Python, чтобы его можно было восстановить позже или в другой системе.

```
>>> import pickle
```

```
>>> pickle.dumps([5, 6, 7])
```

```
b'\x80\x03]q\x94(K\x05K\x06K\x07e.'
```

Символ `'.'` в конце предыдущей строки — это самый интересный аспект этих выходных данных. Таким образом формат отмечает окончание элемента `pickle`. Когда загрузчик доходит до этого символа, он останавливается и возвращает значение, не продолжая чтение. Получается, мы можем взять предыдущий `pickle`, добавить в конец какую-то бессмыслицу, а метод `loads()` проигнорирует лишнее и вернет исходный список.

```
>>> pickle.loads(b'\x80\x03]q\x94(K\x05K\x06K\x07e.blahblahblah')
```

```
[5, 6, 7]
```

Конечно, этот метод неэффективен для сетевых данных, потому что он не указывает, сколько байтов было обработано для загрузки этого элемента `pickle`, и мы не знаем, какую часть строки составляют данные из `pickle`. Если мы используем функцию `pickle.load()` для чтения из файла, указатель файла будет находиться в конце данных `pickle`, и мы можем начать чтение с этого места, если хотим узнать, что именно идет после `pickle`.

```
>>> from io import BytesIO
```

```
>>> f = BytesIO(b'\x80\x03]q\x94(K\x05K\x06K\x07e.blahblahblah')
```

```
>>> pickle.load(f)
```

```
[5, 6, 7]
```



```
>>> f.tell()
14
>>> f.read()
b'blahblahblah'
```

Также мы можем создать протокол, который будет только передавать элементы pickle между двумя приложениями Python. Библиотека pickle умеет читать из файла и знает, что иногда нужно повторять операции чтения, пока pickle не будет прочитан до конца, поэтому нам не нужен будет цикл с функцией `recvall()`, как в листинге 5.2. Если мы хотим обернуть сокет в объект файла Python, чтобы он мог использоваться, например, функцией `pickle.load()`, нам понадобится метод сокета `makefile()`, описанный в *главе 3*.

Разделение больших структур данных на элементы pickle имеет много нюансов, особенно если они содержат объекты Python, кроме простых встроенных типов (целые числа, строки, списки и словари).

JSON и XML

Форматы данных JSON и XML довольно популярны в ситуациях, когда протокол должен читаться другими языками программирования или вы просто предпочитаете универсальные, а не специфичные для Python стандарты. Эти два формата не поддерживают кадрирование, поэтому нужно понять, как получить целую строку текста по сети, чтобы проанализировать ее.

Сегодня для передачи данных между компьютерами чаще всего используется JSON. Начиная с Python 2.6 в стандартную библиотеку входит модуль `json`. Он предоставляет универсальный метод для сериализации простых структур данных.

```
>>> import json
>>> json.dumps([49, 'hello!'])
'[49, "hello!"]'
>>> json.dumps([49, 'hello!'], ensure_ascii=False)
'[49, "hello!"]'
>>> json.loads('{"name": "bob", "quest": "how are you?"}')
{'quest': 'how are you?', 'name': 'bob'}
```

JSON не только поддерживает символы Unicode в строках, но и может включать их в полезные данные, если мы сообщим модулю `json`, что выходные данные не должны ограничиваться символами ASCII. Стоит отметить, что JSON работает со строками, поэтому входные и выходные данные у модуля `json` — это полные строки, а не байтовые объекты Python. По стандарту JSON строки должны передаваться по сети в кодировке UTF-8.

Формат XML принимает строки и помечает их, окружая угловыми скобками. Обычно он используется для документов. Пока вам достаточно знать, что мы обя-

заны использовать XML только с протоколом HTTP. Иногда он хорошо сочетается с другими стандартами, если нам нужна текстовая разметка.

Двоичные форматы, вроде Thrift или Google Protocol Buffers, отличаются от описанных выше, потому что клиенту и серверу требуется описание того, что будет входить в каждое сообщение. Это довольно популярные альтернативные форматы, которые я советую вам изучить. Они поддерживают разные версии, т. е. мы можем запускать в рабочем окружении новые серверы и при этом поддерживать связь с компьютерами, использующими более старую версию протокола, которую мы пока не успели обновить. Они быстрые и простые и хорошо работают с двоичными данными.

Сжатие

Поскольку для передачи данных по сети обычно требуется больше времени, чем центральный процессор тратит на подготовку данных к этой передаче, иногда имеет смысл сжимать данные перед отправкой. Как мы увидим в *главе 9*, популярный протокол HTTP позволяет клиенту и серверу определять, поддерживают ли они оба сжатие.

Модуль `zlib` в стандартной библиотеке Python предоставляет один из самых распространенных форматов сжатия в Интернете и имеет встроенные возможности кадрирования. Если мы начнем передавать ему сжатый поток данных, он сообщит, когда заканчивается поток, и предоставит доступ к несжатой полезной нагрузке, которая может идти следом.

Большинство протоколов предпочитают сами кадрировать данные, а затем предоставлять получившиеся блоки библиотеке `zlib` для распаковки. Однако можно воспитать у себя полезную привычку — добавлять немного несжатых данных в конец каждой сжатой строки (здесь я буду использовать `b'.'`) и ждать, что объект сжатия расшифрует эти дополнительные данные как сигнал завершения.

Вот пример объединения сжатых потоков данных:

```
>>> import zlib
>>> data = zlib.compress(b'Python') + b'.' + zlib.compress(b'zlib')
+ b'.'
>>> data
b'\x9c\x0b\xa8,\xc9\xc8\xcf\x03\x00\x08\x97\x02\x83.\x9c\xab\xca\x9c\x02\x00\x04d\x01\xb2.'
>>> len(data)
28
```

Если полезных данных немного, большинство алгоритмов сжатия только увеличивают, а не уменьшают размер, потому что издержки сжатия перекрывают экономию. Допустим, мы отправляем эти 28 байт в 8-байтовых пакетах. После обработки

первого пакета неиспользованный слот данных объекта распаковки останется пустым, и это будет указывать на то, что поступит больше данных.

```
>>> d = zlib.decompressobj()
>>> d.decompress(data[0:8]), d.unused_data
(b'Pytho', b'')
```

Придется выполнить `recv()` для сокета еще раз. Когда мы отправляем второй блок из восьми символов в объект `decompress`, он завершит сжатые данные, которые мы ждали, и вернет непустое значение `unused_data`, указывая, что мы наконец получили байт `b'.'`.

```
>>> d.decompress(data[8:16]), d.unused_data
('n', '.x')
```

После точки должен идти первый байт полезных данных, которые следуют за этим первым фрагментом сжатых данных. Поскольку мы ожидаем больше сжатых данных, мы передаем `'x'` новому объекту `decompress`, а затем отправляем ему последний 8-байтовый пакет.

```
>>> d = zlib.decompressobj()
>>> d.decompress(b'x'), d.unused_data
(b'', b'')
>>> d.decompress(data[16:24]), d.unused_data
(b'zlib', b'')
>>> d.decompress(data[24:]), d.unused_data
(b'', b'.')
```

Теперь `unused_data` содержит значения, а значит, мы прошли конец второго пакета сжатых данных и можем изучать его содержимое, точно зная, что он пришел целиком и полностью. Обычно разработчики протоколов не требуют обязательного сжатия и создают собственное кадрирование. Однако если вы знаете, что всегда будете использовать библиотеку `zlib`, можно воспользоваться ее встроенными возможностями завершения потоков и автоматического определения конца каждого сжатого потока.

Исключения в сети

Обычно примеры кода в этой книге перехватывают исключения, которые относятся к рассматриваемой теме. Например, в листинге 2.2 для иллюстрации времени ожидания сокета приводится исключение `socket.timeout`. Мы игнорировали остальные исключения, которые происходят, когда мы указываем неверное имя хоста в командной строке, используем метод `bind()` с удаленным IP-адресом, порт, для которого вызван `bind()`, уже занят, или узел не может ответить либо перестает отвечать.

При работе с сокетами могут возникать разные проблемы. Возможных ошибок при использовании сетевого соединения очень много, включая все, что только может

пойти не так на каждом уровне сложного протокола TCP/IP. К счастью, на нашу программу может повлиять относительно небольшое число исключений. Далее перечислены исключения, связанные с сокетами.

- ◆ **OSError.** Это исключение вызывается почти для любой проблемы, которая может возникнуть при передаче данных по сети. Это может случиться в любой момент во время вызова сокета, даже когда мы этого не ожидаем. Например, если предыдущий метод `send()` вызвал пакет сброса (RST) с удаленного хоста, такую ошибку будет вызывать следующая операция с этим сокетом.
- ◆ **socket.gaierror.** `Getaddrinfo()` вызывает это исключение, когда не может найти нужное имя или сервер. GAI расшифровывается как Get Address Info. Это исключение может возникать не только при явном вызове `getaddrinfo()`, но и когда мы вызываем `bind()` или `connect()` с именем хоста вместо IP-адреса, а поиск имени хоста завершается сбоем. Если возникает это исключение, мы можем найти номер ошибки и сообщение внутри объекта исключения.

```
>>> import socket
>>> s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
>>> try:
...     s.connect(('nonexistent.hostname.foo.bar', 80))
... except socket.gaierror as e:
...     raise
...
Traceback (most recent call last):
...
socket.gaierror: [Errno -2] Name or service not known
>>> e.errno
-2
>>> e.strerror
'Name or service not known'
```

Это исключение возникает только в случае, если вы или библиотека, которую вы используете, устанавливаете время ожидания и не ждете завершения вызовов `send()` или `recv()` бесконечно. То есть время ожидания истекает до того, как операция завершается обычным образом.

Также в документации по стандартной библиотеке для модуля `socket` описано исключение `herror`. К счастью, оно возникает, только если мы используем устаревшие вызовы поиска адреса.

Когда мы используем высокоуровневые протоколы на основе сокетов в Python, нужно решить, код будет получать ошибки самих сокетов или преобразовывать их в ошибки собственных типов. В стандартной библиотеке Python есть решения для обоих вариантов. `httplib`, например, работает на достаточно низком уровне, чтобы отображать ошибку сокета при подключении к нераспознанному имени хоста.

```
>>> import http.client
>>> h = http.client.HTTPConnection('nonexistent.hostname.booo.far')
```

```
>>> h.request('GET', '/')

```

```
Traceback (most recent call last):

```

```
...
```

```
socket.gaierror. [Errno -2] Name or service not known

```

A urllib2, наоборот, скрывает эту проблему и вызывает исключение URLError, чтобы сохранить семантику аккуратной системы для разрешения URL в документы.

```
>>> import urllib.request

```

```
>>> urllib.request.urlopen('http://nonexistent.hostname.boofar/')

```

```
Traceback (most recent call last):

```

```
...
```

```
socket.gaierror. [Errno -2] Name or service not known

```

```
During handling of the above exception, another exception occurred:

```

```
Traceback (most recent call last):

```

```
...
```

```
urllib.error.URLError: <urlopen error [Errno -2] Name or service not known>

```

В зависимости от реализации используемого протокола мы можем работать только с исключениями протокола или с исключениями протокола и сокетов. В документации к библиотеке указывается, какой подход используется. Я постарался привести фрагменты для ключевых пакетов, которые будут рассматриваться в следующих главах, чтобы определить исключения, которые могут возникать в коде при использовании той или иной библиотеки. Также вы всегда можете запустить библиотеку, передать ей несуществующее имя хоста или даже запустить ее без подключения к сети и посмотреть, какие ошибки она будет выдавать.

Как предусмотреть обработку всех возможных ошибок при создании сетевой программы? Этот вопрос касается не только сети. В любых программах Python нужно обрабатывать исключения, и решения, о которых я рассказываю в этой главе, относятся к разным сценариям. Наш подход будет зависеть от того, как мы поступаем с исключениями, — упаковываем их для обработки другими программистами, которые используют ваш API, или перехватываем исключения, чтобы сообщить о них конечному пользователю.

Специфические исключения

Существуют два метода предоставлять исключения пользователям вашего API. Конечно, нередко только вы будете использовать созданный вами модуль или процедуру. Однако представьте, что однажды вы забудете, как создавали модуль, и тогда будете благодарны за простой и ясный подход к исключениям.

Один из методов — игнорировать исключения в сети. Они будут видны вызывающему объекту, который сможет перехватить их или сообщить о них. Этот метод подходит для низкоуровневых сетевых программ, в которых вызывающий объект

понимает, почему мы создаем сокет и почему при его создании или использовании произошел сбой. Разработчик, создающий вызывающий код, ожидает сетевой сбой, только если связь между вызываемыми объектами API и низкоуровневыми сетевыми действиями очевидна.

Альтернатива — упаковать сетевые сбои в собственные исключения. Это упростит жизнь авторам, которые незнакомы с реализацией ваших программ, потому что их код сможет перехватывать исключения, связанные с действиями, выполняемыми вашим кодом, и им не придется понимать, как работают сокеты. Пользовательские исключения также позволяют создавать сообщения об ошибках, которые проясняют, что именно библиотека пыталась сделать, когда столкнулась с проблемами в сети.

Если мы создаем, например, метод `тусору()` для копирования файла из одной удаленной системы в другую, ошибка `socket.error` не позволит вызывающему объекту понять, где произошел сбой — на исходном компьютере, в месте назначения или где-то еще. В этом примере лучше создавать собственные исключения, например `SourceError` (ошибка на исходном компьютере) и `DestinationError` (ошибка на целевом компьютере), которые семантически тесно связаны с вашим API. Если пользователь вашего API захочет копнуть глубже, можно добавить исходный сокет, используя цепочку `raise...from`.

```
class DestinationError(Exception):
    def __str__(self):
        return '%s: %s' % (self.args[0], self.__cause__.strerror) # ...
    try:
        host = sock.connect(address)
    except socket.error as e:
        raise DestinationError('Error connecting to destination') from e
```

В этом коде предполагается, что `DestinationError` будет включать только потомков `OSError`, например `socket.error`. Для обработки сценариев, когда текстовая информация исключения хранится не в атрибуте `strerror`, метод `__str__()` был бы более сложным. По крайней мере, это иллюстрирует тренд. После получения `DestinationError` вызывающий объект может изучить его причину.

Исключения в сети: обнаружение и сообщение об ошибках

Обработка исключений может быть детальной и общей.

Если это *детальная обработка*, мы заключаем каждый сетевой вызов в конструкцию `try...except` и выводим содержательное сообщение об ошибке. В коротких программах этот подход можно считать эффективным, но для длинных он будет слишком громоздким, при этом не факт, что пользователь будет получать полезную информацию. Всегда думайте, будет ли очередной блок `try...except` с сообщением

о конкретной ошибке предоставлять нужную информацию. Альтернатива — *общие обработчики исключений*. В этом случае мы должны объединить фрагменты кода, выполняющие конкретные задачи, в более крупные блоки.

- ◆ "Единственная задача этой процедуры — подключиться к серверу лицензий".
- ◆ "Все действия этой функции с сокетом извлекают ответ из базы данных".
- ◆ "Весь код по очистке и завершению работы находится в этом разделе".

Затем части программы, которые собирают входные данные, аргументы командной строки и параметры конфигурации до начала операций, могут обертыть эти масштабные действия в обработчики:

```
import sys
...
try:
    deliver_updated_keyfiles(...)
except (socket.error, socket.gaierror) as e:
    print('cannot deliver remote keyfiles: {}'.format(e), file=sys.stderr)
exit(1)
```

Можно вызывать пользовательские ошибки, которые указывают на конкретные проблемы.

the program and print error output for the user.

```
except:
    FatalError('cannot send replies: {}'.format(e))
```

Затем в начале программы можно перехватывать все исключения `FatalError` и выводить сообщения об ошибках. Когда придет время добавить параметр командной строки, чтобы отправлять ошибки `FatalError` в системные журналы ошибок, а не выводить их на экран, достаточно будет изменить один фрагмент кода, а не десятки.

Есть еще одна причина добавить обработчик исключений в сетевую программу — повторно выполнять неудавшиеся операции. Это распространенная задача в долгосрочных приложениях. Представьте утилиту, которая регулярно рассылает сообщения электронной почты со статусом. Если вдруг она не сможет их отправлять, вряд ли она завершит работу из-за временной проблемы. Вместо этого поток сообщений может записать в журнал проблему, подождать несколько минут и попробовать снова.

В таком случае мы создаем обработчики событий для указанных последовательностей сетевых действий, которые мы считаем отдельной составной операцией, способной завершиться успехом или сбоем. Если что-то в этой последовательности пойдет не так, программа подождет 10 минут и попытается отправить сообщение снова. Использование оператора `try...except` зависит от структуры и логики выполняемых сетевых операций, а не от удобства пользователя или программиста.

Резюме

Для передачи по Интернету данные нужно преобразовывать таким образом, чтобы независимо от используемых механизмов хранения на компьютере данные были представлены в стандартном и воспроизводимом формате, доступном для чтения другими системами, программами и даже языками программирования.

Поскольку в IP-сети обычно используются 8-битные байты, мы должны выбирать такие кодировки, чтобы символы можно было преобразовывать в байты. Модуль `struct` в Python помогает упорядочить байты таким образом, чтобы с ними могли работать разные компьютеры. Наконец, структуры данных и документы иногда лучше передавать через JSON или XML, потому что это стандартные механизмы для передачи структурированных данных.

При работе с потоками TCP/IP мы должны учитывать кадрирование. Как узнать, когда начинается и заканчивается одно сообщение в потоке данных? Для этого есть разные методы, но все их нужно использовать с осторожностью, потому что метод `recv()` может вернуть только часть входящей передачи каждого вызова. Специальные символы-разделители, сообщения фиксированной длины и механизм `chunked encoding` — все это варианты разграничения блоков данных.

Элементы `pickle` в Python не только преобразуют структуры данных в строки, которые можно отправлять по сети, но и сообщают модулю `pickle`, где заканчивается входящий элемент `pickle`. Элементы `pickle` можно использовать для кадрирования отдельных сообщений в потоке в дополнение к кодированию данных. Модуль сжатия `zlib`, который часто используется с HTTP, также может определять конец сжатого сегмента, позволяя кадрировать сообщения без лишних затрат.

Сокеты и сетевые протоколы, используемые программами, могут вызывать разные ошибки. Если мы создаем библиотеку для других разработчиков или утилиту для конечных пользователей, можно использовать операторы `try...except`. В `try...except` можно обертывать целые части программы, если только они не выполняют слишком масштабные задачи. Наконец, в `try...except` следует отдельно инкапсулировать действия, которые можно выполнять повторно, если ошибка является временной.

Протокол SSL/TLS

Протокол TLS (Transport Layer Security — протокол защиты транспортного уровня), изначально называвшийся SSL (Secure Sockets Layer — уровень защищенных сокетов), был впервые представлен компанией Netscape в 1995 г. и стал стандартом в 1999 г. Сегодня это, пожалуй, самая популярная форма шифрования в Интернете. Он используется со многими базовыми протоколами для проверки подлинности сервера и защиты данных при передаче.

Подходы к реализации и использованию TLS постоянно меняются. Каждый год возникают новые атаки на его алгоритмы шифрования, поэтому разрабатываются новые шифры и методологии. На момент написания этой книги последней версией была TLS 1.2, хотя в будущем, конечно, появятся новые версии. Я постараюсь обновлять примеры скриптов в репозитории с кодом для этой книги, чтобы они соответствовали последним изменениям. Переходите по адресу перед каждым скриптом в этой главе и копируйте фрагмент кода из репозитория.

Содержание главы

- ◆ От чего не защищает TLS.
- ◆ Что худшее может случиться?
- ◆ Создание сертификатов.
- ◆ TLS Offloading.
- ◆ Контексты по умолчанию в Python 3.4.
- ◆ Подходы к обертке сокетов.
- ◆ Выбор шифров вручную и Perfect Forward Security.
- ◆ Поддержка протокола TLS.
- ◆ Дальнейшее изучение.
- ◆ Резюме.

Цель

В начале главы мы определим цели TLS и методы их достижения. Затем мы посмотрим, как активировать и настроить TLS на TCP-сокете с помощью простых и сложных примеров кода Python. Наконец, мы увидим, как TLS работает в протоколах, которые мы будем рассматривать далее.

От чего не защищает TLS

Если мы посмотрим на данные, проходящие через корректно настроенный сокет TLS, мы ничего не поймем. Более того, если все сделать правильно, то расшифровать сообщение не сможет ни другой компьютер, ни даже правительственная организация с огромным бюджетом. Злоумышленники не смогут узнать, какой URL вы запрашиваете, какой контент получаете, какие пароли вводите или какие файлы cookie могут передаваться через сокет. (Подробнее о паролях и cookie мы поговорим в *главе 9*, посвященной протоколу HTTP).

Надо понимать, что TLS шифрует не все данные сообщения, и какая-то информация доступна третьим сторонам.

- ◆ В IP-заголовке каждого пакета адрес вашего компьютера и хоста на той стороне отображаются в виде открытых байтов.
- ◆ Каждый TCP-заголовок также включает номера портов клиента и сервера.
- ◆ DNS-запрос, который клиент отправил, чтобы узнать IP-адрес сервера, тоже не шифруется.

Кроме того, можно узнать размер блоков данных, которые проходят через TLS-сокет в каждом направлении. Даже когда TLS пытается скрыть фактическое количество переданных байтов, мы все равно можем наблюдать общий паттерн запросов и ответов по примерному размеру фрагментов.

Давайте посмотрим эти недостатки на примере. Допустим, мы загружаем **<https://pypi.python.org/pypi/skyfield/>** по бесплатному Wi-Fi в кафе, используя защищенного клиента HTTPS (например, браузер).

Что увидит наблюдатель, т. е. другой человек, который тоже подключен к этой сети в кафе или обслуживает роутер? Сначала наблюдатель увидит, что наша система делает DNS-запрос для адреса **pypi.python.org**, и если возвращенный IP-адрес не содержит много других сайтов, наблюдатель может предположить, что дальше мы взаимодействуем с этим IP-адресом через порт 443, чтобы посещать страницы по адресу **https://pypi.python.org**. Поскольку HTTP сначала записывает запрос целиком и только потом записывает ответ, наблюдатель сможет различать наши запросы и ответы сервера.

Он также будет понимать примерный размер каждого возвращенного документа и порядок передачи.

Что можно узнать по этому размеру? У страниц <https://pypi.python.org> разный размер, и наблюдатель может узнать его, просканировав сайт с помощью веб-скрейпера (см. главу 11). Изображения и другие ресурсы в HTML (которые нужно загрузить при первом просмотре или если они стерлись из кеша браузера) зависят от типа страницы. Наблюдатель может не знать, что именно вы ищете или какие пакеты просматриваете и загружаете, но он сможет догадаться об этом по примерному размеру файлов.

В этой книге мы не будем говорить о том, как скрыть свое поведение в Интернете или защитить другие персональные данные при передаче, потому что для этого нужно изучить анонимные сети (например, Tor) и анонимные почтовые серверы. Какие бы средства защиты мы ни применяли, наша система все равно будет отправлять и получать блоки данных, по размеру которых можно примерно догадаться, что мы делаем. При наличии подходящих средств злоумышленник может даже заметить, что наши запросы совпадают с полезными данными, которые выходят из анонимной сети к определенному месту назначения.

В этой главе мы рассмотрим только возможности TLS и их использование в программах Python.

Что худшее может случиться?

Для того чтобы понять основные аспекты TLS, давайте рассмотрим ряд проблем, с которыми сталкивается этот протокол при установке соединения.

Допустим, мы хотим установить TCP-соединение с определенным именем хоста и номером порта в Интернете, и мы неохотно соглашаемся с тем фактом, что любой сторонний наблюдатель сможет увидеть наш DNS-поиск имени хоста, а также номер порта, к которому мы подключаемся (т. е. используемый нами протокол, если только владелец сервиса не назначил нестандартные номера портов). Через TCP-соединение мы подключаемся к IP-адресу и порту. Если протокол требует представиться, прежде чем включить шифрование, эти первые несколько байтов тоже будут видны всем.

(В этом отношении протоколы ведут себя по-разному. Например, HTTPS ничего не отправляет перед включением шифрования, а SMTP посылает несколько строк текста. О поведении протоколов мы поговорим чуть позже в этой главе.)

Когда мы наконец подключимся к сокету и обменяемся с ним рукопожатиями, чтобы включить шифрование, в дело вступает TLS, который предоставляет гарантии относительно собеседника и предлагает способы защитить от посторонних глаз данные, которыми вы будете обмениваться.

Сначала TLS-клиент попросит удаленный сервер предоставить двоичный документ, называемый *сертификатом* и включающий *открытый ключ* — целое число, которое можно использовать для шифрования данных. Расшифровать и прочитать данные сможет только владелец *закрытого ключа*, связанного с этим открытым ключом. Если удаленный сервер настроен правильно и не взломан, на нем будет копия

закрытого ключа. Ни у одного другого сервера в Интернете (не считая иные компьютеры в этом кластере) не будет этого ключа.

Как убедиться, что у удаленного сервера есть закрытый ключ в вашей реализации TLS? Очень просто! Библиотека TLS доставляет зашифрованные данные с открытым ключом и просит удаленный сервер выдать контрольную сумму, чтобы доказать, что данные были расшифрованы с помощью закрытого ключа.

TLS-стек должен учитывать возможность подделки удаленного сертификата. В конце концов, любой человек с доступом к инструменту командной строки OpenSSL (или аналогичному инструменту) может создать сертификат с общим именем `cn=www.google.com`, `cn=pypi.python.org` или любым другим. Можно ли ему верить? В качестве решения можно хранить в TLS-сеансе список доверенных *центров сертификации*, чтобы проверять подлинность хостов в Интернете. По умолчанию библиотека TLS в операционной системе или браузер используют несколько сотен сертификатов со всего мира от разных организаций, которые предоставляют верификацию сайтов.

Если параметры по умолчанию нам не нравятся или мы хотим выбрать частный центр сертификации, созданный нашей компанией для бесплатного подписания частных сертификатов, можно указать свой список центра сертификации. Это хороший выбор, когда мы не ожидаем подключения клиентов извне и хотим наладить взаимодействие только между внутренними сервисами.

Подпись — это знак, поставленный центром сертификации на сертификате, чтобы подтвердить, что он был одобрен.

Прежде чем признать законность сертификата, библиотека TLS сверит подпись с открытым ключом соответствующего сертификата центра сертификации.

TLS проверяет поля данных сертификата, убедившись, что тело сертификата было отправлено доверенной третьей стороне и подписано доверенной третьей стороной. Особенно важны два типа поля. Во-первых, это поля `notBefore` и `notAfter`, которые обозначают период действия сертификата, чтобы сертификаты, связанные с украденными закрытыми ключами, не действовали бесконечно. Поскольку стек TLS проверяет эти поля по системным часам, проблемы с часами могут помешать вам использовать TLS.

Во-вторых, общее имя сертификата должно соответствовать имени хоста, к которому мы подключаемся. В конце концов, если мы пытаемся подключиться к **<https://pypi.python.org>**, вряд ли нам понравится, что в ответ сайт присылает сертификат для другого имени хоста.

Один сертификат можно использовать для нескольких имен хоста. Современные сертификаты могут хранить дополнительные имена в поле `subjectAltName`, которые дополняют общее имя в поле `subject`. Кроме того, эти имена могут содержать подстановочные символы, например `*.python.org`, чтобы охватить несколько имен хоста. Современные алгоритмы TLS автоматически выполняют это сопоставление, и модуль `ssl` для Python тоже это умеет.

Наконец, агенты TLS клиента и сервера согласовывают общий секретный ключ и шифр для шифрования данных, которые будут передаваться по этому соединению. Это последняя точка, на которой может произойти сбой TLS, потому что правильно настроенное программное обеспечение отклонит шифр или ключ неподходящей длины. Сбой TLS может произойти на двух уровнях: версия TLS, которую хочет использовать одна из сторон соединения, устарела и не считается безопасной, или шифры одной из сторон недостаточно надежные, чтобы им можно было доверять.

Контроль передается обратно приложению на каждом конце, когда шифр согласован и обе стороны создали ключи для шифрования и подписывания каждого блока данных. Каждый передаваемый блок данных зашифрован с помощью ключа шифрования, и получившийся блок подписан с помощью ключа подписания, чтобы другая сторона была уверена, что он создан законным отправителем, а не злоумышленником в ходе атаки посредника. Данные могут свободно передаваться в обоих направлениях, как через обычный TCP-сокет, пока TLS не будет отключен, а сокет не будет закрыт или переведен в режим открытого текста.

Далее мы рассмотрим библиотеку `ssl` для Python, потому что она позволяет выполнять все эти задачи в коде. Узнать больше можно в официальных источниках, а также на сайтах и в книгах, например книгах Брюса Шнайера (Bruce Schneier), в блоге Google Online Security, блоге Адама Лэнгли (Adam Langley) и в других источниках. Я нахожу полезным выступление Хинека Шлавака (Hynek Schlawack), "Печальное состояние SSL" ("Sorry State Of SSL") на PyCon 2014. На момент, когда вы читаете эту книгу, могли появиться и более актуальные материалы по криптографии.

Создание сертификатов

Каталог `certs` также содержит различные сертификаты, которые используются в сети (см. главу 1), включая сертификаты, которые вы будете использовать в командной строке в примерах из этой главы. Все остальные сертификаты были подписаны сертификатом `ca.crt`. Это небольшой независимый центр сертификации, которому наш код Python будет доверять при использовании других сертификатов с TLS.

В общих чертах, создание сертификата обычно начинается с двух элементов данных: один создается человеком, а другой — машиной. Это текстовое описание сущности, указанной сертификатом, и закрытый ключ, который был создан с помощью генератора случайных значений в операционной системе.

Когда нужно вручную ввести описание идентификационных данных, я обычно сохраняю его на будущее в файл с контролем версий, но многие администраторы просто вводят информацию в OpenSSL. В листинге 6.1 приводится файл `www.cnf` для веб-сервера **www.example.com**, с помощью которого был создан сертификат.

Листинг 6.1. Конфигурация командной строки OpenSSL для сертификата X.509

```
[ req ]
prompt = no
```

```

distinguished_name = req_distinguished_name
[ req_distinguished_name ]
countryName = India
stateOrProvinceName = New Delhi
localityName = New Delhi
0.organizationName = Example from Bpbonline
organizationalUnitName = Network Programming in Python: The Basics
commonName = www.example.com
emailAddress = root@example.com
[ ssl_client ]
basicConstraints = CA:FALSE
nsCertType = client
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = clientAuth

```

Если помните, TLS сравнивает `commonName` и `subjectAltName` (в нашем примере альтернативных имен нет) с именем хоста, чтобы убедиться, что это нужный хост.

Эксперты спорят по поводу длины и типа закрытого ключа, который должен подкреплять сертификат. Одни администраторы выбирают RSA, другие — алгоритм Диффи — Хеллмана. Мы не будем вдаваться в подробности, а просто посмотрим пример команды для создания ключа RSA с длиной, которая сейчас считается допустимой:

```
$ openssl genrsa -out www.key 4096
```

Generating RSA private key, 4096 bit long modulus

```

.....
.....++
.....++
e is 65537 (0x10001)

```

Используя эти два компонента, администратор может создать запрос на подпись сертификата (certificate signing request, CSR) к собственному или стороннему центру сертификации.

```
$ openssl req -new -key www.key -config www.cnf -out www.csr
```

Для того чтобы понять, как OpenSSL создает частный центр сертификации и подписывает запрос на подпись сертификата, чтобы создать файл `www.crt`, соответствующий созданному ранее запросу, загляните в `Makefile`. Если мы обращаемся к общедоступному центру сертификации, мы можем получить файл `www.crt` по электронной почте (не волнуйтесь, этот сертификат и должен быть общедоступным). Когда подписанный сертификат будет готов, мы также сможем загрузить его в учетной записи на сайте выбранного центра сертификации. Наконец мы объединяем сертификат и секретный ключ в один файл, чтобы его было удобно использовать

в коде Python. Если файлы созданы в стандартном формате PEM, объединить их можно будет с помощью простой команды конкатенации UNIX.

```
$ cat www.crt www.key > www.pem
```

Готовый файл должен включать текстовое объяснение содержимого сертификата, сам сертификат и закрытый ключ. Будьте осторожны при работе с этим файлом. Если кто-то получит доступ к файлу `www.key` или `www.pem`, где хранится закрытый ключ, он сможет имитировать ваш сервис в течение нескольких месяцев или лет, пока срок действия ключа не истечет.

Существуют более сложные конфигурации, чем подписание сертификатов для сервера через центр сертификации напрямую. Некоторые компании, например, используют на серверах только сертификаты, которые действуют несколько дней или недель. Если кто-то взломает сервер и украдет закрытый ключ, ущерб будет не таким значительным. Вместо того чтобы платить центру сертификации за замену ключа каждые несколько дней, компания может использовать промежуточный сертификат с более длительным сроком действия. Закрытый ключ этого сертификата держится в секрете и используется для подписания видимых пользователям сертификатов, которые будут действовать на серверах. Получившаяся цепочка сертификатов, или цепочка доверия, сочетает в себе гибкость, т. к. мы сами подписываем новые сертификаты, когда захотим, и удобство, потому что мы используем общедоступный центр сертификации и нам не приходится устанавливать пользовательский сертификат центра сертификации в каждом браузере или клиенте, который хочет взаимодействовать с нашими сервисами. У клиентского программного обеспечения не будет проблем с подтверждением идентификации, если наши серверы с поддержкой TLS предоставляют собственный сертификат сервера и промежуточный сертификат, который связан с доверенным сертификатом центра сертификации.

Если вам поручили обеспечить шифрование с помощью ключей, ознакомьтесь с книгами и материалами по подписанию сертификатов.

TLS Offloading

Прежде чем мы посмотрим, как использовать TLS в коде Python, особенно для разработки сервера, нужно отметить, что многие эксперты советуют не торопиться с реализацией шифрования в приложении. В конце концов, существуют инструменты, которые применяют TLS к клиентским соединениям от нашего имени и доставляют незашифрованные данные приложению, если оно выполняется на другом порту.

Будет проще обновлять и настраивать отдельный демон или сервис, который предоставляет терминацию TLS для приложения Python, чем писать собственный код и использовать библиотеку OpenSSL. Более того, сторонние инструменты часто предлагают функции, которые мы пока не можем настраивать с помощью модуля `ssl` даже в Python 3.4. Например, криптография на эллиптических кривых ECDSA или настройка повторного согласования сеанса пока недоступны в модуле `ssl`. По-

вторное согласование требует особого внимания, потому что может значительно сократить потребление ресурсов центрального процессора при использовании TLS, но при неправильной настройке ставит под угрозу Perfect Forward Security (см. в разд. "Выбор шифров вручную и Perfect Forward Security" далее в этой главе). Почитайте статью про Perfect Forward Security "How to botch TLS forward secrecy" (<https://www.imperialviolet.org/2013/06/27/botchingpfs.html>), где очень доступно излагается эта концепция. Сторонние демоны, которые предоставляют терминацию TLS, включают фронтенд-серверы HTTPS. Поскольку стандарт HTTPS требует, чтобы клиент и сервер согласовали шифрование, прежде чем можно будет передавать по этому каналу другие сообщения, сторонний инструмент может взять на себя эти заботы. TLS может полностью исчезнуть из кода Python, если мы развертываем Apache, nginx или другой обратный прокси перед веб-сервисом Python в качестве дополнительного уровня защиты или подписываемся на сеть доставки содержимого, вроде Fastly, которая пересылает запросы на наши серверы.

Даже если вы создаете собственный сырой сокет, для которого нет сторонних инструментов, но хотите передать TLS другому инструменту, пролистайте эту главу, чтобы в общих чертах понимать тему, прежде чем углубиться в документацию по выбранному инструменту. Этот инструмент (а не код Python) будет загружать сертификат и закрытый ключ, и его нужно тщательно настроить, чтобы обеспечить надлежащую безопасность при использовании слабых шифров. Единственная проблема — как выбранный фронтенд будет сообщать вашему сервису Python удаленный IP-адрес и (если используются клиентские сертификаты) идентификатор каждого подключенного клиента. В HTTP-запросы можно добавить дополнительные заголовки, чтобы включить информацию о клиенте. Дополнительную информацию, например IP-адрес клиента, нужно будет добавить в начало входящего потока данных, если используются более примитивные технологии, например stunnel или HAProxy, которые не умеют полноценно обрабатывать HTTP. В любом случае этот инструмент будет предоставлять все возможности TLS, которых мы обсудим в оставшейся части этой главы, используя только сокеты Python.

Контексты по умолчанию в Python 3.4

Нам доступно несколько реализаций TLS с открытым кодом. Стандартная библиотека Python использует популярную библиотеку OpenSSL, которая считается лучшим вариантом для большинства систем и языков. Некоторые дистрибутивы Python включают свою версию OpenSSL, а другие просто обертывают OpenSSL, которая поставляется с операционной системой. `ssl` — это устаревшее обозначение модуля из стандартной библиотеки. В этой книге мы будем рассматривать `ssl`, но сообщество Python предлагает и другие криптографические проекты, например `pyOpenSSL`, который позволяет более детально работать с API базовой библиотеки. С Python 3.4 приложениям Python проще правильно использовать TLS благодаря добавлению функции `ssl.create_default_context()` (листинг 6.2). Это хорошая иллюстрация API с разумными параметрами по умолчанию, как требуется большинству

клиентов. Спасибо Кристиану Хаймсу (Christian Heimes) и Дональду Стафту (Donald Stufft) за концепцию контекста по умолчанию (default context) в стандартной библиотеке, а также за готовность принимать обратную связь. Они обещали не уничтожать обратную совместимость при выходе новых версий Python, поэтому различные процедуры, которые модуль `ssl` предлагает для установки TLS-соединений, продолжают работать с более старыми и менее безопасными параметрами по умолчанию. Однако если вы используете шифр TLS или длину ключа, которые сейчас считаются небезопасными, `create_default_context()` все же выдаст исключение при следующем обновлении Python.

Нарушая обещание о том, что Python можно обновлять, не меняя поведение приложений, `create_default_context()` выбирает поддерживаемые шифры, так что вам не нужно разбираться в TLS и читать статьи по безопасности, а достаточно просто следовать рекомендациям и поддерживать актуальность версии Python на своем компьютере. После каждого обновления тестируйте приложения, чтобы убедиться, что они по-прежнему могут взаимодействовать с клиентами или серверами через TLS.

Листинг 6.2. В Python 3.4 или более поздней версии: защищаем сокет с TLS для клиента и сервера

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Простой клиент и сервер TLS, использующие безопасные параметры по умолчанию.

import argparse, socket, ssl

def client(host, port, cafile=None):
    purpose = ssl.Purpose.SERVER_AUTH
    context = ssl.create_default_context(purpose, cafile=cafile)
    raw_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    raw_sock.connect((host, port))
    print('Connected to host {!r} and port {}'.format(host, port))
    ssl_sock = context.wrap_socket(raw_sock, server_hostname=host)
    while True:
        data = ssl_sock.recv(1024)
        if not data:
            break
        print(repr(data))

def server(host, port, certfile, cafile=None):
    purpose = ssl.Purpose.CLIENT_AUTH
    context = ssl.create_default_context(purpose, cafile=cafile)
    context.load_cert_chain(certfile)
```

```

listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
listener.bind((host, port))
listener.listen(1)
print('Listening at interface {!r} and port {}'.format(host, port))
raw_sock, address = listener.accept()
print('Connection from host {!r} and port {}'.format(*address))
ssl_sock = context.wrap_socket(raw_sock, server_side=True)
ssl_sock.sendall('Simple is better than complex.'.encode('ascii'))
ssl_sock.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Safe TLS client and server')
    parser.add_argument('host', help='hostname or IP address')
    parser.add_argument('port', type=int, help='TCP port number')
    parser.add_argument('-a', metavar='cafile', default=None,
                        help='authority: path to CA certificate PEM file')
    parser.add_argument('-s', metavar='certfile', default=None,
                        help='run as server: path to server PEM file')
    args = parser.parse_args()
    if args.s:
        server(args.host, args.port, args.s, args.a)
    else:
        client(args.host, args.port, args.a)

```

Как видите, привязать сокет можно всего за три шага. Для начала создаем объект контекста TLS, который содержит все параметры шифра и проверки сертификата. Затем используем метод `wrap_socket()` контекста, чтобы разрешить библиотеке OpenSSL взять под контроль TCP-соединение, обменяться рукопожатиями со второй стороной и установить зашифрованный канал. Наконец, используем возвращенный `ssl_sock` для последующих коммуникаций, чтобы слой TLS всегда шифровал данные перед передачей в сеть. Эта обертка предлагает все те же функции, что и обычный TCP-сокет, включая `send()`, `recv()` и `close()`, которые мы рассмотрели в *главе 3*.

Некоторые параметры в новом контексте зависят от того, создаем мы контекст для клиента, который пытается проверить сервер (`Purpose.SERVER_AUTH`), или сервер, который должен принимать клиентские соединения (`Purpose.CLIENT_AUTH`). У нас есть два отдельных набора параметров, потому что мы хотим, чтобы TLS-клиенты все-таки принимали старые шифры, ведь некоторые серверы могли устареть, и мы не можем на это повлиять. При этом для собственных серверов мы, конечно, должны использовать только современные и безопасные шифры. Па-

параметры `create_default_context()` могут зависеть от версии Python. Ниже приводятся примеры для Python 3.4.

- ◆ Поскольку `create_default_context()` устанавливает `PROTOCOL_SSLv23` в качестве протокола при создании нового объекта `SSLContext`, клиент и сервер должны будут согласовать используемую версию TLS.
- ◆ Клиент и сервер откажутся взаимодействовать через устаревшие протоколы `SSLv2` и `SSLv3` из-за задокументированных проблем в обеих версиях. Вместо этого они будут требовать у собеседника версию не ниже `TLSv1`. (Это значит, что мы не сможем взаимодействовать с Internet Explorer 6 в Windows XP, но это настолько устаревшая комбинация, что даже Microsoft ее больше не поддерживает.)
- ◆ Главное различие между параметрами клиента и сервера в том, что сжатие TLS отключено, потому что связано с рисками. Большинство TLS-соединений в Интернете — это общение клиента (например, стандартного веб-браузера) с сервером (PyPI, Google, сервер банка), у которого есть действительный подписанный сертификат.

Еще одно различие между клиентами и серверами — используемые шифры. Параметры клиента допускают более широкий набор шифров, включая устаревшее потоковое шифрование RC4. У сервера параметры более строгие, и предпочтение отдается более новым шифрам с поддержкой Perfect Forward Security (PFS), чтобы в случае компрометации ключа сервера (из-за действий злоумышленников или судебного предписания) невозможно было раскрыть предыдущее взаимодействие.

Узнать всю эту информацию несложно: нужно открыть `ssl.py` в стандартной библиотеке и посмотреть код функции `create_default_context()` и ее параметры. Вы можете сделать это сами, особенно если вышла новая версия Python и предыдущая информация устарела. Если вам интересно, код в `ssl.py` содержит список шифров для клиента и сервера, которые помечены как `_DEFAULT_CIPHERS` и `_RESTRICTED_SERVER_CIPHERS`. Для того чтобы узнать, что определяют значения в каждой строке, изучите последнюю документацию по OpenSSL.

При создании контекста в листинге 6.2 параметр `cafile` указывает, какому центру сертификации (ca, certificate authority) будет доверять скрипт при проверке удаленного сертификата. `create_default_context()` выполняет метод `load_default_certs()` нового контекста, прежде чем вернуть значение `None` — значение по умолчанию, если не указано ключевое слово `keyword`. Также можно проверить общедоступные веб-сайты и другие сервисы, которые приобрели сертификат у надежного публичного центра сертификации. Если `cafile` — это имя файла, а не строка, из операционной системы не импортируются никакие сертификаты, и только сертификаты центра сертификации из этого файла имеют право проверять вторую сторону TLS-соединения. (Если мы создаем контекст и для `cafile` задано `None`, а затем мы выполняем метод `load_verify_locations()`, чтобы установить дополнительные сертификаты, могут быть доступны оба типа сертификатов.)

Наконец, метод `wrap_socket()` в листинге 6.2 предоставляет два важных параметра: один для сервера и один для клиента. Поскольку одна из двух сторон должна пред-

полагать, что сервер может не выполнить обязательства или не пройти согласование, у сервера есть параметр `server side=True`. Клиентскому вызову требуется дополнительная информация: имя хоста, к которому мы подключились с помощью `connect()`, чтобы его можно было сравнить с полями `subject` в сертификате сервера. Если мы постоянно предоставляем ключевое слово имени хоста сервера для метода `wrapssocket()`, как в листинге 6.2, эта проверка выполняется автоматически.

Подходы к обертке сокетов

Все скрипты в этой главе показывают, как с помощью модуля `ssl` устанавливать TLS-соединение. Для этого мы создаем настроенный объект `SSLContext`, который описывает наши требования к безопасности, устанавливаем соединение между клиентом и сервером через обычный сокет, а затем вызываем метод `wrap_socket()` контекста, чтобы выполнить согласование TLS. Я использую этот паттерн во всех примерах, потому что это самый надежный, эффективный и универсальный способ применять API этого модуля. С помощью этого паттерна можно писать клиенты и серверы, которые будет легко читать, потому что мы используем единообразный подход, и код будет просто сравнивать с примерами и друг с другом. Модуль `ssl` в стандартной библиотеке содержит еще несколько функций, которые вы можете увидеть в других скриптах. Давайте рассмотрим эти функции и их недостатки.

Иногда код вызывает функцию `ssl.wrap_socket()`, предварительно не создав контекст. Особенно часто это встречается в старых скриптах, потому что это был единственный способ установить TLS-соединение до добавления объектов контекста в Python 3.2. У этого подхода есть несколько недостатков.

- ◆ Это неэффективно, потому что при каждом вызове создается новый объект контекста. Создавая и настраивая свой контекст, мы можем впоследствии использовать его много раз.
- ◆ Этому контексту не хватает гибкости. У него есть целых девять ключевых слов, чтобы его можно было хоть как-то настроить, но мы все равно не можем указать, например, какие шифры хотим использовать. Из-за обещания гарантировать обратную совместимость допускается использование очень слабых шифров.
- ◆ Наконец, никто не проверяет имена хостов, а это небезопасно. Мы не можем узнать, относится ли сертификат, полученный от другой стороны, к имени хоста, к которому мы якобы подключаемся, пока не выполним `match_hostname()` после "успешного" подключения.

Как видите, не стоит использовать `ssl.wrap_socket()` в новом коде и лучше заменить его в уже имеющихся программах. Следуйте рекомендациям в листинге 6.2. Еще один подход — обернуть сокет до подключения, будь то клиентский сокет, который выполняет `connect()`, или серверный сокет, который выполняет `accept()`. Обернутый сокет в любом случае не сможет сразу согласовать TLS-соединение, и ему придется ждать подключения. Очевидно, это работает только для протоколов, которые активируют TLS сразу после подключения, например HTTPS. Поскольку та-

ким протоколам, как SMTP, требуется открытый текст для начала взаимодействия, при обертывании указывается ключевое слово, которое требует рукопожатия при подключении. Мы можем указать `False` и отложить согласование TLS на потом, чтобы оно выполнялось с методом `do_handshake()`. Да, предварительное обертывание сокета само по себе не снижает безопасность, но я не рекомендую такой подход по нескольким причинам.

- ◆ Функция обертывания выполняется отдельно от согласования TLS, и тот, кто будет читать вызов `connect()` или `accept()` в коде, может не заметить, что протокол TLS вообще используется.
- ◆ Теперь методы `connect()` и `accept()` могут выдавать исключения, связанные не только с сокетом или DNS, но и с TLS, если при согласовании что-то пошло не так. Любая инструкция `try...except` вокруг этого вызова теперь должна будет учитывать два типа ошибок, поскольку в одном вызове метода будут выполняться две процедуры.
- ◆ Наконец, у нас появится объект `SSLSocket`, который может выполнять шифрование, а может и не делать этого. `SSLSocket` будет предоставлять шифрование только после установки соединения или когда явно вызывается функция `do_handshake()` (если автоматическое согласование отключено). Метод, который используется в листингах в этой книге, переходит к `SSLSocket`, только если включено шифрование, так что связь между классом текущего объекта сокета и статусом подключения имеет гораздо больше смысла.

Предварительное обертывание полезно лишь в одном случае: когда мы пытаемся использовать старую библиотеку, которая поддерживает только открытый текст. Мы можем добавить TLS к любому протоколу так, что он этого даже не заметит, предоставив предварительно обернутый сокет и установив для ключевого слова `do_handshake_on_connect` значение по умолчанию `True`. Это уникальная ситуация, в которой мы должны сообщить библиотеке о TLS и передать ей контекст TLS в качестве аргумента, если это вообще возможно.

Выбор шифров вручную и Perfect Forward Security

Если вас волнует безопасность данных, можно использовать функцию `create_default_context()`, чтобы указать, какие именно шифры может использовать OpenSSL, не полагаясь на параметры по умолчанию.

Криптография постоянно развивается, будут появляться проблемы и решения, о которых мы пока не знаем. На момент написания книги существует проблема, связанная с Perfect Forward Security (PFS): может ли кто-то, кто получает (или крадет) старый закрытый ключ, читать прошлые зашифрованные взаимодействия, которые он записал и сохранил для будущей дешифровки. Самые популярные современные шифры защищают от этого риска, шифруя каждый новый сокет с помощью временного ключа. Часто мы стараемся вручную указать характеристики объекта контекста именно затем, чтобы обеспечить свойство PFS.

Хотя контексты по умолчанию в модуле `ssl` не требуют шифр с поддержкой PFS, если клиент и сервер используют недавние версии OpenSSL, скорее всего, будет выбран именно такой шифр. Например, мы можем выполнить скрипт `safe_tls.py` в листинге 6.2 в режиме сервера и подключиться к нему с помощью скрипта `test_tls.py` из листинга 6.4. На своем ноутбуке со своей операционной системой я получил следующий результат:

```
$ python3.4 test_tls.py -a ca.crt localhost 1060
```

```
...
```

```
Cipher chosen for this connection... ECDHE-RSA-AES256-GCM-SHA384
```

```
Cipher defined in TLS version..... TLSv1/SSLv3
```

```
Cipher key has this many bits..... 256
```

```
Compression algorithm in use..... none
```

Python часто принимает умные решения без нашего участия, но если мы хотим убедиться, что будет использоваться определенная версия протокола или алгоритм, нужно просто ограничить контекст. Например, на момент написания этой книги следующий код считался хорошей конфигурацией сервера (если сервер не ожидает, что клиенты будут предоставлять TLS-сертификаты, и значит, может использовать `CERT_NONE` в качестве режима верификации):

```
context = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2)
```

```
context.verify_mode = ssl.CERT_NONE
```

```
context.options |= ssl.OP_CIPHER_SERVER_PREFERENCE # выбираем предпочитаемый шифр
```

```
context.options |= ssl.OP_NO_COMPRESSION           # защита от эксплойта CRIME
```

```
context.options |= ssl.OP_SINGLE_DH_USE            # для PFS
```

```
context.options |= ssl.OP_SINGLE_ECDH_USE          # для PFS
```

```
context.set_ciphers('ECDH+AES128 ')                # по мнению Шнайера, это лучше, чем AES256
```

Эти строки кода можно заменить в программе, вроде листинга 6.2, когда создается серверный сокет.

Для указания конкретной версии и шифра TLS использовалось всего несколько явных параметров. Если клиент пытается установить соединение, но не поддерживает эти параметры, он не сможет подключиться. Даже если у клиента будет чуть более старая версия TLS, например 1.1, или чуть более слабый шифр, например 3DES, установить соединение не получится, если вместо контекста по умолчанию добавить предыдущий код в листинг 6.3.

```
$ python3.4 test_tls.py -p TLSv1_1 -a ca.crt localhost 1060
```

```
Address we want to talk to..... ('localhost', 1060)
```

```
Traceback (most recent call last):
```

```
...
```

```
ssl.SSLError: [SSL: TLSV1_ALERT_PROTOCOL_VERSION] tlsv1 alert protocol
version (_ssl.c:598)
```

```
$ python3.4 test_tls.py -C 'ECDH+3DES' -a ca.crt localhost 1060
```

```
Address we want to talk to..... ('localhost', 1060)
```

```
Traceback (most recent call last):
```

...

```
ssl.SSLError: [SSL: SSLV3_ALERT_HANDSHAKE_FAILURE] sslv3 alert handshake failure
(_ssl.c:598)
```

В таких случаях сервер выдаст исключение Python, анализируя сбой со своей точки зрения. Если соединение будет установлено, данные будут защищены с помощью актуальной версии TLS (1.2) и одного из лучших доступных шифров. Однако при замене контекстов по умолчанию из модуля `ssl` на установленные вручную параметры возникает проблема: нам не только приходится самим изучать свои потребности и выбирать версию TLS и шифра при написании приложения, но и вовремя обновлять параметры, если будут обнаружены новые уязвимости. Если сочетать TLS 1.2 с алгоритмом Диффи — Хеллмана на эллиптических кривых, мы получим идеальный вариант, во всяком случае для этой книги. Правда, в будущем этот подход наверняка будет считаться устаревшим или даже небезопасным. Сможем ли мы вовремя заменить эти параметры во всех своих программах?

Пока `create_default_context()` не позволит нам настаивать на Perfect Forward Secrecy, у нас будет только два варианта: доверять контексту по умолчанию и соглашаться на то, что некоторые клиенты или серверы, с которыми мы взаимодействуем, не будут защищены с помощью PFS, или жестко кодировать шифр и регулярно обновлять его. Помните, что PFS не поможет, если мы не будем часто сбрасывать состояния сеанса сервера или ключ удостоверения сеанса. Достаточно перезапустить серверный процесс каждый вечер, чтобы получать новые ключи, но если у нас очень много серверов и мы хотим, чтобы они обслуживали пул TLS-клиентов, лучше поискать другой вариант. (Если мы хотим координировать перезапуск сеансов на целом кластере серверов без риска для PFS, возможно, стоит рассмотреть другие инструменты для терминации TLS, кроме Python.) Наконец, если мы разрабатываем или конфигурируем и клиента, и сервер (например, если мы настраиваем зашифрованные коммуникации в своем центре обработки данных или между своими серверами), будет гораздо проще задать фиксированные шифры. Когда мы взаимодействуем с другими программами, негибкий набор шифров мешает другим клиентам обращаться к нашим сервисам, особенно если их инструменты используют другие реализации TLS. Если мы указываем всего несколько вариантов, нужно явно объяснить их тем, кто создает и настраивает клиенты, чтобы они понимали, почему более старые клиенты не могут установить соединение.

Поддержка протокола TLS

Поддержка TLS реализована в большинстве популярных интернет-протоколов. При их использовании через стандартную библиотеку Python или сторонние библиотеки нужно обращать внимание на то, как указать параметры и шифр TLS, чтобы запретить подключение с использованием старых версий протокола, слабых шифров или небезопасных функций вроде сжатия. Для этого можно использовать определенные вызовы API или просто отправлять объект `SSLContext` с параметрами конфигурации.

Стандартная библиотека Python включает следующие протоколы с поддержкой TLS.

- ◆ **http.client.** Мы можем использовать ключевое слово `context` конструктора, чтобы передать `SSLContext` с собственными параметрами при создании объекта `HTTPSConnection` (см. главу 9). К сожалению, `urllib.request` и библиотека `Requests`, описанная в главе 9, не принимают аргумент `SSLContext` в своих API.
- ◆ **smtplib.** Мы можем использовать ключевое слово `context` конструктора, чтобы передать `SSLContext` с собственными параметрами при создании объекта `SMTP_SSL` (см. главу 13). Если мы создадим простой объект `SMTP` и вызовем его метод `starttls()` позже, вызову метода будет передан параметр контекста.
- ◆ **poplib.** Мы можем использовать ключевое слово `context` в функции `Object()` { [код] } объекта `POP3_SSL` (см. главу 14), чтобы передать `SSLContext` с нашими параметрами. Если мы создадим обычный объект `POP3` и вызовем его метод `stls()` позже, вызову метода будет передан параметр контекста.
- ◆ **imaplib.** Мы можем использовать ключевое слово `ssl context` в функции `Object()` { [код] } объекта `IMAP4_SSL` (см. главу 15), чтобы передать `SSLContext` с нашими параметрами. Если мы создадим простой объект `IMAP4` и вызовем его метод `starttls()` позже, вызову метода будет передан параметр контекста `ssl`.
- ◆ **ftplib.** Мы можем использовать ключевое слово `context` в функции `Object()` { [код] } объекта `FTP_TLS` (см. главу 17), чтобы передать `SSLContext` с нашими параметрами. Прежде чем включить шифрование, стороны обмениваются открытым текстом по FTP (например, приветственное сообщение 220, которое часто раскрывает имя хоста сервера). Прежде чем метод `login()` предоставит имя пользователя и пароль, FTP-объект `TLS` автоматически включит шифрование. Если мы не входим на удаленный сервер, но все равно хотим включить шифрование, нужно вручную выполнить метод `auth()` как первое действие после подключения.
- ◆ **nnntplib.** В этой книге мы не будем рассматривать протокол NNTP (Usenet), но просто знайте, что его тоже можно защитить. При создании NNTP SSL мы можем использовать ключевое слово `ssl context` в функции `Object()` { [код] }, чтобы передать `SSLContext` с нашими параметрами. Если мы создадим простой объект NNTP и вызовем его метод `starttls()` позже, вызову метода будет передан параметр контекста.

Стоит отметить, что между всеми этими протоколами есть кое-что общее: мы можем использовать TLS для расширения более старого стандарта, использующего открытый текст, одним из двух способов. Первый способ — ввести новую команду, которая позволяет перейти на TLS во время взаимодействия. Второй способ — назначить второй определенный номер порта TCP для соединений, защищенных TLS, и в этом случае согласование TLS будет происходить автоматически, без запроса. Большинство из рассмотренных выше протоколов предлагают оба способа, но HTTP поддерживает только второй, потому что не хранит состояние.

Если мы подключаемся к серверу, который поддерживает версию TLS одного из предыдущих протоколов, этот сервер настроен другой командой или организацией

и у нас нет документации, нужно провести тесты, чтобы узнать, открыт ли специальный порт для TLS или поддерживается только переход на TLS поверх протокола с открытым текстом. Если вы используете для сетевых соединений не стандартную библиотеку, а сторонний пакет, изучите его документацию, чтобы узнать, как с его помощью предоставлять объект `SSLContext`. Если вы не предоставили никакой механизм, а на момент написания книги даже популярные сторонние библиотеки для Python 3.4 не всегда предлагали такую возможность, придется экспериментировать с доступными параметрами и тестировать результаты (возможно, с помощью кода в листинге 6.4 из следующего раздела), чтобы проверить, гарантирует ли сторонняя библиотека достаточно надежный протокол и шифр, чтобы обеспечить конфиденциальность данных.

Дальнейшее изучение

В листинге 6.3 приводится скрипт Python 3.4, который создает зашифрованное соединение и сообщает о его характеристиках, чтобы мы узнали о версии протокола TLS и вариантах шифра, доступных клиентам и серверам. Для этого в листинге используются новые возможности объекта `SSLSocket` из модуля `ssl` стандартной библиотеки, которые позволяют скриптам Python проверять статус соединений с использованием `OpenSSL` и узнавать их параметры.

Ниже перечислены используемые функции.

- ◆ `getpeercert()` возвращает список сертификатов сторон соединения, который представляет собой словарь Python с атрибутами из сертификата X.509 стороны, с которой установлен сеанс TLS. Эта возможность `SSLSocket` доступна уже давно и во многих предыдущих версиях Python. В последующих версиях Python появилось больше возможностей, связанных с сертификатами.
- ◆ `cypher()` возвращает имя шифра, который в итоге был согласован между объектом `OpenSSL` и реализацией TLS на той стороне, а значит, используется для текущего соединения.
- ◆ `compression()` возвращает применяемый алгоритм сжатия или `None`, если алгоритм сжатия не используется.

Скрипт в листинге 6.3 также пытается использовать `ctypes`, чтобы понять применяемый протокол TLS (в идеале это будет нативная возможность модуля `ssl` начиная с версии Python 3.5) и сообщить информацию о нем. Листинг 6.3 позволяет нам подключиться к клиенту или серверу, которые мы создали, и понять, какие шифры и протоколы они будут или не будут согласовывать.

Листинг 6.3. Подключение к любой конечной точке через TLS и информирование о согласованных шифрах

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Код пытается установить TLS-соединение и, если получилось, сообщить о его свойствах.
```

```
import argparse, socket, ssl, sys, textwrap
import ctypes
from pprint import pprint

def open_tls(context, address, server=False):
    raw_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    if server:
        raw_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        raw_sock.bind(address)
        raw_sock.listen(1)
        say('Interface where we are listening', address)
        raw_client_sock, address = raw_sock.accept()
        say('Client has connected from address', address)
        return context.wrap_socket(raw_client_sock, server_side=True)
    else:
        say('Address we want to talk to', address)
        raw_sock.connect(address)
        return context.wrap_socket(raw_sock)

def describe(ssl_sock, hostname, server=False, debug=False):
    cert = ssl_sock.getpeercert()
    if cert is None:
        say('Peer certificate', 'none')
    else:
        say('Peer certificate', 'provided')
        subject = cert.get('subject', [])
        names = [name for names in subject for (key, name) in names
                  if key == 'commonName']
        if 'subjectAltName' in cert:
            names.extend(name for (key, name) in cert['subjectAltName']
                          if key == 'DNS')
        say('Name(s) on peer certificate', *names or ['none'])
        if (not server) and names:
            try:
                ssl.match_hostname(cert, hostname)
            except ssl.CertificateError as e:
                message = str(e)
            else:
                message = 'Yes'
            say('Whether name(s) match the hostname', message)
        for category, count in sorted(context.cert_store_stats().items()):
            say('Certificates loaded of type {}'.format(category), count)
```

```

try:
    protocol_version = SSL_get_version(ssl_sock)
except Exception:
    if debug:
        raise
else:
    say('Protocol version negotiated', protocol_version)

cipher, version, bits = ssl_sock.cipher()
compression = ssl_sock.compression()

say('Cipher chosen for this connection', cipher)
say('Cipher defined in TLS version', version)
say('Cipher key has this many bits', bits)
say('Compression algorithm in use', compression or 'none')

return cert
    
```

```

class PySSLSocket(ctypes.Structure):
    """Первые несколько полей PySSLSocket (см. Modules/_ssl.c)."""

    _fields_ = [('ob_refcnt', ctypes.c_ulong), ('ob_type', ctypes.c_void_p),
                ('Socket', ctypes.c_void_p), ('ssl', ctypes.c_void_p)]

def SSL_get_version(ssl_sock):
    """Обращение к версии протокола TLS сокета."""
    lib = ctypes.CDLL(ssl._ssl. file)
    lib.SSL_get_version.restype = ctypes.c_char_p
    address = id(ssl_sock._sslobj)
    struct = ctypes.cast(address, ctypes.POINTER(PySSLSocket)).contents
    version_bytestring = lib.SSL_get_version(struct.ssl)
    return version_bytestring.decode('ascii')

def lookup(prefix, name):
    if not name.startswith(prefix):
        name = prefix + name
    try:
        return getattr(ssl, name)
    except AttributeError:
        matching_names = (s for s in dir(ssl) if s.startswith(prefix))
        message = 'Error: {!r} is not one of the available names:\n {}'.format(
    
```

```
        name, ' '.join(sorted(matching_names)))
    print(fill(message), file=sys.stderr)
    sys.exit(2)
```

```
def say(title, *words):
    print(fill(title.ljust(36, '.') + ' ' + ' '.join(str(w) for w in words)))
```

```
def fill(text):
    return textwrap.fill(text, subsequent_indent=' ',
                          break_long_words=False, break_on_hyphens=False)
```

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Protect a socket with TLS')
    parser.add_argument('host', help='hostname or IP address')
    parser.add_argument('port', type=int, help='TCP port number')
    parser.add_argument('-a', metavar='cafile', default=None,
                        help='authority: path to CA certificate PEM file')
    parser.add_argument('-c', metavar='certfile', default=None,
                        help='path to PEM file with client certificate')
    parser.add_argument('-C', metavar='ciphers', default='ALL',
                        help='list of ciphers, formatted per OpenSSL')
    parser.add_argument('-p', metavar='PROTOCOL', default='SSLv23',
                        help='protocol version (default: "SSLv23")')
    parser.add_argument('-s', metavar='certfile', default=None,
                        help='run as server: path to certificate PEM file')
    parser.add_argument('-d', action='store_true', default=False,
                        help='debug mode: do not hide "ctypes" exceptions')
    parser.add_argument('-v', action='store_true', default=False,
                        help='verbose: print out remote certificate')
    args = parser.parse_args()
    address = (args.host, args.port)
    protocol = lookup('PROTOCOL_', args.p)
    context = ssl.SSLContext(protocol)
    context.set_ciphers(args.C)
    context.check_hostname = False
    if (args.s is not None) and (args.c is not None):
        parser.error('you cannot specify both -c and -s')
    elif args.s is not None:
        context.verify_mode = ssl.CERT_OPTIONAL
        purpose = ssl.Purpose.CLIENT_AUTH
        context.load_cert_chain(args.s)
```

```

else:
    context.verify_mode = ssl.CERT_REQUIRED
    purpose = ssl.Purpose.SERVER_AUTH
    if args.c is not None:
        context.load_cert_chain(args.c)
if args.a is None:
    context.load_default_certs(purpose)
else:
    context.load_verify_locations(args.a)

print()
ssl_sock = open_tls(context, address, args.s)
cert = describe(ssl_sock, args.host, args.s, args.d)
print()
if args.v:
    pprint(cert)
    
```

Можно просто вызвать справку с помощью `-h`, чтобы узнать о параметрах командной строки для этого инструмента. Он пытается предоставить все основные возможности `SSLContext` через параметры командной строки, чтобы мы могли поэкспериментировать с ними и посмотреть, как они влияют на согласование. Мы видим, что при использовании `create_default_context()` параметры сервера строже, чем параметры клиента. Запустите скрипт из листинга 6.2 как сервер в окне терминала. Предполагается, что у вас уже есть файлы сертификата `ca.crt` и `localhost.pem` из каталога `chapter06` репозитория к этой книге.

```
$ /usr/bin/python3.4 safe_tls.py -s localhost.pem '' 1060
```

Сервер принимает соединения, используя самые современные версии протокола и шифра. По возможности он согласует надежную конфигурацию с использованием Perfect Forward Security. В листинге 6.3 мы видим, что будет, если использовать только параметры Python по умолчанию:

```
$ /usr/bin/python3.4 test_tls.py -a ca.crt localhost 1060
```

```

Address we want to talk to..... ('localhost', 1060)
Peer certificate..... provided
Name(s) on peer certificate..... localhost
Whether name(s) match the hostname.. Yes
Certificates loaded of type crl..... 0
Certificates loaded of type x509.... 1
Certificates loaded of type x509_ca. 0
Protocol version negotiated..... TLSv1.2
Cipher chosen for this connection... ECDHE-RSA-AES128-GCM-SHA256
Cipher defined in TLS version..... TLSv1/SSLv3
    
```

```
Cipher key has this many bits..... 128
Compression algorithm in use..... none
```

Комбинация ECDHE-RSA-AES128-GCM-SHA256 — это один из лучших вариантов, которые OpenSSL может сейчас предложить. Сервер `safe_tls.py` откажется общаться с клиентом, который поддерживает только шифрование на уровне Windows XP. Перезапустим сервер `safe_tls.py` и на этот раз выполним подключение со следующими аргументами:

```
$ /usr/bin/python3.4 test_tls.py -p SSLv3 -a ca.crt localhost 1060
```

```
Address we want to talk to..... ('localhost', 1060)
```

```
Traceback (most recent call last):
```

```
...
```

```
ssl.SSLError: [SSL: SSLV3_ALERT_HANDSHAKE_FAILURE] sslv3 alert handshake failure
(_ssl.c:598)
```

Строгие параметры сервера, предложенные Python, отвергают устаревший протокол SSLv3. Даже при использовании с современными протоколами старые шифры, вроде RC4, не будут приняты.

```
test_tls.py /usr/bin/python3.4 -
```

```
localhost 1060 C 'RC4' -a ca.crt
```

```
('localhost', 1060) is the address we wish to talk to.
```

```
(Last call) Traceback (most recent call):
```

```
...
```

```
ssl.SSLError: [SSL: SSLV3_ALERT_HANDSHAKE_FAILURE] sslv3 alert handshake failure
(_ssl.c:598)
```

Однако если тот же скрипт запустить как клиент, его поведение серьезно изменится. Мы уже говорили, что сервер должен решать, насколько защищенным должно быть соединение, а авторы клиента просто хотят, чтобы все работало, лишь бы при этом не пришлось полностью оставить данные без защиты. Как вы помните, безопасный сервер отказался использовать шифр RC4. Давайте посмотрим, что будет, если мы используем клиента `safe_tls.py` с тем же RC4. Для начала завершим работу всех серверов и выполним тестовый скрипт как сервер, задав для шифра -C.

```
$ /usr/bin/python3.4 test_tls.py -C 'RC4' -s localhost.pem '' 1060
```

```
Interface where we are listening.... ('', 1060)
```

Затем попробуем подключиться с помощью скрипта `safe_tls.py`, который использует контекст по умолчанию из Python 3.4, в новом окне терминала.

```
$ /usr/bin/python3.4 safe_tls.py -a ca.crt localhost 1060
```

Соединение успешно установлено, хотя мы используем защищенный контекст по умолчанию. Мы видим, что RC4 был выбран как шифр потоковой передачи в окне сервера. Мы можем проверить, что RC4 — это минимальный принимаемый шифр, используя параметр -C с разными строками. Такие шифры и алгоритмы, как MD5, будут отвергнуты, потому что не подходят для клиента, который пытается обеспечить максимальную совместимость с любым сервером, с которым пользователь захочет связаться.

Для того чтобы узнать больше о создании пользовательских протоколов и шифров, изучите документацию по модулю `ssl` и официальную документацию к `OpenSSL`. Если ваша система это поддерживает, можно использовать нативную командную строку `OpenSSL`, чтобы вывести все шифры, которые соответствуют заданной строке шифров. Это тот же текст, который можно было предоставить в листинге 6.3 с параметром `-C` или указать с методом `set_cipher()` в коде. По мере развития `OpenSSL` можно будет с помощью командной строки оценивать, как меняется влияние разных правил шифров. Вот шифры, которые соответствуют строке `ECDH+AES128` на моем ноутбуке с `Ubuntu` на момент написания книги:

```
$ openssl ciphers -v 'ECDH+AES128'
```

```

ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH      Au=RSA  Enc=AESGCM(128) Mac=AEAD
ECDHE-ECDSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH      Au=ECDSA Enc=AESGCM(128) Mac=AEAD
ECDHE-RSA-AES128-SHA256      TLSv1.2 Kx=ECDH      Au=RSA  Enc=AES(128)
Mac=SHA256
ECDHE-ECDSA-AES128-SHA256    TLSv1.2 Kx=ECDH      Au=ECDSA Enc=AES(128)
Mac=SHA256
ECDHE-RSA-AES128-SHA         SSLv3   Kx=ECDH      Au=RSA  Enc=AES(128)   Mac=SHA1
ECDHE-ECDSA-AES128-SHA       SSLv3   Kx=ECDH      Au=ECDSA Enc=AES(128)   Mac=SHA1
AECDH-AES128-SHA             SSLv3   Kx=ECDH      Au=None  Enc=AES(128)   Mac=SHA1
ECDH-RSA-AES128-GCM-SHA256   TLSv1.2 Kx=ECDH/RSA  Au=ECDH  Enc=AESGCM(128) Mac=AEAD
ECDH-ECDSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH/ECDSA Au=ECDH  Enc=AESGCM(128) Mac=AEAD
ECDH-RSA-AES128-SHA256      TLSv1.2 Kx=ECDH/RSA  Au=ECDH  Enc=AES(128)
Mac=SHA256
ECDH-ECDSA-AES128-SHA256    TLSv1.2 Kx=ECDH/ECDSA Au=ECDH  Enc=AES(128)
Mac=SHA256
ECDH-RSA-AES128-SHA         SSLv3   Kx=ECDH/RSA  Au=ECDH  Enc=AES(128)   Mac=SHA1
ECDH-ECDSA-AES128-SHA       SSLv3   Kx=ECDH/ECDSA Au=ECDH  Enc=AES(128)   Mac=SHA1
    
```

Если мы укажем `set_cipher('ECDH+AES128')`, библиотека `OpenSSL` будет принимать все эти комбинации. По возможности следует использовать контекст по умолчанию. В противном случае протестируйте конкретного клиента и сервер, которые планируете использовать, и попробуйте выбрать один или два надежных шифра, которые будут поддерживаться обеими сторонами. Если для этого придется проделывать слишком много работы, листинг 6.3 поможет настроить поведение `OpenSSL`. Загрузите новую версию листинга 6.3 по приведенной в начале книги ссылке, потому что версия, указанная в этой книге, могла устареть.

Резюме

В этой главе мы рассматривали тему, в которой разбирается не так много людей: использование криптографии для защиты данных при передаче через `TCP`-сокеты и реализация протокола `TLS` (ранее — `SSL`) в коде `Python`.

Обычно при установке `TLS`-соединения клиент запрашивает у сервера сертификат — цифровой документ, который подтверждает подлинность. Сертификат дол-

жен быть подписан центром, которому доверяют клиент и сервер, и включать общедоступный ключ, который сервер обязан предъявить. Клиент должен проверить, совпадает ли идентификационная информация в сертификате с именем хоста, к которому он, по его мнению, подключается. Наконец, клиент и сервер должны согласовать параметры шифра, сжатия и ключа, которые будут использоваться для защиты данных, передаваемых через сокет в обоих направлениях.

Многие администраторы даже не задумываются об использовании TLS в своих приложениях, а просто скрывают приложения за надежными прокси, вроде Apache, nginx или HAProxy, которые обрабатывают TLS. Сервисы за сетями доставки содержимого тоже могут не реализовывать TLS сами.

Для реализации TLS в Python существуют сторонние библиотеки, но можно использовать модуль `ssl` на основе OpenSSL в стандартной библиотеке. Простейшие зашифрованные каналы можно создавать посредством сертификата сервера, если `ssl` корректно работает в вашей операционной системе и версии Python.

Для этого мы создаем объект контекста, открываем соединение, а затем вызываем метод `wrap_socket()` контекста, чтобы передать контроль над соединением протоколу TLS. Приложения, написанные на Python 3.4 и более новых версий (я советую брать версию не ниже 3.4, если приложение должно само реализовывать TLS), обычно следуют этому паттерну: они создают объект контекста, открывают соединение и вызывают метод `wrap_socket()`. Это самый универсальный подход несмотря на то, что модуль `ssl` предлагает несколько сокращенных функций, которые могут встречаться в более старых программах.

Многие клиенты и серверы Python просто принимают объект контекста по умолчанию, возвращенный функцией `ssl.create_default_context()`. В нем предусмотрены более строгие параметры для сервера и менее строгие — для клиентов, чтобы они могли подключаться к серверам с более старыми версиями TLS. Некоторые приложения Python создают собственные объекты `SSLContext`, чтобы настроить протокол и шифр по своим потребностям. В любом случае можно использовать тестовый скрипт из этой главы или другой инструмент TLS, чтобы изучить, как разные параметры влияют на поведение. Стандартная библиотека поддерживает несколько протоколов, которые при желании можно защитить с помощью TLS, и большинство из них мы рассмотрим в следующих главах книги. Все они будут работать с объектом `SSLContext`. Поскольку кое-где еще используется Python 2, некоторые сторонние библиотеки могут некорректно поддерживать контексты. Со временем эта ситуация улучшится.

Если вы реализовали TLS в приложении, протестируйте его на разных соединениях с разными параметрами. Существуют сторонние инструменты и сайты для тестирования клиентов и серверов TLS. Инструменты, описанные в листинге 6.3, можно использовать с Python 3.4 напрямую в системе, если вы хотите поэкспериментировать с разными параметрами OpenSSL и увидеть, как происходит согласование.

Архитектура сервера

При создании сетевого сервиса мы должны выполнить две главные задачи. Во-первых, нужно создать код, который будет корректно реагировать на входящие запросы и генерировать допустимые ответы. Во-вторых, мы должны внедрить сетевой код в сервис Windows или демон UNIX, который будет автоматически запускаться при загрузке системы, вести журнал своих действий в постоянном хранилище, выдавать оповещения, если не удастся подключиться к базе данных или хранилищу и либо полностью защищаться от всех возможных сбоев, либо быстро перезапускаться в случае сбоя.

В этой главе мы рассмотрим первую из этих сложностей. Вторую тему мы обсуждать не будем, потому что, во-первых, ей можно посвятить отдельную книгу, а во-вторых, она не связана с сетевым программированием. Мы рассмотрим развертывание только в одной части этой главы, а затем перейдем к обсуждению главной темы — созданию программного обеспечения для сетевых серверов.

Сетевые серверы можно разделить на три категории. Мы начнем с простого однопоточного сервера, похожего на серверы UDP и TCP, о которых мы говорили в *главах 2 и 3*, и рассмотрим его недостатки. Например, он может обслуживать только одного клиента за раз, так что остальным клиентам приходится ждать, при этом центральный процессор практически бездействует даже во время взаимодействия с этим клиентом. Далее мы поговорим о двух других решениях: о дублировании однопоточного сервера на несколько потоков или процессов и о самостоятельной реализации мультиплексирования с помощью асинхронных операций.

Содержание главы

- ◆ Несколько слов о развертывании.
- ◆ Базовый протокол.
- ◆ Однопоточный сервер.
- ◆ Многопроцессорный и многопоточный серверы.
- ◆ Фреймворк SocketServer из прошлого.
- ◆ Асинхронные серверы.
- ◆ Фреймворк `asyncio` с обратными вызовами.

- ◆ Фреймворк `asyncio` с сопрограммами.
- ◆ Устаревший модуль `asyncore`.
- ◆ Комбинированный подход.
- ◆ Под влиянием `inetd`.
- ◆ Резюме.

Цель

Сначала мы с нуля реализуем каждый паттерн, изучая потоковое и асинхронное сетевое программирование, а затем рассмотрим фреймворки для удобной реализации тех же паттернов. Мы будем работать с фреймворками из стандартной библиотеки Python, но если есть хорошие сторонние решения, я о них упомяну. Большую часть скриптов в этой главе можно выполнять на Python 2, но, например, модуль `asyncio` доступен только в Python 3, и его можно считать стандартом.

Несколько слов о развертывании

Сетевой сервис можно развернуть на одном компьютере или группе компьютеров. Для того чтобы подключиться к сервису, размещенному на одном компьютере, клиенты могут указать его IP-адрес. Если сервис выполняется на нескольких компьютерах, потребуется более сложная стратегия. Мы, конечно, можем дать каждому клиенту адрес и имя хоста одного экземпляра сервиса, если он работает в том же центре обработки данных, что и клиент, но этот подход плохо масштабируется. Клиенты, жестко привязанные к имени хоста и IP-адресу, не смогут установить подключение при сбое сервиса.

Если мы обращаемся к сервису по имени, лучше настроить DNS-сервер, который будет сообщать все IP-адреса, по которым расположен сервис, и написать клиентов таким образом, чтобы они переходили на второй или третий IP-адрес в случае сбоя первого.

Самый эффективный современный подход — разместить сервисы за балансировщиком нагрузки, чтобы клиенты подключались к нему напрямую, а он перенаправлял входящие подключения на сервер. В случае сбоя сервера балансировщик нагрузки просто перестанет отправлять ему запросы, пока он не восстановится, так что клиенты даже не заметят, что сервер не работал. Крупнейшие сервисы в Интернете сочетают эти подходы: в каждом центре обработки данных есть балансировщик нагрузки и ферма серверов, а также общедоступное DNS-имя, которое предоставляет IP-адреса балансировщика нагрузки, расположенного ближе к вам.

Какой бы простой или сложной ни была архитектура сервиса, вам нужен механизм для развертывания серверного кода Python на физический или виртуальный компьютер. Есть два основных подхода к развертыванию. При традиционном подходе мы включаем все возможности сервиса в каждую серверную программу, которую

мы пишем: двойной fork для использования в качестве демона UNIX (или регистрация в качестве сервиса Windows), журналирование на уровне системы, файл конфигурации и способ запустить, остановить и перезапустить программу. Для этого можно использовать стороннюю библиотеку или написать свой код.

Затем появились новые методологии, например приложение двенадцати факторов (The Twelve-Factor App). Его разработчики пропагандируют минималистский подход: каждый сервис пишется как обычная программа, которая выполняется на переднем плане и не пытается стать демоном. Вместо файла конфигурации, общего для всей системы, программа получает необходимые параметры конфигурации из своего окружения (словарь `sys.envirop` в Python). Программа подключается к бэкенд-сервисам, указанным окружением. Также она выводит сообщения журнала на экран, используя простую функцию `print()` в Python. Она ожидает сетевые запросы по любому порту, который рекомендован в настройках окружения.

Разработчики могут легко тестировать сервис, созданный с помощью этого простого подхода, запуская его из командной строки. При необходимости с помощью скаффолдинга можно превратить приложение в демон или системный сервис или развернуть на ферме серверов. Например, скаффолдинг может получать переменные окружения из центрального сервиса конфигурации, выводить потоки вывода и ошибок на удаленный сервер журналирования и перезапускать сервис, если он отключился или завис. Мы можем быть уверены, что код сервиса будет вести себя в рабочем окружении так же, как при разработке, потому что сама программа просто возвращает результат в стандартный поток вывода.

Провайдеры решений "платформа как услуга" размещают за вас эти приложения, запуская десятки и даже сотни его экземпляров за одним общедоступным доменным именем и балансировщиком нагрузки TCP, а затем собирая все журналы для анализа. В некоторые из этих решений можно загружать код приложения Python напрямую. Для других необходимо упаковать код, интерпретатор Python и другие зависимости в контейнер (например, Docker), который можно протестировать на ноутбуке перед развертыванием, и в рабочем окружении код будет выполняться из образа, полностью идентичного тому, который мы использовали при тестировании. В любом случае нам не приходится разрабатывать сервис, который запускает множество процессов. Платформа обрабатывает избыточность и дубликацию за нас.

В сообществе Python давно предпринимаются попытки прекратить создание отдельных сервисов. Хороший пример — популярная утилита `supervisord`. Она может выполнять несколько копий программы, перенаправлять стандартные потоки вывода и ошибок в файлы журналов, перезапускать процессы в случае сбоя и даже выдавать оповещения, если сбои происходят слишком часто.

Если вы все же несмотря ни на что решите создать процесс, который умеет превращаться в демон, поищите разумные способы сделать это в сообществе Python. Например, в PEP 3143 (<http://python.org>) в разделе "Другие реализации демонов" ("Other daemon implementations") приводится подборка ресурсов и инструкций. Также изучите код `supervisord` и документацию для модуля `logging` в стандартной библиотеке Python.

При использовании сетевого стека операционной системы с процессом операционной системы с целью обслуживать сетевые запросы мы сталкиваемся с одинаковыми сложностями независимо от того, работаем мы с отдельным процессом Python или масштабным сервисом на основе платформы. В оставшейся части этой главы мы будем обсуждать, как решить эти проблемы, чтобы при этом система оставалась максимально производительной и клиентам не приходилось долго ждать ответы на сетевые запросы.

Базовый протокол

В примерах в этой главе используется простой протокол TCP, в котором клиент отправляет один из трех запросов открытым текстом в кодировке ASCII, а затем ждет ответ от сервера. Мы рассмотрим несколько вариантов серверов. Клиент может отправлять сколько угодно запросов, пока сокет открыт, а затем закроем соединение без предупреждения, когда запросы закончатся, как при использовании HTTP. Вопросительный знак в кодировке ASCII будет обозначать конец запроса.

Beautiful is better than?

(Красивое лучше, чем?) Затем возвращается ответ с точкой в конце.

Ugly.

(Уродливое.) Каждая пара вопросов и ответов представляет собой правило из манифеста "Дзена Python", где описывается философия Python. Советую перечитывать этот манифест, когда требуется вдохновение. Вызвать его можно командой `import this`.

В листинге 7.1 определено несколько программ для создания клиента и нескольких серверов на основе этого протокола, у которого, как мы увидим, нет собственного интерфейса командной строки. Этот модуль существует исключительно для того, чтобы можно было импортировать его в следующих листингах и не пришлось повторять код.

Листинг 7.1. Данные и программы для протокола Zen-of-Python

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Константы и программы для поддержания взаимодействия по сети.

import argparse, socket, time

aphorisms = {b'Beautiful is better than?': b'Ugly.',
              b'Explicit is better than?': b'Implicit.',
              b'Simple is better than?': b'Complex.'}

def get_answer(aphorism):
    """Возвращает ответ, чтобы продолжить афоризм из "Дзен Python"."""
```

```
time.sleep(0.0)      # увеличение для имитации дорогостоящей операции
return aphorisms.get(aphorism, b'Error: unknown aphorism.')
```

```
def parse_command_line(description):
```

```
    """Анализирует командную строку и возвращает адрес сокета."""
    parser = argparse.ArgumentParser(description=description)
    parser.add_argument('host', help='IP or hostname')
    parser.add_argument('-p', metavar='port', type=int, default=1060,
                        help='TCP port (default 1060)')
    args = parser.parse_args()
    address = (args.host, args.p)
    return address
```

```
def create_srv_socket(address):
```

```
    """Создает и возвращает слушающий сокет сервера."""
    listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    listener.bind(address)
    listener.listen(64)
    print('Listening at {}'.format(address))
    return listener
```

```
def accept_connections_forever(listener):
```

```
    """Неопределенное время отвечает на входящие соединения по слушающему сокету."""
    while True:
        sock, address = listener.accept()
        print('Accepted connection from {}'.format(address))
        handle_conversation(sock, address)
```

```
def handle_conversation(sock, address):
```

```
    """Общается с клиентом через sock, пока разговор не будет окончен."""
    try:
        while True:
            handle_request(sock)
    except EOFError:
        print('Client socket to {} has closed'.format(address))
    except Exception as e:
        print('Client {} error: {}'.format(address, e))
    finally:
        sock.close()
```

```
def handle_request(sock):
    """Получает один запрос клиента через sock и отправляет ответ."""
    aphorism = recv_until(sock, b'?')
    answer = get_answer(aphorism)
    sock.sendall(answer)

def recv_until(sock, suffix):
    """Получает байты через сокет sock, пока не поступит suffix."""
    message = sock.recv(4096)
    if not message:
        raise EOFError('socket closed')
    while not message.endswith(suffix):
        data = sock.recv(4096)
        if not data:
            raise IOError('received {!r} then socket closed'.format(message))
        message += data
    return message
```

В словаре `aphorisms` три вопроса, которые клиент задает серверу, перечислены как ключи, а ответы хранятся как значения. Функция `get_answer()` ищет ответ в словаре, и если вопрос не распознан, возвращается сообщение об ошибке. Стоит отметить, что вопросы клиента всегда заканчиваются вопросительным знаком, а ответы, включая сообщение об ошибке, — точкой. Эти знаки используются для кадрирования.

Следующие две функции предоставляют общий код запуска, который будет использоваться всеми серверами. `create_srv_socket()` создает слушающий TCP-сокет, который нужен серверу для обработки входящих соединений, а `parse_command_line()` предоставляет общий метод обработки параметров командной строки.

В последних четырех процедурах листинга 7.1 мы видим основные паттерны функционирования сервера.

Каскад из четырех функций повторяет действия, которые мы рассматривали в *главе 3*, когда создавали TCP-сервер для прослушивающего сокета, и *главе 5*, когда кадрировали данные и обрабатывали ошибки.

- ◆ `accept_connections_forever()` — это простой цикл `listen()`, который с помощью `print()` объявляет каждый подключающийся клиент, прежде чем передать сокет следующей функции.
- ◆ `handle_conversation()` — это процедура перехвата ошибок, которая включает бесконечное количество циклов запросов и ответов таким образом, что любые сложности с клиентским сокетом не приводят к сбою программы. Единственное исключение, `EOFError`, не выходит за пределы программы, потому что с его помощью внутренний цикл приема данных обозначает, что клиент завершил запросы и прервал соединение. В этом протоколе (как и в HTTP) такое случается


```
args = parser.parse_args()
address = (args.host, args.p)
client(address, args.e)
```

Если `cause_error` имеет значение `False`, клиент запускает TCP-сокеты и отправляет три вопроса, после каждого ожидая ответ от сервера. Если мы хотим проверить, что будут делать серверы из этой главы в случае ошибки, можно использовать параметр `-e`, чтобы клиент отправил незавершенный запрос и сразу прервал соединение. Если сервер работает нормально, мы увидим три вопроса и три ответа на них.

```
$ python client.py 127.0.0.1
```

```
b'Beautiful is better than?' b'Ugly.'
b'Simple is better than?' b'Complex.'
b'Explicit is better than?' b'Implicit.'
```

Клиент и серверы в данной главе используют порт 1060, как и многие другие примеры в этой книге, но принимают параметр `-p`, чтобы можно было указать альтернативу, если этот порт в вашей системе недоступен.

Однопоточный сервер

Модуль `zen_utils` в листинге 7.1 предоставляет широкий набор утилит, которые упрощают создание простого однопоточного сервера (это самая базовая архитектура, как мы видели в *главе 3*): достаточно написать функцию в три строки.

Листинг 7.3. Максимально простой однопоточный сервер

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Однопоточный сервер, который обслуживает по одному клиенту за раз,
# пока другие клиенты ждут.

import zen_utils

if __name__ == '__main__':
    address = zen_utils.parse_command_line('simple single-threaded server')
    listener = zen_utils.create_srv_socket(address)
    zen_utils.accept_connections_forever(listener)
```

Этот сервер, как и серверы, которые мы создавали в *главах 2 и 3*, требует только один аргумент командной строки: интерфейс, по которому сервер должен ждать исходящие соединения. Укажите типичный IP-адрес `localhost`, чтобы сервер был недоступен для других пользователей в вашей сети.

```
$ python srv_single.py 127.0.0.1
```

```
Listening at ('127.0.0.1', 1060)
```


Конечно, можно открыть сервис на всех интерфейсах системы, указав пустую строку, которую Python интерпретирует как "любой интерфейс на текущем компьютере".

```
$ python srv_single.py''
```

```
Listening at ('', 1060)
```

В любом случае сервер публикует строку, которая указывает, что он успешно открыл свой порт и ждет входящие соединения. Если хотите что-то изменить, откройте справку (-h) или используйте параметр -p, чтобы указать другой порт. Для того чтобы протестировать работу сервера после запуска, выполним скрипт клиента из предыдущего раздела. Сервер сообщит об активности клиентов в окне терминала, где он выполняется, пока клиенты подключаются и отключаются.

```
Accepted connection from ('127.0.0.1', 40765)
```

```
Client socket to ('127.0.0.1', 1060) has closed
```

```
Accepted connection from ('127.0.0.1', 40768)
```

```
Client socket to ('127.0.0.1', 1060) has closed
```

Этого будет достаточно, если сетевой сервис будет обслуживать по одному клиенту за раз. Когда соединение завершается, он готов для следующего. Пока соединение установлено, сервер либо будет выполнять функцию `recv()`, ожидая, что операционная система сообщит, когда поступят новые данные, либо будет максимально быстро создавать ответ и сразу же отправлять его. Если клиент не готов принять данные, операция `send()` или `sendall()` будет заблокирована. Когда клиент будет готов, данные будут переданы, сервер будет разблокирован и вернется к своей функции `recv()`. Ответы всегда отправляются клиенту так быстро, как это возможно.

Когда второй клиент пытается подключиться к серверу, который занят взаимодействием с первым клиентом, мы замечаем все недостатки однопоточной архитектуры. Если целочисленный параметр для `listen()` больше нуля, операционная система попытается установить соединение со вторым входящим клиентом, используя тройное рукопожатие TCP. Это экономит время, потому что эту процедуру уже не придется выполнять, когда сервер будет готов для взаимодействия. Однако пока сервер не закончит обмен данными с первым клиентом, второе соединение будет находиться в очереди. Сервер сможет связаться со вторым клиентом через этот сокет только после того, как завершит взаимодействие с первым, и функция сервера начнет следующий цикл с вызова `accept()`.

Однопоточные серверы очень чувствительны для атак типа "отказ в обслуживании" (Denial of Service, DoS) — достаточно просто подключиться к нему и никогда не отключаться. Сервер навечно застрянет на функции `recv()`, ожидая данных. Правда, автор сервера может указать время ожидания с помощью `sock`. Для того чтобы не ждать бесконечно, используйте `settimeout()` и настройте инструмент защиты от DoS-атаки таким образом, чтобы часто выдавать запросы и избежать окончания времени ожидания. Сервер будет недоступен для других клиентов.

Наконец, однопоточная архитектура зря тратит процессорные и системные ресурсы, потому что сервер не может выполнять другие задачи, пока ждет следующего

запроса клиента. Запустите однопоточный сервер с модулем `trace` из стандартной библиотеки и посмотрите, сколько времени на обработку занимает каждая строка.

```
$ python3.4 -m trace -tg --ignore-dir=/usr srv_single.py ''
```

Каждая строка указывает время в секундах с момента запуска сервера, когда строка кода Python начала выполняться. Вы заметите, что большинство строк начинают выполняться сразу после завершения предыдущей, в ту же сотую долю секунды или в следующую. Однако когда серверу приходится ждать ответа от клиента, выполнение останавливается. Пример:

```
3.02 zen_utils.py(40): print('Accepted connection...')...
```

```
3.02 zen_utils.py(41): handle_conversation(sock, address)
```

```
[?]
```

```
zen_utils.py(57): aphorism = recv_until(sock, b'?')
```

```
zen_utils.py(63): message = sock.recv(4096)
```

```
3.03 zen_utils.py(64): if not message:
```

```
3.03 zen_utils.py(66): while not message.endswith(suffix):
```

```
[?]
```

```
3.03 zen_utils.py(57): aphorism = recv_until(sock, b'?')
```

```
3.03 zen_utils.py(63): message = sock.recv(4096)
```

```
3.08 zen_utils.py(64): if not message:
```

```
3.08 zen_utils.py(66): while not message.endswith(suffix):
```

```
[?]
```

```
3.08 zen_utils.py(57): aphorism = recv_until(sock, b'?')
```

```
3.08 zen_utils.py(63): message = sock.recv(4096)
```

```
3.12 zen_utils.py(64): if not message:
```

```
3.12 zen_utils.py(66): while not message.endswith(suffix):
```

```
[?]
```

```
3.12 zen_utils.py(57): aphorism = recv_until(sock, b'?')
```

```
3.12 zen_utils.py(63): message = sock.recv(4096)
```

```
3.16 zen_utils.py(64): if not message:
```

```
3.16 zen_utils.py(65): raise EOFError('socket closed')
```

```
[?]
```

```
3.16 zen_utils.py(48): except EOFError:
```

```
3.16 zen_utils.py(49): print('Client socket...has closed...')
```

```
3.16 zen_utils.py(53): sock.close()
```

```
3.16 zen_utils.py(39): sock, address = listener.accept()
```

Это полное взаимодействие с программой `client.py`, которое включает три запроса и три ответа. Программа выполняется 0,14 секунды от первой до последней строки и

при этом в общей сложности ждет $0,05 + 0,04 + 0,04 = 0,13$ секунды. Получается, что центральный процессор занят только $0,01 : 0,14 = 7\%$ времени. Конечно, это приблизительные расчеты. Сама по себе трассировка замедляет сервер и увеличивает потребление центрального процессора, так что это неточные значения. Однако более эффективные технологии подтвердят эти выводы. Если во время каждого запроса не выполняются очень ресурсоемкие операции, однопоточные серверы нерационально используют центральный процессор. Клиенты ждут обработки запросов, а процессор в это время простаивает.

Стоит упомянуть о паре технических аспектов. Первый вызов `recv()` возвращает данные сразу, только второй и третий вызовы `recv()` с вопросами, а также последний вызов `recv()`, чтобы узнать о закрытии соединения, возвращают данные с задержкой. Это связано с тем, что сетевые стеки операционной системы включают текст начального запроса в тройное рукопожатие для установки TCP-соединения. В итоге, когда соединение уже установлено и `accept()` может вернуть данные, данные уже ждут возврата из функции `recv()`.

Еще нужно отметить, что `send()` выполняется без задержек. Это связано с тем, что в системе POSIX он возвращает результат, как только исходящие данные были записаны в исходящие буферы операционной системы. Если `send()` возвращает данные, это еще не значит, что система передала их. Приложение может заставить операционную систему остановиться и дожидаться результата передачи, если будет ждать от клиента еще данные.

Давайте вернемся к исходной теме. Как обойти ограничения однопоточного сервера? В оставшейся части этой главы мы будем рассматривать две альтернативные стратегии, чтобы помешать одному клиенту полностью захватить сервер. Обе стратегии позволяют серверу общаться с несколькими клиентами одновременно. Сначала мы поговорим о потоках (или процессах), которые поручают операционной системе переключать внимание сервера между разными клиентами. Затем мы рассмотрим асинхронную серверную архитектуру и узнаем, как самостоятельно переключать сервер, чтобы он мог общаться с несколькими клиентами одновременно в одном потоке управления.

Многопроцессорный и многопоточный серверы

Если мы хотим, чтобы сервер общался с несколькими клиентами одновременно, можно использовать встроенную возможность операционной системы, которая позволяет нескольким потокам проходить по одному фрагменту кода отдельно друг от друга. Для этого используются потоки с общей памятью или процессы, которые работают независимо.

Преимущество этого подхода — в его простоте. Мы можем легко запустить несколько копий одного кода однопоточного сервера.

Недостаток в том, что количество клиентов, с которыми может общаться сервер, ограничено возможностями параллельного выполнения операционной системы.

Даже простаивающий или медленный клиент задействует весь поток или процесс, потребляет ОЗУ и занимает слот в таблице процессов, даже если застрял на вызове `recv()`. Операционные системы редко хорошо масштабируются, когда одновременно выполняются тысячи потоков, и переключение контекстов при переходе с одного клиента на другой замедляет работу сервиса, когда клиентов становится больше.

Многопоточный или многопроцессорный сервер включает главный поток, который выполняет цикл `accept()`, прежде чем передать входящие клиентские сокеты очереди рабочих потоков. К счастью, операционная система упрощает нам жизнь: у каждого потока может быть своя копия слушающего серверного сокета и своя команда `accept()`. Если в данный момент все потоки заняты, операционная система назначает каждое клиентское соединение потоку, который ждет завершения вызова `accept()`, или ставит соединение в очередь, пока один из потоков не освободится. Давайте посмотрим пример в листинге 7.4.

Листинг 7.4. Многопоточный сервер

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Используем несколько потоков для обслуживания нескольких клиентов параллельно.

import zen_utils
from threading import Thread

def start_threads(listener, workers=4):
    t = (listener,)
    for i in range(workers):
        Thread(target=zen_utils.accept_connections_forever, args=t).start()

if __name__ == '__main__':
    address = zen_utils.parse_command_line('multi-threaded server')
    listener = zen_utils.create_srv_socket(address)
    start_threads(listener)
```

Это один из вариантов многопоточной архитектуры: главный поток порождает *n* серверных потоков и завершается, потому что эти *n* потоков должны без конца поддерживать активность процессов. Есть и другие возможности. Например, главный поток мог бы остаться и превратиться в один из рабочих потоков. Или он мог бы служить монитором и время от времени проверять активность своих потоков, а если кто-то из них неактивен, запускать ему замену.

У каждого управляющего потока есть свой образ памяти и собственная область дескрипторов файлов в соответствии с процессами. Это требует дополнительных ресурсов операционной системы, но позволяет лучше изолировать потоки и снижает вероятность того, что они перегрузят главный поток монитора.

Все эти паттерны, о которых вы сможете узнать больше в документации к модулям `threading` и `multiprocessing`, а также в книгах и статьях по параллелизму в Python, имеют кое-что общее: все они выделяют на уровне операционной системы относительно ресурсоемкий управляющий поток каждому подключенному клиенту, даже если в этот момент клиент не делает запросы. Поскольку серверный код может остаться без изменений, пока его контролирует несколько потоков (при условии, что каждый поток устанавливает собственное соединение с базой данных и открывает файлы, так что не приходится координировать ресурсы между потоками), мы можем легко протестировать многопоточный подход на серверной рабочей нагрузке. Если у программы получится обработать наши запросы, то благодаря своей простоте это будет очень привлекательной стратегией для внутренних сервисов, недоступных извне, в которых злоумышленник не сможет просто открывать бездействующие соединения, занимая свободные потоки или процессы.

Фреймворк `SocketServer` из прошлого

Описанный в предыдущем разделе паттерн с использованием управляющих потоков, видимых для операционной системы, для одновременной обработки нескольких клиентских взаимодействий так популярен, что в стандартной библиотеке Python для него есть фреймворк. Он был создан в 1990-х годах, и мы кратко рассмотрим его, чтобы вы понимали, как работает многопоточность, и познакомились с модулем на случай, если придется обслуживать старый код, в котором он используется.

Модуль `socketserver` (`SocketServer` в Python 2) отделяет паттерн сервера, который знает, как открыть слушающий сокет, и принимает новые клиентские соединения от паттерна обработчика, который понимает, как передавать данные через открытый сокет. Как показано в листинге 7.5, эти два паттерна вместе создают серверный объект с классом `handler` в качестве одного из аргументов.

Листинг 7.5. Многопоточный сервер, созданный с помощью модуля `socketserver`

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Сервер написан с помощью устаревшего модуля socketserver из стандартной библиотеки.

from socketserver import BaseRequestHandler, TCPServer, ThreadingMixIn
import zen_utils

class ZenHandler(BaseRequestHandler):
    def handle(self):
        zen_utils.handle_conversation(self.request, self.client_address)
```

```

class ZenServer(ThreadingMixIn, TCPServer):
    allow_reuse_address = 1
    # address_family = socket.AF_INET6 # раскомментируйте эту строку,
    # если вам нужны IPv6

if __name__ == '__main__':
    address = zen_utils.parse_command_line('legacy "SocketServer" server')
    server = ZenServer(address, ZenHandler)
    server.serve_forever()

```

Вместо потоков мы можем создать полностью изолированные процессы, которые будут обслуживать входящих клиентов, заменив `ThreadingMixIn` на `ForkingMixIn`.

В листинге 7.4 мы начали с фиксированного количества потоков, которое администратор сервера мог выбрать в зависимости от того, сколько управляющих потоков может поддерживать операционная система без потери производительности. В листинге 7.5 мы разрешаем пулу подключающихся клиентов выбирать, сколько потоков нужно запустить, и общее количество потоков не ограничено. Недостатки такого подхода очевидны: злоумышленник легко может перегрузить сервер. Этот модуль из стандартной библиотеки лучше не использовать в рабочем окружении и для сервисов, ориентированных на клиентов.

Асинхронные серверы

Чем занять процессор между ответом клиенту и получением следующего запроса, чтобы при этом не пришлось создавать по управляющему потоку на каждого клиента? Можно создать сервер, работающий асинхронно. Вместо того чтобы замирать в ожидании данных или отключения от клиента, код будет слушать длинный список клиентских сокетов и отвечать, когда один из них будет готов для взаимодействия.

Этот паттерн возможен благодаря двум аспектам сетевых стеков современных операционных систем.

Во-первых, они предоставляют системную функцию, которая разрешает процессу ожидать запросов от целого списка клиентских сокетов, а не только одного, так что один поток может обслуживать сотни и даже тысячи клиентских сокетов одновременно. Во-вторых, сокет можно настроить как неблокирующий, т. е. он никогда не будет блокировать вызывающий поток в вызове `send()` или `recv()` и всегда будет сразу возвращать данные из системного вызова `send()` или `recv()` независимо от дальнейшего развития взаимодействия. Если взаимодействие продвигается медленно, вызывающий объект может попробовать позже, когда клиент будет готов для общения.

Термин "асинхронный" означает, что код не ждет определенного клиента, а управляющий поток, который выполняет код, не синхронизирован и не должен ждать,

когда закончится взаимодействие с одним клиентом. Вместо этого можно обслуживать все подключенные клиенты.

Операционная система поддерживает асинхронный режим с помощью различных вызовов. Вызов POSIX `select()` — самый старый способ со множеством недостатков. Его более новые альтернативы — `poll()` в Linux и `epoll()` в BSD. Книга Уильяма Ричарда Стивенса "UNIX: разработка сетевых приложений"¹ — классический труд на эту тему. Поскольку в этой главе мы не будем фокусироваться на создании собственного асинхронного цикла управления, мы рассмотрим только вызов `poll()`. Он покажет, как работает полностью асинхронный фреймворк, с помощью которого вы сможете реализовывать асинхронность в своих проектах. В следующих разделах мы рассмотрим другие фреймворки.

Полный код асинхронного сервера для нашего протокола Zen приводится в листинге 7.6.

Листинг 7.6. Асинхронный цикл обработки событий

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Асинхронный ввод-вывод осуществляется напрямую системным вызовом poll().

import select, zen_utils

def all_events_forever(poll_object):
    while True:
        for fd, event in poll_object.poll():
            yield fd, event

def serve(listener):
    sockets = {listener.fileno(): listener}
    addresses = {}
    bytes_received = {}
    bytes_to_send = {}

    poll_object = select.poll()
    poll_object.register(listener, select.POLLIN)

    for fd, event in all_events_forever(poll_object):
        sock = sockets[fd]

        # Сокет закрыт: удаляем его из структур данных.
```

¹ Stevens W. Richard. UNIX Network Programming. — Prentice Hall, 2003.

```
if event & (select.POLLHUP | select.POLLERR | select.POLLNVAL):
    address = addresses.pop(sock)
    rb = bytes_received.pop(sock, b'')
    sb = bytes_to_send.pop(sock, b'')
    if rb:
        print('Client {} sent {} but then closed'.format(address, rb))
    elif sb:
        print('Client {} closed before we sent {}'.format(address, sb))
    else:
        print('Client {} closed socket normally'.format(address))
    poll_object.unregister(fd)
    del sockets[fd]

# Новый сокет: добавляем его в структуры данных.

elif sock is listener:
    sock, address = sock.accept()
    print('Accepted connection from {}'.format(address))
    sock.setblocking(False)    # усилим socket.timeout, если мы ошибемся
    sockets[sock.fileno()] = sock
    addresses[sock] = address
    poll_object.register(sock, select.POLLIN)

# Входящие данные: продолжаем получать данные, пока не получим suffix.

elif event & select.POLLIN:
    more_data = sock.recv(4096)
    if not more_data:         # конец файла
        sock.close()         # следующий poll() выполнит POLLNVAL
                              # и тем самым произведет очистку
        continue
    data = bytes_received.pop(sock, b'') + more_data
    if data.endswith(b'?):
        bytes_to_send[sock] = zen_utils.get_answer(data)
        poll_object.modify(sock, select.POLLOUT)
    else:
        bytes_received[sock] = data

# Сокет готов к отправке: данные отправляются,
# пока не будут доставлены все байты.
```



```

elif event & select.POLLOUT:
    data = bytes_to_send.pop(sock)
    n = sock.send(data)
    if n < len(data):
        bytes_to_send[sock] = data[n:]
    else:
        poll_object.modify(sock, select.POLLIN)

if __name__ == '__main__':
    address = zen_utils.parse_command_line('low-level async server')
    listener = zen_utils.create_srv_socket(address)
    serve(listener)

```

Этот цикл обработки событий не ждет, пока операционная система сменит контексты при переключении с одного клиента на другой, а хранит состояние взаимодействия с каждым клиентом в своих структурах данных. Поскольку `poll()` может выдавать несколько событий в одном вызове, у сервера есть два цикла: цикл `while` постоянно вызывает `poll()`, а внутренний цикл обрабатывает каждое событие, возвращенное вызовом `poll()`. Для того чтобы не пришлось писать главный цикл сервера на втором уровне вложенности, мы скрываем два уровня итерации за генератором.

Когда `poll()` указывает, что дескриптор файла *n* готов к действию, используется словарь сокетов, где можно найти соответствующий сокет Python. Даже если сокет был закрыт и операционная система больше не напоминает нам о конечной точке, к которой он был подключен, мы помним адреса сокетов и можем вывести диагностические сообщения с соответствующим удаленным адресом.

Асинхронный сервер ведет словарь полученных байтов, где хранятся входящие данные в ожидании выполнения запроса, и словарь байтов для отправки, где исходящие данные ждут, пока операционная система не назначит их отправку.

Эти структуры данных в сочетании с событием, при котором мы сообщаем вызову `poll()`, что ждем данных на каждом соquete, составляют полный конечный автомат для постепенной обработки взаимодействия с клиентом.

1. Клиент, готовый к подключению, сначала проявляется как активность на слушающем соquete сервера, который всегда поддерживается в состоянии `POLLIN` (`poll input`). Когда это происходит, мы запускаем `accept()`, сохраняем сокет и его адрес в словарях и говорим объекту `poll`, что готовы получать данные из нового клиентского сокета.
2. С помощью функции `recv()` мы получаем до 4 Кбайт данных, когда событие `POLLIN` предоставляет клиентский сокет. Если в запросе нет вопросительного знака, который отмечает конец запроса, данные сохраняются в словарь полученных байтов, а затем цикл начинается сначала, и `poll()` повторяется. Если вопрос получен до конца, мы можем ответить на него, найдя подходящий ответ и доба-

вив его в словарь байтов для отправки. Это приводит к важному изменению: сокет переходит из режима `POLLIN`, в котором мы получаем уведомления о поступлении данных, в режим `POLLOUT`, в котором мы ждем оповещения об освобождении исходящих буферов, потому что теперь мы отправляем, а не получаем данные.

3. Когда исходящие буферы на клиентском сокете будут способны получить хотя бы еще один байт, функция `poll()` сообщает нам об этом с помощью `POLLOUT`, а мы отвечаем, пытаясь с помощью функции `send()` отправить все, что хотим передать, оставив только байты, которые не помещаются в исходящие буферы.
4. Наконец, появляется `POLLOUT`, и его функция `send()` позволяет передать оставшиеся исходящие данные. На этом этапе цикл запроса и ответа завершается, и сокет снова переходит в состояние `POLLIN`, чтобы обработать следующий запрос.
5. Мы удаляем клиентский сокет и очищаем исходящие и входящие буферы, когда он выдает ошибку или закрывается. По крайней мере это взаимодействие (из множества других) будет завершено.

Главное преимущество асинхронного подхода в том, что один управляющий поток может обрабатывать взаимодействие с сотнями, если не тысячами, клиентов. Когда сокет клиента готовится к следующему событию, код переходит к следующему действию для этого сокета, получает или отправляет данные, а затем возвращается к `poll()`, чтобы отслеживать активность. Один управляющий поток взаимодействует со множеством клиентов, потому что хранит состояние всех соединений с клиентами в одном наборе словарей, индексированном по клиентским сокетам, не требуя переключений между контекстами на уровне операционной системы (не считая эскалаций и деэскалаций в привилегированном режиме, связанных с системными вызовами `poll()`, `recv()`, `send()` и `close()`). По сути, мы заменяем полноценные переключения контекстов в операционной системе, которые нужны многопоточному или многопроцессорному серверу для переключения внимания между клиентами, на поиск ключей в словарях Python.

Технически предыдущему коду не нужен `sock`, чтобы устанавливать каждый новый клиентский сокет в неблокирующий режим `setblocking(False)`. Почему? Потому что листинг 7.6 не вызывает `recv()`, если только нет данных, ожидающих получения, и `recv()` не блокирует выполнение, если готов хотя бы один байт входных данных. Также он не вызывает `send()`, если нельзя отправить данные, и `send()` не блокирует выполнение, если есть хотя бы один байт, который можно записать в исходящие сетевые буферы операционной системы. Однако на всякий случай лучше все-таки использовать `setblocking()`. В противном случае неправильно адресованный вызов `send()` или `recv()` вызовет блокировку, и сервер будет отвечать только одному клиенту. Ошибка с нашей стороны создаст сокет с вызовом `setblocking()`, и мы получим уведомление о том, что операционная система не может обработать этот вызов прямо сейчас.

Если к одному серверу обращается много клиентов, мы заметим, что один поток легко управляет всеми параллельными взаимодействиями. Однако в листинге 7.6

нам пришлось углубиться в детали работы операционной системы. А если мы поработаем с клиентским кодом и делегируем обработку `select()`, `poll()` и `epoll()` кому-то еще?

Фреймворк `asyncio` с обратными вызовами

С Python 3.4 в стандартную библиотеку добавлен фреймворк `asyncio`, частично разработанный Гвидо ван Россумом (Guido van Rossum), создателем Python. Он пытается объединить сферу, которая была разделена в эпоху Python 2, предоставляя стандартный интерфейс для циклов обработки событий на основе `select()`, `epoll()` и соответствующих технологий.

Возможно, вы уже представляете, какие задачи выполняет этот фреймворк, если изучили листинг 7.6 и заметили, какая малая часть кода в нем связана собственно с протоколом вопроса и ответа, который мы рассматриваем в этой главе. Он поддерживает выполнение центрального цикла в стиле `select`. Он поддерживает таблицу сокетов, по которым ожидается ввод-вывод, а также добавляет и удаляет их из цикла `select`. После закрытия сокетов он очищает и забывает их. Наконец, при получении фактических данных пользовательский код выбирает подходящий ответ.

Фреймворк `asyncio` поддерживает два стиля программирования. Первый похож на старый фреймворк `Twisted` из Python 2. Он использует экземпляр объекта, чтобы отслеживать открытые клиентские соединения. При таком подходе этапы в листинге 7.6, которые продвигают взаимодействие с клиентом, становятся вызовами метода для экземпляров объекта. В листинге 7.7 мы видим знакомые этапы чтения вопроса и создания ответа, записанные так, как требует фреймворк `asyncio`.

Листинг 7.7. Сервер `asyncio` в стиле обратных вызовов

```
#!/usr/bin/env python3
# Network Programming in Python: The Basics
# Асинхронный ввод-вывод в методах обратного вызова asyncio.

import asyncio, zen_utils

class ZenServer(asyncio.Protocol):

    def connection_made(self, transport):
        self.transport = transport
        self.address = transport.get_extra_info('peername')
        self.data = b''
        print('Accepted connection from {}'.format(self.address))

    def data_received(self, data):
        self.data += data
```

```

    if self.data.endswith(b'?'):
        answer = zen_utils.get_answer(self.data)
        self.transport.write(answer)
        self.data = b''

def connection_lost(self, exc):
    if exc:
        print('Client {} error: {}'.format(self.address, exc))
    elif self.data:
        print('Client {} sent {} but then closed'
              .format(self.address, self.data))
    else:
        print('Client {} closed socket'.format(self.address))

if __name__ == '__main__':
    address = zen_utils.parse_command_line('asyncio server using callbacks')
    loop = asyncio.get_event_loop()
    coro = loop.create_server(ZenServer, *address)
    server = loop.run_until_complete(coro)
    print('Listening at {}'.format(address))
    try:
        loop.run_forever()
    finally:
        server.close() loop.close()

```

В листинге 7.7 мы видим, что объект сокета корректно изолирован от кода протокола. Удаленный адрес мы узнаем у фреймворка, не у сокета. Вызов метода используется для доставки данных и просто отображает поступившую строку. Ответ, который мы хотим отправить, передается фреймворку. Код выйдет из цикла, когда данные будут переданы операционной системе для отправки клиенту, благодаря вызову метода `write()`. Фреймворк гарантирует, что это произойдет максимально быстро, если только это не помешает обработке других клиентских соединений.

В большинстве случаев асинхронные рабочие процессы устроены сложнее. Достаточно часто ответы для клиентов невозможно собрать максимально быстро, а нужно читать из файлов в файловой системе или обращаться к бэкенд-сервисам, например базе данных. В таких случаях коду придется работать в двух направлениях: он будет использовать фреймворк при обмене данными не только с клиентом, но и с файловой системой. Тогда методы обратных вызовов могут создавать объекты, порождающие еще больше обратных вызовов, которые сработают после обмена данными с базой данных или диском.

Детали см. в официальной документации по `asyncio`.

Фреймворк `asyncio` с сопрограммами

Еще один способ написать код протокола для фреймворка `asyncio` — создать сопрограмму, представляющую собой функцию, которая не блокирует программу ввода-вывода, а останавливает процесс и возвращает контроль вызывающему объекту, когда ему нужно получить или отправить данные. Обычно Python создает сопрограммы через генераторы — функции, которые содержат одну или несколько инструкций вывода, а значит, считывают последовательность объектов, а не возвращают одно значение при вызове.

Если вам приходилось создавать обычные генераторы с инструкциями `yield`, которые просто предоставляют данные, вы удивитесь, как работают генераторы с `asyncio`. Они используют расширенный синтаксис `yield` на основе PEP 380. Благодаря улучшенному синтаксису генератор может не только считать все элементы, выданные другим генератором через инструкцию `yield from`, но и вернуть значение внутрь сопрограммы и даже выдать исключение, если это нужно потребителю. Это позволяет создать паттерн, при котором сопрограмма выполняет `result = yield` объекта с описанием нужной операции (чтение из другого сокета или доступ к файловой системе) и либо получает результат успешной операции, либо сталкивается с исключением, указывающим, что операция не удалась.

Этот протокол реализован как сопрограмма в листинге 7.8.

Листинг 7.8. Сервер `asyncio` с сопрограммой

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Асинхронный ввод-вывод с помощью сопрограммы asyncio.

import asyncio, zen_utils

@asyncio.coroutine
def handle_conversation(reader, writer):
    address = writer.get_extra_info('peername')
    print('Accepted connection from {}'.format(address))
    while True:
        data = b''
        while not data.endswith(b'?'):
            more_data = yield from reader.read(4096)
            if not more_data:
                if data:
                    print('Client {} sent {!r} but then closed'
                          .format(address, data))
                else:
                    print('Client {} closed socket normally'.format(address))
```

```

        return
    data += more_data
    answer = zen_utils.get_answer(data)
    writer.write(answer)

if __name__ == '__main__':
    address = zen_utils.parse_command_line('asyncio server using coroutine')
    loop = asyncio.get_event_loop()
    coro = asyncio.start_server(handle_conversation, *address)
    server = loop.run_until_complete(coro)
    print('Listening at {}'.format(address))
    try:
        loop.run_forever()
    finally:
        server.close()
        loop.close()

```

Вы узнаете весь код в этом листинге, если сравните его с предыдущими попытками: операция кадрирования в цикле `while`, который многократно вызывал `recv()`, за ней идет запись ответа ожидающему клиенту, и все это обернуто в цикл `while`, который готов отвечать на столько запросов, сколько клиент хочет сделать. Однако есть здесь и ключевое отличие, из-за которого мы не можем просто использовать прежние реализации этой логики. Это генератор, который выполняет `yield`, когда предыдущий код просто осуществлял блокировку и ждал ответа от операционной системы. Благодаря этому различию генератор подключается к подсистеме `asyncio` без блокировки, не мешая нескольким рабочим потокам одновременно обрабатывать взаимодействия.

РЕР 380 рекомендует этот метод для сопрограмм, потому что с ним проще увидеть, когда генератор приостановлен. Каждый раз, когда выполняется `yield`, выполнение может прекращаться на неопределенный период времени. Некоторым программистам не нравится вставлять явные инструкции `yield` в код, поэтому такие фреймворки, как `gevent` и `eventlet` в Python 2, берут типичный сетевой код с обычными блокирующими запросами ввода-вывода и перехватывают их, чтобы обеспечивать асинхронный ввод-вывод.

На момент написания книги эти модули не были переписаны под Python 3. Если их адаптируют, мы сможем выбирать между многословным, но явным подходом с использованием сопрограммы и `asyncio`, при котором мы вставляем `yield` везде, где может возникнуть пауза, и неявным, но более компактным кодом, в котором вызовы `recv()` возвращают контроль асинхронному циклу ввода-вывода, а в самом коде выглядят невинными вызовами метода.

Устаревший модуль `asyncore`

В листинге 7.9 показан модуль `asyncore` из стандартной библиотеки для создания примера протокола. Он может встретиться вам в чем-то коде.

Листинг 7.9. Использование устаревшего модуля `asyncore`

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Сервер написан с помощью устаревшего модуля asyncore из стандартной библиотеки.

import asyncore, asyncchat, zen_utils

class ZenRequestHandler(asyncchat.async_chat):

    def init (self, sock):
        asyncchat.async_chat.__init__(self, sock)
        self.set_terminator(b'?')
        self.data = b''

    def collect_incoming_data(self, more_data):
        self.data += more_data

    def found_terminator(self):
        answer = zen_utils.get_answer(self.data + b'?')
        self.push(answer)
        self.initiate_send()
        self.data = b''

class ZenServer(asyncore.dispatcher):

    def handle_accept(self):
        sock, address = self.accept()
        ZenRequestHandler(sock)

if __name__ == '__main__':
    address = zen_utils.parse_command_line('legacy "asyncore" server')
    listener = zen_utils.create_srv_socket(address)
    server = ZenServer(listener)
    server.accepting = True      # мы уже вызвали listen()
    asyncore.loop()
```

Если вы опытный программист на Python, вы сразу заметите в этом листинге что-то странное. Несмотря на то что объект `ZenServer` явно не передается методу `asyncore.loop()` и вообще не регистрируется, цикл управления каким-то образом

знает, что сервис доступен. Очевидно, модуль использует глобальные переменные или другие хитрые способы установить связи между главным управляющим циклом, объектом сервера и обработчиками запросов, но мы этого не видим.

Зато мы видим, что за кулисами выполняются многие из тех действий, которые мы наблюдали с `asynscio`. Каждое клиентское соединение создает новый экземпляр `ZenRequestHandler`, в котором можно хранить любое состояние, чтобы отслеживать, как разворачивается взаимодействие с клиентом.

Более того, мы видим асимметрию между отправкой и получением, типичную для асинхронных фреймворков. Получение данных приводит к возврату из функции и передаче контроля фреймворку, а при получении нового блока байтов функция вызывается снова. Однако отправка данных выполняется как однократная операция, в которой мы передаем фреймворку управление всеми исходящими полезными данными и надеемся, что он сделает столько вызовов `send()`, сколько необходимо для передачи данных.

Наконец, мы видим, что асинхронные фреймворки, если только они не используют магию `gevent` и `eventlet` (на момент написания книги эти модули доступны только для Python 2), заставляют нас писать серверный код не так, как при создании простого сервера в листинге 7.3. Когда мы использовали много потоков и много процессов, мы просто запускали код однопоточного сервера, а асинхронный подход требует разделять код на небольшие фрагменты, которые могут выполняться независимо, без блокировок. Каждый неблокируемый фрагмент кода должен быть включен в метод в стиле обратного вызова. Если мы используем сопрограмму, каждая неблокируемая операция должна выполняться между инструкциями `yield` и `yield from`.

Комбинированный подход

Переходя с одного объекта протокола на другой (или, в случае с простейшим кодом в листинге 7.6, с одной записи словаря на другую), эти асинхронные серверы могут легко переключаться между трафиком разных клиентов. В результате обслуживание клиентов обходится дешевле, чем при переключении контекстов операционной системой.

Зато у асинхронного сервера есть установленный лимит. Поскольку он делает всю работу в одном потоке операционной системы, он достигает потолка и не может больше обрабатывать клиентские запросы, когда 100% центральных процессоров занято. Это паттерн, который в чистом виде всегда ограничен одним центральным процессором, сколько бы ядер ни было у сервера.

К счастью, у этой проблемы есть решение. Когда нам нужна высокая производительность, мы можем использовать объект асинхронного обратного вызова или сопрограмму, чтобы создать сервис и выполнять его в асинхронном фреймворке. Затем мы можем настроить операционную систему сервера таким образом, чтобы она запускала столько процессов цикла обработки событий, сколько ядер у нас есть.

(Лучше уточнить у администратора сервера, не нужно ли оставить одно или два ядра для операционной системы.) В этом случае мы получим преимущества обоих подходов. Асинхронный фреймворк может быстро работать на своем центральном процессоре, переходя между активными клиентскими сокетами как угодно часто, не переключая контексты. Новые входящие соединения операционная система может распределять между всеми активными серверными процессами, балансируя нагрузку по всему серверу.

Возможно, эти процессы следует заключить в демон, который сможет проверять их работоспособность и перезапускать их или отправлять предупреждения в случае сбоя, как объясняется в *разд. "Несколько слов о развертывании" ранее в этой главе*. Для асинхронного сервиса подойдут любые описанные там технологии, от `supervisord` до контейнеров в рамках модели "платформа как услуга".

Под влиянием `inetd`

Я обязательно должен упомянуть о заслуженном демоне `inetd`, который доступен практически во всех дистрибутивах BSD и Linux. Он решает проблему с запуском демонов по числу сетевых сервисов на одном сервере. Этот демон был придуман еще на заре Интернета.

Нужно просто указать каждый порт, который мы слушаем, в файле `/etc/inetd.conf`.

Каждый из них получает `bind()` и `listen()` от демона `inetd`, но запускает серверный процесс только при подключении клиента. Поскольку `inetd` открывает порты с маленьким номером, этот метод упрощает работу с сервисами по таким портам, которые работают в обычном пользовательском аккаунте. Демон `inetd` может запускать по одному процессу на клиентское соединение или ожидать, что сервер продолжит ждать новые соединения после того, как примет первое соединение с TCP-сервисом, как в этой главе (см. документацию `inetd(8)`, если хотите почитать о более сложном случае с использованием датаграмм UDP).

Для этого требуется больше ресурсов, потому что сервер создает по процессу на соединение, но это более простой подход. Строка `nowait` в четвертом поле файла `inetd.conf` сервиса обозначает одноразовые сервисы.

Когда такой сервис запускается, он видит, что клиентский сокет уже подключен к стандартным потокам ввода, вывода и ошибок. Сервис должен будет общаться только с этим клиентом. В листинге 7.10 приводится пример, который можно использовать с файлом `inetd.conf`.

Листинг 7.10. Общение с одним клиентом, чей сокет связан с потоками `stdin/stdout/stderr`

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Одноразовый сервер для использования inetd(8).

import socket, sys, zen_utils
```

```

if __name__ == '__main__':
    sock = socket.fromfd(0, socket.AF_INET, socket.SOCK_STREAM)
    sys.stdin = open('/dev/null', 'r')
    sys.stdout = sys.stderr = open('log.txt', 'a', buffering=1)
    address = sock.getpeername()
    print('Accepted connection from {}'.format(address))
    zen_utils.handle_conversation(sock, address)

```

Нам редко нужны необработанные обратные трассировки и сообщения о статусах (Python или его библиотеки могут направлять их напрямую в стандартный поток вывода или ошибок), которые прерывают взаимодействие с клиентом, так что этот скрипт заменяет стандартные потоки ввода, вывода и ошибок Python на файлы. Мы затрагиваем объекты файлов внутри системы, а не дескрипторы реальных файлов, так что этот подход распространяется только на ввод-вывод в коде Python. Закройте файловые дескрипторы 0, 1 и 2, если сервер вызывает низкоуровневые библиотеки C, которые выполняют свой стандартный ввод-вывод. В этом сценарии мы создаем что-то вроде песочницы, и для этого лучше использовать `supervisord`, модуль демонизации, или контейнеризацию с помощью платформы, как мы описывали в *разд. "Несколько слов о развертывании" ранее в этой главе*.

Если у выбранного порта немаленький номер, можно проверить листинг 7.10, выполнив в командной строке от имени пользователя `inetd -d inetd.conf` (файл конфигурации будет содержать строку, предоставленную ранее) и подключившись к порту, как обычно, используя `client.py`.

Альтернативный подход — использовать строку `wait` в четвертом поле `inetd.conf`, которая предоставит скрипту доступ к слушающему сокету. Тогда скрипт будет отвечать за вызов `accept()` для клиента, который находится в ожидании. Преимущество этого подхода в том, что сервер может поддерживать активность и выполнять `accept()`, чтобы принимать соединения новых клиентов, без использования `inetd`. Потенциально это более эффективный подход, чем начинать новый процесс для каждого входящего соединения. Если клиент на какое-то время прекратит подключение, сервер может выполнить `exit()`, чтобы экономить память, пока клиент снова не запросит сервис; `inetd` определит, что сервис завершил работу, и возобновит слушание.

В состоянии ожидания может использоваться листинг 7.11. Он может принимать сколько угодно новых соединений, но если пройдет несколько секунд без новых клиентских соединений, время ожидания закончится и код завершится, чтобы сервер не хранил его в памяти.

Листинг 7.11. Код отвечает на несколько клиентских вызовов, а затем выходит, потому что закончилось время ожидания.

```

#!/usr/bin/env python3
# Programming in Python: The Basics
# Многоразовый сервер для использования inetd(8).

```

```
import socket, sys, zen_utils

if __name__ == '__main__':
    listener = socket.fromfd(0, socket.AF_INET, socket.SOCK_STREAM)
    sys.stdin = open('/dev/null', 'r')
    sys.stdout = sys.stderr = open('/tmp/zen.log', 'a', buffering=1)
    listener.settimeout(8.0)
    try:
        zen_utils.accept_connections_forever(listener)
    except socket.timeout:
        print('Waited 8 seconds with no further connections; shutting down')
```

Сервер использует тот же однопоточный подход, с которого мы начали эту главу.

В рабочем окружении, скорее всего, понадобится более надежная структура. Вы можете почерпнуть идеи в этой главе. Единственный критерий: программа должна бесконечно принимать (accept()) уже слушающий сокет. Это очень простой подход, если вы не возражаете, что серверный процесс не будет завершаться после того, как inetd его запустит. Если вы хотите, чтобы сервер завершал работу после некоторого периода неактивности, придется приложить больше усилий. В этой книге мы не будем рассматривать, как по окончании времени ожидания завершать работу группы потоков или процессов, чтобы ни один из них больше не взаимодействовал с клиентом. При этом нужно убедиться, что ни один из них недавно не получал достаточно клиентских соединений, чтобы оправдать его поддержание в активном состоянии.

В некоторых версиях inetd есть простой механизм контроля доступа на основе IP-адреса и имени хоста. Этот механизм создан на базе tcpd — старой программы, которая работала параллельно inetd, пока их не объединили в общий процесс. При таком подходе можно использовать файлы /etc/hosts.allow и /etc/hosts.ban, чтобы разрешать или запрещать доступ к конкретным хостам с определенных (или всех) IP-адресов. Если возникли проблемы с подключением клиентов к сервису, который использует inetd, изучите документацию системы и посмотрите, как системный администратор настроил эти файлы.

Резюме

Серверы, представленные в *главах 2 и 3*, умели работать с клиентами только по одному, а остальным клиентам приходилось ждать, пока закроется сокет предыдущего клиента. Есть два способа преодолеть это препятствие.

С точки зрения программирования, самым простым вариантом будет использовать много потоков (или процессов), при этом код сервера практически не меняется, а операционная система берет на себя переключение между рабочими потоками,

чтобы ожидающие клиенты как можно быстрее получали результаты, а простаивающие клиенты не тратили ресурсы центрального процессора. При таком подходе можно поддерживать взаимодействие сразу с несколькими клиентами, а центральный процессор используется эффективнее, потому что не простаивает в ожидании одного клиента.

Более сложная, но эффективная альтернатива — асинхронное программирование, с помощью которого один управляющий поток может переключать внимание между любым количеством клиентов, предоставляя операционной системе полный список сокетов, с которыми он в данный момент общается. Проблема в том, что в этом случае нужно разделить логику обработки клиентского запроса и создавать ответ с помощью отдельных неблокирующих фрагментов кода, которые возвращают управление асинхронному фреймворку, когда приходится ждать клиента. В принципе, асинхронный сервер можно написать самим, используя вызовы `select()` или `poll()`, но большинство программистов предпочитают использовать фреймворки, например `asyncio` из стандартной библиотеки в Python 3.4 и более новых версий.

Развертывание — это процесс размещения разработанного сервиса на сервере, чтобы он запускался вместе с системой. Эту задачу можно автоматизировать с помощью различных технологий, например `supervisord` или решения "платформа как услуга". Старый демон `inetd`, который предлагает способ гарантировать, что сервис будет запускаться, как только впервые понадобится клиенту, — это, пожалуй, самый простой вариант развертывания для базового сервера Linux.

В этой книге мы еще не раз поговорим о серверах. В *главе 8* мы рассмотрим несколько базовых сетевых сервисов, которыми пользуются программисты Python, а *главы 9–11* будут посвящены протоколу HTTP и инструментам Python для клиента и сервера. Код из этой главы мы будем использовать снова, выбирая между ветвлением при использовании Gunicorn и фреймворками асинхронного программирования вроде Tornado.

Очереди сообщений и кеши

Это короткая, но, пожалуй, самая важная глава в книге. Мы рассмотрим две технологии — кеши и очереди сообщений, на которых строятся масштабные системы. В предыдущих главах мы рассмотрели API сокетов и узнали, как Python использует примитивные операции в IP-сети, чтобы создавать каналы коммуникаций. В следующих главах мы будем рассматривать протоколы, основанные на сокетах, и узнаем, как извлекать документы из Всемирной паутины, отправлять электронную почту и отдавать команды удаленным серверам.

Инструменты, которым посвящена эта глава, имеют свои сходства и различия.

- ◆ Эти технологии популярны, потому что эффективны. Мы используем Memcached или очередь сообщений, потому что это продуманные сервисы, которые решают за нас некоторые задачи, а не потому, что они реализуют какой-то особенный протокол для взаимодействия с другими инструментами.
- ◆ Обычно мы используем эти инструменты внутри компании. Снаружи не всегда можно сказать, используются ли для веб-сайта или сетевого сервиса кеши, очереди или механизмы распределения нагрузки.
- ◆ Такие протоколы, как HTTP и SMTP, были созданы для передачи конкретных полезных данных (гипертекстовые документы и сообщения электронной почты соответственно), а кеши и очереди сообщений обычно не знают, какие данные передают.

В этой главе мы подробно рассмотрим эти технологии. Для каждой из библиотек, которые мы здесь упоминаем, есть подробная онлайн-документация, а по самым популярным из них написаны целые книги.

Содержание главы

- ◆ Использование Memcached (кеширование в памяти).
- ◆ Хеширование и сегментирование.
- ◆ Очереди сообщений.
- ◆ Очереди сообщений в Python.
- ◆ Резюме.

Цель

В этой главе вы узнаете, какие задачи решает каждая технология, как мы можем использовать сервисы для решения этих задач и как работать с этими инструментами с помощью Python. Программист должен всю жизнь учиться и, конечно, распознавать распространенные проблемы, для которых уже есть готовые решения. К сожалению, программисты славятся склонностью снова и снова изобретать колесо. Считайте, что в этой главе вы получите два готовых колеса.

Использование Memcached (кеширование в памяти)

Memcached — это демон кеширования в памяти. Он создает один большой кеш с алгоритмом LRU (least recently used — вытеснение давно не используемого) из свободной оперативной памяти на серверах, где он установлен. Общеизвестно, что он кардинально изменил некоторые важные сервисы в Интернете. Сначала мы посмотрим, как применять его в Python, а затем обсудим его реализацию, чтобы понять, как работает сегментирование — один из ключевых аспектов современных сетей.

Использовать Memcached довольно просто.

- ◆ На каждом сервере, где есть свободная память, запускается демон Memcached.
- ◆ Мы создаем список IP-адресов и номеров портов новых демонов Memcached и распространяем его по всем клиентам, которые будут обращаться к кешу.
- ◆ Теперь клиентские программы смогут использовать очень быстрый корпоративный кеш с ключами и значениями, который служит большим словарем Python для всех наших серверов. Кеш использует алгоритм LRU, т. е. удаляет старые записи, заменяя их новыми.

Существует много клиентов Python для Memcached. Посмотрите список, если интересно: <http://code.google.com/p/memcached/wiki/Clients>.

Поскольку первый клиент написан полностью на Python, ему не понадобятся библиотеки для компиляции. Он находится в Python Package Index, а значит, его можно легко установить в виртуальном окружении (см. главу 1). В Python 3 для этого достаточно одной строки:

```
$ pip install python3-memcached
```

API у пакета простой и понятный. Мы могли бы ожидать что-то более похожее на словарь Python с его собственными методами, вроде `__getitem__()`, но создатель этого API предпочел использовать те же имена методов, что и в других языках, которые поддерживает Memcached. Это было разумное решение, потому что так переводить примеры Memcached на Python будет гораздо проще. Если у вас на компьютере Memcached работает на порту по умолчанию 11211, простое взаимодействие с ним через командную строку Python могло бы выглядеть так:

```
>>> import memcache
>>> mc = memcache.Client(['127.0.0.1:11211'])
```

```
>>> mc.set('user:19', 'Simple is better than complex.')
True
>>> mc.get('user:19')
'Simple is better than complex.'
```

Как видите, очень похоже на словарь Python. Когда мы отправляем строку в качестве значения, она напрямую записывается в Memcached в кодировке UTF-8, а затем декодируется при извлечении в дальнейшем. Если это будет любой другой объект Python, кроме простейшей строки, модуль memcache автоматически преобразует его в поток байтов и сохранит двоичный pickle в Memcached (см. главу 5). Помните об этой особенности, если будете создавать приложение Python, которое использует кеш Memcached с клиентами, написанными на других языках. Только значения, сохраненные как строки, будут доступны для чтения другими клиентами.

Помните, что сервер удаляет данные из Memcached. Цель кеша — хранить результаты, чтобы не тратить время и ресурсы на их повторное вычисление. Он не предназначен для хранения информации, которую можно восстановить из других источников. Если выполнить предыдущие инструкции при загруженном Memcached, а между set() и get() пройдет достаточно времени, действие get() может определить, что срок хранения строки в кеше истек, и ее там больше нет.

В листинге 8.1 мы видим базовый подход к использованию Memcached. Код заглядывает в Memcached, чтобы узнать, сохранен ли в кеше ответ на заданный вопрос, прежде чем выполнять ресурсоемкую операцию по вычислению квадрата. Если ответ есть, его можно вернуть сразу, не вычисляя заново. Если он еще не сохранен в кеше, то будет сохранен там, прежде чем нам вернется результат.

Листинг 8.1. Используем Memcached, чтобы ускорить обработку

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Используем memcached для кеширования дорогого результата.

import memcache, random, time, timeit

def compute_square(mc, n):
    value = mc.get('sq:%d' % n)
    if value is None:
        time.sleep(0.001) # делаем вид, что вычисление квадрата требует много времени
        value = n * n
        mc.set('sq:%d' % n, value)
    return value

def main():
    mc = memcache.Client(['127.0.0.1:11211'])
```

```
def make_request():
    compute_square(mc, random.randint(0, 5000))

print('Ten successive runs:')
for i in range(1, 11):
    print(' %.2fs' % timeit.timeit(make_request, number=2000), end='')
print()

if __name__ == '__main__':
    main()
```

Для того чтобы этот пример получился, демон Memcached должен быть запущен на вашем компьютере на порту 11211.

Программа будет работать с нормальной скоростью в первые несколько сотен запросов. Если у нее впервые спросят квадрат какого-то числа, а она не найдет его в кеше оперативной памяти, ей придется вычислить его. Однако когда программа будет снова и снова встречать те же числа, она начнет работать быстрее, потому что будет находить в кеше готовые результаты.

Программа будет работать гораздо быстрее через несколько тысяч запросов, поскольку мы выбираем из 5000 целых чисел. Десятый пакет из 2000 вычислений на моем компьютере выполняется более чем в шесть раз быстрее, чем первый.

\$ python squares.py

Десять выполнений подряд:

```
2.89s 2.14s 1.55s 1.20s 0.97s 0.79s 0.64s 0.51s 0.49s 0.44s
```

Это типичный паттерн для кеширования. Когда в кеше накапливается достаточно ключей и значений, скорость выполнения возрастает, но когда Memcached наполняется и начинает включать большой процент возможных значений, снижение времени выполнения замедляется.

Какие данные мы записываем в кеш в реальном приложении?

Многие программисты сохраняют в кеше нижний уровень ресурсоемких вызовов, например поиск в базе данных, чтение из файловой системы или запросы к внешним сервисам. Обычно довольно просто понять, какие элементы будут кешированы и на какой срок, чтобы информация не устаревала. Если запись в базе данных изменится, кеш можно заранее очистить от устаревших объектов, связанных с новым значением. Однако мы можем получить много пользы и от кеширования промежуточных выходных данных на более высоких уровнях приложения, например структур данных, фрагментов HTML или даже целых веб-страниц. Успешное обращение к кешу позволяет сэкономить не только потому, что нам не придется обращаться к базе данных, но и потому, что нам не придется преобразовывать результат в структуру данных, а затем в отрисованный HTML.

По Memcached есть много подробной документации и инструкций.

Во-первых, ключи должны быть уникальными, поэтому разработчики часто используют префиксы и кодировки, чтобы различать типы хранимых объектов. `User:19`, `mypage:/node/14` и даже полный текст SQL-запроса часто используются как ключи. Длина ключа не должна превышать 250 символов, но можно включить в поиск и более длинную строку, если использовать надежный хеш. Кстати, значения в Memcached могут быть длиннее ключей, но не более 1 Мбайт.

Во-вторых, не забывайте, что Memcached — это кэш. Он временный, хранит данные в оперативной памяти, а после перезапуска все забудет. Если кэш будет утрачен, приложение всегда сможет восстановить все данные из него.

В-третьих, кэш не должен возвращать слишком старые данные.

Определение "слишком старых" зависит от предметной области. Например, баланс на банковском счете должен быть актуальным, а главная новость на новостном сайте может держаться несколько минут.

Есть три метода удалять устаревшие данные, чтобы они не возвращались пользователю, когда потеряют свою актуальность.

- ◆ Мы можем определить срок годности для каждого элемента в кеше, и Memcached будет сам удалять устаревшие объекты.
- ◆ Если мы найдем способ сопоставить фрагмент данных со всеми ключами в кеше, которые могут его включать, мы можем аннулировать все записи в кеше, как только они потеряют актуальность.
- ◆ Вместо удаления некорректных записей их можно перезаписывать. Этот вариант хорошо подходит для записей, которые меняются десятки раз в секунду. Тогда клиенты, вместо того чтобы обнаружить недостающее значение и попытаться вычислить его снова, найдут измененную запись.

Декораторы — это популярный способ добавления кеширования в Python, потому что они окружают вызовы функций, не влияя на имена и подписи. В PyPI существует несколько библиотек с декораторами, которые могут использовать Memcached.

Хеширование и сегментирование

Структура Memcached иллюстрирует ключевое понятие, которое встречается в разных базах данных и которое лучше учитывать при создании собственной архитектуры. Когда клиент Memcached получает список экземпляров Memcached, он сегментирует базу данных, хешируя строковое значение каждого ключа и позволяя хешу определять, какой сервер в кластере Memcached хранит этот ключ.

Представим конкретную комбинацию ключа и значения, например ключ `sq:42` и значение `1764`, которое мог сохранить код из листинга 8.1. Кластер Memcached хочет сохранить этот ключ с этим значением ровно один раз, чтобы зря не занимать оперативную память. При этом, чтобы сервис работал быстро, он хочет отказаться

от дублирования, т. е. избегать координации между множеством серверов или коммуникаций между всеми клиентами.

Это значит, что без дополнительной информации, кроме ключа и списка серверов Memcached, с которыми они настроены, всем клиентам понадобится схема, чтобы понимать, к чему относится этот фрагмент данных. Если они этого не понимают, ключ и значение нужно реплицировать на множество сервисов. При таком подходе будет доступно меньше памяти, а если клиент удалит один недопустимый элемент, его копии могут остаться на других серверах.

Решение проблемы: все клиенты могут использовать один стабильный процесс, чтобы преобразовывать ключ в целое число, которое выбирает один сервер из списка. Для этого можно использовать хеш, который преобразует строку в уникальный набор символов, скрывая ее содержимое.

В листинге 8.2 видно, почему пары "ключ — значение" нужно исключить. Этот код загружает английский словарь (возможно, вам потребуется загрузить свой словарь или изменить путь, чтобы скрипт работал на вашем компьютере) и проверяет, как эти слова были бы распределены по четырем серверам, если бы использовались как ключи.

Листинг 8.2. Существует два подхода к назначению данных серверам: биты из хеша и паттерны в данных

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Хеши - это отличный способ разделить работу.

import hashlib

def alpha_shard(word):
    """Плохо справляется с задачей, распределяя данные по серверам на основе первой
    буквы."""
    if word[0] < 'g':          # abcdef
        return 'server0'
    elif word[0] < 'n':       # ghijklm
        return 'server1'
    elif word[0] < 't':       # nopqrs
        return 'server2'
    else:                     # tuvwxzyz
        return 'server3'

def hash_shard(word):
    """Назначает данные серверам, используя встроенную функцию Python hash()."""
    return 'server%d' % (hash(word) % 4)
```

```
def md5_shard(word):
    """Назначает данные серверам, используя общедоступный хеш-алгоритм."""
    data = word.encode('utf-8')
    return 'server%d' % (hashlib.md5(data).digest()[-1] % 4)

if __name__ == '__main__':
    words = open('/usr/share/dict/words').read().split()
    for function in alpha_shard, hash_shard, md5_shard:
        d = {'server0': 0, 'server1': 0, 'server2': 0, 'server3': 0}
        for word in words:
            d[function(word.lower())] += 1
        print(function. __name__[:-6])
        for key, value in sorted(d.items()):
            print(' {} {} {:.2}'.format(key, value, value / len(words)))
        print()
```

Функция `hash()` — это встроенная процедура хеширования в Python, которая оптимизирована для быстрой работы, потому что используется для реализации поиска по словарю Python. Алгоритм хеширования MD5 больше не считается безопасным, но его можно использовать для распределения нагрузки по серверам (хотя он работает медленнее, чем встроенный хеш Python). Результаты явно показывают, почему опасно распределять нагрузку по критериям, которые могут привести к неравномерности.

\$ python hashing.py

alpha

```
server0  35285  0.37
server1  22674  0.28
server2  29097  0.39
server3  12115  0.15
```

hash

```
server0  24768  0.25
server1  25004  0.25
server2  24713  0.25
server3  24686  0.25
```

md5

```
server0  24777  0.25
server1  24820  0.25
server2  24717  0.25
server3  24857  0.25
```

Как видите, если мы разделим данные по начальным буквам и назначим каждому из четырех серверов примерно одинаковое количество букв, сервер 0 получит в три

раза больше, чем сервер 3, хотя у него всего шесть букв, а не семь. Зато хеширование справилось с задачей. Несмотря на закономерности в первых буквах, структуре и окончании слов в английском языке, хеш-функции равномерно распределили слова по четырем воображаемым серверам.

Конечно, многие наборы данных не содержат столько закономерностей, сколько английские слова, но сегментированные базы данных, вроде Memcached, всегда сталкиваются с какими-то паттернами во входящих данных.

Довольно часто попадают ключи, которые начинаются с одинакового префикса или содержат символы из алфавита с ограниченной областью распространения, например десятичные числа. Из-за явных закономерностей сегментирование следует всегда выполнять с помощью хеш-функции.

Разумеется, при использовании баз данных вроде Memcached, для которых клиентские библиотеки допускают внутреннее сегментирование, можно не задумываться об этой проблеме. Эта информация пригодится, если вам придется создавать собственный сервис, распределяющий работу или данные по узлам в кластере таким образом, чтобы это можно было воспроизвести на множестве клиентов в том же хранилище данных.

Очереди сообщений

Как мы видели в *главе 2*, датаграммы полезны для ненадежных сервисов, когда данные могут теряться, повторяться или путаться. Протоколы очереди сообщений позволяют организовать надежную доставку сообщений без использования датаграмм. Очередь сообщений гарантирует согласованность и атомарность при доставке: либо сообщение придет полностью, либо не придет совсем. Протокол очереди сообщений сам отвечает за кадрирование. Клиенты очереди сообщений не выполняют цикл с вызовов `recv()`, пока сообщение не поступит целиком.

Еще одно преимущество очередей сообщений в том, что они позволяют создавать разные топологии взаимодействия клиентов, не ограничиваясь двусторонним соединением, поддерживаемым IP-протоколом вроде TCP. Очереди сообщений можно использовать в разных целях.

- ◆ Когда мы указываем адрес электронной почты, чтобы зарегистрировать аккаунт на сайте, обычно мы видим страницу с подобным сообщением: "Благодарим за регистрацию. Мы отправили вам сообщение с подтверждением". Эта страница появляется сразу, она не ждет несколько минут, которые могут потребоваться для доставки сообщения. Сайт обычно сохраняет адрес в очереди сообщений, из которой бэкэнд-серверы могут его извлечь, когда будут готовы установить новое исходящее SMTP-соединение (см. *главу 13*). Если по какой-то причине доставить сообщение не удалось, адрес снова попадет в очередь сообщений, чтобы можно было повторить попытку.
- ◆ Очереди сообщений могут лежать в основе пользовательского сервиса, использующего удаленный вызов процедур (`remote procedure call`, RPC) (см. *главу 18*).

В такой архитектуре загруженные фронтенд-серверы избегают ресурсоемких задач, отправляя запросы к очереди сообщений, которая содержит десятки или сотни бэкэнд-серверов. Затем мы просто ждем ответа.

- ◆ Объемные данные о событиях, которые нужно собирать, хранить и обрабатывать централизованно, часто отправляются в виде крошечных сообщений через очередь сообщений. Очереди сообщений полностью заменили журналирование на локальных жестких дисках и предыдущие технологии передачи журналов, например syslog, на некоторых сайтах.

С помощью очередей сообщений можно создать единую систему, к которой могут подключаться все клиенты и серверы, или издатели и подписчики.

Очереди сообщений меняют наш подход к разработке приложений. Один управляющий поток может включать чтение данных HTTP из сокета, аутентификацию и интерпретацию запроса, использование API для настраиваемой обработки изображений и запись результата на диск в типичной монолитной программе. Все API, используемые этим управляющим потоком, можно установить в одной системе и загрузить в один экземпляр среды выполнения Python. Однако если у нас есть очередь сообщений, почему такая ресурсоемкая, специализированная и не связанная с веб-сервисами рабочая нагрузка, как обработка изображений, должна делить ресурсы центрального процессора и диска с фронтенд-сервисом HTTP? В итоге мы переходим на специализированные компьютеры, объединенные в кластеры, которые обслуживают один сервис, вместо того чтобы создавать сервисы на крупных компьютерах с десятками разнородных библиотек. Если специалисты понимают топологию системы сообщений и протокол для отделения сервера без потери сообщений, они легко могут останавливать, обновлять и снова включать серверы обработки изображений, никак не затрагивая пул HTTP-сервисов за балансировщиком нагрузки, который находится перед очередью сообщений.

Существует несколько топологий очередей сообщений.

- ◆ **Канал** — это структура, которая больше всего похожа на очередь в бытовом понимании: издатель создает сообщения и отправляет их в очередь, откуда их забирает потребитель. Например, фронтенд-компьютеры сайта с фотоизображениями могут получать от конечных пользователей фотографии и ставить их во внутреннюю очередь. Сообщения из этой очереди читаются генераторами эскизов. Каждый агент получает за раз одно сообщение с изображением, для которого нужно создать много эскизов. В часы пиковой загрузки очередь может быть длинной, в более спокойные периоды она может пустовать, но в любом случае фронтенд-серверы могут быстро отвечать ожидающему клиенту, сообщая, что фотография успешно загружена и скоро появится в ленте.
- ◆ **Модель "издатель — подписчик"**, или **fanout**, похожа на классическую очередь, но с одним важным исключением. С одной стороны, в классической очереди каждое сообщение отправляется только одному потребителю — нет смысла отправлять одну фотографию на два сервера создания эскизов. Подписчики, с другой стороны, часто должны получать все сообщения от издателей или отфильтровывать их часть. Внешние сервисы, которые должны сообщать о собы-

тиях внешнему миру, используют очередь такого типа. Также с ее помощью можно создать фабрику, через которую можно узнавать, какие системы работают, какие остановлены для обслуживания, а какие могут публиковать адреса других очередей сообщений по мере их создания и удаления.

- ◆ Наконец, **паттерн "запрос — ответ"** считается самым сложным, потому что требует кругового пути. В предыдущих топологиях у издателя было довольно мало ответственности: он просто подключался к очереди и передавал сообщение. Клиент очереди сообщений, с другой стороны, должен после отправки запроса не прерывать подключение и ждать ответа. Для этого у очереди должен быть метод адресации, который позволит отправлять ответы только нужному клиенту, хотя подключенных клиентов могут быть тысячи. Несмотря на всю сложность, это один из самых эффективных паттернов, потому что десятки и сотни клиентов распределяются по огромному количеству серверов, которые отвечают только за очередь сообщений. Поскольку хорошая очередь сообщений позволяет подключать и отключать серверы без потери сообщений, эта топология позволяет останавливать серверы для обслуживания так, что клиенты этого даже не заметят.

Очереди по модели "запрос — ответ" — это отличный метод подключить легковесные рабочие потоки, которые могут выполняться тысячами на одном компьютере, например фронтенд-сервере, к клиентам базы данных или файловым серверам, которые берут на себя основную работу. Паттерн "запрос — ответ" хорошо сочетается с RPC и дает бонус, который обычно отсутствует в более простых системах RPC: много потребителей и издателей можно назначить одной очереди, так что группы клиентов не будут знать друг о друге.

Очереди сообщений в Python

Самые популярные очереди сообщений реализуются на отдельных серверах. Все компоненты нашего приложения (издатели, потребители, фильтры и сервисы RPC) можно подключить к очереди сообщений, и они не будут знать адреса и идентификаторы друг друга. Протокол AMQP — это один из самых популярных протоколов очередей сообщений для любых языков, который поддерживается самыми разными серверами с открытыми источниками (opensource), включая RabbitMQ, Apache Qpid и др.

Многие программисты не знают, как использовать протокол сообщений, потому что есть сторонние библиотеки, которые объединяют все преимущества очереди сообщений в удобный API. Вместо того чтобы изучать AMQP, многие программисты на Python, использующие веб-фреймворк Django, применяют, например, популярную распределенную очередь задач Celery. Поддерживая различные бэкенд-сервисы, библиотека может предоставлять независимость от протокола. Celery позволяет вместо специальной системы сообщений использовать простое хранилище пар "ключ — значение" Redis в качестве очереди сообщений.

Для этой книги удобнее взять пример, который не потребует установки полноценного сервера очереди сообщений, так что мы будем использовать ØMQ (Zero Message Queue), которая была разработана компанией, создавшей AMQP, но логика обработки сообщений перенесена из централизованного брокера в клиенты сообщений. Иными словами, внедрение библиотеки ØMQ в каждую программу позволяет коду создавать отдельную фабрику сообщений без единого брокера. Этот подход отличается от архитектуры с централизованным брокером, которая обеспечивает надежность, избыточность, повторную передачу и сохранение на диске. На сайте ØMQ подробно описаны преимущества и недостатки этого решения:

www.zeromq.org/docs:welcome-from-amqp

В листинге 8.3 решается простейшая задача, которая не требует очереди сообщений: мы определяем значение p , используя простой, хоть и неэффективный метод Монте-Карло. На рис. 8.1 изображена структура сообщений. Процедура `bitsource` создает строку из $2n$ символов, которая содержит единицы и нули. Нечетные биты будут обозначать n -значную целочисленную координату x , а четные — n -значную целочисленную координату y . Значение координаты не может превышать радиус круга. Нужно определить, находится точка внутри или снаружи четверти круга с центром в точке начала координат.

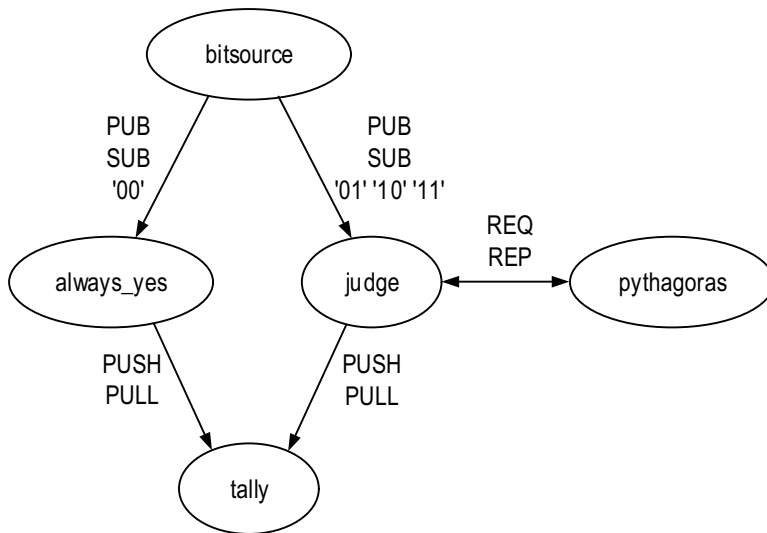


Рис. 8.1. Топология для вычисления числа π методом Монте-Карло

Для этих бинарных строк мы можем создать аудиторию из двух слушателей, используя топологию "издатель — подписчик". Обе координаты начинаются с нуля, точка должна находиться в левом нижнем квадранте поля, всегда в пределах круга. Слушатель `always_yes` будет получать только цифровые строки, начинающиеся с 00, и всегда может выдавать ответ Y. Функция `judge`, которая выполняет проверку, должна обработать остальные три возможных паттерна для первых двух битов. Она будет просить `pythagoras` вычислить сумму квадратов двух целочисленных координат.

нат, чтобы определить, находится указанная точка в круге или за его пределами, а затем отправит T (True) или F (False) в исходящую очередь.

Метод `tally` получает T или F для каждого случайного значения и вычисляет π , сравнивая число ответов T с общим числом ответов (T и F). Если вам интересно, как происходят вычисления, поищите в Интернете, как вычислить число π методом Монте-Карло.

В листинге 8.3 реализована архитектура из пяти рабочих процессов, которые выполняются 30 секунд, а затем программа завершается.

Нам понадобится библиотека `ØMQ`, которую можно получить в Python, выполнив в новом виртуальном окружении команду:

```
$ pip install pyzmq
```

Если в вашей операционной системе предустановлен Python или вы используете отдельную установку Python, например Anaconda, этот пакет может уже быть установлен. В любом случае листинг 8.3 сможет выполняться без ошибок импорта и без дополнительных настроек.

Листинг 8.3. Фабрика сообщений ØMQ с пятью рабочими процессами

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Небольшое приложение, которое использует несколько разных очередей сообщений.

import random, threading, time, zmq

B = 32 # количество битов точности в каждом случайном целом числе

def ones_and_zeros(digits):
    """n выражается минимум как d двоичных чисел, без специального
    префикса."""
    return bin(random.getrandbits(digits)).lstrip('0b').zfill(digits)

def bitsource(zcontext, url):
    """Создаются случайные точки в единичном квадрате."""
    zsock = zcontext.socket(zmq.PUB)
    zsock.bind(url)
    while True:
        zsock.send_string(ones_and_zeros(B * 2))
        time.sleep(0.01)

def always_yes(zcontext, in_url, out_url):
    """Координаты в левом нижнем квадранте находятся внутри единичного круга."""
    isock = zcontext.socket(zmq.SUB)
```



```

isock.connect(in_url)
isock.setsockopt(zmq.SUBSCRIBE, b'00')
osock = zcontext.socket(zmq.PUSH)
osock.connect(out_url)
while True:
    isock.recv_string() osock.send_string('Y')

```

```

def judge(zcontext, in_url, pythagoras_url, out_url):
    """Определяем, находится ли каждая полученная координата внутри
    единичного круга."""
    isock = zcontext.socket(zmq.SUB) isock.connect(in_url)
    for prefix in b'01', b'10', b'11':
        isock.setsockopt(zmq.SUBSCRIBE, prefix)
        psock = zcontext.socket(zmq.REQ)
        psock.connect(pythagoras_url)
        osock = zcontext.socket(zmq.PUSH)
        osock.connect(out_url)
        unit = 2 ** (B * 2)
        while True:
            bits = isock.recv_string()
            n, m = int(bits[:2], 2),
                int(bits[1:2], 2)
            psock.send_json((n, m))
            sumsquares = psock.recv_json()
            osock.send_string('Y' if sumsquares < unit else 'N')

```

```

def pythagoras(zcontext, url):
    """Возвращается сумма квадратов последовательностей чисел."""
    zsock = zcontext.socket(zmq.REP)
    zsock.bind(url)
    while True:
        numbers = zsock.recv_json()
        zsock.send_json(sum(n * n for n in numbers))

```

```

def tally(zcontext, url):
    """Считаем, сколько точек попало в единичный круг, и выводим число пи."""
    zsock = zcontext.socket(zmq.PULL)
    zsock.bind(url)
    p = q = 0
    while True:
        decision = zsock.recv_string()

```

```

q += 1
if decision == 'Y':
    p += 4
    print(decision, p / q)

```

```

def start_thread(function, *args):
    thread = threading.Thread(target=function, args=args)
    thread.daemon = True      # можно нажать Ctrl+C и прервать программу
    thread.start()

```

```

def main(zcontext):
    pubsub = 'tcp://127.0.0.1:6700'
    reqrep = 'tcp://127.0.0.1:6701'
    pushpull = 'tcp://127.0.0.1:6702'
    start_thread(bitsource, zcontext, pubsub)
    start_thread(always_yes, zcontext, pubsub, pushpull)
    start_thread(judge, zcontext, pubsub, reqrep, pushpull)
    start_thread(pythagoras, zcontext, reqrep)
    start_thread(tally, zcontext, pushpull)
    time.sleep(30)

```

```

if __name__ == '__main__':
    main(zmq.Context())

```

Делить один сокет сообщений на несколько потоков небезопасно, поэтому каждый поток создает собственный сокет или сокеты для коммуникации. При этом у потоков один объект контекста, который обеспечивает согласованность. Все они находятся в общей области URL-адресов, сообщений и очередей.

Хотя у этих сокетов есть методы с именами, похожими на известные операции с сокетами, например `recv()`, здесь они работают иначе. Сообщения всегда хранятся в хронологическом порядке и никогда не дублируются. Они существуют не как один непрерывный поток, а как отдельные единицы.

Очевидно, что этот пример немного надуманный, просто чтобы в нескольких строках кода показать, как работает основной паттерн обмена сообщениями. Функции `always_yes` и `judge` устанавливают соединения с `bitsource`, формируя систему "издатель — подписчик", в которой каждый подключенный клиент получает копию каждого сообщения от издателя (кроме сообщений, которые отфильтровываются, как в нашем случае). Выбирая сообщение, у которого первые цифры совпадают со строкой фильтра, фильтры в сокетах добавляют, а не вычитают его из общего числа сообщений. Поскольку эти четыре фильтра включают все возможные комбинации двух начальных двоичных чисел, наша пара подписчиков гарантированно получит каждую битовую строку, выданную `bitsource`.

Отношения между `judge` и `pythagoras` представляют собой типичную связь RPC "запрос — ответ", в которой клиент, контролирующий сокет `REQ`, должен первым подать голос, чтобы назначить свое сообщение одному из ожидающих агентов, привязанных к этому сокету. (В нашем сценарии привязан только один агент.) За кулисами фабрика обмена сообщениями добавляет к запросу обратный адрес. Когда агент выполняет задачу и отвечает, ответ отправляется по обратному адресу через сокет `REP` соответствующему клиенту.

Наконец, рабочий процесс `tally` использует схему `push-pull`, чтобы каждый отправленный элемент был передан одному и только одному агенту, подключенному к сокету. Если у нас несколько рабочих процессов `tally`, каждый новый фрагмент данных, поступающий сверху, передается только одному из них, и они все рассчитывают число π независимо.

В данном листинге, в отличие от предыдущих примеров программирования сокетов в этой книге, нам не нужно беспокоиться о вызове `bind()` или `connect()`. Это функции очереди сообщений, которые используют время ожидания и опросы, чтобы за кулисами повторять неудавшийся вызов `connect()`, если конечная точка, представленная этим URL-адресом, станет доступна позднее. Получается, агенты не могут входить в приложение и выходить из него, пока оно выполняется.

На моем ноутбуке система рабочих процессов смогла вычислить первые три цифры числа π за отведенное время.

```
$ python queuepi.py
```

```
...
```

```
Y 3.1406089633937735
```

Из этого короткого примера можно подумать, что программировать очередь сообщений очень просто. В реальности мы используем более сложные паттерны, чтобы гарантировать доставку сообщений, сохранять их, если в данный момент их нельзя обработать, и управлять потоком, чтобы защитить агента от перегрузки, если в очереди будет слишком много сообщений. В официальной документации можно найти подробное описание того, как применять эти паттерны для сервиса в рабочем окружении. В конце концов, многие разработчики считают, что лучше использовать полноценный брокер сообщений за `Celery`, например `RabbitMQ`, `Qpid` или `Redis`, чтобы получить все необходимые возможности без лишних усилий и риска сделать ошибку.

Резюме

Современные приложения нередко обслуживают тысячи и миллионы потребителей. Для этого существует несколько удобных технологий, и все они хорошо сочетаются с Python.

`Memcached` — это популярный сервис, который объединяет всю свободную оперативную память на всех серверах, где он развернут, в огромный кэш LRU. `Memcached` может заметно разгрузить базу данных или другое бэкенд-хранилище,

если у вас есть система для аннулирования или замены устаревших записей — или если вы работаете с данными, которые устаревают по предсказуемому расписанию. Его можно вставить в разные точки рабочего процесса. Вместо того чтобы кешировать результат ресурсоемкого запроса к базе данных, можно кешировать только получившуюся страницу.

Очереди сообщений — еще один популярный механизм координации и интеграции различных частей приложения, для которого может требоваться отдельное оборудование, стратегии балансировки нагрузки или даже языки программирования. Очереди могут распределять сообщения среди очень большого числа потребителей или серверов. Это было бы невозможно при использовании прямых связей, которые устанавливаются между двумя сокетами TCP. Кроме того, они могут использовать базу данных или другое постоянное хранилище, чтобы сообщения не терялись при сбое сервера. Очереди сообщений обеспечивают гибкость и устойчивость, потому что если один компонент системы станет узким местом, нагрузка будет перераспределена. Очередь сообщений скрывает серверы и процессы, которые обслуживают тот или иной запрос, поэтому мы можем легко отключать, обновлять, перезапускать или добавлять серверы, не прерывая работу.

Очереди сообщений обычно используются за понятным API, например Celery, который популярен в сообществе Django. Redis также можно применять в качестве бэкенда. Советую почитать побольше про Redis.

Мы изучили базовые и специфические технологии поверх TCP/IP и переходим к протоколу HTTP, на котором работает Интернет. Ему будут посвящены следующие три главы.

Это первая из трех глав об HTTP. Проектирование и развертывание HTTP-серверов мы рассмотрим в *главе 10*. В обеих главах мы будем изучать протокол на базовом уровне, т. е. как механизм извлечения и загрузки документов.

HTTP может передавать самые разные документы, включая фотографии, PDF, музыку и видео, но в *главе 11* мы сосредоточимся на веб-страницах, потому что именно благодаря этому типу документов HTTP повсеместно используется в Интернете. Всемирная паутина представляет собой коллекцию гипертекстов, связанных друг с другом с помощью URL-адреса, о котором мы также поговорим в *главе 11*. Там мы узнаем о шаблонах библиотек, формах, паттернах программирования на основе Аякх, а также веб-фреймворках, которые объединяют все эти паттерны так, чтобы упростить жизнь разработчикам.

RFC с 7230 по 7235 описывают HTTP версии 1.1. Если у вас будут вопросы по этому протоколу, изучите данные документы. Также советую прочесть более техническое описание структуры протокола в главе 5 в знаменитой диссертации Роя Томаса Филдинга "Архитектурные стили и проектирование сетевых программ" (Roy Thomas Fielding. Architectural Styles and the Design of Network-based Software Architectures).

В этой главе мы обсудим, как отправить запрос серверу и получить документы в ответ.

Содержание главы

- ◆ Библиотеки клиентов Python.
- ◆ Кадрирование, шифрование и порты.
- ◆ Методы.
- ◆ Хосты и пути.
- ◆ Коды состояний.
- ◆ Валидация и кеширование.
- ◆ Кодирование содержимого.

- ◆ Согласование содержимого.
- ◆ Тип содержимого.
- ◆ Аутентификация по HTTP.
- ◆ Файлы cookie.
- ◆ Поддержание соединения и httplib.
- ◆ Резюме.

Цель

В этой главе вы узнаете, как использовать HTTP в клиентском приложении, которое хочет извлечь и кешировать документы, а также отправить запросы или данные на сервер. Вы также узнаете, по каким правилам работает протокол.

Библиотеки клиентов Python

Протокол HTTP и всевозможные ресурсы данных, которые он предоставляет, уже давно интересуют программистов Python, как видно по длинному списку сторонних клиентов, которые заявляются как улучшенные альтернативы модуля urllib из стандартной библиотеки.

Сегодня, правда, одно стороннее решение стоит особняком. Оно не только вытеснило всех конкурентов, но и заменило urllib как универсальный инструмент для написания коммуникаций по HTTP на Python.

Речь о модуле requests, созданном Кеннетом Райцем (Kenneth Reitz) на базе логики объединения соединений в пулы из urllib3, написанной Андреем Петровым.

В этой главе мы еще будем рассматривать достоинства и недостатки urllib и requests в контексте возможностей HTTP. У них очень похожие базовые интерфейсы: они предоставляют вызываемый объект, который устанавливает HTTP-соединение, отправляет запрос и ждет заголовки ответа, прежде чем предоставить объект ответа, который отображает их. Тело ответа ставится в очередь на входящем соquete и читает данные только по запросу программиста.

В большинстве примеров в этой главе мы будем тестировать эти две библиотеки HTTP на <http://httpbin.org> — небольшом тестовом сайте, созданном Кеннетом Райцем, который можно выполнять локально, установив его через pip и запустив в контейнере WSGI, например Gunicorn (см. главу 10). Просто введите следующий код, чтобы запустить его на порте localhost 8000 и выполнять примеры из этой главы на своем компьютере, не обращаясь к общедоступной версии httpbin.org:

```
$ pip install gunicorn httpbin requests
```

```
$ gunicorn httpbin:app
```

Теперь мы сможем использовать `urllib` и `requests` для извлечения одной из его страниц, чтобы посмотреть, как их интерфейсы похожи на первый взгляд.

```
>>> import requests
>>> r = requests.get('http://localhost:8000/headers')
>>> print(r.text)
{
    "headers": {
        "Accept": "*/*",
        "Accept-Encoding": "gzip, deflate",
        "Host": "localhost:8000",
        "User-Agent": "python-requests/2.3.0 CPython/3.4.1 Linux/3.13.0- 34-generic"
    }
}
>>> from urllib.request import urlopen
>>> import urllib.error
>>> r = urlopen('http://localhost:8000/headers')
>>> print(r.read().decode('ascii'))
{
    "headers": {
        "Accept-Encoding": "identity",
        "Connection": "close",
        "Host": "localhost:8000",
        "User-Agent": "Python-urllib/3.4"
    }
}
```

Здесь уже видны два различия, и по ним можно представить себе, что нас ждет в этой главе. Модуль `requests` сразу объявляет, что поддерживает `gzip` и сжатие HTTP-ответов с помощью алгоритма `deflate`, а `urllib` не знает об этих форматах. Модуль `requests` корректно определил, как преобразовать HTTP-ответ из байтов в текст, а библиотека `urllib` вернула просто байты, предоставив нам самим декодировать их.

Предпринимались и другие попытки создания HTTP-клиентов Python, и многие из них были направлены на имитацию браузера. Их разработчики хотели не только работать с протоколом HTTP, о котором мы будем говорить в этой главе, но и реализовать идеи, которые мы обсудим в *главе 11*, объединяя структуру HTML, семантику его форм и правила, которым должен следовать браузер при отправке формы. Например, некоторое время была популярна библиотека `mechanize`.

Наконец, веб-страницы часто слишком сложны, так что с ними можно работать только через полноценные браузеры, а формы нередко работают благодаря аннотациям или корректировкам на JavaScript. У многих современных форм даже нет физической кнопки **Отправить**, и эта задача реализуется через скрипт. Технологии

управления в браузере оказались более эффективными, чем модуль `mechanize`. Некоторые из них мы обсудим в *главе 11*.

В этой главе мы поговорим о том, как устроен протокол HTTP и какие его функции доступны через библиотеки `requests` и `urllib`, а также какие существуют ограничения при использовании пакета `urllib` из стандартной библиотеки. Если вы, например, не можете устанавливать сторонние библиотеки, но хотите выполнять сложные операции с HTTP, просмотрите не только документацию по библиотеке `urllib`, но и статью о ней в блоге "Python Module of the Week", а также главу об HTTP в онлайн-книге "Dive Into Python".

<http://pymotw.com/2/urllib2/index.html#module-urllib2>

Эти материалы были написаны еще во времена Python 2, так что в них обсуждается библиотека `urllib2`, а не `urllib.request`, но все равно по ним можно составить представление об устаревшем объектно-ориентированном подходе, который используется в `urllib`.

Кадрирование, шифрование и порты

Для HTTP-сообщений открытым текстом по умолчанию используется порт 80. Порт 443 предназначен для клиентов, которые хотят сначала начать зашифрованное TLS-соединение (см. *главу 6*), а затем перейти на HTTP после установки шифрования. Эта вариация протокола называется HTTPS, где "S" означает "secure" (безопасный). Дальше обмен по HTTP осуществляется так же, как через незашифрованный сокет, только внутри зашифрованного канала.

В *главе 11* мы увидим, что выбор пользователя между HTTP и HTTPS, а также между стандартным и нестандартным портом обычно выражается в предоставляемом URL.

Цель TLS — проверить подлинность сервера, к которому подключается клиент, и если предоставлен клиентский сертификат, позволить серверу проверить подлинность клиента в ответ. Никогда не используйте клиента HTTPS, не проверяющего, что сертификат сервера соответствует имени хоста, к которому клиент пытается подключиться. Все клиенты в этой главе выполняют эту задачу.

Сначала клиент устанавливает HTTP-соединение, отправляя запрос с указанием документа. Когда весь запрос будет передан, клиент ждет полный ответ от сервера либо с ошибкой, либо с запрошенной информацией. Клиент не может отправить второй запрос по тому же соединению, пока не получит весь ответ. Во всяком случае, так работает HTTP/1.1.

Запрос и ответ используют одинаковые стандарты форматирования и кадрирования, и в HTTP важно поддерживать эту симметрию. Ниже приводится пример запроса и ответа, на который можно опираться, читая дальнейшее описание протокола:

```
GET /ip HTTP/1.1
```

```
User-Agent: curl/7.35.0 Host: localhost:8000 Accept: */*
```



```

HTTP/1.1 200 OK
Server: gunicorn/19.1.1
Date: Sat, 21 Sep 2020 00:18:00 GMT
Connection: close
Content-Type: application/json Content-Length: 27
Access-Control-Allow-Origin: *

Access-Control-Allow-Credentials: true
{
  "origin": "127.0.0.1"
}

```

Запрос представляет собой текстовый блок, который начинается с GET. Ответ начинается с указания версии (HTTP/1.1), а затем следуют три строки текста JSON под заголовками на пустой строке. Этот стандарт относится к запросу и ответу. Каждое HTTP-сообщение состоит из трех элементов.

- ◆ Первая строка в запросе, которая указывает метод и документ, и обратный код и описание в ответе. Символы возврата каретки и перевода строки (CR-LF, коды ASCII 13 и 10) обозначают конец строки.
- ◆ Имя, двоеточие и значение содержатся в одном или нескольких заголовках. Имена в заголовках нечувствительны к регистру. В конце каждого заголовка стоят символы CR-LF. Весь список заголовков заканчивается пустой строкой — четырьмя байтами CR-LF-CR-LF. Они составляют пару последовательностей окончания строки, между которыми ничего нет.
- ◆ Необязательное тело, которое может следовать после пустой строки, отделяющей заголовки, независимо от того, есть ли выше заголовки. Скоро вы увидите, что для кадрирования объекта существует много вариантов.

Первая строка и заголовки отделяются последовательностями CR-LF, а весь набор отделяется пустой строкой в конце. Сервер или клиент могут дойти до этого конца, выполняя `recv()`, пока не появится последовательность из четырех символов CR-LF-CR-LF. Поскольку ничто не указывает на длину строки и заголовков, многие серверы устанавливают разумные лимиты, чтобы злоумышленник не смог занять всю оперативную память, отправив заголовки бесконечной длины.

Если в сообщении есть тело, у нас имеются три варианта его кадрирования.

Чаще всего используется заголовок `Content-Length`, значение которого представляет десятичное число, указывающее длину тела в байтах. Это очень простой способ. Клиент может просто повторять операцию `recv()`, пока общее число байтов не будет равно указанной длине. Когда данные генерируются динамически, мы не всегда можем указать `Content-Length`, потому что длину данных невозможно определить до завершения процесса.

Если в заголовках указано `chunked` в качестве значения `Transfer-Encoding`, используется более сложный метод. Длина всего сообщения не передается заранее, но оно

поступает частями, перед каждой из которых указана ее длина. У каждого фрагмента есть шестнадцатеричное (а не десятичное, как в Content-Length) поле длины, два символа CR-LF, блок данных точно указанной длины и еще два символа CR-LF. Поток фрагментов заканчивается финальным фрагментом, который заявляет, что у него нулевая длина. Он содержит ноль, пару CR-LF и еще одну пару CR-LF.

Отправитель может вставить точку с запятой после длины фрагмента и перед CR-LF, а затем указать расширение для этого фрагмента. Отправитель может добавить в конец несколько финальных HTTP-заголовков, когда последний фрагмент отправил нулевую длину и символы CR-LF. Если вы реализуете HTTP самостоятельно, изучите эти детали в RFC 7230.

Сервер может указать, что соединение закрыто (Connection: close), отправить тело ответа, а затем закрыть TCP-сокеты. Это альтернатива Content-Length. При этом возрастает риск того, что клиент не сможет понять, почему закрыт сокет: потому что вся информация передана или потому что на сервере либо в сети возникла ошибка. Кроме того, снижается эффективность протокола, ведь клиенту придется подключаться заново для каждого запроса.

(По стандарту клиент не может отправить Connection: close, иначе он не получит ответ от сервера. Хотя, если помните, мы видели, что можно закрыть сокет в одном направлении (shutdown()) и при этом получать данные от сервера.)

Методы

Первое слово в HTTP-запросе показывает, какое действие клиент требует от сервера. Если сервер готов предоставить хорошо задокументированный API программам, которые захотят получить к нему доступ (обычно это JavaScript в браузере), существует два стандартных способа, GET и POST, и несколько менее распространенных методов.

Операции чтения и записи в HTTP осуществляются через два базовых метода: GET и POST.

Когда мы вводим HTTP URL в браузере, метод GET запрашивает документ, указанный в пути запроса. В нем не может быть тела. Серверы ни при каких обстоятельствах не должны разрешать клиентам редактировать данные с помощью этого подхода. Все параметры пути (подробнее об URL мы поговорим в *главе 11*) могут влиять только на возвращаемый документ, например ?q=python или ?results=10, и не могут запрашивать изменения на сервере. Поскольку GET не может менять данные, клиент может безопасно повторять этот запрос, если первая попытка не удалась, ответы GET можно кешировать (подробнее об этом чуть позже), а программы веб-скрейпинга (см. *главу 11*) могут посещать сколько угодно URL без риска создавать или удалять содержимое сайтов, по которым они проходят.

Если клиент хочет отправить на сервер новые данные, он отправляет POST. Традиционные онлайн-формы обычно используют POST, чтобы отправить запрос, если не просто копируют поля формы в URL. POST также используется в API, ориенти-

рованных на разработчика, чтобы отправлять новые документы, комментарии и строки базы данных. Поскольку повторение запроса POST может привести к повторному выполнению действия, например повторному платежу в интернет-магазине, результат запроса POST нельзя кешировать для использования в будущем или автоматически повторять, когда ответ не поступает.

Оставшиеся методы HTTP можно разделить на две группы: похожие на GET и похожие на POST. OPTIONS и HEAD — это методы из группы GET. Метод OPTIONS определяет, какие значения заголовка совместимы с данным путем, а метод HEAD велит серверу начать процесс подготовки к передаче ресурса, но затем останавливает процесс и передает только заголовки. Это позволяет клиенту проверять такие значения, как `ContentType`, не загружая все тело запроса.

PUT и DELETE похожи на POST, потому что могут вносить потенциально необратимые изменения в содержимое, хранящееся на сервере. Запросы PUT создают новые документы на сервере, которые будут храниться по пути, указанному в запросе, а запросы DELETE удаляют на сервере путь и содержимое по нему. Что интересно, эти два метода более безопасные, чем POST, т. к. являются идиempотентными, т. е. их можно повторять сколько угодно раз, потому что результат всегда будет одним и тем же.

Наконец, существует метод TRACE для отладки и метод CONNECT для переключения на другие протоколы (как мы увидим в *главе 11*, он используется для включения WebSocket).

Однако они используются нечасто и не связаны с доставкой документов, а это главная функция HTTP. Подробнее об этом можно почитать в стандарте.

Стоит отметить, что `urlopen()` в стандартной библиотеке выбирает метод HTTP невидимо для нас: POST, если вызывающий объект указывает параметр данных; в противном случае GET. Это не лучший подход, потому что выбор метода HTTP очень важен для защиты клиента и сервера. Гораздо лучше использовать `get()` и `post()` в библиотеке `requests`.

Хосты и пути

Изначально HTTP разрешал указывать в запросе только команду и маршрут.

```
GET /html/rfc7230
```

Это работало, пока на каждом сервере размещался только один веб-сайт, но затем стали появляться крупные HTTP-серверы, обслуживающие десятки и даже сотни сайтов. Как сервер может предсказать, какое имя хоста пользователь разместил в URL, если указан только путь, особенно для путей вроде `/`, которые мы видим почти на каждом сайте?

Нужен был хотя бы один заголовок — `Host`. В современных версиях в запросе также указывается версия протокола:

```
GET /html/rfc7230 HTTP/1.1 Host: tools.ietf.org
```

Если клиент не предоставил в URL заголовок Host с именем хоста, HTTP-сервер, скорее всего, выдаст ошибку. Обычно это 400 Bad Request (некорректный запрос). Ниже мы рассмотрим коды ошибок и их значения.

Коды состояний

Строка ответа, в отличие от строки запроса, начинается с версии протокола, а затем в ней указываются стандартный код состояния и неформальное описание состояния, которое будет отображаться пользователю и попадет в файл журнала. Когда все в порядке, мы получаем код состояния 200, а ответ выглядит примерно следующим образом:

```
HTTP/1.1 200 OK
```

Поскольку после кода идет неформальное описание, сервер может заменить OK на Okay, Yippee (ура), It Worked (все получилось) или даже текст на другом языке.

RFC 7231 предлагает почти два десятка кода состояний для общих и специфических сценариев. Изучите этот стандарт, если интересно. Коды 2xx обозначают успех, 3xx — перенаправление, 4xx — некорректность клиентского запроса, а 5xx — неожиданное событие по вине сервера.

В этой главе мы рассмотрим самые актуальные для нас коды.

- ◆ **200 OK.** Запрос выполнен успешно. Если это был POST, мы добились нужного эффекта.
- ◆ **301 Permanently Moved** (постоянно перемещено). Мы указали верный путь, но это не канонический путь к ресурсу (хотя раньше мог им быть), и клиенту следует запросить URL, указанный в заголовке Location в ответе. Если клиент захочет его кешировать, все будущие запросы смогут обходить старый URL и переходить сразу по новому.
- ◆ **303 Other** (другое). Клиент может узнать результат этого запроса, выполнив GET по URL, указанному в заголовке Location ответа. Последующие попытки получить доступ к этому ресурсу должны быть сделаны с указанием этого расположения. Этот статус очень важен при проектировании сайтов, как мы увидим в *главе 11*. Любая форма, отправленная с помощью POST, должна возвращать 303, чтобы фактическая страница, которую видит клиент, поступала через безопасную идемпотентную операцию GET.
- ◆ **304 Not Modified** (не изменено). Поскольку заголовки запросов указывают, что у клиента уже есть актуальная версия документа в кеше (*см. разд. "Валидация и кеширование" далее в этой главе*), тело документа не нужно включать в запрос.
- ◆ **307 Temporary Redirect** (временное перенаправление). Клиентский запрос (GET или POST) нужно повторить по другому URL, указанному в заголовке Location в ответе. Последующие попытки получить доступ к этому ресурсу должны быть сделаны с указанием данного расположения. Это позволяет доставлять формы на другой адрес, если сервер отключен или недоступен.

- ◆ **400 Bad Request** (некорректный запрос). Эта ошибка возникает из-за проблем с HTTP-запросом.
- ◆ **403 Forbidden** (запрещено). В запросе нет пароля, cookie или других идентификационных данных, по которым сервер может проверить, что у клиента есть разрешение на доступ к нему (подробнее об этом чуть позже в этой главе).
- ◆ **404 Not Found** (не найдено). Путь не соотносится с существующим ресурсом. Это самый известный код, потому что вместо кода 200 клиенты видят просто запрошенный документ.
- ◆ **405 Method Not Allowed** (метод не разрешен). Сервер распознает метод и путь, но этот метод не может использоваться с этим путем.
- ◆ **500 Server Error** (ошибка сервера). Сервер хочет выполнить запрос, но пока не может этого сделать из-за внутренних проблем.
- ◆ **501 Not Implemented** (не реализовано). Сервер не распознал команду HTTP.
- ◆ **502 Bad Gateway** (неверный шлюз). Сервер работает как шлюз или прокси (см. главу 10), но не может взаимодействовать с сервером, который должен предоставить ответ по этому пути.

У ответов с кодами состояния 3xx не должно быть тела, а ответы с кодами 4xx и 5xx обычно содержат понятное человеку описание ошибки. Стандартные страницы ошибок для определенного языка или фреймворка, на котором был создан сервер, часто не содержат полезной информации. Создатели серверов стараются создавать более информативные сообщения, чтобы помочь пользователям и разработчикам устранять ошибки.

При изучении конкретного клиента Python HTTP следует задать два важных вопроса о кодах состояний.

Первый вопрос: обрабатывает ли библиотека перенаправления автоматически? Если нет, мы должны будем отслеживать коды состояний 3xx и вручную переходить в расположение, указанное в заголовке Location. Низкоуровневый модуль `httplib` в стандартной библиотеке требует перенаправления вручную, а модуль `urllib` обрабатывает перенаправление автоматически в соответствии со стандартом. Библиотека `requests` также автоматизирует эти задачи, но у нее есть атрибут `history`, который включает все перенаправления до конечного места назначения.

```
>>> r = urlopen('http://httpbin.org/status/301')
>>> r.status, r.url
(200, 'http://httpbin.org/get')
>>> r = requests.get('http://httpbin.org/status/301')
>>> (r.status, r.url)
(200, 'http://httpbin.org/get')
>>> r.history
[<Response [301]>, <Response [302]>]
```

Мы можем отключить перенаправление с помощью ключевого слова при использовании библиотеки `requests`. В `urllib` это возможно, но гораздо сложнее.

```
>>> r = requests.get('http://httpbin.org/status/301',
... allow_redirects=False)
>>> r.raise_for_status()
>>> (r.status_code, r.url, r.headers['Location'])
(301, 'http://localhost:8000/status/301', '/redirect/1')
```

Если программа Python будет обнаруживать сбои 301 и пытаться избегать этих URL в будущем, это снизит нагрузку на запрашиваемые сервисы. Если программа хранит состояние, она сможет кэшировать сбои 301, чтобы не посещать эти страницы снова, или можно напрямую перезаписать URL там, где он хранится. Если URL запрашивается интерактивно, можно вывести сообщение о том, что страница переместилась.

Префикс `www` перед именем хоста — это одно из самых распространенных перенаправлений.

```
>>> r = requests.get('http://google.com/')
>>> r.url
'http://www.google.com/'
>>> r = requests.get('http://www.twitter.com/')
>>> r.url
'https://twitter.com/'
```

Два известных сайта применяют противоположные стратегии: один включает префикс `www` в официальное имя хоста, другой — нет. Они используют перенаправление, чтобы реализовать желаемый подход и избежать видимости того, что сайт доступен по двум разным URL. Если URL создаются из неверного имени хоста, мы будем выполнять два HTTP-запроса вместо одного для каждого запрашиваемого ресурса, если только приложение не запоминает эти перенаправления, чтобы их не пришлось повторять.

Также нужно обратить внимание на то, как HTTP-клиент уведомляет нас о попытке получить URL, если возвращен код состояния 4xx или 5xx. Метод `urlopen()` выдает для таких кодов исключение, поэтому наш код не может случайно обработать страницу с ошибкой, полученную от сервера, как обычные данные.

```
>>> urlopen('http://localhost:8000/status/500')
```

Traceback (most recent call last):

...

`urllib.error.HTTPError: HTTP Error 500: INTERNAL SERVER ERROR`

Когда `urlopen()` выдает исключение, он не может изучить данные ответа. Узнать о проблеме можно, изучив объект исключения, который одновременно является объектом ответа, с заголовками и телом.

```
>>> try:
... urlopen('http://localhost:8000/status/500')
```

```
... except urllib.error.HTTPError as e:
... print(e.status, repr(e.headers['Content-Type']))
500 'text/html; charset=utf-8'
```

Библиотека requests поступает еще интереснее: даже неверные коды состояния приводят к тому, что вызывающий клиент получает объект ответа без комментария. Затем клиент может либо проверить код состояния в ответе, либо выполнить метод `raise_for_status()`, который выдаст исключение для кода состояния 4xx или 5xx.

```
>>> r = requests.get('http://localhost:8000/status/500')
>>> r.status_code
500
```

```
>>> r.raise_for_status()
```

```
Traceback (most recent call last):
```

```
...
```

```
requests.exceptions.HTTPError: 500 Server Error: INTERNAL SERVER ERROR
```

Если мы не хотим делать проверку состояния при каждом запросе, можно создать функцию-обертку, которая будет выполнять проверку за нас.

Валидация и кеширование

В HTTP есть много продуманных методов избежать повторных запросов GET для часто используемых ресурсов, но все они работают только в том случае, если сервер добавляет заголовки к ресурсу, который их поддерживает. Разработчики сервера должны включать кеширование всегда, когда это целесообразно, потому что оно позволяет сократить сетевой трафик и нагрузку на серверы и при этом ускорить работу приложения. Все эти процессы подробно рассматриваются в RFC 7231 и 7232. В этом разделе приводится только самое общее их описание.

При добавлении заголовков в поддержку кеширования архитектор сервиса должен хорошо подумать, следует ли возвращать одинаковую страницу, если в двух запросах указаны идентичные пути. Есть ли в запросах что-то такое, из-за чего они могут вернуть два разных ресурса? Если да, сервис должен предоставить заголовок `Vary` в каждом ответе с указанием других заголовков, от которых зависит содержимое документа. Если нужно возвращать разные документы разным пользователям, можно использовать заголовки `Host`, `Accept-Encoding` и `Cookie`.

Если настроен заголовок `Vary`, уровней кеширования может быть несколько.

Некоторые ресурсы запрещено сохранять в кеше клиента, потому что они быстро устаревают. Мы можем предоставить пользователю выбор: следует ли сохранять копию ресурса на диск.

```
HTTP/1.1 200 OK
```

```
Cache-control: no-store
```

```
...
```


Если сервер выбирает кеширование, нужно предотвратить вероятность того, что клиент будет доставлять кешированную копию ресурса каждый раз, как пользователь будет ее запрашивать, пока она полностью не устареет. Если сервер использует определенный путь только для одной постоянной версии документа или изображения, ему не надо беспокоиться о том, что ресурсы хранятся в кеше бесконечно. Если, например, номер версии или хеш в конце URL увеличивается или изменяется каждый раз, как дизайнер публикует новую версию логотипа компании, любую версию логотипа можно отправлять с разрешением хранить ее сколько угодно времени.

Сервер может ограничить время хранения клиентской копии ресурса двумя способами. Он может определить дату и время окончания срока действия, после которых ресурс нельзя будет использовать без запроса к серверу.

```
HTTP/1.1 200 OK
```

```
Expires: Thu, 01 Dec 1999 16:00:00 GMT
```

```
...
```

Риск в том, что у клиента могут быть неправильно настроены часы, и тогда срок действия кешированной копии истечет раньше или позже. Более разумный метод — установить количество секунд, в течение которых ресурс хранится в кеше после получения. Он будет работать, если часы клиента не заблокированы.

```
HTTP/1.1 200 OK
```

```
Cache-control: max-age=3600
```

```
...
```

Два заголовка здесь позволяют клиенту использовать старую копию ресурса в течение неограниченного времени, не обращая к серверу повторно.

А что если сервер хочет сам выбирать — можно использовать кешированный ресурс или нужно получить новую версию? В этом случае клиенту придется делать HTTP-запрос каждый раз, как он захочет использовать этот ресурс. Это будет дороже, чем просто использовать кешированную копию, не передавая трафик по сети, но позволит сэкономить время, потому что серверу придется отправлять новую копию ресурса, только если старая копия действительно устарела.

Есть два способа заставить клиента выполнять проверку после каждого использования ресурса, при этом разрешая ему повторно использовать кешированную копию, если это возможно. Это так называемые *условные запросы*, поскольку тело ответа доставляется только в том случае, если будет обнаружено, что копия в кеше клиента устарела.

Первый механизм требует, чтобы сервер знал о последнем изменении ресурса. Если у ресурса есть файл в файловой системе, определить это довольно просто, но если ресурс берется из таблицы базы данных без журнала аудита или даты последнего изменения, это может быть трудно или даже невозможно. Сервер может включать эту информацию в каждый ответ, если она доступна.

```
HTTP/1.1 200 OK
```

```
Last-Modified: Tue, 15 Nov 1999 12:45:26 GMT
```

```
...
```


Если клиент хочет использовать кешированную копию ресурса, он может сохранить дату и отправить ее серверу при следующем запросе к этому ресурсу. Если сервер определит, что ресурс не изменился с последнего использования клиентом, он может отправить только заголовки и специальный код состояния 304 вместо тела.

GET / HTTP/1.1

If-Modified-Since: Tue, 15 Nov 1994 12:45:26 GMT

...

HTTP/1.1 304 Not Modified

...

Второй подход работает не с временем изменения, а с идентификатором ресурса. В этом случае серверу нужен способ установить уникальный тег для каждой версии ресурса, который будет гарантированно меняться на новое уникальное значение при каждом изменении ресурса. Можно использовать, например, контрольную сумму или UUID в базе данных. Когда сервер создает ответ, он должен включить тег в заголовке ETag.

HTTP/1.1 200 OK

ETag: "d41d8cd98f00b204e9800998ecf8427e"

...

Когда клиент, у которого в кеше хранится эта версия ресурса, хочет снова ее использовать, он может отправить запрос серверу и включить тег хешированной версии, чтобы узнать, не потеряла ли она актуальность.

GET / HTTP/1.1

If-None-Match: "d41d8cd98f00b204e9800998ecf8427e"

...

HTTP/1.1 304 Not Modified

...

ETag и If-None-Match используют кавычки с целью указания, что схема может выполнять более эффективные сравнения, чем сравнение двух строк на равенство. Если хотите узнать больше, см. раздел 3.2 в RFC 7232. Стоит отметить, что If-Modified-Since и If-None-Match помогают экономить время и сетевой трафик, потому что ресурс не приходится передавать дважды. Прежде чем клиент сможет получить доступ к ресурсу, приходится сделать круговой путь — до сервера и обратно.

Кеширование — очень эффективный инструмент, обеспечивающий производительность современных сайтов. При этом ни одна из рассмотренных клиентских библиотек Python не позволяет работать с кешированием. Библиотеки `urllib` и `requests` отвечают только за сетевые HTTP-запросы в реальном времени и не управляют кешем, который помог бы нам обойтись без сетевых запросов. Если нам нужна оболочка, которой можно указать локальное постоянное хранилище, чтобы она использовала заголовки `Expires` и `Cache-control`, даты изменения и `ETags`, чтобы сократить задержку и сетевой трафик, можно поискать сторонние решения.

Если вы настраиваете или обслуживаете прокси, вам также пригодится кеширование, но об этом мы поговорим в *главе 10*.

Кодирование содержимого

Между кодированием HTTP и кодированием содержимого существует заметная разница.

Кодирование протокола — это метод преобразования ресурса в тело HTTP-ответа. В итоге выбор кодировки для протокола не имеет никакого значения. Например, клиент получит один и тот же документ или изображение независимо от того, что использовалось для кадрирования: Content-Length или механизм chunked encoding. Байты могли быть доставлены напрямую или в сжатом виде для скорости, но ресурс будет выглядеть одинаково.

Кодирование протокола просто обортывает передачу данных, не меняя сами данные.

Современные браузеры поддерживают различные методы кодировки, но среди программистов наибольшей популярностью пользуется gzip. Клиент, который может принимать эту кодировку, должен объявить об этом в заголовке Accept-Encoding и подготовиться к чтению заголовка Transfer-Encoding в ответе, чтобы узнать, принял ли его сервер.

```
GET / HTTP/1.1
Accept-Encoding: gzip
...
HTTP/1.1 200 OK
Content-Length: 3913 Transfer-Encoding: gzip
...
```

Поскольку библиотека urllib не поддерживает этот метод, придется написать собственный код, чтобы создавать и обнаруживать эти заголовки, а также распаковывать тело ответа, если мы используем сжатие.

Если сервер отвечает допустимым значением Transfer-Encoding, библиотека requests сразу объявляет Accept-Encoding gzip, deflate и распаковывает содержимое. Благодаря этому сжатие выполняется автоматически и невидимо для пользователей requests, если сервер это поддерживает.

Согласование содержимого

Тип содержимого и кодировка содержимого видны конечному пользователю и клиентской программе, которая делает HTTP-запрос. Они определяют, какой формат файла будет использоваться для конкретного ресурса, а также какая кодировка будет применяться для преобразования текста в байты, если выбран текстовый формат.

Эти заголовки позволяют более старым браузерам, которые не поддерживают PNG, указывать, что они предпочитают GIF и JPG, а также с их помощью можно отображать ресурсы на языке, указанном пользователем. Вот пример того, как выглядят эти заголовки при создании современным браузером:

```
GET / HTTP/1.1
Accept: text/html;q=0.9,text/plain,image/jpg,*/*;q=0.8
```

Accept-Charset: unicode-1-1;q=0.8

Accept-Language: en-US,en;q=0.8,ru;q=0.6

User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/537.36 (KHTML)

...

Типы и языки, первыми указанные в заголовке, имеют самое высокое значение предпочтения: 1.0, затем это значение часто снижается до 0.9 или 0.8, чтобы сервер понимал, что это нелучшие варианты. Многие простые HTTP-сервисы и сайты полностью игнорируют эти заголовки, предпочитая использовать разные URL для разных версий ресурсов. Если сайт, например, поддерживает английский и французский языки, главная страница может быть доступна в двух вариантах: /en/index.html и /fr/index.html. На корпоративном сайте могут существовать два идентичных логотипа в форматах /logo.png и /logo.gif, и пользователю можно предоставить оба варианта для загрузки. Разные параметры запроса URL, например as?f=json и ?f=xml, часто указываются в документации для RESTful-реализации веб-сервиса (см. главу 10), чтобы можно было выбрать желаемое представление.

Однако HTTP задумывался не так.

Целью HTTP было предоставить для ресурса только один путь независимо от того, сколько форматов или человеческих языков используется для его отображения, а сервер должен выбирать ресурс по заголовкам согласования содержимого.

Почему мы часто игнорируем это согласование?

Для начала согласование содержимого предоставляет пользователю минимальный контроль над происходящим. Вернемся к сайту со страницами на английском и французском языках. Сервер не контролирует ситуацию, когда на сайте отображается язык в соответствии с заголовком Accept-Language, а пользователь хочет видеть страницу на другом языке. Он предложит пользователю изменить язык по умолчанию в настройках браузера. А если пользователь не может найти эти настройки? Или он использует общий компьютер и менять настройки запрещено?

Вместо того чтобы перекладывать выбор языка на браузер, который может быть плохо написанным, неудобным или нестабильным, многие сайты создают несколько наборов путей, по одному для каждого предоставляемого языка. Они могут оценивать заголовок Accept-Language при первом входе пользователя, чтобы автоматически выбрать наиболее вероятный язык. Однако у пользователя должна быть возможность выбрать другой вариант, если этот не подходит.

Кроме того, поскольку API HTTP-клиента (используемые JavaScript в браузере или предлагаемые другими языками в иных средах выполнения) часто затрудняют управление заголовками Ассерта, согласование контента игнорируется (или существует наряду с механизмом на основе URL, который принудительно возвращается к правильной версии содержимого). Когда мы размещаем компоненты управления в пути в URL, любой пользователь с самым базовым инструментом получения URL может вносить в него изменения.

Наконец, для согласования содержимого HTTP-серверы должны создавать или выбирать содержимое с учетом нескольких параметров. Можно подумать, что у логи-

ки сервера есть постоянный доступ к заголовкам Асцепт, но это не всегда так. Если убрать из уравнения согласование содержимого, серверное программирование становится гораздо проще. Зато с его помощью можно сократить количество возможных URL в сложных сервисах и при этом все равно предоставлять умным HTTP-клиентам способ получать содержимое, скорректированное с учетом предпочтений. Если планируете применять согласование содержимого, см. RFC 7231 с описанием синтаксиса различных заголовков Асцепт.

Наконец, давайте поговорим о строке User-Agent.

Она не предназначена для использования при согласовании содержимого. Она должна была стать временным обходным путем для ограничений браузера. Иными словами, это был способ предоставить некоторым клиентам тщательно подготовленные обновления, пока другие клиенты могли обращаться к странице без проблем.

Однако разработчики приложений при поддержке колл-центров быстро поняли, что можно запретить доступ к сайту всем браузерам, кроме определенных, чтобы решить проблемы с доступностью и сократить число обращений в центр поддержки. В результате "гонки вооружений" между клиентами и браузерами появились огромные строки User-Agent, как описано в статье <http://webaim.org/blog/user-agent-string-history/>.

Обе клиентские библиотеки, urllib и requests, позволяют включать любые заголовки Асцепт. Также обе они поддерживают разработку клиента, который автоматически использует предпочитаемые заголовки. Эта возможность напрямую включена в концепцию Session, которая используется в библиотеке requests.

```
>>> s = requests.Session()
```

```
>>> s.headers.update({'Accept-Language': 'en-US,en;q=0.8'})
```

Если не указано иное значение, все последующие вызовы к методам, вроде s.get(), будут использовать это значение по умолчанию для заголовка. У библиотеки urllib есть свои способы настроить обработчики по умолчанию, но они сложнее и используют объектно-ориентированный подход, так что советую изучить документацию.

Тип содержимого

После проверки всех заголовков Асцептс от клиента и представления ресурса сервер меняет заголовок Content-Type в исходящем ответе.

Тип содержимого выбирается из списка типов MIME, которые уже разработаны для мультимедийного содержимого, отправляемого по электронной почте (см. главу 12). Самые распространенные форматы: text/plain, text/html, image/gif, image/jpg и image/png. Документы можно отправлять в разных форматах, например application/pdf. Тип содержимого application/octet-stream назначается серии байтов, которым не нужна дальнейшая интерпретация. При использовании заголовка Content-Type через HTTP следует помнить об одном важном моменте. Если главный тип (то, что указано слева от косой черты) — text (текст), у сервера есть не-

сколько вариантов кодирования символов для передачи клиенту. Он объявляет о своих предпочтениях, добавляя точку с запятой и кодировку символов, используемую для преобразования текста в байты, в заголовке Content-Type.

Content-Type: text/html; charset=utf-8

Получается, что мы не можем просто сравнить заголовок Content-Type со списком типов MIME, пока не найдем точку с запятой и не разделим заголовок на два. Большинство библиотек нам в этом не помогут. Если мы создадим код, который должен проверять тип содержимого, нужно будет провести разделение по точке с запятой при использовании библиотек urllib и requests (requests хотя бы использует параметр кодировки типа содержимого, если мы запросим объект ответа для уже декодированного атрибута текста).

Библиотека WebOb Йена Бикинга (Ian Bicking) — единственная в этой книге, которая позволяет работать с типом содержимого и кодировкой отдельно по умолчанию. В объекте ответа есть отдельные атрибуты типа содержимого и кодировки, которые объединяются через точку с запятой в заголовке Content-Type в соответствии со стандартом.

Аутентификация по HTTP

Аутентификация проверяет, поступает ли запрос от клиента, у которого есть разрешения делать такие запросы. Например, когда мы звоним в банк, у нас спрашивают адрес и персональные данные, чтобы убедиться, что это действительно владелец счета. Точно так же HTTP-запрос часто требует подтверждения подлинности устройства или пользователя.

Если сервер не может проверить вашу подлинность или с вами все в порядке, но у вас нет разрешения на этот ресурс, он объявит об этом с помощью кода состояния 401 Not Authorized (не авторизовано).

Поскольку ресурсы созданы исключительно для людей, многие реальные HTTP-серверы не возвращают 401. Если мы попытаемся извлечь ресурс без необходимого идентификатора, скорее всего, мы получим ошибку 303 See Other (прочее) на странице входа. Это полезная информация для людей, но не для программ Python, которым важно видеть разницу между сбоем аутентификации и безобидным перенаправлением на другой ресурс. Поскольку каждый HTTP-запрос отличается от остальных, в том числе следующих за ним, информация для аутентификации должна включаться в каждый запрос на том же сожете. Это дает независимость, благодаря которой прокси-серверы и балансировщики нагрузки могут распределять HTTP-запросы по любому количеству серверов, даже если они поступают по одному сокету. Для того чтобы узнать больше о самых современных методах аутентификации в HTTP, см. RFC 7235.

Итак, сначала Basic Auth (базовая аутентификация) требует, чтобы сервер включал строку с областью (realm) в заголовки 401 Not Authorized. Поскольку браузер может отслеживать соответствие паролей и областей, один браузер способен зашиф-

ровать разные части дерева документа отдельными паролями. Затем клиент отправляет еще один запрос с заголовком `Authorization`, который включает имя пользователя и пароль (в кодировке Base64) и ожидает ответ 200.

```
GET / HTTP/1.1
```

```
...
```

```
HTTP/1.1 401 Unauthorized
```

```
WWW-Authenticate: Basic realm="engineering team"
```

```
...
```

```
GET / HTTP/1.1
```

```
Authorization: Basic YnJhbmRvbjphdGlnZG5nbmF0d3dhaA==
```

```
...
```

```
HTTP/1.1 200 OK
```

```
...
```

Сейчас передавать логин и пароль открытым текстом не стоит, но раньше не было беспроводных сетей, а коммутаторы были физическими, а не виртуальными, т. е. не такими уязвимыми. Когда архитекторы протоколов начали продумывать риски, был разработан новый подход — дайджест-аутентификация доступа (digest access authentication), при котором сервер отправляет значение, а клиент отвечает хешем MD5, куда входят значение и пароль. Правда, при таком подходе имя пользователя по-прежнему передается открытым текстом. Все данные формы тоже открыты, как и ресурсы, возвращенные с сайта. Злоумышленник может использовать атаку посредника, убедив вас подписать значение, только что полученное от сервера, а затем с его помощью олицетворять вас. Когда мы хотим посмотреть остаток на счете в банке или указать данные карты в интернет-магазине, мы рассчитываем на полную безопасность. В результате был создан SSL для разработки HTTPS, а затем его заменил TLS, который мы рассматривали в *главе 6*.

В теории внедрение TLS позволяло обойти проблему с Basic Auth. Сегодня она используется во многих простых API и веб-приложениях, защищенных HTTPS. Requests поддерживает Basic Auth с помощью одного ключевого слова, а urllib поддерживает ее, только если мы создадим последовательность объектов для установки в средстве открытия URL (см. документацию).

```
>>> r = requests.get('http://example.com/api/',
... auth=('bpbonline'))
```

Можно создать объект `Session` в requests для аутентификации, чтобы не пришлось прописывать ее для каждого `get()` и `post()`.

```
>>> s = requests.Session()
>>> s.auth = 'bpbonline'
>>> s.get('http://httpbin.org/basic-auth/bpbonline')
<Response [200]>
```

Помните, что этот метод, реализуемый модулем requests или другими библиотеками, не является полноценным протоколом. Предоставленное имя пользователя и пароль не относятся к определенной области. Поскольку имя пользователя и пароль

предоставляются в одностороннем порядке в запросе, и никто не проверяет, нужны ли они серверу, мы не получаем ответ 401, который может предоставить область. Ключевое слово `auth` или аналогичная настройка `Session` — это просто способ установить заголовок `Authorization` без кодирования в `Base64`.

Многие разработчики предпочитают этот простой подход полноценному протоколу на основе области. Единственная цель — проверить подлинность запросов `GET` и `POST` к API независимо от пользователя или приложения, которое делает запрос. Для этого удобно использовать заголовок `Authorization`. Есть и еще одно преимущество: мы не тратим время и пропускную способность на получение исходного кода 401, когда у клиента есть основания полагать, что потребуется пароль.

Библиотека `requests` не поможет, если мы работаем с очень старой системой, которая требует различных паролей для разных областей на одном сервере. В этом случае нам пришлось бы самим следить за использованием правильного пароля с правильными URL. Это редкий случай, когда `urllib` может сделать то, чего не может `requests`. Правда, я никогда не слышал, чтобы кто-то жаловался на этот недостаток в `requests`, и это показывает, как редко сейчас используется `Basic Auth`.

Файлы cookie

Сегодня аутентификация через HTTP используется редко.

Почему? Обычно дизайнеры сайтов создают персонализированную страницу входа, которая соответствует их правилам взаимодействия с пользователем. Когда у браузера запрашивают аутентификацию через HTTP, он отображает скучное всплывающее окно, которое содержит мало информации и не вписывается в концепцию сайта. Более того, если у пользователя не получилось правильно ввести имя пользователя и пароль, окно будет возникать снова и снова, и пользователь не будет понимать, что именно пошло не так и как это исправить.

К счастью, на выручку пришли файлы `cookie`.

С точки зрения клиента, `cookie` — это непрозрачная пара "ключ — значение", которую можно включить в каждый успешный ответ сервера, полученный клиентом.

```
GET /login HTTP/1.1
```

```
...
```

```
HTTP/1.1 200 OK
```

```
Set-Cookie: session-id=d41d8cd98f00b204e9800998ecf8427e; Path=/
```

```
...
```

Клиент добавляет имя и значение в заголовке `Cookie` во все последующие запросы к этому серверу.

```
GET /login HTTP/1.1
```

```
Cookie: session-id=d41d8cd98f00b204e9800998ecf8427e
```

```
...
```


Благодаря этому появилась возможность предлагать страницы входа, создаваемые сайтами. Если пользователь неправильно ввел логин или пароль, сервер может отобразить подсказки или ссылки, отформатированные в стиле сайта. Когда форма успешно отправлена, клиент может получить cookie, созданный специально для того, чтобы подтверждать подлинность пользователя в последующих запросах.

Даже страница входа, которая не является настоящей веб-формой, а использует Ajax, чтобы мы оставались на той же странице (*см. главу 11*), может использовать cookie, если API размещен по тому же имени хоста. Когда вызов API для входа подтверждает имя пользователя и пароль и возвращает 200 OK с заголовком Cookie, все последующие запросы к тому же сайту (не только вызовы API, но и запросы страниц, изображений и данных) предоставляют cookie, чтобы сервер знал, что они исходят от пользователя, прошедшего аутентификацию.

Следует отметить, что cookie должны быть непрозрачными. Это должны быть случайные строки UUID, направляющие сервер к записи базы данных, которая содержит имя пользователя, или зашифрованные строки, которые может расшифровать только сервер, чтобы узнать идентификатор пользователя. Если бы пользователь мог прочесть cookie, например, если бы cookie содержал значение THIS-USER-IS-bpbonline, он мог бы изменить его значение и отправить со следующим запросом, чтобы выдать себя за другого пользователя, имя которого он знает или может вывести.

Реальные заголовки Set-Cookie могут быть еще более сложными, чем в этом примере. Подробно это описано в RFC 6265. Тут нужно сказать пару слов о безопасности. При отправке незашифрованных запросов к сайту HTTP-клиент не должен отправлять cookie, иначе любой, кто подключен к той же сети Wi-Fi в кафе, сможет видеть значение этого cookie и олицетворять пользователя. Некоторые сайты размещают на вашем компьютере cookie, когда вы посещаете их. Это позволяет отслеживать ваши посещения. Собранную информацию можно использовать для таргетированной рекламы в браузере или даже записать в постоянную историю аккаунта, если позже вы войдете под своим именем пользователя.

Без cookie не получилось бы отследить ваш идентификатор и доказать, что вы прошли аутентификацию, а значит, многие HTTP-сервисы, взаимодействующие с пользователем, не работали бы. Для отслеживания cookie с помощью urllib требуется объектно-ориентированный подход. Подробности см. в документации.

Cookies автоматически отслеживаются в requests, если Session корректно создается и используется.

Поддержание соединения и httplib

Если соединение уже открыто, можно пропустить тройное рукопожатие для установки TCP-соединения (*см. главу 3*), что позволило разрешать HTTP-соединениям оставаться открытыми, пока браузер получает HTTP-ресурс, затем JavaScript и, наконец, стили CSS и изображения. Стоимость установки нового соединения возросла

в связи с развитием TLS (см. главу 6), который рекомендуется для всех HTTP-соединений, и это только повысило ценность повторного использования соединений.

Теперь по умолчанию HTTP-соединение остается открытым после запроса для версии HTTP/1.1. Этот параметр может быть указан клиентом или сервером. Если они планируют разорвать соединение после завершения запроса, соединение можно закрыть. В противном случае можно использовать одно TCP-соединение, пока клиент не получит с сервера столько ресурсов, сколько пожелает. Браузеры часто удерживают четыре или больше TCP-соединений для одного сайта, чтобы загрузить страницу и все ее данные и изображения одновременно и как можно быстрее показать их пользователю. Если вам интересны детали реализации, см. раздел 6 в RFC 7230, где подробно описан механизм контроля соединения.

К сожалению, модуль `urllib` не разрешает повторно использовать соединение. Только низкоуровневый модуль `httplib` в стандартной библиотеке позволяет делать два запроса по одному соединению.

```
>>> import http.client
>>> h = http.client.HTTPConnection('localhost:8000')
>>> h.request('GET', '/ip')
>>> r = h.getresponse()
>>> r.status
200
>>> h.request('GET', '/user-agent')
>>> r = h.getresponse()
>>> r.status
200
```

Когда мы просим выполнить еще один запрос, зависший объект `HTTPConnection` не вернет ошибку, а создаст новое TCP-соединение на замену предыдущему. Версия того же объекта, защищенная TLS, предоставляется классом `HTTPSConnection`. Объект `Session` в библиотеке `requests`, с другой стороны, поддерживается сторонним пакетом `urllib3`. Он отслеживает открытые соединения с HTTP-серверами, с которыми вы недавно взаимодействовали, и пытается автоматически использовать их повторно, когда мы запрашиваем другой ресурс с того же сайта.

Резюме

Протокол HTTP используется для извлечения ресурсов, у которых есть имя хоста и путь. Клиент `urllib` в стандартной библиотеке работает в простых сценариях, но у него не так много возможностей, как у `requests` — библиотеки Python, которая стала универсальным инструментом для программистов, создающих код для передачи информации в Интернете.

HTTP использует базовую сетевую схему клиентского запроса и ответа: строка информации, за которой следуют заголовки с именем и значением, затем пустая стро-

ка и тело (необязательно) с разными кодировками и разделителями. Клиент начинает общение, отправляя запрос, а затем ждет ответ от сервера. Самые распространенные методы HTTP: GET для извлечения ресурса и POST для доставки обновленных данных на сервер. Есть и другие методы, но все они работают по принципу GET или POST. Каждый ответ от сервера включает код состояния, который указывает, был ли запрос успешным или нет либо нужно ли перенаправить клиента на другой ресурс для выполнения запроса. HTTP состоит из нескольких уровней. Заголовки кеширования дают возможность кешировать и повторно использовать ресурс на клиенте, не загружая его снова, если он не изменился. Это позволяет значительно повысить производительность загруженных сайтов.

Согласование содержимого помогает корректировать форматы данных и языки отображения в зависимости от предпочтений клиента и пользователя, но есть проблемы, из-за которых этот подход используется нечасто. Встроенная аутентификация HTTP плохо подходит для интерактивного использования, поэтому ее заменили пользовательские страницы входа и cookie, а Basic Auth по-прежнему иногда используется для API, защищенных TLS. По умолчанию соединения HTTP/1.1 могут сохраняться и использоваться повторно, и библиотека requests поддерживает эти возможности. В следующей главе мы применим эти знания на практике и создадим сервер.

Серверы для работы с HTTP

Каким образом программа Python может отвечать на HTTP-запросы как сервер? В *главе 7* мы рассмотрели принципы параллелизма и работы сокетов для создания сервера на основе TCP. Поскольку протокол HTTP очень популярен, существуют готовые решения для всех ключевых паттернов сервера, но вряд ли вы когда-нибудь будете работать на таком низком уровне.

Хотя это возможно даже из командной строки.

```
$ python3 -m http.server
```

```
Serving HTTP on 0.0.0.0 port 8000 ...
```

Для поставки файлов из файловой системы этот сервер соблюдает правила из 1990-х годов. Путь в HTTP-запросе преобразуется в путь в локальной файловой системе для выполнения поиска. Сервер будет обращаться только к файлам в текущем рабочем каталоге или под ним. Файлы предоставляются как обычно. Когда мы указываем каталог, сервер отправляет содержимое файла `index.html`, если он существует, или динамически создает список файлов внутри него. Когда раньше мне нужно было передать данные между компьютерами, а специфические протоколы передачи файлов были недоступны, меня выручал небольшой веб-сервер на Python. А если мы хотим создать собственную программу, которая будет отвечать на HTTP-запросы?

Мы ответим на этот вопрос за две главы в контексте кода для предоставления документов или API для программирования. В *главе 11* мы поговорим об Интернете и методах отображения HTML-страниц и взаимодействия с браузером пользователя.

Содержание главы

- ◆ Стандарт WSGI.
- ◆ Серверные фреймворки для асинхронной обработки.
- ◆ Прямые и обратные прокси.
- ◆ Четыре архитектурных стиля.
- ◆ Python на Apache.
- ◆ HTTP-серверы на Python.

- ◆ Преимущество обратных прокси.
- ◆ Платформа как услуга.
- ◆ REST и паттерны GET и POST.
- ◆ WSGI без фреймворка.
- ◆ Резюме.

Цель

В этой главе мы посмотрим на модули из стандартной библиотеки и сторонние инструменты, а также обсудим архитектуру и развертывание сервера.

Стандарт WSGI

На заре HTTP-программирования многие сервисы Python создавались как базовые скрипты CGI, которые вызывались один раз на входящий запрос. Сервер разбивал HTTP-запрос на фрагменты и хранил их в переменных окружения скрипта CGI. Программисты Python могли просмотреть их напрямую и выдать HTTP-ответ в стандартный поток вывода или использовать модуль `cgi` в стандартной библиотеке. Поскольку запуск нового процесса для каждого входящего HTTP-запроса заметно замедлял производительность сервера, среды выполнения языков стали создавать собственные HTTP-серверы.

У Python в стандартной библиотеке есть модуль `http.server`, с помощью которого мы можем создавать собственные сервисы, добавляя методы `do_GET()` и `do_POST()` к базовым подклассам `BaseHTTPRequestHandler`.

Некоторые программисты хотели предоставлять динамические страницы через веб-сервер, который так же мог выдавать статические материалы, вроде изображений и таблицы стилей. В результате был создан `mod_python` — модуль Apache, который позволял правильно зарегистрированным функциям Python предоставлять пользовательские обработчики Apache для аутентификации, журналирования и содержания. У Apache был уникальный API. Обработчики Python получали определенный объект запроса Apache в качестве аргумента и могли использовать специальные функции модуля `apache` для взаимодействия с веб-сервером. Приложения, созданные с помощью `mod_python`, были совсем не похожи на приложения, написанные с помощью CGI или `http.server`.

Получилось, что каждое приложение для работы с HTTP, написанное на Python, было привязано к одному методу коммуникации с веб-сервером. Для работы с `http.server` сервис, созданный для CGI, приходилось частично переписывать, и оба варианта нужно было менять, чтобы они работали с Apache. Это затрудняло перевод веб-сервисов Python на новые платформы. В PEP 333 был предложен новый стандарт — WSGI.

Как сказал Дэвид Уилер (David Wheeler): "В информатике все сложности можно разрешить с помощью дополнительного уровня косвенности". Стандарт WSGI (Web Server Gateway Interface) как раз и предоставлял такой уровень косвенности, который требовался сервису Python HTTP для работы с любым веб-сервером. Он определял соглашение о вызовах, которое (если бы его приняли все крупные веб-серверы) позволило бы подключать к любому веб-серверу низкоуровневые сервисы и целые веб-фреймворки. Попытка применять WSGI повсеместно сразу же оказалась успешной, и сейчас это механизм по умолчанию, с помощью которого Python взаимодействует с HTTP.

Приложение WSGI определено стандартом как вызываемый объект с двумя аргументами. В листинге 10.1 показан пример такого вызываемого объекта в виде базовой функции Python. (Альтернативные варианты — класс Python, еще один тип вызываемого объекта, или даже экземпляр класса с методом `__call__()`.) Первый параметр, `environ`, получает словарь, который содержит более полную версию предыдущего набора переменных окружения CGI. Второй параметр — вызываемый объект с именем `start_response()`, который приложение WSGI должно использовать для объявления заголовков ответа. Приложение может начать отправку байтовых строк (если это генератор) или вернет итерируемый объект, который выдает байтовые строки при итерации после вызова (вернуться может простой список Python).

Листинг 10.1. Простой HTTP-сервис как клиент WSGI

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Простой HTTP-сервис, созданный напрямую по низкоуровневым спецификациям WSGI.

from pprint import pformat
from wsgiref.simple_server import make_server

def app(environ, start_response):
    headers = {'Content-Type': 'text/plain; charset=utf-8'}
    start_response('200 OK', list(headers.items()))
    yield 'Here is the WSGI environment:\r\n\r\n'.encode('utf-8')
    yield pformat(environ).encode('utf-8')

if __name__ == '__main__':
    httpd = make_server('', 8000, app)
    host, port = httpd.socket.getsockname()
    print('Serving on', host, 'port', port)
    httpd.serve_forever()
```

По листингу 10.1 может показаться, что работать с WSGI очень просто, но это только потому, что здесь он работает упрощенно. При реализации спецификации на

стороне сервера код будет сложнее, потому что он должен быть готов к работе с приложениями, которые используют все возможности этого стандарта. Подробности о версии WSGI для Python 3 см. в PEP 3333.

После выпуска WSGI стала набирать популярность идея использовать ряд оберток из WSGI в качестве промежуточного слоя для будущих HTTP-сервисов на Python. Одна обертка могла бы обеспечивать аутентификацию. Прежде чем возвращать страницу 500 Internal Server Error (внутренняя ошибка сервера), еще одна обертка могла бы перехватывать исключения и заносить их в журнал. Еще обертка могла бы перенаправлять пользователя или сервис, указавший устаревший URL, на более актуальные страницы с помощью Diazo (этот проект еще существует). Хотя некоторые разработчики продолжают создавать и использовать промежуточный слой в WSGI, большинство программистов Python сейчас применяют его только для соединения приложения или фреймворка с веб-сервером, который ожидает входящие HTTP-запросы.

Серверные фреймворки для асинхронной обработки

Остался, правда, один паттерн, который не затронула революция, произведенная WSGI: асинхронные серверы, допускающие сопрограммы или зеленые потоки.

Поскольку вызываемый объект WSGI работает со стандартным многопоточным или многопроцессорным сервером, он должен блокироваться во время операций ввода-вывода. WSGI не предоставляет способ, с помощью которого вызываемый объект мог бы вернуть управление главному потоку сервера, чтобы он мог работать с другими вызываемыми объектами, которые его ждут. (В *главе 7* мы увидели, как асинхронный сервис разделяет свою логику на короткие неблокирующие фрагменты кода.) В результате для каждого серверного фреймворка, поддерживающего асинхронность, требуется отдельный набор правил для написания веб-сервисов. Эти паттерны могут различаться по простоте и удобству, но часто они обрабатывают парсинг входящих HTTP-запросов и предоставляют дополнительные возможности, например автоматическое распределение URL и подключение к базе данных (см. *главу 11*).

Вот почему этот раздел посвящен серверным фреймворкам. Проекты, которые экспериментируют с модулем `async` в Python, должны сначала создать веб-сервер HTTP на своем движке, а затем разработать соглашение о вызовах для передачи информации из запросов в код. В отличие от экосистемы WSGI, нельзя по отдельности выбрать асинхронный HTTP-сервер и веб-фреймворк. Обычно они оба входят в один пакет. Сервер Twisted поддерживает различные обработчики протоколов и предоставляет собственный набор правил для разработки веб-сервисов. Facebook создал и представил свой движок Tornado, который не поддерживает целый ряд протоколов, а сосредотачивается исключительно на производительности HTTP. Twisted не поддерживает тот же набор соглашений о вызовах. Есть еще проект Eventlet, чьи зеленые потоки являются неявно асинхронными, но явно не передают управление при каждой транзакции ввода-вывода. Он позволяет писать вызываемые

мые объекты, которые выглядят как традиционный WSGI, но при этом передают управление при попытке выполнить блокирующие действия.

Гвидо ван Россум, создатель Python, продвигал новый модуль `asyncio` в Python 3.4 (см. главу 7), чтобы предоставить стандартизированный интерфейс для разных реализаций цикла обработки событий для интеграции в различные фреймворки асинхронного протокола. Это помогает объединить разнородные низкоуровневые циклы обработки событий, но напрямую не влияет на тех, кто хочет разрабатывать асинхронные HTTP-сервисы, потому что им не предоставляется API для работы с HTTP-запросами и ответами. Если вы разрабатываете HTTP-сервис, который использует движок асинхронной обработки, вроде `asyncio`, `Tornado` или `Twisted`, помните, что надо выбрать и HTTP-сервер, и фреймворк, который будет отвечать за парсинг запросов и составление ответов. Использовать серверы и фреймворки из разных наборов нельзя.

Прямые и обратные прокси

HTTP-прокси, прямой или обратный, — это HTTP-сервер, который получает входящие запросы и в некоторых случаях превращается в клиента, делающего исходящий HTTP-запрос к серверу за ним, прежде чем вернуть ответ исходному клиенту. Подробную информацию о прокси и о том, как HTTP предвосхищает их потребности, см. в разделе 2.3 в RFC 7230: <https://tools.ietf.org/html/rfc7230#section-2.3>.

На заре веб-технологий чаще всего использовались *прямые прокси*. Компания может предоставить HTTP-прокси, чтобы браузеры сотрудников обращались к удаленным серверам через него, а не напрямую. Если в начале рабочего дня сотня сотрудников запросит логотип Google, прокси может запросить у Google логотип один раз, а затем кешировать его для обслуживания последующих запросов. Если заголовки `Expires` и `Cache-Control` настроены должным образом, запросы будут обслуживаться быстрее. С повсеместным распространением TLS для защиты конфиденциальности и учетных данных пользователей, прямые прокси больше невозможно использовать. Прокси не может проверить или кешировать запрос, который он не понимает.

Обратные прокси часто применяются в крупных HTTP-сервисах. Обратный прокси используется как часть веб-сервиса и полностью скрыт от HTTP-клиентов. Клиенты считают, что обращаются к **python.org**, а на самом деле взаимодействуют с обратным прокси. Если серверы **python.org** указывают заголовки `Expires` и `Cache-Control`, прокси может предоставлять много статических и динамических страниц напрямую из кеша. Поскольку HTTP-запросы доставляются на серверы, только если ресурс невозможно кешировать или срок его хранения в кеше истек, обратный прокси может обрабатывать большую часть нагрузки по общению с сервисом.

Терминация TLS должна выполняться обратным прокси, и это должен быть сервис, которому принадлежат сертификат и закрытый ключ сервиса, для которого он выступает в качестве прокси. Прокси не сможет выполнять кеширование и перенаправление, если не будет изучать каждый входящий HTTP-запрос.

Когда мы используем обратный прокси, будь то фронтенд веб-сервер, вроде Apache и nginx, или специальный демон, например Varnish, заголовки, связанные с кешированием, такие как Expires и Cache-Control, особенно важно использовать правильно. Они передают информацию между уровнями нашего сервиса, а не только браузеру конечного пользователя.

Обратные прокси могут помочь даже с данными, которые, как вы думаете, не следует кешировать, например страница заголовков или запись журнала событий, если вы не против, что они будут запаздывать на несколько секунд. В конце концов, извлечение ресурса в большинстве случаев занимает долю секунды. Так ли страшно, что ресурс будет на секунду старше? Допустим, мы задаем заголовок Cache-Control для критически важного канала или журнала событий, который получает сотню запросов в секунду, указывая, что возраст записей не должен превышать одну секунду. Обратный прокси потенциально может снизить нагрузку на сервер в сто раз: нужно будет просто извлекать ресурс один раз в начале каждой секунды, а потом поставлять всем клиентам результат из кеша.

Если вы планируете создать и реализовать крупный HTTP-сервис за прокси, см. подробную информацию о принципах работы кеширования HTTP и ожидаемых преимуществах в RFC 7234. Существуют специальные параметры, например проху-revalidate и s-maxage, явным образом нацеленные на промежуточные кеши, вроде Varnish, а не на HTTP-клиент конечного пользователя.

Содержание страницы часто зависит не только от пути и метода, но и, например, от заголовка Host, идентификатора пользователя, который делает запрос, и иногда заголовков, определяющих, какой тип контента поддерживает клиент. См. описание заголовка Vary в разделе 7.1.4 из RFC 7231, а также в главе 9. По очевидным причинам для корректного поведения часто требуются файлы cookie.

Четыре архитектурных стиля

Архитекторы могут придумать сколько угодно сложных методов скомпоновать HTTP-сервис из небольших элементов, но сообщество Python договорилось о четырех главных подходах (рис. 10.1). Какие есть варианты развернуть HTTP-сервис, если мы написали код Python для создания динамического содержимого и выбрали API или фреймворк с поддержкой WSGI?

- ◆ Использовать сервер, написанный на Python, и вызывать конечную точку WSGI из кода. Сервер Gunicorn сейчас самый популярный, но существуют и специальные серверы для Python, которые можно применять в рабочем окружении. Например, проверенный сервер CherryPy по-прежнему иногда используется в проектах, пока Flup продолжает набирать пользователей. (Если речь не о сервисе с небольшой нагрузкой, который будет использоваться только внутри организации, лучше избегать прототипов, вроде wsgiref.) Если вы используете движок асинхронного сервера, нужно запустить сервер и фреймворк в одном процессе.

- ◆ Использовать Apache с модулем `mod_wsgi`, чтобы код Python выполнялся в отдельном `WSGIDaemonProcess`. При этом мы получаем гибридное решение, в котором два языка работают на одном сервере. Статические ресурсы поступают от движка Apache на C, а динамические пути проходят через `mod_wsgi`, который затем вызывает интерпретатор Python, чтобы выполнить код приложения. (Этот вариант недоступен для асинхронных веб-фреймворков, поскольку WSGI не позволяет приложению моментально передать управление и выполнить работу позже.)
- ◆ За веб-сервером запустить HTTP-сервер на Python, например Gunicorn (или аналогичный), который будет предоставлять статические файлы напрямую, одновременно выступая как обратный прокси для динамических ресурсов, созданных на Python. Для этого часто используются фронтенд-серверы Apache и nginx. Если приложение Python не помещается на одном компьютере, они могут выполнять балансировку нагрузки по нескольким бэкенд-серверам.
- ◆ Создать третий уровень, взаимодействующий с внешним миром, запустить HTTP-сервер на Python за Apache или nginx, которые, в свою очередь, будут находиться за чистым обратным прокси вроде Varnish. Обратные прокси можно распространить на глобальном уровне, чтобы кешированные ресурсы предоставлялись из расположений рядом с клиентом, а не только с одного континента. Fastly и другие сети доставки содержимого разворачивают множество серверов Varnish в центрах обработки данных по всему миру, а затем с их помощью предоставляют готовый сервис для терминирования внешних TLS-сертификатов и запросов к центральным серверам.

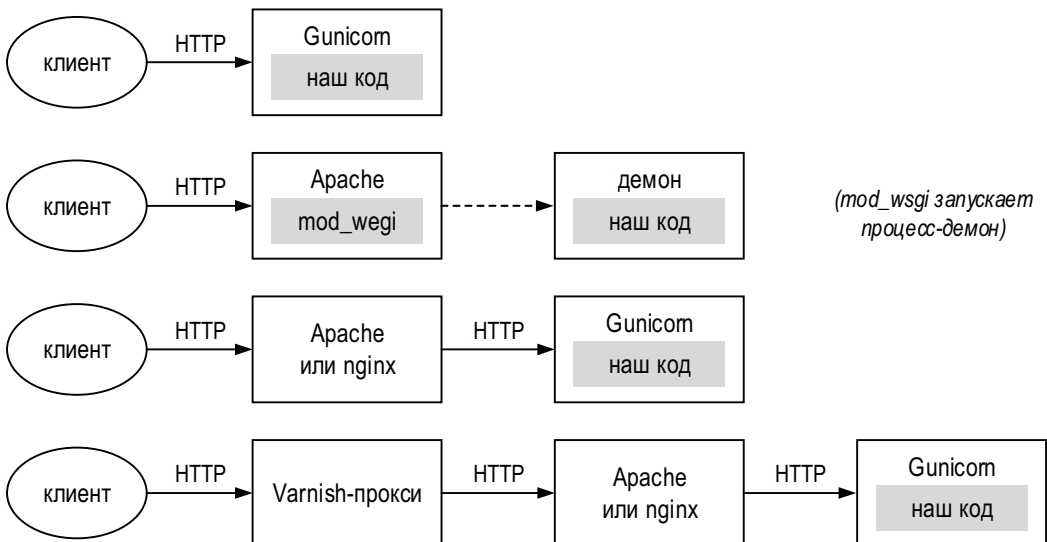


Рис. 10.1. Четыре стандартных способа развернуть код Python отдельно или через обратные прокси HTTP

Интерпретатор большой и работает медленно, а глобальная блокировка интерпретатора мешает нескольким потокам выполнять код Python одновременно, так что обычно выбираются три последних варианта, с языком C.

Из-за блокировки интерпретатора было решено использовать отдельные процессы Python, а не несколько потоков Python в одном процессе. Правда, из-за размеров интерпретатора только ограниченное число экземпляров Python помещается в оперативной памяти, что ограничивает количество процессов.

Python на Apache

Представьте ранний веб-сайт Python на Apache со старым модулем `mod_python`. Это поможет понять проблемы, описанные выше. Большинство запросов к нормальному веб-сайту (см. главу 11) касаются статических ресурсов: на каждый запрос для динамической сборки страницы на Python приходится десяток запросов на CSS, JavaScript и изображения. Несмотря на это, `mod_python` устанавливал копию среды выполнения интерпретатора Python на каждом рабочем узле Apache, причем большинство из них простаивало. В любой момент только один из десятка рабочих узлов выполнял код Python, пока остальные поставляли файлы с помощью Apache и C. Для того чтобы избавиться от этой блокировки, нужно отделить интерпретаторы Python от рабочих веб-серверов, которые поставляют статический контент с диска к ожидающим сокетами. В результате появились два взаимоисключающих метода.

Первый вариант — применять современный модуль `mod_wsgi` с возможностью использовать процесс-демон, чтобы не добавлять к каждому потоку Apache интерпретатор Python. Рабочие потоки и процессы Apache избавляются от необходимости выполнять Python и только динамически связываются с `mod_wsgi`. `Mod_wsgi` же создает и контролирует отдельный пул рабочих процессов Python, в которые он может отправлять запросы и где приложение WSGI действительно выполняется. Для каждого огромного интерпретатора Python, который отвечает за динамическое содержимое веб-сайта, есть десяток крошечных рабочих процессов Apache, поставляющих статические файлы.

HTTP-серверы на Python

Если Python не будет выполняться в главном серверном процессе, а HTTP-запросы нужно сериализовать и перенаправлять из процесса Apache в процесс Python, почему бы просто не использовать HTTP? Почему не настроить Apache таким образом, чтобы он перенаправлял каждый динамический запрос к Gunicorn, где выполняется сервис?

Да, придется запускать и обслуживать два отдельных демона (Apache и Gunicorn), когда раньше достаточно было запустить Apache и позволить модулю `mod_wsgi` заниматься созданием интерпретаторов Python. Зато мы получаем универсальное решение. Например, Apache и Gunicorn больше не обязаны работать на одном обо-

рудовании. Можно запустить Apache на сервере, оптимизированном для большого количества одновременных подключений и частого доступа к диску, а Gunicorn — на втором сервере, приспособленном под динамические языки и запросы к базам данных.

Мы даже можем поменять Apache, раз мы понизили его статус с контейнера приложения до сервера статических файлов с возможностями обратного прокси. В конце концов, nginx, как и многие другие современные веб-серверы, могут предоставлять файлы и одновременно выступать как обратный прокси для других путей.

Модуль `mod_wsgi` представляет собой ограниченную и проприетарную версию настоящего обратного прокси. Мы могли бы взаимодействовать с настоящим HTTP и выполнять Python на том же компьютере или на другом, в зависимости от наших потребностей, а сами используем внутренний протокол между процессами, которые должны находиться на одном компьютере.

Преимущество обратных прокси

Как насчет приложений HTTP, которые предлагают только динамическое содержимое, создаваемое кодом Python, а статические ресурсы не используются совсем? В таких случаях серверам Apache и nginx почти нечего делать, и возникает искушение не использовать их и напрямую предоставить доступ к Gunicorn или другому веб-серверу на Python.

Не забывайте о безопасности, которую в таких случаях обеспечивает обратный прокси. Просто подключите сервис с n рабочих процессов и n сокетов, доставьте несколько начальных байтов запроса, а затем остановите процесс и веб-сервис. Все сотрудники будут ждать полного запроса, который может поступить или не поступить.

Запросы, которые сначала проходят через Apache или nginx и занимают много времени, медленно собираются в буферах обратного прокси, которые обычно не перенаправляют вам запрос, пока не получают его полностью. Этой ситуацией могут воспользоваться злоумышленники. Кроме того, при таком подходе трудности возникают у клиентов с низкой пропускной способностью.

Разумеется, прокси, которые собирает запросы полностью, прежде чем поставлять их, не на 100% защищен от атак типа "отказ в обслуживании" (к сожалению, абсолютной защиты от них не существует), но он позволяет среде выполнения динамического языка не зависать, пока данные от клиента недоступны. Он также защищает Python от других типов ошибочных входных данных, например имен заголовков длиной в мегабайт или некорректно сформированных запросов, потому что Apache или nginx просто отклонят их с ошибкой 4xx, так что код бэкенд-приложения ничего не будет знать о них.

Из этих вариантов архитектуры я советую использовать три. Мой любимый вариант: Gunicorn за nginx или Apache (зависит от системного администратора).

Если мой сервис — просто API без статических компонентов, иногда я использую Gunicorn отдельно или сразу за Varnish, когда хочу, чтобы динамические ресурсы

использовали его первоклассную логику кеширования. Только если я создаю огромные веб-сервисы, я использую все три уровня: Gunicorn с Python, nginx или Apache и локальный или распределенный кластер Varnish.

Конечно, возможны и другие конфигурации, и я надеюсь, что вы уже достаточно хорошо понимаете достоинства и недостатки разных компонентов и сможете принимать верные решения в своих проектах.

Когда появились среды выполнения Python вроде PyPy, в которых код работает гораздо быстрее, возник вопрос: почему бы не поставлять статическое и динамическое содержимое через Python, если он теперь работает так же быстро, как Apache? Будет интересно посмотреть, смогут ли серверы на основе быстрых сред выполнения Python бросить вызов проверенным решениям вроде Apache и nginx. Если эти фавориты отрасли так хорошо задокументированы, известны и любимы системными администраторами, что могут предложить серверы Python, чтобы привлечь внимание?

Разумеется, эти типы архитектуры можно использовать с вариациями. Если статические файлы не нужны или вы не возражаете, что Python будет сам извлекать их с диска, можно разместить Gunicorn сразу за Varnish. Еще один вариант — использовать nginx или Apache с обратным кешированием, чтобы обеспечивать базовое кеширование в стиле Varnish без дополнительного третьего уровня. Некоторые сайты пробуют использовать альтернативные протоколы для взаимодействия между фронтенд-сервером и Python, например проекты Flup и uwsgi. Эти четыре варианта архитектуры просто самые популярные, но существуют и другие.

Платформа как услуга

Многие темы в предыдущих разделах, включая балансировку нагрузки, несколько уровней прокси-сервера и развертывание приложений, затрагивают тему системного администрирования и планирования операций. Python не единственный язык, для которого нужно выбрать балансировщика нагрузки на фронтенде или обеспечить физическую и географическую избыточность HTTP-сервиса. Мы не будем обсуждать эти вопросы, потому что они далеки от сетевого программирования на Python.

Если вы хотите использовать Python при разработке сетевого сервиса, почитайте про автоматические развертывания, непрерывную интеграцию и высокопроизводительное масштабирование. Здесь мы эти темы рассматривать не будем. Правда, стоит упомянуть о популярности решений "платформа как услуга" (Platforms as a Service, PaaS) и о том, как упаковать приложение для развертывания на такой платформе.

Большая часть задач по подготовке и запуску HTTP-сервиса автоматизирована, или, во всяком случае, ею занимается провайдер PaaS. Нам не приходится покупать серверы, предоставлять им хранилище и IP-адреса, настраивать корневой доступ для администрирования, перезагружать их, устанавливать правильную версию

Python и копировать приложение на каждый сервер вместе с системными скриптами, чтобы запускать сервис автоматически после перезапуска сервера или перебоев с питанием. За все это отвечает провайдер PaaS, который может устанавливать тысячи компьютеров, сотни серверов с базами данных и десятки балансировщиков нагрузок. Когда все эти задачи автоматизированы, нам остается только отправить провайдеру файл конфигурации. Затем провайдер может добавить наше доменное имя в DNS, привязать его к одному из своих балансировщиков нагрузки, установить подходящую версию Python и все необходимые зависимости Python в образе операционной системы — и приложение будет готово к работе. С помощью этой процедуры можно легко публиковать новый код или делать откаты, если новая версия вызвала проблемы при использовании реальными потребителями. Heroku — популярное решение PaaS, которое включает первоклассную поддержку приложений Python в своей экосистеме. Небольшие компании, которым не хватает навыков или времени самостоятельно подготавливать и обслуживать балансировщиков нагрузки и прочие решения, могут воспользоваться предложением Heroku и других аналогичных провайдеров.

Например, экосистема Docker позволяет создавать и запускать контейнеры на вашем компьютере Linux, упрощая тестирование и отладку по сравнению с долгими и медленными процедурами на Heroku. Казалось бы, решение PaaS должно принять программу на Python с поддержкой WSGI и выполнить ее без лишних усилий с нашей стороны.

К сожалению, это не так. Мы по-прежнему должны выбрать веб-сервер, даже если работаем через Heroku или Docker.

Провайдеры PaaS предлагают балансировку нагрузки, контейнеризацию, конфигурацию с контролем версий, кеширование образов контейнеров и администрирование базы данных, но наше приложение все равно должно предоставлять необходимый компонент для взаимодействия с HTTP: открытый порт, к которому балансировщик нагрузки PaaS может подключиться, чтобы отправлять HTTP-запросы. И, конечно, нам нужен сервер, чтобы превратить фреймворк или приложение WSGI в слушающий сетевой порт.

Некоторые разработчики, зная, что сервис PaaS будет обрабатывать балансировку нагрузки, выбирают простой однопоточный сервер и делегируют провайдеру PaaS задачу по запуску нужного количества экземпляров приложения.

Однако многие разработчики используют Unicorn или подобное решение, чтобы в каждом контейнере было несколько рабочих потоков одновременно. При таком подходе один контейнер принимает несколько запросов, если логика кругового распределения в балансировщике нагрузки PaaS возвращается к тому же контейнеру до выполнения первого запроса.

Это проблема, если некоторым ресурсам сервиса требуется несколько секунд для отрисовки, из-за чего последующие запросы становятся в очередь.

Стоит отметить, что большинство провайдеров PaaS не разрешают поставлять статическое содержимое, если только мы не используем Python или не устанавливаем

Apache или nginx в контейнере. Конечно, мы можем создать пространство URL таким образом, чтобы у статических и динамических ресурсов было разное имя хоста, и разместить статические ресурсы в другом месте, но многие архитекторы предпочитают совмещать статические и динамические ресурсы в одном пространстве имен.

REST и паттерны GET и POST

Один из авторов текущих стандартов HTTP, доктор Рой Филдинг (Roy Fielding), защитил по нему диссертацию. Он придумал термин REST (representational state transfer — передача репрезентативного состояния), чтобы описать архитектуру, которая использует все возможности гипертекстовой системы, такой как HTTP. Его диссертацию можно найти и прочитать онлайн.

В главе 5 он расписывает концепцию REST на основе ряда базовых принципов.

www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Как пишет доктор Филдинг, REST определяется четырьмя ограничениями:

- ◆ идентификацией ресурсов;
- ◆ работой с ресурсами через представления;
- ◆ самоописываемыми сообщениями;
- ◆ гипермедиа для изменения состояния приложения (Hypermedia as an application state engine, HATEOAS).

Многие разработчики сервисов стараются создать продукт, к которому в полной мере можно было бы применить термин RESTful (т. е. отвечающий требованиям REST), чтобы их идеи сочетались с принципами HTTP, а не противоречили им. Доктор Филдинг утверждает, что у большинства разработчиков это не получается. В чем их ошибка?

Первое правило (идентификация ресурсов) исключает практически все традиционные виды RPC. На уровне HTTP-протокола ни JSON-RPC, ни XML-RPC (см. главу 18) не раскрывают идентификаторы ресурсов. Допустим, клиент хочет извлечь статью блога, изменить заголовок, а затем снова извлечь статью и сравнить их. Если бы эти действия были реализованы как вызовы метода RPC, следующие методы и пути были бы доступны для HTTP:

POST /rpc-endpoint/ @ 200 OK

POST /rpc-endpoint/ @ 200 OK

POST /rpc-endpoint/ @ 200 OK

Каждый из запросов, наверное, указывает что-то вроде "post 1022" как ресурс, который клиент хочет извлечь или изменить в теле каждого запроса POST. Однако RPC скрывает эти детали от HTTP. Интерфейс RESTful использовал бы путь ресурса, чтобы описать, какая статья меняется, например /post/1022/.

Второе ограничение (работа с ресурсами через представления) запрещает разработчику определять специальный механизм для обновления заголовка, уникальный для этого сервиса, чтобы автору клиента потом не приходилось изучать документацию по сервису каждый раз, как нужно обновить информацию. Поскольку представление статьи (в формате HTML, JSON, XML или каком-то другом) — это единственная форма, в которой в REST могут быть выражены операции чтения и записи, нет нужды узнавать конкретную стратегию изменения заголовка статьи. Клиент просто получает текущее представление, обновляет заголовок и отправляет новое представление обратно сервису.

```
GET /post/1022/ @ 200 OK
```

```
PUT /post/1022/ @ 200 OK
```

```
GET /post/1022/ @ 200 OK
```

Многим разработчикам не нравится тот факт, что для запроса или обновления десятка ресурсов требуется десяток круговых путей к сервису, так что существует потребность внести исключения в эту архитектуру. Однако симметрия между операциями чтения и записи ресурса, а также использование осмысленной семантики в протоколе HTTP — это преимущества REST, которые всем нравятся. Поскольку теперь протокол видит разницу между чтением и записью, если ответы GET включают корректные заголовки, кеширование и условные запросы разрешены, даже если программы общаются не через браузер.

Ограничение, связанное с самоописываемыми сообщениями, требует явных заголовков кеширования, чтобы коммуникации содержали всю необходимую информацию. Разработчику клиента не приходится обращаться к документации по API, чтобы узнать, что `/post/1022/` находится в формате JSON или его можно кешировать, только если используются условные запросы, чтобы кешированная копия была актуальна, в то время как запросы вроде `/post/?q=news` можно обслуживать напрямую из кеша в течение 60 секунд после извлечения. Вся эта информация предоставляется в заголовках каждого HTTP-ответа. Если первые три критерия REST удовлетворяются, сервис становится полностью прозрачным для HTTP-протокола, а значит, и для всего набора прокси, кешей и клиентов, которые будут использовать эту семантику.

Это возможно независимо от того, что делает сервис: создает HTML-страницы с формами и JavaScript (см. главу 11) для человека или поставляет короткие URL, которые ведут на представления JSON или XML для компьютера.

Последнее правило соблюдается гораздо реже. Фраза "гипермедиа для изменения состояния приложения" породила столько обсуждений, что часто используется в сокращенном виде — HATEOAS. В своей статье о том, что REST API должны управляться гипертекстом, доктор Филдинг выражал сожаление, что это ограничение не соблюдается.

<http://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven>

Он разделяет ограничение HATEOAS на шесть пунктов, последний из которых, пожалуй, является наиболее полным. "Обращаться к REST API нужно без предва-

рительной информации, кроме первого URI (закладки) и набора стандартизированных типов медиа, которые соответствуют целевой аудитории", — пишет он.

Почти все API на основе HTTP уже нарушают это правило. Документация по ним почти всегда начинается со слов: "Каждая статья находится по URL, наподобие /post/1022/, в котором указан уникальный идентификатор статьи". При этом API отдаляется от принципов RESTful, и к соответствующему ресурсу клиента ведут конкретные правила, скрытые в документации, а не гипертекстовые ссылки.

Для того чтобы API мог по-настоящему называться RESTful, у него должна быть только одна точка входа. Может вернуться, например, последовательность форм, в одну из которых можно вести идентификатор статьи, чтобы вернуть ее URL-адрес. Документация не понадобится, потому что сервис динамически свяжет концепцию "статья с ID 1022" с определенным местоположением. По словам доктора Филдинга, концепция гипертекста — это критически важное требование для сервисов, которые планируют существовать десятилетия, потому что он сможет поддерживать множество поколений HTTP-клиентов, а затем и археологию данных через много-много лет. Однако первые три аспекта HTTP предоставляют большую часть преимуществ (работа без отслеживания состояния, избыточность и ускорение кеша), поэтому мало сервисов соблюдают все ограничения REST.

WSGI без фреймворка

В *главе 7* мы рассмотрели несколько методов создания сетевого сервиса, и все их можно использовать для ответа на HTTP-запросы.

Однако нам редко требуется программировать собственный низкоуровневый код для протокола. Многие детали протокола можно делегировать веб-серверу и веб-фреймворку. В чем разница между ними?

Веб-сервер — это программа, которая поддерживает слушающий сокет в открытом состоянии, вызывает `accept()` для приема новых соединений и анализирует каждый поступающий HTTP-запрос. Сервер обрабатывает такие ситуации, как обращение клиента, который подключается, но не отправляет запросы, или клиента, запросы которого нельзя обработать как HTTP без вызова нашего кода. Некоторые серверы отсчитывают время ожидания и закрывают бездействующий клиентский сокет, а также отклоняют запросы с очень длинными путями или заголовками.

При обращении к вызываемым объектам WSGI, зарегистрированным на сервере, только хорошо сформированные полные запросы передаются фреймворку или коду. Сервер часто выдает HTTP-коды, как описано в *главе 9*:

- ◆ 4xx, если входящий HTTP-запрос некорректный или превышает установленный лимит по размеру;
- ◆ 500 Server Error, если вызываемый объект WSGI выдает исключение.

Существуют два метода создать вызываемый объект WSGI, который веб-сервер будет использовать после поступления и парсинга HTTP-запросов. Мы можем на-

писать вызывающий код сами или создать код, который подключается к веб-фреймворку, предоставляющему собственный вызываемый объект WSGI. В чем разница?

Главная функция веб-фреймворка — отправка запросов к месту назначения. Каждый HTTP-запрос указывает необходимый метод, имя хоста и путь. Скорее всего, наш сервис будет работать по одному или двум именам хостов, а не по всем. Сервис может быть готов выполнять запросы GET или POST, а запрос может указывать любой другой метод — даже еще не существующий. Мы можем предоставлять ответы по самым разным путям, но будет еще больше путей, по которым мы не сможем предоставить ответ. Фреймворк позволяет указывать, какие пути и методы мы поддерживаем, чтобы он автоматически отправлял остальные сообщения об ошибках:

- ◆ 404 Not Found (не найдено);
- ◆ 405 Method Not Allowed (метод не разрешен);
- ◆ 501 Not Implemented (не реализовано).

В *главе 11* мы увидим, как традиционные и асинхронные фреймворки передают запросы и какие еще возможности они дают программистам. Как выглядел бы наш код без них?

А если бы код напрямую общался с WSGI и распределял запросы?

Существуют два подхода к разработке такого приложения: нужно понять стандарт WSGI и научиться читать его словарь переменных окружения или использовать обертку, например предоставляемую инструментами WebOb и Werkzeug, доступными в Python Package Index. Для работы с окружением WSGI требуется много словный код, как показано в листинге 10.2.

Листинг 10.2. Вызываемый объект WSGI для возврата текущего времени в необработанном виде

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Простой HTTP-сервис, созданный напрямую по низкоуровневым спецификациям WSGI.

import time

def app(environ, start_response):
    host = environ.get('HTTP_HOST', '127.0.0.1')
    path = environ.get('PATH_INFO', '/')
    if ':' in host:
        host, port = host.split(':', 1)
    if '?' in path:
        path, query = path.split('?', 1)
    headers = [('Content-Type', 'text/plain; charset=utf-8')]
    if environ['REQUEST_METHOD'] != 'GET':
```

```

    start_response('501 Not Implemented', headers)
    yield b'501 Not Implemented'
elif host != '127.0.0.1' or path != '/':
    start_response('404 Not Found', headers)
    yield b'404 Not Found'
else:
    start_response('200 OK', headers)
    yield time.ctime().encode('ascii')

```

Без фреймворка коду пришлось бы самому решать, какие имена хостов, маршруты и методы не подходят для наших сервисов. Мы должны возвращать ошибку для каждого отклонения от этой комбинации параметров запроса, выполняя метод GET для пути/по имени хоста 127.0.0.1. Может показаться странным, что такой маленький сервис не принимает любое имя хоста. Однако давайте сделаем вид, что у нас большой сервис с разнообразным контентом и десятками имен хостов. Если клиент предоставляет заголовок Host с указанием 127.0.0.1:8000, мы должны выделить отсюда имя хоста и порт. Кроме того, если в URL есть строка запроса, например `?name=value`, в конце нужно разделить путь по символу `?`. (В листинге 10.2 предполагается, что мы игнорируем лишние строки запросов, а не возвращаем ошибку 404.)

В следующих двух листингах показано, как сторонние библиотеки, которые можно добавить через `pip`, упрощают работу с WSGI (см. главу 1).

```
$ pip install WebOb
```

```
$ pip install Werkzeug
```

WebOb — это легкий интерфейс объекта, который служит оберткой для словаря WSGI и предоставляет более удобный доступ к нему. Он был создан Йеном Бикингом (Ian Bicking). В листинге 10.3 показано, как он позволяет избежать типичные паттерны из предыдущего примера.

Листинг 10.3. Вызываемый объект WSGI, написанный с помощью WebOb, возвращает текущее время

```

#!/usr/bin/env python3
# Programming in Python: The Basics
# Вызываемый объект WSGI, созданный с помощью WebOb.

import time, webob

def app(environ, start_response):
    request = webob.Request(environ)
    if environ['REQUEST_METHOD'] != 'GET':
        response = webob.Response('501 Not Implemented', status=501)

```

```

elif request.domain != '127.0.0.1' or request.path != '/':
    response = webob.Response('404 Not Found', status=404)
else:
    response = webob.Response(time.ctime())
return response(envIRON, start_response)

```

WebOb уже поддерживает два распространенных паттерна: считывать имя хоста из заголовка Host отдельно от необязательных номеров порта и обрабатывать путь без строки запроса в конце. Также у него есть объект Response, который знает все необходимое о типах контента и кодировках (по умолчанию это открытый текст), так что нам остается только предоставить строку в теле ответа, а WebOb позаботится об остальном.

Библиотека WebOb выделяется среди множества реализаций объекта HTTP-ответа на Python. Класс Response в WebOb позволяет считать два элемента заголовка Content-Type, например text/plain; charset=utf-8, как два разных значения, которые он предоставляет как атрибуты типа содержимого и кодировки.

Библиотека Werkzeug, созданная Армином Ронахером (Armin Ronacher), которая также лежит в основе его фреймворка Flask, не так популярна, как WebOb, для работы с чистым кодом WSGI, но у нее есть свои преданные поклонники (см. главу 11). Объекты запроса и ответа являются неизменными. В листинге 10.4 видно, чем эта библиотека отличается от WebOb в плане простоты.

Листинг 10.4. Вызываемый объект WSGI, написанный с помощью Werkzeug, возвращает текущее время

```

#!/usr/bin/env python3
# Programming in Python: The Basics
# Вызываемый объект WSGI, созданный с помощью Werkzeug.

import time
from werkzeug.wrappers import Request, Response

@Request.application
def app(request):
    host = request.host
    if ':' in host:
        host, port = host.split(':', 1)
    if request.method != 'GET':
        return Response('501 Not Implemented', status=501)

```

```
elif host != '127.0.0.1' or request.path != '/':  
    return Response('404 Not Found', status=404)  
else:  
    return Response(time.ctime())
```

С Werkzeug нам не придется запоминать правильные подписи вызываемого объекта WSGI. Вместо этого мы предоставляем декоратор, который преобразует функцию в более простой синтаксис вызова. Мы принимаем объект Request в Werkzeug как единственный аргумент и возвращаем объект Response — остальным занимается библиотека.

Единственное отличие от WebOb: здесь мы должны делить имена хостов 127.0.0.1:8000 самостоятельно. Несмотря на небольшое различие, обе библиотеки выполняют одну задачу: позволяют нам работать с запросами и ответами HTTP на более высоком уровне, чем WSGI. Обычно разработчикам не стоит тратить время на такое низкоуровневое программирование и лучше использовать веб-фреймворк. Однако если нам нужно как-то преобразовать входящие HTTP-запросы, прежде чем передать их веб-фреймворку для обработки, можно написать код на уровне WSGI. Если вы разрабатываете специализированный обратный прокси или другой HTTP-сервис на Python, возможно, чистый WSGI будет идеальным вариантом.

Чистые вызываемые объекты WSGI занимают в программировании на Python то же место, что прямые и обратные прокси в экосистеме HTTP. Лучше всего они работают с низкоуровневыми операциями, такими как фильтрация, нормализация и распределение запросов, а не с предоставлением ресурсов по конкретным именам хостов и URL, как HTTP-сервис. Прочтите спецификации или документацию по библиотекам WebOb и Werkzeug, чтобы узнать, как вызываемый объект WSGI может изменить запрос, прежде чем передать его другому вызываемому объекту.

Резюме

При запуске из командной строки модуль http.server в Python предоставляет файлы под текущим рабочим каталогом. Это полезно в экстренном случае или при проверке сайта, хранящегося на диске, но сегодня этот модуль редко используется для создания новых HTTP-сервисов.

В Python стандарт WSGI часто используется для работы с обычным синхронным HTTP. Серверы сканируют входящие запросы, чтобы создать словарь с информацией, которую приложения просматривают, прежде чем предоставить HTTP-заголовки и необязательное содержимое ответа. Это позволяет нам использовать любой веб-фреймворк Python с любым веб-сервером. Экосистема WSGI не включает асинхронные веб-серверы. Поскольку вызываемые объекты WSGI не являются полноценными сопрограммами, каждый асинхронный HTTP-сервер должен использовать собственный паттерн для написания сервисов в своем фреймворке. В этой ситуации сервер и фреймворк упакованы вместе, а значит, идеально совмес-

тимы. Существует четыре основных архитектурных паттерна использования HTTP через Python. Gunicorn или другой сервер на Python вроде CherryPy можно использовать как отдельный сервер. Другие архитекторы предпочитают применять модуль `mod_wsgi`, чтобы выполнять Python под контролем Apache. Некоторые считают, что проще использовать Gunicorn или аналогичный сервер за обратным прокси `nginx` или Apache как отдельный HTTP-сервис, который предоставляет динамические ресурсы. Кроме того, можно поместить перед этой конструкцией обратный прокси Varnish для кеширования. Экземпляры кеша можно хранить в том же центре обработки данных (или даже на том же компьютере), но часто они распределены по разным регионам, чтобы обслуживать ближайших HTTP-клиентов. Провайдер PaaS обычно предоставляет готовые инструменты кеширования, обратный прокси и балансировщики нагрузки. Единственная задача нашего приложения — отвечать на HTTP-запросы, обычно через простой контейнер, например на Gunicorn. Также нужно учитывать, является ли сервис RESTful, т. е. соблюдает ли ограничения, описанные Роем Филдингом. Хотя многие сервисы отказались от непрозрачного выбора метода и пути, который скрывал от нас важные действия, редкие сервисы полностью реализовали видение доктора Филдинга и используют гипертекст вместо семантики, обозначенной в документации. Маленькие сервисы, особенно те, которые фильтруют или преобразуют HTTP-запросы, можно писать как вызываемые объекты WSGI. Библиотеки `WebOb` и `Werkzeug` позволяют сократить окружение WSGI до удобного объекта `Request`, а также помогают строить ответ с помощью класса `Response`.

В следующей главе мы рассмотрим Всемирную паутину — огромную коллекцию взаимосвязанных документов. Вы узнаете, как извлекать и обрабатывать гипертекстовые документы, а также создавать сайты с помощью популярных веб-фреймворков.

Всемирная паутина

В главах 9 и 10 мы узнали, что HTTP — это универсальный способ, с помощью которого клиенты могут запрашивать документы, а серверы — предоставлять их.

Но почему HTTP называется протоколом передачи гипертекста (Hypertext Transfer Protocol)? Чем он отличается от старого протокола передачи файлов — FTP (см. главу 17)? Почему Всемирная паутина построена на HTTP?

Содержание главы

- ◆ URL и гипермедиа.
- ◆ Создание и парсинг URL.
- ◆ Относительные URL.
- ◆ Язык гипертекстовой разметки HTML.
- ◆ Чтение и запись с использованием базы данных.
- ◆ Ужасное веб-приложение на Flask.
- ◆ Методы и формы HTTP.
- ◆ Когда формы используют неподходящие методы.
- ◆ Опасные и безопасные cookie.
- ◆ Непостоянный межсайтовый скриптинг.
- ◆ Постоянный межсайтовый скриптинг.
- ◆ Подделка межсайтовых запросов.
- ◆ Улучшенная программа.
- ◆ Приложение для оплаты на Django.
- ◆ Выбор фреймворка для веб-сайта.
- ◆ Веб-сокеты.
- ◆ Веб-скрейпинг.
- ◆ Получение страниц.

- ◆ Страницы для веб-скрейпинга.
- ◆ Рекурсивный веб-скрейпинг.
- ◆ Резюме.

Цель

В этой главе вы узнаете, что HTTP преследует гораздо более амбициозную цель, чем передача отдельных документов вроде книг, фотографий и видео. Он позволяет серверам по всему миру публиковать документы, которые превращаются в единую взаимосвязанную сеть знаний с помощью перекрестных ссылок.

URL и гипермедиа

Тысячелетиями в книгах встречаются отсылки на другие книги. Мы можем взять с полки нужную книгу, пролистать ее и найти упоминаемый абзац. Всемирная паутина (World Wide Web, WWW) упрощает эту задачу.

Мы можем превратить текст "обсуждение cookie в главе 9" в гиперссылку. Он будет подчеркнутым, и на него можно будет нажать, чтобы перейти к соответствующему разделу. *Гипертекстовые документы* — это документы с гиперссылками в тексте. Если в печатной книге мы видим слова "см. с. 103", мы не сможем волшебным образом перенестись к нужной странице. А в браузере это возможно.

URL (Uniform Resource Location — унифицированный указатель ресурса) помогает работать с гипермедиа. Он предоставляет единый механизм для ссылок не только на современные гипертекстовые документы, но и на старые файлы FTP и серверы Telnet. Подобные примеры можно увидеть в адресной строке браузера.

Примеры URL-адресов

`https://www.python.org/`

`http://en.wikipedia.org/wiki/Python_(programming_language)`

`http://localhost:8000/headers`

`ftp://ssd.jpl.nasa.gov/pub/eph/planets/README.txt`

`telnet://rainmaker.wunderground.com`

Сначала указан протокол, через который мы получаем страницу, например `https` или `http`. Затем идут две косые черты, имя хоста и необязательный номер порта после двоеточия. Наконец, указан один документ из доступных. Этот синтаксис можно использовать не просто для описания материала, который нужно извлечь из сети.

URI (Uniform Resource Identifier — унифицированный идентификатор ресурса) — это более общая концепция, которая позволяет определять физические документы в сети. Кроме того, ресурсы определяются с помощью URN (Uniform Resource

Name — унифицированное имя ресурса). Все ссылки на другие материалы в этой книге будут гиперссылками.

URL дополняется строкой запроса, которая начинается с вопросительного знака (?). Символ амперсанда (&) разграничивает следующие параметры, предоставленные пользователем, на основе которых автоматически создается страница. В каждом параметре должно быть имя, знак равенства и значение.

`https://www.google.com/search?q=apod&btnI=yes`

Наконец, у URL может быть суффикс, который указывает на определенное место на странице.

`https://tools.ietf.org/html/rfc2324#section-2.3.2`

Этот фрагмент выделяется из остальных компонентов URL. Поскольку веб-браузер предполагает, что нужно извлечь полную страницу, указанную в маршруте, чтобы найти элемент, на который указывает фрагмент, сам фрагмент не отправляется в HTTP-запросе. Когда браузер загружает URL HTTP, сервер может узнать только имя хоста, путь и запрос. Имя хоста предоставляется в заголовке Host, а путь и запрос объединяются, образуя полный путь, который следует за HTTP-методом в первой строке запроса, как мы видели в *главе 9*.

Если внимательно изучить RFC 3986, можно заметить дополнительные возможности, которые редко используются. Обращайтесь к этому ресурсу, если вам встретятся эти возможности, например возможность включать строку аутентификации пользователь@пароль прямо в URL.

Создание и парсинг URL

Модуль `urllib.parse` из стандартной библиотеки Python содержит инструменты для интерпретации и создания URL. Достаточно вызова одной функции, чтобы разделить URL на составляющие. Возвращается кортеж, к которому можно обращаться через индексы или распаковку в инструкции присваивания в более ранних версиях Python.

```
>>> from urllib.parse import urlsplit
>>> u = urlsplit('https://www.google.com/search?q=apod&btnI=yes')
>>> tuple(u)
('https', 'www.google.com', '/search', 'q=apod&btnI=yes', '')
```

Кортеж также поддерживает доступ к элементам по именованным атрибутам, чтобы код было удобнее читать, когда мы проверяем URL.

```
>>> u.scheme
'https'
>>> u.netloc
'www.google.com'
```



```
>>> u.path
'/search'
>>> u.query
'q=apod&btnI=yes'
>>> u.fragment
''
```

У `netloc` (network location — расположение в сети) может быть множество подчиненных компонентов, но они достаточно редкие, так что `urlsplit()` не разделяет их в кортеже. Они доступны лишь как свойства в результате.

```
>>> u = urlsplit('https://bpb:online@localhost:8000/')
>>> u.netloc
'bpb:online@localhost:8000'
>>> u.username
'bpb'
>>> u.password
'online'
>>> u.hostname
'localhost'
>>> u.port
8000
```

Только половина процесса парсинга включает разделение URL на части. Символы, которые нужно экранировать, прежде чем включать в URL, могут встречаться и в пути, и в запросе. Символы `&` и `#`, например, нельзя использовать без экранирования, потому что они служат разделителями в URL. Символ `/` тоже нужно экранировать, если он встречается в пути, т. к. косая черта разделяет компоненты пути.

Часть URL, которая содержит запрос, предъявляет свои требования к синтаксису. Значения запроса часто содержат пробелы (представьте типичный запрос в поисковой системе), вместо них используется символ `+` или `%20` (в шестнадцатеричной нотации).

Единственный правильный способ парсинга URL, который обращается к разделу Q&A (часто задаваемые вопросы), чтобы найти там часть про TCP/IP с информацией про packet loss (потеря пакетов):

```
>>> from urllib.parse import parse_qs, parse_qsl, unquote
>>> u = urlsplit('http://example.com/Q%26A/TCP%2FIP?q=packet+loss')
>>> path = [unquote(s) for s in u.path.split('/')]
>>> query = parse_qsl(u.query)
>>> path
['', 'Q&A', 'TCP/IP']
>>> query
[('q', 'packet loss')]
```

Поскольку это абсолютный путь, который начинается с косой черты, если мы разделим его с помощью `split()`, получим начальную пустую строку.

Строка запроса в URL разрешает указывать параметр запроса несколько раз, так что запрос предстает как список кортежей, а не как простой словарь. Мы можем предоставить список кортежей в `dict()` и увидим только последнее значение для каждого параметра, если мы создаем код, в котором это не важно. Если мы хотим разрешать несколько значений для параметра, `parse_qs()` нужно заменить на `parse_qsl()`, чтобы получить словарь со списками в качестве значений.

```
>>> parse_qs(u.query)
{'q': ['packet loss']}
```

Стандартная библиотека содержит все необходимые процедуры, чтобы обратить процесс. Мы берем в кавычки каждый компонент пути, объединяем их через косую черту, кодируем запрос и передаем результат процедуре `unsplit`, которая выполняет действие, обратное функции `urlsplit()`. Таким образом Python может восстановить URL из фрагментов.

```
>>> from urllib.parse import quote, urlencode, urlunsplit
>>> urlunsplit(('http', 'example.com',
... '/'.join(quote(p, safe='') for p in path),
... urlencode(query), ''))
'http://example.com/Q%26A/TCP%2FIP?q=packet+loss'
```

Если мы правильно делегировали парсинг URL этим методам стандартной библиотеки, мы заметим, что они обрабатывают за нас детали спецификации.

Код в предыдущих примерах можно было бы даже назвать навороченным, потому что в нем так много всего. Как часто в компонентах пути встречается косая черта? Большинство сайтов стараются тщательно продумать элементы пути, чтобы некрасивое экранирование требовалось как можно реже. Если сайт разрешает использовать в этой части URL только буквы, цифры, дефисы и подчеркивания, риск случайной вставки косой черты минимален. Если вы уверены, что в пути нет экранированной косой черты, можно предоставить его методам `quote()` и `unquote()` без разделения.

```
>>> quote('Q&A/TCP IP')
'Q%26A/TCP%20IP'
>>> unquote('Q%26A/TCP%20IP')
'Q&A/TCP IP'
```

В реальности метод `quote()` ожидает, что в запросе не будет косой черты, потому что у него по умолчанию установлен параметр `safe='/'`, который не затрагивает косую черту. В длинной версии кода этот параметр был заменен на `safe=""`.

Модуль `urllib.parse` в стандартной библиотеке предлагает более специализированные процедуры, чем уже рассмотренные нами, например `urldefrag()`, которая разделяет URL по символу `#`. Прочтите документацию, чтобы узнать больше об этой и других специальных функциях.

Относительные URL

Команда смены рабочего каталога в командной строке файловой системы определяет положение, с которого система начнет рассматривать относительные пути без начальной косой черты. Если путь начинается с косой черты, поиск ведется от корня файловой системы. Это *абсолютные пути*, т. е. они всегда указывают на одно и то же расположение, где бы мы ни работали.

```
$ wc -l /var/log/dmesg
977 dmesg
$ wc -l dmesg
wc: dmesg: No such file or directory
$ cd /var/log
$ wc -l dmesg
977 dmesg
```

Гипертекст работает по тому же принципу. Если все ссылки в документе являются *абсолютными URL*, как в предыдущем разделе, очевидно, на какой ресурс ведет каждая из них. Если документ содержит относительные URL, следует учитывать расположение самого документа.

У Python есть функция `urljoin()`, которая позволяет работать со всеми тонкостями стандарта. Мы можем использовать `urljoin()`, чтобы заполнить недостающую информацию из URL, взятого из гипертекстового документа. Если URL и так абсолютный, он вернется без изменений. Метод `urljoin()` использует тот же порядок аргументов, что и `os.path.join()`. Сначала предоставляем базовый URL документа, а затем URL, который мы в нем обнаружили. Относительный URL может перезаписать элементы базового разными способами.

```
>>> from urllib.parse import urljoin
>>> base = 'http://tools.ietf.org/html/rfc3986'
>>> urljoin(base, 'rfc7320')
'http://tools.ietf.org/html/rfc7320'
>>> urljoin(base, '.')
'http://tools.ietf.org/html/'
>>> urljoin(base, '..')
'http://tools.ietf.org/'
>>> urljoin(base, '/dailydose/')
'http://tools.ietf.org/dailydose/'
>>> urljoin(base, '?version=1.0')
'http://tools.ietf.org/html/rfc3986?version=1.0'
>>> urljoin(base, '#section-5.4')
'http://tools.ietf.org/html/rfc3986#section-5.4'
```

Повторю, что мы можем спокойно предоставить абсолютный URL методу `urljoin()`, потому что он определит, что он абсолютный, и вернет его без изменений.

```
>>> urljoin(base, 'https://www.google.com/search?q=apod&btnI=yes')
'https://www.google.com/search?q=apod&btnI=yes'
```

Поскольку *относительный URL* может опустить схему и указать все остальное, мы можем создавать веб-страницы, которые не знают, поставляются они через HTTP или HTTPS, даже в статических частях сайта. В этой ситуации из базового URL воспроизводится только схема.

```
>>> urljoin(base, '//www.google.com/search?q=apod')
'http://www.google.com/search?q=apod'
```

Если вы планируете использовать относительные URL на своем сайте, следите за косой чертой в конце адресов страниц, потому что относительный URL может иметь два разных значения в зависимости от наличия косой черты в конце.

```
>>> urljoin('http://tools.ietf.org/html/rfc3986', 'rfc7320')
'http://tools.ietf.org/html/rfc7320'
>>> urljoin('http://tools.ietf.org/html/rfc3986/', 'rfc7320')
'http://tools.ietf.org/html/rfc3986/rfc7320'
```

Такая незначительная деталь имеет критическое значение. Первый URL эквивалентен обращению к каталогу `html` с целью показать файл `rfc3986`, который там находится, т. е. `html` считается текущим рабочим каталогом. Поскольку в реальной файловой системе только у каталогов в конце может стоять косая черта, второй URL рассматривает `rfc3986` как каталог.

В результате относительная ссылка, сформированная из второго URL, начинается с компонента `rfc3986`, а не с родительского каталога `html`. Всегда создавайте сайт таким образом, чтобы пользователь, который вводит некорректный URL, быстро переходил в нужное место. Например, если мы пытаемся перейти по второму URL из предыдущего примера, мы будем перенаправлены на сайт IETF. Сервер заметит ошибочную косую черту в конце и предоставит заголовок `Location` с правильным URL.

Если вы будете создавать веб-клиента, помните, что относительные URL не всегда будут относительными для пути, который мы указываем в HTTP-запросе. Если сайт будет отправлять заголовок `Location`, относительный URL должен создаваться относительно альтернативного расположения.

Язык гипертекстовой разметки HTML

Существуют целые библиотеки для работы с основными форматами документов в Интернете. Также имеются активные стандарты, которые описывают формат гипертекстовых документов, механизмы их форматирования с помощью CSS (*cascading style sheets* — каскадные таблицы стилей) и API, через который встроенный в браузер язык, например JavaScript, вносит динамические изменения в документ,

когда пользователь взаимодействует с ним или с сервера поступают дополнительные данные. Ниже описаны базовые стандарты и ресурсы.

Поскольку это книга о сетевом программировании, мы посмотрим, как эти технологии взаимодействуют с сетью.

HTML (HyperText Markup Language — язык гипертекстовой разметки документа — это способ оформления обычного текста, который использует огромное количество угловых скобок. Каждая пара угловых скобок — это тег, который открывает или закрывает элемент в документе. Примерно так будет выглядеть абзац с жирным и курсивным текстом:

```
<p>Слова в этом абзаце выделены <b>жирным</b> и <i>курсивом</i>.</p>
```

Некоторые теги, например `<br>`, создают новую строку и не имеют закрывающего тега. Иногда его называют самозакрывающимся тегом и записывают как `<br/>`. Это обязательно в XML, но можно пропустить в HTML.

В HTML допускается многое, включая пропуск закрывающих тегов. Даже если нам не встретился тег `</li>`, когда заканчивается элемент списка `<ul>`, средство парсинга само распознает, что текущий элемент, начавшийся с тега `<li>`, уже закончился.

Дизайнер может располагать элементы по-разному, в том числе вкладывая их друг в друга. Обычно одни и те же элементы из ограниченного набора HTML используются повторно в разных проектах. Да, новый стандарт HTML5 позволяет создавать новые элементы на лету в середине страницы, но дизайнеры предпочитают придерживаться стандартных подходов. На больших страницах для разметки текста можно использовать множество общих тегов `<div>` или `<span>`. Если в тексте много одинаковых тегов `<div>`, как стили CSS их различают и как JavaScript позволяет по-разному с ними взаимодействовать?

Дело в том, что автор HTML может присваивать элементам классы, по которым к ним можно обращаться. Есть два подхода к использованию классов.

Можно назначать уникальный класс каждому HTML-элементу.

```
<div class="weather">
  <h5 class="city">Provo</h5>
  <p class="temperature">61°F</p>
</div>
```

Затем в CSS и JavaScript можно обращаться к этим элементам по селекторам, например `.city`, `.temperature` или `h5.city` и `p.temperature`. Простейший селектор CSS состоит из имени тега, за которым через точку следует имя класса. Также дизайнер может решить, что заголовок `<h5>` будет служить только одной цели, абзац будет служить этой же цели, а значит, им можно присвоить общий класс.

```
<div class="weather"><h5>Provo</h5><p>61°F</p></div>
```

Придется использовать более сложные схемы, чтобы обращаться к `<h5>` и `<p>` внутри `<div>` по общему классу для `<div>`. Можно также после класса внешнего тега через пробел указать внутренний тег.

```
.weather h5
.weather p
```

Для того чтобы узнать больше, читайте документацию по CSS. Если вам интересно, как можно использовать селекторы для выбора компонентов в динамическом режиме в браузере, прочтите справку по JavaScript или документацию по хорошо продуманной библиотеке работы с документами, например jQuery.

В современном браузере, например Google Chrome или Firefox, нажмите комбинацию клавиш <Ctrl>+<U>, чтобы посмотреть HTML-код страницы.

Также можно открыть вкладку **Network** (Сеть) в инспекторе и посмотреть остальные ресурсы, которые были загружены и отрисованы при переходе на страницу. Стоит отметить, что по умолчанию панель **Network** пуста. Открыв вкладку, щелкните **Reload** (Перезагрузить), чтобы появились данные.

Если JavaScript добавил или удалил какие-то элементы на странице после первой загрузки, документ, который вы изучаете с помощью **Inspect Element**, может совершенно не походить на HTML, полученный изначально. Если какой-то элемент в инспекторе привлекает ваше внимание, но вы не можете найти его в исходном коде, воспользуйтесь вкладкой **Network** (Сеть) в отладчике, чтобы понять, какие дополнительные ресурсы JavaScript извлекает и как они использовались для создания дополнительных элементов на странице. Когда мы будем экспериментировать с небольшими веб-приложениями в следующих листингах, используйте функцию **Inspect Element** в браузере, чтобы проверять страницы, возвращаемые браузером.

Чтение и запись с использованием базы данных

Представим простое банковское приложение, в котором владельцы счетов могут переводить деньги друг другу. Как минимум в таком приложении должна быть таблица переводов, метод вставки нового перевода, метод извлечения и отображения всех переводов для текущего пользователя.

В листинге 11.1 мы подключаем небольшую библиотеку, которая использует базу данных SQLite, входящую в стандартную библиотеку Python, чтобы показать все три свои возможности. Этот код должен работать везде, где установлен Python.

Листинг 11.1. Создание базы данных и взаимодействие с ней

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Небольшая библиотека с процедурами базы данных для приложения
# банковских переводов.

import os, pprint, sqlite3
from collections import namedtuple

def open_database(path='bank.db'):
    new = not os.path.exists(path)
```

```

db = sqlite3.connect(path)
if new:
    c = db.cursor()
    c.execute('CREATE TABLE payment (id INTEGER PRIMARY KEY, '
              ' debit TEXT, credit TEXT, dollars INTEGER, memo TEXT)')
    add_payment(db, 'john', 'psf', 125, 'Registration for PyCon')
    add_payment(db, 'john', 'liz', 200, 'Payment for writing that code')
    add_payment(db, 'jason', 'john', 25, 'Gas money-thanks for the ride!')
    db.commit()
return db

def add_payment(db, debit, credit, dollars, memo):
    db.cursor().execute('INSERT INTO payment (debit, credit, dollars, memo)'
                        ' VALUES (?, ?, ?, ?)', (debit, credit, dollars, memo))

def get_payments_of(db, account):
    c = db.cursor()
    c.execute('SELECT * FROM payment WHERE credit = ? or debit = ?'
              ' ORDER BY id', (account, account))
    Row = namedtuple('Row', [tup[0] for tup in c.description])
    return [Row(*row) for row in c.fetchall()]

if __name__ == '__main__':
    db = open_database()
    pprint.pprint(get_payments_of(db, 'john'))

```

Поскольку библиотека SQLite хранит каждую базу данных в одном файле на диске, функция `open_database()` может проверить существование файла и определить, создаем мы базу данных или просто повторно открываем ее. Когда мы создаем базу данных, она создает одну таблицу переводов и заполняет ее тремя примерами переводов, чтобы в веб-приложении не отображался пустой список. Это очень упрощенная схема, но нам этого будет достаточно. В реальной жизни потребуется таблица с именами пользователей и безопасными хешами паролей, а также официальная таблица банковских счетов, откуда и куда поступают деньги. Эта программа позволяет пользователю создать пример счета при вводе данных.

Обратите внимание, что все входные данные в вызовах SQL в этом примере содержат экранированные символы. Ошибки с экранированием в интерпретируемом языке вроде SQL представляют серьезные риски для безопасности, потому что злоумышленники могут найти способ вставлять код SQL в текстовое поле. Безопаснее всего в этом вопросе будет полагаться на саму базу данных, а не на нашу логику.

Вместо того чтобы выполнять экранирование и интерполяцию самостоятельно, листинг 11.1 отправляет в SQLite вопросительный знак в тех местах, где значение нуждается в интерполяции.

Еще один важный шаг — объединять необработанные записи из базы данных во что-то более осмысленное. Метод `fetchall()` входит в DB-API 2.0, который все новые коннекторы баз данных Python предоставляют для совместимости. Более того, для каждой строки, возвращаемой из базы данных, он возвращает не объект или словарь, а кортеж:

```
(1, 'john', 'psf', 125, 'Registration for PyCon')
```

Работа с кортежами напрямую может иметь неприятные последствия. Записи `row[2]` или `row[3]` в коде могут означать, что деньги поступили на счет, или указывать сумму платежа, но их сложно будет правильно интерпретировать. В результате `bank.py` создает класс именованного кортежа, который отвечает на имена атрибутов вроде `row.credit` и `row.dollars`. Хотя создание нового класса при каждом вызове `SELECT` считается неэффективным, мы получаем нужную семантику в одну-две строки кода, чтобы в оставшееся время сосредоточиться на коде самого веб-приложения.

Ужасное веб-приложение на Flask

Давайте посмотрим на приложение `app_insecure.py` из листинга 11.2. Внимательно изучите код, прежде чем ответить на следующие вопросы. Это действительно ужасный и ненадежный код, который приведет к утечке данных и потере репутации? Он правда выглядит настолько опасным?

Листинг 11.2. (Это не вина Flask!) Небезопасное приложение

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Плохо написанное и небезопасное приложение для переводов.
# (Это не вина Flask, просто мы неправильно его используем)

import bank
from flask import Flask, redirect, request, url_for
from jinja2 import Environment, PackageLoader

app = Flask(__name__)
get = Environment(loader=PackageLoader(__name__, 'templates')).get_template

@app.route('/login', methods=['GET', 'POST'])
def login():
    username = request.form.get('username', '')
```



```
password = request.form.get('password', '')
if request.method == 'POST':
    if (username, password) in [('john', 12345678), ('sam', 'abcde')]:
        response = redirect(url_for('index'))
        response.set_cookie('username', username)
        return response
return get('login.html').render(username=username)
```

```
@app.route('/logout')
def logout():
    response = redirect(url_for('login'))
    response.set_cookie('username', '')
    return response
```

```
@app.route('/')
def index():
    username = request.cookies.get('username')
    if not username:
        return redirect(url_for('login'))
    payments = bank.get_payments_of(bank.open_database(), username)
    return get('index.html').render(payments=payments, username=username,
                                    flash_messages=request.args.getlist('flash'))
```

```
@app.route('/pay', methods=['GET', 'POST'])
def pay():
    username = request.cookies.get('username')
    if not username:
        return redirect(url_for('login'))
    account = request.form.get('account', '').strip()
    dollars = request.form.get('dollars', '').strip()
    memo = request.form.get('memo', '').strip()
    complaint = None
    if request.method == 'POST':
        if account and dollars and dollars.isdigit() and memo:
            db = bank.open_database()
            bank.add_payment(db, username, account, dollars, memo)
            db.commit()
            return redirect(url_for('index', flash='Payment successful'))
        complaint = ('Dollars must be an integer' if not dollars.isdigit()
                     else 'Please fill in all three fields')
```

```

return get('pay.html').render(complaint=complaint, account=account,
                               dollars=dollars, memo=memo)

if __name__ == '__main__':
    app.debug = True
    app.run()

```

Этот код уязвим для многих самых распространенных векторов атак в Интернете. Далее мы рассмотрим главные недостатки этого приложения, чтобы узнать, чего ему не хватает. Все ошибки связаны с обработкой данных на сайте и не имеют отношения к защите сайта с помощью TLS. Даже если трафик зашифрован, например с помощью обратного прокси перед сервером (см. главу 10), злоумышленник сможет нанести вред, не видя данные, передаваемые между пользователем и программой.

Программа использует веб-фреймворк Flask для обработки основных операций онлайн-приложения Python: обработка входных данных из HTML-форм (об этом мы поговорим в следующем разделе), отправка сообщения об ошибке 404 для страниц, которые приложение не определяет, а также упрощение создания HTTP-ответов с HTML-содержимым из одного источника или перенаправление на другой URL. Ознакомьтесь с документацией по Flask на сайте <http://flask.pocoo.org/> или читайте дальше эту главу.

Допустим, этот код написан программистом, который не знаком с Интернетом. Он что-то слышал о языках шаблонов, с помощью которых можно легко добавлять пользовательский текст в HTML, и узнал, как загрузить и запустить Jinja2. Он также узнал, что микрофреймворк Flask уступает по популярности только Django. Он выбрал Flask, чтобы приложение помещалось в один файл.

В коде есть страницы `login()` и `logout()`. На экране входа жестко закодированы два возможных пользовательских аккаунта и пароли к ним, потому что у приложения нет базы данных пользователей. Скоро мы узнаем больше о логике форм, но уже сейчас мы видим, что при входе и выходе создается и удаляется cookie (см. главы 9 и 10), который указывается в последующих запросах и обозначает, что запрос поступил от пользователя, прошедшего аутентификацию.

Остальные две страницы сайта защищают себя от неавторизованных посетителей, проверяя наличие этого cookie, а если его нет, пользователь перенаправляется на страницу входа. Помимо проверки на аутентификацию `login()` включает только две строки кода (на самом деле три, потому что одна из них длинная): берет информацию о переводах пользователя из базы данных и объединяет ее с другой информацией, чтобы представить эти сведения в шаблоне HTML-страницы.

Кажется очевидным, что странице нужно знать имя пользователя, но почему код ищет сообщение flash в параметрах URL (которые Flask предоставляет как словарь `request.args`)?

Если почитать страницу `pay()`, ответ будет очевиден. Пользователь будет перенаправлен на страницу `index` после успешного перевода, но, скорее всего, ему потре-

буется какой-то индикатор того, что в форме были указаны все нужные данные. Для этого в верхней части страницы отображается флеш-сообщение, как это называется во Flask. (Flash переводится как вспышка, и сообщение так называется потому, что появляется, а затем исчезает. Это никак не связано с Adobe Flash.) Флеш-сообщение передается как строка запроса в URL при первой итерации веб-приложения.

`http://example.com/?flash=Payment+successful`

Оставшаяся часть процедуры `pay()` понятна: приложение проверяет, была ли форма отправлена успешно, и если да, выполняет какие-то операции. Поскольку пользователь или браузер мог предоставить или пропустить аргументы формы, код использует метод `get()` запроса, чтобы конфиденциально и безопасно проверить их. Если какой-то ключ отсутствует, словарь формы может вернуть значение по умолчанию (здесь это пустая строка).

Если запрос одобрен, перевод сохраняется в базе данных на неопределенный срок. В противном случае форма будет показана пользователю, но не пустая, а с уже введенными значениями, чтобы их не пришлось вводить повторно.

Изучите три шаблона HTML в листинге 11.2, чтобы лучше понимать обсуждение форм и методов в следующем разделе. Поскольку стандартные характеристики дизайна HTML входят в базовый шаблон, который является самым популярным паттерном для многостраничных сайтов, у нас здесь четыре шаблона. Шаблон в листинге 11.3 указывает структуру с точками вставок для заголовка страницы и тела страницы, которые могут добавляться из других шаблонов. Так как язык шаблонов Jinja2 хорошо структурирован (он написан Армином Ронахером, создателем Werkzeug (см. главу 10) и Flask), заголовок можно использовать дважды: в элементе `<title>` и в элементе `<h1>`.

Листинг 11.3. Страница `base.html` шаблона Jinja2

```
<html>
  <head>
    <title>{% block title %}{% endblock %}</title>
    <link rel="stylesheet" type="text/css" href="/static/style.css">
  </head>
  <body>
    <h1>{{ self.title() }}</h1>
    {% block body %}{% endblock %}
  </body>
</html>
```

Шаблон Jinja2 определяет, например, что мы можем попросить заменить значение в шаблоне с помощью двойных фигурных скобок, как для имени пользователя, и с

помощью сочетания фигурных скобок с символами процента можно сделать цикл и снова вывести тот же шаблон HTML. Дополнительную информацию о синтаксисе и возможностях см. на сайте <http://jinja.pocoo.org/>.

Единственные элементы на странице входа из листинга 11.4 — это заголовок и сама форма. В первый раз мы увидим в элементе формы начальное значение (`value="..."`), которое уже должно присутствовать в редактируемом элементе, когда он впервые отображается на экране.

Листинг 11.4. Страница login.html шаблона Jinja2

```
{% extends "base.html" %}
{% block title %}Please log in{% endblock %}
{% block body %}
<form method="post">
  <label>User: <input name="username" value="{{ username }}"></label>
  <label>Password: <input name="password" type="password"></label>
  <button type="submit">Log in</button>
</form>
{% endblock %}
```

При заполнении этой формы пользователю не придется снова вводить имя пользователя, если он ошибся при вводе пароля, потому что в значении `value="..."` будет вставляться его имя.

Как видно в листинге 11.5, на странице `index` по адресу / происходит еще много интересного. Если есть флеш-сообщения, они будут отображаться сразу под заголовком. В следующем разделе содержится неупорядоченный список (`<ul>`) элементов (`<li>`), соответствующих сделанным переводам, а сверху заголовок "Your Payments" ("Ваши переводы"). Наконец, мы видим ссылки на страницу нового перевода и выхода.

Листинг 11.5. Страница index.html шаблона Jinja2

```
{% extends "base.html" %}
{% block title %}Welcome, {{ username }}{% endblock %}
{% block body %}
{% for message in flash_messages %}
<div class="flash_message">{{ message }}<a href="/">&times;</a></div>
{% endfor %}
<p>Your Payments</p>
<ul>
  {% for p in payments %}
    {% set prep = 'from' if (p.credit == username) else 'to' %}
```

```

    {% set acct = p.debit if (p.credit == username) else p.credit %}
    <li class="{{ prep }}">${{ p.dollars }} {{ prep }} <b>{{ acct }}</b>
    for: <i>{{ p.memo }}</i></li>
  {% endfor %}
</ul>
<a href="/pay">Make payment</a> | <a href="/logout">Log out</a>
{% endblock %}

```

Стоит отметить, что код не хочет постоянно отображать имя счета текущего пользователя, проходя по входящим и исходящим переводам. Вместо этого он для каждого перевода определяет правильность имени счета зачисления или списания, которое соответствует текущему пользователю, а затем выводит имя второго счета с подходящим предлогом, чтобы пользователь видел направление перевода. Для этого используется команда Jinja2 `% set... %`, которая упрощает реализацию этих расчетов прямо в шаблоне. Пользователь может допустить множество разных ошибок при заполнении формы, и листинг 11.6 предусматривает получение ошибки и отображает ее в верхней части формы.

В остальном этот код почти такой же: три поля формы, которые, если форма заполнена неправильно, отображаются снова, заполненные значениями, которые пользователь ввел, когда пытался отправить форму.

Листинг 11.6. Страница `pay.html` шаблона Jinja2

```

{% extends "base.html" %}
{% block title %}Make a Payment{% endblock %}
{% block body %}
<form method="post" action="/pay">
  {% if complaint %}<span class="complaint">{{ complaint }}</span>{% endif %}
  <label>To account: <input name="account" value="{{ account }}"></ label>
  <label>Dollars: <input name="dollars" value="{{ dollars }}"></ label>
  <label>Memo: <input name="memo" value="{{ memo }}"></label>
  <button type="submit">Send money</button> | <a href="/">Cancel</a>
</form>
{% endblock %}

```

У каждой кнопки отправки формы должна быть альтернатива — "путь отступления". Эксперименты показывают, что пользователи делают меньше ошибок, если путь отступления проще, чем изначальные действия по заполнению формы, и эта альтернатива не должна выглядеть как кнопка. Поэтому в `pay.html` путь отступления **Cancel** (Отмена) выглядит как ссылка и визуально отделен от кнопки символом `()`.

Для того чтобы испробовать это приложение, перейдите в репозиторий исходного кода и введите следующие команды в каталоге `chapter11`, который содержит файлы `bank.py`, `app_insecure.py` и каталог `templates/`.

```
$ pip install flask
```

```
$ python3 app_insecure.py
```

Мы должны получить сообщение о том, что приложение запущено, а также URL.

```
* Running on http://127.0.0.1:5000/
```

```
* Restarting with reloader
```

Flask сам себя перезапустит и перезагрузит приложение, если мы изменим один из листингов в режиме отладки (см. предпоследнюю строку в листинге 11.2), поэтому можно сразу видеть последствия любых изменений в коде.

Нам еще кое-чего не хватает: где файл `style.css`, упомянутый в листинге 11.3? Он находится в каталоге `static/`, рядом с приложением в репозитории. Изучите его, если вас интересует не только сетевое программирование, но и веб-дизайн.

Методы и формы HTTP

Действие по умолчанию для HTML-формы — GET, а форма может содержать всего одно поле ввода.

```
<form action="/search">
  <label>Search: <input name="q"></label>
  <button type="submit">Go</button>
</form>
```

Мы не будем подробно рассматривать создание форм, потому что это обширная тема со множеством технических нюансов. Кроме текстовых полей вроде этого, существуют и другие возможности. Даже у текстовых полей есть варианты. Можно, например, с помощью CSS3 добавить пример текста, который исчезнет, как только пользователь начнет печатать. Можно сделать так, чтобы кнопка отправки была неактивной, пока пользователь не введет поисковой запрос, используя код JavaScript. Можно включить инструкции или примеры ключевых слов под полем ввода, чтобы помочь пользователю. Можно написать на кнопке отправки "Отправить" или более конкретное действие, которое описывает отправку формы на сервер. Можно даже удалить кнопку **Поиск**, чтобы упростить дизайн, но тогда пользователю нужно догадаться, что после ввода запроса нужно нажать клавишу <Enter>.

Эти темы подробно описаны в книгах по веб-дизайну. В этой книге мы будем работать с формами только применительно к сети.

Поля ввода формы GET указываются напрямую в URL, а значит, и в маршруте, переданном с HTTP-запросом.

GET /search?q=Programming+in+Python++The+Basics HTTP/1.1

Host: example.com

Понимаете, что это значит? Параметры GET сохраняются в истории браузера, где их могут увидеть. Значит, нельзя использовать GET для передачи конфиденциальных данных, например пароля. При отправке формы с помощью GET мы как бы говорим браузеру, куда хотим перейти дальше, и браузер создает для нас URL-адрес, ведущий на страницу, которую сервер должен для нас создать. Если мы введем в поле поиска три разных запроса, будут созданы три независимые страницы, три записи в истории браузера, чтобы мы могли вернуться на эти страницы, и три URL, которыми можно поделиться с другими людьми, чтобы они увидели те же результаты поиска.

Для этого мы просто указываем данные в форме, а они включаются в запрос GET.

Кроме этого метода, существуют POST, PUT и DELETE.

При их использовании данные из формы не включаются в URL и в путь HTTP-запроса.

```
<form method="post" action="/donate">
  <label>Charity: <input name="name"></label>
  <label>Amount: <input name="dollars"></label>
  <button type="submit">Donate</button>
</form>
```

Когда мы отправляем HTML-форму, браузер помещает все данные в тело запроса, оставляя путь пустым.

POST /donate HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 39

name=PyCon%20scholarships&dollars=35

Мы переходим на страницу "\$35 for PyCon scholarships" ("35 долларов на стипендию PyCon") не потому, что нам просто любопытно. Мы совершаем действие, и если мы выполним POST дважды, итоговая сумма увеличится вдвое. "\$35 for PyCon scholarships" — это не имя сайта, который мы хотим посетить, поэтому параметры формы не включаются в URL. Это похоже на то, что философ Джон Лэнгшо Остин (J. L. Austin) называл речевым актом — словами, которые меняют действительность. Кстати, браузеры могут загружать большой объем полезных данных, целые файлы, используя альтернативное кодирование multipart/form на основе стандарта MIME (см. главу 12). Семантика формы POST будет одинаковой в любом случае.

Подтверждение повторной отправки формы

На странице, которую вы ищете, использовалась введенная вами информация. При возврате на эту страницу может потребоваться повторить выполненные ранее действия. Продолжить?

Вы наверняка видели подобные предупреждения в браузере. При просмотре формы пользователем очевидно, что форма не была отправлена, однако у браузера нет возможности понять, что запрос POST не прошел. Он отправил POST, получил страницу. Возможно, на странице отображается сообщение с благодарностью за пожертвование в 1000 долларов, и если он отправит форму, деньги спишутся снова.

Сайты могут использовать одну из двух стратегий, чтобы не оставлять пользователя на странице, которую он получил в результате POST и на которой невозможно спокойно воспользоваться кнопками обновления и перехода вперед и назад.

- ◆ Предотвратить ввод пользователем ошибочных значений с помощью ограничений, прописанных в JavaScript и HTML5. Если кнопка отправки не активна, пока форма не готова к отправке, или если можно обработать данные в форме с помощью JavaScript без перезагрузки страницы, тогда ошибочная отправка (например, с пустыми полями) не приведет нас на проблемную страницу с методом POST.
- ◆ Когда форма наконец будет корректно отправлена и действие будет выполнено успешно, веб-приложение не должно сразу отправлять код 200 OK. Отправьте 303 See Other (прочее), чтобы перенаправить пользователя по другому URL, предоставленному в заголовке Location. В итоге после успешного POST браузер сразу через метод GET перенаправит пользователя на другую страницу. Теперь пользователь может нажимать кнопки обновления и перехода вперед и назад сколько угодно раз — запросы GET будут безопасно повторяться без попыток отправить форму повторно.

Приложение в листинге 11.2 слишком простое, чтобы помешать пользователю видеть ответ POST, если форма заполнена некорректно, но оно отправляет 303 See Other с помощью функции `Flask redirect() Object() { [код] }`, когда форма `/login` или `/pay` отправлены успешно.

Когда формы используют неподходящие методы

Неправильное использование методов HTTP в веб-приложениях вызывает проблемы с автоматизированными инструментами, ожиданиями пользователей и браузером.

У меня есть друг, который размещал сайт своего маленького бизнеса в системе управления контентом на PHP у местного провайдера. В кабинете администратора отображались ссылки на фотографии, которые использовались на сайте.

Он загрузил фотографии по всем ссылкам, чтобы сохранить копию на своем компьютере. Через несколько минут ему написал друг — спросить, почему с сайта исчезли все фотографии.

Оказалось, что кнопка удаления рядом с каждой фотографией не была настоящей кнопкой, которая запускала операцию POST. Это были ссылки с обычным URL, которые удаляли изображение, которое вы посещаете. Поскольку GET всегда и при любых обстоятельствах должен быть безопасной операцией, браузер выполнил

GET для сотен ссылок на странице. Хостинговый провайдер подвел и пришлось восстанавливать сайт из резервной копии.

Обратное действие — использование POST для чтения — не имеет таких серьезных последствий. Этот метод не удаляет файлы, а просто мешает работе.

Однажды мне довелось использовать поисковый движок, разработанный одной компанией для собственных нужд. Я сделал несколько поисковых запросов, нашел нужное и скопировал ссылку на страницу с результатами, чтобы отправить ее своему начальнику.

Я был в ужасе, когда увидел URL. Я понятия не имел, как работает сервер, но был уверен, что когда начальник перейдет по ссылке, /search.pl не выдаст ему те же результаты.

Дело в том, что форма поиска использовала POST, так что запрос не отображался в строке браузера, т. е. URL для каждого поиска был одинаковым, и его нельзя было отправлять другим или сохранять в закладки. Я нажимал кнопки перехода назад и вперед в браузере, чтобы просмотреть свои поисковые запросы, и каждый раз видел окошко, в котором меня спрашивали, действительно ли я хочу отправить запрос повторно. Браузер думал, что эти операции POST могли иметь негативные последствия.

Мы должны использовать GET для поиска и POST для действий не только потому, что так велят правила. Это влияет и на пользовательский опыт.

Опасные и безопасные cookie

В листинге 11.2 приводится веб-приложение, которое пытается защитить конфиденциальность пользователя. В ответ на GET для страницы / требуется, чтобы пользователь выполнил вход, прежде чем можно будет показать список его переводов. Также пользователь должен войти, прежде чем можно будет принять операцию POST для формы /pay, через которую пользователь совершает перевод. К сожалению, таким приложением может воспользоваться злоумышленник, чтобы делать переводы от имени другого пользователя.

Злоумышленник может создать счет, чтобы посмотреть, как работает приложение. Он может запустить инструменты отладки в браузере, а затем войти на сайт и просмотреть входящие и исходящие заголовки на вкладке **Сеть**. Что он увидит, когда введет свое имя пользователя и пароль?

HTTP/1.0 302 FOUND

...

Set-Cookie: username=badguy; Path=

...

Интересно, правда? Браузер получил cookie с именем username (имя пользователя) и значением имени пользователя (badguy). Получается, сайт считает, что если запрос содержит этот cookie, пользователь правильно ввел логин и пароль.

Разумеется, вызывающий объект может поставить для cookie любое значение.

Он может подделать cookie с помощью определенных параметров конфиденциальности в браузере или использовать Python для посещения сайта. С помощью модуля request он может попробовать получить первую страницу. Как и ожидалось, если запрос не прошел аутентификацию, пользователь перенаправляется на страницу /login.

```
>>> import requests
>>> r = requests.get('http://localhost:5000/')
>>> print(r.url)
http://localhost:5000/login
```

А если злоумышленник разместит cookie и сделает вид, что некий пользователь john прошел аутентификацию и выполнил вход?

```
>>> r = requests.get('http://localhost:5000/', cookies={'username': 'john'})
>>> print(r.url)
http://localhost:5000/
```

Получилось! Сайт видит значение cookie и считает, что HTTP-запросы с этим cookie исходят от соответствующего пользователя. Злоумышленнику достаточно получить логин другого пользователя. чтобы подделать запрос и отправить перевод куда угодно.

```
>>> r = requests.post('http://localhost:5000/pay',
... {'account': 'hacker', 'dollars': 100, 'memo': 'Auto-pay'},
... cookies={'username': 'john'})
>>> print(r.url)
http://localhost:5000/?flash=Payment+successful
```

Все получилось. Злоумышленник перевел \$100 со счета пользователя john на свой банковский счет. Из этого следует, что мы никогда не должны создавать cookie таким образом, чтобы пользователь мог менять их значение по своему желанию. Если у нас очень умные пользователи, они поймут, что вы скрываете их логины с помощью кодировки Base64, меняете местами буквы или используете исключяющее ИЛИ для значения с постоянной маской. Есть три безопасных и эффективных метода защитить cookie от подделок.

- ◆ Можно подписывать cookie цифровой подписью и оставлять его на виду. Это обескуражит злоумышленников: они видят cookie со своим именем пользователя и пробуют переписать его, чтобы получить доступ к чужому счету, но не могут подделать цифровую подпись для этого нового cookie, и сайт не поверит в его подлинность.
- ◆ Можно полностью зашифровать cookie, чтобы пользователь не мог расшифровать его значение.
- ◆ Можно с помощью стандартной библиотеки UUID создавать совершенно случайные и бессмысленные строки для cookie и хранить их в базе данных, чтобы сопоставлять эти cookie с реальными пользователями. Если много HTTP-

запросов от одного пользователя направляются на отдельные серверы, это постоянное хранилище сеансов должно быть доступно всем фронтенд-серверам. Некоторые приложения хранят сеансы в главной базе данных, а другие используют Redis или иное краткосрочное хранилище, чтобы не перегружать основное постоянное хранилище данных запросами.

В этом примере приложения мы можем использовать Flask для цифрового подписания cookie, чтобы их нельзя было подделать. На реальном рабочем сервере ключ подписания должен храниться отдельно от исходного кода, но в этом примере мы поместим его в верхней части файла. Если мы разместим реальный ключ в исходном коде, любой пользователь с доступом в нашу систему контроля версий сможет прочитать этот ключ. Более того, ключи будут доступны на ноутбуках разработчиков и в системе непрерывной интеграции.

```
app.secret_key = 'saigeij8AiS2ahleahMo5dahveixuV4J'
```

Flask будет оперировать секретным ключом каждый раз, как мы используем его уникальный объект сеанса, чтобы задать cookie, например во время входа.

```
session['username'] = username
```

```
session['csrf_token'] = uuid.uuid4().hex
```

Flask использует ключ еще раз, прежде чем поверить значению cookie, извлеченному из входящего запроса. Cookie с неверной подписью будет считаться поддельным и игнорироваться.

```
username = session.get('username')
```

В листинге 11.8 мы увидим эти улучшения в действии.

Кстати, cookie не следует отправлять по незашифрованному каналу HTTP, потому что иначе они будут видны любому пользователю той же беспроводной сети в кофейне. Многие сайты используют HTTPS для страницы входа, чтобы защитить cookie, а затем полностью их раскрывают, когда браузер загружает CSS, JavaScript и изображения через HTTP с хоста с тем же именем.

Узнайте, как сделать так, чтобы ваш веб-фреймворк требовал защищенной передачи для каждого cookie, который вы отправляете в браузер, чтобы избежать раскрытия cookie. В этом случае он не будет включаться в незашифрованные запросы к ресурсам, к которым есть доступ у всех.

Непостоянный межсайтовый скриптинг

Если злоумышленник не может украсть или подделать cookie, чтобы выдать себя за другого пользователя, он может применить другую тактику. Ему не понадобятся cookies, если он поймет, как контролировать браузер другого пользователя, пока он работает в системе. Cookie будет сам добавляться при каждом запросе, когда мы работаем через этот браузер.

У этой атаки есть три основных подхода. Сервер в листинге 11.2 уязвим для всех трех. Первый подход — непостоянный межсайтовый скриптинг (cross-site scripting,

XSS), при котором злоумышленник пытается выдать написанный им текст за обычный текст сайта. Предположим, злоумышленник хочет отправить \$110 на свой банковский счет. Он может написать JavaScript-код, как в листинге 11.7.

Листинг 11.7. attack.js — скрипт для незаконных переводов

```
<script>
var x = new XMLHttpRequest(); x.open('POST', 'http://localhost:5000/pay');
x.setRequestHeader('Content-Type','application/x-www-form-urlencoded');
x.send('account=hacker&dollars=110&memo=Theft');
</script>
```

Если этот код отображается на сайте, только когда пользователь выполнил вход в приложение, запрос POST будет срабатывать и делать переводы от имени законного пользователя. Поскольку код в тегах `<script>` мы не видим на странице, пользователь не заметит ничего подозрительного, пока не нажмет комбинацию клавиш `<Ctrl>+<U>`, чтобы прочесть исходный код. И даже тогда он еще должен понять, что элемент `<script>` выглядит странно и не должен присутствовать на странице.

Как злоумышленник может отобразить этот HTML? Злоумышленник может вставить этот HTML в шаблон страницы / через флеш-параметр. Автор кода из листинга 11.2, очевидно, невнимательно читал документацию и не знает, что Jinja2 в чистом виде автоматически не экранирует специальные символы вроде `>`, потому что сам по себе не знает, что вы используете его для HTML.

Злоумышленник может создать URL, который включает скрипт во флеш-параметре.

```
>>> with open('/home/john/py3/chapter11/attack.js') as f:  
... query = {'flash': f.read().strip().replace('\n', ' ')}  
>>> print('http://localhost:5000/?' + urlencode(query))  
http://localhost:5000/?flash=%3Cscript%3E+var+x+%3D+new+XMLHttpRequest  
%28%29%3B+x.open%28%27+POST%27%2C+%27http%3A%2F%2Flocalhost%3A5000%2Fpay%27  
%29%3B+x.setRequestHeader%28%27Content-Type%27%2C+%27application%2Fxml-www-form-urlencoded%27%29%3B+x.send%28%27account%3Dhacker%26dolars%3D110%26memo%3DTheft%27%29%3B+%3C%2Fscript%3E
```

Наконец, злоумышленник должен придумать способ убедить пользователя щелкнуть по ссылке.

Это сложно, если он нацелен на конкретного пользователя. Возможно, придется имитировать электронное сообщение от настоящего знакомого жертвы и оформить его так, чтобы пользователь захотел нажать на ссылку. Тут нужно проявить изобретательность. Злоумышленник может общаться с пользователем в открытом чате и скинуть ему ссылку под видом статьи на тему, по которой пользователь высказался. Ссылка выше может вызвать у пользователя подозрения, так что злоумышленник может поделиться укороченной ссылкой, которая развернется, только если на нее нажать.

Если злоумышленник атакует большой сайт с миллионами пользователей, можно придумать что-то попроще. Достаточно разослать по электронной почте миллион убедительных сообщений со ссылкой, и кто-нибудь обязательно перейдет по ней и станет жертвой этой схемы.

Попробуйте создать ссылку с помощью кода с использованием модуля `requests`, который мы написали ранее. Затем перейдите по ссылке после входа в приложение и без такого входа. Войдя в приложение, вы заметите, что при каждой перезагрузке главной страницы отправляется еще один перевод, который обрабатывается автоматически нажатой ссылкой. Для того чтобы убедиться, что JavaScript в тегах `<script>` попал на страницу, нажмите комбинацию клавиш `<Ctrl>+<U>` в Firefox или Google Chrome.

Если вы заметите, что атака не работает, перейдите в консоль JavaScript в браузере. В моей версии Chrome я вижу сообщение о том, что XSS Auditor отказался выполнять скрипт, потому что его исходный код был обнаружен в запросе. Хороший современный браузер может поддаться этой атаке, только если защита выключена или злоумышленник найдет более продуманный способ использовать флеш-сообщение.

Даже если атака пройдет успешно, пользователь может что-то заподозрить, когда на экране появится пустое поле сообщения без текста. В качестве упражнения попытайтесь исправить этот недостаток в предыдущем URL: за пределами тегов `<script>` попробуйте передать какой-нибудь контент, например сообщение "Welcome back" ("С возвращением"), чтобы раздел сообщения выглядел более правдоподобно. Для защиты от атаки, написанной в листинге 11.8, необходимо полностью удалить флеш-сообщение — контекстную информацию о том, какую операцию выполнила форма `/pay`, которую приложение хочет отобразить на следующей странице, куда перейдет пользователь. Вместо этого можно хранить флеш-сообщение на сервере до поступления следующего запроса. Flask предлагает для этого функции `flash()` и `get_flashed_messages()`.

Постоянный межсайтовый скриптинг

Мы можем вставить свой код JavaScript в поле Мемо, куда отправитель записывает сообщение для получателя. Войдите в приложение как пользователь `sam`, указав пароль из листинга 11.2, а затем попробуйте отправить мне перевод. В сообщении напишите благодарность за эту замечательную книгу, чтобы я не удивился, откуда поступил платеж. Поля будут отображаться следующим образом, когда вы уже добавите элемент скрипта, но до того, как нажмете **Send money** (Отправить перевод):

To account: john

Dollars: 1

Memo: A small thank-you.<script>...</script>

Затем нажмите кнопку отправки. Выйдите из приложения, войдите как `john` и начните нажимать кнопку перезагрузки. Перевод будет списываться со счета пользователя `john` каждый раз, как он будет попадать на страницу.

Как видите, постоянная межсайтовая атака работает очень эффективно: JavaScript выполняется каждый раз, как пользователь заходит на сайт, и атака происходит снова и снова, пока данные не будут удалены с сервера, тогда как предыдущая ссылка (при непостоянной атаке) работала, только если пользователь щелкал на ней. Когда XSS-атаки впервые были запущены через открытые сообщения на уязвимых сайтах, пострадали сотни тысяч людей, пока проблему не исправили. Поскольку автор использовал шаблоны Jinja2, не до конца понимая их, листинг 11.2 уязвим для таких атак. Судя по документации, экранирование не происходит автоматически. Jinja2 сохраняет специальные символы HTML вроде `<` и `>`, если вы знаете, как включить их экранирование.

Используя Jinja2 в функции Flask `render_template()`, листинг 11.8 защищает от XSS-атак. Когда он видит расширение `html`, он автоматически включает экранирование HTML. Мы можем защититься от ошибок, которые можно наделать при написании собственного кода, если будем использовать стандартные шаблоны, предлагаемые веб-фреймворком.

Подделка межсайтовых запросов

XSS-атаки не должны вызывать проблемы на сайте, когда все содержимое корректно экранировано. Однако у злоумышленника есть еще одна возможность: отправить форму с другого сайта, потому что совсем не обязательно начинать атаку с нашего. Он может заранее предсказать значение всех полей, чтобы отправить запрос странице `/pay` с любой другой страницы. Например, он может использовать форум на сайте, где не прописано корректное экранирование или не удаляются теги `<script>` из комментариев, и убедить пользователей посетить сайт, на котором он скрыл код JavaScript, или встроить его в комментарий. Злоумышленнику достаточно создать форму, которая активирует отправку ему денег, а затем сделать для этой формы привлекательную кнопку.

```
<form method="post" action="http://localhost:5000/pay">
  <input type="hidden" name="account" value="pam">
  <input type="hidden" name="dollars" value="220">
  <input type="hidden" name="message" value="Someone won big">
  <button type="submit">Reply</button>
</form>
```

Поскольку JavaScript наверняка включен у вас в браузере, злоумышленник может просто скопировать и вставить элемент `<script>` из листинга 11.7 на страницу, в пост на форуме или примечание, а затем сидеть и ждать зачисления денег на свой счет.

Это классическая межсайтовая подделка запроса (cross-site request forgery, CSRF), для которой злоумышленнику не нужно придумывать, как попасть в платежную систему. Достаточно использовать простые формы перевода на любом сайте в лю-

бой точке мира, куда можно внедрить код JavaScript. Каждый посещаемый вами сайт должен защищаться от такого внедрения.

В приложениях тоже должна быть такая защита.

Как приложение может защититься от атак CSRF? Затруднив заполнение и отправку форм. Например, форма может содержать не только минимальный набор полей, необходимый для отправки перевода, но и дополнительное поле с секретной информацией, которую видит только законный пользователь или его браузер. Она не обязательно должна отображаться для пользователя, читающего и использующего форму. Поскольку злоумышленник не узнает значение секрета, скрытого в каждой отправляемой форме /pay, он не сможет создать убедительный POST для адреса.

В листинге 11.8 используется способность Flask безопасно хранить секреты в cookie, чтобы присваивать каждому пользователю случайную секретную строку при каждом входе. Этот пример предполагает, что платежный сайт будет защищен с помощью HTTPS, так что никто не увидит этот секрет или cookie при передаче. Платежный сайт может добавлять случайный секрет сеанса к каждой форме /pay, которая отображается пользователю. HTML включает скрытые поля формы как раз для таких случаев. В файле pay2.html, заменяющем листинг 11.6, который будет использоваться в листинге 11.8, следующее поле добавляется в форму:

```
<input name="csrf_token" type="hidden" value="{{ csrf_token }}">
```

При каждой отправке формы проводится дополнительная проверка, чтобы значение CSRF из формы соответствовало тому, что было отправлено пользователю в HTML-версии формы. Если они не совпадают, сайт считает, что злоумышленник пытается отправить форму от имени пользователя, и возвращает ошибку 403 Forbidden (запрещено).

Защита от CSRF в листинге 11.8 написана вручную, чтобы продемонстрировать все компоненты и использование дополнительного поля, которое мешает злоумышленнику наугад собрать форму. На практике защита от CSRF должна быть встроена в выбранный веб-фреймворк или как минимум предлагаться как стандартный плагин. Сообщество Flask предлагает несколько способов, в том числе вариант из популярного пакета Flask-WTF для создания и парсинга HTML-форм.

Улучшенная программа

В листинге 11.8 приводится приложение app_improved.py — улучшенное, но не идеальное, потому что невозможно гарантировать абсолютную защиту от всех рисков.

Листинг 11.8. Платежное приложение: app_improved.py

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Платежное приложение с улучшенной безопасностью.
```

```

import bank, uuid
from flask import (Flask, abort, flash, get_flashed_messages,
                   redirect, render_template, request, session, url_for)

app = Flask( name )
app.secret_key = 'saiGeij8AiS2ahleahMo5dahveixuV4J'

@app.route('/login', methods=['GET', 'POST'])
def login():
    username = request.form.get('username', '')
    password = request.form.get('password', '')
    if request.method == 'POST':
        if (username, password) in [('john', '123456789'), ('pam', 'abcde')]:
            session['username'] = username
            session['csrf_token'] = uuid.uuid4().hex
            return redirect(url_for('index'))
    return render_template('login.html', username=username)

@app.route('/logout')
def logout():
    session.pop('username', None)
    return redirect(url_for('login'))

@app.route('/')
def index():
    username = session.get('username')
    if not username:
        return redirect(url_for('login'))
    payments = bank.get_payments_of(bank.open_database(), username)
    return render_template('index.html', payments=payments, username=username,
                           flash_messages=get_flashed_messages())

@app.route('/pay', methods=['GET', 'POST'])
def pay():
    username = session.get('username')
    if not username:
        return redirect(url_for('login'))
    account = request.form.get('account', '').strip()
    dollars = request.form.get('dollars', '').strip()
    memo = request.form.get('memo', '').strip()
    complaint = None

```



```

if request.method == 'POST':
    if request.form.get('csrf_token') != session['csrf_token']:
        abort(403)
    if account and dollars and dollars.isdigit() and memo:
        db = bank.open_database()
        bank.add_payment(db, username, account, dollars, memo)
        db.commit()
        flash('Payment successful')
        return redirect(url_for('index'))
    complaint = ('Dollars must be an integer' if not dollars.isdigit()
                 else 'Please fill in all three fields')
    return render_template('pay2.html', complaint=complaint, account=account,
                           dollars=dollars, memo=memo,
                           csrf_token=session['csrf_token'])

if __name__ == '__main__':
    app.debug = True
    app.run()

```

Уязвимость Shellshock, например, затрагивает командную оболочку Bash, которая выполняет любой переданный ей код как специально отформатированные переменные окружения — вроде тех, которые старый механизм CGI задавал на основе недоверенных входящих HTTP — незаметно для всех. Я не могу гарантировать безопасность приложения в этом листинге, потому что злоумышленники постоянно придумывают новые способы. Однако в этом коде выполняется корректное экранирование, флеш-сообщения отправляются во внутреннее хранилище, а не передаются через браузер, и в каждой форме есть скрытый случайный UUID, защищающий ее от подделок. Стоит отметить, что два важных улучшения (хранение флеш-сообщений во внутреннем хранилище и экранирование символов через Jinja2 до их добавления в HTML) выполнены с помощью стандартных функций Flask, а не специально написанного кода. Советую внимательно читать документацию по фреймворкам и использовать максимум их возможностей, чтобы сделать код приложения более лаконичным, а также безопасным — ведь эти шаблоны созданы профессионалами и постоянно улучшаются усилиями сообщества. Во многих ситуациях эти возможности помогут вам исправить проблемы с безопасностью и производительностью, о которых вы могли даже не подозревать.

Что касается взаимодействия с сетью, приложение можно довольно эффективно автоматизировать, но в плане представлений и форм нужно сгладить еще немало шероховатостей. Нужно написать код, который будет проверять, что пользователь вошел в систему. Значение полей в форме нужно копировать из запроса в HTML, чтобы пользователю не пришлось вводить их заново. Взаимодействие с базой данных осуществляется на довольно низком уровне. Если мы хотим, чтобы переводы отправлялись на постоянное хранение в SQLite, нужно открывать сессии с базой данных вручную и сохранять там информацию.

В сообществе Flask вы найдете полезные рекомендации и сторонние инструменты для выполнения этих задач. Последним примером будет то же приложение, созданное в одном фреймворке, которое решает эти вопросы за нас.

Приложение для оплаты на Django

Django — это фулстек-решение, где есть все необходимое для начинающих программистов. Пожалуй, это самый популярный веб-фреймворк среди разработчиков Python. С помощью Django можно не только работать с шаблонами и URL, но и взаимодействовать с базой данных, представлять результаты как объекты Python и даже составлять и интерпретировать формы, не устанавливая сторонние библиотеки. Фреймворк с упорядоченными и безопасными паттернами может быть удобнее, чем более гибкий инструмент, при использовании которого программистам приходится отдельно подбирать ORM (Object Relational Mapper — объектно-реляционный преобразователь) и библиотеку форм, хотя они могут слабо представлять себе, как все эти компоненты сочетаются. Приложение Django целиком приводится в репозитории кода для этой книги. URL репозитория для этой главы¹:

<https://bpbonline.com/Programming in Python: The Basics/py3/chapter11>

Там есть несколько стандартных файлов, которые не стоит целиком приводить в этой книге.

- ◆ **manage.py.** Исполняемый скрипт, который позволяет выполнять команды Django для подготовки и запуска приложения в режиме разработки.
- ◆ **djbank/init.py.** Это пустой файл, который уведомляет Python о том, что каталог является пакетом Python, который можно использовать для импорта модулей.
- ◆ **djbank/admin.py.** Файл содержит три строки кода, которые отображают модель Payment в разделе администратора, как описывается в *разд. "Выбор вер-фреймворка для веб-сайта" далее в этой главе.*
- ◆ **djbank/wsgi.py.** Этот файл содержит вызываемый объект WSGI, который веб-сервер, соответствующий требованиям WSGI, например Gunicorn или Apache (см. главу 10), может использовать, чтобы запускать приложение для переводов.

Далее представлены четыре интересных скрипта, которые демонстрируют, что фреймворк уже поддерживает множество распространенных паттернов, которые код Python может использовать без изменений. Django устраняет необходимость создавать в приложении собственные SQL-запросы благодаря встроенному ORM. Таким образом мы полностью избегаем проблем с правильными кавычками для значений SQL. В листинге 11.9 приводятся поля таблицы базы данных в декларативном классе Python, которые будут использоваться для представления строк таблицы после извлечения. Если наши требования к данным невозможно описать од-

¹ В репозитории к книге, к сожалению, отдельных файлов приложения Django нет. Кроме того, URL, представленный автором, в принципе ошибочный. Однако он здесь приведен, т. к. автор ссылается на него в дальнейшем повествовании. — *Прим. ред.*

ними типами полей, Django позволяет использовать с подобным классом расширенную логику валидации.

Листинг 11.9. Приложение Django, models.py

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Определения модели для приложения Django.

from django.db import models
from django.forms import ModelForm

class Payment(models.Model):
    debit = models.CharField(max_length=200)
    credit = models.CharField(max_length=200, verbose_name='To account')
    dollars = models.PositiveIntegerField()
    memo = models.CharField(max_length=200)

class PaymentForm(ModelForm):
    class Meta:
        model = Payment
        fields = ['credit', 'dollars', 'memo']
```

Нижнее объявление класса велит Django создать и изменить записи базы данных с помощью формы. Он спросит пользователя о трех предоставленных полях, оставив поле счета списания (debit) пустым, чтобы мы могли заполнить его своим именем пользователя.

Позже мы увидим, что этот класс может обрабатывать обе стороны взаимодействия пользователя с веб-приложением: он может отрисовывать форму как набор полей HTML, а затем выполнять парсинг данных HTTP POST, которые возвращаются после отправки формы, чтобы можно было создать или изменить строку базы данных Payment (Перевод).

Если мы будем использовать микрофреймворк вроде Flask, для выполнения подобных задач потребуется сторонняя библиотека. Например, есть популярный ORM SQLAlchemy, и многие программисты предпочитают не использовать Django, потому что им нравится функциональность и элегантность SQLAlchemy.

Однако поскольку SQLAlchemy не знает об HTML-формах, понадобится еще одна сторонняя библиотека, чтобы обрабатывать вторую половину задач, которые выполняет файл models.py при использовании Django.

Вместо того чтобы использовать декоратор в стиле Flask для привязки путей URL к функциям представлений Python, Django требует создать файл urls.py, как в листинге 11.10. В этом случае представления не зависят от позиции и помогают централи-

зывать управление пространством URL, но при этом у каждого представления будет меньше контекста при просмотре отдельно от остальных.

Листинг 11.10. Приложение Django: файл `urls.py`

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Паттерны URL для приложения Django.

from django.conf.urls import patterns, include, url
from django.contrib import admin
from django.contrib.auth.views import login

urlpatterns = patterns('',
    url(r'^admin/', include(admin.site.urls)),
    url(r'^accounts/login/$', login),
    url(r'^$', 'djbank.views.index_view', name='index'),
    url(r'^pay/$', 'djbank.views.pay_view', name='pay'),
    url(r'^logout/$', 'djbank.views.logout_view'),
)
```

Если URL содержит несколько разделов, Django принимает странное решение и использует регулярное выражение, чтобы сопоставить их, так что мы видим сложные для чтения паттерны. По личному опыту могу сказать, что при таком синтаксисе сложнее искать ошибки.

Кроме того, что путь к странице входа находится там, где модуль аутентификации Django ожидает его найти, эти паттерны предоставляют почти такое же пространство URL, что и предыдущие приложения Flask. Этот метод использует стандартную страницу входа Django, так что нам не приходится писать ее самим и надеяться, что мы не наделаем ошибок. Представления в листинге 11.11, которые наконец связывают приложение Django воедино, получаются одновременно проще и продуманнее, чем то, что мы написали при использовании Flask.

Листинг 11.11. Приложение Django: файл `views.py`

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Функция для каждого представления в приложении Django.

from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.contrib.auth import logout
from django.db.models import Q
from django.shortcuts import redirect, render
```

```

from django.views.decorators.http import require_http_methods, require_safe
from .models import Payment, PaymentForm

def make_payment_views(payments, username):
    for p in payments:
        yield {'dollars': p.dollars, 'memo': p.memo,
              'prep': 'to' if (p.debit == username) else 'from',
              'account': p.credit if (p.debit == username) else p.debit}

@require_http_methods(['GET'])
@login_required
def index_view(request):
    username = request.user.username
    payments = Payment.objects.filter(Q(credit=username) | Q(debit=username))
    payment_views = make_payment_views(payments, username)
    return render(request, 'index.html', {'payments': payment_views})

@require_http_methods(['GET', 'POST'])
@login_required
def pay_view(request):
    form = PaymentForm(request.POST or None)
    if form.is_valid():
        payment = form.save(commit=False)
        payment.debit = request.user.username
        payment.save()
        messages.add_message(request, messages.INFO, 'Payment successful.')
        return redirect('/')
    return render(request, 'pay.html', {'form': form})

@require_http_methods(['GET'])
def logout_view(request):
    logout(request)
    return redirect('/')

```

Где защита от межсайтового скриптинга? Хороший вопрос. Она была добавлена в файл `settings.py`, когда я попросил Django создать скелет этого приложения командой `manage.py startapp`.

Формы не будут работать, если мы не добавим `{% csrf_token %}` в шаблон формы, даже если мы не знаем, что защита CSRF существует. Если мы об этом забудем, Django напомним нам, выдав сообщение об ошибке в режиме разработки `runserver`. Это очень эффективный подход для неопытного веб-разработчика, который мало знаком с этими сложностями: Django по умолчанию защищает их от распространенных проблем с формами и полями, в отличие от микрофреймворков.

Этот код использует встроенные возможности Django практически для всего, так что нам не приходится самим продумывать вход и управление сеансами, в итоге

представления в этом приложении получаются гораздо проще, чем их аналоги, созданные с помощью Flask. Поскольку `urls.py` просто использует Django, мы даже не видим прописанную страницу входа. Страница выхода просто вызывает `logout()`, не заботясь о деталях. Используя атрибут `@login` для представлений, мы не волнуемся о том, выполнил пользователь вход или нет. Единственный вспомогательный метод, который напрямую соответствует аналогичному функционалу в приложении Flask, — декоратор `@require http_methods()`, который предоставляет ту же защиту от недопустимых или неподдерживаемых методов HTTP, что и декораторы Flask.

Работать с базой данных очень просто. Модуль `bank.py` с его синтаксисом SQL больше не нужен.

Django использует базу данных SQLite (один из параметров по умолчанию в `settings.py`) и готов открыть сеанс связи с базой данных в тот момент, когда код запросит класс модели из файла `models.py`.

Единственный минус — логику, которая должна входить в шаблон (формулировки и презентация переводов на главной странице), пришлось переместить в код Python, потому что через систему шаблонов Django эту логику выразить не так-то просто. Представление `index()` в Python, с другой стороны, вызывает генератор, который создает словарь с информацией о каждом переводе, превращая необработанный объект в нужные данные.

Некоторые программисты недовольны такой нефункциональной системой работы с шаблонами. Другие научились создавать в Django теги шаблонов, чтобы вызывать логику из глубины шаблона. Некоторые сочтут, что код, как в листинге 11.11, лучше в долгосрочной перспективе, потому что тесты для процедуры, вроде `make_payment_views()`, проще создавать, чем логику, вплетенную в шаблон.

Для того чтобы выполнить это приложение Django, возьмите исходный код для главы 11 по ссылке выше, установите Django 1.7 в Python 3 и выполните следующие три команды:

```
$ python manage.py syncdb
$ python manage.py loaddata start
$ python manage.py runserver
```

Перейдите по адресу <http://localhost:8000/> и посмотрите, как Django позволяет создать приложение, похожее на то, которое мы написали с помощью Flask ранее.

Выбор фреймворка для веб-сайта

Активное сообщество Python постоянно создает и развивает веб-фреймворки. Через несколько лет эта информация устареет, но мы все равно приведем в этой книге описание наиболее популярных фреймворков, чтобы вы получили общее представление о них.

◆ **Django.** Отличный вариант для новичков в веб-программировании. Он предлагает встроенные возможности, например защиту от CSRF, ORM и синтаксис

шаблонов. Это не только избавляет новичков от необходимости искать сторонние библиотеки, но и гарантирует, что все сторонние инструменты Django могут использовать одинаковый набор интерфейсов для взаимодействия с HTML и базой данных. Посетите страницу `/admin` после выполнения листинга 11.11, чтобы увидеть пример того, как администраторы могут напрямую взаимодействовать с базой данных, используя автоматически сгенерированные формы создания, изменения и удаления.

- ◆ **Tornado.** Этот веб-фреймворк, в отличие от остальных в данном списке, использует асинхронный механизм обратных вызовов из главы 9, чтобы обрабатывать десятки и сотни соединений с клиентами в одном потоке операционной системы. Кроме того, он не ограничен поддержкой WSGI, а поддерживает WebSocket напрямую, как описано в следующем разделе. Правда, многим библиотекам сложно взаимодействовать с его структурой обратных вызовов, так что программистам приходится искать асинхронные альтернативы стандартным ORM и коннекторам для базы данных.
- ◆ **Flask.** Самый популярный микрофреймворк, использующий надежные инструменты и поддерживающий широкий набор возможностей (если знать, как их найти и использовать). Часто применяется в паре с SQLAlchemy или нереляционной базой данных.
- ◆ **Bottle.** Альтернатива Flask, но требуется установить всего один файл, `bottle.py`, а не несколько отдельных пакетов. Особенно понравится разработчикам, которые еще не интегрировали инструмент установки `pip` в рабочий процесс. Предлагает хорошо продуманный язык шаблонов.
- ◆ **Pyramid.** Создан с учетом уроков, усвоенных сообществом в работе с Zope и Pylons. Это универсальный фреймворк для разработчиков, работающих в гибких пространствах URL, например созданных при разработке системы управления контентом. Пользователи могут создавать подпапки и дополнительные веб-страницы одним щелчком мыши. Он может поддерживать предопределенные структуры URL, как и предыдущие фреймворки, а также проход по объектам, т. е. фреймворк понимает последовательные компоненты URL, которые похожи на путь по папкам до файла в файловой системе. Активное сообщество Python постоянно создает и развивает веб-фреймворки. Через несколько лет эта информация устареет, но мы все равно приведем в этой книге описание наиболее популярных фреймворков, чтобы вы получили общее представление о них.

У вас может возникнуть искушение выбрать веб-фреймворк на основе его репутации. Например, исходя из приведенного здесь описания, а также рекомендаций на сайтах вроде Stack Overflow.

Однако я советую пообщаться с коллегами и знакомыми, которые уже полюбили какой-то фреймворк и могут помочь вам лично, и выбрать их любимый фреймворк. Помощь от людей, которые на своем опыте познали все недостатки и проблемы работы с тем или иным фреймворком, часто оказывается ценнее, чем возможности самого инструмента.

Веб-сокеты

Веб-сайты, которые используют JavaScript, часто разрешают пользователям редактировать контент в реальном времени. Когда кто-то публикует твит, Twitter хочет обновить страницу, которую мы просматриваем, чтобы при этом браузеру не приходилось каждую секунду проверять, появилось ли что-то новое. Лучшее решение проблемы с длинными опросами — протокол WebSocket (RFC 6455).

Раньше использовались методы Comet: клиент отправляет HTTP-запрос к пути, сервер зависает, оставляя сокет открытым, и ждет возможности ответить, пока не произойдет какое-то событие (например, публикация нового твита). Поскольку WSGI работает только на базе традиционного HTTP, для поддержки веб-сокетов недостаточно веб-фреймворков и серверов с возможностью WSGI вроде Gunicorn, Apache и nginx.

Одна из главных причин популярности отдельных серверов Tornado — WSGI не поддерживает веб-сокеты.

В отличие от HTTP, когда клиент отправляет один запрос и ждет, что сервер предоставит ответ, прежде чем отправить следующий запрос, сокет в режиме веб-сокета пропускает сообщения в оба направления в любое время, т. е. они не ждут друг друга.

Для того чтобы узнать больше, читайте документацию для модуля `tornado.websocket` с кодом Python и JavaScript, которые могут взаимодействовать друг с другом через пару симметричных обратных вызовов. Поищите идеи использования подобной системы для реализации динамических изменений на веб-страницах в источниках об асинхронном фронтенд-программировании для браузера.

Веб-скрейпинг

Обычно веб-программисты начинают свою карьеру со скрейпинга чужого сайта, а не с создания собственного. В конце концов, гораздо проще использовать уже имеющиеся материалы, чем где-то взять полный набор новых данных.

Главный совет о скрейпинге: избегайте его всеми силами.

Кроме базового скрейпинга есть и другие подходы к получению данных, менее дорогостоящие не только для программистов, но и для самого сайта. Например, Internet Movie Database (IMDB) позволяет загрузить статистику по фильмам на странице www.imdb.com/interfaces, чтобы главному сайту не приходилось создавать сотни тысяч дополнительных страниц, которые нам затем придется интерпретировать. Многие сайты, включая Google и Yahoo, предлагают API для ключевых сервисов, чтобы нам не пришлось получать в ответ необработанный HTML.

Если вы ищете данные в Google, но не находите готовых загрузок или API, помните несколько правил. Найдите страницу с условиями использования на нужном сайте. Поищите файл `/robots.txt`, где можно посмотреть, какие URL предназначены для

загрузки поисковыми движками, а каких лучше избегать. Это поможет вам не собирать много версий одной статьи с разными рекламами и не перегружать сайт.

Если вы будете следовать условиям использования и файлу robots.txt, это снизит вероятность того, что ваш IP-адрес добавят в черный список за генерацию чрезмерного трафика.

Для скрейпинга веб-сайта обычно требуется знать, как работает HTTP и как веб-браузеры используют его (см. главы 9–11).

- ◆ Методы GET и POST, а также как метод, путь и заголовки вместе образуют HTTP-запрос.
- ◆ Коды состояний в HTTP-ответе и структура ответа, включая различия между успешным выполнением, перенаправлением, временным сбоем и постоянным сбоем.
- ◆ Аутентификация с помощью форм и задание cookie, которые будут включаться в последующие запросы, чтобы их можно было считать допустимыми.
- ◆ Базовая аутентификация HTTP: как она требуется ответом сервера и доставляется в запросе клиента.
- ◆ Аутентификация на основе JavaScript: форма входа отправляет прямой запрос POST на веб-сервер, не вовлекая браузер в процесс отправки.
- ◆ Разница между запросом или действием, которое добавляет данные в конец URL и выполняет GET для этого расположения, и действием, которое выполняет POST для отправки данных из тела запроса на сервер напрямую (для защиты от атак CSRF).
- ◆ Разница между методом POST в URL, созданных для получения данных формы из браузера, и URL, которые предназначены для прямого взаимодействия с фронтенд-кодом JavaScript, а значит, с большей вероятностью ожидают и доставляют данные в JSON и другом удобном для программиста виде.

Скрейпинг сложного сайта может потребовать многочасовых экспериментов с инструментами разработчика в браузере, прежде чем вы поймете, что происходит. Потребуется три вкладки, и если щелкнуть на странице правой кнопкой мыши и выбрать **Inspect Element** (Исследовать), в Firefox и Google Chrome откроются все три. На вкладке **Elements** (Инспектор) отображается динамический контент, даже если JavaScript добавляет и удаляет элементы в процессе. Здесь мы видим, какие элементы находятся внутри других элементов. На вкладке **Network** (Сеть) можно нажать кнопку **Reload** (Обновите) и просмотреть все HTTP-запросы и ответы, в том числе инициированные кодом JavaScript, которые создали текущую страницу. На вкладке **Console** (Консоль) мы видим ошибки на странице, даже если как пользователи мы не получаем уведомления о них.

Существуют два варианта автоматизировать процесс.

Первый вариант: забросить большую сеть, потому что мы хотим загрузить много информации. Для этого нужно войти на сайт, чтобы получить нужные cookie, а затем выполнять операции GET, которые запустят другие операции GET, проходя по ссылкам на загружаемых страницах. По такому принципу работают "пауки", ис-

пользуемые поисковыми движками для просмотра всех страниц на каждом сайте. Пауками их называют потому, что они перемещаются по Всемирной паутине.

Второй вариант пригодится, если мы будем работать с одной или двумя страницами, а не всем сайтом или его большими разделами. Это ситуации, когда нам нужны данные только с некоторых страниц. Например, мы хотим вывести температуру с погодного сайта или пытаемся автоматизировать задачу, для которой в противном случае потребовался бы браузер, например провести платеж или просмотреть вчерашние транзакции по кредитной карте, чтобы проверить их на мошенничество. Для этого часто требуется больше внимание к деталям.

Поскольку банк использует JavaScript на странице, чтобы запретить автоматические попытки получить незаконный доступ к счетам, для этого часто требуется полноценный браузер, а не просто код Python.

Прежде чем запустить автоматизированную программу на сайте, обязательно прочитайте условия использования и файлы `robot.txt`. Вас могут остановить, если программа начинает потреблять гораздо больше ресурсов, чем при просмотре человеком. Я не буду даже упоминать OAuth и другие приемы, которые затрудняют использование программ, выполняющих задачи, обычно осуществляемые через браузер. Если используются незнакомые тактики или протоколы, привлекайте сторонние библиотеки и отслеживайте исходящие заголовки, чтобы они в точности совпадали с тем, что отправляется при успешной отправке формы или просмотре сайта в браузере. В зависимости от правил на сайте, даже поле `user-agent` может быть важным.

Получение страниц

Существуют три способа извлекать страницы из Интернета и изучать их содержимое в программе Python.

- ◆ Используйте библиотеку Python для прямых запросов GET и POST. Запросите объект `Session` из библиотеки `requests`, чтобы он отслеживал `cookie` и объединял соединения в пул. Если вы ищете вариант из стандартной библиотеки, попробуйте `urllib.request` для несложных ситуаций.
- ◆ Раньше существовали компромиссные варианты, которые действовали как рудиментарный браузер, чтобы находить компоненты `<form>` и помогать вам в создании HTTP-запроса, опираясь на те же правила, которые применял бы браузер для возврата входных данных формы с сервера. Самым известным инструментом был `mechanize`, но сейчас он, кажется, не используется. Может быть, потому что сайты стали настолько сложными, что без JavaScript уже невозможно обойтись.
- ◆ Можно использовать сам браузер. В примерах ниже мы применяем библиотеку `Selenium Webdriver` и `Firefox`, хотя существуют инструменты, которые работают по принципу браузеров, но не открывают полное окно. Обычно для этого создается экземпляр `WebKit`, не привязанный к реальному окну. `PhantomJS` популяризовал этот подход в мире JavaScript, а для Python существует `Ghost.py`.

Алгоритм будет очень простым, если мы уже знаем, какие URL хотим посетить. Возьмите список URL, отправьте каждому по HTTP-запросу, а затем сохраните и проанализируйте результаты. Если список URL неизвестен заранее и приходится узнавать их по ходу, все будет гораздо сложнее. Придется отслеживать уже посещенные URL, чтобы не переходить по ним по несколько раз.

В листинге 11.2 приводится простой скрейпер с узкой областью действия. Он входит в платежное приложение и сообщает о поступлениях на счет. Откройте копию приложения в одном окне, прежде чем запустить программу.

\$ python app_improved.py

Листинг 11.12. Подсчет поступлений в платежном приложении

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Скрейпинг вручную с переходом к определенной странице и извлечением данных.

import argparse, bs4, lxml.html, requests
from selenium import webdriver from urllib.parse import urljoin

ROW = '{:>12} {}'

def download_page_with_requests(base):
    session = requests.Session()
    response = session.post(urljoin(base, '/login'),
                            {'username': 'john', 'password': '12345678'})
    assert response.url == urljoin(base, '/')
    return response.text

def download_page_with_selenium(base):
    browser = webdriver.Firefox()
    browser.get(base)
    assert browser.current_url == urljoin(base, '/login')
    css = browser.find_element_by_css_selector
    css('input[name="username"]').send_keys('john')
    css('input[name="password"]').send_keys('12345678')
    css('input[name="password"]').submit()
    assert browser.current_url == urljoin(base, '/')
    return browser.page_source

def scrape_with_soup(text):
    soup = bs4.BeautifulSoup(text)
    total = 0
```

```

for li in soup.find_all('li', 'to'):
    dollars = int(li.get_text().split()[0].lstrip('$'))
    memo = li.find('i').get_text() total += dollars
    print(ROW.format(dollars, memo))
print(ROW.format('-' * 8, '-' * 30))
print(ROW.format(total, 'Total payments made'))

def scrape_with_lxml(text):
    root = lxml.html.document_fromstring(text) total = 0
    for li in root.cssselect('li.to'):
        dollars = int(li.text_content().split()[0].lstrip('$'))
        memo = li.cssselect('i')[0].text_content()
        total += dollars
        print(ROW.format(dollars, memo))
    print(ROW.format('-' * 8, '-' * 30))
    print(ROW.format(total, 'Total payments made'))

def main():
    parser = argparse.ArgumentParser(description='Scrape our payments site.')
    parser.add_argument('url', help='the URL at which to begin')
    parser.add_argument('-l', action='store_true', help='scrape using lxml')
    parser.add_argument('-s', action='store_true', help='get with selenium')
    args = parser.parse_args()
    if args.s:
        text = download_page_with_selenium(args.url)
    else:
        text = download_page_with_requests(args.url)
    if args.l:
        scrape_with_lxml(text)
    else:
        scrape_with_soup(text)

if __name__ == ' main ':
    main()

```

Мы можем запустить `msscrape.py` в другом окне терминала, когда это приложение Flask начнет выполняться на порту 5000. Загрузите и установите стороннюю библиотеку Beautiful Soup и библиотеку requests.

```
$ pip install beautifulsoup4
```

```
$ pip install requests
```

```
$ python msscrape.py http://127.0.0.1:5000/
```

```
125 Registration for PyCon
200 Payment for writing that code
```

```
-----
325 Total payments made
```

Когда файл `mscrape.py` выполняется в режиме по умолчанию, сначала он с помощью библиотеки `request` входит на сайт через форму входа. В результате объект `Session` получит `cookie`, чтобы извлечь главную страницу. Затем скрипт выполняет парсинг страницы, получает элементы списка и несколько раз вызывает `print()`, чтобы вывести исходящие переводы.

Передав параметр `-s` файлу `mscrape.py`, мы увидим кое-что поинтереснее: запустится полноценный браузер `Firefox`, если он установлен, и именно с его помощью мы будем просматривать сайт. Этот режим работает, только если установлен пакет `Selenium`.

```
$ pip install selenium
```

```
$ python mscrape.py -s http://127.0.0.1:5000/
```

```
125 Registration for PyCon
200 Payment for writing that code
```

```
-----
325 Total payments made
```

Когда скрипт выведет выходные данные, нажмите комбинацию клавиш `<Ctrl>+<W>`, чтобы закрыть `Firefox`. Мы можем, конечно, написать скрипты `Selenium`, чтобы браузер закрывался автоматически, но я предпочитаю оставлять его открытым во время разработки и отладки, чтобы видеть, что пошло не так в браузере, если программа наткнется на ошибку.

Нужно понимать разницу между этими двумя методами. Для того чтобы написать код на основе `requests`, сначала нужно зайти на сайт, прочитать форму входа и скопировать информацию в данные, которые метод `post()` использует для входа. У кода нет способов узнавать об изменениях формы входа в будущем. Даже если жестко закодированное имя пользователя и пароль потеряют актуальность, он все равно будет их использовать.

В таком виде код с использованием `requests` совершенно не похож на функционал браузера. Он не заходит на страницу входа и не смотрит форму. Он предполагает, что страница входа существует, а затем обходит ее, чтобы методом `POST` отправить форму, которая появляется в результате ее заполнения. Разумеется, если форма входа получает секретный токен, чтобы помешать подбору паролей, эта стратегия не работает. В этом случае первый `GET` на странице `/login` должен будет получить секретный токен, который нужно будет приложить к имени пользователя и паролю, чтобы создать действительный `POST`.

В `mscrape.py` код с `Selenium` использует противоположный подход. Он действует так, будто видит форму, выбирает ее элементы и начинает печатать как настоящий пользователь в браузере. Затем он "нажимает" на кнопку отправки формы. `Selenium` делает в `Firefox` то, что делали бы мы, т. е. если его селекторы `CSS` правильно опре-

деляют поля формы, код успешно входит на сайт, даже если используются секретные токены или специальный код JavaScript для подписания или автоматизации.

Конечно, Selenium работает гораздо медленнее, чем request, особенно когда нам сначала приходится ждать загрузки Firefox. Зато он быстро выполняет действия, для которых потребовались бы часы проб и ошибок в Python. Можно ли реализовать гибридный подход к скрейпингу: использовать Selenium для входа и получения нужных cookie, а затем уведомить requests о них, чтобы массово извлечь страницы и не ждать браузер?

Страницы для веб-скрейпинга

Когда сайт отправляет данные в CSV, JSON или другом распознаваемом формате данных, мы можем использовать подходящий модуль в стандартной библиотеке или стороннюю библиотеку для парсинга. А если нужная информация скрыта в HTML, видимом пользователю?

Конечно, можно нажать комбинацию клавиш <Ctrl>+<U> в Google Chrome или Firefox, чтобы прочитать весь HTML, но это будет довольно утомительно. Гораздо удобнее щелкнуть правой кнопкой мыши, выбрать команду **Inspect Element** и просмотреть свертываемое дерево элементов, если, конечно, HTML написан корректно и ошибка в разметке не скрывает нужные данные от браузера. Как мы видели, недостаток инспектора элементов в том, что к моменту, когда мы увидим документ, программы JavaScript на этой странице могут уже до неузнаваемости изменить документ.

Для просмотра таких страниц есть два варианта. Первый вариант: отключить JavaScript в браузере и перезагрузить страницу, на которой мы находимся. Теперь она будет отображаться в инспекторе элементов без изменений: мы увидим в точности то, что "увидит" код Python при загрузке этой страницы. Второй вариант: использовать инструмент вроде пакета tidy от W3C на Debian и Ubuntu. Эти возможности присутствуют в обеих библиотеках для парсинга, которые использовались в листинге 11.12. Когда объект soup будет создан, мы сможем прибегнуть к следующему приему, чтобы отобразить его элементы на экране:

```
print(soup.prettify())
```

Для отображения дерева документов lxml требуется чуть больше усилий:

```
from lxml import etree
print(etree.tostring(root, pretty_print=True).decode('ascii'))
```

Читать результат будет гораздо проще, чем необработанный HTML, если сайт не размещает элементы в отдельных строках с правильными отступами для четкой структуры документа, хотя это будет неудобно и увеличит трафик для сайта.

Изучение HTML будет состоять из трех действий.

1. Мы используем выбранную библиотеку для парсинга HTML. Поскольку в Интернете очень много HTML-кода с ошибками и проблемами разметки, библио-

теке придется сложно. Дизайнеры обычно не знают об этом, потому что браузеры пытаются восстановить и интерпретировать разметку. В конце концов, создатели браузеров не хотят, чтобы только их браузер возвращал ошибку на популярном сайте, пока другие браузеры отображают его корректно. Обе библиотеки в листинге 11.12 хорошо подходят для парсинга HTML.

2. Мы используем селекторы — текстовые паттерны, которые автоматически находят нужные элементы, чтобы подробнее изучить документ. Мы можем сделать это самостоятельно, терпеливо проходя по дочерним элементам каждого компонента и просматривая нужные теги и атрибуты, но с селекторами это гораздо быстрее. Обычно с ними код Python выглядит проще и понятнее.
3. Мы запрашиваем значения текста и атрибута от каждого объекта элемента.

В листинге 11.12 этот трехэтапный процесс повторяется дважды с двумя разными библиотеками.

Библиотека Beautiful Soup используется функцией `scrape_with_soup()` и является универсальным ресурсом для программистов по всему миру. У нее необычный API, потому что это была первая библиотека Python, которая упростила парсинг документов, но со своей задачей она справляется.

У всех объектов `soup` есть метод `find_all()`, который ищет подчиненные элементы, соответствующие заданному имени тега и (необязательно) имени класса HTML. При этом объект может представлять весь документ или один подчиненный компонент. Наконец, когда мы найдем нужный элемент и будем готовы прочитать его содержимое, мы используем метод `get_text()`. Код может выполнять скрейпинг на этом простом сайте, используя только эти два способа, и даже для сложных сайтов часто бывает достаточно десятка действий или меньше.

Документацию по библиотеке Beautiful Soup можно найти по адресу: www.crummy.com/software/BeautifulSoup/.

Функция `scrape_with_lxml()` использует библиотеку `lxml`, созданную поверх `libxml2` и `libxslt`. Если вы работаете в более старой версии операционной системы, в которой не установлены компиляторы, или если вы не установили пакет `python-dev` или `python-devel`, операционная система не сможет поддерживать скомпилированные пакеты Python. Эта библиотека уже скомпилирована в системе Python в операционных системах на основе Debian и обычно называется `python-lxml`.

Даже на Mac OS X и Windows современные дистрибутивы Python вроде Anaconda поддерживают `lxml`: <http://continuum.io/downloads>.

В листинге 11.12 эта библиотека используется для парсинга HTML.

```
$ pip install lxml
```

```
$ python msccraper.py -l http://127.0.0.1:5000/
```

```
125 Registration for PyCon
```

```
200 Payment for writing that code
```

```
-----
```

```
325 Total payments made
```

Основные действия такие же, как при использовании библиотеки Beautiful Soup. Мы начали с верхней части документа, использовали метод поиска — в нашем случае `cssselect()`, чтобы найти нужные элементы, а затем выполнили дополнительные процедуры поиска, чтобы получить дочерние элементы, и, наконец, запросили содержащийся в них текст, чтобы проанализировать и отобразить его.

Библиотека `lxml` не только работает быстрее, чем Beautiful Soup, но и дает больше возможностей для выбора элементов.

- ◆ Она использует `cssselect()` для поддержки паттернов CSS. Это особенно важно при поиске элементов по классу, потому что элемент считается принадлежащим классу `x` независимо от того, как указан его атрибут класса: `class="x"`, `class="x y"` или `class="w y"`.
- ◆ Она поддерживает выражения XPath и метод `xpath()`, который пользуется популярностью у фанатов XML. Для того чтобы найти все абзацы, например, можно указать `'./p'`. Одно из главных преимуществ выражения XPath: мы можем в конце написать `'.../text()'` и просто получить текст каждого элемента, а не объекты Python, из которых придется запрашивать текст.
- ◆ Методы `find()` и `findall()` нативно поддерживают операции XPath.

Инструменту скрейпинга пришлось поработать чуть усерднее в обоих примерах, потому что в поле описания перевода есть тег `<i>`, а сумма в начале каждой строки не размещается в тегах. Это распространенная проблема: какая-то информация существует отдельно, в собственных тегах, а какая-то — скрывается в другом контенте, и приходится использовать методы Python вроде `split()` и `strip()`, чтобы отделить ее от окружающего текста.

Рекурсивный веб-скрейпинг

В репозитории с кодом к этой книге есть небольшой статический сайт², доступ к страницам которого затруднен для инструмента веб-скрейпинга:

`https://bpbonline.com/ Programming in Python: The Basics`

Если вы загрузили весь репозиторий, вы можете обслуживать его локально с помощью встроенного веб-сервера Python.

\$ `python -m http.server`

Serving HTTP on 0.0.0.0 port 8000 ...

Если изучить код страницы и использовать инструмент отладки в браузере, можно увидеть, что не все ссылки на главной странице по адресу `http://127.0.0.1:8000/` предоставляются одновременно. Только две из них (`page1` и `page2`) являются настоящими якорями с атрибутами `href=""` в HTML-коде.

² К сожалению, в репозитории к книге никакого сайта нет. Кроме того, ссылка, предоставленная автором книги, синтаксически неверная. — *Прим. ред.*

Следующие две страницы скрыты за формой с кнопкой отправки под названием **Search** (Поиск), и мы не сможем получить к ним доступ, не нажав на кнопку. Небольшой фрагмент динамического кода JavaScript приводит к тому, что в нижней части экрана отображаются две последние ссылки (page5 и page6). Это имитирует поведение сайтов, которые сразу показывают структуру страницы, но совершают еще один круг до сервера и обратно, прежде чем отобразить нужные данные.

Возможно, на этом этапе вам потребуется движок веб-скрейпинга, который поможет вам выполнить полноценный рекурсивный поиск всех URL на сайте или хотя бы их части. Так же как веб-фреймворки учитывают типичные паттерны в веб-приложениях, например необходимость вернуть ошибку 404, если сайт не существует, они могут следить за тем, какие страницы мы уже посетили, а какие — еще нет.

Scrapy (<http://scrapy.org/>) — очень популярный инструмент веб-скрейпинга. Советую изучить его документацию.

В листинге 11.13 приводится упрощенный пример реального инструмента скрейпинга. Он использует библиотеку lxml, так что по возможности установите ее, следуя инструкциям в предыдущем разделе.

Листинг 11.13. Инструмент рекурсивного веб-скрейпинга, использующий операции GET

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Инструмент рекурсивного скрейпинга с библиотекой request.

import argparse, requests
from urllib.parse import urljoin, urlsplit
from lxml import etree

def GET(url):
    response = requests.get(url)
    if response.headers.get('Content-Type', '').split(';')[0] != 'text/html':
        return
    text = response.text
    try:
        html = etree.HTML(text)
    except Exception as e:
        print('    {}: {}'.format(e.__class__.__name__, e))
        return
    links = html.findall('//*[a[@href]]')
    for link in links:
        yield GET, urljoin(url, link.attrib['href'])

def scrape(start, url_filter):
    further_work = {start}
```

```

already_seen = {start}
while further_work:
    call_tuple = further_work.pop()
    function, url, *etc = call_tuple
    print(function.__name__, url, *etc)
    for call_tuple in function(url, *etc):
        if call_tuple in already_seen:
            continue
        already_seen.add(call_tuple)
        function, url, *etc = call_tuple
        if not url_filter(url):
            continue
        further_work.add(call_tuple)

def main(GET):
    parser = argparse.ArgumentParser(description='Scrape a simple site.')
    parser.add_argument('url', help='the URL at which to begin')
    start_url = parser.parse_args().url
    starting_netloc = urlsplit(start_url).netloc
    url_filter = (lambda url: urlsplit(url).netloc == starting_netloc)
    scrape((GET, start_url), url_filter)

if __name__ == '__main__':
    main(GET)

```

В листинге 11.13 есть только два действующих элемента (не считая задание и чтение аргументов командной строки).

Самый простой из них — функция GET(), которая пытается загрузить содержимое по URL и проанализировать его, если это HTML. Если это действие выполняется успешно, извлекаются атрибуты href="" якорных тегов, чтобы понять, на какие страницы ссылается текущая страница. Поскольку эти ссылки могут быть относительными URL, для каждой из них используется urljoin(), чтобы вставить недостающие компоненты.

Функция GET() возвращает кортеж для каждого URL в тексте страницы, указывая, что инструмент скрейпинга должен вызвать себя по обнаруженному URL, если он еще этого не делал. Инструменту остается отслеживать уже вызванные комбинации функций и URL, чтобы не переходить по одному и тому же адресу несколько раз. Он хранит коллекцию просмотренных URL и коллекцию непросмотренных и повторяется, пока последняя не окажется пустой.

Инструмент скрейпинга можно использовать для крупных сайтов, таких как [httpbin](http://httpbin.org/).

\$ python rscrape1.py http://httpbin.org/

В листинге 11.12 эта библиотека используется для парсинга HTML.

```
$ python rscrape1.py http://127.0.0.1:8000/
```

```
GET http://127.0.0.1:8000/
```

```
GET http://127.0.0.1:8000/page1.html
```

```
GET http://127.0.0.1:8000/page2.html
```

Если инструмент должен изучать сайт дальше, ему потребуются два элемента.

Для начала откройте HTML в настоящем браузере, чтобы код JavaScript выполнялся и оставшаяся часть страницы загрузилась.

Затем в дополнение к GET() потребуется вторая операция, которая "нажимает" кнопку поиска, чтобы вы смогли увидеть, что появится.

Этот тип операции никогда не следует включать в инструмент автоматического скрейпинга, созданный для извлечения общего содержимого с общедоступного сайта, потому что, как вы уже знаете, отправка форм предназначена для действий пользователя, особенно если она сопровождается операцией POST. (В этом случае форма выполняет запрос GET, более безопасный.) В этой ситуации мы изучили сайт и решили, что нажимать на кнопку безопасно.

Поскольку инструмент может вызывать разные функции, в листинге 11.14 можно использовать движок из предыдущего примера.

Листинг 11.14. Selenium для рекурсивного скрейпинга

```
#!/usr/bin/env python3
# Programming in Python: The Basics
# Инструмент рекурсивного скрейпинга с Selenium Webdriver.

from urllib.parse import urljoin
from rscrape1 import main
from selenium import webdriver

class WebdriverVisitor:
    def __init__(self):
        self.browser = webdriver.Firefox()

    def GET(self, url):
        self.browser.get(url)
        yield from self.parse()
        if self.browser.find_elements_by_xpath('//*[form]'):
            yield self.submit_form, url

    def parse(self):
        # (Could also parse page.source with lxml yourself, as in scraper1.py)
```

```

url = self.browser.current_url
links = self.browser.find_elements_by_xpath('//*[a[@href]]')
for link in links:
    yield self.GET, urljoin(url, link.get_attribute('href'))

def submit_form(self, url):
    self.browser.get(url)
    self.browser.find_element_by_xpath('//*[form]').submit()
    yield from self.parse()

if __name__ == '__main__':
    main(WebdriverVisitor().GET)

```

Не используйте функцию `Firefox()` всякий раз, как нужно извлечь URL, потому что для создания экземпляров Selenium требуется много ресурсов (им ведь приходится запускать копию Firefox). Вместо простой функции процедура `GET()` написана как метод, чтобы можно было сохранять свойство браузера между вызовами `GET()` и оно было доступно для вызова `submit_form()`.

Этот листинг значительно отличается от предыдущего методом `submit_form()`. Когда метод `GET()` встречает форму поиска на странице, он возвращает программе кортеж. Он возвращает кортеж для каждой ссылки на странице, а также кортеж для загрузки страницы и нажатия кнопки поиска. Поэтому данный код копает глубже, чем инструмент скрейпинга в предыдущем примере.

\$ python rscrape2.py http://127.0.0.1:8000/ GET http://127.0.0.1:8000/

```

GET http://127.0.0.1:8000/page1.html
GET http://127.0.0.1:8000/page2.html
submit_form http://127.0.0.1:8000/
GET http://127.0.0.1:8000/page5.html
GET http://127.0.0.1:8000/page6.html
GET http://127.0.0.1:8000/page4.html
GET http://127.0.0.1:8000/page3.html

```

Несмотря на то что некоторые ссылки загружаются динамически через JavaScript, а другие можно посетить только через отправку формы методом POST, инструмент скрейпинга находит каждую страницу на сайте. Таким образом вы сможете автоматизировать взаимодействие с любым сайтом с помощью Python.

Резюме

HTTP создан для работы со Всемирной паутиной — коллекцией документов, связанных гиперссылками, каждая из которых содержит URL другой страницы или фрагмента страницы. Для того чтобы перейти на них, достаточно щелкнуть по та-

кой ссылке. Стандартная библиотека Python содержит функции для чтения и создания URL, а также преобразования относительных URL в абсолютные путем восстановления недостающих компонентов.

В большинстве веб-приложений постоянное хранилище данных, например база данных, привязаны к коду, который отвечает на входящие HTTP-запросы и создает HTML-страницы в ответ. База данных должна корректно расставлять кавычки, чтобы защититься от ненадежной информации. Для этого можно использовать DB-API 2.0 и любой ORM в Python.

Веб-фреймворки бывают базовыми и полноценными. Мы можем выбрать свой язык шаблонов и ORM (или другой уровень постоянного хранения), если используем базовый фреймворк. Полнофункциональный фреймворк предоставляет свои реализации этих инструментов. В любом случае существует способ подключить URL к своему коду, который поддерживает статические URL и URL с переменными компонентами пути, например `/person/123/`. Кроме того, они предоставляют простой способ рендеринга и возвращения шаблонов, а также перенаправления и обработки ошибок HTTP.

Всегда существует риск того, что злоумышленник воспользуется сложной системой взаимодействия компонентов в Интернете, чтобы навредить самому сайту или другим сайтам через него. Например, с помощью межсайтового скриптинга или подделки межсайтовых запросов. Прежде чем писать код, который принимает данные через маршрут URL, строку запроса URL, метод POST или файлы, нужно продумывать защиту от этих рисков. Взвесьте все за и против, прежде чем выбирать между полноценным решением вроде Django, которое ограничивает нас в выборе инструментов, но предлагает хорошие параметры по умолчанию (например, включенная защита от CSRF в формах), и более простыми решениями вроде Flask или Bottle, которые дают больше гибкости при выборе компонентов, но подразумевают более глубокое их понимание. При разработке приложения с помощью Flask вы можете случайно остаться без защиты CSRF, если не будете знать, что она вам нужна.

Tornado славится своей асинхронной архитектурой и позволяет обслуживать несколько клиентов через один управляющий поток на уровне операционной системы. С появлением `asyncio` в Python 3 такие решения, как Tornado, развивают стандартный набор паттернов, похожий на то, что предлагает WSGI для веб-фреймворков на базе потоков.

Веб-скрейпинг требует хорошего понимания принципов работы и устройства сайтов, чтобы скрипт мог имитировать поведение пользователя, например вход или заполнение и отправку форм. В Python существует несколько методов запроса и обработки страниц. Сейчас чаще всего используются `requests` или `Selenium` для извлечения и `Beautiful Soup` или `lxml` для парсинга.

Мы закончили рассмотрение HTTP и Всемирной паутины. Далее мы поговорим об электронной почте и формате сообщений и рассмотрим менее известные протоколы, поддерживаемые стандартной библиотекой Python.

Составление и парсинг сообщений электронной почты

Это первая из четырех глав, посвященных электронной почте.

- ◆ В данной главе мы не будем обсуждать сетевое взаимодействие, а поговорим о том, как создаются электронные письма, и обратим особое внимание на включение в них мультимедиа и использование символов из других языков.
- ◆ В *главе 13* вы познакомитесь с протоколом SMTP, который используется для передачи электронных писем с вашего компьютера на сервер, откуда они будут прочитаны нужным получателем.
- ◆ В *главе 14* мы поговорим об устаревшем и неэффективном протоколе POP, с помощью которого пользователь может загружать и просматривать новые письма в своем почтовом ящике на сервере электронной почты.
- ◆ В *главе 15* мы рассмотрим протокол IMAP — продвинутую версию SMTP, которая используется на вашем почтовом сервере. IMAP позволяет не только извлекать и просматривать письма, но также отмечать их прочитанными, а затем сохранять в разных каталогах на сервере.

Таким образом мы рассмотрим весь жизненный цикл электронных писем. *Электронное письмо* состоит из текста, мультимедиа и метаданных, таких как отправитель и получатель. SMTP передает письмо от адресата на целевой сервер. Наконец, почтовый клиент получателя (например, Mozilla Thunderbird или Microsoft Outlook) с помощью POP или IMAP загружает копию письма на устройство. Правда, последний шаг используется все реже, потому что многие люди читают электронную почту прямо в браузере, в HTML, так что письма даже не покидают почтовый сервер.

Содержание главы

- ◆ Форматирование электронного письма.
- ◆ Составление электронного письма.
- ◆ HTML и мультимедиа.
- ◆ Создание контента.

- ◆ Парсинг электронного письма.
- ◆ Использование MIME.
- ◆ Кодирование заголовков.
- ◆ Парсинг дат.
- ◆ Резюме.

Цель

Раньше самым популярным сервисом был Hotmail, а теперь — Gmail. Как бы письмо ни было создано и представлено позже (через SMTP, POP или IMAP), оно будет следовать одинаковым правилам, которые мы рассмотрим в этой главе.

Форматирование электронного письма

Знаменитый RFC 822, созданный в 1982 г., служил определением электронной почты почти 20 лет, пока окончательно не устарел. Ему на смену в 2001 г. пришел RFC 2822, а затем и RFC 5322 в 2008 г. Опирайтесь на эти правила при написании особенно важного кода для работы с электронными письмами. В данном разделе мы рассмотрим только несколько аспектов форматирования.

- ◆ В сообщениях электронной почты используется **открытый текст в кодировке ASCII** (коды символов с 1 по 127).
- ◆ Символы возврата каретки и перевода строки (**CR-LF**), использовавшиеся еще в телетайпах, остаются стандартной последовательностью для обозначения конца строки в современных интернет-протоколах.
- ◆ Электронное письмо состоит из заголовков, пустой строки и тела.
- ◆ Заголовки не чувствительны к регистру. Они состоят из имени, двоеточия и значения, которое может занимать несколько строк, с отступом в виде пробела.
- ◆ Поскольку обычный текст не поддерживает символы Unicode или двоичные данные, другие стандарты, которые мы рассмотрим позже в этой главе, предоставляют кодировки, которые позволяют превращать более сложные данные в простой текст ASCII для передачи и хранения.

В листинге 12.1 приводится настоящее письмо, которое пришло на мой адрес.

Листинг 12.1. Настоящее электронное письмо

```
X-From-Line: rms@gnu.org Fri Dec 3 04:00:59 1999
Return-Path: <rms@gnu.org>
Delivered-To: john@yahoo.com
Received: from Esther.edu (pele.santafe.edu [192.12.12.119])
  by europa.gtri.gatech.edu (Postfix) with ESMTP id 6C4774809
```

for <john@yahoo.com>; Fri, 8 nov 2019 04:00:58 -0500 (EST)
 Received: from Esther.edu (Esther [192.12.12.49])
 by Esther.edu (8.9.1/8.9.1) with ESMTP id CAA27250
 for <john@yahoo.com>; Fri, 8 nov 2019 02:00:57 -0700 (MST)
 Received: (from rms@localhost)
 by Esther.edu (8.9.1b+Sun/8.9.1) id CAA29939;
 Fri, 8 nov 2019 02:00:56 -0700 (MST)
 Date: Fri, 8 nov 2019 02:00:56 -0700 (MST)
 Message-Id: <201911080900.CAA29939@Esther.edu>
 X-Authentication-Warning: Esther.edu: rms set sender to rms@gnu.org
 using -f
 From: stanley david <rms@gnu.org>
 To: john@yahoo.com
 In-reply-to: <m3k8my7x1k.fsf@europa.gtri.gatech.edu> (message from John
 chris rodrigues on 07 nov 2019 00:04:55 -0500)
 Subject: Re: Please proofread this license
 Reply-To: rms@gnu.org
 References: <201911070547.WAA21685@aztec.santafe.edu>
 <m3k8my7x1k.fsf@europa.gtri.gatech.edu>
 Xref: 38-74.clients.speedfactory.net scrapbook:11
 Lines: 1
 Thanks.

Хотя в теле этого письма передана только одна строка текста, мы видим очень много дополнительных данных.

Хотя все заголовки, начиная со строки From (От кого), скорее всего, присутствовали при написании письма, многие заголовки над ними были добавлены по ходу передачи. Каждый клиент и сервер, который обрабатывает письмо, может добавлять свои заголовки. Это значит, что в процессе передачи по сети письмо накапливает собственную историю, которую можно прочесть, начиная с последнего заголовка и двигаясь вверх. В этом примере письмо поступило от компьютера aztec из Santa Fe (Санта-Фе), к которому отправитель был подключен напрямую через внутренний интерфейс localhost. Компьютер aztec переслал письмо pele, который, по-видимому, отвечал за обработку писем в отделе или в целом кампусе, через SMTP. Наконец, pele установил SMTP-соединение с моим компьютером europa в Технологическом институте Джорджии, который сохранил письмо на диск, чтобы я прочел его позже.

Давайте рассмотрим несколько заголовков (полный список см. в документации).

- ◆ Имя отправителя в поле From. Последующие заголовки похожи на те, которые идут перед ними. В угловых скобках принимается имя человека и его адрес электронной почты.

- ◆ Если отправитель, указанный в заголовке From, не является одновременно получателем, в поле To (Кому) указан получатель.
Получателей может быть несколько.
- ◆ В поле Cc указываются получатели, которые получают копию письма, но явно оно им не адресовано.
- ◆ В поле Bcc указываются получатели скрытой копии, которых не видят остальные получатели. Почтовый клиент удаляет Bcc перед отправкой письма.
- ◆ В поле Subject указывается тема письма.
- ◆ Дата указывается почтовым клиентом отправителя, и принимающий почтовый сервер не будет ее перезаписывать. Если отправитель не указал дату, она может быть добавлена после получения письма.
- ◆ Поле Message-Id содержит уникальный идентификатор письма.
- ◆ Идентификаторы Message-Id предыдущих писем, на которые отвечает это письмо, указаны в поле In-Reply-To (В ответ на). Это удобно, если мы просматриваем целую цепочку писем.
- ◆ Каждый раз, как письмо совершает очередной прыжок по маршруту через SMTP, оно добавляется в список полученных. Почтовый сервер просматривает этот путь, чтобы убедиться, что письмо отправлено верно.

В таком простом примере ограничения, связанные с ASCII, отражаются как на заголовках, так и на тексте. Мы рассмотрим стандарты, которые регулируют использование символов других языков в заголовке, а также стандарты, которые позволяют включать символы других языков и двоичные данные в теле.

Составление электронного письма

Класс `EmailMessage`, который мы будем использовать в каждой программе в этой главе, является главным интерфейсом Python для создания писем. Я хотел бы поблагодарить Дэвида Мюррея (David Murray), специалиста по Python-модулю `email`, за помощь и советы при написании скриптов для этой главы. В листинге 12.2 мы видим простейший пример.

Листинг 12.2. Простейшее текстовое сообщение электронной почты

```
#!/usr/bin/env python3
# Programming in Python: The Basics

import email.message, email.policy, email.utils, sys

text = """Hello,
This is a basic message from Chapter 12.
- john"""
```

```
def main():
    message = email.message.EmailMessage(email.policy.SMTP)
    message['To'] = 'recipient@example.com'
    message['From'] = 'Test Sender <sender@example.com>'
    message['Subject'] = 'Test Message, Chapter 12'
    message['Date'] = email.utils.formatdate(localtime=True)
    message['Message-ID'] = email.utils.make_msgid()
    message.set_content(text)
    sys.stdout.buffer.write(message.as_bytes())

if __name__ == '__main__':
    main()
```

Код в этой главе работает только с Python 3.4 и выше, потому что в предыдущих версиях не было класса `EmailMessage`.

Мы можем еще упростить электронные письма, исключив указанные выше заголовки, но это считается стандартным минимумом в современном Интернете.

Благодаря `EmailMessage` код очень похож на текст письма. Мы можем задавать заголовки и предоставлять контент в любой последовательности, но обычно сначала задаются заголовки, а затем тело, чтобы в коде письмо выглядело так же, как в почтовом клиенте.

Стоит отметить, что я здесь включил два обязательных заголовка, но их значения не задаются автоматически. Я использую метод `formatdate()`, который уже входит в стандартный набор инструментов Python для работы с электронной почтой, чтобы предоставить дату в определенном формате, как того требуют стандарты. Идентификатор `Message-Id` представляет собой случайную строку, которая в идеале должна быть уникальной среди всех подобных идентификаторов в прошлом и будущем.

Полный скрипт выводит электронное письмо в стандартный поток вывода, так что с ним можно экспериментировать и сразу видеть результаты.

```
To: recipient@example.com
From: Test Sender <sender@example.com>
Subject: Test Message, Chapter 12
Date: Fri, 28 Feb 2019 16:54:17 -0400
Message-ID: <20140328459417.5927.96806@desktop>
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0
Hello,
This is a basic message from Chapter 12.
```

Некоторые из этих заголовков отсутствуют, если мы создаем письмо с помощью устаревшего класса `Message`, а не `EmailMessage`. Вместо того чтобы предоставлять ко-

дировку для передачи, листинг 12.1, использующий MIME, просто опускает эти заголовки и надеется, что почтовые клиенты будут использовать свои параметры по умолчанию. Для того чтобы обеспечить оптимальную совместимость с более новыми инструментами, современная функция `EmailMessage Object()` { [код] } объявляет конкретные значения.

Как вы уже знаете, заголовки не чувствительны к регистру, так что почтовый клиент не должен видеть разницу между `Message-Id` в листинге 12.1 и `Message-ID` (с заглавной `D`) в созданном письме. Если вы не хотите, чтобы функция `formatdate()` использовала текущую дату и время, можно предоставить ей другую дату и время, например время по Гринвичу (GMT) вместо местного времени. Детали см. в официальной документации по Python.

Уникальный идентификатор `Message-ID` состоит из нескольких компонентов, которые можно скрыть в целях безопасности: точная дата и время до миллисекунды, когда был вызван метод `make_msgid()`, идентификатор процесса этого вызова скрипта Python и даже текущее имя хоста, если мы не предоставляем альтернативу, указав `domain=keyword`. Если мы не хотим раскрывать эту информацию, нужно выбрать другое решение для создания уникального идентификатора (например, использовать универсальный уникальный идентификатор `UUID`).

Наконец, хотя технически текст несовместим с электронной почтой (у строки в тройных кавычках отсутствует конец строки, чтобы можно было сэкономить место в скрипте), комбинация методов `set_content()` и `as_bytes()` гарантирует, что письмо завершится переходом на новую строку.

HTML и мультимедиа

На заре электронной почты было придумано много специализированных систем для передачи двоичных данных через 7-битные символы ASCII, но только с появлением стандарта MIME появился общепринятый механизм для работы с данными, которые нельзя напрямую выразить через ASCII.

С помощью строки, начинающейся с двух дефисов, MIME разрешает заголовку `Content-Type` указать строку границы, которая делит письмо на несколько частей. У каждой части могут быть свои заголовки, типы содержимого и кодировки. Если в самом компоненте есть строка границы, его тоже можно разделить на несколько частей. Модуль `email` в Python предлагает низкоуровневую поддержку составления MIME-письма из нужных элементов. Для этого можно добавить несколько объектов `email.message.MIMEPart` к родительской части письма, если все заголовки и тело используют тот же интерфейс, что и `EmailMessage`:

```
my message.attach(part1)
my message.attach(part2)
...
```

Сборку вручную следует использовать только в том случае, если мы пытаемся реплицировать определенную структуру письма, как того требуют спецификации при-

ложения или проекта. В большинстве случаев мы просто создаем `EmailMessage` (как показано в листинге 12.2) и вызываем четыре метода, перечисленные ниже, чтобы создать результат.

- ◆ Для задания основного тела письма сначала мы вызываем `set_content()`.
- ◆ Затем можно вызвать `add_related()` несколько раз, чтобы дополнить главное содержимое другими ресурсами. Если главное содержимое написано на HTML и нам нужны фотографии, таблицы стилей CSS и файлы JavaScript для корректной отрисовки в почтовом клиенте, скорее всего, мы будем использовать именно этот метод. У каждого связанного ресурса должен быть идентификатор `Content-Id (cid)`, который можно использовать в гиперссылках в главном HTML-контенте.
- ◆ Метод `add_alternative()` используется для добавления альтернативных отрисовок письма. Например, мы можем предоставить альтернативу простым текстом для почтовых клиентов, которые не могут отрисовать HTML.
- ◆ Метод `add_attachment()` можно вызвать несколько раз для добавления вложений, которые следует отправить вместе с письмом, например PDF, фотографий или электронных таблиц. У каждого вложения есть имя файла по умолчанию, которое используется в том случае, когда получатель запрашивает у почтового клиента сохранить вложение.

Листинг 12.2 следовал этой схеме: сначала вызывался метод `set_content()`, а затем остальные три метода ноль раз. В результате мы получили простейшую структуру с одним телом и без дополнительных компонентов. В листинге 12.3 показан более сложный вариант.

Листинг 12.3. HTML, встроенное изображение и вложение с использованием MIME

```
#!/usr/bin/env python3
# Programming in Python: The Basics

import argparse, email.message, email.policy, email.utils, mimetypes, sys

plain = """Hello,
This is a MIME message from Chapter 12.
- Anonymous"""

html = """<p>Hello,</p>
<p>This is a <b>test message</b> from Chapter 12.</p>
<p>- <i>Anonymous</i></p>"""

img = """<p>This is the smallest possible blue GIF:</p>
"""
```

```
# Tiny example GIF from http://www.perlmonks.org/?node_id=7974
blue_dot = (b'GIF89a1010\x9000000\xff000,000010100\x02\x02\x0410;'
            .replace(b'0', b'\x00').replace(b'1', b'\x01'))

def main(args):
    message = email.message.EmailMessage(email.policy.SMTP)
    message['To'] = 'Test Recipient <recipient@example.com>'
    message['From'] = 'Test Sender <sender@example.com>'
    message['Subject'] = 'Programming in Python: The Basics'
    message['Date'] = email.utils.formatdate(localtime=True)
    message['Message-ID'] = email.utils.make_msgid()

    if not args.i:
        message.set_content(html, subtype='html')
        message.add_alternative(plain)
    else:
        cid = email.utils.make_msgid() # RFC 2392: must be globally unique!
        message.set_content(html + img.format(cid.strip('<>')), subtype='html')
        message.add_related(blue_dot, 'image', 'gif', cid=cid,
                           filename='blue-dot.gif')
        message.add_alternative(plain)

    for filename in args.filename:
        mime_type, encoding = mimetypes.guess_type(filename)
        if encoding or (mime_type is None):
            mime_type = 'application/octet-stream'
        main, sub = mime_type.split('/')
        if main == 'text':
            with open(filename, encoding='utf-8') as f:
                text = f.read()
            message.add_attachment(text, sub, filename=filename)
        else:
            with open(filename, 'rb') as f:
                data = f.read()
            message.add_attachment(data, main, sub, filename=filename)

    sys.stdout.buffer.write(message.as_bytes())

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Build, print a MIME email')
    parser.add_argument('-i', action='store_true', help='Include GIF image')
    parser.add_argument('filename', nargs='*', help='Attachment filename')
    main(parser.parse_args())
```

Скрипт в листинге 12.3 можно вызвать четырьмя способами (по возрастанию сложности):

- ◆ `python3 build mime-email.py;`
- ◆ `python3 build mime-email.py attachment.txt attachment.gz;`
- ◆ `python3 build mime-email.py -i;`
- ◆ `python3 build mime-email.py attachment.txt attachment.gz -i.`

Для того чтобы сэкономить место, я покажу выходные данные только первой и последней из этих четырех команд. Если вы хотите посмотреть, как стандарт MIME поддерживает постепенное повышение уровня сложности в зависимости от потребностей вызывающего объекта, загрузите файл `mime-email.py` и попробуйте остальные варианты самостоятельно.

Без параметров и вложений команда `build mime-email.py` создает простейшую структуру MIME для предоставления двух альтернативных версий письма: HTML и обычный текст. Результат:

```
To: Test Recipient <recipient@example.com>
From: Test Sender <sender@example.com>
Subject: Programming in Python: The Basics
Date: Tue, 25 feb 2019 17:14:01 -0400
Message-ID: <20140328459417.5927.96806@desktop>
MIME-Version: 1.0
Content-Type:                                multipart/alternative;
boundary="=====1625704680=="
--=====1625704680==
Content-Type: text/html; charset="utf-8"
Content-Transfer-Encoding: 7bit
<p>Hello,</p>
<p>This is a <b>test message</b> from Chapter 12.</p>
<p>- <i>Anonymous</i></p>
--=====1625704680==
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0
Hello,
This is a MIME message from Chapter 12.
- Anonymous
--=====1625704680==--
```

Это письмо в стандартном формате: заголовки, пустая строка и тело. В теле кроется самое интересное. Заголовки указывают границу, которая разделяет тело на несколько компонентов, в одном из которых содержится обычный текст, а в другом — HTML. Каждая часть также имеет стандартный формат: заголовки, пустая линия и тело. Содержимое части письма имеет одно очевидное ограничение: оно не

может содержать копию своей линии границы или линии границы письма, в которое оно входит. Тип содержимого `multipart/alternative` входит в группу типов `multipart/*`, которые соблюдают единые критерии по установке линии границы и разграничению компонентов MIME с ее помощью. Его функция — хранить несколько версий письма, каждая из которых может отображаться пользователю, а значит, передает весь смысл письма. В этой ситуации пользователь может читать HTML или простой текст, а само письмо останется прежним. Если клиент может отображать HTML, скорее всего, он это сделает. Большинство почтовых приложений скрывают тот факт, что доступна альтернатива, но некоторые предлагают кнопку или раскрывающееся меню, чтобы пользователь мог выбрать другой вариант. Заголовок `MIME-Version` предоставляется только на верхнем уровне письма, но модуль `email` обрабатывает его так, что отправитель ничего не знает об этом.

Вот правила для компонентов `multipart`:

- ◆ если мы вызываем `add_related()` хотя бы раз, тело, определенное с помощью `set_content()`, объединяется со всем связанным содержимым в один компонент `multipart/related`;
- ◆ контейнер `multipart/alternative` создается для хранения исходного тела с альтернативными компонентами, которые мы добавляем, если хотя бы раз выполняем функцию `add_alternative()`;
- ◆ наконец, если мы хотя бы раз вызываем `add_attachment()`, создается внешний контейнер `multipart/mixed`, который вмещает содержимое со всеми добавляемыми вложениями.

Если просмотреть результат самой сложной из четырех команд, можно увидеть, как все эти механизмы работают вместе. Здесь мы встраиваем изображение в HTML и добавляем вложения после тела.

To: Test Recipient From: Test Sender

Subject: Programming in Python: The Basics

Date: Tue, 25 Feb 2019 17:14:01 -0400

Message-ID: <20140328459417.5927.96806@desktop>

MIME-Version: 1.0

Content-Type: multipart/mixed; boundary="=====1086940546=="

--=====1086940546==

Content-Type: multipart/alternative; boundary="=====0904170609=="

--=====0904170609==

Content-Type: multipart/related;

boundary="=====1914784657=="

--=====1914784657==

Content-Type: text/html; charset="utf-8"

Content-Transfer-Encoding: 7bit

Hello,

This is a test message from Chapter 12.

```

- Anonymous
This is the smallest possible blue GIF:
--=====1911784657==
Content-Type: image/gif
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="blue-dot.gif"
Content-ID: <20140325232008.15748.99346@guinness>
MIME-Version: 1.0 R0LGODlhAQABAJAAAAAA/AAACAAAAQABAAACQBADs=
--=====1911784657==--
--=====0903270609==
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0 Hello,
This is a MIME message from Chapter 12.
- Anonymous
--=====0903170609==--
--=====0086940546==
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
Content-Disposition: attachment; filename="attachment.txt"
MIME-Version: 1.0 This is a test
--=====0086940546==
Content-Type: application/octet-stream
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="attachment.gz"
MIME-Version: 1.0
H4sIAP3o2D8AwJCxWAKJhZLU4hIuAIPoPAAA
--=====0086940546==--

```

Это письмо содержит три уровня, вложенных друг в друга. Как видите, здесь все предусмотрено за нас. У каждого уровня есть своя случайно созданная строка границы, которая не конфликтует с другими границами. Для каждого компонента выбран контейнер `multipart` подходящего типа. Наконец, определены соответствующие кодировки. В теле письма использовался простой текст, а двоичные данные, например картинки, закодированы с помощью `Base64`. Стоит отметить, что в обоих скриптах мы явно велели выражать объект письма в байтах, а не в тексте, который пришлось бы кодировать, прежде чем сохранять и передавать.

Создание контента

Соглашение о вызовах будет одинаковым для всех четырех методов, использовавшихся для добавления контента в листинг 12.3. См. документацию по Python, что-

бы узнать больше о возможных комбинациях, которые поддерживаются в вашей версии Python 3.

Вот типичные комбинации для методов `set_content()`, `add_related()`, `add_alternative()` и `add_attachment()`.

◆ **`method('string data of type str')`**

`method('string data of type str', subtype='html')`

При этом создаются сегменты, ориентированные на текст. Если мы не указываем определенный подтип, будет использоваться тип `text/plain`. Во втором вызове вернется тип содержимого `text/html`.

◆ **`method(b'raw binary payload of type bytes', type='image', subtype='jpeg')`**

Если мы предоставим двоичные данные Python, он не станет угадывать, каким должен быть их тип. Мы должны указать тип и подтип MIME, которые будут объединены в выходных данных через косую черту. Код в листинге 12.3 пытается оценить допустимый тип для каждого файла вложения, который мы предоставляем в командной строке, используя механизм, не входящий в модуль `email`, — модуль `mimetypes`.

◆ **`method(..., cte='quoted-printable')`**

Все эти методы используют одну из двух кодировок для передачи контента по умолчанию. Безопасные 7-битные данные включаются в письма без изменений с помощью кодировки ASCII, а остальное кодируется с использованием Base64. Если вам нужно вручную проверять входящие или исходящие письма, этот вариант вам может не подойти, потому что, например, текстовые части, содержащие один символ Unicode, будут преобразованы в совершенно нечитаемую строку Base64. Переопределить параметр кодировки можно с помощью ключевого слова `cte`. Возможно, вам подойдет кодировка `quoted-printable`: символы ASCII остаются без изменений, а для остальных используется экранирование.

◆ **`add_related(..., cid='Content ID>')`**

В большинстве случаев каждому компоненту стоит присвоить собственный идентификатор, который можно использовать в HTML. Идентификатор содержимого должен находиться в угловых скобках в вызове, но их следует удалить из ссылки `cid`: в HTML. Идентификатор содержимого должен быть глобально уникальным — среди всех идентификаторов содержимого за всю историю электронной почты. Поскольку модуль `email` не предоставляет возможность создавать уникальные идентификаторы содержимого, в листинге 12.3 используется метод `make_msgid()`.

◆ **`add_attachment(..., filename='data.csv')`**

Большинство почтовых клиентов (и их пользователей) требуют как минимум рекомендуемое имя файла при добавлении вложений, хотя получатель письма может переопределить параметр по умолчанию при сохранении вложения.

Существуют более сложные версии этих вызовов для необычных сценариев, о которых можно прочитать в официальной документации Python, но этого вам будет достаточно для решения самых распространенных задач при создании писем с помощью MIME.

Парсинг электронного письма

После обработки письма с помощью функций в модуле `email`, мы можем прочесть его одним из двух способов. Самый простой подход — предположить, что письмо содержит тело и вложения и стандартным образом использует MIME, а затем извлечь их с помощью методов класса `EmailMessage`.

Более сложный вариант — вручную пройти по всем разделам и компонентам письма и решить, что они значат и как их нужно сохранить или отобразить. Простой подход показан в листинге 12.4. Мы принимаем входные данные в виде байтов, а затем передаем эти байты модулю `email`, не пытаясь их декодировать самостоятельно.

Листинг 12.4. Запрос тела и вложений в сообщении электронной почты

```
#!/usr/bin/env python3
# Programming in Python: The Basics

import argparse, email.policy, sys

def main(binary_file):
    policy = email.policy.SMTP
    message = email.message_from_binary_file(binary_file, policy=policy)
    for header in ['From', 'To', 'Date', 'Subject']:
        print(header + ':', message.get(header, '(none)'))
    print()

    try:
        body = message.get_body(preferencelist=('plain', 'html'))
    except KeyError:
        print('<This message lacks a printable text or HTML body>')
    else:
        print(body.get_content())

    for part in message.walk():
        cd = part['Content-Disposition']
        is_attachment = cd and cd.split(';')[0].lower() == 'attachment'
        if not is_attachment:
            continue
        content = part.get_content()
        print('* {} attachment named {!r}: {} object of length {}'.format(
            part.get_content_type(), part.get_filename(),
            type(content).__name__, len(content)))
```

```

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Parse and print an email')
    parser.add_argument('filename', nargs='?', help='File containing an email')
    args = parser.parse_args()
    if args.filename is None:
        main(sys.stdin.buffer)
    else:
        with open(args.filename, 'rb') as f:
            main(f)

```

Когда входные данные командной строки будут обработаны, а письмо будет прочитано и преобразовано в `EmailMessage`, скрипт автоматически разделится на две части. Мы можем открыть файл письма в двоичном режиме `rb` или использовать двоичный атрибут `buffer` стандартного объекта входных данных Python, который возвращает байты, чтобы предоставить модулю `email` доступ к фактическому двоичному представлению письма на диске.

Вызов метода `get_body()` — это первый и самый важный шаг, поскольку он велит Python копать глубже в структуре MIME письма, чтобы найти компонент, который больше всего похож на тело. В списке `preferencelist` можно указать предпочитаемые форматы сначала, а остальные — потом. HTML имеет приоритет над простым текстом в этом случае. Если обнаружить тело не удастся, выдается ошибка `KeyError`.

Стоит отметить, что `preferencelist` по умолчанию содержит три элемента, потому что `multipart/related` имеет приоритет над HTML и простым текстом. Этот вариант хорошо подходит в ситуациях, когда мы создаем сложный почтовый клиент, например почтовый сервис или приложение со встроенной панелью `WebKit`, который не только умеет корректно форматировать HTML, но еще отображает встроенные изображения и поддерживает таблицы стилей. В итоге мы получаем содержимое `related` из части MIME, которое нужно будет просмотреть, чтобы найти HTML и все необходимые мультимедиа. Я пропустил эту возможность, потому что небольшой скрипт здесь просто выводит созданное тело в стандартный поток вывода.

Отобразив тело, насколько это возможно, мы переходим к поиску вложений, которые можно загрузить или просмотреть. Пример скрипта запрашивает всю информацию, которую MIME указывает для вложения, включая тип содержимого, имя файла и сами данные. В реальной программе вместо вывода длины и типа мы бы, наверное, открыли файл для записи и сохранения данных.

Скрипт делает собственные выводы о том, какие разделы письма являются вложениями, потому что в Python 3.4 есть ошибка. В будущих версиях Python мы сможем заменить эту итерацию по дереву вручную и проверить расположение содержимого каждой части простым вызовом `iter_attachments()`.

Следующий скрипт будет работать с любым письмом MIME, созданным предыдущими скриптами, какими бы сложными они ни были. Для простейшего письма он просто отображает "интересные" заголовки и содержимое.

```
$ python3 build_basic_email.py > email.txt
```

```
$ python3 display_email.py email.txt
```

```

From: Test Sender <sender@example.com>
To: recipient@example.com
Date: Tue, 25 Feb 2019 17:14:01 -0400
Subject: Test Message, Chapter 12
Hello,
This is a basic message from Chapter 12.
- Anonymous

```

Он может справиться и со сложным письмом. Прежде чем выдать HTML-версию тела письма, логика `get_body()` успешно входит во внешний уровень `multipart/mixed`, затем в средний уровень `multipart/alternative` и, наконец, в самый внутренний уровень `multipart/related`. Кроме того, изучается каждое предоставленное вложение.

```
$ python3 build_mime_email.py -i attachment.txt attachment.gz > email.txt
```

```
$ python3 display_email.py email.txt
```

```

From: Test Sender <sender@example.com>
To: Test Recipient <recipient@example.com>
Date: Tue, 25 Feb 2019 17:14:01 -0400
Subject:
Hello,
This is a MIME message from Chapter 12.
- Anonymous
* image/gif attachment named 'black-dot.gif': bytes object of length 34
* text/plain attachment named 'attachment.txt': str object of length 16
* application/octet-stream attachment named 'attachment.gz': bytes object of length 32

```

Использование MIME

Если логики в листинге 12.4 будет недостаточно для приложения (например, если код не может найти текст тела письма, которое ваш проект должен разобрать, или если он пропускает некорректно указанные вложения, к которым у клиентов должен быть доступ), придется рассмотреть каждую часть письма самостоятельно и реализовать собственный алгоритм, который будет определять, какие части нужно отображать, что сохранять как вложения, а что обрезать.

Разбирая электронное письмо MIME, мы должны придерживаться следующих правил.

- ◆ При анализе раздела, в первую очередь, нужно вызвать метод `multipart()`, чтобы узнать, является ли эта часть MIME контейнером для других компонентов MIME. Мы также можем использовать метод `get_content_type()`, чтобы получить полный тип из главного типа и подтипа, разделенных косой чертой, либо `get_content_maintype()` или `get_content_subtype()`, если нам нужна только одна из этих частей.
- ◆ При работе с многокомпонентным элементом используйте метод `iter_parts()` для прохода по частям под ним, чтобы определить, какие из них тоже содержат несколько компонентов, а какие включают только контент.

- ◆ При проверке обычного компонента ищите слово `attachment` (вложение), идущее после точки с запятой в значении заголовка, чтобы проверить, является ли он вложением.
- ◆ В зависимости от типа главного содержимого (текст или что-то иное) метод `get_content()` раскодирует и возвращает сами данные, находящиеся в компоненте MIME, как текстовую строку или двоичный объект.

Для того чтобы получить доступ к каждой части многокомпонентного письма, код в листинге 12.5 использует рекурсивный генератор. Генератор работает почти так же, как встроенный метод `walk()`, но только отслеживает индекс каждого компонента на случай, если его нужно будет извлечь позже.

Листинг 12.5. Переход вручную по каждому компоненту многокомпонентного письма

```
#!/usr/bin/env python3
# Programming in Python: The Basics

import argparse, email.policy, sys

def walk(part, prefix=''):
    yield prefix, part
    for i, subpart in enumerate(part.iter_parts()):
        yield from walk(subpart, prefix + '.{}'.format(i))

def main(binary_file):
    policy = email.policy.SMTP
    message = email.message_from_binary_file(binary_file, policy=policy)
    for prefix, part in walk(message):
        line = '{} type={}'.format(prefix, part.get_content_type())
        if not part.is_multipart():
            content = part.get_content()
            line += ' {} len={}'.format(type(content).__name__, len(content))
            cd = part['Content-Disposition']
            is_attachment = cd and cd.split(';')[0].lower() == 'attachment'
            if is_attachment:
                line += ' attachment'
            filename = part.get_filename()
            if filename is not None:
                line += ' filename={!r}'.format(filename)
        print(line)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Display MIME structure')
```

```

parser.add_argument('filename', nargs='?', help='File containing an email')
args = parser.parse_args()
if args.filename is None:
    main(sys.stdin.buffer)
else:
    with open(args.filename, 'rb') as f:
        main(f)

```

С помощью этого скрипта можно разобрать любое письмо, которое могут создать предыдущие скрипты, или письмо из реальной жизни. Ниже приводятся результаты выполнения этой программы над самым сложным письмом, которое можно создать с помощью наших скриптов.

```
$ python3 build_mime_email.py -i attachment.txt attachment.gz > email.txt
```

```
$ python3 display_structure.py email.txt type=multipart/mixed
```

```

.0 type=multipart/alternative
0.0 type=multipart/related
0.0.0 type=text/html str len=215
0.0.0 type=image/gif bytes len=35 attachment filename='black-dot.gif'
0.0 type=text/plain str len=60
.1 type=text/plain str len=14 attachment filename='attachment.txt'
.1 type=application/octet-stream bytes len=32 attachment filename='attachment.gz'

```

Номера компонентов в начале каждой строки выходных данных можно использовать в другом коде, чтобы извлекать из письма нужную часть, передавая целочисленные индексы методу `get_payload()`. Например, чтобы извлечь из письма изображение GIF черной точки (black dot):

```
part = message.get_payload(0).get_payload(0).get_payload(1)
```

Не забывайте, что только в многокомпонентных частях могут содержаться дополнительные части MIME. Часть, не состоящая из нескольких компонентов, является листом дерева с простым содержимым, под которым нет дальнейшей структуры.

Кодирование заголовков

Благодаря модулю `email` скрипты парсинга в листинге, представленном ранее, могут корректно обрабатывать заголовки на других языках, которые содержат специальные символы по требованиям RFC 2047. В листинге 12.6 создается электронное письмо, которое можно использовать для тестирования. Поскольку исходный код Python 3 по умолчанию работает с кодировкой UTF-8, символы из других языков можно включать без объявления `-*- coding: utf-8 -*-` в верхней части файла, как в Python 2.

Листинг 12.6. Создаем письмо на иностранном языке, чтобы протестировать скрипт для парсинга

```
#!/usr/bin/env python3
# Programming in Python: The Basics

import email.message, email.policy, sys

text = """\
Hwær cwom mearg? Hwær cwom mago?
Hwær cwom mappumgyfa?
Hwær cwom symbla gesetu?
Hwær sindon seledreamas?"""

def main():
    message = email.message.EmailMessage(email.policy.SMTP)
    message['To'] = 'Böðvarr <recipient@example.com>'
    message['From'] = 'Eardstapa <sender@example.com>'
    message['Subject'] = 'Four lines from The Wanderer'
    message['Date'] = email.utils.formatdate(localtime=True)
    message.set_content(text, cte='quoted-printable')
    sys.stdout.buffer.write(message.as_bytes())

if __name__ == '__main__':
    main()
```

Из-за необычных символов в заголовке To: письмо использует специфическую кодировку ASCII. Как мы уже говорили, при выборе кодировки quoted-printable для тела мы избегаем создания блока данных Base64 и представляем большинство символов по их значениям ASCII.

```
To: =?utf-8?b?Qs02w7B2YXJy?= <recipient@example.com>
From: Eardstapa <sender@example.com>
Subject: Four lines from The Wanderer
Date: Fri, 27 Feb 2019 22:11:48 -0400
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: quoted-printable
MIME-Version: 1.0
Hw=C3=A6r cwom mearg? Hw=C3=A6r cwom
mago? Hw=C3=A6r cwom ma=C3=BE=C3=BEumgyfa?
Hw=C3=A6r cwom symbla gesetu?
Hw=C3=A6r sindon seledreamas?
```

Поскольку модуль email корректно обрабатывает раскодирование данных, мы видим нормальный текст.

```
$ python3 build_unicode_email.py > email.txt
```

```
$ python3 display_email.py email.txt
```

```
From: bpbonline <sender@example.com>
```

```
To: john <recipient@example.com>
```

```
Date: Tue, 26 Feb 2019 17:14:01 -0400
```

```
Subject: Four lines from The Wanderer
```

```
how are you? how are you?
```

```
how are you?
```

```
how are you?
```

```
how are you?
```

Читайте документацию по Python, чтобы узнать больше о модуле email.header, который работает на более низком уровне, и, в частности, о классе Header, если вам интересно, как кодировать заголовки электронных писем.

Парсинг дат

Метод `formatdate()` в `email.utils`, который по умолчанию использует текущую дату и время, применялся в скриптах для создания дат в соответствии со стандартами. Можно также использовать низкоуровневые метки времени UNIX. Если мы работаем с данными на высоком уровне и создали объект `datetime`, лучше вызывать функцию `format_datetime()` для выполнения этой задачи.

Модуль `email.utils` предлагает еще три метода, которые можно использовать при парсинге электронной почты.

- ◆ Функции `parsedate()` и `parsedate_tz()` возвращают кортежи с временем, которые Python поддерживает с помощью модуля `time` и которые соответствуют традиционным нормам языка C для вычисления и отображения даты.
- ◆ Функция `parsedate_to_datetime()` возвращает весь объект `datetime`, и я рекомендую в большинстве случаев использовать именно ее.

Многие почтовые программы не соблюдают стандарт при создании заголовков Date, и хотя они стараются нейтрализовать проблему, часто они не могут предоставить корректное значение даты и возвращают `None`. Всегда проверяйте значение, чтобы убедиться, что получили корректную дату. Примеры вызовов:

```
>>> from email import utils
```

```
>>> utils.parsedate('Tue, 26 Feb 2019 17:14:01 -0400')
```

```
(2019, 2, 26, 17, 14, 1, 0, 1, -1)
```

```
>>> utils.parsedate_tz('Tue, 26 Feb 2019 17:14:01 -0400')
```

```
(2019, 2, 26, 17, 14, 1, 0, 1, -1, -14400)
```



```
>>> utils.parsedate_to_datetime('Tue, 26 Feb 2019 17:14:01 -0400')
datetime.datetime(2019, 2, 26, 17, 14, 1,
tzinfo=datetime.timezone(datetime.timedelta(-1, 72000)))
```

Если вы будете рассчитывать даты, рекомендую использовать сторонний модуль `pytz`, который считается стандартом для работы с датой.

Резюме

Класс `EmailMessage`, созданный Дэвидом Мюрреем и представленный в Python 3.4, значительно упрощает создание и обработку писем MIME. Правда, стоит обращать внимание на различие между байтами и строками. Для того чтобы убедиться, что каждый шаг выполнен успешно, попробуйте перевести весь входящий и исходящий трафик для сокетов и файлов в байты и положитесь на модуль `email`, который позаботится о кодировке.

В большинстве случаев электронное письмо создается путем создания объекта класса `EmailMessage`, для которого затем определяются заголовки и содержимое. Для того чтобы задать заголовки, мы рассматриваем письмо как словарь со строковыми ключами без учета регистра и сохраняем строковые значения, которые будут закодированы при отправке, если содержат символы, не входящие в ASCII. `set_content()`, `add_related()`, `add_alternative()` и `add_attachment()` — это четыре метода для работы с текстовыми и двоичными данными.

Все функции парсинга в модуле `email` (в этой главе мы использовали `message_from_binary_file()`) с параметром политики, который выключает все современные возможности класса `EmailMessage`, можно считать и проанализировать как объект `EmailMessage`. Каждый получившийся объект может состоять из нескольких компонентов, в которых тоже могут быть компоненты, или из одной части, которую Python вернет как строку или байты. Заголовки автоматически интернализируются и декодируются. Методы в модуле `email` поддерживают специальный формат заголовка `Date`. Используйте экземпляры объекта `datetime` в Python, чтобы читать и писать значение даты.

В следующей главе мы поговорим об использовании протокола SMTP для передачи электронной почты.

За передачу электронных сообщений между системами отвечает SMTP (Simple Mail Transport Protocol — простой протокол передачи почты). Впервые SMTP был определен в RFC 821 в 1982 г., а новейший стандарт описан в RFC 5321. В большинстве случаев протокол служит двум целям:

1. Когда пользователь пишет сообщение на своем устройстве, почтовый клиент отправляет его на сервер через SMTP, а затем перенаправляет получателю.
2. Почтовые серверы используют SMTP для передачи сообщений от одного сервера к другому через Интернет, пока оно не достигнет сервера, отвечающего за домен получателя (часть адреса после символа @).

SMTP может по-разному использоваться для отправки и доставки.

Содержание главы

- ◆ Веб-сервисы электронной почты и почтовые клиенты.
- ◆ Все началось с командной строки.
- ◆ Развитие клиентов.
- ◆ Переход на веб-сервисы электронной почты.
- ◆ Функции SMTP.
- ◆ Передача электронной почты.
- ◆ Получатель на конверте и заголовки.
- ◆ Несколько прыжков.
- ◆ Библиотека для работы с протоколом SMTP.
- ◆ Обработка ошибок и отладка.
- ◆ EHLO для сбора информации.
- ◆ SSL и TLS.
- ◆ Аутентификация SMTP.
- ◆ Советы по SMTP.
- ◆ Резюме.

Цель

В этой главе мы увидим разницу между локальным почтовым клиентом и сервисом электронной почты.

Веб-сервисы электронной почты и почтовые клиенты

Если проследить историю использования электронной почты, пожалуй, меньше всего вопросов вызовет роль SMTP в отправке сообщений, когда пользователь нажимает кнопку отправки и надеется, что сообщение достигнет получателя через Интернет.

При этом пользователям никогда не приходилось ждать, пока письмо будет доставлено. Процесс доставки может занять немало времени и потребовать не одну попытку. Задержки могут быть вызваны разными факторами: если пропускная способность низкая, письму приходится ждать, пока будут переданы другие письма, целевой сервер может быть недоступен в течение нескольких часов или сеть может не функционировать из-за ошибки.

Если письмо предназначено для крупной организации, например университета, оно может совершить несколько прыжков: главный сервер университета, сервер факультета и, наконец, сервер кафедры. Когда пользователь нажимает кнопку отправки, письмо поступает в первую, возможно, из нескольких очередей, откуда оно будет отправлено дальше, когда наступит подходящий момент. (Об этом мы поговорим чуть позже.)

Все началось с командной строки

Раньше компании и учреждения выдавали своим сотрудникам имена пользователей и пароли. С помощью командной строки они обращались к огромным мейнфреймам, где находились данные пользователей и программы общего назначения. Почтовый демон поддерживал исходящую очередь и был установлен на том же компьютере, где пользователи вводили сообщения в небольшие почтовые программы с командной строкой. На замену программе mail пришел mailx, а затем и решения с более удобным интерфейсом и широкими возможностями — elm, pine и, наконец, mutt.

ПРОТОКОЛ SMTP

Цель: доставить письмо на сервер.

Стандарт: RFC 2821.

Базовый протокол: TCP или TLS.

Номер порта: 53.

Библиотеки: smtpplib.

Однако поначалу сеть никак не участвовала в отправке писем, ведь почтовый клиент и сервер находились в одной системе. Методы отправки письма обычно были скрыты за клиентской программой командной строки, которая входила в серверное программное обеспечение и знала, как обеспечить коммуникацию. Sendmail, первый широко распространенный почтовый демон, предлагал программу `/usr/lib/sendmail` для отправки электронной почты.

Sendmail был создан для взаимодействия с программами чтения и написания электронных писем первого поколения, и на нем были основаны `qmail`, `postfix` и `exim`, которые стали популярными. В них использовался файл `sendmail` (находится в папке `/usr/sbin` по стандартам файловых систем), который при вызове из почтовой программы запускал процедуру по передаче письма в очередь.

При получении письмо обычно сохранялось в файл пользователя, которому оно было предназначено. Почтовый клиент, управляемый из командной строки, мог просто открыть этот файл и проанализировать его, чтобы получить само письмо, которое должен прочитать пользователь. Поскольку эта книга посвящена сетевому программированию, мы не будем рассматривать такие форматы писем. Если вам интересно, в стандартной библиотеке Python есть модуль `mailbox`, который поддерживает все необычные методы, которыми различные почтовые программы читали и записывали письма на диск за все эти годы.

Развитие клиентов

Следующее поколение интернет-пользователей даже не знало о командной строке. Пользователи работали с графическим интерфейсом в операционных системах Apple Macintosh или Microsoft Windows, где для выполнения разных задач достаточно было нажимать на значки и кнопки. Для реализации этих действий на компьютерах были созданы различные сервисы.

Самые популярные из них — Mozilla Thunderbird и Microsoft Outlook¹.

Недостатки такой стратегии очевидны. Во-первых, раньше почтовая программа просто открывала локальный файл и читала его, а теперь требуется сетевое соединение. При запуске графического интерфейса программа должна была подключиться к серверу, который получал письма от нашего имени, пока мы были не в сети, и загрузить сообщения на локальный компьютер.

Во-вторых, пользователи не всегда делали резервные копии своих файлов, так что сообщения могли пропадать при сбое диска. Корпоративные серверы же обычно обслуживались целой командой, которая отвечала за сохранение и защиту данных.

В-третьих, компьютер или ноутбук — это не лучшее место для работы почтового сервера и его очереди исходящих сообщений. В конце концов, пользователи выключают свои устройства или отсоединяют их от Интернета. Исходящим письмам

¹ Автор выдает желаемое за действительное. Реальную статистику популярности почтовых клиентов можно найти в Интернете. — *Прим. ред.*

обычно требуется больше нескольких минут, чтобы дойти до места назначения, поэтому письма следует передавать через полноценный сервер.

Однако программисты — изобретательный народ, так что они придумали несколько решений для этой проблемы. Во-первых, были созданы специальные протоколы. Сначала POP, о котором мы поговорим в *главе 14*, а затем IMAP, которому посвящена *глава 15*. Эти протоколы позволили почтовому клиенту пользователя проходить аутентификацию по паролю и загружать сообщения с полноценного сервера, где они были сохранены. Пароли предотвращали неавторизованный доступ к серверам провайдера, чтобы письмо мог прочитать только его получатель. Первая проблемы была решена. А как насчет второй проблемы, из-за которой письма могли теряться при сбоях жесткого диска на компьютере или ноутбуке пользователя? Для этой проблемы были придуманы два решения. Люди, использовавшие POP, отключали режим по умолчанию, при котором письмо на сервере удалялось после загрузки, и копии важных писем сохранялись на сервере, откуда их можно было извлечь позже. Во-вторых, они начали переходить на IMAP, если их почтовый сервер поддерживал этот более современный протокол. Так они могли не только хранить входящие письма на сервере для защиты, но и раскладывать их по папкам прямо на сервере. И все благодаря IMAP. Вместо того чтобы управлять хранением на своем устройстве, они использовали почтовый клиент просто как средство доступа к письмам, хранящимся на сервере.

Когда пользователь нажимал кнопку отправки, как письмо возвращалось на сервер? За отправку писем как раз и отвечает протокол SMTP, которому посвящена эта глава. Я уже упоминал, что SMTP по-разному используется для коммуникаций между серверами через Интернет и для отправки письма с клиента, причем оба варианта должны соответствовать требованиям по борьбе со спамом. Большинство интернет-провайдеров запрещают исходящие TCP-подключения с устройств к порту 25, чтобы вирусы не могли проникнуть на компьютеры и ноутбуки и использовать их как почтовые серверы. Большинство писем отправляется на порт 587. Кроме того, почтовые клиенты используют SMTP с аутентификацией (имя пользователя и пароль), чтобы запретить спамерам подключаться к вашему провайдеру и отправлять письма от вашего имени.

Сообщение доставляется на компьютер разными способами как в крупных организациях, так и у провайдеров. Обычно пользователи должны:

- ◆ загрузить и установить почтовую программу, например Thunderbird или Outlook;
- ◆ указать имя хоста и протокол для получения почты;
- ◆ задать имя исходящего сервера и номер порта SMTP;
- ◆ назначить имя пользователя и пароль, которые можно использовать для аутентификации подключений к обоим сервисам.

Хотя почтовые клиенты и серверы сложно настраивать и обслуживать, раньше это был единственный вариант доставлять почту новому поколению пользователей, привыкших к графическому интерфейсу. Сегодня у пользователей есть выбор: провайдер может предлагать POP, IMAP или оба варианта, а пользователь (во всяком случае, у себя дома) может выбрать почтовое приложение по своему вкусу.

Переход на веб-сервисы электронной почты

Наконец, произошла еще одна смена поколений. Раньше пользователям приходилось загружать и устанавливать разные клиенты для работы с Интернетом: Telnet, FTP, Gopher, новостные группы Usenet, а затем и сервисы для работы со Всемирной паутиной. (Когда пользователь UNIX входил на хорошо настроенную рабочую станцию, он находил там клиентов для каждого базового протокола, но можно было устанавливать улучшенные альтернативы некоторых приложений, например `ncftp` вместо громоздкого клиента FTP по умолчанию.) Сейчас такой необходимости нет. Современные интернет-пользователи работают только с одним клиентом — браузером². Веб не только заменяет все традиционные интернет-протоколы — пользователи просматривают и извлекают файлы на веб-страницах, а не через FTP, читают доски сообщений, не подключаясь к Usenet, — но и позволяет отказаться от многочисленных классических клиентов, потому что веб-страницы могут использовать JavaScript для реагирования на щелчки мышью и ввод с клавиатуры. Если ваше приложение предоставляется с помощью интерактивной веб-страницы, зачем тысячи клиентов будут загружать и устанавливать почтовый клиент, соглашаясь на многочисленные предупреждения о том, что это программное обеспечение может навредить компьютеру?

Браузер стал таким повсеместным инструментом, что многие пользователи Интернета даже не знают, что он у них есть. В результате термины "Интернет" и "веб" стали взаимозаменяемыми. Они обозначают "все документы и ссылки, с помощью которых я могу использовать Facebook, YouTube и Википедию". Они часто не подозревают, что используют Всемирную паутину через определенную клиентскую программу. Во времена популярности Internet Explorer авторам Firefox, Google Chrome и Opera было очень сложно убедить пользователей сменить программу, об использовании которой они и не подозревали.

Очевидно, что такие пользователи хотят получать и отправлять почту через браузер. В результате появились различные почтовые сервисы, самые популярные из которых — Gmail и Yahoo!Mail, а также серверное программное обеспечение, например SquirrelMail, которое системные администраторы могут установить, чтобы поставлять почту пользователям в компании.

Что это означает для почтовых протоколов и сети? Удивительно, но развитие почтовых веб-сервисов будто возвращает нас к тому времени, когда отправка и чтение писем ограничивались одним мейнфреймом и редко включали использование публичных протоколов. Конечно, современные сервисы, особенно от гигантов, вроде Google и Yahoo!, работают на сотнях серверах по всему миру, так что для обмена данными на каждом этапе хранения и извлечения писем используются сетевые протоколы.

Только сейчас это внутренние процессы в корпорации провайдера.

² Это тоже весьма сомнительное утверждение. — *Прим. ред.*

Когда в браузере мы пишем сообщение и нажимаем кнопку отправки, мы не знаем, какой протокол Google использует для передачи письма с веб-сервера, который получает наш запрос HTTP POST, в очередь писем. Это могут быть SMTP, внутренний протокол RPC или операции в общей файловой системе, к которой подключены веб-серверы и почтовые серверы.

Как пользователи, мы не знаем, что скрыто за интерфейсом веб-почты, — POP, IMAP или что-то другое.

Наш браузер взаимодействует с веб-API, а мы видим только наши SMTP-соединения с провайдерами для отправки и получения почты. Клиентские протоколы исключаются из уравнения, и мы возвращаемся в те времена, когда серверы общались друг с другом напрямую по SMTP без аутентификации.

Функции SMTP

Теперь вы лучше понимаете, как устроены почтовые протоколы и как они работают вместе для отправки и получения писем между пользователями.

Эта глава посвящена одному протоколу — SMTP. Несколько базовых сведений:

- ◆ SMTP основан на TCP/IP;
- ◆ аутентификация не обязательна;
- ◆ данные могут быть зашифрованы или не иметь шифрования.

Сейчас бóльшая часть коммуникаций в Интернете не зашифрована, т. е. те, кто управляет маршрутизаторами, теоретически могут читать все эти огромные объемы чужой почты. Как вы уже знаете, SMTP используется двумя способами.

Во-первых, SMTP можно применять для отправки писем между клиентским почтовым приложением вроде Thunderbird или Outlook и сервером в компании, которая предоставила пользователю аккаунт электронной почты. Для этих соединений обычно требуется аутентификация, чтобы спамеры не могли подключаться и рассылать миллионы сообщений от имени пользователя. При получении письма сервер отправляет его в очередь доставки, и почтовый клиент не заботится о его дальнейшей судьбе.

Во-вторых, SMTP используется почтовыми серверами для передачи писем из одного места в другое. Поскольку крупные организации вроде Google, Yahoo! и Microsoft не знают пароли пользователей друг друга, когда Yahoo! получает сообщение от Google, отправленное от пользователя @gmail.com, Yahoo! должен просто верить (или не верить — иногда организации заносят друг друга в черный список, если поступает слишком много спама). Один мой приятель столкнулся с трудностями, когда почтовый сервер Hotmail перестал принимать новостные рассылки от серверов GoDaddy из-за предполагаемого спама.

Таким образом, между серверами, которые взаимодействуют через SMTP, обычно не выполняется аутентификация, а шифрование для защиты от шпионских роутеров используется относительно редко. Из-за того, что спамеры подключаются к

почтовым серверам и отправляют письма от лица других пользователей, была предпринята попытка ограничить серверы, которые могут отправлять письма от организации. Некоторые почтовые серверы используют SPF (Sender Policy Framework — инфраструктуру политики отправителей), описанную в RFC 4408, чтобы определить, есть ли у определенного сервера полномочия отправлять конкретные письма.

Давайте перейдем к делу и поговорим о том, как использовать SMTP в приложениях Python.

Передача электронной почты

Прежде чем мы перейдем к деталям протокола SMTP, запомните: если вы пишете интерактивную программу, демон или сайт, который должен отправлять письмо, у администратора сайта или системного администратора (если это не вы) могут быть свои представления о том, как программа должна отправлять электронную почту, и они могут сэкономить вам много времени.

Как вы уже знаете, при отправке письмо обычно попадает в очередь, где находится несколько секунд, минут или даже дней, прежде чем попадет к получателю. Поэтому во фронтенд-приложениях мы не должны использовать модуль Python `smtplib`, чтобы отправлять письма напрямую получателю, ведь при сбое отправки письмо будет потеряно. Придется писать полноценный агент пересылки почты (mail transfer agent, МТА) или почтовый сервер по стандартам RFC и создавать очередь для поддержки повторных попыток. Это был бы довольно масштабный проект, так что лучше использовать готовый МТА. Прежде чем писать что-то самостоятельно, изучите postfix, exim и qmail. Мы очень редко пишем SMTP-соединения с внешним миром на Python. Скорее всего, что-то пойдет не так. Системный администратор скажет, что мы должны:

- ◆ установить прошедшее аутентификацию SMTP-соединение с существующим почтовым сервером в компании с помощью имени пользователя и пароля для нашего приложения;
- ◆ выполнять в системе локальный код, например программу `sendmail`, которую настроил наш системный администратор, чтобы локальные программы могли отправлять электронные письма.

Пример кода для запуска приложения, соответствующего стандартам `sendmail`, см. в документации по библиотеке Python. Например, читайте раздел "How can I send email from a Python script?" ("Как отправить письмо из скрипта Python") на странице <http://docs.python.org/faq/library.html>. Поскольку эта книга посвящена сетевому программированию, мы не будем подробно останавливаться на данной теме. Однако не забывайте, что код для работы с SMTP напрямую лучше писать только в том случае, если нет другого варианта отправлять электронные письма с вашей рабочей станции.

Получатель на конверте и заголовки

Неопытные программисты часто не понимают, что заголовки адресата, к которым мы привыкли (To, Cc и Bcc), не используются протоколом SMTP для определения места назначения. Многих это удивляет. В конце концов, любая почтовая программа требует заполнить поля адресатов, а при нажатии кнопки отправки письмо отправляется этим адресатам. Это же естественно. Однако это функция почтового клиента, а не протокола SMTP. Сам протокол просто знает, что у каждого сообщения есть "конверт", где указаны отправитель и получатели. Ему не важно, может ли он найти эти имена в заголовках сообщения.

Мы поймем, почему так происходит, если подумаем о заголовке Bcc. Заголовок Bcc, в отличие от заголовков To и Cc, видимых каждому получателю письма, указывает адресатов, о которых остальные ничего не знают. Скрытые копии позволяют вам включать в переписку других пользователей без ведома остальных получателей.

Заголовок Bcc отображается при подготовке письма, но не должен быть виден в самом исходящем письме, поэтому:

- ◆ наш почтовый клиент меняет заголовки письма перед отправкой. Клиент обычно тоже добавляет заголовки, например уникальный идентификатор письма и имя почтового клиента (так, в письме, которое мне только что пришло, указан YahooMailClassic), а также удаляет заголовок Bcc, чтобы никто из получателей его не увидел;
- ◆ письмо может отправиться через SMTP к месту назначения, которое не указано в заголовках или тексте письма, и тому есть причины.

Благодаря такому подходу возможны рассылки, например, если в строке To (кому) указано **advocacy@python.org**, письмо может быть отправлено десяткам и сотням людей, подписанным на рассылку, при этом их адреса не увидят остальные получатели, а заголовки даже не придется переписывать.

Читая следующее описание SMTP, помните, что заголовки и тело письма существуют отдельно от "отправителя на конверте" и "получателя на конверте", как описано в спецификации протокола. Когда мы используем `/usr/sbin/sendmail`, Thunderbird или GMail, мы вводим адрес получателя один раз, но затем он используется дважды: в заголовке To в верхней части письма и за пределами письма, когда SMTP получает запрос отправить письмо.

Несколько прыжков

Когда-то письма делали всего один "прыжок" от мейнфрейма, где они были созданы, до компьютера, на диске которого находился почтовый ящик получателя. Сегодня письма обычно проходят через несколько серверов, и в процессе у них несколько раз меняется конверт SMTP.

Давайте рассмотрим эту концепцию на примере. Некоторые детали мы выдумали, но они позволят вам получить представление о том, как письмо передается через Интернет в реальном мире.

Представьте себе сотрудника в крупной IT-компании, который говорит коллеге, что его адрес — **brandon@gatech.edu**. Когда коллега отправляет ему письмо, провайдер обращается к DNS (см. главу 4), получает серию записей MX в ответ и подключается к одному из IP-адресов для отправки письма. Просто, правда?

Однако сервер **gatech.edu** обслуживает весь кампус. Он обращается к таблице, чтобы найти, где находится **brandon**, находит кафедру и узнает, что официальный адрес на самом деле выглядит как **brandon.rhodes@oit.gatech.edu**, где **oit** обозначает факультет.

Сервер **gatech.edu** выполняет DNS-поиск, чтобы найти **oit.gatech.edu**, а затем отправляет письмо на почтовый сервер факультета через SMTP (второй прыжок). Однако факультет уже давно не хранит все письма на одном сервере UNIX. Теперь там работает сложная система, с которой можно взаимодействовать через веб-сервис, POP и IMAP.

Входящее письмо на **oit.gatech.edu** сначала проходит через один из серверов фильтрации спама (третий прыжок), например **spam3.oit.gatech.edu**. Затем, если оно проходит проверку на спам и не удаляется, оно случайным образом направляется на один из восьми почтовых серверов, и после четвертого прыжка оказывается в очереди **mail7.gatech.edu**. Серверы маршрутизации, например **mail7**, затем могут запрашивать центральную службу каталогов, чтобы узнать, в каком бэкенд-хранилище, подключенном к большому массиву дисков, размещается почтовый ящик этого пользователя. Итак, **mail7** выполняет LDAP-поиск пользователя **brandon** и определяет, что его ящик хранится на сервере **anvil.oit.gatech.edu**. Письмо доставляется на **anvil** и записывается в дисковый массив. Это пятый и последний прыжок SMTP.

Отправка и получение письма занимают как минимум несколько секунд, ведь крупные компании и интернет-провайдеры обычно используют несколько уровней серверов, через которые письмо должно пройти. Как отследить путь письма? Мы знаем, что протокол SMTP не читает заголовки, но понимает, куда нужно отправить письмо. При этом, как мы видели, место назначения может меняться с каждым прыжком. Почтовые серверы добавляют новые заголовки, чтобы отслеживать путь письма от отправителя к получателю.

Это заголовки **Received** (получено), и именно по ним системные администраторы пытаются определить проблему, когда почтовая система не работает. Попросите своего почтового клиента показать все заголовки любого электронного письма. Вы увидите все этапы, через которые прошло письмо на пути к получателю. Спамеры часто подделывают заголовки **Received**, чтобы казалось, что письмо исходит из законного источника. Наконец, когда последний сервер в цепочке успешно запишет письмо на физическое хранилище чьего-то ящика, создается заголовок **Delivered**.

Поскольку каждый сервер добавляет свой заголовок **Received** в верхнюю часть письма, мы экономим время, и каждому серверу не приходится проходить через все ранее написанные заголовки **Received**. Читать их нужно в обратном порядке: самый старый заголовок **Received** будет идти последним, так что путь письма можно восстановить, читая заголовки снизу вверх. Путь оказался длиннее или короче, чем вы ожидали?

Библиотека для работы с протоколом SMTP

В модуле `smtpplib` стандартной библиотеки Python есть встроенная реализация SMTP, которая позволяет работать с этим протоколом напрямую для выполнения простых задач.

Программы в примерах выше принимают несколько параметров командной строки, включая имя сервера SMTP, адрес отправителя и адреса получателей. Используйте эти возможности с осторожностью. Работайте только с сервером SMTP, который принадлежит вам или точно будет готов принять ваши тестовые сообщения, иначе ваш IP-адрес будет добавлен в черный список за спам.

Если вы не знаете, где найти такой сервер SMTP, установите почтовый демон локально, например `postfix` или `exim`, а затем свяжите его с `localhost`. В некоторых операционных системах UNIX, Linux и Mac OS X уже есть SMTP, который ожидает подключений от локальной рабочей станции.

В противном случае узнайте имя хоста и порт у сетевого администратора или интернет-провайдера.

Важно помнить, что нельзя выбрать почтовый сервер случайным образом — многие из них хранят или пересылают сообщения только от разрешенных клиентов.

Теперь можно переходить к листингу 13.1, который демонстрирует базовое приложение SMTP.

Листинг 13.1. Использование `smtpplib.sendmail()` для отправки сообщений

```
#!/usr/bin/env python3
# Programming in Python: The Basics

import sys, smtpplib

message_template = """To: {}
From: {}
Subject: Test Message from simple.py
Hello,
This is a test message sent to you from the simple.py program
in Programming in Python: The Basics.
"""

def main():
    if len(sys.argv) < 4:
        name = sys.argv[0]
        print("usage: {} server fromaddr toaddr [toaddr...]").format(name))
        sys.exit(2)
```

```

server, fromaddr, toaddrs = sys.argv[1], sys.argv[2], sys.argv[3:]
message = message_template.format(' ', '.join(toaddrs), fromaddr)

connection = smtplib.SMTP(server)
connection.sendmail(fromaddr, toaddrs, message)
connection.quit()

s = '' if len(toaddrs) == 1 else 's'
print("Message sent to {} recipient{}".format(len(toaddrs), s))

if __name__ == '__main__':
    main()

```

Здесь используется очень эффективная функция из стандартной библиотеки Python, так что программа устроена довольно просто. Сначала создается простое письмо с помощью аргументов командной строки (в *главе 12* описано, как создать более сложное письмо, которое содержит не только простой текст). Создается объект `smtplib.SMTP`, который подключается к выбранному серверу. Наконец, требуется вызов `sendmail()`. Если он выполняется успешно, почтовый сервер принял письмо без ошибок.

Как мы говорили ранее, на этом этапе информация о получателе письма существует независимо от его содержания. Программа создает заголовок `To` с адресами получателей письма, однако это всего лишь текст, он мог бы быть любым. (Другой вопрос, что благодаря этому тексту письмо будет доставлено получателю или удалено как спам.)

Если запустить эту программу в тестовой среде для этой книги, соединение будет выполнено успешно следующим образом:

```

mail.example.com sender@example.com recipient@example.com $ python3 simple.py
1 recipient received the message successfully.

```

Метод `sendmail()`, благодаря стараниям авторов стандартной библиотеки Python, может быть последним вызовом SMTP, который нам понадобится. Давайте посмотрим, как работает SMTP и что происходит за кулисами.

Обработка ошибок и отладка

При работе с `smtplib` мы можем встречать разные исключения:

- ◆ `socket.gaierror` — ошибки поиска адреса;
- ◆ `socket.error` — общие ошибки сети и взаимодействия;
- ◆ `socket.herror` — другие проблемы с адресацией.
- ◆ `smtplib.SMTPException` или его подкласс — проблемы взаимодействия с SMTP.

Первые три ошибки возникают в стеке TCP операционной системы как исключения в сетевом коде Python. Они передаются через модуль `smtpplib` в нашу программу. Все ошибки, которые затрагивают коммуникацию через SMTP, приведут к исключению `smtpplib.SMTPException`, если работает сокет TCP. Модуль `smtpplib` также позволяет получить подробные сообщения о процедурах, выполняемых при отправке письма. Следующий параметр дает возможность установить уровень детализации:

```
connection.set_debuglevel(1)
```

Это позволит обнаружить потенциальные проблемы. В листинге 13.2 приводится пример приложения с базовой обработкой ошибок и отладкой.

Листинг 13.2. Более продуманный клиент SMTP

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import sys, smtpplib, socket

message_template = """To: {}
From: {}
Subject: Test Message from simple.py Hello,
This is a test message sent to you from the debug.py program
in Programming in Python: The Basics.
"""

def main():
    if len(sys.argv) < 4:
        name = sys.argv[0]
        print("usage: {} server fromaddr toaddr [toaddr...]".format(name))
        sys.exit(2)

    server, fromaddr, toaddrs = sys.argv[1], sys.argv[2], sys.argv[3:]
    message = message_template.format(' '.join(toaddrs), fromaddr)

    try:
        connection = smtpplib.SMTP(server)
        connection.set_debuglevel(1)
        connection.sendmail(fromaddr, toaddrs, message)
    except (socket.gaierror, socket.error, socket.herror,
            smtpplib.SMTPException) as e:
        print("Your message may not have been sent!")
        print(e)
        sys.exit(1)
```

```

else:
    s = ' ' if len(toaddrs) == 1 else 's'
    print("Message sent to {} recipient{}".format(len(toaddrs), s))
    connection.quit()

if __name__ == '__main__':
    main()

```

Эта программа похожа на предыдущую, но выходные данные будут заметно отличаться, как видно в листинге 13.3.

Листинг 13.3. Данные для отладки из модуля smtplib

```

$ python3 debug.py mail.example.com sender@example.com recipient@example.com
send: 'hello [127.0.1.1]\r\n'
reply: b'250-guinness\r\n'
reply: b'250-SIZE 33444432\r\n'
reply: b'250 HELP\r\n'
reply: retcode (250); Msg: b'guinness\nSIZE 33554432\nHELP'
send: 'mail FROM:<sender@example.com> size=212\r\n'
reply: b'250 OK\r\n'
reply: retcode (250); Msg: b'OK'
send: 'rcpt TO:<recipient@example.com>\r\n' reply: b'250 OK\r\n'
reply: retcode (250); Msg: b'OK'
send: 'data\r\n'
reply: b'354 End data with <CR><LF>.<CR><LF>\r\n'
reply: retcode (354); Msg: b'End data with <CR><LF>.<CR><LF>'
data: (354, b'End data with <CR><LF>.<CR><LF>')
send: b'To: recipient@example.com\r\nFrom: sender@example.com\r\nSubject: Test
Message from
simple.py\r\n\r\nHello,\r\n\r\nThis is a test message sent to you from the debug.py
program\r\nin
Programming in Python: The Basics.\r\n.\r\n'
reply: b'250 OK\r\n'
reply: retcode (250); Msg: b'OK'
data: (250, b'OK')
send: 'quit\r\n'
reply: b'221 Bye\r\n'
reply: retcode (221); Msg: b'Bye'
Message sent to 1 recipient

```

В этом примере показано взаимодействие между smtplib и SMTP-сервером по сети.

Особенно эти детали актуальны, если наш код использует более сложные возможности SMTP.

Сначала клиент (библиотека `smtpplib`) отправляет команду `EHLO` с нашим именем хоста (это улучшенный вариант прежней команды `HELO`). Удаленный сервер отвечает, предоставляя свое имя хоста и список дополнительных возможностей SMTP.

Затем клиент выполняет команду `mail from`, в которой указан адрес отправителя на конверте и размер письма. На этом этапе сервер может отклонить письмо (например, если считает отправителя спамером), но в этом случае мы получаем `250 Ok`. (Здесь важен код `250`, потому что сообщение — предназначенный для пользователя текст, который зависит от сервера.)

Затем клиент отправляет команду `rcpt`, указывая получателя с конверта.

При использовании SMTP мы увидим, что письмо передается независимо от содержащегося в нем текста. В строке `rcpt to` будет указан каждый получатель, если мы отправляем письмо нескольким людям. Наконец, клиент отправляет `data`, передает само письмо (используя символы возврата каретки и перевода строки для обозначения конца строки) и завершает взаимодействие.

В этом примере `smtpplib` выполняет за нас все действия. Позже я объясню, как усилить контроль процесса, чтобы использовать расширенные возможности.

Не стоит думать, что если во время первого прыжка не обнаружены ошибки, письмо будет гарантированно доставлено. Почтовый сервер может принимать письмо, но затем не доставлять его. Перечитайте *разд. "Несколько прыжков"* ранее в этой главе и поймете, сколько раз письмо рискует потеряться в дороге.

EHLO для сбора информации

Полезно знать, какие письма удаленный сервер SMTP может принимать. Например, у большинства серверов SMTP есть ограничение по размеру письма, и если мы его не проверим, можем случайно отправить слишком большое письмо, которое будет отклонено.

При использовании исходной версии SMTP клиент отправляет инструкцию `HELO` на сервер в качестве первого приветствия. ESMTP, расширение протокола SMTP, поддерживает более надежное взаимодействие. `EHLO` начинает диалог от имени клиентов с поддержкой `EHLO` и сообщает серверу с поддержкой ESMTP, что он может предоставить в ответе больше информации. Максимальный размер письма и другие расширенные возможности SMTP, предлагаемые сервером, также входят в эту информацию.

Стоит перепроверить возвращаемый код. Не все серверы поддерживают ESMTP. `EHLO` вернет ошибку для таких серверов, и значит, нужно будет отправить команду `HELO`.

Поскольку в предыдущих примерах в коде `sendmail()` вызывается сразу после создания объекта SMTP, `smtpplib` автоматически отправляет серверу приветственное

сообщение, чтобы начать диалог от нашего имени. Если метод Python `sendmail()` определяет, что мы пытаемся отправить команду EHLO или HELO сами, он не будет отправлять приветственное сообщение. В листинге 13.4 приводится программа, которая узнает ограничения по размеру письма на сервере и возвращает ошибку перед отправкой слишком большого сообщения.

Листинг 13.4. Проверка ограничений по размеру письма

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import smtplib, socket, sys

message_template = """To: {}
From: {}
Subject: Test Message from simple.py Hello,
This is a test message sent to you from the ehlo.py program
in Programming in Python: The Basics."""

def main():
    if len(sys.argv) < 4:
        name = sys.argv[0]
        print("usage: {} server fromaddr toaddr [toaddr...].format(name))
        sys.exit(2)

    server, fromaddr, toaddrs = sys.argv[1], sys.argv[2], sys. argv[3:]
    message = message_template.format(' ', '.join(toaddrs), fromaddr)

    try:
        connection = smtplib.SMTP(server)
        report_on_message_size(connection, fromaddr, toaddrs, message)
    except (socket.gaierror, socket.error, socket.herror,
            smtplib.SMTPException) as e:
        print("Your message may not have been sent!")
        print(e)
        sys.exit(1)
    else:
        s = ' ' if len(toaddrs) == 1 else 's'
        print("Message sent to {} recipient{}".format(len(toaddrs), s))
        connection.quit()

def report_on_message_size(connection, fromaddr, toaddrs, message):
    code = connection.ehlo()[0]
```



```

uses_esmtp = (200 <= code <= 299)
if not uses_esmtp:
    code = connection.helo()[0]
    if not (200 <= code <= 299):
        print("Remote server refused HELO; code:", code)
        sys.exit(1)

if uses_esmtp and connection.has_extn('size'):
    print("Maximum message size is", connection.esmtp_features['size'])
    if len(message) > int(connection.esmtp_features['size']):
        print("Message too large; aborting.")
        sys.exit(1)

connection.sendmail(fromaddr, toaddrs, message)

if __name__ == '__main__':
    main()

```

Если мы запустим это приложение и удаленный сервер укажет максимальный размер письма, программа отобразит размер на экране и проверит, что письмо не превышает его, прежде чем отправить. (Если письмо небольшое, как у нас, такая проверка кажется странной, но в листинге просто приводится пример того, как это работает.) Пример запуска программы:

```
$ python3 ehlo.py mail.example.com sender@example.com recipient@example.com
```

```
Maximum message size is 33444432
```

```
Message successfully sent to 1 recipient
```

Изучите часть кода, в которой проверяются выходные данные от вызова `ehlo()` или `helo()`. В первой части эти две функции возвращают числовой код от удаленного сервера SMTP. Код от 200 до 299 указывает на успешное выполнение, остальные — на ошибку. Если мы получаем код успешного выполнения, сервер обработал сообщение.

Однако помните: если первый сервер SMTP принял сообщение, это еще не значит, что оно будет доставлено, ведь у последующего сервера могут быть установлены более строгие ограничения на размер.

Кроме размера доступна информация, предоставляемая ESMTP. Некоторые серверы, например, могут принимать данные в 8-битном формате, если поддерживают протокол 8BITMIME. Другие, как описано в следующем разделе, могут поддерживать шифрование. Изучите RFC 1869 или документацию по вашему серверу, чтобы узнать больше об ESMTP и его возможностях, которые могут зависеть от конкретного сервера.

SSL и TLS

Письма, отправленные через SMTP простым текстом, может прочитать любой человек с доступом к шлюзу или маршрутизатору, через который передаются пакеты, включая беспроводную сеть в кафе, к которой подключен почтовый клиент. Идеальное решение этой проблемы — зашифровать каждое письмо открытым ключом, а закрытый ключ будет только у отправителя. Например, можно использовать GNU Privacy Guard. Для шифрования и аутентификации отдельных взаимодействий по SMTP между парами компьютеров можно использовать протокол SSL/TLS, как описано в *главе 6*, независимо от того, защищены ли сами сообщения. В этом разделе мы поговорим о том, как SSL/TLS работает с соединениями SMTP.

Помните, что TLS защищает прыжки SMTP, только если для них настроено шифрование. Даже если мы используем TLS для отправки письма на сервер, мы не можем повлиять на то, использует ли сервер TLS для пересылки этого письма дальше.

Подход к использованию TLS в SMTP:

1. Создаем объект SMTP, как обычно.
2. С помощью команды EHLO отправляем письмо. TLS не будет поддерживаться, если удаленный сервер не поддерживает EHLO.
3. Проверяем, присутствует ли `starttls`, с помощью `s.has_extn()`. Если нет, удаленный сервер не поддерживает TLS и письмо можно отправить только открытым текстом.
4. Создаем объект контекста SSL, чтобы проверять подлинность сервера.
5. Для того чтобы открыть зашифрованный канал, вызываем `starttls()`.
6. Выполняем `ehlo()` вновь, на этот раз с шифрованием.
7. Наконец, отправляем письмо.

При работе с TLS первым делом нужно решить, стоит ли возвращать ошибку, если TLS недоступен. Обычно следует сообщать об ошибке в следующих сценариях, в зависимости от приложения:

- ◆ TLS не поддерживается на удаленном сервере;
- ◆ удаленный сервер не может корректно создать сеанс TLS или предоставляет сертификат, который невозможно проверить;
- ◆ давайте рассмотрим каждую ситуацию и решим, когда следует выдавать сообщение об ошибке.

Иногда нужно считать ошибкой отсутствие поддержки TLS. Допустим, если мы создаем приложение, которое взаимодействует только с небольшим количеством почтовых серверов — например, почтовые серверы, управляемые вашей компанией или учреждением, которые должны поддерживать TLS.

Поскольку TLS поддерживается небольшим процентом почтовых серверов, обычно почтовая программа не должна выдавать ошибку, если TLS не поддерживается. Если TLS доступен, многие клиенты SMTP с поддержкой TLS будут его использо-

вать, а если нет, данные будут передаваться в незашифрованном виде. Это так называемое оппортунистическое шифрование, и хотя это не так безопасно, как шифровать все соединения, таким образом мы шифруем письма хотя бы в тех случаях, когда это возможно.

Удаленный сервер может сообщить, что TLS доступен, но все равно не устанавливает успешное TLS-соединение. Часто это связано с некорректной конфигурацией сервера. Передачу на такие серверы следует повторять уже без попытки шифрования, чтобы письмо все-таки прошло. В некоторых случаях мы не можем проверить подлинность удаленного сервера. Валидация между серверами описывается в *главе 6*. Если наша политика безопасности требует отправлять письма только на доверенные серверы, невозможность аутентификации должна приводить к ошибке.

В листинге 13.5 приводится клиент общего назначения, который поддерживает TLS. Если TLS доступен, он будет использоваться для подключения к серверу. В противном случае письмо будет доставлено открытым текстом. Если мы пытаемся установить TLS-соединение с сервером, который заявляет, что поддерживает его, но попытка не удалась, мы получим ошибку.

Листинг 13.5. Использование TLS

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import sys, smtplib, socket, ssl

message_template = """To: {}
From: {}
Subject: Test Message from simple.py Hello,
This is a test message sent to you from the tls.py program
in Programming in Python: The Basics.
"""

def main():
    if len(sys.argv) < 4:
        name = sys.argv[0]
        print("Syntax: {} server fromaddr toaddr [toaddr...].format(name))
        sys.exit(2)

    server, fromaddr, toaddrs = sys.argv[1], sys.argv[2], sys.argv[3:]
    message = message_template.format(' ', ' '.join(toaddrs), fromaddr)
    try:
        connection = smtplib.SMTP(server)
        send_message_securely(connection, fromaddr, toaddrs, message)
```

```

except (socket.gaierror, socket.error, socket.herror,
        smtpplib.SMTPException) as e:
    print("Your message may not have been sent!")
    print(e)
    sys.exit(1)
else:
    s = '' if len(toaddrs) == 1 else 's'
    print("Message sent to {} recipient{}".format(len(toaddrs), s))
    connection.quit()

def send_message_securely(connection, fromaddr, toaddrs, message):
    code = connection.ehlo()[0]
    uses_esmtp = (200 <= code <= 299)
    if not uses_esmtp:
        code = connection.helo()[0]
        if not (200 <= code <= 299):
            print("Remove server refused HELO; code:", code)
            sys.exit(1)

    if uses_esmtp and connection.has_extn('starttls'):
        print("Negotiating TLS...")
        context = ssl.SSLContext(ssl.PROTOCOL_SSLv23)
        context.set_default_verify_paths()
        context.verify_mode = ssl.CERT_REQUIRED
        connection.starttls(context=context)
        code = connection.ehlo()[0]
        if not (200 <= code <= 299):
            print("Couldn't EHLO after STARTTLS")
            sys.exit(5)
        print("Using TLS connection.")
    else:
        print("Server does not support TLS; using normal connection.")

    connection.sendmail(fromaddr, toaddrs, message)

if __name__ == '__main__':
    main()

```

Стоит отметить, что вызов `sendmail()` в последних нескольких листингах остается одинаковым независимо от того, включен ли TLS. TLS скрывает этот уровень сложности после включения.

Аутентификация SMTP

Наконец, SMTP может требовать аутентификации, т. е. почтовый сервер провайдера или компании требует, чтобы вы указали имя пользователя и пароль и подтвердили, что не являетесь спамером.

TLS следует использовать вместе с аутентификацией для максимальной безопасности. В противном случае любой человек, который видит соединение, увидит и ваше имя пользователя и пароль. Следует сначала установить TLS-соединение, а затем отправить информацию для аутентификации по зашифрованному каналу. Аутентификация выполняется очень просто: метод `login()` в `smtplib` принимает имя пользователя и пароль.

Давайте посмотрим пример в листинге 13.6. Для того чтобы не повторять код, который вы уже видели в предыдущих листингах, этот пример игнорирует предупреждение из предыдущего абзаца и отправляет имя пользователя и пароль открытым текстом.

Листинг 13.6. Аутентификация через SMTP

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import sys, smtplib, socket
from getpass import getpass

message_template = """To: {}
From: {}
Subject: Test Message from simple.py
Hello,
This is a test message sent to you from the login.py program
in Programming in Python: The Basics.
"""

def main():
    if len(sys.argv) < 4:
        name = sys.argv[0]
        print("Syntax: {} server fromaddr toaddr [toaddr...].format(name))
        sys.exit(2)

    server, fromaddr, toaddrs = sys.argv[1], sys.argv[2], sys.argv[3:]
    message = message_template.format(' ', '.join(toaddrs), fromaddr)

    username = input("Enter username: ")
    password = getpass("Enter password: ")
```

```

try:
    connection = smtplib.SMTP(server)
    try:
        connection.login(username, password)
    except smtplib.SMTPException as e:
        print("Authentication failed:", e)
        sys.exit(1)
    connection.sendmail(fromaddr, toaddrs, message)
except (socket.gaierror, socket.error, socket.herror,
        smtplib.SMTPException) as e:
    print("Your message may not have been sent!")
    print(e)
    sys.exit(1)
else:
    s = '' if len(toaddrs) == 1 else 's'
    print("Message sent to {} recipient{}".format(len(toaddrs), s))
    connection.quit()

if __name__ == '__main__':
    main()

```

Аутентификация не поддерживается большинством исходящих почтовых серверов, и в этом случае попытка вызвать метод `login()` вернет ошибку сбой аутентификации. Для того чтобы этого избежать, мы можем проверить `connection.has_extn('auth')`, если удаленный сервер поддерживает ESMTP.

Эту программу можно выполнить так же, как предыдущие. Нужно будет ввести имя пользователя и пароль, если она выполняется на сервере, который поддерживает аутентификацию. Если они будут приняты, программа отправит письмо получателю.

Советы по SMTP

Несколько советов по началу работы с SMTP:

- ◆ Мы не можем быть уверены, что письмо дойдет до получателя. Хотя мы сразу можем увидеть сбой, отсутствие сообщения об ошибке еще не означает, что дальше все пойдет так же гладко.
- ◆ Если у одного из получателей произошел сбой, функция `sendmail()` выдает исключение, а письмо можно отправить остальным получателям. Подробности можно найти в самом исключении. Возможно, потребуется выполнить `sendmail()` отдельно для каждого получателя, чтобы знать, для какого адреса произошел сбой. Например, чтобы попробовать отправить письмо позже только ему, а не

остальным получателям. Правда, при таком подходе тело письма будет отправлено несколько раз, по одному для каждого адресата.

- ◆ Без проверки сертификата SSL/TLS не обеспечивает безопасность: мы можем общаться со старым сервером, который временно находится по IP-адресу другого сервера.
- ◆ Не забудьте установить контекст объекта SSL, как описано в предыдущем примере TLS, и передать его как аргумент для `starttls()`, чтобы поддерживать проверку сертификата.
- ◆ Модуль `smtplib` в Python не должен использоваться как универсальный инструмент для передачи почты. Он предназначен для отправки писем на локальный сервер SMTP, который и будет отвечать за доставку почты.

Резюме

SMTP — это протокол для отправки электронных писем на почтовые серверы. Для SMTP-клиентов Python предоставляет модуль `smtplib`. Мы можем отправлять письма с помощью метода `sendmail()` объектов SMTP. Заголовки To, Cc и Bcc в тексте письма существуют отдельно от реального списка получателей. Во время SMTP-соединения могут возникать разные исключения. Интерактивные программы должны эффективно обрабатывать эти исключения.

ESMTP — это расширение для SMTP. До отправки письма мы с его помощью можем определить максимальный размер письма, который поддерживается сервером SMTP. ESMTP также поддерживает TLS, метод шифрования взаимодействия с удаленным сервером (см. главу 6).

Некоторые серверы SMTP требуют аутентификацию. Для этого можно использовать метод `login()`. У SMTP нет возможности загружать письма из почтового ящика на ваш компьютер — за это отвечают протоколы, которые мы рассмотрим в следующих двух главах. В главе 14 мы рассмотрим POP — простой метод получения и загрузки писем.

Глава 15 будет посвящена более сложному и функциональному протоколу IMAP.

Протокол POP

POP (Post Office Protocol — протокол почтового отделения) — это простой метод извлечения электронной почты с сервера. Обычно он реализуется через почтовый клиент вроде Thunderbird или Outlook. В *главе 13* кратко описано место почтовых клиентов и протоколов вроде POP в истории развития электронной почты.

Если вы вдруг захотите использовать POP вместо IMAP; прочтите *главу 15*, где подробно описываются преимущества IMAP над этим довольно примитивным протоколом.

Самая распространенная реализация POP — версия 3, которую также называют POP3. Поскольку версия 3 так популярна, под POP обычно подразумевают POP3. Простота POP — главное преимущество и главный недостаток этого протокола. Если нам нужно просто получить доступ к удаленному почтовому ящику, загрузить новые письма и удалить письмо после загрузки, протокол POP отлично подойдет для этого. Он сделает все быстро и без сложного кода.

Обычно он используется для загрузки и удаления файлов. Он не поддерживает несколько почтовых ящиков и не предоставляет надежную идентификацию сообщений. Это значит, что мы не можем использовать POP для синхронизации почты, чтобы хранить на сервере исходную копию письма и загружать на компьютер локальную копию для чтения, поскольку мы не можем определить, какие сообщения мы уже загрузили.

Если мы хотим синхронизировать почту, нам поможет протокол IMAP.

Модуль `poplib` в стандартной библиотеке Python предоставляет удобный интерфейс для использования POP.

Правда, он предназначен для создания клиента, а не сервера. Если вы хотите создать сервер, поищите подходящий модуль Python.

Содержание главы

- ◆ Серверы POP и стандарты.
- ◆ Аутентификация и подключение.
- ◆ Получение информации о почтовом ящике.

- ◆ Загрузка и удаление писем.
- ◆ Резюме.

Цель

В этой главе вы узнаете, как подключаться к серверу POP, получать информацию о почтовом ящике, загружать письма и удалять их оригиналы с сервера с помощью модуля `poplib`.

Серверы POP и стандарты

Серверы POP имеют ужасную репутацию, потому что не соответствуют стандартам. Для некоторых действий POP стандартов просто не существует, так что реализация остается на совести разработчиков сервера. В большинстве случаев базовые операции должны работать корректно, но детали зависят от сервера.

Некоторые серверы, например, помечают все письма как прочитанные при каждом подключении к серверу, даже если на самом деле мы их не загружаем. Другие серверы отмечают письмо как прочитанное, если мы его загрузили. Какие-то серверы никогда не помечают письма как прочитанные. Стандарт вроде бы поддерживает последнее поведение, но это не точно. Помните об этих различиях, читая эту главу.

Аутентификация и подключение

POP поддерживает различные механизмы аутентификации. Самые популярные из них — базовая аутентификация с именем пользователя и паролем и APOP, расширение POP, которое помогает защищать пароли от отправки открытым текстом, если мы используем старый сервер POP, не поддерживающий SSL.

В Python подключение к удаленному серверу и аутентификация выглядят следующим образом:

1. Мы создаем объект `POP3_SSL` или стандартный объект `POP3` и передаем ему имя хоста и порт.
2. Для отправки имени пользователя и пароля мы используем функции `user()` и `pass_()`. Не забудьте символ подчеркивания в `pass_()`. В Python есть ключевое слово `pass`, поэтому мы не можем использовать в качестве имени метода.
3. Если выдается исключение `poplib.error_proto`, попытка входа провалилась, и строковое значение исключения содержит сообщение об ошибке сервера.

Выбор между `POP3` и `POP3_SSL` зависит от того, разрешает ли провайдер (или даже требует) подключение по зашифрованному соединению, хотя обычно SSL следует использовать всегда, когда это возможно (см. главу 6). Методы в листинге 14.1 применяются для входа на удаленный сервер POP. Код подключается к серверу и

вызывает метод `stat()`, который возвращает базовый кортеж с указанием количества писем в почтовом ящике и их общего размера. Наконец, программа вызывает `quit()`, чтобы завершить POP-соединение.

ПРОТОКОЛ POP3

Цель: загружать письма из почтового ящика.

Стандарт: RFC 1939 (май 1996 г.).

Базовый протокол: TCP/IP.

Порт по умолчанию: 110 (cleartext), 995 (SSL).

Библиотеки: poplib.

Листинг 14.1. Простейший сеанс POP

```
import getpass, poplib, sys

def main():
    if len(sys.argv) != 3:
        print('usage: %s hostname username' % sys.argv[0])
        exit(2)

    hostname, username = sys.argv[1:]
    passwd = getpass.getpass()

    p = poplib.POP3_SSL(hostname) # или "POP3" если SSL не поддерживается
    try:
        p.user(username)
        p.pass_(passwd)
    except poplib.error_proto as e:
        print("Login failed:", e)
    else:
        status = p.stat()
        print("You have %d messages totaling %d bytes" % status)
    finally:
        p.quit()

if __name__ == '__main__':
    main()
```

Программа никак не меняет сообщения, но некоторые серверы POP меняют флаги в почтовом ящике просто потому, что мы подключились к нему. Если выполнять

примеры из этой главы с реальным почтовым ящиком, мы не сможем отличать прочитанные письма от непрочитанных и новые от старых. К сожалению, это поведение зависит от сервера, клиенты POP не могут это контролировать. Настоятельно рекомендую запускать эти примеры на тестовых почтовых ящиках, а не на своем.

В командной строке необходимо указать имя хоста сервера POP и имя пользователя. Если вы их не знаете, спросите у своего провайдера или сетевого администратора. Стоит отметить, что в некоторых сервисах имя пользователя может быть простой строкой (например, `guido`), а в других нужно указать полный адрес (`guido@example.com`). Затем у нас попросят ввести пароль. Наконец, мы увидим состояние почтового ящика, но это никак не затронет письма в нем.

Мы можем использовать среду Mininet:

```
$ python3 popconn.py mail.example.com bpbonline Password: abc12345
```

```
You have 3 messages totaling 5660 bytes
```

Если вы получили подобные выходные данные, вам удалось наладить взаимодействие с помощью POP.

Когда серверы POP не поддерживают SSL для защиты соединений, они могут поддерживать хотя бы APOP, который использует механизм запроса-ответа, чтобы пароль не приходилось отправлять простым текстом. (Хотя третья сторона, которая видит пакеты, сможет просматривать нашу электронную почту.) Стандартная библиотека Python значительно упрощает нам задачу: мы просто вызываем метод `apop()`, и если сервер POP, с которым мы взаимодействуем, нас не понимает, мы должны использовать базовую аутентификацию.

Используйте строку конфигурации, как в листинге 14.2, в программе POP, чтобы применять APOP по умолчанию, а базовую аутентификацию — в случае недоступности APOP.

Листинг 14.2. Попытка использовать APOP и переход на другой вариант

```
print("Attempting APOP authentication...")
try:
    p.apop(user, passwd)
except poplib.error_proto:
    print("Attempting standard authentication...")
    try:
        p.user(user)
        p.pass_(passwd)
    except poplib.error_proto as e:
        print("Login failed:", e)
        sys.exit(1)
```

Некоторые старые серверы POP могут блокировать почтовый ящик после удачного входа. Если почтовый ящик заблокирован, в нем нельзя вносить изменения, и он не

может принимать письма, пока блокировка не будет снята. Проблема в том, что некоторые серверы POP не могут корректно определять проблемы, и если сообщение зависает, не вызывая `quit()`, ящик остается заблокированным. Так вел себя даже самый популярный сервер POP. Поэтому в приложениях Python следует всегда вызывать `quit()` при завершении сеанса. Все листинги в этой главе вызывают `quit()` в последнем блоке.

Получение информации о почтовом ящике

Функция `stat()` возвращает номера писем в почтовом ящике и их общий размер.

Метод `list()` возвращает более подробную информацию о каждом письме.

Нас интересует номер письма, по которому к нему можно будет обращаться позже. Номера сообщений могут идти не подряд. Например, это может быть 1, 2, 5, 6 и 9. Кроме того, номер письма может меняться в каждом соединении с сервером POP.

Команда `list()` в листинге 14.3 отображает информацию о каждом сообщении.

Листинг 14.3. Использование метода POP `list()`

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import getpass, poplib, sys

def main():
    if len(sys.argv) != 3:
        print('usage: %s hostname username' % sys.argv[0])
        exit(2)

    hostname, username = sys.argv[1:]
    passwd = getpass.getpass()

    p = poplib.POP3_SSL(hostname)
    try:
        p.user(username)
        p.pass_(passwd)
    except poplib.error_proto as e:
        print("Login failed:", e)
    else:
        response, listings, octet_count = p.list()
        if not listings:
            print("No messages")
```

```

    for listing in listings:
        number, size = listing.decode('ascii').split()
        print("Message %s has %s bytes" % (number, size))
    finally:
        p.quit()

if __name__ == '__main__':
    main()

```

Функция `list()` возвращает кортеж из трех элементов. Нам важен второй из них. Вот пример выходных данных для реального почтового ящика с тремя письмами:

```
('+OK 3 messages (5676 bytes)', ['1 2405', '2 1625', '3 1664'], 25)
```

Для каждого из трех писем три строки во втором элементе указывают номер и размер письма. В листинге 14.3 используется простой парсинг, чтобы отобразить эту информацию в более понятном виде. Вот как можно выполнить эту программу для сервера POP, который мы развернули в тестовой среде этой книги (*см. главу 1*):

```
$ python3 mailbox.py mail.example.com bpbonline
```

```
Password: abc12345
```

```
Message 1 has 355 bytes
```

```
Message 2 has 446 bytes
```

```
Message 3 has 1183 bytes
```

Загрузка и удаление писем

При использовании библиотеки `poplib` мы отправляем простые команды, которые всегда возвращают кортеж с разными строками и списками строк. Мы можем работать с письмами, обозначая их целочисленным идентификатором, возвращенным методом `list()`. Нам доступны три метода.

- ◆ `getr(идентификатор)`. Эта функция загружает простое письмо и возвращает кортеж с кодом результата и текстом письма в виде списка строк. Большинство серверов POP устанавливают для флага `seen` письма значение `true`, тем самым помечая его как прочитанное, чтобы мы не читали его снова через POP (если только почтовый ящик не предоставляет способ снова отметить письмо как непрочитанное).
- ◆ `top(идентификатор, число строк)`. Этот метод дает те же результаты, что и `getr()`, но не помечает сообщение как прочитанное. Вместо того чтобы отображать сообщение целиком, он показывает заголовки и указанное число строк тела. Так пользователи могут решить, стоит ли загружать письмо.
- ◆ `delete(идентификатор)`. Этот метод помечает письмо на удаление с сервера POP после завершения сеанса. Поскольку мы больше не сможем извлечь это письмо с сервера, используйте этот метод только в том случае, если пользователь явно за-

прашивает необратимое удаление письма или если мы сделали его резервную копию и убедились, что данные действительно записаны, с помощью функции `fsync()`.

В листинге 14.4 приводится почтовый клиент, использующий протокол POP. Он проверяет почтовый ящик, чтобы узнать, сколько у нас писем и какие у них номера, а затем с помощью `top()` показывает предварительный просмотр каждого письма. По запросу пользователя он может извлечь все письмо и удалить его из почтового ящика.

Листинг 14.4. Простой почтовый клиент, который читает сообщения, используя POP

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import email, getpass, poplib, sys

def main():
    if len(sys.argv) != 3:
        print('usage: %s hostname username' % sys.argv[0])
        exit(2)

    hostname, username = sys.argv[1:]
    passwd = getpass.getpass()

    p = poplib.POP3_SSL(hostname)
    try:
        p.user(username)
        p.pass_(passwd)
    except poplib.error_proto as e:
        print("Login failed:", e)
    else:
        visit_all_listings(p)
    finally:
        p.quit()

def visit_all_listings(p):
    response, listings, octets = p.list()
    for listing in listings:
        visit_listing(p, listing)

def visit_listing(p, listing):
    number, size = listing.decode('ascii').split()
```

```

print('Message', number, '(size is', size, 'bytes):')
print()
response, lines, octets = p.top(number, 0)
document = '\n'.join( line.decode('ascii') for line in lines )
message = email.message_from_string(document)
for header in 'From', 'To', 'Subject', 'Date':
    if header in message:
        print(header + ':', message[header])
print()
print('Read this message [ny]?')
answer = input()
if answer.lower().startswith('y'):
    response, lines, octets = p.retr(number)
    document = '\n'.join( line.decode('ascii') for line in lines )
    message = email.message_from_string(document)
    print('-' * 72)
    for part in message.walk():
        if part.get_content_type() == 'text/plain':
            print(part.get_payload())
            print('-' * 72)
print()
print('Delete this message [ny]?')
answer = input()
if answer.lower().startswith('y'):
    p.dele(number)
    print('Deleted.')

if __name__ == '__main__':
    main()

```

Как видите, в этом листинге активно используется модуль `email`, с которым мы познакомились в *главе 12*, потому что даже современные письма MIME с HTML и графикой обычно включают компонент `text/plain`, с которым работает модуль `email`. Если выполнить эту программу в тестовой среде, мы получим следующий результат:

\$ python3 download-and-delete.py mail.example.com bpbonline

password: abc12345

Message 1 (size is 356 bytes):

From: Administrator <admin@mail.example.com>

To: Bpbonline <bpbonline@mail.example.com>

Subject: Welcome to example.com!

Read this message [ny]? y

We are happy that you have chosen to use example.com's industry- leading Internet e-mail service and we hope that you experience is a pleasant one. If you ever need your password reset, simply contact our staff! - example.com

Delete this message [ny]? y

Deleted.

Резюме

С помощью POP можно легко загружать электронные письма с удаленного сервера. Мы можем получить информацию о номере и размере каждого письма с помощью модуля poplib в Python. По этим номерам можно извлекать и удалять отдельные сообщения.

При подключении к серверу POP почтовый ящик может блокироваться, поэтому важно, чтобы сеансы POP были максимально короткими и обязательно завершались методом quit().

Для того чтобы защитить пароли и содержимое электронных писем, по возможности POP следует использовать с SSL.

Если SSL недоступен, как минимум следует использовать APOP. Отправляйте пароль открытым текстом только в том случае, если вам действительно необходимо использовать POP, а альтернатив нет. POP — это простой и популярный протокол со своими недостатками, которые делают его непригодным для некоторых целей. Например, он может работать только с одной папкой и не присваивает сообщениям постоянные идентификаторы.

В следующей главе мы рассмотрим IMAP — протокол, который превосходит POP по своим возможностям.

Протокол IMAP

IMAP (Internet Message Access Protocol — протокол доступа к интернет-сообщениям) похож на протокол POP, который мы рассматривали в предыдущей главе.

Более того, в *главе 13* мы говорили, что оба протокола служат одной цели: соединяют устройство пользователя с удаленным интернет-сервером для управления почтой.

На этом их сходство заканчивается. В отличие от POP, который используется для загрузки новых писем на компьютер, протокол IMAP позволяет сортировать и архивировать письма на сервере, чтобы экономить место на устройстве.

У IMAP очень много преимуществ по сравнению с POP.

- ◆ Письма можно разложить по папкам.
- ◆ Для каждого письма можно устанавливать флаги, например "прочитано", "ответчено", "просмотрено", "удалено".
- ◆ Можно искать в письмах текстовые строки, не загружая их.
- ◆ Письмо из локального хранилища можно легко загружать в удаленные папки.
- ◆ У писем есть постоянные уникальные идентификаторы, что позволяет синхронизировать сообщения в локальном хранилище и на сервере.
- ◆ Пользователи могут делиться папками с другими или предоставлять их только для чтения.
- ◆ Некоторые серверы IMAP могут отображать как папки с письмами не только почту, но и, например, новостные группы Usenet.
- ◆ Клиент IMAP может загружать часть письма, например определенное вложение или только заголовки, не ожидая загрузки остального письма.

Как видите, у IMAP гораздо больше возможностей, чем у POP. Многие почтовые клиенты, например Thunderbird и Outlook, могут отображать папки IMAP и работать с ними так же, как с локальными. Почтовый клиент загружает письмо с сервера IMAP не заранее, а только когда пользователь нажимает на это письмо. Одновременно с этим для письма можно установить флаг "прочитано".

ПРОТОКОЛ IMAP

Цель: читать, сортировать и удалять письма из папок.

Стандарт: RFC 3501 (2003 г.).

Базовый протокол: TCP/IP.

Порт по умолчанию: 143 (cleartext), 993 (SSL).

Библиотеки: imaplib, IMAPClient.

Исключения: socket.error, socket.gaierror,
IMAP4.error, IMAP4.abort, IMAP4.readonly.

Клиенты IMAP тоже могут синхронизироваться с сервером IMAP. Можно, например, загрузить папку IMAP, отправляясь в командировку, а затем в дороге просматривать и удалять письма или отвечать на них. Когда ноутбук снова подключится к сети, почтовый клиент расставит флаги "прочитано" и "отвечено", а также удалит с сервера нужные сообщения.

Это одно из главных преимуществ IMAP над POP — почтовый ящик пользователя будет выглядеть одинаково с любого устройства. Если почтовый клиент POP удаляет письма после загрузки, письма пользователя будут разбросаны по всем устройствам, с которых он проверяет почту. У пользователей IMAP такой проблемы не возникает.

Конечно, IMAP можно использовать как POP — загружать письма и сохранять их локально. Если расширенные возможности нам не нужны, мы можем сразу после этого удалять письмо с сервера.

Протокол IMAP доступен в нескольких версиях. Текущая версия — IMAP4rev1, и обычно именно эту версию мы имеем в виду, когда говорим про IMAP. В этой главе все серверы используют IMAP4rev1. Некоторые из возможностей, описанных здесь, могут не поддерживаться очень старыми серверами IMAP, но они встречаются крайне редко.

Пара хороших источников с инструкциями по написанию клиента IMAP:

www.dovecot.org/imap-client-coding-howto.html

www.imapwiki.org/ClientImplementation

Если вы планируете создать что-то более сложное, чем небольшой клиент, который только показывает входящие письма или автоматически загружает вложения, внимательно изучите эти ресурсы или даже прочтите книгу об IMAP, чтобы учесть все варианты серверов и реализаций протокола.

Содержание главы

- ◆ Реализация IMAP в Python.
- ◆ Клиент IMAP.

- ◆ Просмотр папок.
- ◆ UID и номера писем.
- ◆ Интервалы между письмами.
- ◆ Общая информация.
- ◆ Получение всего почтового ящика.
- ◆ Загрузка отдельных писем.
- ◆ Добавление и удаление флагов.
- ◆ Удаление писем.
- ◆ Поиск.
- ◆ Работа с папками.
- ◆ Асинхронность.
- ◆ Резюме.

Цель

В этой главе мы рассмотрим только основные принципы работы протокола с упором на его реализацию на Python.

Реализация IMAP в Python

Для работы с IMAP в стандартной библиотеке Python есть модуль `imaplib`. Правда, он умеет только отправлять запросы и получать ответы на них и не может реализовывать требования IMAP по парсингу возвращенных данных. В листинге 15.1 мы видим, что `imaplib` выдает информацию в слишком неудобном формате. Это простой скрипт, который подключается к аккаунту IMAP с помощью `imaplib`, перечисляет все возможности, заявленные сервером, а затем отображает код состояния и данные, возвращенные командой `LIST`.

Листинг 15.1. IMAP-соединение и вывод папок

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Открываем IMAP-соединение с помощью стандартной библиотеки.

import getpass, imaplib, sys

def main():
    if len(sys.argv) != 3:
        print('usage: %s hostname username' % sys.argv[0])
        sys.exit(2)
```

```

hostname, username = sys.argv[1:]
m = imaplib.IMAP4_SSL(hostname)
m.login(username, getpass.getpass())
try:
    print('Capabilities:', m.capabilities)
    print('Listing mailboxes ')
    status, data = m.list()
    print('Status:', repr(status))
    print('Data:')
    for datum in data:
        print(repr(datum))
finally:
    m.logout()

if __name__ == '__main__':
    main()

```

При запуске этого скрипта с правильными аргументами у нас запрашивают пароль. Для аутентификации IMAP почти всегда требуются имя пользователя и пароль:

```
open imaplib.py $ python imap.example.com bpbonline@example.com Password:
```

Если мы указали правильный пароль, мы получим ответ, похожий на листинг 15.2. Сначала идет раздел capabilities с возможностями IMAP, которые поддерживает этот сервер. Как видите, формат этого списка соответствует принципам Python: как бы ни выглядел этот список в реальности, мы видим удобный кортеж из строк.

Листинг 15.2. Пример выходных данных предыдущего листинга.

```
Capabilities: ('IMAP4REV1', 'UNSELECT', 'IDLE', 'NAMESPACE', 'QUOTA', 'XLIST',
'CHILDREN', 'XYZZY', 'SASL-IR', 'AUTH=XOAUTH')
```

```
Listing mailboxes
```

```
Status: 'OK'
```

```
Data:
```

```
b'(\HasNoChildren) "/" "INBOX" b'(\HasNoChildren) "/" "Personal"
```

```
b'(\HasNoChildren) "/" "Receipts"
```

```
b'(\HasNoChildren) "/" "Travel"
```

```
b'(\HasNoChildren) "/" "Work"
```

```
b'(\Noselect \HasChildren) "/" "[Gmail]"
```

```
b'(\HasChildren \HasNoChildren) "/" "[Gmail]/All Mail"
```

```
b'(\HasNoChildren) "/" "[Gmail]/Drafts"
```

```
b'(\HasChildren \HasNoChildren) "/" "[Gmail]/Sent Mail"
```

```
b'(\HasNoChildren) "/" "[Gmail]/Spam"
```

```
b'(\HasNoChildren) "/" "[Gmail]/Starred"
```

```
b'(\HasChildren \HasNoChildren) "/" "[Gmail]/Trash"
```

Когда мы доходим до выходных данных метода `list()`, все идет не так. Во-первых, возвращается код состояния в виде строки `OK`, так что коду, который использует `imaplib`, придется постоянно проверять, что мы получили: `OK` или ошибку. Это не в духе Python, потому что программы Python обычно могут выполняться без проверки ошибок: если ошибка возникнет, будет выдано исключение.

Во-вторых, `imaplib` не помогает расшифровать результаты. Список папок в аккаунте IMAP подчиняется структуре, требуемой протоколом: каждый элемент в списке указывает флаги папки, затем указывается символ, который разделяет папки и подпапки (в нашем случае это косая черта), а затем имя папки в кавычках. Однако затем все это преобразуется обратно в необработанные данные, и нам приходится расшифровывать подобные строки:

```
"/" "[Gmail]/Sent Mail" (\HasChildren\HasNoChildren)
```

В-третьих, мы получаем смешение отдельных последовательностей: флаги остаются не переработанными байтовыми строками, но каждый разделитель и имя папки преобразуются в строку символов Unicode.

Если мы не планируем реализовывать различные элементы протокола самостоятельно, придется найти более сложную клиентскую библиотеку IMAP.

Клиент IMAP

К счастью, существует популярная и надежная библиотека IMAP для Python, которую можно легко установить с помощью Python Package Index. Пакет `IMAPClient`, созданный Менно Смитсом (Menno Smits), использует модуль `imaplib` из стандартной библиотеки.

Установите `IMAPClient` в виртуальном окружении, как описано в *главе 1*, и поэкспериментируйте. После установки выполните программу из листинга 15.3 в интерпретаторе в виртуальном окружении.

Листинг 15.3. Просмотр папок IMAP с помощью `IMAPClient`

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Открываем IMAP-соединение с помощью IMAPClient

import getpass, sys
from imapclient import IMAPClient

def main():
    if len(sys.argv) != 3:
        print('usage: %s hostname username' % sys.argv[0])
        sys.exit(2)
```

```

hostname, username = sys.argv[1:]
c = IMAPClient(hostname, ssl=True)
try:
    c.login(username, getpass.getpass())
except c.Error as e:
    print('Could not log in:', e)
else:
    print('Capabilities:', c.capabilities())
    print('Listing mailboxes:')
    data = c.list_folders()
    for flags, delimiter, folder_name in data:
        print(' %-30s%s %s' % (' '.join(flags), delimiter, folder_name))
finally:
    c.logout()

if __name__ == '__main__':
    main()

```

Эта библиотека берет на себя детали работы с протоколом. Например, мы не получаем код состояния, который приходится проверять при каждом выполнении команды. Теперь библиотека выполняет эту проверку и выдает исключение, если что-то пошло не так.

Во-вторых, каждый результат команды LIST, которая в этой библиотеке выглядит как метод `list_folders()`, а не `list()`, как в `imaplib`, уже разложен на типы данных Python. Каждая строка данных возвращается как кортеж, который содержит флаги папки, разделитель, имя папки и сами флаги в виде строковой последовательности.

В листинге 15.4 мы видим, что выдает второй скрипт.

Листинг 15.4. Корректный парсинг флагов и имен папок

```

Capabilities: ('IMAP4REV1', 'UNSELECT', 'IDLE', 'NAMESPACE', 'QUOTA', 'XLIST',
'CHILDREN', 'XYZZY', 'SASL-IR', 'AUTH=XOAUTH')

```

```

Listing mailboxes:

```

```

\HasNoChildren / INBOX
\HasNoChildren / Personal
\HasNoChildren / Receipts
\HasNoChildren / Travel
\HasNoChildren / Work
\Noselect \HasChildren / [Gmail]
\HasChildren \HasNoChildren / [Gmail]/All Mail
\HasNoChildren / [Gmail]/Drafts
\HasChildren \HasNoChildren / [Gmail]/Sent Mail

```

```
\HasNoChildren / [Gmail]/Spam
\HasNoChildren / [Gmail]/Starred
\HasChildren \HasNoChildren / [Gmail]/Trash
```

Стандартные флаги для папок:

- ◆ **NoInferiors** — папка не содержит и никогда не будет содержать подпапки. Если мы попробуем создать подпапку в этой папке через клиента IMAP, то получим ошибку;
- ◆ **NoSelect** — нельзя использовать метод `select_folder()` с этой папкой, т. е. папка не содержит и не может содержать писем (она только вмещает подпапки);
- ◆ **Marked** — сервер почему-то пометил эту папку. Обычно это означает, что в папке есть новые письма. Однако если отсутствует флаг **Marked**, это еще не значит, что новых писем нет, просто некоторые серверы не поддерживают этот флаг;
- ◆ **Unmarked** — в папке нет новых сообщений.

У серверов могут быть нестандартные флаги, которые наш код может принимать или игнорировать.

Просмотр папок

Мы должны выбрать папку, прежде чем можно будет загружать, искать или менять сообщения. Это означает, что протокол IMAP отслеживает состояние: он помнит, в какой папке мы находимся, и выполняет операции в этой папке, не заставляя нас повторять ее имя. Для того чтобы начать с начала, нужно разорвать соединение и подключиться заново. Так удобнее работать, но наша программа должна всегда следить за тем, какая папка открыта, иначе мы можем выполнять операции в неверной папке. При выборе папки мы говорим серверу IMAP, что все последующие инструкции будут применяться к этой папке, пока мы не выйдем из этой папки или не перейдем в другую.

С помощью аргумента `readonly=True` можно открыть папку в режиме "только для чтения". Если мы попытаемся удалить или отредактировать письмо, мы получим сообщение об ошибке. Если мы планируем только читать письма, сервер может оптимизировать доступ к папке и защитить ее содержимое от случайного изменения или удаления.

Например, мы можем заблокировать папку на диске для чтения, но не для записи.

UID и номера писем

IMAP позволяет ссылаться на определенное сообщение в папке одним из двух способов: временный номер сообщения (обычно 1, 2, 3 и т. д.) или уникальный идентификатор UID. Разница в том, что второй вариант более постоянный. Номера сообщений назначаются, когда мы подключаемся к папке. Они удобны для воспри-

ятия, но меняются каждый раз, когда мы подключаемся к папке снова. Это поведение (как при использовании POP) хорошо подходит для примитивных программ для считывания и загрузки писем, когда мы не возражаем против того, что номера будут каждый раз меняться. UID, с другой стороны, не меняется, даже когда мы отключаемся от сервера. Если сегодня у сообщения UID 1053, он останется таким и завтра, и ни у одного письма в этой папке никогда не будет UID 1053.

Это поведение хорошо подходит для синхронизации: мы достоверно знаем, какие операции выполняются над конкретным сообщением. Это одна из причин, по которым использовать IMAP гораздо приятнее, чем POP.

Если мы возвращаемся в аккаунт IMAP и пользователь без нашего ведома удалил папку и создал новую с тем же именем, программа может подумать, что это та же папка, но UID рассинхронизировались. Даже если мы пропустим переименование папки, мы потеряем связь между письмами в аккаунте и уже загруженными письмами.

Оказывается, IMAP может защитить нас от этого и (как мы увидим чуть позже) предлагает атрибут папки UIDVALIDITY, с помощью которого можно сравнивать UID в папке между сеансами. Номера или UID писем можно использовать в большинстве процедур IMAP, которые работают с письмами. IMAPClient обычно использует UID и игнорирует временные номера. Если нам все же нужны временные номера, достаточно указать `uid=False` при создании экземпляра IMAPClient или менять значение атрибута `use_uid` с `True` на `False` на лету во время сеанса.

Интервалы между письмами

Большинство команд IMAP для писем могут работать с несколькими письмами одновременно. Это экономит время, когда нужно обработать много писем, потому что не приходится отправлять команды и ждать ответа для каждого письма отдельно.

Получается, мы можем указать не один номер сообщения, а список номеров через запятую или диапазон номеров через двоеточие. Звездочка означает все письма до конца. Пример:

2,4:6,20:*

В этот диапазон входят письмо 2, письма с 4 по 6 и письма с 20 до конца.

Общая информация

Когда мы поначалу выбираем папку, сервер IMAP отображает сводную информацию о ней, в том числе о письмах в ней.

IMAPClient возвращает эту информацию в виде словаря. При вызове `select_folder()` на большинстве серверов IMAP могут возвращаться следующие ключи:

- ◆ EXISTS — целое число, обозначающее количество писем в папке;
- ◆ FLAGS — список флагов, которые можно применить к письмам в этой папке;

- ◆ RECENT — примерное число писем, которые появились в папке с последнего вызова `select_folder()`;
- ◆ PERMANENTFLAGS — список пользовательских флагов, которые можно установить для сообщений; обычно пуст;
- ◆ UIDNEXT — предположения сервера по UID, который будет назначен следующему сообщению;
- ◆ UIDVALIDITY — строка, которая подтверждает, что нумерация UID не изменилась. Если мы возвращаемся к папке, а это значение изменилось по сравнению с предыдущим визитом, UID был сброшен и ранее сохраненные значения недействительны;
- ◆ UNSEEN — количество непросмотренных писем (без флага Seen).

Из этих флагов серверы обычно возвращают FLAGS, EXISTS и RECENT, хотя большинство используют еще и UIDVALIDITY. Пример программы, которая читает и отображает сводную информацию из папки INBOX (Входящие), приведен в листинге 15.5.

Листинг 15.5. Сводная информация о папке

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Открываем IMAP-соединение с помощью IMAPClient и выводим информацию о папке.

import getpass, sys
from imapclient import IMAPClient

def main():
    if len(sys.argv) != 4:
        print('usage: %s hostname username foldername' % sys.argv[0])
        sys.exit(2)

    hostname, username, foldername = sys.argv[1:]
    c = IMAPClient(hostname, ssl=True)
    try:
        c.login(username, getpass.getpass())
    except c.Error as e:
        print('Could not log in:', e)
    else:
        select_dict = c.select_folder(foldername, readonly=True)
        for k, v in sorted(select_dict.items()):
            print('%s: %r' % (k, v))
```

```

finally:
    c.logout()

if __name__ == '__main__':
    main()

```

При запуске эта программа возвратит следующие результаты:

```

$ ./folder_info.py imap.example.com bpbonline@example.com
Password:
EXISTS: 3
PERMANENTFLAGS: ('\\Answered', '\\Flagged', '\\Draft', '\\Deleted',
'\\Seen', '\\*')
READ-WRITE: True
UIDNEXT: 2626
FLAGS: ('\\Answered', '\\Flagged', '\\Draft', '\\Deleted', '\\Seen')
UIDVALIDITY: 1
RECENT: 0

```

Как видите, у меня есть три письма в папке INBOX, новых писем с момента последнего входа нет. Не забудьте сравнить UIDVALIDITY со значением, сохраненным в предыдущем сеансе, если программа будет использовать UID из прошлых сеансов.

Получение всего почтового ящика

Для загрузки писем с помощью IMAP мы используем команду FETCH, которая в библиотеке IMAPClient выполняется через метод `fetch()`.

Самый очевидный способ — загрузить все письма одним большим пакетом. Этот вариант самый простой и требует меньше всего трафика, поскольку нам не приходится отправлять много запросов и получать много ответов. Это значит, что приложению придется хранить все возвращенные письма в памяти одновременно, пока они не будут обработаны. Очевидно, что это не лучший подход для больших почтовых ящиков со множеством вложений.

В листинге 15.6 используется структура данных Python для загрузки всех писем из папки INBOX в память компьютера. Затем приводится сводная информация по каждому из них.

Листинг 15.6. Загрузка всех писем в папке

```

#!/usr/bin/env python3
# Programming in Python: The Basics.
# Открываем IMAP-соединение с помощью IMAPClient и извлекаем письма.

```

```

import email, getpass, sys
from imapclient import IMAPClient

def main():
    if len(sys.argv) != 4:
        print('usage: %s hostname username foldername' % sys.argv[0])
        sys.exit(2)

    hostname, username, foldername = sys.argv[1:]
    c = IMAPClient(hostname, ssl=True)
    try:
        c.login(username, getpass.getpass())
    except c.Error as e:
        print('Could not log in:', e)
    else:
        print_summary(c, foldername)
    finally:
        c.logout()

def print_summary(c, foldername):
    c.select_folder(foldername, readonly=True)
    msgdict = c.fetch('1:*', ['BODY.PEEK[]'])
    for message_id, message in list(msgdict.items()):
        e = email.message_from_string(message['BODY[]'])
        print(message_id, e['From'])
        payload = e.get_payload()
        if isinstance(payload, list):
            part_content_types = [ part.get_content_type() for part in payload ]
            print('  Parts:', ' '.join(part_content_types))
        else:
            print('  ', ' '.join(payload[:60].split()), '...')

if __name__ == '__main__':
    main()

```

Помните, что IMAP отслеживает состояние: мы должны сначала вызвать `select_folder()`, чтобы войти в папку, и только потом `fetch()`, чтобы запросить содержимое писем. Закрывать папку можно методом `close_folder()`. Поскольку идентификаторы сообщений, временные номера или UID, всегда представляют собой положительные целые числа, диапазон `1:*` указывает все письма от первого до последнего.

Странная строка `BODY.PEEK[]` используется для запроса полного тела сообщения. Строка `BODY[]` подразумевает все письмо. Мы также можем запрашивать некоторые части письма в квадратных скобках. `PEEK` указывает, что мы обращаемся к письмам, только чтобы получить сводные данные, и мы не хотим, чтобы сервер автоматически присвоил им флаг `Seen` и считал, что они прочитаны.

Возвращенный словарь сопоставляет `UID` сообщений со словарями, где содержится информация о письмах. Мы заглядываем в словарь каждого письма в поисках записи `BODY[]`, в которой `IMAP` указал запрошенную информацию о письме: его текст, возвращенный как большая строка.

Скрипт просит Python взять строку `From:` и часть содержимого письма и вывести их на экран с помощью модуля `email`, который мы рассматривали в *главе 12*. Если бы мы хотели адаптировать этот скрипт, чтобы сохранять письма в файле или базе данных, можно было бы пропустить фазу парсинга и считать тело письма одной строкой, которую нужно сохранить и проанализировать позже.

Ниже приводятся выходные данные скрипта:

```
$ ./mailbox_summary.py imap.example.com john INBOX
```

```
Password:
```

```
2592 "Amazon.com" <order-update@amazon.com>
```

```
Dear john, Portable Power Systems, Inc. shipped the follo ...
```

```
2470 Meetup Reminder <info@meetup.com>
```

```
Parts: text/plain text/html
```

```
2472 billing@linode.com
```

```
Thank you. Please note that charges will appear as "Linode.c ...
```

Конечно, если бы письма содержали много вложений и мы решили бы загрузить их все, только чтобы предоставить сводку, это было бы чересчур. Однако это очень простая функция, и я подумал, что лучше начать с нее.

Загрузка отдельных писем

Электронные письма, как и папки с письмами, могут быть довольно большими. Многие почтовые системы позволяют пользователям хранить сотни и тысячи писем размером по 10 Мбайт или больше. Если мы начнем загружать целый почтовый ящик, как мы делали в предыдущем примере, оперативная память очень быстро переполнится.

`IMAP` поддерживает сетевые почтовые клиенты, которые не хотят хранить локальные копии каждого письма, и позволяет не только извлекать письма целиком, но и выполнять другие операции:

- ◆ заголовки письма можно извлечь отдельно в виде блока текста;
- ◆ мы можем также запрашивать конкретные заголовки, не загружая все письмо;

- ♦ сервер может с помощью рекурсии проверять письма и возвращать структуру MIME письма, а также текст его конкретных компонентов.

Это позволяет клиентам IMAP делать эффективные запросы, загружая только данные, которые нужно показать пользователю, чтобы не перегружать сервер и сеть и ускорить отображение результатов.

В листинге 15.7 сочетаются различные способы работы с аккаунтом IMAP. Мы увидим, как работает простой клиент IMAP. Этот пример даст нам больше контекста, чем если бы мы рассматривали эти возможности по отдельности в нескольких более коротких листингах. Клиент состоит из трех циклов, каждый из которых получает данные от пользователя, просматривающего список папок, а затем выводит письма в нужной папке и, наконец, показывает части определенного письма.

Листинг 15.7. Простой клиент IMAP

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Пользователь просматривает папки, письма и части письма.

import getpass, sys
from imapclient import IMAPClient

banner = '-' * 72

def main():
    if len(sys.argv) != 3:
        print('usage: %s hostname username' % sys.argv[0])
        sys.exit(2)

    hostname, username = sys.argv[1:]
    c = IMAPClient(hostname, ssl=True)
    try:
        c.login(username, getpass.getpass())
    except c.Error as e:
        print('Could not log in:', e)
    else:
        explore_account(c)
    finally:
        c.logout()

def explore_account(c):
    """Отобразите папки в этой учетной записи IMAP
    и позвольте пользователю выбрать одну из них."""
```

```
while True:
```

```
    print()
    folderflags = {}
    data = c.list_folders()
    for flags, delimiter, name in data:
        folderflags[name] = flags
    for name in sorted(folderflags.keys()):
        print('%-30s %s' % (name, ' '.join(folderflags[name])))
    print()

    reply = input('Type a folder name, or "q" to quit: ').strip()
    if reply.lower().startswith('q'):
        break
    if reply in folderflags:
        explore_folder(c, reply)
    else:
        print('Error: no folder named', repr(reply))
```

```
def explore_folder(c, name):
```

```
    """Перечислите сообщения в папке `name` и позвольте пользователю
    выбрать одно из них."""
```

```
while True:
```

```
    c.select_folder(name, readonly=True)
    msgdict = c.fetch('1:*', ['BODY.PEEK[HEADER.FIELDS (FROM SUBJECT)]',
                              'FLAGS', 'INTERNALDATE', 'RFC822.SIZE'])
    print()
    for uid in sorted(msgdict):
        items = msgdict[uid]
        print('%6d %20s %6d bytes %s' % (
            uid, items['INTERNALDATE'], items['RFC822.SIZE'],
            ' '.join(items['FLAGS'])))
        for i in items['BODY[HEADER.FIELDS (FROM SUBJECT)]'].splitlines():
            print(' ' * 6, i.strip())

    reply = input('Folder %s - type a message UID, or "q" to quit: '
                  % name).strip()
    if reply.lower().startswith('q'):
        break
    try:
        reply = int(reply)
```

```

except ValueError:
    print('Please type an integer or "q" to quit')
else:
    if reply in msgdict:
        explore_message(c, reply)

c.close_folder()

def explore_message(c, uid):
    """Пусть пользователь просматривает различные части данного сообщения."""

    msgdict = c.fetch(uid, ['BODYSTRUCTURE', 'FLAGS'])

    while True:
        print()
        print('Flags:', end=' ')
        flaglist = msgdict[uid]['FLAGS']
        if flaglist:
            print(' '.join(flaglist))
        else:
            print('none')
        print('Structure:')
        display_structure(msgdict[uid]['BODYSTRUCTURE'])
        print()
        reply = input('Message %s - type a part name, or "q" to quit: '
                      % uid).strip()

        print()
        if reply.lower().startswith('q'):
            break
        key = 'BODY[%s]' % reply
        try:
            msgdict2 = c.fetch(uid, [key])
        except c._imap.error:
            print('Error - cannot fetch section %r' % reply)
        else:
            content = msgdict2[uid][key]
            if content:
                print(banner)
                print(content.strip())
                print(banner)
            else:
                print('(No such section)')

```

```

def display_structure(structure, parentparts=[]):
    """Привлекательно отображать заданную структуру сообщения."""

    # Все тело сообщения называется TEXT.

    if parentparts:
        name = '.'.join(parentparts)
    else:
        print('  HEADER')
        name = 'TEXT'

    # Выводим простую часть MIME, которая не состоит из нескольких частей.
    # Указываем ее расположение, если оно доступно.

    is_multipart = not isinstance(structure[0], str)

    if not is_multipart:
        parttype = ('s/%s' % structure[:2]).lower()
        print('  %-9s' % name, parttype, end=' ')
        if structure[6]:
            print('size=%s' % structure[6], end=' ')
        if structure[9]:
            print('disposition=%s' % structure[9][0],
                  ' '.join('{}={}'.format(k, v) for k, v in structure[9][1:]),
                  end=' ')
        print()
        return

    # Для многокомпонентных частей выводим все вложенные части.

    parttype = 'multipart/%s' % structure[1].lower()
    print('  %-9s' % name, parttype, end=' ')
    print()
    subparts = structure[0]
    for i in range(len(subparts)):
        display_structure(subparts[i], parentparts + [ str(i + 1) ])

if __name__ == '__main__':
    main()

```

Внешняя функция, как и в предыдущих листингах, использует простой вызов `list_folders()`, чтобы показать пользователям список папок. Также показаны флаги IMAP для каждой папки.

Это позволяет пользователю делать выбор:

```
INBOX          \HasNoChildren
Receipts       \HasNoChildren
Travel         \HasNoChildren
Work           \HasNoChildren
```

Type a folder name, or "q" to quit:

Когда пользователь выбирает папку, выводится сводка по каждому письму. Почтовые клиенты принимают разные решения о том, сколько информации о каждом письме нужно отображать. В листинге 15.7 код выбирает несколько полей заголовков, а также дату и размер письма. Стоит отметить, что слово BODY используется с осторожностью. Поскольку сервер IMAP пометил бы сообщения как просмотренные (\Seen), потому что они отображаются в сводке, мы используем PEEK вместо BODY. После выбора папки результаты этого вызова retrieve() выводятся на экран:

```
2704 2019-10-28 21:32:13 19129 bytes \Seen
```

```
From: John Jebaraj
```

```
Subject: Digested Articles
```

```
2705 2019-10-28 23:03:45 15354 bytes
```

```
Subject: Re: [venv] Building a virtual environment for offline testing
```

```
From: "W. Angel Trader"
```

```
2706 2019-10-28 08:11:38 10694 bytes
```

```
Subject: Re: [venv] Building a virtual environment for offline testing
```

```
From: Esther Lopes Tavares
```

Folder INBOX - type a message UID, or "q" to quit:

Как видите, благодаря возможности передать несколько нужных элементов функции IMAP fetch() мы можем получать очень сложные сводки по письмам всего за один цикл "запрос — ответ". Когда пользователь выбирает письмо, вызывается метод fetch(), чтобы извлечь BODYSTRUCTURE (структуру тела) письма, и это позволяет увидеть компоненты MIME без загрузки всего письма. BODYSTRUCTURE перечисляет компоненты MIME в виде рекурсивной структуры данных, и нам не приходится загружать по сети большие объемы данных с вложениями.

Возвращается кортеж с простыми компонентами MIME:

```
('TEXT', 'PLAIN', ('CHARSET', 'US-ASCII'), None, None, '7BIT', 2280, 46)
```

Ниже приводятся элементы кортежа, как описано в разделе 7.4.2 в RFC 3501 (начиная с нулевого индекса):

1. Тип MIME.
2. Подтип MIME.
3. Параметры тела в виде кортежа (имя, значение, имя, значение, ...).
4. Идентификатор содержимого.
5. Описание содержимого.

6. Кодировка содержимого.

7. Размер содержимого в байтах.

Из этого складывается длина содержимого в строках для текстовых типов MIME.

Когда сервер IMAP определяет, что письмо или его часть содержит несколько компонентов (подробнее *см. главу 12*), он возвращает получателю кортеж, начинающийся со списка элементов, каждый из которых представляет собой кортеж по той же схеме. В конце указывается информация о многокомпонентном контейнере, который подключен к этим частям:

```
(((...), (...)), "MIXED", ('BOUNDARY', '==--=='), None, None)
```

Параметр MIXED (смешанный) указывает тип многокомпонентного контейнера, т. е. целиком тип называется `multipart/mixed`. Другие популярные подтипы `multipart`: `ALTERNATIVE` (альтернативный), `DIGEST` (дайджест) и `PARALLEL` (параллельный). Остальные элементы необязательны, но если они присутствуют, это серия параметров "имя — значение" (в этом случае указывается строка границы многокомпонентного письма MIME), порядок, язык и расположение (обычно по URL).

Учитывая эти правила, рекурсивный метод вроде `display_structure()` в листинге 15.7 идеально подходит для разбора и представления иерархии частей письма.

Когда сервер IMAP предоставляет BODYSTRUCTURE, мы получаем примерно следующие выходные данные:

```
Folder INBOX - type a message UID, or "q" to quit: 2701
```

```
Flags: \Seen
```

```
HEADER
```

```
TEXT multipart/mixed
```

```
1 multipart/alternative
```

```
1.1 text/plain size=253
```

```
1.2 text/html size=508
```

```
2 application/octet-stream size=5448 ATTACHMENT FILENAME='test.py'
```

```
Message 2701 - type a part name, or "q" to quit:
```

Как видите, у этого письма довольно типичная структура. Его можно просмотреть в виде сложного HTML-кода в браузере или современном почтовом клиенте либо в виде простого текста в более старых приложениях. Оно также включает вложение с предлагаемым именем файла на случай, если пользователь захочет сохранить его в локальной файловой системе. Для простоты и безопасности эта программа не пытается ничего сохранить на жесткий диск. Вместо этого пользователь может выбрать любую часть письма (например, разделы `HEADER` и `TEXT` или одну из указанных частей, вроде 1.1), и его содержимое будет выведено на экран. Эти возможности поддерживаются с помощью метода `IMAP fetch()`, как мы видим в листинге. `HEADER` и 1.1 — это просто параметры, которые можно указать в вызове `fetch()`. Их можно использовать наряду с другими значениями, например `BODY.PEEK` и `FLAGS`. Единственная разница в том, что последние значения применяются ко всему письму, а имя части 2.1.3 существует только для многокомпонентных писем, у которых в структуре есть такое обозначение.

Обратите внимание, что протокол IMAP на самом деле не предоставляет имена компонентов, которые разрешает письмо. Вместо этого необходимо сосчитать число компонентов, указанных в BODYSTRUCTURE, начиная с индекса 1, чтобы решить, какую часть запросить. Функция `display_structure()` считает элементы, используя простой цикл.

Наконец, команда `fetch()` не только позволяет извлечь нужные части письма, но и обрезает их, если они слишком длинные, а мы хотим посмотреть только начало. Для этого необходимо после имени компонента в угловых скобках указать желаемое число символов. Это похоже на операцию `slice`.

```
BODY[ ]<0..100>
```

Мы получим 100 байт тела письма, с индекса 0 до индекса 100. Это позволяет просмотреть текст и начало вложения, чтобы получить представление о его содержании до загрузки.

Добавление и удаление флагов

При работе с листингом 15.7 или просмотре примера выходных данных вы могли заметить, что IMAP назначает письмам флаги, которые обычно выглядят как слово с обратной косой чертой впереди. Например, `\Seen` означает, что письмо просмотрено. Некоторые из них описаны в RFC 3501 и используются всеми серверами IMAP. Самые важные флаги:

- ◆ `\Answered` — пользователь ответил на письмо;
- ◆ `\Draft` — пользователь что-то написал, но не отправил письмо;
- ◆ `\Flagged` — письмо помечено по какой-то причине; цель и важность этого флага зависят от почтового клиента;
- ◆ `\Recent` — письмо еще не просмотрено клиентом IMAP. Этот флаг отличается от остальных тем, что его нельзя добавить или удалить стандартными командами. Он удаляется автоматически после посещения почтового ящика;
- ◆ `\Seen` — письмо получено и прочитано.

Как видите, эти флаги примерно соответствуют статусам, которые предлагают многие почтовые клиенты, хотя формулировки могут меняться (например, "новое" вместо "непрочитанное"). Есть флаги, которые поддерживаются только некоторыми серверами и не всегда начинаются с обратной косой черты. Более того, поскольку не все серверы надежно реализуют флаг `Recent`, обычные клиенты IMAP используют его только как подсказку.

В библиотеке `IMAPClient` есть много методов для работы с флагами. Например, можно получить флаги, как с помощью `fetch()`, но ответ предоставляется не в словаре:

```
>>> c.get_flags(2703)
{2703: ('\Seen',)}
```

Также флаги можно добавлять и удалять:

```
c.remove_flags(2703, ['\\Seen'])
c.add_flags(2703, ['\\Answered'])
```

С помощью `set_flags()` можно заменить целый список флагов на новый для определенного письма, не используя цепочку операций добавления и удаления:

```
c.set_flags(2703, ['\\Seen', '\\Answered'])
```

Каждая функция может принимать не один, а несколько UID.

Удаление писем

IMAP позволяет удалять письма с помощью флагов. Для безопасности этот процесс разделен на два этапа: сначала клиент помечает письма флагом `Delete`, а затем вызывает `expunge()`, чтобы выполнить все ожидающие запросы на удаление за один раз.

Библиотека `IMAPClient` не требует делать это вручную (хотя можно и вручную). Она скрывает от нас тот факт, что простой метод `delete_messages()` использует флаги. Изменения вступают в силу также после вызова `expunge()`:

```
c.delete_messages([2703, 2704])
c.expunge()
```

Еще одна причина использовать UID вместо временных номеров заключается в том, что `expunge()` меняет временные номера в почтовом ящике.

Поиск

Если все наши письма хранятся на сервере, протокол должен работать с поиском, иначе нам пришлось бы загружать все письма на компьютер, чтобы выполнять полнотекстовый поиск. Суть поиска проста: мы используем функцию `search()` в экземпляре клиента IMAP и получаем UID писем, которые соответствуют нашим критериям (если мы не меняли для нашего клиента значение `IMAPClient` по умолчанию — `uid=True`):

```
>>> c.select_folder('INBOX')
>>> c.search('SINCE 13-Jul-2013 TEXT Apress')
[2590L, 2652L, 2653L, 2654L, 2655L, 2699L]
```

С помощью UID в функции `fetch()` можно получать нужную информацию о каждом письме, чтобы пользователь видел сводные результаты поиска.

Запрос в предыдущем примере включает два критерия: письмо должно быть получено после 13 июля 2013 г. и содержать слово `Apress`. В результатах я получил одно сообщение, которое удовлетворяет обоим критериям. Критерии были указаны через пробел. Если бы мы хотели, чтобы письмо соответствовало одному из двух критериев, мы бы использовали следующий запрос:

```
OR (SINCE 20-Aug-2010) (TEXT BPBONLINE)
```

В запросе можно сочетать самые разные критерии, которые так же определены в RFC 3501, как и остальные характеристики IMAP. Некоторые требования довольно просты и относятся к двоичным свойствам вроде флага:

ALL: все письма в почтовом ящике

UID (id, ...): письма с заданными UID LARGER

n: письма длиной более n байт SMALLER

m: письма длиной менее m байт

ANSWERED: есть флаг \Answered

DELETED: есть флаг \Deleted

DRAFT: есть флаг \Draft

FLAGGED: есть флаг \Flagged

KEYWORD: содержит заданное ключевое слово

NEW: есть флаг \Recent

OLD: нет флага \Recent

UNANSWERED: нет флага \Answered

UNDELETED: нет флага \Deleted

UNDRAFT: нет флага \Draft

UNFLAGGED: нет флага \Flagged

UNKEYWORD: не содержит заданное ключевое слово

UNSEEN: нет флага \Seen

Кроме того, существуют флаги для заголовков. Кроме слова send, которое связано с заголовком Date, они ищут строки в заголовках с тем же именем:

BCC string

CC string

FROM string

HEADER name string

SUBJECT string

TO string

Письмо IMAP содержит две даты: дату отправки, которая указывается в заголовке Date и добавляется отправителем, и дату поступления на сервер IMAP.

Первое значение можно подделать, а второе зависит от надежности сервера IMAP и его часов. В зависимости от того, какую дату мы хотим запросить, существует два набора критериев:

BEFORE 01-Jan-1970

ON 01-Jan-1970

SINCE 01-Jan-1970

SENTBEFORE 01-Jan-1970

SENTON 01-Jan-1970

SENTSINCE 01-Jan-1970

Наконец, есть две операции поиска по тексту письма. С их помощью пользователи выполняют полнотекстовый поиск, вводя запрос в поле поиска в почтовом клиенте:

Строка BODY: тело письма должно содержать эту строку.

Строка TEXT: тело или заголовок письма должно содержать эту строку.

Изучите документацию к серверу IMAP, чтобы узнать, поддерживает ли он приблизительное соответствие, как современные поисковые движки, или возвращает только точные совпадения. Если строки содержат специальные символы, заключите их в двойные кавычки и экранируйте кавычки внутри:

```
>>> c.search(r'TEXT "Quoth the raven, \'Nevermore.\'''')
[2652L]
```

Для того чтобы использовать одинарные кавычки, здесь я использовал строку `г'...'`.

Работа с папками

В IMAP для создания и удаления папок достаточно указать имя папки:

```
c.create_folder('Personal')
c.delete_folder('Work')
```

Некоторые серверы или конфигурации IMAP могут запрещать эти операции или накладывать ограничения на имена. Выполняйте проверку на ошибки, вызывая эти функции.

В аккаунте IMAP есть два варианта получить письмо не от другого пользователя. Во-первых, можно скопировать сообщения из исходной папки в новую. Для того чтобы перейти в папку, где хранятся письма, вызываем `select_folder()` и используем метод `copy()`:

```
c.select_folder('INBOX')
c.copy([2653L, 2654L], 'TODO')
```

Во-вторых, с помощью IMAP можно добавить письмо в ящик. Для передачи письма нам не понадобится SMTP. Только IMAP. Добавить письмо просто, но нужно кое-что учитывать. Самое важное — окончание строк. Для обозначения конца текстовой строки многие компьютеры UNIX используют один символ перевода строки ASCII (0x0a или `\n` в Python). В Windows используются два символа: CR-LF (0x0D или `\r` в Python) — возврат каретки и перевод строки. На старых компьютерах Mac используется только возврат вручную. IMAP, как и многие другие интернет-протоколы (например, HTTP), использует CR-LF (`\r\n` в Python). Если мы загружаем письмо с другим символом для обозначения конца строки, на некоторых серверах IMAP могут возникнуть проблемы. Поэтому при преобразовании загружаемых текстов необходимо проследить за обозначением конца строки. Большинство локальных почтовых клиентов заканчивают строку только с помощью `\n`, так что эта проблема встречается куда чаще, чем можно подумать.

Будьте осторожны при изменении символа конца строки, потому что некоторые письма могут содержать `\r\n` внутри, а первые несколько десятков строк могут заканчиваться только на `\n`, и в клиентах IMAP возникают ошибки, если письмо использует два разных варианта конца строки. Решить проблему можно с помощью метода Python `splitlines()`, который распознает все три возможных варианта. Просто выполните эту функцию над сообщением, а затем объедините строки с помощью стандартного конца строки.

```
>>> 'one\rtwo\nthree\r\nfour'.splitlines()
['one', 'two', 'three', 'four']
>>> '\r\n'.join('one\rtwo\nthree\r\nfour'.splitlines())
'one\r\ntwo\r\nthree\r\nfour'
```

Указав корректные символы конца строки, мы выполняем метод `append()` для клиента IMAP:

```
c.append('INBOX', my_message)
```

Мы можем отправить стандартный объект Python `datetime` в качестве аргумента ключевого слова, а также список флагов и `msg_time` в качестве времени поступления письма.

Асинхронность

До сих пор я описывал IMAP как синхронный протокол, но он может работать с клиентами, которые отправляют десятки запросов по сокету на сервер, а затем получают ответы в любом порядке, в каком серверу удобно их отправить.

Постоянно отправляя запрос, ожидая ответ и возвращая результат, библиотека `IMAPClient` скрывает гибкость протокола. Однако есть и другие библиотеки, например `Twisted Python`, которые позволяют работать с IMAP асинхронно. Синхронные приемы из этой главы подходят для большинства программ Python, которые работают с почтовыми ящиками. Если вы решите использовать асинхронную библиотеку, вам пригодятся все команды IMAP из этой главы. Останется только изучить, как передавать эти команды с помощью API асинхронной библиотеки.

Резюме

С помощью IMAP можно легко загружать электронные письма с удаленного сервера. Для работы с IMAP есть несколько библиотек Python. Модуль `imaplib` из стандартной библиотеки подходит для базовых задач, но мы сами отвечаем за парсинг ответов. Библиотека `IMAPClient`, написанная Менно Смитсом, входит в PyPI и предлагает гораздо больше возможностей.

Электронные письма хранятся на сервере IMAP в папках. В дополнение к папкам по умолчанию мы можем создавать пользовательские папки. Клиенты IMAP могут

создавать и удалять папки, добавлять письма в существующие папки и перемещать письма между папками. Письма можно легко выводить и извлекать после выбора папки (эквивалент команды смены каталога в файловой системе). Вместо того чтобы загружать каждое письмо целиком (хотя и это возможно), клиент может запросить из письма определенную информацию, например несколько заголовков или структуру, чтобы вывести сводку и выбрать для загрузки отдельные компоненты приложения.

Клиент может помечать письма флагами, которые бывают стандартными или специфическими. Для удаления письма мы сначала помечаем его флагом `Delete`, а затем удаляем все письма с этим флагом. Наконец, IMAP предоставляет удобные возможности поиска прямо на сервере, без загрузки на устройство пользователя.

В следующей главе мы рассмотрим совершенно другую тему: отправку команд на удаленный сервер и получение ответа.

Протоколы SSH и Telnet

Прежде всего советую прочесть статью Нила Стивенсона "В начале была командная строка"¹.

Эта глава посвящена передаче текстовых команд на другой компьютер и, пожалуй, станет самой интересной для большинства читателей. После того как мы настроим имена доменов и список приложений на панели управления хостинговой компании, командная строка становится главным инструментом установки и выполнения кода для сайтов.

Для управления виртуальными и физическими серверами почти всегда используется SSH. Amazon AWS, например, позволяет получить доступ к новому хосту, если мы укажем ключ SSH и установим его, чтобы можно было входить в новые экземпляры сразу, не вводя повторно пароль. С тех пор как мы научили компьютеры принимать текстовые команды и отвечать текстовыми данными, мы не придумали ничего лучше. Щелчки и перетаскивания мыши даже близко не сравнятся с возможностями текстовых команд, в том числе на таком строгом языке, который использует оболочка UNIX.

Содержание главы

- ◆ Автоматизация с помощью командной строки.
- ◆ Раскрытие выражения и экранирование в командной строке.
- ◆ Аргументы в командах UNIX.
- ◆ Экранирование символов.
- ◆ Ужасная командная строка Windows.
- ◆ Терминал.
- ◆ Терминалы и буферизация.
- ◆ Telnet.
- ◆ SSH: безопасная оболочка.

¹ Stephenson Neal. In the Beginning... Was Command Line. — William Morrow Paperbacks, 1999.

- ◆ SSH: краткий обзор.
- ◆ Ключи хоста для SSH.
- ◆ Аутентификация в SSH.
- ◆ Отдельные команды и сеансы.
- ◆ Протокол SFTP.
- ◆ Дополнительные возможности.
- ◆ Резюме.

Цель

Эта глава посвящена командной строке. Вы узнаете, как установить сетевое подключение к ней и устранять возможные ошибки при ее использовании.

Автоматизация с помощью командной строки

Если вы планируете заниматься удаленным администрированием сетей, вам лучше поискать другие решения. Сообщество Python использует три подхода к автоматизации удаленного управления:

1. Fabric позволяет программировать скрипты с действиями, которые выполняются по SSH-соединениям с серверами (www.fabfile.org/).
2. Ansible — простой и эффективный инструмент, с помощью которого можно автоматизировать управление десятками и сотнями удаленных компьютеров через SSH-соединение. Это эффективное решение выбирают многие системные администраторы (см. <http://docs.ansible.com/index.html>).
3. SaltStack не использует SSH, а требует установить собственного агента в клиентской системе и с его помощью отправлять команды на удаленные компьютеры гораздо быстрее, чем при использовании сотен или тысяч параллельных SSH-соединений, даже если мы работаем с огромными системами и кластерами (www.saltstack.com/).

Наконец, следует упомянуть `rexpect`. Технически эта программа не умеет работать с сетью, но она часто используется для управления командами `ssh` или `telnet`, когда программист на Python хочет автоматизировать взаимодействие с удаленной командной строкой. Обычно это происходит, когда у устройства нет API и нужно вводить команды каждый раз, как появляется командная строка. Для настройки простого сетевого оборудования часто требуется такое неудобное пошаговое взаимодействие. Подробности см. на странице `rexpect`: <http://pypi.python.org/pypi/pexpect>.

Конечно, есть вероятность, что эти решения не подойдут для вашего проекта, и тогда вам придется изучать, как работают протоколы удаленной оболочки. Как раз этой теме и посвящена глава.

Раскрытие выражения и экранирование в командной строке

Если вы когда-нибудь набирали команды в командной строке UNIX, то знаете, что не каждый символ воспринимается буквально. Возьмем для примера следующую команду. (Символ доллара \$ в этой главе будет обозначать запрос на ввод команды, который показывает, что сейчас наша очередь вводить текст.)

```
$ echo *
```

```
sftp.py shell.py ssh_commands.py ssh_simple.py ssh_simple.txt ssh_threads.py
telnet_codes.py telnet_login.py
```

Звездочка * в этой команде не означает, что нужно буквально вывести символ звездочки на экране.

Оболочка понимает, что я пытаюсь создать шаблон, который должен соответствовать всем файлам в текущем каталоге. Мне нужно будет указать еще один специальный символ, чтобы экранировать звездочку и вывести на экран именно ее.

```
$ echo Here is a lone asterisk: \*
```

```
Here is a lone asterisk: *
```

```
$ echo And here are '*' two '*' more asterisks
```

```
And here are * two * more asterisks
```

Оболочка может выполнять подпроцессы, выходные данные которых затем используются в тексте другой команды. Кроме того, она может выполнять математические вычисления. Мы можем попросить bash (стандартную оболочку на большинстве систем Linux) разделить количество слов в статье Нила Стивенсона на количество строк, чтобы узнать, сколько слов содержит каждая строка.

```
$ echo $(( $(wc -w < command.txt) / $(wc -l < command.txt) )) words per line
```

```
46 words per line
```

Как видно в этом примере, мы используем очень сложные правила для ввода специальных символов в командной строке. В обычном окне терминала размером 80 × 24 мы разместили 5375 строк, или 223 экрана с текстом. Мне не хватит этой главы, чтобы описать проблемы, которые могут возникнуть при вводе команды в оболочку. В следующих разделах мы рассмотрим всего два важных аспекта, чтобы эффективно использовать командную строку.

- ◆ Специальные символы интерпретируются как специальные используемой оболочкой, например bash. Они не имеют особого значения для операционной системы.
- ◆ Когда мы отдаем команды оболочке, локально или по сети, мы экранируем специальные символы, чтобы они не превратились в неожиданные значения в удаленной системе.

Далее мы рассмотрим эти аспекты. Мы будем работать в самых популярных серверных операционных системах, Linux и Mac OS X, а не более примитивных, вроде Windows, которую мы рассмотрим отдельно.

Аргументы в командах UNIX

В низкоуровневой командной строке UNIX нет специальных или зарезервированных символов. Это нужно понимать. Если вы использовали оболочку вроде `bash`, вероятно, вы научились относиться к ней, как к минному полю. Однако с помощью специальных символов можно легко указать все файлы в текущем каталоге как аргументы команды. Правда, бывает сложно вывести на экран сообщение, которое содержит одинарные и двойные кавычки, или понять, какие символы можно вводить, а какие считаются зарезервированными.

Важно понимать, что правила командной оболочки относительно специальных символов не связаны с операционной системой. Они относятся к самой оболочке, например `bash` или `arcane`. Даже если вам сложно представить UNIX-подобную систему без этих правил, если мы уберем оболочку, эти правила уйдут вместе с ней.

Для того чтобы в этом убедиться, запустите процесс и попробуйте указать специальные символы в простой команде.

```
>>> import subprocess
>>> args = ['echo', 'Sometimes', '*', 'is just an asterisk']
>>> subprocess.call(args)
Sometimes * is just an asterisk
```

Мы запустили новый процесс с аргументами, не призывая на помощь оболочку. Символ `*` не был преобразован в список файлов: команда `echo` вывела его буквально. Звездочка используется часто, но это не самый популярный специальный символ в оболочке, самый популярный — пробел. Он используется как разделитель между аргументами. Если включить пробел в имя файла в UNIX, а затем переместить файл, нам гарантированы часы мучений.

```
$ mv Smith Contract.txt ~/Documents
mv: cannot stat `Smith': No such file or directory
mv: cannot stat `Contract.txt': No such file or directory
```

Для того чтобы оболочка поняла, что мы имеем в виду один файл с пробелом в имени, а не два файла, попробуйте следующее:

```
$ mv Smith\ Contract.txt ~/Documents
$ mv "Smith Contract.txt" ~/Documents
$ mv Smith*Contract.txt ~/Documents
```

Последний вариант отличается от остальных тем, что будет искать все файлы, имя которых начинается на `Smith` и заканчивается на `Contract.txt`, даже если между ними не только пробел, а целая строка символов. Новички часто используют подстановочный символ вместо других обозначений пробела. В листинге 16.1 приводится базовый скрипт для оболочки на Python, в котором только символ пробела считается специальным символом, а остальное передается буквально.

Листинг 16.1. оболочка поддерживает аргументы, разделенные пробелами

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Простая оболочка для выполнения команд
# без специальных символов (кроме пробелов для разделения).
```

```
Import subprocess
```

```
def main():
    while True:
        args = input('] ').strip().split()
        if not args:
            pass
        elif args == ['exit']:
            break
        elif args[0] == 'show':
            print("Arguments:", args[1:])
        else:
            try:
                subprocess.call(args)
            except Exception as e:
                print(e)

if __name__ == '__main__':
    main()
```

Очевидно, что раз мы не используем символы экранирования, то не можем указывать файлы с пробелами в имени, потому что пробел всегда используется только для разделения параметров. Если запустить эту оболочку и протестировать специальные символы, которые раньше мы так боялись использовать, мы заметим, что они интерпретируются буквально. (Для того чтобы отличаться от вашей оболочки, оболочка в листинге 16.2 использует] для обозначения запроса команды.)

```
$ python shell.py
```

```
] echo Hi there!
```

```
Hi there!
```

```
] echo An asterisk * is not special.
```

```
An asterisk * is not special.
```

```
] echo The string $HOST is not special, nor are "double quotes".
```

```
The string $HOST is not special, nor are "double quotes".
```

```
] echo What? No *>!$ special characters?
```

What? No *<>!\$ special characters?

```
] show "The 'show' built-in lists its arguments."
```

```
Arguments: ['The', "'show'", 'built-in', 'lists', 'its', 'arguments.']."]
```

```
] exit
```

Как видите, команды UNIX — в этом случае `/bin/echo` — не обращают внимание на специальные символы в параметрах. Двойные кавычки, знак доллара и звездочка — все эти символы воспринимаются буквально. Python сводит аргументы к набору строк, которые операционная система может использовать для создания нового процесса, как показывает предыдущая команда `show`.

Что произойдет, если мы не разделим команду на аргументы, а передадим операционной системе одну строку, которая содержит имя команды и аргумент?

```
>>> import subprocess
```

```
>>> subprocess.call(['echo hello'])
```

```
Traceback (most recent call last):
```

```
...
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'echo hello'
```

Видите, что происходит? Операционная система не понимает, что пробелы должны быть специальными, так что система думает, будто ей велят выполнить команду с точными именем `echo[пробел]hello`, но не может найти ее, если мы не создали ее в текущем каталоге. Только символ `null` (с нулевым кодом Unicode и ASCII) считается по-настоящему уникальным в системе. В UNIX-подобных системах символ `null` обозначает конец аргумента командной строки в памяти. Если мы используем символ `null` в аргументе, UNIX считает, что аргумент закончился, и игнорирует остальной текст. Python укажет нам на ошибку, если мы попытаемся включить символ `null` в аргумент командной строки.

```
>>> subprocess.call(['echo', 'Sentences can end\0 abruptly.'])
```

```
Traceback (most recent call last):
```

```
...
```

```
TypeError: embedded NUL character
```

К счастью, каждая команда в системе предназначена для работы в этом ограничении, так что у нас почти нет причин использовать символы `null` в параметрах командной строки. (Они не могут встречаться в именах файлов по той же причине, по которой они не могут существовать в списках аргументов: имена файлов представляются операционной системой как строки, которые оканчиваются на `null`.)

Экранирование символов

В предыдущем разделе мы использовали процедуры в модуле Python `subprocess`, чтобы напрямую вызывать команды. Это было здорово, потому что мы могли передавать символы, которые считаются зарезервированными в традиционной оболочке. Если у нас есть длинный список имен файлов с пробелами и другими специаль-

ными символами, было бы очень удобно передать их в вызов программы, чтобы команда на том конце поняла нас. При использовании протоколов удаленной оболочки по сети мы обычно общаемся с оболочкой вроде `bash`, а не вызываем команды напрямую, как при использовании модуля `subprocess`. Это значит, что протоколы удаленной оболочки больше похожи на процедуру `system()` из модуля `os`, которая запускает оболочку для интерпретации команды, а значит, связана со всеми сложностями командной строки UNIX.

```
>>> import os
>>> os.system('echo *')
sftp.py shell.py ssh_commands.py ssh_simple.py ssh_simple.txt ssh_threads.py
telnet_codes.py
telnet_login.py
```

Сетевые приложения могут подключаться к разным системным и встроенным оболочкам с разными правилами экранирования и использования подстановочных символов. Иногда они бывают по-настоящему запутанными. Если на том конце сетевого соединения находится обычная оболочка UNIX из семейства `sh`, например `bash` или `zsh`, нам повезло: модуль Python `pipes`, который обычно используется для создания очень сложных командных строк оболочки, предлагает вспомогательную функцию для экранирования параметров. Она называется `quote()`, и нам достаточно просто передать ей строку.

```
>>> from pipes import quote
>>> print(quote("filename"))
filename
>>> print(quote("file with spaces"))
'file with spaces'
>>> print(quote("file 'single quoted' inside!"))
'file "'"single quoted'"' inside!'
>>> print(quote("danger!; rm -r *"))
'danger!; rm -r *'
```

В результате для подготовки командной строки к удаленному выполнению команд достаточно вызвать `quote()` для каждого параметра и вставить выходные данные с пробелами. Стоит отметить, что при отправке команд в удаленную оболочку с помощью Python мы обычно избегаем всех ужасов двухуровневого экранирования, с которыми вы могли сталкиваться при попытке составить командную строку SSH со сложными кавычками. Для создания команд с передачей аргументов в удаленную оболочку обычно требуется несколько попыток:

```
$ echo $HOST
guinness
$ ssh asaph echo $HOST
guinness
$ ssh asaph echo \$HOST
asaph
```

```
$ ssh asaph echo \\$HOST
guinness
$ ssh asaph echo \\\$HOST
$HOST
$ ssh asaph echo \\\\$HOST
\guinness
```

Мы можем убедиться, что каждый из этих ответов разумен. Для того чтобы посмотреть, как текст обрабатывается в удаленной командной строке SSH, сначала мы указываем `echo`, чтобы увидеть, как выглядит каждая команда в локальной оболочке, а затем вставляем этот текст в удаленную командную строку SSH. Эти команды сложно записывать, и даже опытные администраторы могут ошибиться при попытке предсказать, какой результат выдаст серия команд выше.

Ужасная командная строка Windows

Мы посмотрели, как работает оболочка UNIX и как параметры предоставляются процессу. Если вы подключаетесь к компьютеру с ОС Windows с помощью протокола удаленной оболочки, забудьте все, о чем мы говорили выше. В Windows все устроено совсем не так удобно. Вместо того чтобы передавать аргументы командной строки новому процессу в виде отдельных строк, эта операционная система передает полную командную строку. Процессу приходится самому догадываться, как пользователь записал имена файлов с пробелами. Пользователи Windows, конечно, пришли к более-менее устоявшимся соглашениям о том, как команды будут читать аргументы. Например, можно заключить имя файла из нескольких слов в двойные кавычки, и почти любая программа поймет, что мы имеем в виду один файл, а не несколько. В большинстве команд звездочка интерпретируется как подстановочный символ, но это решение принимается приложением, а не командной строкой.

Древний сетевой протокол Telnet отправляет команды открытым текстом, прямо как Windows, так что если программа отправляет параметры с пробелами или специальными символами, их нужно как-то экранировать. Если мы используем современный удаленный протокол вроде SSH, который позволяет отправлять параметры в виде списка строк, а не в одной строке, помните, что в Windows протокол SSH может только собрать нашу тщательно продуманную командную строку и надеяться, что команда Windows интерпретирует ее правильно.

При отправке команд в Windows лучше использовать метод `list2cmdline()` из модуля Python `subprocess`. Он принимает список аргументов, идентичных тем, которые применяются для команды UNIX, и пытается собрать их вместе, используя двойные кавычки и обратные косые черты по необходимости, чтобы стандартные программы Windows могли корректно извлечь эти аргументы.

```
>>> from subprocess import list2cmdline
>>> args = ['rename', 'salary "Smith".xls', 'salary-smith.xls']
>>> print(list2cmdline(args))
rename "salary \"Smith\".xls" salary-smith.xls
```


Поэкспериментируйте с сетевой библиотекой и протоколом удаленной оболочки, чтобы понять, что требует Windows в ваших обстоятельствах. В этой главе предполагается, что вы подключаетесь к серверам с современными UNIX-подобными операционными системами, в которых аргументы командной строки существуют отдельно и не требуют дополнительного экранирования.

Терминал

Обычно мы используем удаленное соединение на основе Python не только для того, чтобы общаться просто с оболочкой. Разумно будет следить за входящим потоком данных и ошибками, выдаваемыми командами. Также мы можем отправлять данные обратно — в качестве входных данных для удаленной программы или в ответ на запросы. При выполнении таких операций иногда приложения зависают навсегда, не передавая ожидаемые выходные данные. Случается, отправляемые данные не доходят до места назначения. В таких ситуациях вам пригодятся навыки работы с терминалами UNIX. *Терминал* — это устройство, на котором пользователь вводит запросы и получает ответы от компьютера. Если компьютер UNIX оснащен физическими последовательными портами, к которым можно подключить физический терминал, в каталоге устройств мы увидим запись вроде `/dev/ttyS1`, благодаря которой приложения смогут обмениваться строками с этим устройством. Сейчас терминалы обычно представляют собой другие программы, например `xterm`, `Gnome` или `KDE`, `Mac OS X iTerm` или `Terminal` или даже клиент `PuTTY` в системе Windows, подключенный через протокол удаленной оболочки. Программы в терминале обычно пытаются понять, общаются ли они с пользователем, и только если они подключены к устройству терминала, они предполагают, что нужно подготовить выходные данные для человека. Операционная система UNIX включает целый набор псевдотерминалов (виртуальных терминалов) с именами вроде `/dev/tty42`, к которым можно подключать программы, чтобы они думали, что общаются с человеком. Когда пользователь открывает `xterm` или устанавливает соединение по SSH, `xterm` или демон SSH создает новый псевдотерминал, настраивает его и выполняет оболочку.

TTY — это сокращенное обозначение устройства терминала в UNIX, т. к. первым примером терминала был `TeleType`. Поэтому для проверки того, поступают ли входные данные от терминала, используется вызов `isatty()` (*Is a teletype? — Это телетайп?*).

Важно понимать, что оболочка запрашивает команду, потому что думает, будто подключена к терминалу. Если мы запустим оболочку стандартным вводом без терминала, допустим, отправим выходные данные из другой команды, мы не увидим запрос команды, но команды будут приниматься.

\$ cat | bash

```
echo Here we are inside of bash, with no prompt
```

```
Here we are inside of bash, with no prompt
```

```
python3
```

```
print('Python has not printed a prompt, either.')
import sys
print('Is this a terminal?', sys.stdin.isatty())
```

Ни Python, ни bash не запросили ввод команды. Python ведет себя на удивление тихо. В то время как bash выводит строку текста в ответ на нашу команду echo, мы ввели уже три строки в Python и не получили никакого ответа. Что происходит?

Python считает, что раз выходные данные поступают не с терминала, нужно просто считать полный скрипт из стандартного потока ввода. В конце концов, входные данные представляют собой файл, а файлы могут содержать целые скрипты. Для того чтобы прервать это бесконечное чтение до конца файла, нужно нажать комбинацию клавиш <Ctrl>+<D>, чтобы отправить команду конца файла в cat, который закроет выходные данные и завершит пример. Python разберет и выполнит наш трехстрочный скрипт (все после слова python3 в сеансе), и мы увидим результаты в терминале, а за ними запрос команды от оболочки.

```
Python has not printed a prompt, either.
Is this a terminal? False
```

Некоторые программы автоматически меняют формат вывода независимо от того, общаются они с человеком или нет. При работе в интерактивном режиме команда ps обрезает строки выходных данных под ширину терминала, но при использовании в конвейере или файле ширина не регулируется. Традиционный вывод команды ls в столбец также преобразуется, и имена файлов выводятся строкой, потому что другой программе будет легче читать их в таком виде.

\$ ls

```
sftp.py ssh_commands.py ssh_simple.txt telnet_codes.py
shell.py ssh_simple.py ssh_threads.py telnet_login.py
```

\$ ls | cat

```
sftp.py
shell.py
ssh_commands.py
ssh_simple.py
ssh_simple.txt
ssh_threads.py
telnet_codes.py
telnet_login.py
```

Как все это связано с сетевым программированием? Когда программы запрашивают команду при подключении к терминалу, но не запрашивают при чтении из файла или получении данных от другой команды, это проявляется на удаленном конце протоколов оболочки, которые мы рассматриваем в этой главе. Например, если программа использует Telnet, она всегда общается с терминалом, так что наши скрипты и программы должны всегда ожидать запрос на команду от оболочки. При использовании более продвинутого протокола SSH мы можем выбирать, что именно наша программа будет считать источником входных данных: терминал или файл

и конвейер. Если у вас есть еще один компьютер, к которому можно подключиться, это можно легко проверить из командной строки:

```
$ ssh -t asaph
```

```
asaph$ echo "Here we are, at a prompt."
```

```
Here we are, at a prompt.
```

```
Asaph$ exit
```

```
$ ssh -T asaph
```

```
echo "The shell here on asaph sees no terminal; so, no prompt."
```

```
The shell here on asaph sees no terminal; so, no prompt.
```

```
exit
```

```
$
```

При отправке команд через современный протокол вроде SSH мы должны решить, должна ли удаленная программа считать, что получает данные из конвейера/файла или общается с пользователем через терминал. При общении с терминалом программа не должна вести себя иначе, но она может подстраиваться под нас для нашего удобства. Для этого она вызывает `isatty()`, как мы видели выше, и меняет поведение в зависимости от ответа. Вот несколько примеров различий.

- ◆ При взаимодействии с терминалом программы, часто используемые интерактивно, запрашивают команду в понятном пользователю виде. Если они считают, что входные данные поступают из файла, они не запрашивают команды, иначе экран был бы заполнен тысячами таких запросов при выполнении длинного скрипта или программы Python.
- ◆ Если входные данные поступают через ТТУ, большинство интерактивных программ предлагают командную строку. Многие символы являются уникальными, потому что используются для просмотра истории командной строки и выполнения команд редактирования. Без взаимодействия с терминалом приложения отключают редактирование командной строки и принимают символы управления в обычном потоке входных данных.
- ◆ Ожидая данные от терминала, многие программы считывают входные данные строка за строкой, поскольку пользователи предпочитают получать ответ сразу после каждой команды. При чтении из конвейера или файла программы сначала получают тысячи символов и только затем пытаются понять их. Даже при чтении из файла `bash` считывает входные данные построчно, а Python читает весь скрипт, прежде чем попытается выполнить хотя бы одну строку. Еще чаще программы меняют выходные данные при взаимодействии с терминалом.

Пользователь ждет, что каждая строка или даже буква будут отображаться моментально. Если программа общается с другой программой или файлом, она собирает выходные данные в большие фрагменты перед отправкой.

Два последних аспекта, в том числе тот факт, что они используют буферизацию, вызывают много проблем, когда мы пытаемся автоматизировать процесс, обычно выполняемый вручную, потому что программа теперь получает входные данные из конвейера или файла, а не через терминал, и внезапно начинает вести себя по-

другому. Поскольку инструкция `print` не предоставляет выходные данные моментально, а сохраняет результат, чтобы отправить все сразу при наполнении выходного буфера, может создаваться впечатление, что программа зависла.

Из-за этой проблемы многие продуманные программы на Python и других языках выполняют `flush()` для выходных данных, чтобы отправлять данные из буфера, даже если на том конце их ждет не терминал. Получается, что при отказе от терминалов программы меняют свое поведение и активно используют буферы, если думают, что записывают данные в файл или конвейер, а не предоставляют пользователю.

Терминалы и буферизация

С терминалами связаны и другие проблемы. Что если мы хотим, чтобы программа считывала данные по одному символу за раз, а терминал UNIX отправляет все введенные нами символы в буфер и передает их одной строкой? Это связано с тем, что по умолчанию терминал UNIX использует каноническую обработку входных данных, при которой пользователь вводит целую строку и может редактировать ее по ходу, а затем нажимает клавишу `<Enter>`, чтобы отправить команду. Мы можем использовать команду изменения текущих параметров TTY в `stty`, чтобы отключить традиционную обработку и передавать программе каждый символ по мере ввода.

\$ stty -icanon

Еще одна проблема заключается в том, что в терминалах UNIX можно было использовать сочетание клавиш, чтобы остановить выходные данные и прочитав весь экран текста, прежде чем переходить к следующему фрагменту. Обычно использовались сочетания `<Ctrl>+<S>` для остановки и `<Ctrl>+<Q>` для продолжения, и это было очень неудобно, когда мы отправляли двоичные данные по автоматизированному Telnet-соединению, потому что первое же сочетание `<Ctrl>+<S>` приостанавливало терминал и почти наверняка прерывало сеанс. С помощью `stty` можно отключить этот параметр.

\$ stty -ixon -ixoff

Это две самые распространенные проблемы с терминалами и буферизацией, хотя есть и другие проблемные параметры. Команда `stty` поддерживает два режима, потому что существует очень много параметров и масса реализаций UNIX. Эти два режима — `cooked` и `raw`, и они включают и отключают сразу десятки параметров вроде `icanon` и `ixon`.

\$ stty raw

\$ stty cooked

Если мы перемудрим с параметрами, в большинстве систем UNIX есть команда для сброса терминала до настроек по умолчанию. (Если вы запутались в параметрах `stty`, нажмите комбинацию клавиш `<Ctrl>+<J>` для отправки команды сброса, потому что клавиша `<Return>` (`<Ctrl>+<M>`) только отправляет команды из-за конфигурации терминала, которая называется `icrnl`.)

\$ reset

Если вы взаимодействуете с терминалом через скрипт Python и не хотите использовать Telnet или SSH, попробуйте модуль `termios` из стандартной библиотеки. С его помощью вы сможете управлять параметрами, которые меняли через `stty`, если помните, как работает булева алгебра. В этой книге мы не сможем подробно описать работу терминалов (на одни только примеры ушла бы целая глава), но на эту тему есть много интересных материалов, например глава 19 в книге Уильяма Ричарда Стивенса "UNIX: разработка сетевых приложений"².

Telnet

Этому устаревшему протоколу посвящен только небольшой раздел. Почему? Он небезопасный, потому что любой человек, который видит передаваемые пакеты, видит и ваш аккаунт, пароль и все действия. Он неудобный и почти не используется.

ПРОТОКОЛ TELNET

Цель: доступ к удаленной оболочке.

Стандарт: RFC 854 (1989 г.).

Базовый протокол: TCP/IP.

Порт по умолчанию: 23.

Библиотека: `telnetlib`.

Исключения: `socket.error`, `socket.gaierror`,
`EOFError`, `select.error`.

Я бы стал использовать Telnet только при подключении к небольшому устройству вроде роутера, DSL-модема или сетевого коммутатора внутри корпоративной сети с хорошим межсетевым экраном. Вот несколько рекомендаций по использованию модуля Python `telnetlib`, если вы планируете создать программу Python, которая должна взаимодействовать с этими устройствами через Telnet. Протокол Telnet просто устанавливает канал (простейший сокет TCP, см. главу 3), а затем копирует данные по этому каналу в оба направления. Telnet передает все данные, которые мы вводим, и выводит все, что получает, на экран. Это значит, что Telnet не знает обо всех тонкостях, о которых должен знать протокол удаленной оболочки.

Когда мы подключаемся через Telnet к компьютеру UNIX, например, обычно мы видим запрос `login:` и вводим имя пользователя, видим запрос `password:` и вводим пароль. Небольшие встроенные устройства, которые используют Telnet, могут выполнять простой скрипт, но почти всегда требуют аутентификации. В любом случае, Telnet ничего не знает об этом методе коммуникации. Строка `password:` для не-

² Stevens W. Richard. UNIX Network Programming. — Addison-Wesley Professional, 1992.

го — это просто несколько непонятных символов, которые поступают по TCP-соединению и которые клиент Telnet должен вывести на экран. Поскольку Telnet не знает об аутентификации, мы не можем отправлять аргументы в команду Telnet для предварительной аутентификации в удаленной системе, и мы не можем пропустить ввод имени пользователя и пароля, которые запрашиваются при первом подключении. Если вы собираетесь использовать Telnet, нужно считывать эти запросы и давать на них подходящие ответы.

Не существует стандартных сообщений об ошибках в случае сбоя пароля, если различные системы по-разному запрашивают имя пользователя и пароль. Поэтому сложно писать программы для Telnet на таких языках, как Python. Если мы не знаем все сообщения об ошибках, которые только может вывести удаленная система в ответ на логин и пароль (в том числе о том, что не хватает памяти, не подключен домашний каталог или превышена квота), будут регулярно возникать ситуации, когда программа будет ждать запрос команды или определенное сообщение об ошибке, но так и не дожидается.

Получается, что при использовании Telnet мы работаем только с текстом. Мы ждем поступления текста и пытаемся дать удаленной системе понятный ответ. Библиотека Python `telnetlib` помогает работать с Telnet, предоставляя не только базовые возможности отправки и получения данных, но и несколько функций, которые будут ждать определенной строки от удаленной системы. В этом отношении модуль `telnetlib` похож на сторонний пакет Python `pexpect`, о котором я говорил чуть выше, а значит, и на UNIX-команду `expect`. Одна процедура в `telnetlib` даже называется `expect()` в честь своего предшественника.

Листинг 16.2 подключается к хосту, автоматизирует запрос и передачу имени пользователя и пароля, а затем выполняет простую команду для отображения результатов.

Листинг 16.2. Вход на удаленный хост с помощью Telnet

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Подключаемся к localhost, ждем запроса логина и пытаемся войти.

import argparse, getpass, telnetlib

def main(hostname, username, password):
    t = telnetlib.Telnet(hostname)
    # t.set_debuglevel(1)          # раскомментируйте, чтобы
                                # получать отладочные сообщения

    t.read_until(b'login:')
    t.write(username.encode('utf-8'))
    t.write(b'\r')
    t.read_until(b'assword:')      # первый символ должен быть 'p' или 'P'
    t.write(password.encode('utf-8'))
    t.write(b'\r')
```

```

n, match, previous_text = t.expect([br'Login incorrect', br'\$'], 10)
if n == 0:
    print('Username and password failed - giving up')
else:
    t.write(b'exec uptime\r')
    print(t.read_all().decode('utf-8')) # читать до закрытия сокета

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Use Telnet to log in')
    parser.add_argument('hostname', help='Remote host to telnet to')
    parser.add_argument('username', help='Remote username')
    args = parser.parse_args()
    password = getpass.getpass('Password: ')
    main(args.hostname, args.username, password)

```

Если скрипт выполнен успешно, он показывает результаты простой команды `uptime` в удаленной системе.

```
$ python telnet_login.py example.com john
```

```
Password: abc12345
```

```
10:24:43 up 5 days, 12:13, 14 users, load average: 1.44, 0.92, 0.74
```

Если вы хотите, чтобы объект `Telnet` выводил все отправленные и полученные за сеанс строки, вызовите `set_debuglevel(1)`. Оказалось, что это важно даже для самого простого скрипта, как в листинге, потому что скрипт зависал дважды и мне приходилось перезапускать его с включенными отладочными сообщениями, чтобы изучить выходные данные и исправить ситуацию. (Один раз я забыл `\r` в конце команды `uptime`, а в другой раз я ошибся в возвращенном тексте.) Обычно я отключаю отладку, когда программа работает, а затем включаю, когда возобновляю работу над скриптом. `Telnet` использует сокет TCP, и исключения `socket.error` и `socket.gaierror` будут переданы программе. После создания сеанса `Telnet` обычно начинаются отправка и получение данных, причем мы ждем запрос команды или ответ от удаленного компьютера, прежде чем что-то отправить. Есть два способа ждать поступления текста.

- ◆ Простая функция `read_until()` ждет литеральную строку и возвращает строку, которая содержит весь текст, поступивший с момента запуска метода до появления заданной строки.
- ◆ Более сложный метод `expect()` принимает список регулярных выражений Python. Получив с того конца текст, соответствующий одному из регулярных выражений, `expect()` возвращает три элемента: индекс совпавшего регулярного выражения в списке, сам объект `SRE_Match` регулярного выражения и текст, полученный до совпадения. Изучите документацию по модулю `re` в стандартной библиотеке, чтобы узнать больше о возможностях `SRE_Match`, включая поиск значений для вложенных выражений.

Будьте очень внимательны при написании вложенных выражений. Когда я изначально программировал этот скрипт, я использовал шаблон `$` в `expect()`, чтобы ожидать запрос команды от оболочки. Однако оказалось, что в регулярном выражении этот символ имеет особое значение, так что пришлось его экранировать.

Скрипт прекращает выполнение, если получает сообщение об ошибке из-за неправильного пароля, а не ждет бесконечно запрос имени пользователя и пароля, который может никогда не поступить или поступить в неожиданном виде.

```
$ python telnet_login.py example.com john
```

```
Password: wrongpass
```

```
Username and password failed - giving up
```

Если вам нужно использовать Telnet в скрипте Python, учтите, что это будет более длинная или сложная версия простой структуры, приведенной здесь. Методы `read_until()` и `expect()` принимают второй аргумент, `timeout`, который задает максимальный период в секундах, в течение которого вызов будет ждать следующего совпадения, прежде чем сдастся и вернет контроль скрипту Python. Если это случится, ошибка не возникнет. Просто вернется весь уже полученный текст, и мы сами должны будем решить, содержит текст нужный шаблон или нет. У объекта `Telnet` есть несколько особенностей, которые мы не будем рассматривать здесь. Читайте документацию по модулю `telnetlib` в стандартной библиотеке, где в том числе описывается метод `interact()` для прямого взаимодействия через соединение Telnet и терминал. Такие вызовы были распространены раньше, когда нужно было автоматизировать вход, но сохранить контроль и отправлять стандартные команды. У протокола Telnet есть стандарт по внедрению информации управления, и `telnetlib` строго следует этому стандарту, чтобы отделять наши данные от управляющих кодов. В результате вы можете использовать объект `Telnet` для отправки и получения двоичных данных, при этом игнорируя управляющие коды, которые могут поступить. Если вы работаете над сложным проектом с использованием Telnet, возможно, вам нужно будет предусмотреть обработку опций.

Обычно `telnetlib` просто отказывается отправлять или принимать запрос на опции от Telnet. Для обработки опций можно указать объект `Telnet` с функцией обратного вызова. В листинге 16.3 приводится простой пример. Он реализует обычное поведение `telnetlib` для большинства опций и отказывается обрабатывать остальные. (Помните, что нужно каким-то образом реагировать на каждую опцию, иначе сеанс Telnet зависнет, потому что сервер будет бесконечно ждать от нас ответа.) Если сервер запрашивает `terminal type` (тип терминала), клиент отвечает, что это `tupython`, и команда оболочки, которую он выполняет после входа, распознает это значение как свою переменную окружения `$TERM`.

Листинг 16.3. Работа с кодами опций Telnet

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Как может выглядеть код, если мы обрабатываем опции Telnet сами.
```



```

import argparse, getpass
from telnetlib import Telnet, IAC, DO, DONT, WILL, WONT, SB, SE, TTYPE

def process_option(tsocket, command, option):
    if command == DO and option == TTYPE:
        tsocket.sendall(IAC + WILL + TTYPE)
        print('Sending terminal type "mypython"')
        tsocket.sendall(IAC + SB + TTYPE + b'\0' + b'mypython' + IAC + SE)
    elif command in (DO, DONT):
        print('Will not', ord(option))
        tsocket.sendall(IAC + WONT + option)
    elif command in (WILL, WONT):
        print('Do not', ord(option))
        tsocket.sendall(IAC + DONT + option)

def main(hostname, username, password):
    t = Telnet(hostname)
    # t.set_debuglevel(1)          # раскомментируйте для получения отладочных сообщений
    t.set_option_negotiation_callback(process_option)
    t.read_until(b'login:', 10)
    t.write(username.encode('utf-8') + b'\r')
    t.read_until(b'assword:', 10) # первый символ должен быть 'p' или 'P'
    t.write(password.encode('utf-8') + b'\r')
    n, match, previous_text = t.expect([br'Login incorrect', br'\$'], 10)
    if n == 0:
        print("Username and password failed - giving up")
    else:
        t.write(b'exec echo My terminal type is $TERM\n')
        print(t.read_all().decode('ascii'))

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Use Telnet to log in')
    parser.add_argument('hostname', help='Remote host to telnet to')
    parser.add_argument('username', help='Remote username')
    args = parser.parse_args()
    password = getpass.getpass()
    main(args.hostname, args.username, password)

```

Больше информации о том, как работают опции Telnet, можно найти в соответствующих RFC.

Далее мы рассмотрим более современный и безопасный метод выполнения удаленных операций.

SSH: безопасная оболочка

SSH (Secure Shell Protocol) — это один из самых известных безопасных зашифрованных протоколов (самый известный — HTTPS).

ПРОТОКОЛ SSH

Цель: защищенная удаленная оболочка, передача файлов, проброс портов.

Стандарт: RFC 4250–4256 (2006 г.).

Базовый протокол: TCP/IP.

Порт по умолчанию: 22.

Библиотека: paramiko.

Исключения: `socket.error`, `socket.gaierror`,
`paramiko.SSHException`.

SSH произошел от протокола, который поддерживал операции удаленного входа (`rlogin`), удаленной оболочки (`rsh`) и удаленного копирования файлов (`rscp`), и в то время был гораздо популярнее, чем Telnet.

Операция `rscp` стала настоящим спасением для тех, кто часами ждал передачи двоичных файлов между компьютерами с помощью Telnet и скрипта, который пытался ввести пароль, а затем оказывалось, что файл содержит бит, похожий на управляющий символ, так что приходилось экранировать символы или думать, как их отключить. Операция `rlogin` не пыталась бездумно повторять запрос имени пользователя и пароля. Она участвовала во всем процессе аутентификации, и мы могли даже записать в главный каталог файл с инструкциями в стиле "пускай пользователя `john` без пароля, когда он входит с компьютера `asaph`". И системные администраторы и пользователи UNIX могли не тратить по несколько часов в месяц на ввод паролей.

Более того, скопировать десять файлов из одной системы в другую с помощью `rscp` стало почти так же просто, как скопировать их в локальную папку. SSH сохранил все удобные возможности своего предшественника и добавил безопасность и надежное шифрование, так что теперь он используется на важнейших серверах по всему миру. В этой главе мы рассмотрим сторонний пакет `paramiko` для Python, который работает с протоколом SSH и так хорошо с этим справляется, что был адаптирован и для Java, когда пользователи Java захотели так же просто и удобно взаимодействовать с SSH.

SSH: краткий обзор

До этого момента мы видели примеры того, как протокол используется для выполнения только одной задачи — загрузки веб-страницы или отправки электронного

письма. Мы не пытались выполнять несколько операций через один сокет. SSH настолько продвинутый, что реализует собственное мультиплексирование. Один SSH может использоваться сразу несколькими информационными каналами, потому что SSH помечает каждый передаваемый блок данных идентификатором канала. Такой подход обладает как минимум двумя преимуществами. Во-первых, идентификатор канала хоть и имеет вес, но он незначителен по сравнению с объемом дополнительных данных, которые SSH передает для согласования и поддержания шифрования. Во-вторых, установка SSH-соединения требует больших затрат. Согласование ключа хоста и аутентификация могут занимать несколько секунд, поэтому будет разумно использовать уже установленное соединение для максимального количества задач. Для того чтобы компенсировать дорогостоящее соединение, мы пытаемся выполнить в нем как можно больше операций, и благодаря каналам SSH это возможно.

После подключения можно создавать различные каналы:

- ◆ интерактивный сеанс оболочки (вроде того, который поддерживает Telnet);
- ◆ выполнение одной команды изолированно;
- ◆ проброс портов с перехватом TCP-соединений;
- ◆ сеанс передачи файлов с доступом к удаленной файловой системе.

В следующих разделах мы рассмотрим разные типы каналов.

Ключи хоста для SSH

Когда SSH-клиент подключается к удаленному хосту впервые, они обмениваются временными открытыми ключами, чтобы зашифровать остальную коммуникацию. Клиент хочет удостовериться в подлинности сервера, прежде чем передавать ему дальнейшую информацию. Это логично, ведь мы не хотим, чтобы SSH раскрывал даже имя пользователя, не говоря уже о пароле, если вдруг злоумышленник временно завладел IP-адресом удаленного сервера.

Одно из решений — использовать инфраструктуру открытых ключей (см. главу 6). Для начала нужно выбрать группу компаний, называемых центрами сертификации, которые будут выдавать сертификаты. Затем мы устанавливаем список их открытых ключей во всех браузерах и других SSL-клиентах, которые сейчас доступны. Центры сертификации берут с вас плату за подтверждение того, что вы действительно **google.com**, например, и SSL-сертификат **google.com** можно безопасно подписать. Наконец, можно установить сертификат на веб-сервер, чтобы все знали, кто вы.

С точки зрения SSH, у этого метода много недостатков. Мы действительно можем создать внутреннюю инфраструктуру открытых ключей, в которой мы будем распространять сертификаты своего центра сертификации по веб-браузерам и приложениям, а затем подписывать свои сертификаты сервера, не платя третьим сторонам, но для SSH это очень долгий процесс. Администраторы серверов обычно

предпочитают устанавливать, использовать и отключать серверы, не спрашивая разрешения у центральных органов.

В итоге SSH предполагает, что когда сервер разворачивается, он создает случайную комбинацию открытого и закрытого ключей, которая никем не подписывается. Используется один из двух методов распространения ключей.

- ◆ Системный администратор создает скрипт, который собирает все открытые ключи хоста в организации, готовит файл `ssh_known_hosts` со списком всех известных SSH-хостов и сохраняет его в каталог `/etc/ssh` на каждом компьютере. Он может предоставлять его любым настольным клиентам вроде PuTTY в Windows. Перед первым подключением каждый SSH-клиент будет знать о каждом ключе SSH-хоста.
- ◆ Администратор может просто отказаться от предоставления ключей хостов заранее и заставить каждого SSH-клиента запоминать их при первом подключении. Это знакомо тем, кто использует командную строку SSH: клиент говорит, что не узнает хост, к которому мы подключаемся, мы отвечаем `yes` (да), и ключ клиента сохраняется в файл `/.ssh/known_hosts`. При первом подключении у нас нет возможности узнать, действительно ли этот хост тот, за кого себя выдает. Зато при последующих подключениях мы можем быть уверены, что это правильный хост, а не другой сервер с таким же IP-адресом (если только кто-то не украдет ключи хоста).

Когда в командной строке SSH попадает незнакомый хост, она выдает запрос:

\$ ssh dns.google

The authenticity of host 'dns.google (8.8.8.8)' can't be established.

RSA key fingerprint is 85:8f:32:4r:ac:1f:a9:bc:35:58:c1:d4:25:e3:c7:8c.

Are you sure you want to continue connecting (yes/no)? **yes**

Warning: Permanently added 'dns.google 8.8.8.8' (RSA) to the list of known hosts.

Ответ `yes` я ввел с клавиатуры, разрешив SSH создать подключение и запомнить ключ на будущее. Если SSH подключается к хосту и обнаруживает другой ключ, он предупреждает нас об этом.

\$ ssh dns.google

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

```
@ WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED! @
```

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

```
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
```

Someone could be eavesdropping on you right now (man-in-the-middle attack)!

Если вам приходилось воссоздавать сервер с нуля и вы забывали сохранять старые ключи SSH, вы уже видели подобное предупреждение. Без старых ключей новый хост будет использовать ключи, созданные при переустановке. Придется приложить немало усилий, чтобы пройти по всем SSH-клиентам и удалить проблемный старый ключ, чтобы они получили новый при повторном подключении. Все стандартные методы SSH, использующие ключи хоста, полностью поддерживаются библиотекой `paramiko`. По умолчанию она ведет себя очень экономно и не загружа-

ет файлы с ключами хоста, поэтому при первом подключении к хосту вызывается исключение, ведь проверить ключи не получится.

```
>>> import paramiko
>>> client = paramiko.SSHClient()
>>> client.connect('example.com', username='test')
Traceback (most recent call last):
```

```
...
```

```
paramiko.ssh_exception.SSHException: Server 'example.com' not found in known_hosts
```

Перед подключением загрузите известные ключи хоста системы и текущего пользователя, чтобы получить типичную команду SSH.

```
>>> client.load_system_host_keys()
>>> client.load_host_keys('/home/John/.ssh/known_hosts')
>>> client.connect('example.com', username='test')
```

С помощью библиотеки `paramiko` можно выбрать, как поступать с неизвестными хостами. После создания объекта клиента ему нужно присвоить класс для принятия решений, который будет вызываться в случае, когда ключ хоста не распознается. Этот класс можно создать путем наследования от класса `MissingHostKeyPolicy`.

```
>>> class AllowAnythingPolicy(paramiko.MissingHostKeyPolicy):
... def missing_host_key(self, client, hostname, key):
... return
...
>>> client.set_missing_host_key_policy(AllowAnythingPolicy())
>>> client.connect('example.com', username='test')
```

Стоит отметить, что входные данные для метода `missing_host_key()` предоставляют различную информацию для принятия решения. Например, можно разрешить соединения без ключа хоста с компьютерами в подсети нашего сервера. В `paramiko` есть разные классы для принятия решений, которые уже реализуют различные опции ключей хоста.

- ◆ **`paramiko.AutoAddPolicy`.** Когда мы впервые видим ключ хоста, он сразу добавляется в хранилище ключей хоста пользователя (в системах UNIX — файл `/.ssh/known_hosts`), но если потом ключ хоста изменится, это вызовет критическое исключение.
- ◆ **`paramiko.RejectPolicy`.** При подключении к хостам с неизвестными ключами выдается исключение.
- ◆ **`paramiko.WarningPolicy`.** При подключении к неизвестному хосту в журнал записывается предупреждение, но соединение продолжается.

При создании скрипта SSH я обычно начинаю с подключения к удаленному хосту вручную с помощью инструмента командной строки `ssh`, чтобы ответить `yes` на запрос и сохранить ключ удаленного хоста в файл с ключами хоста. Таким образом программам не приходится беспокоиться об отсутствии ключей, а если ключа нет, они выдают ошибку. Если вы не хотите обрабатывать подключение вручную, ис-

пользуйте метод `AutoAddPolicy()`. Он не требует участия человека, зато гарантирует, что при последующих подключениях вы будете общаться с тем же компьютером. Даже если это злоумышленник, который записывает все наши контакты с ним и пароль (если мы его указываем), у него по крайней мере должен быть тот же секретный ключ при каждом подключении.

Аутентификация в SSH

По аутентификации в SSH в Интернете можно найти много документации и статей, в том числе о том, как настроить стандартный SSH-клиент, создать SSH-сервер на хосте с UNIX или Windows и использовать открытые ключи, чтобы каждый раз не вводить пароль. Здесь я скажу об аутентификации всего пару слов. Существует три стандартных метода доказать свою подлинность удаленному серверу при взаимодействии с ним через SSH.

- ◆ Можно указать имя пользователя и пароль.
- ◆ Можно указать имя пользователя, чтобы затем клиент выполнил аутентификацию с запросом и ответом с помощью открытого ключа. Так можно доказать, что у нас есть скрытый ключ идентификации, при этом не раскрывая его содержимое удаленной системе.
- ◆ Можно использовать Kerberos для аутентификации. Если удаленная система поддерживает Kerberos, а мы используем инструмент командной строки `kinit` для подтверждения подлинности при взаимодействии с одним из главных серверов Kerberos в домене аутентификации SSH-сервера, можно будет выполнить вход без пароля.

Мы рассмотрим первые два варианта, потому что третий используется редко. Использовать имя пользователя и пароль с `paramiko` очень просто — достаточно указать их в вызове метода `connect()`.

```
>>> client.connect('example.com', username='john', password=abc12345)
```

Код Python выглядит еще проще в случае применения открытых ключей, когда мы с помощью `ssh-keygen` создаем пару ключей (обычно сохраняемых в каталоге `/.ssh`), чтобы проходить аутентификацию без пароля.

```
'my.example.com' >>> client.connect('my.example.com')
```

Если файл с ключом идентификации не находится в стандартном каталоге `/.ssh/id_rsa`, можно указать имя файла или список с именами файлов в методе `connect()`.

```
>>> client.connect('my.example.com', key filename='/home/john/.ssh/id_sysadmin', key
filename='/home/john/.ssh/id_sysadmin', key filename='/home/john/.ssh/id_sysadmin')
```

Такой подход сработает только в том случае, если открытый ключ в файле `id_sysadmin.pub` добавлен в файл авторизованных хостов на удаленном конце, который называется примерно следующим образом:

```
/home/john/.ssh/authorized_keys
```

Всегда проверяйте разрешения файлов в удаленном каталоге `.ssh` и файлах внутри, если не получается наладить аутентификацию с открытыми ключами. Некоторым SSH-серверам не нравится, когда файлы доступны для чтения или записи в группе. SSH обычно предпочитает режим `0700` для каталога `.ssh` и режим `0600` для файлов внутри. В последних версиях операция копирования ключей SSH в другие аккаунты была автоматизирована с помощью короткого скрипта, который гарантирует, что разрешения файлов будут заданы корректно.

```
myaccount@example.com ssh-copy-id -i /.ssh/id_rsa.pub
```

После успешного выполнения метода `connect()` мы можем приступить к удаленным операциям, которые будут передаваться через один физический сокет без необходимости повторно согласовывать ключ хоста, идентификацию или шифрование для защиты SSH-соединения.

Отдельные команды и сеансы

После подключения SSH-клиента мы получим доступ к SSH-операциям. Мы можем легко устанавливать сеансы с удаленной оболочкой, выполнять определенные команды, запускать сеансы для передачи файлов и настраивать проброс портов. Все эти операции мы рассмотрим по отдельности. С помощью SSH можно установить прямой сеанс с оболочкой, которая выполняется на том конце в псевдотерминале и позволяет программам общаться с пользователем, как через терминал. Это соединение похоже на Telnet. В листинге 16.4 в удаленную оболочку отправляется простая команда `echo`, а затем мы просим завершить работу.

Листинг 16.4. Используем SSH для запуска интерактивной оболочки

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Используем SSH как Telnet: подключаемся и выполняем две команды.

import argparse, paramiko, sys

class AllowAnythingPolicy(paramiko.MissingHostKeyPolicy):
    def missing_host_key(self, client, hostname, key):
        return

def main(hostname, username):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(AllowAnythingPolicy())
    client.connect(hostname, username=username) # password=''

    channel = client.invoke_shell()
    stdin = channel.makefile('wb')
    stdout = channel.makefile('rb')
```

```

stdin.write(b'echo Hello, world\rexit\r')
output = stdout.read()
client.close()

sys.stdout.buffer.write(output)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Connect over SSH')
    parser.add_argument('hostname', help='Remote machine name')
    parser.add_argument('username', help='Username on the remote machine')
    args = parser.parse_args()
    main(args.hostname, args.username)

```

Как видите, скрипт связан ограничениями программы на основе терминала. Вместо того чтобы аккуратно инкапсулировать каждую из двух команд и разделить их аргументы, он надеется, что удаленная оболочка корректно разделит входные данные по пробелам и символам возврата каретки. Этот скрипт предполагает, что у нас уже есть файл идентификации и файл с удаленными ключами, так что вводить пароль не нужно. Если вы хотите вводить пароль, раскомментируйте аргумент пароля в скрипте. Мы можем вызвать `getpass()`, как в примере с Telnet, чтобы не вводить пароль в файле Python. В этом случае мы заметим, что вводимые команды отображаются дважды, и мы не можем на первый взгляд отличить эхо команды от ее выходных данных.

```
Welcome to Ubuntu 13.10 (GNU/Linux 3.11.0-19-generic x86_64)
```

```
Last login: Wed Mar 23 15:06:03 2019 from localhost
```

```
echo Hello, world
```

```
exit
```

```
test@john:~$ echo Hello, world
```

```
Hello, world
```

```
test@john:~$ exit
```

```
logout
```

Понимаете, что произошло?

Текст команды был передан удаленному хосту, пока он еще отправлял нам приветственное сообщение, потому что мы не сделали паузу и не дождались запроса, прежде чем отдать команды `echo` и `exit` (для которых потребовался бы цикл с повторяющимися вызовами `read()`). Команды были написаны сразу под строкой `Last login` (последний вход), потому что терминал UNIX по умолчанию работает в режиме `cooked`, т. е. выводит нажатия клавиш пользователем. Затем оболочка `bash` начала считывать команды символ за символом, установив терминал в режим `raw`, потому что предпочитает предоставлять собственный интерфейс командной строки. Поскольку она предполагает, что вы хотите видеть, что делаете (хотя вы уже перестали печатать и символы считываются из буфера с запаздыванием в несколько миллисекунд), она повторяет все команды на экране.

Конечно, пришлось бы приложить немалые усилия, чтобы без парсинга и сложных функций создать программу Python, которая отделяет выходные данные команды (слова `Hello, world`) от остальных выходных данных, которые мы получаем через SSH-соединение. Из-за этого странного поведения, определяемого терминалом, вызывайте `invoke_shell()`, только если разрабатываете интерактивную программу с терминалом, в которой пользователь будет вводить команды. В остальных случаях лучше используйте `exec_command()` для удаленных команд, чтобы выполнять отдельные команды, а не начинать целый сеанс с оболочкой. Этот вызов позволяет управлять стандартными потоками ввода, вывода и ошибок, как если бы мы выполняли команду локально с помощью модуля `subprocess` из стандартной библиотеки.

Такой скрипт приводится в листинге 16.5. Разница между `exec_command()` и локальным подпроцессом (кроме того факта, что команда выполняется на удаленном компьютере) заключается в том, что мы не можем предоставлять параметры командной строки удаленному серверу в виде отдельных строк. Вместо этого мы предоставляем удаленной оболочке командную строку целиком, чтобы она сама ее интерпретировала.

Листинг 16.5. Выполнение отдельных команд SSH

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Выполняем три отдельные команды и читаем три набора отдельных выходных данных.

import argparse, paramiko

class AllowAnythingPolicy(paramiko.MissingHostKeyPolicy):
    def missing_host_key(self, client, hostname, key):
        return

def main(hostname, username):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(AllowAnythingPolicy())
    client.connect(hostname, username=username) # password=''

    for command in 'echo "Hello, world!'", 'uname', 'uptime':
        stdin, stdout, stderr = client.exec_command(command)
        stdin.close()
        print(repr(stdout.read()))
        stdout.close()
        stderr.close()

    client.close()
```

```

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Connect over SSH')
    parser.add_argument('hostname', help='Remote machine name')
    parser.add_argument('username', help='Username on the remote machine')
    args = parser.parse_args()
    main(args.hostname, args.username)

```

В отличие от предыдущих примеров взаимодействия через Telnet и SSH, скрипт получит выходные данные этих трех команд в виде отдельных потоков данных. Нет риска перепутать выходные данные одной команды с выходными данными другой.

```
$ python3 ssh_commands.py localhost john
```

```
'Hello, world!\n'
```

```
'Linux\n'
```

```
'15:30:18 up 5 days, 22:55, 5 users, load average: 0.79, 0.84, 0.71\n'
```

SSH не только обеспечивает безопасность, но и семантически разделяет операции на удаленном компьютере, не требуя устанавливать отдельные соединения.

Если нужно заключить в кавычки аргументы командной строки, чтобы удаленная оболочка корректно интерпретировала имена файлов, содержащие пробелы и специальные символы, можно использовать метод `quotes()` из модуля Python `pipes` при создании командных строк для функции `exec_command()`, как описано в *разд. "Telnet" ранее в этой главе*. Каждый раз, как мы вызываем `invoke_shell()` или `exec_command()`, чтобы инициализировать новый SSH-сеанс для подключения к оболочке или выполнить команду, создается новый канал SSH, который предлагает объекты Python, напоминающие файлы, для взаимодействия со стандартными потоками ввода, вывода и ошибок удаленной команды. Эти каналы работают параллельно, и SSH эффективно чередует их данные в одном SSH-соединении, чтобы взаимодействие происходило одновременно, но не смешивалось.

В листинге 16.6 приводится простой пример. Две командные строки запускаются удаленно, и каждая из них представляет собой простой скрипт оболочки с командами `echo` и паузами `sleep`. Можно представить, что это команды файловой системы, которые возвращают данные при проходе по файловой системе, или операции, потребляющие много ресурсов центрального процессора, которые создают и возвращают результаты медленно. SSH не видит разницы. Канал приостанавливается на несколько секунд, а затем возобновляет работу при поступлении новых данных.

Листинг 16.6. Каналы SSH, работающие одновременно

```

#!/usr/bin/env python3
# Programming in Python: The Basics.
# Параллельное выполнение двух удаленных команд по разным каналам.

import argparse, paramiko, threading

```

```

class AllowAnythingPolicy(paramiko.MissingHostKeyPolicy):
    def missing_host_key(self, client, hostname, key):
        return

def main(hostname, username):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(AllowAnythingPolicy())
    client.connect(hostname, username=username) # password='')

    def read_until_EOF(fileobj):
        s = fileobj.readline()
        while s:
            print(s.strip())
            s = fileobj.readline()

    ioe1 = client.exec_command('echo One;sleep 2;echo Two;sleep 1;echo Three')
    ioe2 = client.exec_command('echo A;sleep 1;echo B;sleep 2;echo C')
    thread1 = threading.Thread(target=read_until_EOF, args=(ioe1[1],))
    thread2 = threading.Thread(target=read_until_EOF, args=(ioe2[1],))
    thread1.start()
    thread2.start()
    thread1.join()
    thread2.join()

    client.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Connect over SSH')
    parser.add_argument('hostname', help='Remote machine name')
    parser.add_argument('username', help='Username on the remote machine')
    args = parser.parse_args()
    main(args.hostname, args.username)

```

Мы начинаем два потока и предоставляем им по одному каналу для чтения, чтобы обрабатывать оба потока данных одновременно. Оба потока выводят новую строку данных, как только она поступает, и завершают работу, когда команда `readline()` возвращает пустую строку, обозначая конец файла. Выходные данные этого скрипта:

```
$ python3 ssh_threads.py localhost john
```

```
One
```

```
A
```

B
Two
Three
C

Как видите, каналы SSH в одном TCP-соединении действуют независимо друг от друга. Они получают и отправляют данные в своем темпе и могут завершаться отдельно, когда заканчивается их команда. Точно так же можно выполнять еще две операции: передачу файлов и проброс портов.

Протокол SFTP

SFTP (SSH File Transfer Protocol — протокол передачи файлов поверх SSH) позволяет проходить по дереву удаленного каталога, создавать и удалять каталоги и файлы, а также копировать файлы с локального на удаленный компьютер и обратно. SFTP предлагает настолько широкие возможности, что может использоваться в средствах просмотра графических файлов и даже для монтирования удаленных файловых систем локально. (Найдите информацию о системе sshfs в Интернете.)

До протокола SFTP мы копировали файлы с помощью неэффективных скриптов через Telnet путем тщательного экранирования двоичных данных. SSH не заставляет нас использовать командную строку `sftp` для передачи файлов, а использует подход RSH и предоставляет инструмент командной строки `scp`, который работает почти как `ср`, но позволяет указывать перед именем файла имя хоста, чтобы обозначить, что файл находится в удаленной системе. Это значит, что команды удаленного копирования сохраняются в истории командной строки среди остальных команд оболочки, а не в отдельном буфере истории отдельной командной строки, которую нужно вызывать, а затем завершать (это очень раздражало при использовании традиционных клиентов FTP). Более того, SFTP и команды `sftp` и `scp` не только предлагают аутентификацию по паролю, но и позволяют копировать файлы с помощью того же метода с открытыми ключами, благодаря которому нам не приходится вводить пароль снова и снова при выполнении задач на удаленном компьютере с помощью команды `ssh`.

Для того чтобы получить представление о действиях, поддерживаемых SFTP, читайте *главу 17* о протоколе FTP. Большинство операций SFTP называется так же, как локальные команды, которые мы используем для работы с файлами в оболочке UNIX, например `chmod` и `mkdir`, или как системные вызовы UNIX, например `lstat` и `unlink`. Вы наверняка знакомы с ними по модулю Python `os`. Поскольку это очень знакомые операции, при написании команд SFTP мне достаточно документации по `paramiko` для клиента Python SFTP (www.lag.net/paramiko/docs/paramiko.SFTPClient-class).

О чем нужно помнить при использовании SFTP.

- ♦ Как FTP и обычный доступ через оболочку, протокол SFTP отслеживает состояние. В результате мы можем либо указать все имена файлов и каталогов в виде абсолютного пути, начиная с корня файловой системы, либо использовать

`getcwd()` и `chdir()`, чтобы переходить по файловой системе, а затем указывать относительные пути в каталогах, куда мы вошли.

- ◆ Мы можем открыть файл методом `file()` или `open()` (так же, как в Python есть встроенная вызываемая функция с обоими именами) и получим объект в стиле файла, который подключен к каналу SSH, существующему отдельно от канала SFTP. То есть мы можем и дальше выдавать инструкции SFTP, перемещаясь по файловой системе и копируя или открывая другие файлы, а исходный канал останется подключенным к своему файлу и будет готов для чтения или записи. Файлы можно передавать асинхронно, потому что у каждого открытого удаленного файла есть свой канал. Мы можем открыть несколько удаленных файлов одновременно и записывать их все на жесткий диск или отправлять в них данные. Следите за тем, чтобы не открыть сразу много каналов, иначе каждый будет работать очень медленно.
- ◆ Наконец, для имен файлов, передаваемых через SFTP, не поддерживается раскрытие выражений. Если мы укажем имя файла, которое начинается с символа `*` или содержит пробелы и специальные символы, это будет считаться частью имени файла. При использовании SFTP нам не нужна оболочка. Благодаря поддержке внутри SSH-сервера мы можем напрямую взаимодействовать с удаленными файловыми системами. Это значит, что если мы хотим предложить пользователям возможность поиска совпадений по шаблону, мы должны сами получить содержимое каталога и поискать этот шаблон, используя процедуру, похожую на то, что предлагает `fnmatch` из стандартной библиотеки Python.

Простой сеанс SFTP показан в листинге 16.7. Он выполняет простую задачу, которая иногда требуется системным администраторам (хотя они могут с тем же успехом использовать команду `scp`): он подключается к удаленной системе и копирует файлы журнала сообщений из каталога `/var/log` на локальный компьютер, например для сканирования и анализа.

Листинг 16.7. Вывод каталога и извлечение файлов с помощью SFTP

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Получение файлов с помощью SFTP.

import argparse, functools, paramiko

class AllowAnythingPolicy(paramiko.MissingHostKeyPolicy):
    def missing_host_key(self, client, hostname, key):
        return

def main(hostname, username, filenames):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(AllowAnythingPolicy())
    client.connect(hostname, username=username) # password='')
```

```

def print_status(filename, bytes_so_far, bytes_total):
    percent = 100. * bytes_so_far / bytes_total
    print('Transfer of %r is at %d/%d bytes (%.1f%%)' % (
        filename, bytes_so_far, bytes_total, percent))

sftp = client.open_sftp()
for filename in filenames:
    if filename.endswith('.copy'):
        continue
    callback = functools.partial(print_status, filename)
    sftp.get(filename, filename + '.copy', callback=callback)
client.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Copy files over SSH')
    parser.add_argument('hostname', help='Remote machine name')
    parser.add_argument('username', help='Username on the remote machine')
    parser.add_argument('filename', nargs='+', help='Filenames to fetch')
    args = parser.parse_args()
    main(args.hostname, args.username, args.filename)

```

Хотя я подробно объяснил, что у каждого файла, который мы открываем с помощью SFTP, есть свой независимый канал, простые функции `get()` и `put()`, предоставляемые библиотекой `paramiko`, которые, по сути, представляют собой обертку для `open()` с циклом для чтения и записи, не пытаются выполнять операции асинхронно. Они просто блокируются и ждут поступления каждого файла целиком. Это значит, что предыдущий скрипт передает по одному файлу за раз, и выходные данные выглядят примерно так:

```
$ python sftp.py localhost john W-2.pdf miles.png
```

```

Transfer of 'W-2.pdf' is at 32768/115065 bytes (28.5%)
Transfer of 'W-2.pdf' is at 65536/115065 bytes (57.0%)
Transfer of 'W-2.pdf' is at 98304/115065 bytes (85.4%)
Transfer of 'W-2.pdf' is at 115065/115065 bytes (100.0%)
Transfer of 'W-2.pdf' is at 115065/115065 bytes (100.0%)
Transfer of 'miles.png' is at 15577/15577 bytes (100.0%)
Transfer of 'miles.png' is at 15577/15577 bytes (100.0%)

```

Для того чтобы изучить весь набор операций с файлами, предоставляемый SFTP, прочитайте продуманную документацию для `paramiko` по предоставленной выше ссылке.

Дополнительные возможности

Мы рассмотрели все операции SSH, предоставляемые через методы для базового объекта `SSHClient`. Более сложные возможности, например удаленные сеансы X11 и проброс портов, требуют глубже погрузиться в интерфейс `paramiko` и взаимодействовать напрямую с объектом `transport` клиента.

Класс `transport` отвечает за интерпретацию низкоуровневых операций, которые лежат в основе SSH-соединения. Мы можем быстро запросить у клиента передачу.

```
>>> client.get transport = transport()
```

В этой главе мы не сможем рассмотреть все возможности SSH, но этих знаний будет достаточно, чтобы изучить их самостоятельно в документации по `paramiko`, в том числе по примерам кода в каталоге `demos` проекта `paramiko`, в блогах, на Stack Overflow и в других ресурсах.

SSH открывает порт на локальном или удаленном хосте (как минимум предоставляет порт для соединений с `localhost` и принимает соединения от других компьютеров в Интернете) и перенаправляет эти соединения по каналу SSH для подключения к хосту и порту на том конце, передавая данные между ними. Проброс портов — важная функция. Например, иногда я работаю над веб-приложением, которое я не могу выполнить на своем ноутбуке, потому что оно требует доступ к базе данных и другим ресурсам, которые можно получить только от пула серверов. Однако я не хочу запускать программу на общедоступном порте, потому что придется скорректировать правила межсетевого экрана, а потом настроить HTTPS, чтобы третьи стороны не видели мою работу. Можно запустить разрабатываемое приложение на удаленном компьютере разработки так же, как на локальном (ожидая передачи данных на порту `localhost:8080`, чтобы защититься от подключений с других компьютеров), а затем сообщить SSH, что подключения к локальному порту `8080` на моем ноутбуке нужно перенаправлять, чтобы подключиться к порту `8080` на том локальном компьютере.

```
$ ssh -L 8080:localhost:8080 devel.example.com
```

Если вы собираетесь создать проброс портов с помощью `paramiko` при использовании SSH-соединения, у меня есть для вас хорошая и плохая новости. Плохая новость в том, что `SSHClient` поддерживает стандартные операции, вроде сеансов с оболочкой, и не предоставляет механизм проброса портов напрямую. Придется писать циклы, которые передают данные в оба конца, и использовать объект `transport`. Поскольку данные в таком случае передаются по каналам в SSH-соединении, мы можем не волноваться о том, что это HTTP или другой открытый трафик, потому что внутри SSH он защищен от третьих лиц благодаря шифрованию.

Резюме

Протоколы удаленной оболочки позволяют подключаться к удаленным компьютерам, выполнять команды оболочки и просматривать результаты, как будто в локальном окне терминала. Иногда эти протоколы используются для подключения к

реальной оболочке UNIX, а иногда — для подключения к небольшим встроенным оболочкам в роутерах или другом сетевом оборудовании, которое необходимо настроить. При выполнении команд UNIX не забывайте о буферизации выходных данных, специальных символах и буферизации входных данных из терминала, иначе данные могут испортиться, а соединение с оболочкой — зависнуть. Модуль `telnetlib` из стандартной библиотеки Python нативно поддерживает протокол Telnet. Telnet — старый, небезопасный и сложный для написания протокол, но некоторые устройства поддерживают только его.

Современный протокол SSH используется не только для подключения к командной строке удаленного хоста, но и для передачи файлов и проброса портов TCP/IP. Для работы с SSH на Python есть превосходный сторонний пакет `paramiko`. Три момента, о которых нужно помнить при настройке SSH-соединения:

- ◆ `paramiko` при подключении будет проверять подлинность удаленного компьютера (если мы явно не укажем, что нужно пропустить эту проверку) с помощью ключа хоста;
- ◆ аутентификация обычно выполняется с помощью пароля или пары открытого и закрытого ключей, причем открытый ключ хранится в файле `authorized_keys` удаленного сервера;
- ◆ после аутентификации мы можем выполнять по SSH разные операции, в том числе использовать удаленную оболочку, выполнять отдельные команды или передавать файлы. Все эти операции будут выполняться параллельно по одному соединению благодаря использованию нескольких каналов.

В следующей главе мы рассмотрим протокол передачи файлов FTP — более старый и ограниченный вариант SFTP, который использовался на заре Интернета.

Протокол FTP

FTP (File Transfer Protocol — протокол передачи файлов) раньше был самым распространенным интернет-протоколом, с помощью которого пользователи передавали данные между компьютерами через Интернет. Сейчас есть более удобные альтернативы. FTP использовался для четырех основных задач. Во-первых, его главным предназначением была загрузка файлов. Пользователи подключались к "анонимным" сайтам FTP с открытым доступом, чтобы извлекать документы, исходный код для программ и мультимедийные материалы, например фотографии или видео. (Мы входили с именем пользователя `anonymous` или `ftp`, а затем вводили адрес электронной почты в качестве пароля, чтобы сайт знал, кто использует его сеть.) Поскольку передавать большие файлы с помощью клиентов Telnet часто было рискованно, мы всегда выбирали FTP для передачи файлов между компьютерами.

Во-вторых, FTP часто использовался для анонимной загрузки файлов на сервер. Многие организации предоставляли пользователям за пределами компании возможность отправлять документы и файлы. Для этого они устанавливали FTP-серверы, которые позволяли записывать файлы в каталоги без возможности извлекать содержимое. Пользователи не могли увидеть или угадать имена файлов, отправленных другими пользователями, и получить к ним доступ раньше администраторов сайта.

В-третьих, протокол часто использовался для синхронизации целого дерева файлов между компьютерами. Пользователи могли переносить деревья каталогов из одного аккаунта в другой с помощью клиента, который поддерживал рекурсивные операции FTP, и администраторы сервера могли клонировать или устанавливать новые сервисы, не создавая их с нуля на новом компьютере. Пользователи обычно не знали, как работал сам протокол или какие операции выполнялись для передачи множества отдельных файлов через FTP. Пользователь просто нажимал на кнопку и запускал процесс.

В-четвертых, FTP использовался в своем изначальном предназначении: полноценное интерактивное управление файлами. У первых FTP-клиентов запрос в командной строке был похож на оболочку UNIX. Протокол использует ту же концепцию текущего рабочего каталога и команду `cd` для перехода из одного каталога в другой. Позже клиенты начали имитировать интерфейс Mac, выводя папки и файлы на экран. В любом случае FTP раскрывает весь свой потенциал в процессе работы в файловой системе: мы можем создавать и удалять папки, менять разрешения файлов, переименовывать файлы, выводить каталоги, загружать файлы с сервера и на сервер.

Содержание главы

- ◆ Что делать, если невозможно использовать FTP.
- ◆ Каналы коммуникации.
- ◆ FTP в Python.
- ◆ Двоичные файлы и файлы ASCII.
- ◆ Расширенная загрузка двоичных файлов с сервера.
- ◆ Отправка данных на удаленный компьютер.
- ◆ Расширенная отправка двоичных данных.
- ◆ Обработка ошибок.
- ◆ Поиск по каталогам.
- ◆ Обнаружение каталогов и загрузка в рекурсивном режиме.
- ◆ Создание и удаление каталогов.
- ◆ Безопасное использование FTP.
- ◆ Резюме.

Цель

Эта глава предназначена для тех, кто использует устаревшую систему, с которой нужно общаться через FTP с помощью программы Python, а также для тех, кто хочет узнать больше о протоколах передачи файлов и их истории.

Что делать, если невозможно использовать FTP

Почти для любых возможностей FTP сейчас есть улучшенные альтернативы. Время от времени попадают URL, начинающиеся с `ftp://`, но это бывает все реже и реже. Главный недостаток протокола — отсутствие мер безопасности. Не только файлы, но и имя пользователя и пароль отправляются открытым текстом и видны всем, кто наблюдает за сетевым трафиком. Кроме того, пользователь FTP часто подключается к серверу, выбирает рабочий каталог и выполняет несколько действий по одному сетевому соединению. Учитывая огромное количество пользователей, современные интернет-сервисы предпочитают использовать протокол HTTP (см. главу 9), где обмен данными происходит с помощью коротких и автономных запросов, а не длительных сеансов, во время которых сервер должен помнить о текущем рабочем каталоге и других аспектах.

Наконец, FTP связан с рисками для безопасности файловых систем. Раньше FTP-серверы предоставляли доступ ко всей файловой системе, позволяя пользователям выполнять команду `cd` для перехода на уровень / и смотреть, как устроена система. Да, можно было управлять сервером от имени пользователя `ftp`, чтобы запретить

пользователям доступ к максимальному числу файлов, но многие части файловой системы UNIX должны быть общедоступными, чтобы обычные пользователи могли работать с программами.

Какие есть альтернативы?

- ◆ Сегодня стандартным протоколом для передачи файлов считается HTTP (см. главу 9) с защитой в виде SSL. HTTP предлагает независимые от системы URL, не раскрывая соглашения об именовании в конкретной системе, как это делает FTP.
- ◆ Анонимная отправка файлов на сервер используется реже, но обычно предоставляется веб-страница, которая велит браузеру с помощью HTTP POST отправить файл, выбранный пользователем.
- ◆ С тех пор, как рекурсивное копирование файлов по FTP было единственным вариантом передать файлы на другой компьютер, синхронизация файлов заметно улучшилась. Теперь не нужно копировать каждый файл. Современные процедуры вроде `rsync` и `rdist` сравнивают файлы на обоих концах соединения и копируют только новые и измененные файлы. (Мы не будем рассматривать эти инструкции, поищите информацию в Интернете.) Обычные пользователи предпочитают файловые хостинги вроде Dropbox, написанного на Python.
- ◆ Единственная область, где FTP по-прежнему используется в Интернете, — полный доступ к файловой системе. Да, это небезопасно, но многие экономные провайдеры используют FTP, чтобы пользователи могли копировать мультимедийные материалы и исходный код PHP в свои онлайн-аккаунты. Современные поставщики сервисов обычно используют вместо него SFTP (см. главу 16).

Стандарт FTP описан в RFC 959 (www.faqs.org/rfcs/rfc959.html).

Каналы коммуникации

FTP использует два TCP-соединения по умолчанию. Один канал — управляющий. Он передает команды, подтверждения и коды ошибок. Второй канал передает файлы и другие данные, например список каталогов. Канал данных полностью дуплексный, т. е. файлы могут передаваться в обе стороны одновременно. На практике эта возможность используется редко.

Типичная процедура загрузки файла с FTP-сервера:

1. FTP-клиент устанавливает управляющее соединение, подключаясь к FTP-порту сервера.
2. Клиент подтверждает подлинность, передавая имя пользователя и пароль.
3. Клиент переходит к каталогу сервера, откуда он хочет загрузить файл или куда он хочет отправить файл.
4. Для того чтобы установить соединение для передачи данных, клиент начинает ожидать передачи данных на новом порту и сообщает об этом серверу.

5. Сервер подключается к порту, открытому клиентом.
6. Файл отправляется.
7. Канал данных закрывается.

На заре Интернета подключение сервера к клиенту работало эффективно, потому что практически каждая система с FTP-клиентом предоставляла общедоступный IP-адрес, а межсетевые экраны встречались редко. Сегодня все стало сложнее, и входящие соединения с устройством часто блокируются межсетевыми экранами.

FTP в Python

В Python для работы с FTP существует модуль `ftplib`. Он отвечает за технические детали установки соединений и автоматизацию базовых операций.

Если мы просто хотим загружать файлы, модуль `urllib2` (см. главу 1) поддерживает FTP и может быть удобнее. Он выполняется с URL `ftp://`. В этой главе мы рассмотрим библиотеку `ftplib`, потому что у нее есть функции для работы с FTP, которые отсутствуют в `urllib2`.

Базовый пример использования `ftplib` приводится в листинге 17.1. Приложение устанавливает соединение с удаленным сервером, выводит текущий рабочий каталог и отображает приветственное сообщение.

Листинг 17.1. Создание простого FTP-соединения

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

from ftplib import FTP

def main():
    ftp = FTP('ftp.ibiblio.org')
    print("Welcome:", ftp.getwelcome())
    ftp.login()
    print("Current working directory:", ftp.pwd())
    ftp.quit()

if __name__ == '__main__':
    main()
```

Приветственное сообщение обычно не содержит информации, полезной для программы, но если пользователь вызывает клиента напрямую, его (сообщение) можно отображать. Функция `login()` принимает несколько аргументов, включая имя пользователя, пароль и токен аутентификации, который в FTP называется *аккаунтом*. В

этом случае мы вызывали функцию без аргументов, так что пользователь вошел в систему как аноним с общим паролем. FTP-сеанс может проходить по разным каталогам, примерно как команда `cd` в оболочке может менять папки. Функция `pwd()` возвращает текущий рабочий каталог на удаленном конце соединения. Наконец, функция `quit()` выходит из соединения и закрывает его. После выполнения программы мы получаем следующие выходные данные:

```
$ ./connect.py
```

```
Welcome: 220 ProFTPD Server (Bring it on...)
```

```
Current working directory: /
```

Двоичные файлы и файлы ASCII

При работе с FTP мы должны решить, как будет обрабатываться файл — как монолитный блок двоичных данных или как текстовый файл, — чтобы локальная система могла собрать строки с помощью символа конца строки, который используется на нашей платформе. Когда мы просим Python 3 работать в текстовом режиме, он ожидает и возвращает обычные строки, а если речь о двоичных файлах, он ожидает и возвращает байтовые строки. Файл, отправленный в режиме ASCII, отправляется в программу построчно и без символов конца строки, так что нам приходится склеивать строки обратно вручную. В листинге 17.2 приводится программа Python, которая загружает и сохраняет известный текстовый файл в локальный каталог.

Листинг 17.2. Загрузка файла ASCII

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Загружаем README с удаленного компьютера и записываем его на диск.

import os
from ftplib import FTP

def main():
    if os.path.exists('README'):
        raise IOError('refusing to overwrite your README file')

    ftp = FTP('ftp.kernel.org')
    ftp.login()
    ftp.cwd('/pub/linux/kernel')

    with open('README', 'w') as f:
        def writeline(data):
            f.write(data)
            f.write(os.linesep)
```

```

ftp.retrlines('RETR README', writeline)

ftp.quit()

if __name__ == '__main__':
    main()

```

Функция `cwd()` в листинге выбирает новый рабочий каталог на удаленном компьютере. Передача запускается функцией `retrlines()`. Ее первый параметр указывает удаленную команду, которая будет выполняться (обычно `RETR`), а за ней следует имя файла.

Второй параметр — это функция, которая выполняется при извлечении каждой строки текстового файла. Если этот параметр опустить, данные будут отправляться в стандартный выходной поток. Поскольку строки передаются без символа конца строки, при выводе строк функция `writeline()` добавляет символ, который используется в вашей системе. Попробуйте выполнить эту программу. В конце вы увидите файл `README` в текущем каталоге. Двоичный файл передается почти так же, как текстовый, как показано в листинге 17.3.

Листинг 17.3. Получение двоичного файла

```

#!/usr/bin/env python3
# Programming in Python: The Basics.

import os
from ftplib import FTP

def main():
    if os.path.exists('patch8.gz'):
        raise IOError('refusing to overwrite your patch8.gz file')

    ftp = FTP('ftp.kernel.org')
    ftp.login()
    ftp.cwd('/pub/linux/kernel/v1.0')

    with open('patch8.gz', 'wb') as f:
        ftp.retrbinary('RETR patch8.gz', f.write)

    ftp.quit()

if __name__ == '__main__':
    main()

```

При запуске приложения в текущем рабочем каталоге создается файл `patch8.gz`. Функция `getrbinary()` просто передает блоки данных в указанную функцию. Это удобно, потому что функция `write()` объекта файла ожидает данные, так что специальная функция не требуется.

Расширенная загрузка двоичных файлов с сервера

Вторая функция в модуле `ftplib` для загрузки двоичных файлов — `ntransfercmd()`. У этой команды более примитивный интерфейс, но с ней удобно работать, если мы хотим узнать больше о том, что происходит во время загрузки. Это более сложная команда, которая позволяет отслеживать число переданных байтов и сообщать пользователю о ходе выполнения. В листинге 17.4 приводится пример программы с функцией `ntransfercmd()`.

Листинг 17.4. Загрузка двоичного файла с ходом выполнения

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

import os, sys
from ftplib import FTP

def main():
    if os.path.exists('linux-1.0.tar.gz'):
        raise IOError('refusing to overwrite your linux-1.0.tar.gz file')

    ftp = FTP('ftp.kernel.org')
    ftp.login()
    ftp.cwd('/pub/linux/kernel/v1.0')
    ftp.voidcmd("TYPE I")

    socket, size = ftp.ntransfercmd("RETR linux-1.0.tar.gz")
    nbytes = 0

    f = open('linux-1.0.tar.gz', 'wb')

    while True:
        data = socket.recv(2048)
        if not data:
            break
        f.write(data)
        nbytes += len(data)
```

```

print("\rReceived", nbytes, end=' ')
if size:
    print("of %d total bytes (%.1f%%)"
          % (size, 100 * nbytes / float(size)), end=' ')
else:
    print("bytes", end=' ')
sys.stdout.flush()

print()
f.close()
socket.close()
ftp.voidresp()
ftp.quit()

if __name__ == '__main__':
    main()

```

Здесь нужно обратить внимание на несколько моментов. Во-первых, мы вызываем `voidcmd()`. Он отправляет на сервер команду FTP и проверяет ошибки, но ничего не возвращает. В данном случае мы отдаем команду `TYPE I`. Так мы переходим в режим передачи изображений, чтобы FTP мог передавать двоичные файлы. В предыдущих примерах эта команда выполнялась автоматически функцией `getrbinary()` более высокого уровня, но `ntransfercmd()` так не делает.

Обратите внимание, что `ntransfercmd()` предоставляет кортеж, который включает сокет для передачи данных и размер файла. Размер указывается приблизительно и может быть меньше или значительно больше этого значения. Кроме того, если приблизительный размер файла с FTP-сервера недоступен, возвратится значение `None`.

Объект `datasock` — это стандартный сокет TCP со всеми характеристиками, описанными в первой половине книги (особенно в *главе 3*). Простой цикл в этом примере использует `recv()`, пока не прочитает все данные из соединения, при этом он записывает их на диск и выводит ход выполнения.

Относительно вывода хода выполнения в листинге 17.4 нужно обратить внимание на два аспекта. Во-первых, вместо того чтобы выводить список строк, который прокручивается, так что предыдущие значения выходят за край терминала, каждая строка начинается с символа возврата каретки `'r'`, который смещает курсор к левому краю терминала, перезаписывая предыдущую строку со статусом, так что возникает впечатление, будто процент просто увеличивается. Во-вторых, поскольку каждая инструкция `print` завершает строку пробелом, а не переходом на новую строку, строка никогда не заканчивается, так что надо вызывать `flush()` для стандартного потока вывода, чтобы информация отображалась на экране.

Обязательно нужно закрыть сокет для передачи данных и выполнять `voidresp()` после получения данных, чтобы считать код ответа команды с сервера и выдать исключение, если при передаче возникла проблема. Даже если нам не обязательно определять проблемы, мы обязаны выполнить `voidresp()`, иначе последующие команды будут завершаться сбоем, потому что выходной сокет сервера будет заблокирован, пока мы не считаем данные.

Пример выходных данных этой программы:

```
$. ./advbinarydl.py
```

```
Received 1259161 of 1259161 bytes (100.0% )
```

Отправка данных на удаленный компьютер

FTP также можно применять для отправки данных файла на удаленный компьютер, которая так же использует две операции, как и загрузка файлов с удаленного компьютера: `storbinary()` и `storlines()`. Обе операции требуют выполнить команду и передать объект в стиле файла. Функция `storbinary()` непрерывно вызывает метод `read()` для объекта, пока не получит все его содержимое, тогда как `storlines()` вызывает метод `readline()`. Эти методы, в отличие от аналогичных функций загрузки, не требуют предоставлять собственную вызываемую функцию. (Однако можно предоставить пользовательский объект в стиле файла, методы `read()` или `readline()` которого вычисляют исходящие данные в процессе передачи.)

В листинге 17.5 мы отправляем двоичный файл.

Листинг 17.5. Отправка двоичного файла

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

from ftplib import FTP
import sys, getpass, os.path

def main():
    if len(sys.argv) != 5:
        print("usage:", sys.argv[0],
              "<host> <username> <localfile> <remotedir>")
        exit(2)

    host, username, localfile, remotedir = sys.argv[1:]
    prompt = "Enter password for {} on {}: ".format(username, host)
    password = getpass.getpass(prompt)

    ftp = FTP(host)
    ftp.login(username, password)
```

```

ftp.cwd(remotedir)
with open(localfile, 'rb') as f:
    ftp.storbinary('STOR %s' % os.path.basename(localfile), f)
ftp.quit()

if __name__ == '__main__':
    main()

```

Эта программа очень похожа на предыдущие примеры. Найдите подходящий сервер, чтобы протестировать программу, потому что большинство анонимных FTP-сайтов не поддерживают отправку файлов с локального компьютера. Я просто установил старый `ftpd` на своем ноутбуке на несколько минут и выполнил тест:

```
$ python binaryul.py localhost john test.txt /tmp
```

Я ввел свой пароль по запросу (`john` — мое имя пользователя на этом компьютере). После выполнения программы я убедился, что в `/tmp` теперь есть файл `test.txt`. Помните, что FTP не шифрует и не защищает пароль, так что не пытайтесь выполнить эту программу, подключившись к другому компьютеру по сети. Просто измените `storbinary()` на `storlines()`, чтобы отправить файл в режиме ASCII.

Расширенная отправка двоичных данных

В листинге 17.6 мы видим, как отправить файлы вручную с помощью вызова `nttransfercmd()`. Это похоже на усложненную версию, которую мы рассматривали для загрузки.

Листинг 17.6. Отправка файлов блоками

```

#!/usr/bin/env python3
# Programming in Python: The Basics.

import os, sys
from ftplib import FTP

def main():
    if os.path.exists('linux-1.0.tar.gz'):
        raise IOError('refusing to overwrite your linux-1.0.tar.gz file')

    ftp = FTP('ftp.kernel.org')
    ftp.login()
    ftp.cwd('/pub/linux/kernel/v1.0')
    ftp.voidcmd("TYPE I")

```

```

socket, size = ftp.ntransfercmd("RETR linux-1.0.tar.gz")
nbytes = 0

f = open('linux-1.0.tar.gz', 'wb')

while True:
    data = socket.recv(2048)
    if not data:
        break
    f.write(data)
    nbytes += len(data)
    print("\rReceived", nbytes, end=' ')
    if size:
        print("of %d total bytes (%.1f%%)"
              % (size, 100 * nbytes / float(size)), end=' ')
    else:
        print("bytes", end=' ')
    sys.stdout.flush()

print()
f.close()
socket.close()
ftp.voidresp()
ftp.quit()

if __name__ == '__main__':
    main()

```

После передачи нужно вызвать `datasock.close()`, чтобы закрыть сокет, иначе сервер будет ждать дальнейшие данные. Теперь можно отправлять файлы с отслеживанием хода выполнения:

```
$ python binaryul.py localhost john patch8.gz /tmp
```

```
Enter password for john on localhost:
```

```
Sent 6408 of 6408 bytes (100.0%)
```

Обработка ошибок

Когда возникает ошибка, `ftplib`, как и большинство модулей Python, выдаст исключение. У него есть свой набор исключений, а также он вызывает исключения `socket.error` и `IOError`. Он предоставляет кортеж с именем `ftplib.all_errors` и всеми исключениями, которые `ftplib` может вызывать. Это удобно при написании блока

try...except. Один из недостатков простой функции `getrbinary()` — для ее эффективного использования нам почти всегда приходится открывать файл локально, прежде чем начинать передачу на удаленный компьютер. Если нужный файл не существует при выполнении команды или команда `RETR` не выполняется успешно, закройте и удалите созданный локальный файл, чтобы не захламлять файловую систему файлами нулевой длины. Мы можем использовать функцию `ntransfercmd()`, чтобы проверять наличие ошибок, прежде чем открыть локальный файл. Эти требования соблюдаются в листинге 17.6: если `ntransfercmd()` завершается сбоем, программа завершает работу до того, как локальный файл откроется.

Поиск по каталогам

FTP предлагает два метода узнавать о файлах и каталогах сервера. Они реализованы как методы `nlst()` и `dir()` в `ftplib`. Метод `nlst()` возвращает список записей в каталоге, который включает все файлы и каталоги. Правда, возвращается только список имен, без информации о том, какие записи являются файлами или папками, какие размеры у файлов и т. д.

Более сложная функция `dir()` извлекает удаленный список каталогов. Формат списка определяется системой, но обычно включает имя, размер и тип файла, а также дату его изменения. Как правило, это результат одной из двух команд оболочки на серверах UNIX:

```
$ ls -l
```

```
$ ls -la
```

Выходные данные `dir` могут использоваться серверами Windows. Эти данные могут быть информативны для пользователя, но программе сложно использовать их в таком формате. Некоторые клиенты, которым нужна эта информация, используют средства парсинга для разных форматов, получаемых от `ls` и `dir`. Другие выполняют парсинг только для одного используемого формата. В листинге 17.7 мы вызываем `nlst()`, чтобы получить информацию о каталогах.

Листинг 17.7. Получение информации о каталогах

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

from ftplib import FTP

def main():
    ftp = FTP('ftp.ibiblio.org')
    ftp.login()
    ftp.cwd('/pub/academic/astronomy/')
    entries = ftp.nlst()
    ftp.quit()
```

```

    print(len(entries), "entries:")
    for entry in sorted(entries):
        print(entry)

if __name__ == '__main__':
    main()

```

При выполнении этой программы мы получим следующие выходные данные:

\$ python nlst.py

```

13 entries:
INDEX
README
ephem_4.28.tar.Z
hawaii_scope
incoming
jupitor-moons.shar.Z
lunar.c.Z
lunisolar.shar.Z
moon.shar.Z
planetary
sat-track.tar.Z
stars.tar.Z
xephem.tar.Z

```

Тот же список файлов можно получить, если вручную войти на хост с помощью FTP-клиента. Если использовать другую команду для вывода списка файлов, как в листинге 17.8, выходные данные будут иными.

Листинг 17.8. Получение удобной информации о каталогах

```

#!/usr/bin/env python3
# Programming in Python: The Basics.

from ftplib import FTP

def main():
    ftp = FTP('ftp.ibiblio.org')
    ftp.login()
    ftp.cwd('/pub/academic/astronomy/')
    entries = []
    ftp.dir(entries.append)
    ftp.quit()

```

```

print(len(entries), "entries:")
for entry in entries:
    print(entry)

if __name__ == '__main__':
    main()

```

Информации не стало больше, но имена файлов представлены в формате, удобном для автоматизированной обработки, — это простой список имен файлов. Сравните выходные данные из листинга 17.8, где используется `dir()`, со списком имен файлов, который мы получили раньше:

\$ python dir.py

13 entries:

```

-rw-r--r-- 1 (?) » (?) » »      750 Feb 14 1994 INDEX
-rw-r--r-- 1 root » bin » »      135 Feb 11 1999 README
-rw-r--r-- 1 (?) » (?) » » 341303 Oct  2 1992 ephen_4.28.tar.Z
drwxr-xr-x 2 (?) » (?) » »      4096 Feb 11 1999 hawaii_scope
drwxr-xr-x 2 (?) » (?) » »      4096 Feb 11 1999 incoming
-rw-r--r-- 1 (?) » (?) » »      5983 Oct  2 1992 jupiter-moons.shar.Z
-rw-r--r-- 1 (?) » (?) » »      1751 Oct  2 1992 lunar.c.Z
-rw-r--r-- 1 (?) » (?) » »      8078 Oct  2 1992 lunisolar.shar.Z
-rw-r--r-- 1 (?) » (?) » »      64209 Oct  2 1992 moon.shar.Z
drwxr-xr-x 2 (?) » (?) » »      4096 Jan  6 1993 planetary
-rw-r--r-- 1 (?) » (?) » » 129969 Oct  2 1992 sat-track.tar.Z
-rw-r--r-- 1 (?) » (?) » »      16504 Oct  2 1992 stars.tar.Z
-rw-r--r-- 1 (?) » (?) » » 410650 Oct  2 1992 xephem.tar.Z

```

Метод `dir()` принимает функцию и вызывает ее для каждой строки, предоставляя список каталогов частями, примерно как `getrlines()` для отдельных файлов. Здесь используется метод `append()` списка Python `entries`.

Обнаружение каталогов и загрузка в рекурсивном режиме

Как отличать каталоги от файлов, если невозможно предсказать, какую информацию предоставит FTP-сервер с помощью команды `dir()`? Это важный шаг при загрузке больших деревьев с сервера.

Единственный верный способ, как показано в листинге 17.9, — пытаться вставить `cwd()` в каждое имя, возвращенное вызовом `nlst()`, и если получится, значит, этот объект — каталог. Этот пример ничего не загружает. Он выводит на экран папки, по которым проходит, чтобы не усложнять выходные данные и не заполнять диск данными из примеров.

Листинг 17.9. Рекурсия по каталогам

```
#!/usr/bin/env python3
# Programming in Python: The Basics.

from ftplib import FTP, error_perm

def walk_dir(ftp, dirpath):
    original_dir = ftp.pwd()
    try:
        ftp.cwd(dirpath)
    except error_perm:
        return # игнорируйте не каталоги и те каталоги, в которые мы не можем войти
    print(dirpath)
    names = sorted(ftp.nlst())
    for name in names:
        walk_dir(ftp, dirpath + '/' + name)
    ftp.cwd(original_dir) # return to cwd of our caller

def main():
    ftp = FTP('ftp.kernel.org')
    ftp.login()
    walk_dir(ftp, '/pub/linux/kernel/Historic/old-versions')
    ftp.quit()

if __name__ == '__main__':
    main()
```

Поначалу эта программа будет выполняться небыстро, т. к. в каталоге архива ядра Linux старой версии оказалось немало файлов, но через несколько десятков секунд мы увидим следующее дерево каталогов:

\$ python recursedl.py

```
/pub/linux/kernel/Historic/old-versions
/pub/linux/kernel/Historic/old-versions/impure
/pub/linux/kernel/Historic/old-versions/old
/pub/linux/kernel/Historic/old-versions/old/corrupt
/pub/linux/kernel/Historic/old-versions/tytso
```

Мы можем дополнить список папок, выводя все файлы, которые рекурсивный процесс медленно обнаруживает, добавив несколько инструкций `print`. Более того, мы можем загружать сами файлы в нужные каталоги, созданные локально, добавив еще несколько строк кода. Вся нужная логика для рекурсивной загрузки уже про-

писана в листинге 17.9, но единственный верный способ определить, является ли запись каталогом, к которому у нас есть доступ, — использовать `cwd()` для каждой записи.

Создание и удаление каталогов

Наконец, FTP позволяет удалять файлы, а также создавать и удалять каталоги. Эти сложные вызовы описаны в документации по `ftplib`:

- ◆ `delete(filename)` удаляет файл с сервера;
- ◆ `mkd(dirname)` создает новый каталог;
- ◆ `rmd(dirname)` удаляет каталог; большинство систем требуют, чтобы каталог поначалу был пустым;
- ◆ `rename(oldname, newname)` работает почти так же, как UNIX-команда `mv`: файл переименовывается, если оба имени находятся в одном каталоге, но если указывается имя в другом каталоге, файл перемещается.

Эти инструкции, как и остальные операции FTP, выполняются так, будто мы вошли в командную строку удаленного сервера с тем же именем пользователя, под которым мы вошли на FTP-сервер. Благодаря этим нескольким командам FTP может использоваться для поддержки браузерных приложений, в которых пользователи могут перетаскивать файлы и папки между локальной системой и удаленным хостом.

Безопасное использование FTP

В начале этой главы я упомянул, что есть гораздо более эффективные альтернативы FTP, особенно SFTP поверх SSH (см. главу 16), но стоит отметить, что некоторые FTP-серверы поддерживают TLS-шифрование (см. главу 6), и модуль Python `ftplib` предоставляет эту защиту, если необходимо. Создайте FTP-соединение с классом `FTP_TLS` вместо `FTP`, чтобы использовать TLS. Таким образом вы защитите имя пользователя и пароль, а также весь канал команд FTP, от чужих глаз. FTP-соединение для передачи тоже будет защищено, если вызвать метод `prot_p()` этого класса (не принимает аргументы). Метод `prot_c()` возвращает поток данных в обычный режим, если мы хотим передавать данные без шифрования. При этом, если мы используем класс `FTP_TLS`, команды будут защищены. Больше информации об этом расширении FTP см. в документации по стандартной библиотеке Python: <http://docs.python.org/3/library/ftplib.html>.

Резюме

FTP позволяет передавать файлы между клиентом и удаленным FTP-сервером. Несмотря на то что протокол FTP считается небезопасным и устаревшим по сравне-

нию с улучшенными альтернативами вроде SFTP, иногда попадают сервисы и компьютеры, которые требуют его использования. Пакет `ftplib` в Python используется для взаимодействия с FTP-серверами.

FTP поддерживает передачу двоичных файлов и файлов в кодировке ASCII. Текстовые файлы обычно отправляются в формате ASCII, который позволяет менять окончания строк при передаче. Для остального используется передача в двоичном режиме. Функция `getrlines()` загружает файл в режиме ASCII, а функция `getrbinary()` загружает файл в двоичном режиме. Мы также можем отправлять файлы на удаленный сервер. Функции `storlines()` и `storbinary()` отправляют файлы в формате ASCII и в двоичном виде соответственно. Для передачи двоичных файлов с удаленного сервера и на удаленный сервер используйте функцию `ntransfercmd()`. Она позволяет лучше контролировать процесс передачи и часто применяется для отображения хода выполнения передачи. При ошибках модуль `ftplib` вызывает исключения. Для того чтобы перехватить возможные ошибки, у нас есть кортеж `ftplib.all_errors`.

На том конце можно использовать `cwd()` для перехода в определенный каталог. Команда `nlst()` выдает простой список всех файлов и каталогов в данном каталоге. Команда `dir()` выдает более подробный список, но он отформатирован для сервера. Даже если у нас есть только `nlst()`, мы можем определить, является ли каждая запись файлом или каталогом, если попробуем изменить ее с помощью `cwd()` и проверим, вызовет ли это ошибку. В следующей главе мы перейдем от простой передачи файлов к более общей задаче выполнения удаленных процедур на другом сервере и получения типизированных данных вместо простых строк.

RPC — удаленный вызов процедур

RPC (Remote Procedure Call — удаленный вызов процедур) позволяет вызывать функции в другом процессе или на удаленном сервере, используя тот же синтаксис, с помощью которого мы вызвали бы процедуру в локальном API или библиотеке. Это удобно в двух ситуациях:

- ◆ если нам нужны данные, доступные только на другом жестком диске или в другой сети, интерфейс RPC позволяет легко отправлять запросы в другую систему и получать ответ;
- ◆ программа выполняет много задач, и мы хотим распределить их по нескольким компьютерам, делая вызовы по сети, но не меняя код вызова.

Изначально системы удаленных процедур создавались на низкоуровневых языках вроде C. Когда одна функция C вызывала другую, по сети передавались байты, очень похожие на байты, уже отправленные в стек процессора. Вызовы RPC нельзя было делать, не зная заранее, как будут сериализованы данные, так же как программа на C не могла безопасно вызвать функцию библиотеки без файла заголовков, который указывал, как именно расположить входные данные метода в памяти. Любые ошибки приводили к сбою. На самом деле, полезная нагрузка RPC выглядела как блок двоичных данных, отформатированных с помощью модуля Python struct, о котором мы говорили в *главе 5*. Современные компьютеры и сети работают достаточно быстро, так что мы можем потратить немного памяти и производительности на более надежные протоколы, которые требуют меньше координации между двумя взаимодействующими фрагментами кода.

Более ранние протоколы RPC отправляли бы поток байтов, который выглядит примерно следующим образом:

```
0, 0, 0, 1, 64, 36, 0, 0, 0, 0, 0, 0
```

Получателю нужно было расшифровать эти 12 байт как "целое число 1" и "вещественное число 10.0", узнав, что параметры функции — 32-разрядное целое число и 64-разрядное вещественное число. Современные же протоколы RPC используют самодокументируемые форматы вроде XML, которые устроены таким образом, что почти невозможно интерпретировать аргументы как что-то иное, кроме целого и вещественного числа.

```
<params>
```

```
<param><value><i4>41</i4></value></param>
```

```
<param><value><double>10.</double></value></param>
</params>
```

12 байт двоичных данных превратились в 108 байт протокола, которые отправитель должен создать, а получатель — интерпретировать, для чего требуются сотни циклов центрального процессора, но эти дополнительные затраты считаются оправданными, ведь они устраняют неоднозначность. Разумеется, можно использовать более современный формат, например JSON (JavaScript Object Notation — нотация объектов JavaScript), чтобы передать эту пару значений в более коротком виде.

```
[1,10.0]
```

Мы видим, что понятное текстовое представление заменяет традиционный подход с передачей необработанных двоичных данных, значение которых необходимо было интерпретировать заранее в обоих случаях. В чем уникальность протоколов RPC? Ведь выбор формата данных, отправка запроса и получение ответа — все это применимо к любому значимому сетевому протоколу. Для того чтобы использовать два примера из предыдущих глав, HTTP и SMTP должны сериализовать данные и указать форматы сообщений. У протокола RPC есть три особенности.

Во-первых, у протокола RPC нет строгой семантики. HTTP извлекает документы, SMTP отправляет письма, а RPC не придает получаемым данным особого значения. Он просто поддерживает основные типы данных: целые и вещественные числа, строки и списки. За значение вызовов отвечают API, которые мы создаем с протоколом RPC.

Во-вторых, механизмы RPC позволяют вызывать методы, не определяя их. Читая спецификацию протокола, который служит определенной цели, например HTTP или SMTP, мы видим, что он описывает ограниченный набор базовых операций, например GET и PUT для HTTP и EHLO и MAIL для SMTP. Механизмы RPC, с другой стороны, позволяют нам определять команды или функции, которые будет поддерживать сервер. Они не указывают их заранее.

В-третьих, при использовании RPC код клиента и сервера похожи на любой другой код, который использует вызовы функций. Единственная особенность — нужно проявлять некоторую осторожность в отношении передаваемых объектов. Это могут быть числа, текст и списки, но обычно не активные объекты вроде открытых файлов, если только вы не знаете, что объект представляет удаленный сервер. Тип передаваемых параметров может быть ограничен, но вызовы функций выглядят "нормально" и не требуют дополнительного декорирования или дополнения для передачи по сети.

Содержание главы

- ◆ Характеристики RPC.
- ◆ XML-RPC.
- ◆ JSON-RPC.
- ◆ Самодокументируемые данные.
- ◆ Объекты: Pyro и RPyC.

- ◆ Пример RPyC.
- ◆ Очереди сообщений, RPC и веб-фреймворки.
- ◆ Восстановление после ошибок в сети.
- ◆ Резюме.

Цель

Эта глава посвящена RPC и особенностям работы с ним.

Характеристики RPC

Протоколы RPC имеют определенные характеристики и различия, о которых нужно помнить при выборе и развертывании RPC-клиента или сервера. С их помощью мы вызываем функции и методы на другом сервере, но так, как если бы делали это на локальном компьютере.

Во-первых, у каждого протокола RPC есть ограничения на тип передаваемых данных. В реальности, поскольку эти протоколы должны работать с разными языками программирования, самые общие механизмы RPC обычно имеют больше всего ограничений, потому что поддерживают только возможности, существующие во всех языках.

В результате самые часто используемые протоколы предлагают всего несколько числовых и строковых типов, а также одну последовательность или список и что-то вроде структуры или ассоциативного массива. Поскольку мало языков предоставляют аргументы с ключевыми словами, многие программисты на Python недовольны тем, что поддерживаются только позиционные аргументы.

Когда механизм RPC привязан к языку программирования, он может поддерживать больше параметров. В некоторых обстоятельствах можно передавать даже активные объекты, если протокол может придумать, как восстановить их на удаленном конце. В этой ситуации можно передавать по сети только объекты на основе активных ресурсов операционной системы, например открытые файлы, действующие сокеты или общую область памяти.

Во-вторых, сервер может сообщать об исключении при выполнении удаленной функции. В таких случаях клиентская библиотека RPC обычно выдает исключение, чтобы сообщить вызывающему объекту о проблеме. Разумеется, активные стековые кадры вроде тех, которые Python предоставляет обработчикам исключений, нельзя возвращать. В конце концов, стековый кадр обычно относится к модулям, которые не существуют в клиентской программе. Когда происходит сбой вызова к серверу, нужно как минимум выдавать исключение с сообщением об ошибке на стороне клиента.

В-третьих, многие механизмы RPC поддерживают интроспекцию и позволяют клиентам просматривать список вызовов, поддерживаемых сервисом RPC, и принимаемых им аргументов. Некоторые протоколы RPC требуют, чтобы клиент и сервер

обменялись подробными документами, где определяются поддерживаемая ими библиотека или API. Другие разрешают клиенту только извлечь список имен функций и типов аргументов. Третьи не предоставляют такой информации вовсе. Python не очень хорошо поддерживает интроспекцию, потому что, в отличие от языков со статической типизацией, не знает, какие типы параметров программист подразумевал для каждой функции.

В-четвертых, у каждого механизма RPC должна быть схема адресации, которая позволяет подключиться к определенному удаленному API. Некоторые из этих систем довольно сложные и могут даже подключать нас к нужному серверу в сети для выполнения определенных операций, не требуя знания его имени заранее. Другие подходы требуют IP-адрес, номер порта или URL сервиса, который мы хотим использовать. Вместо того чтобы создавать собственную систему адресации, эти технологии раскрывают имеющуюся схему адресации в сети.

Наконец, когда вызовы RPC исходят от различных клиентских программ с разными учетными данными, некоторые RPC предлагают аутентификацию, контроль доступа и даже полное обезличивание некоторых аккаунтов пользователей. Правда, эти возможности не всегда доступны. Более того, в самых популярных протоколах RPC они отсутствуют. Простые схемы RPC используют базовый протокол вроде HTTP для аутентификации, и мы сами должны позаботиться о паролях, открытых ключах или ограничениях межсетевых экранов, которые требуются для защиты низкоуровневого протокола, если мы хотим защитить сервис RPC от неавторизованного доступа.

XML-RPC

Давайте для начала рассмотрим встроенные в Python возможности XML-RPC. Может показаться, что это не лучший выбор для первого примера, ведь XML славится своей громоздкостью, а XML-RPC все реже используется в новых сервисах.

Зато стандартная библиотека Python поддерживает XML-RPC, и это был один из первых протоколов RPC в Интернете, который нативно работал на базе HTTP и не требовал собственного сетевого протокола. Поэтому в примерах данной главы мы не будем использовать сторонние модули. Хотя это ограничивает возможности RPC-сервера, так примеры будут проще для понимания.

ПРОТОКОЛ XML-RPC

Цель: вызов удаленных процедур.

Стандарт: www.xmlrpc.com/spec.

Базовый протокол: HTTP.

Типы данных: int; float; unicode; list; dict с ключами в Unicode; нестандартные расширения, datetime и None.

Библиотеки: xmlrpclib, SimpleXMLRPCServer, DocXMLRPCServer.

Если когда-нибудь вы работали с обычным XML, то знаете, что у него нет семантики типов данных. Он представляет только элементы, которые содержат другие элементы, текстовые строки и свойства в виде текстовых строк, но не целые числа. В результате стандарт XML-RPC должен добавлять семантику в простой формат документа XML, чтобы описать, как числа должны отображаться при преобразовании в размеченный текст. Стандартная библиотека Python упрощает запись клиента или сервера XML-RPC. В листинге 18.1 приводится простой сервер, который запускает веб-сервер на порте 7001 и отслеживает входящие интернет-соединения.

Листинг 18.1. Сервер XML-RPC

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Сервер XML-RPC.

import operator, math
from xmlrpc.server import SimpleXMLRPCServer
from functools import reduce

def main():
    server = SimpleXMLRPCServer(('127.0.0.1', 7001))
    server.register_introspection_functions()
    server.register_multicall_functions()
    server.register_function(addtogether)
    server.register_function(quadratic)
    server.register_function(remote_repr)
    print("Server ready")
    server.serve_forever()

def addtogether(*things):
    """Сложите вместе все, что есть в списке `things`."""
    return reduce(operator.add, things)

def quadratic(a, b, c):
    """Определите значения `x`, удовлетворяющие: `a` * x*x + `b` * x + c == 0"""
    b24ac = math.sqrt(b*b - 4.0*a*c)
    return list(set([ (-b-b24ac) / 2.0*a,
                      (-b+b24ac) / 2.0*a ]))

def remote_repr(arg):
    """Возвращает рендеринг `repr()` предоставленного `arg`."""
    return arg

if __name__ == '__main__':
    main()
```

Нам не нужно выделять целый порт для RPC-сервиса вроде этого, потому что сервис XML-RPC находится по одному URL веб-сайта. Его можно включить в стандартное веб-приложение, которое предоставляет различные страницы и даже независимые RPC-сервисы по другим URL. Если мы можем выделить целый порт, можно использовать сервер XML-RPC в Python, чтобы легко создать веб-сервер, который обрабатывает только запросы XML-RPC.

Три примера функции, предоставленные сервером через XML-RPC (добавленные к сервису RPC с помощью вызовов `register_function()`), — это стандартные процедуры Python. В этом заключается вся суть XML-RPC: он позволяет предоставлять процедуры для вызова по сети, не переписывая их, как если бы они были обычными функциями в нашей программе.

Модуль `SimpleXMLRPCServer` в стандартной библиотеке Python позволяет создать простейший сервер XML-RPC. Он не может обслуживать другие веб-страницы, он не понимает аутентификацию HTTP, мы не можем просить его предоставить TLS без создания производных классов и написания дополнительного кода. Однако этот модуль демонстрирует основные преимущества и ограничения RPC с помощью всего нескольких строк кода. В дополнение к трем вызовам, которые регистрируют функции, мы делаем два дополнительных вызова конфигурации. Каждый из них поддерживает необязательную, но распространенную возможность серверов XML-RPC — интроспекцию, которая позволяет клиенту узнать, какие вызовы RPC поддерживает сервер, а также возможность вызывать несколько функций за один круговой путь.

Прежде чем переходить к следующим трем программам, нужно запустить этот сервер из командного окна:

```
$ python xmlrpc_server.py
```

```
Server ready
```

Теперь сервер принимает соединения на localhost, порт 7001. К этому TCP-серверу применяются все стандартные правила адресации, которые мы рассматривали в главах 2 и 3, так что нам придется подключиться к нему из другой командной строки в той же системе, если только мы не изменим код, чтобы привязать интерфейс не к localhost. Откройте новое командное окно и приготовьтесь выполнить следующие три листинга. Сначала давайте посмотрим, работает ли возможность интроспекции, которую мы включили для этого сервера. Это необязательно, и не все сервисы XML-RPC предлагают такую возможность.

В листинге 18.2 мы видим интроспекцию с точки зрения клиента.

Листинг 18.2. Какие функции поддерживает сервер XML-RPC?

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Клиент XML-RPC.
```

```
import xmlrpc.client
```

```
def main():
    proxy = xmlrpc.client.ServerProxy('http://127.0.0.1:7001')
    print('Here are the functions supported by this server:')
    for method_name in proxy.system.listMethods():
        if method_name.startswith('system.'):
            continue
    signatures = proxy.system.methodSignature(method_name)
    if isinstance(signatures, list) and signatures:
        for signature in signatures:
            print('%s(%s)' % (method_name, signature))
    else:
        print('%s(...)' % (method_name,))
        method_help = proxy.system.methodHelp(method_name)
        if method_help:
            print(' ', method_help)

if __name__ == '__main__':
    main()
```

Мало того, что механизм интроспекции является необязательным, так он даже не упомянут в спецификации XML-RPC. С его помощью клиенты могут вызывать специальные методы, которые начинаются со строки `system`, чтобы их можно было отличить от обычных методов. Эти методы позволяют узнать о других доступных вызовах. Давайте для начала вызовем `listMethods()`. Если интроспекция поддерживается, мы получим список дополнительных имен методов.

В этом примере давайте проигнорируем системные методы и выведем информацию только об остальных. Мы попытаемся получить сигнатуру каждого метода, чтобы узнать, какие аргументы и типы данных он принимает. Сервер написан на Python, а значит, типы не объявляются, и невозможно узнать, какие типы данных ожидает функция.

\$ python xmlrpc_introspect.py

Here are the functions supported by this server:

`concatenate(...)`

Сложите вместе все, что есть в списке ``things``.

`quadratic(...)`

Определите значения ``x``, удовлетворяющие: ``a` * x*x + `b` * x + c == 0`

`remote_repr(...)`

Возвращает рендеринг ``repr()`` предоставленного ``arg``.

Однако мы видим, что хотя типы аргументов не предоставляются, мы получили строки документации. По сути, SimpleXMLRPCServer извлек и вернул строки `docstring` для функции. В реальности интроспекция используется в двух сценариях.

Если мы создаем программу, которая использует сервис XML-RPC, документация сервиса может содержать информацию, понятную пользователю. Если мы разрабатываем клиента, который будет взаимодействовать с другими такими же сервисами XML-RPC, но отличаться от них предлагаемыми методами, вызов `listMethods()` поможет понять, какие команды поддерживаются серверами. Цель сервиса RPC — сделать так, чтобы вызовы функций выглядели максимально естественно на целевом языке. Более того, как мы видим в листинге 18.3, модуль `xmlrpc.client` из стандартной библиотеки предоставляет прокси-объект для вызова функций сервисов. Эти вызовы похожи на вызовы локальных методов.

Листинг 18.3. Запросы XML-RPC

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# Programming in Python: The Basics.
# Клиент XML-RPC.

import xmlrpc.client

def main():
    proxy = xmlrpc.client.ServerProxy('http://127.0.0.1:7001')
    print(proxy.addtogether('x', 'y', 'z'))
    print(proxy.addtogether(20, 30, 4, 1))
    print(proxy.quadratic(2, -4, 0))
    print(proxy.quadratic(1, 2, 1))
    print(proxy.remote_repr((1, 2.0, 'three')))
    print(proxy.remote_repr([1, 2.0, 'three']))
    print(proxy.remote_repr({'name': 'john',
                              'data': {'age': 49, 'sex': 'M'}}))
    print(proxy.quadratic(1, 0, 1))

if __name__ == '__main__':
    main()
```

Если выполнить этот код для примера сервиса, мы получим выходные данные, из которых многое узнаем об XML-RPC и RPC. Почти все вызовы работают без проблем. Вызовы в листинге, а также сами функции из листинга 18.1 выглядят как обычный Python-код. В них нет ничего, что относилось бы к передаче по сети:

```
$ python xmlrpc_client.py
```

```
xȳz
```

```
55
```

```
[0.0, 8.0]
```

```
[-1.0]
[1, 2.0, 'three']
[1, 2.0, 'three']
```

```
{'data': {'age': [49], 'sex': 'M'}, 'name': 'john'}
```

```
Traceback (most recent call last):
```

```
...
```

```
xmlrpclib.Fault: <Fault 1: "<type 'exceptions.ValueError':>:math domain error">
```

Правда, нужно обратить внимание на несколько моментов. Во-первых, у XML-RPC нет ограничений по типам предоставляемых аргументов. Мы можем вызвать `addtogether()` со строками или числами и передать любое количество параметров. Самому протоколу безразлично, сколько аргументов функция должна принимать и какие типы данных должны быть у этих аргументов. Если бы это был язык, которому это важно, или даже метод Python, который не принимает списки аргументов переменной длины, на том конце могло возникнуть исключение. Однако это была бы проблема языка, а не протокола XML-RPC.

Во-вторых, в Python и подобных языках вызовы функции XML-RPC могут принимать много аргументов, но возвращать только одно результирующее значение. Даже если это значение будет сложной структурой данных, оно будет возвращаться как единый экземпляр. Протоколу не важно, какая у результата форма и какой размер. Количество элементов, возвращенных функцией `quadratic()` (да, мне надоели стандартные математические операции `add()` и `subtract()` в примерах для XML-RPC), может спокойно меняться, не нарушая сетевую логику.

В-третьих, многообразие типов данных Python необходимо ограничить до набора, поддерживаемого XML-RPC. Например, XML-RPC поддерживает только одну последовательность — список. Когда мы передаем в `remote_rep()` кортеж из трех элементов, сервер получает список из трех элементов, а не кортеж. Когда протоколы RPC работают с определенным языком, это стандартная возможность. Если тип не поддерживается, нужно передать его в виде другой структуры данных (в этом случае кортеж преобразуется в список) или выдать исключение, чтобы сообщить, что этот тип параметра невозможно передать.

В-четвертых, в XML-RPC сложные структуры данных могут быть рекурсивными. Аргументы не должны содержать только один уровень сложного типа данных. Как видите, мы можем передать словарь, одним из значений которого будет другой словарь.

В-пятых, исключение, вызванное функцией на сервере, успешно вернулось по сети и отобразилось локально для клиента в виде экземпляра `xmlrpclib.Fault`. Экземпляр указывает имя удаленного исключения, а также сообщение об ошибке. Исключения XML-RPC всегда будут иметь такую структуру, независимо от языка, на котором реализованы функции сервера.

Обратная трассировка не очень информативна. Да, мы видим, какой вызов в коде привел к проблеме, но внутренние уровни стека представляют собой просто код `xmlrpclib`. Мы рассмотрели базовые возможности и ограничения протокола XML-

RPC. Для того чтобы узнать больше, изучите документацию по модулю для клиента и сервера в стандартной библиотеке Python. Передав дополнительные аргументы классу `ServerProxy`, мы можем узнать, как использовать TLS и аутентификацию.

Однако стоит упомянуть еще об одной возможности: совершение нескольких вызовов за один круговой путь по сети, если сервер поддерживает такой подход, как в листинге 18.4.

Листинг 18.4. Несколько вызовов с помощью протокола XML-RPC

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Клиент XML-RPC выполняет несколько вызовов.

import xmlrpc.client

def main():
    proxy = xmlrpc.client.ServerProxy('http://127.0.0.1:7001')
    multical = xmlrpc.client.MultiCall(proxy)
    multical.addtogether('a', 'b', 'c')
    multical.quadratic(2, -4, 0)
    multical.remote_repr([1, 2.0, 'three'])
    for answer in multical():
        print(answer)

if __name__ == '__main__':
    main()
```

При выполнении этого скрипта проверьте окно команд сервера, чтобы убедиться, что для ответа на все три вызова функций требуется только один запрос HTTP:

```
localhost - - [09/Oct/2019 00:16:19] "POST /RPC2 HTTP/1.0" 200 -
```

Кстати, запись сообщений в журнал можно отключить с помощью параметров `SimpleXMLRPCServer`. Можно изучить документацию и настроить сервер и клиента иначе, но по умолчанию они используют URL `/RPC2`. Прежде чем мы перейдем к следующему механизму RPC, давайте рассмотрим еще несколько моментов.

- ◆ Существует два дополнительных типа данных, без которых сложно обойтись, поэтому многие протоколы XML-RPC предлагают их: даты и `None` (`null` или `nil` в других языках). Клиент и сервер на Python могут отправлять и получать нестандартные значения.
- ◆ XML-RPC не поддерживает именованные аргументы, потому что с ними работает мало языков.
- ◆ Наконец, помните, что словари можно передавать, только если все их ключи имеют строковые значения, обычные или в Unicode. Некоторые сервисы обходят

это ограничение, разрешая передавать словарь как последний аргумент функции или удаляя позиционные аргументы полностью и используя один аргумент в виде словаря для каждой функции, которая указывает все свои параметры по имени. Подробнее см. в разд. "Самодокументируемые данные" далее в этой главе.

Цель протокола RPC вроде XML-RPC — взять на себя всю специфику передачи по сети, чтобы мы могли сосредоточиться исключительно на написании кода, но мы все же должны понимать, как наши вызовы будут выглядеть в сети. Пример начального вызова `quadratic()` в клиентской программе:

```
<?xml version='1.0'?>
<methodCall>
  <methodName>quadratic</methodName>
  <params>
    <param>
      <value><int>2</int></value>
    </param>
    <param>
      <value><int>-4</int></value>
    </param>
    <param>
      <value><int>0</int></value>
    </param>
  </params>
</methodCall>
```

Ответ на предыдущий вызов:

```
<?xml version='1.0'?>
<methodResponse>
  <params>
    <param>
      <value><array><data>
        <value><double>0.0</double></value>
        <value><double>8.0</double></value>
      </data></array></value>
    </param>
  </params>
</methodResponse>
```

JSON-RPC

JSON сериализует структуры данных в строке с помощью синтаксиса JavaScript. Это значит, что теоретически с помощью функции `eval()` в браузере строки JSON можно преобразовать обратно в данные. (Поскольку нежелательно выполнять эту

функцию с ненадежными данными, большинство программистов используют официальный парсер JSON, а не существующую совместимость с JavaScript.) Этот механизм вызова удаленной процедуры может сократить объем данных, а также упростить код парсеров и библиотек с помощью синтаксиса, специально созданного для таких данных, в отличие от многословного языка разметки XML.

ПРОТОКОЛ JSON-RPC

Цель: вызов удаленных процедур.

Стандарт: <http://json-rpc.org/wiki/specification>.

Базовый протокол: HTTP.

Типы данных: int; float; unicode; **list**; **dict** с ключами в Unicode; None.

Библиотеки: многие сторонние библиотеки, включая jsonrpclib.

Поскольку стандартная библиотека Python не поддерживает JSON-RPC, ищите сторонние альтернативы. Например, в PyPI есть jsonrpclib-pelix — одна из первых библиотек с поддержкой Python 3. Вы сможете протестировать сервер и клиента из листингов 18.5 и 18.6, если развернете их в виртуальном окружении (см. главу 1).

Листинг 18.5. Сервер JSON-RPC

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Сервер JSON-RPC, для которого требуется "pip install jsonrpclib-pelix".

from jsonrpclib.SimpleJSONRPCServer import SimpleJSONRPCServer

def lengths(*args):
    """Измеряет длину каждого входного аргумента.
    Для N аргументов возвращает список из N списков в виде [len(arg), arg],
    который указывает длину входного аргумента и повторяет сам аргумент. """

    results = []
    for arg in args:
        try:
            arglen = len(arg)
        except TypeError:
            arglen = None
        results.append((arglen, arg))
    return results
```

```
def main():
    server = SimpleJSONRPCServer(('localhost', 7002))
    server.register_function(lengths)
    print("Starting server")
    server.serve_forever()

if __name__ == '__main__':
    main()
```

Код сервера очень простой, каким он и должен быть при использовании протокола RPC. Как и с XML-RPC, мы просто указываем функции, которые хотим предоставлять по сети, и они становятся доступными для запроса. (Также можно передать объект, и все его методы будут зарегистрированы на сервере одновременно.)

Листинг 18.6. Клиент JSON-RPC

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Клиент JSON-RPC, которому требуется "pip install jsonrpclib-pelix".

from jsonrpclib import Server

def main():
    proxy = Server('http://localhost:7002')
    print(proxy.lengths((1,2,3), 27, {'Canopus': -0.74, 'Arcturus': -0.05}))

if __name__ == '__main__':
    main()
```

Код клиента тоже очень простой. Мы можем многое узнать о протоколе, отправляя разные объекты, длину которых мы хотим измерить, и получая эти структуры данных обратно от сервера. Во-первых, протокол разрешает передавать сколько угодно параметров. Ему было не важно, что он не смог понять сигнатуру статического метода по функции. Это поведение очень отличается от механизмов XML-RPC, предназначенных для традиционных языков со статической типизацией.

Во-вторых, значение сервера None в ответе остается без изменений, потому что сам протокол поддерживает это значение, не требуя активировать нестандартные расширения:

\$ python jsonrpc_server.py

Starting server

[In another command window:]

\$ python jsonrpc_client.py

[[3, [1, 2, 3]], [None, 27], [2, {'Canopus': -0.74, 'Arcturus': -0.05}]]

В-третьих, JSON-RPC поддерживает только один тип последовательностей, поэтому кортеж клиента пришлось преобразовать в список, чтобы его можно было передать. Самая большая разница между JSON-RPC и XML-RPC заключается в том, что полезная нагрузка помещается в компактное и элегантное сообщение JSON, которое само знает, как представить каждый тип данных. Здесь мы этого не видим, потому что оба протокола прекрасно скрывают сеть от кода. Когда я использую Wireshark на интерфейсе localhost, выполняя этот пример клиента и сервера, я вижу, как отправляются следующие сообщения:

```
{ "version": "1.1",
  "params": [[1, 2, 3], 27, {'Canopus': -0.74, 'Arcturus': -0.05}], "method": "lengths" }
{ "result": [[3, [1, 2, 3]], [null, 27],
  [2, {'Canopus': -0.74, 'Arcturus': -0.05}]] }
```

Из-за популярности JSON-RPC версии 1 были предприняты многочисленные попытки дополнить протокол новыми возможностями. Поищите в Интернете информацию о текущей ситуации со стандартом и обсуждения по поводу его развития. Мы можем использовать надежную реализацию Python для большинства базовых задач и не задумываться о расширении стандарта.

Важно отметить еще один важный момент. Пример выше выполняется синхронно: клиент отправляет запрос и терпеливо ждет один ответ, ничего не делая в это время. При этом JSON-RPC позволяет присваивать запросам идентификаторы. Получается, что мы можем отправить много запросов, а затем получить ответы с теми же идентификаторами. Здесь я не буду подробно описывать этот процесс, потому что обычно протокол RPC не предназначен для асинхронной работы. В конце концов, вызовы функций в обычных процедурных языках выполняются в синхронном режиме. Если хотите узнать больше про асинхронное выполнение, изучите стандарт и узнайте, могут ли пакеты Python JSON-RPC подойти для вашего сценария.

Самодокументируемые данные

Протоколы XML-RPC и JSON-RPC поддерживают структуру данных, напоминающую словарь Python, но с одним неприятным ограничением: в XML-RPC эта структура данных называется *структурой* (struct), а в JSON — *объектом* (object). Программист Python видит в этом словарь и недоумевает от того, что ключи не могут быть целыми и вещественными числами или кортежами.

Давайте рассмотрим пример. Допустим, у нас есть словарь химических элементов, упорядоченный по порядковому номеру в Периодической системе Д. И. Менделеева:

```
{1: 'H', 2: 'He', 3: 'Li', 4: 'Be', 5: 'B', 6: 'C', 7: 'N', 8: 'O'}
```

Допустим, мы хотим отправить этот словарь с помощью RPC. Нашим первым побуждением было бы преобразовать порядковые номера в строки, чтобы словарь можно было отправить как структуру или объект. Оказывается, так делать не нужно. Структуры и объекты не поддерживают ключи и значения в контейнерах произвольных размеров. Вместо этого они связывают ограниченное количество предо-

предельных имен атрибутов со значениями атрибутов для конкретного объекта. Если мы соединим ключи со значениями в структуре, пользователи, работающие с языками со статической типизацией, будут испытывать большие проблемы, используя наш сервис. Лучше считать словари, передаваемые по протоколу RPC, похожими на объекты Python, которые часто содержат ограниченный набор имен атрибутов, хорошо известных коду. Также словари, передаваемые через RPC, должны содержать всего несколько предопределенных ключей и связанных с ним значений.

Если словарь, который мы видели раньше, будет использоваться методом RPC общего назначения, его нужно сериализовать как список явно именованных значений:

```
[{'number': 1, 'symbol': 'H'},
 {'number': 2, 'symbol': 'He'},
 {'number': 3, 'symbol': 'Li'},
 {'number': 4, 'symbol': 'Be'},
 {'number': 5, 'symbol': 'B'},
 {'number': 6, 'symbol': 'C'},
 {'number': 7, 'symbol': 'N'},
 {'number': 8, 'symbol': 'O'}]
```

Словарь Python в предыдущих примерах показан так, как будет передан в вызов RPC, а не так, как он будет представлен в сети.

Существенное отличие при таком подходе (кроме значительно увеличившейся длины) заключается в том, что предыдущая структура данных была бесполезна, если мы не знали заранее смысл ключей и значений. Для того чтобы придать данным смысл, использовалось соглашение. Однако поскольку мы указали для данных имена, эта структура описывает сама себя: тот, кто увидит эти данные в сети или на компьютере, имеет больше шансов понять, что они представляют. Протоколы XML-RPC и JSON-RPC ожидают, что вы будете использовать типы с ключами и значениями именно таким образом. Они называются структурой и объектом, потому что на языках C и JavaScript соответственно эти термины обозначают сущность, которая содержит именованные атрибуты. В этом смысле они больше похожи на объекты, чем на словари Python.

Если у нас есть словарь Python, как описано здесь, его можно преобразовать в структуру данных, совместимую с RPC, а затем изменить обратно с помощью следующего кода:

```
>>>elements = {1: 'H', 2: 'He'}
>>>t = [{'number': key, 'symbol': value} for key, value in elements.items()]
>>>t
[{'symbol': 'H', 'number': 1}, {'symbol': 'He', 'number': 2}]
>>> {obj['number']: obj['symbol']} for obj in t
{1: 'H', 2: 'He'}
```

Если окажется, что приходится создавать и уничтожать слишком много словарей и такое преобразование будет неудобным, можно использовать именованные кортежи в Python для маршалинга значений перед отправкой.

Объекты: Pyro и RPyC

Если мы хотим использовать RPC, чтобы удаленные вызовы функций выглядели как локальные, два предыдущих механизма RPC нам не подойдут. XML-RPC и JSON-RPC удобно применять, если вызываемые функции используют базовые типы данных в аргументах и возвращаемых значениях. Подумайте, как часто ваши аргументы и возвращаемые значения будут более сложными. А если мы хотим передавать активные объекты? Эту проблему сложно решить, и тому есть две причины.

Во-первых, в разных языках программирования существуют разное поведение и семантика для объектов. Поэтому системы, поддерживающие объекты, ограничиваются одним языком или предоставляют малоинформативное описание того, как может работать объект, выбрав наименьший общий знаменатель для всех языков, которые будут использовать протокол.

Во-вторых, не всегда очевидно, какую часть сведений о состоянии нужно передать вместе с объектом, чтобы он работал на другом компьютере. Да, механизм RPC может просто начать рекурсивно погружаться в характеристики объекта и готовить эти значения для передачи по сети. Правда, даже в умеренно сложных системах такая простая рекурсия по значениям атрибутов приведет к тому, что мы пройдемся по большинству объектов в памяти. А когда мы наберем мегабайты данных для передачи, какова вероятность, что все они действительно понадобятся на том конце?

Вместо того чтобы отправлять полное содержимое каждого объекта, поставляемого как аргумент или возвращаемого как значение, мы можем просто отправить имя объекта, и удаленный компьютер может по этому имени запросить свойства объекта по необходимости. В результате передаются только элементы графа объектов, которые действительно нужны удаленному компьютеру, а не весь граф. Правда, обе стратегии часто приводят к дорогим и медленным сервисам, а также мешают отслеживать, как один объект может повлиять на ответы, предоставляемые другим сервисом на той стороне сети. XML-RPC и JSON-RPC требуют, чтобы мы разбили запрос к удаленному сервису на базовые типы данных, которые можно легко передать, и эта задача в итоге часто ложится на программную архитектуру. Ограничения на типы данных аргументов и возвращаемых значений заставляют нас тщательно продумывать наш сервис, чтобы точно понимать, что именно требуется удаленному сервису и почему. Я не рекомендую переходить на более объектно-ориентированный сервис RPC, только чтобы не продумывать удаленные сервисы и не узнавать, что им нужно для выполнения их задач.

Существуют разные известные протоколы RPC, например SOAP и CORBA, которые пытаются ответить на важные вопросы о том, как поддерживать объекты, передаваемые с одного сервера на другой от имени клиентского приложения, отправляющего сообщение RPC от третьего сервера. Если только в техническом задании не прописано, что необходимо использовать эти механизмы RPC для взаимодействия с существующей системой, программисты Python обычно всеми силами избегают их. Мы не будем рассматривать их в этой книге. Если они вам нужны, придется прочитать еще как минимум по книге по каждому, потому что они очень сложные.

Если мы хотим наладить взаимодействие только между программами Python, лучше использовать сервис RPC, который знаком с объектами Python и их поведением. В Python много сложных типов данных, и было бы неразумно использовать такие ограниченные механизмы, как XML-RPC и JSON-RPC. Особенно если мы эффективно используем словари, множества и объекты даты и времени в Python. Pyро и RPyC — два нативных механизма RPC для Python, о которых стоит упомянуть. Веб-сайт проекта Pyро: <http://pythonhosted.org/Pyro4/>. Эта известная библиотека RPC основана на модуле Python pickle и может отправлять любой тип входных и выходных данных с возможностью "консервации". По сути, если элемент и его атрибуты можно преобразовать в базовые типы, их можно отправить. Pyро не будет работать, если значения, которые мы хотим передать и получить, не обрабатываются модулем pickle. (См. описание модуля pickle в документации по стандартной библиотеке Python.) Если Python не может понять, как "законсервировать" класс, в библиотеке есть инструкции, как это сделать.

Пример RPyC

Веб-сайт проекта RPyC: <http://rpyc.readthedocs.org/en/latest/>. Этот проект использует гораздо более сложный подход к объектам. Он похож на модель CORBA, в которой реальные данные, отправляемые по сети, являются ссылкой на объект, и их можно использовать для обратного вызова и вызова его методов позже, если это потребуется получателю. В текущей версии лучше продумана безопасность, и это важно, если мы хотим, чтобы другие компании могли использовать наш протокол RPC. Ведь если мы позволяем кому-то предоставлять нам данные для расконсервации, по сути, мы позволяем им выполнять на нашем компьютере произвольный код. В листинге 18.7 и 18.8 приводятся примеры клиента и сервера соответственно. Изучите их, чтобы понять, насколько эффективно работает RPyC.

Листинг 18.7. Клиент RPyC

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Клиент RPyC.

import rpyc

def main():
    config = {'allow_public_attrs': True}
    proxy = rpyc.connect('localhost', 18861, config=config)
    fileobj = open('testfile.txt')
    linecount = proxy.root.line_counter(fileobj, noisy)
    print('The number of lines in the file was', linecount)
```

```
def noisy(string):
    print('Noisy:', repr(string))

if __name__ == '__main__':
    main()
```

Листинг 18.8. Сервер RPyC

```
#!/usr/bin/env python3
# Programming in Python: The Basics.
# Сервер RPyC.

import rpyc

def main():
    from rpyc.utils.server import ThreadedServer
    t = ThreadedServer(MyService, port = 18861)
    t.start()

class MyService(rpyc.Service):
    def exposed_line_counter(self, fileobj, function):
        print('Client has invoked exposed_line_counter()')
        for linenum, line in enumerate(fileobj.readlines()):
            function(line)
        return linenum + 1

if __name__ == '__main__':
    main()
```

На первый взгляд, клиент выглядит типичной программой, которая использует сервис RPC. В конце концов, он вызывает функцию `connect()` и указывает сетевой адрес, а затем обращается к функциям возвращенного прокси-объекта, как если бы вызовы выполнялись локально. Однако если присмотреться, можно заметить удивительные отличия. Первый аргумент в функции RPC представляет активный объект файла, который может существовать, а может и не существовать на сервере. Второй аргумент представляет собой функцию, которая также является динамической сущностью, а не инертной структурой данных, поддерживаемой протоколами RPC.

Сервер предлагает один метод, который принимает объект файла и вызываемую функцию в качестве аргументов. Он использует их так же, как обычное приложение Python, которое выполняется в одном процессе. Он вызывает метод `readlines()` объекта файла и ожидает, что он вернет итерируемый объект, который можно ис-

пользовать в цикле `for`. Наконец, сервер вызывает переданный объект функции независимо от того, где находится эта функция (в клиенте). Обновленная архитектура безопасности RPyC требует, чтобы при отсутствии разрешения клиент мог вызывать только методы, которые начинаются со специального префикса `exposed_`.

Посмотрите на выходные данные, полученные в результате выполнения клиента, чтобы понять, что происходит. При условии, что небольшой файл `testfile.txt` действительно существует в текущем каталоге и содержит текст:

```
$ python rpyc_client.py
```

```
Noisy: 'Simple\n'
```

```
Noisy: 'is\n'
```

```
Noisy: 'better\n'
```

```
Noisy: 'than\n'
```

```
Noisy: 'complex.\n'
```

```
The number of lines in the file was 5
```

Здесь нужно обратить внимание на два момента. Во-первых, сервер прошел по нескольким результатам вызова `readlines()`, хотя для этого клиент должен был вызывать логику "файл — объект" несколько раз. Во-вторых, сервер каким-то образом не стал дублировать объект кода метода `noisy()`, чтобы вызывать его напрямую. Вместо этого он несколько раз вызвал функцию на клиентской стороне соединения, и каждый раз с правильным аргументом.

Как это получается? Простыми словами, RPyC ведет себя прямо противоположно механизмам RPC, которые мы рассмотрели ранее. RPyC сериализует только неизменяемые элементы, например целые числа, вещественные числа, строки и кортежи Python, тогда как другие механизмы стремятся сериализовать и передать по сети как можно больше информации, и удаленный код будет выполнен успешно или неудачно, но дополнительная информация не поступит. Для остального он отправляет идентификатор удаленного объекта, чтобы удаленный компьютер мог получать доступ к атрибутам и вызывать методы для этих активных объектов, обращаясь к клиенту. Этот подход приводит к большому объему сетевого трафика. Если выполняется много операций с объектами между клиентом и сервером, могут возникать серьезные задержки. Кроме того, будет сложно обеспечить надлежащую безопасность. Я решил установить клиентское соединение с разрешениями `allow_public_attrs`, чтобы разрешить серверам вызывать `readlines()` для объектов клиента. Если вы не готовы дать серверному коду такие разрешения, придется потратить время на детальную настройку разрешений, чтобы не затруднять работу с одной стороны и не подвергаться слишком большому риску — с другой. Такой подход может требовать много ресурсов. Кроме того, обеспечить безопасность может быть непросто, если клиент и сервер не доверяют друг другу.

Зато никакой другой инструмент не сравнится с RPyC, когда мы хотим наладить взаимодействие между двумя объектами Python на противоположных концах. Процессы могут быть даже больше двух (см. документацию по RPyC).

RPyC прекрасно работает с простыми функциями и объектами Python, не требуя, чтобы они зависели от сетевых возможностей, и это показывает, насколько эффективно Python может перехватывать операции с объектами и обрабатывать их, даже если запрос поступает из сети.

Очереди сообщений, RPC и веб-фреймворки

При работе с сервисами RPC будьте готовы экспериментировать с разными методами передачи. Например, многие программисты Python, которые пишут код для работы с протоколом XML-RPC, не используют классы из стандартной библиотеки. В конце концов, сервис RPC часто разворачивается как часть большого веб-сайта, и обслуживание отдельных серверов на другом порту, просто чтобы обрабатывать запросы такого типа, доставляет много хлопот.

Существует три эффективных подхода, чтобы не запускать новый веб-сервер для каждого сервиса RPC, который мы хотим предоставлять на сайте. Во-первых, можно использовать интерфейс WSGI, чтобы установить сервис RPC, интегрированный в крупный веб-проект. Если запустить веб-приложение и сервис RPC как серверы WSGI за фильтром, который проверяет входящий URL, оба сервиса могут работать по одному имени хоста и номеру порта. Кроме того, веб-сервер WSGI может уже поддерживать потоки и масштабируемость лучше, чем это делает наш сервис RPC.

Если сервис RPC находится в нижней части стека WSGI, мы можем легко добавить аутентификацию.

Во-вторых, мы можем отказаться от применения отдельной библиотеки RPC, если наш веб-фреймворк умеет работать с XML-RPC, JSON-RPC и другими механизмами RPC. Это значит, что мы можем объявлять конечные точки RPC так же легко, как мы определяем представления или сервисы RESTful в нашем веб-фреймворке. Изучите документацию по своему веб-фреймворку и поищите сторонние плагины для работы с RPC.

В-третьих, мы можем доставлять сообщения RPC через альтернативный транспорт, который эффективнее перенаправляет вызовы серверам, готовым их обработать. Если мы хотим, чтобы вызовы распределялись по целой стойке серверов, хорошим вариантом для передачи вызовов RPC станут очереди сообщений, которые мы рассматривали в *главе 8*.

Восстановление после ошибок в сети

У сети есть одна особенность, которую сервисы RPC не могут скрыть: сеть может быть недоступна на момент вызова или сбой может произойти прямо в процессе вызова.

Если вызов прерывается, большинство протоколов RPC просто вызывают исключение. Оно не всегда означает, что удаленный компьютер не обработал запрос.

Может быть, он его обработал, но сеть упала в момент передачи последнего пакета ответа. Теоретически в этой ситуации вызов мог завершиться успешно. Возможно, данные добавились в базу данных или записались в файл, или мы получили иной желаемый результат от вызова RPC. Однако мы будем считать, что вызов не был выполнен, и попробуем выполнить его снова. Вероятно, при этом одни и те же данные сохранятся дважды. При создании программ, которые делегируют вызовы функций через сеть, можно использовать несколько стратегий.

Во-первых, можно создать сервис с идемпотентными операциями, которые можно безопасно повторять. Если операция сформулирована как "переведи 10 долларов с моего банковского счета", при повторном выполнении со счета снимется еще 10 долларов. Однако если мы выполняем операцию "проведи транзакцию 583812, которая переводит 10 долларов с моего счета", ее можно безопасно выполнять снова, потому что по номеру транзакции сервер распознает повтор и объявит об успешном выполнении, не переводя средства снова.

Во-вторых, можно следовать рекомендации из *главы 5*: вместо того, чтобы использовать блок `try...except` для каждого вызова RPC, заключайте в `try` и `except` большие части кода. Так семантика будет выполняться повторно с меньшим риском. Если использовать обработчик исключений для каждого вызова, мы потеряем большинство преимуществ RPC: код должен быть таким, чтобы его было просто программировать и не приходилось постоянно думать, что вызовы будут отправляться по сети. Если вы решите, что программа должна повторять неудавшийся вызов, используйте стратегию экспоненциальной задержки для UDP, как описано в *главе 3*, чтобы не отправлять слишком много запросов на сервер, и без того перегруженный.

Наконец, обратите внимание на детали исключения. Если мы не используем метод RPC специально для Python, внятное исключение `KeyError` или `ValueError` на удаленном компьютере превращается в ошибку, специфическую для RPC, с незнакомым текстом или числовым кодом, по которым сложно понять, что пошло не так.

Резюме

RPC позволяет делать обычные вызовы функций Python, но для передачи по сети и работы на другом сервере. Для этого протокол сериализует аргументы и возвращаемое значение.

Все механизмы RPC работают примерно одинаково: мы устанавливаем сетевое соединение и вызываем предоставленный прокси-объект, чтобы выполнить код на удаленном компьютере. Стандартная библиотека Python предоставляет нативную поддержку более старого протокола XML-RPC, хотя существуют сторонние библиотеки, поддерживающие более элегантный и современный стандарт JSON-RPC. С помощью этих механизмов между клиентом и сервером можно передавать данные всего нескольких типов. Если мы хотим использовать более сложные типы данных Python, нам доступна система Pyro, которая соединяет программы Python

по сети и поддерживает многие типы данных. Система RPyC предлагает гораздо больше возможностей, позволяя передавать объекты между системами и перенаправлять вызовы методов для этих объектов в систему, в которой находится объект.

Если будете пролистывать эту книгу снова, возможно, вам захочется посмотреть на каждую тему через призму RPC, потому что данные передаются между клиентской программой и сервером на основе соглашения о том, к чему приведет запрос и как будет отображаться ответ. Теперь вы знаете, как работает протокол RPC, который поддерживает произвольную коммуникацию, а не выполнение заданных действий. Всегда обдумывайте, требуется ли для вашего сценария такое гибкое решение, как RPC, или для всех транзакций между клиентом и сервером достаточно будет одного из более простых и специализированных протоколов, которые мы рассматривали в предыдущих главах. Выберите протокол, подходящий для вашего случая, чтобы избежать лишних сложностей и наладить простое и надежное сетевое взаимодействие.

Предметный указатель

A

American Standard Code for Information Interchange (ASCII) 113
AMQP 196
Apache, сервер 231, 232
asyncio, фреймворк 177

B

Bottle 278
Byte order marker (BOM) 115

C

Cascading style sheets (CSS) 251
Celery 196
codecs, модуль 114
Cookie 221, 264
Cross-site request forgery (CSRF) 269
Cross-site scripting (XSS) 266

D

Denial of Service (DoS) 167
DF, флаг 37
Digest access authentication 220
Django 273, 277
dnspython3, библиотека 105
Docker 235
Domain Name Service (DNS) 90
Domain Name System (DNS) 34, 101

E

EHLO 326
email, модуль 296
email.utils, модуль 311
ESMTP 326

F

File Transfer Protocol (FTP) 401
Flask 257, 278
ftplib, модуль 404

G

Getaddrinfo 93

H

HELO 326
Heroku 235
http.server, модуль 226
httplib, модуль 211, 223
HTTP-заголовок:

- ◊ Accept-Language 217
- ◊ Authorization 220
- ◊ Content-Type 218
- ◊ Cookie 221
- ◊ Location 251
- ◊ Transfer-Encoding 216
- ◊ User-Agent 218
- ◊ Vary 213

HTTP-прокси 229

- ◊ обратный 229
- ◊ прямой 229

HyperText Markup Language (HTML) 252
Hypertext Transfer Protocol (HTTP) 26, 203

I

IMAPClient, модуль 349
imaplib, модуль 347
inetd, демон 183
Internet Assigned Numbers Authority (IANA) 43

Internet Control Message Protocol (ICMP) 37
Internet Message Access Protocol (IMAP)
345

Internet Movie Database (IMDB) 279

Internet Protocol (IP) 31, 33

IP-адрес:

- ◊ IPv4 34
- ◊ IPv6 35, 92

J

JavaScript Object Notation (JSON) 126, 420
json, модуль 126

L

Least recently used (LRU) 188
localhost 36
logging, модуль 161
lxml, библиотека 286

M

Mail transfer agent (MTA) 319
mailbox, модуль 315
Maximum transmission unit (MTU) 37, 46
Memcached, демон 188
mod_python, модуль 226
mod_wsgi, модуль 231

O

OpenSSL, библиотека 142

P

paramiko, модуль 388
Perfect Forward Security (PFS) 147, 155
pickle, модуль 125
Platforms as a Service (PaaS) 234
Post Office Protocol (POP) 335
PPPoE, протокол 37
pygeocoder, пакет 23
Pyramid 278
Pyro 435
Python Package Index (PyPI) 23
pytz, модуль 312

R

random, модуль 56
Remote procedure call (RPC), 194, 419
Representational state transfer (REST) 236
Requests for comment (RFC) 38
requests, библиотека 25
requests, модуль 204, 211
RPyC 435

S

Secure Shell Protocol (SSH) 386
Secure Sockets Layer (SSL) 135
Simple Mail Transport Protocol (SMTP) 313
smtplib, модуль 322
socket, модуль 44, 92, 93
socketserver, модуль 171
SQLite 254
SSH File Transfer Protocol (SFTP) 396
ssl, модуль 146
struct, модуль 118
subprocess, модуль 376
supervisord, утилита 161

T

Telnet 381
telnetlib, модуль 381
termios, модуль 381
Tornado 278
Transmission Control Protocol (TCP) 31, 41,
67

- ◊ размер окна 68
- ◊ сокет:
 - подключенный 75
 - слушающий 75

Transport Layer Security (TLS) 135

U

Unicode 113
Uniform Resource Identifier (URI) 246
Uniform Resource Location (URL) 246

- ◊ абсолютный 250
- ◊ относительный 251

Uniform Resource Name (URN) 246
urllib, модуль 204, 211, 223

urllib.parse, модуль 247, 249
User Datagram Protocol (UDP) 41

V

virtualenv, пакет 23

W

Web Server Gateway Interface (WSGI) 227
WebOb 219, 240
WebSocket, протокол 279

Werkzeug, библиотека 241
World Wide Web (WWW) 246

X

XML 126

Z

zen_utils, модуль 166
Zero Message Queue (ØMQ) 197
zlib, модуль 127

A

Адрес 33
Аккаунт 404
Арифметика с дополнением до двух 112
Атака посредника 49
Аутентификация 219
 ♦ в SSH 390

Б

Байт 32, 111, 112
Библиотека Python, стандартная 23
Бит 112

В

Веб-сервер 238
Веб-сокеты 279
Веб-фреймворк 239
Взаимоблокировка 79
Всемирная паутина 246
Вызов процедур удаленный 194

Д

Дайджест-аутентификация доступа 220
Датаграмма 37
Декодирование 32, 114
Документ гипертекстовый 246
Домен верхнего уровня 90

Е

Единица передачи максимальная 37, 46

З

Задержка экспоненциальная 53
Запрос условный 214

И

Идентификатор запроса 56
Исключение 128
 ♦ `herror` 129
 ♦ `OSError` 129
 ♦ `socket.gaierror` 129
 ♦ обработка 131

К

Кадрирование 119
Канал 195
Кеширование 213, 215
Ключ:
 ♦ закрытый 137
 ♦ открытый 137
Код состояния 210
Кодирование 32, 114
Кодирование протокола HTTP 216
Кодировка:
 ♦ ASCII 113
 ♦ Unicode 113
 ♦ UTF-16 115
 ♦ UTF-32 114, 115
 ♦ UTF-8 114, 115
 ♦ многобайтовая 114
 ♦ однобайтовая 114

М

Маркер последовательности байтов 115

Маршрутизация 35

Метка порядка байтов 115

Метод:

- ◊ CONNECT 209
- ◊ DELETE 209
- ◊ encode() 32
- ◊ GET 208
- ◊ getaddrinfo() 95, 100
- ◊ getservbyname() 44
- ◊ getsockaddr() 100
- ◊ HEAD 209
- ◊ OPTIONS 209
- ◊ pack() 118
- ◊ POST 208
- ◊ PUT 209
- ◊ repr() 113
- ◊ TRACE 209
- ◊ unpack() 118

Мультивещание 62

Мультиплексирование 41, 42

О

Октет 112

Отказ в обслуживании 167

П

Пакет 33

Парсинг:

- ◊ дат 311
- ◊ электронного письма 305

Письмо электронное 293

- ◊ парсинг 305

Платформа как услуга 161, 234

Подделка запроса межсайтовая 269

Порядок байтов:

- ◊ обратный 117
- ◊ прямой 117

Путь абсолютный 250

Р

Развертывание сервиса 160

С

Сервер:

- ◊ асинхронный 172
- ◊ многопоточный 170
- ◊ многопроцессорный 170

Сервис, развертывание 160

Сертификат 137

Сжатие данных 127

Скрейпинг 279

Скриптинг межсайтовый 266

- ◊ непостоянный 266

- ◊ постоянный 268

Служба доменных имен 90

Сокет 45

- ◊ TCP 70
 - подключенный 75
 - слушающий 75
- ◊ UDP 55

Спуфинг 56

Т

Тег HTML 252

Терминал 377

Точка кодовая 32

У

Управление потоком 68

Ф

Файл /etc/services 44

Фрагментация 37

- ◊ флаг DF 37

Функция:

- ◊ decode() 32
- ◊ getaddrinfo() 44, 93
- ◊ hex() 117
- ◊ recv() 119

Ц

Центр сертификации 138

Цепочка сертификатов 141

Ш

Широковещание 62

Отдел оптовых поставок:

e-mail: opt@bhv.ru



Вы научитесь:

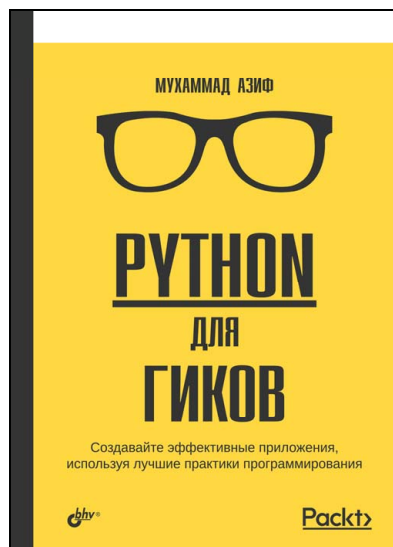
- Писать код для контроллера привода двигателя
- Строить карты на основе данных лидара
- Создавать собственные алгоритмы автономного планирования траектории движения
- Писать код для автоматической отправки путевых точек контроллеру привода
- Создавать программы картографии и навигации для автономных роботов

Рассказано о технологии создания автономных роботов на базе одноплатного компьютера Raspberry Pi и о разработке программ для них на языке C++. Показаны принципы написания и даны примеры кода для контроллера привода двигателя, продемонстрированы способы использования датчиков для обнаружения препятствий и построения карт на основе данных лидара. Описаны методы разработки собственных алгоритмов автономного планирования траектории движения, приведен код для автоматической отправки путевых точек контроллеру привода. Рассмотрены библиотеки C++ для написания программ картографии и навигации автономных роботов, даны сведения об использовании контактов аппаратного интерфейса Raspberry Pi GPIO.

Электронный архив на сайте издательства содержит код описанных в книге программ.

Бромбах Ллойд, инженер, программист и энтузиаст электроники и робототехники. Участвовал в соревнованиях по робототехнике, таких как финансируемый НАСА конкурс Lunar Regolith Excavation Challenge 2007 и 27-й конкурс Intelligent Ground Vehicle Challenge.

Отдел оптовых поставок:
e-mail: opt@bhv.ru



Вы изучите:

- Принципы разработки и управления сложными проектами
- Способы автоматизации тестирования приложений и разработки через тестирование (TDD)
- Многопоточность и многопроцессорность в Python
- Написание приложений с использованием кластера Apache Spark для обработки больших данных
- Разработку и развертывание бессерверных программ в облаке на примере Google Cloud Platform (GCP)
- Создание на Python веб-приложений и REST API, использование среды Flask
- Использование Python для извлечения данных с сетевых устройств и систем управления сетью (NMS)
- Применение Python для анализа данных и машинного обучения

Книга подробно рассказывает о разработке, развертывании и поддержке крупномасштабных проектов на Python. Представлены такие концепции, как итераторы, генераторы, обработка ошибок и исключений, обработка файлов и ведение журналов. Приведены способы автоматизации тестирования приложений и разработки через тестирование (TDD). Рассказано о написании приложений с использованием кластера Apache Spark для обработки больших данных, о разработке и развертывании бессерверных программ в облаке на примере Google Cloud Platform (GCP), о создании веб-приложений и REST API, использовании среды Flask. Показаны способы применения языка для создания, обучения и оценки моделей машинного обучения, а также их развертывания в облаке, описаны приемы использования Python для извлечения данных с сетевых устройств и систем управления сетью (NMS).

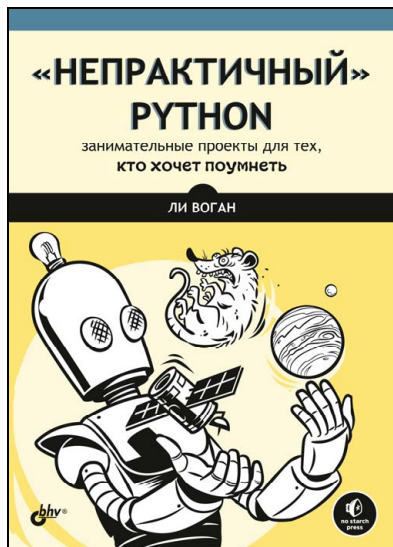
Мухаммад Азиф, программный архитектор, обладающий обширным опытом в области веб-разработки, автоматизации сетей и облаков, виртуализации и машинного обучения. Возглавлял многие крупномасштабные проекты в различных коммерческих компаниях. В 2012 году получил степень доктора философии в области компьютерных систем в Карлтонском университете (Оттава, Канада) и в настоящее время работает в компании Nokia в качестве ведущего специалиста.

Воган Л.
«Непрактичный» Python:
занимательные проекты для тех,
кто хочет поумнеть

Отдел оптовых поставок

E-mail: opt@bhv.ru

Моделируй, экспериментируй, играй



Потренируйтесь в решении задач, чтобы:

- Помочь Джеймсу Бонду взломать высокотехнологичный сейф с использованием алгоритма восхождения к вершине холма
- Сочинить стихи с помощью анализа марковских цепей
- Породить расу гигантских крыс, применив генетические алгоритмы
- Запланировать обеспеченную жизнь на пенсии с использованием метода Монте-Карло
- Смоделировать Млечный Путь и рассчитать наши шансы обнаружить инопланетные цивилизации
- Нарисовать карту Марса и изучить орбитальную механику с использованием вашего собственного космического спутника

Данная книга — это набор забавных, в том числе образовательных, проектов, предназначенных для развлечения программистов и одновременного повышения их навыков. Это хорошее дополнение к традиционным самоучителям, отличная «следующая книга», расширяющая полученные ранее навыки и знакомящая с новыми полезными инструментами.

Каждый проект включает в себя интригующий поворот с историческими событиями, литературными персонажами или ссылками на поп-культуру — и все это используя модули `tkinter`, `matplotlib`, `cProfile`, `Pylint`, `pygame`, `pillow` и `python-docx`.

Ли Воган — программист, энтузиаст поп-культуры и педагог. Профессионал в сфере построения и анализа компьютерных моделей, разработки, тестирования и коммерциализации программного обеспечения, а также подготовки ИТ-специалистов. Эта книга написана им с целью помочь читателям отточить свои навыки программирования на Python и получить от этого удовольствие!



ИНТЕРНЕТ-МАГАЗИН

BHV.RU

КНИГИ, РОБОТЫ,
ЭЛЕКТРОНИКА

Интернет-магазин издательства «БХВ»

- Более 30 лет на российском рынке
- Книги и наборы по электронике и робототехнике по издательским ценам
- Электронные архивы книг и компакт-дисков
- Ответы на вопросы читателей



Интернет-магазин БХВ-Электроника
Скоро открытие!

